

Appendix K: ResNet50V2 Tests 6

In []: !nvidia-smi

Wed Mar 19 16:16:47 2025

```
+-----+
+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version:
12.4    |
|-----+-----+-----+
+-----+
| GPU  Name                       Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|              |                    |              |
MIG M. |
|=====+=====+=====+
=====|
|   0   NVIDIA A100-SXM4-40GB         Off |  00000000:00:04.0 Off |
0 |
| N/A   30C    P0              45W / 400W |      0MiB / 40960MiB |      0%
Default |
|              |                    |              |
Disabled |
+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|       ID    ID                                   |                    Us
age      |
|=====+=====+=====+
=====|
| No running processes found
|
+-----+-----+-----+
+-----+
```

In []: !nvidia-smi

Wed Mar 19 17:41:48 2025

```

+-----+
+-----+
| NVIDIA-SMI 550.54.15                  Driver Version: 550.54.15          CUDA Version:
12.4    |
+-----+-----+-----+
+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|                |                    |                      |
MIG M. |
+=====+=====+=====+
=====+
|   0   NVIDIA A100-SXM4-40GB          Off |  00000000:00:04.0 Off |
0 |
| N/A   29C    P0              46W / 400W |      0MiB / 40960MiB |      0%
Default |
|                |                    |                      |
Disabled |
+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|       ID    ID                                   |
age      |
+=====+=====+=====+
=====+
| No running processes found
|
+-----+-----+-----+
+-----+

```

```

In [3]: #!/pip install tensorflow
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))

```

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Import libraries

```

In [2]: import tensorflow as tf # Access to models, datasets and training
from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100
from tensorflow.keras.applications import ResNet50 # Access to pre-trained model
from tensorflow.keras import layers, models, optimizers # Access to building blo
from tensorflow.keras.models import load_model # Ability to load best_saved mode
from tensorflow import keras # Access to stuff for model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLRC
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis o
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data

```

```
import seaborn as sns
import matplotlib.pyplot as plt # Plot data
```

Function definitions

```
In [4]: def display_imgs(imgs, labels):
        plt.subplots(figsize=(10,10))
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.imshow(imgs[k])
            #plt.title(labels[k])
            plt.axis('off')
        plt.tight_layout()
        plt.show()

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224)) # Resize images
    x = tf.image.random_flip_left_right(x) # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2) # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2) # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32)) # One-hot en
    return x, y

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

    # Plot Training and Validation Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_acc, label='Training Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Training and Validation Accuracy')
```

```
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```

Having weight decay in fine_tuning as well

```
In [ ]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")

        # Split temp data into equal validation (4000) and testing (4000) data
        X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
        print(f"X_temp_val.shape: {X_temp_val.shape}")
        print(f"y_temp_val.shape: {y_temp_val.shape}")
        print(f"X_temp_test.shape: {X_temp_test.shape}")
        print(f"y_temp_test.shape: {y_temp_test.shape}\n")

        # Split test data into validation (5000) and testing (5000)
        X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

        # Add temp_val to validation (9000) and temp_test to testing (9000) to get a
        X_val = np.concatenate((X_val, X_temp_val), axis=0)
        y_val = np.concatenate((y_val, y_temp_val), axis=0)
        X_test = np.concatenate((X_test, X_temp_test), axis=0)
        y_test = np.concatenate((y_test, y_temp_test), axis=0)

        print(f"X_train.shape: {X_train.shape}")
        print(f"y_train.shape: {y_train.shape}")
        print(f"X_val.shape: {X_val.shape}")
        print(f"y_val.shape: {y_val.shape}")
        print(f"X_test.shape: {X_test.shape}")
        print(f"y_test.shape: {y_test.shape}\n")

        display_imgs(X_train, y_train)

        # Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
        X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
        display_imgs(X_train, y_train)

        labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                        'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                        'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                        'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                        'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                        'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                        'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                        'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                        'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                        'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
```



```

        'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                                     .batch(batch_size)
                                     .prefetch(tf.data.experimental.AUTOTUNE))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                    .map(augment_dataset, num_parallel_calls=tf.data.experimen
#                    .batch(batch_size)
#                    .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
              .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                               tf.squeeze(tf.one_hot(y, depth=100, dtype=
              .batch(batch_size)
              .prefetch(tf.data.experimental.AUTOTUNE))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

```

```

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.7), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'>
    print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)

#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)
#tensorboard = TensorBoard(log_dir='./Logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/acc
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v5.keras', # Path where the best model is saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when validation loss is best

```

```

mode='min', # Looking to minimise the validation
verbose=1) # Print out message when saving mode
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v5.keras', # Pa
monitor='val_loss', # What to monitor (validati
save_best_only=True, # Only save model when val
mode='min', # Looking to minimise the validation
verbose=1) # Print out message when saving mode
#cvs_logger = CSVLogger('training_log.csv', separator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping, checkp

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v5.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

```

```

#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5, weight_decay=3e-
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score']))

model.summary()

#### <<<<<<<<<Modify Dataset>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v5.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")

```

```

print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>

169001437/169001437 ————— 2s 0us/step

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

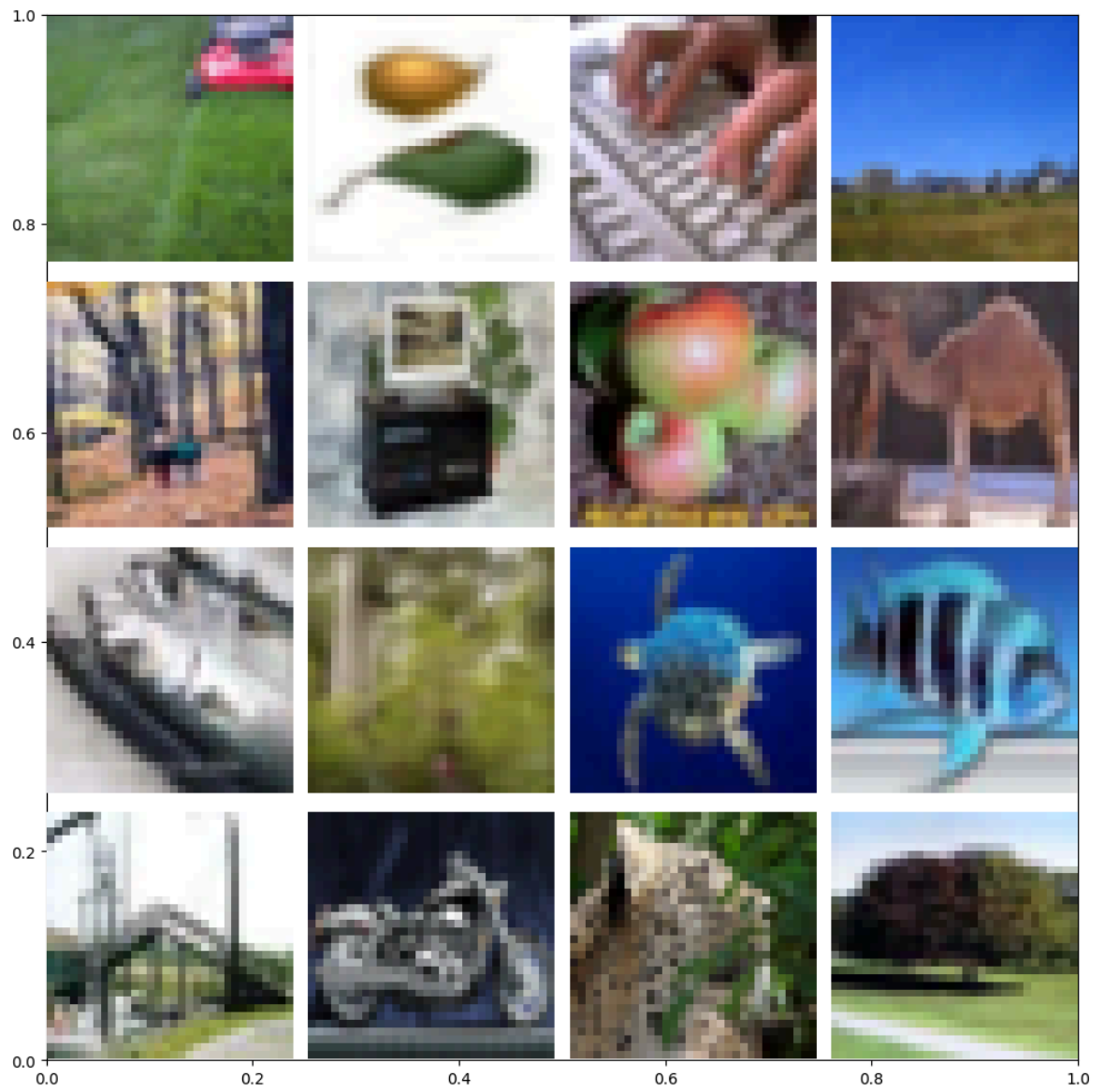
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

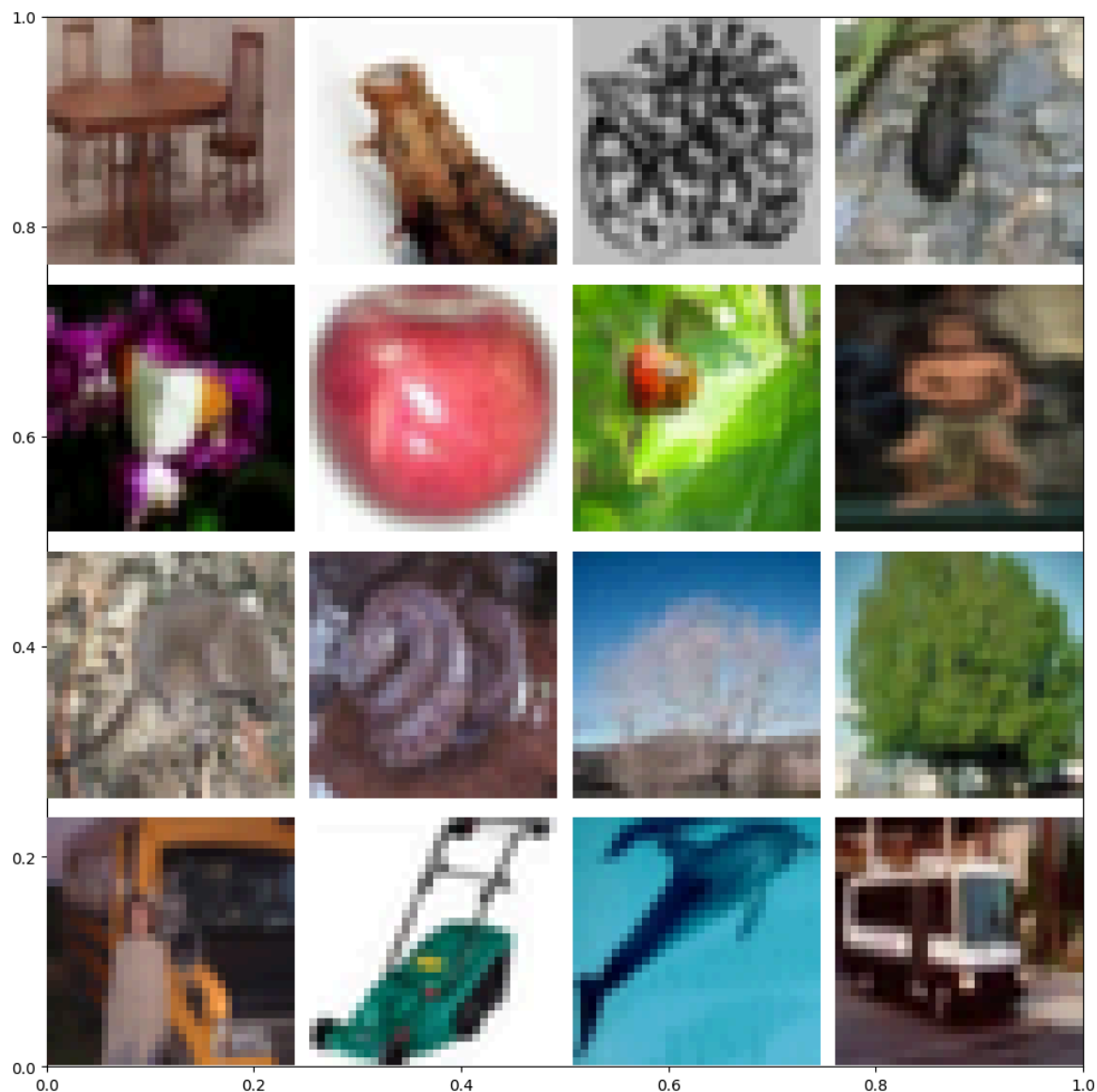
labels[30142].shape: (1,)

imgs[30142].shape: (32, 32, 3)



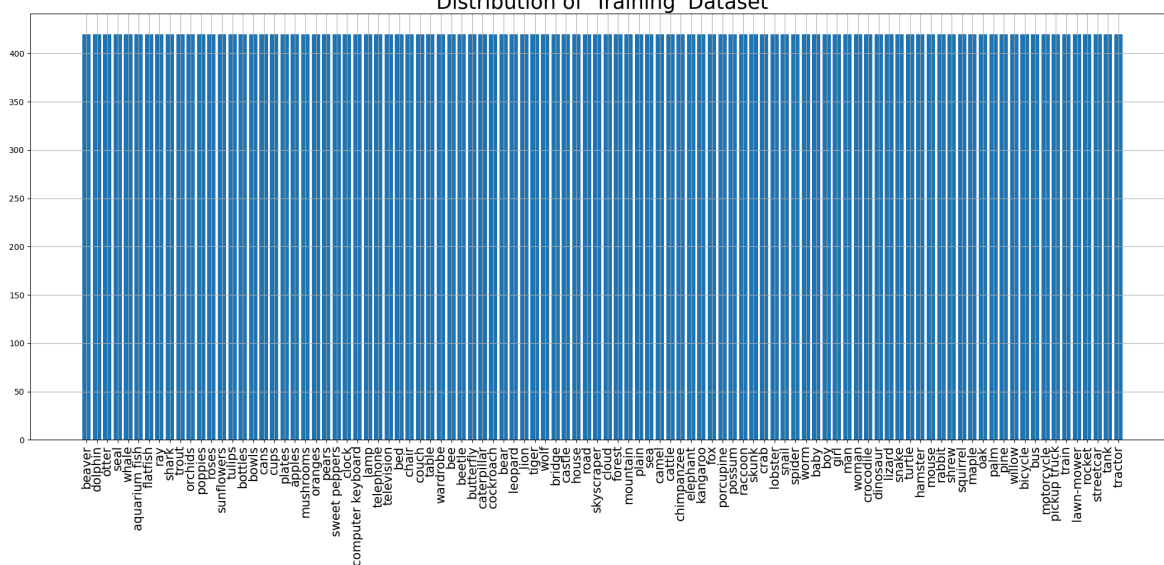
labels[27523].shape: (1,)

imgs[27523].shape: (32, 32, 3)

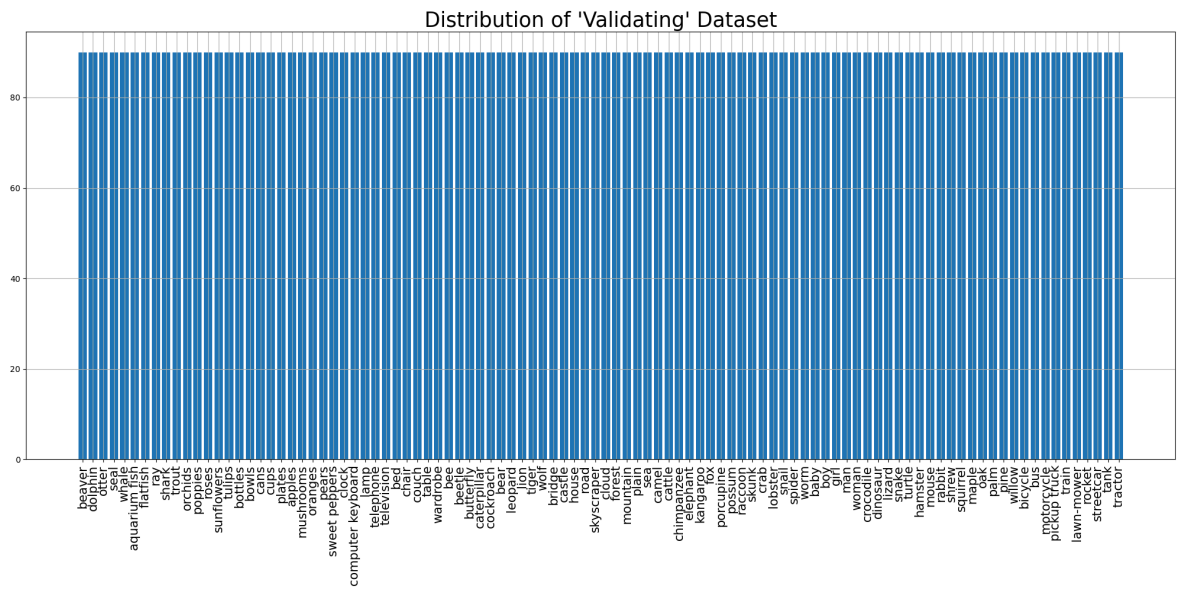


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels names, rotation=90, fontsize=15)
```

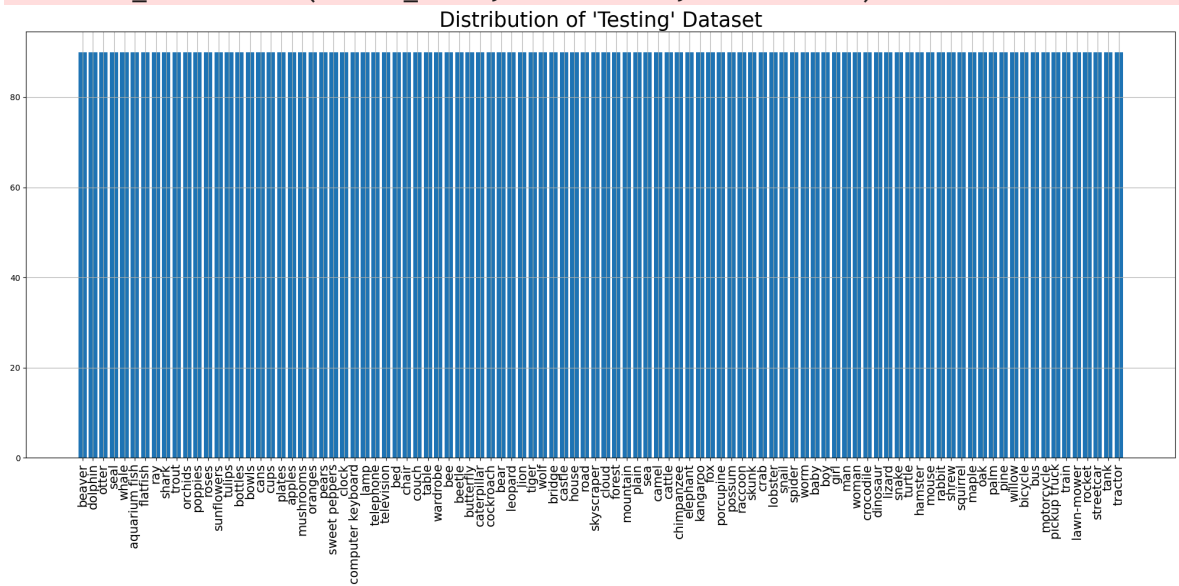
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5

94668760/94668760 ————— **11s** 0us/step

<class 'tuple'>

2

<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

<class 'numpy.ndarray'> <class 'numpy.ndarray'>

(64, 224, 224, 3) (64, 100)

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)

◀ ▶

Total params: 24,665,188 (94.09 MB)

Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

Epoch 1/25
657/657 ————— 0s 34ms/step - accuracy: 0.2656 - f1_score: 0.2600 - loss: 3.1657 - precision: 0.6380
Epoch 1: val_loss improved from inf to 1.56987, saving model to best_model_v5.keras
657/657 ————— 56s 58ms/step - accuracy: 0.2658 - f1_score: 0.2602 - loss: 3.1648 - precision: 0.6381 - val_accuracy: 0.5726 - val_f1_score: 0.5686 - val_loss: 1.5699 - val_precision: 0.8216
Epoch 2/25
657/657 ————— 0s 23ms/step - accuracy: 0.4752 - f1_score: 0.4700 - loss: 1.9388 - precision: 0.7285
Epoch 2: val_loss improved from 1.56987 to 1.44740, saving model to best_model_v5.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4752 - f1_score: 0.4700 - loss: 1.9387 - precision: 0.7285 - val_accuracy: 0.5980 - val_f1_score: 0.5916 - val_loss: 1.4474 - val_precision: 0.8236
Epoch 3/25
655/657 ————— 0s 23ms/step - accuracy: 0.5156 - f1_score: 0.5110 - loss: 1.7692 - precision: 0.7509
Epoch 3: val_loss improved from 1.44740 to 1.42039, saving model to best_model_v5.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5156 - f1_score: 0.5110 - loss: 1.7690 - precision: 0.7509 - val_accuracy: 0.6051 - val_f1_score: 0.6012 - val_loss: 1.4204 - val_precision: 0.8254
Epoch 4/25
657/657 ————— 0s 23ms/step - accuracy: 0.5249 - f1_score: 0.5214 - loss: 1.7077 - precision: 0.7561
Epoch 4: val_loss improved from 1.42039 to 1.40238, saving model to best_model_v5.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5249 - f1_score: 0.5214 - loss: 1.7077 - precision: 0.7561 - val_accuracy: 0.6090 - val_f1_score: 0.6051 - val_loss: 1.4024 - val_precision: 0.8336
Epoch 5/25
655/657 ————— 0s 23ms/step - accuracy: 0.5409 - f1_score: 0.5368 - loss: 1.6389 - precision: 0.7674
Epoch 5: val_loss improved from 1.40238 to 1.37673, saving model to best_model_v5.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5410 - f1_score: 0.5368 - loss: 1.6388 - precision: 0.7674 - val_accuracy: 0.6144 - val_f1_score: 0.6104 - val_loss: 1.3767 - val_precision: 0.8271
Epoch 6/25
655/657 ————— 0s 23ms/step - accuracy: 0.5476 - f1_score: 0.5439 - loss: 1.6022 - precision: 0.7663
Epoch 6: val_loss improved from 1.37673 to 1.36658, saving model to best_model_v5.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5476 - f1_score: 0.5439 - loss: 1.6022 - precision: 0.7663 - val_accuracy: 0.6187 - val_f1_score: 0.6132 - val_loss: 1.3666 - val_precision: 0.8204
Epoch 7/25
655/657 ————— 0s 23ms/step - accuracy: 0.5549 - f1_score: 0.5513 - loss: 1.5735 - precision: 0.7665
Epoch 7: val_loss improved from 1.36658 to 1.34462, saving model to best_model_v5.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5550 - f1_score: 0.5513 - loss: 1.5734 - precision: 0.7665 - val_accuracy: 0.6219 - val_f1_score: 0.6187 - val_loss: 1.3446 - val_precision: 0.8211
Epoch 8/25
655/657 ————— 0s 23ms/step - accuracy: 0.5617 - f1_score: 0.5583 - loss: 1.5569 - precision: 0.7670
Epoch 8: val_loss did not improve from 1.34462

657/657 ————— **18s** 27ms/step - accuracy: 0.5617 - f1_score: 0.5583
- loss: 1.5568 - precision: 0.7670 - val_accuracy: 0.6173 - val_f1_score: 0.6157
- val_loss: 1.3588 - val_precision: 0.8189
Epoch 9/25
657/657 ————— **0s** 23ms/step - accuracy: 0.5656 - f1_score: 0.5615 -
loss: 1.5359 - precision: 0.7742
Epoch 9: val_loss improved from 1.34462 to 1.33982, saving model to best_model_v
5.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5656 - f1_score: 0.5615
- loss: 1.5359 - precision: 0.7742 - val_accuracy: 0.6196 - val_f1_score: 0.6145
- val_loss: 1.3398 - val_precision: 0.8146
Epoch 10/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5658 - f1_score: 0.5608 -
loss: 1.5269 - precision: 0.7749
Epoch 10: val_loss did not improve from 1.33982
657/657 ————— **18s** 27ms/step - accuracy: 0.5658 - f1_score: 0.5609
- loss: 1.5268 - precision: 0.7749 - val_accuracy: 0.6199 - val_f1_score: 0.6159
- val_loss: 1.3437 - val_precision: 0.8246
Epoch 11/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5679 - f1_score: 0.5644 -
loss: 1.5122 - precision: 0.7746
Epoch 11: val_loss improved from 1.33982 to 1.32804, saving model to best_model_v
5.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5679 - f1_score: 0.5644
- loss: 1.5121 - precision: 0.7747 - val_accuracy: 0.6247 - val_f1_score: 0.6213
- val_loss: 1.3280 - val_precision: 0.8163
Epoch 12/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5735 - f1_score: 0.5693 -
loss: 1.4967 - precision: 0.7736
Epoch 12: val_loss did not improve from 1.32804
657/657 ————— **18s** 27ms/step - accuracy: 0.5735 - f1_score: 0.5693
- loss: 1.4967 - precision: 0.7736 - val_accuracy: 0.6196 - val_f1_score: 0.6154
- val_loss: 1.3420 - val_precision: 0.8209
Epoch 13/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5765 - f1_score: 0.5727 -
loss: 1.4841 - precision: 0.7714
Epoch 13: val_loss improved from 1.32804 to 1.32650, saving model to best_model_v
5.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5765 - f1_score: 0.5727
- loss: 1.4840 - precision: 0.7715 - val_accuracy: 0.6227 - val_f1_score: 0.6180
- val_loss: 1.3265 - val_precision: 0.8166
Epoch 14/25
657/657 ————— **0s** 23ms/step - accuracy: 0.5765 - f1_score: 0.5724 -
loss: 1.4843 - precision: 0.7782
Epoch 14: val_loss improved from 1.32650 to 1.31837, saving model to best_model_v
5.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5765 - f1_score: 0.5724
- loss: 1.4842 - precision: 0.7782 - val_accuracy: 0.6292 - val_f1_score: 0.6250
- val_loss: 1.3184 - val_precision: 0.8271
Epoch 15/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5734 - f1_score: 0.5697 -
loss: 1.4824 - precision: 0.7739
Epoch 15: val_loss did not improve from 1.31837
657/657 ————— **18s** 27ms/step - accuracy: 0.5734 - f1_score: 0.5698
- loss: 1.4823 - precision: 0.7740 - val_accuracy: 0.6267 - val_f1_score: 0.6236
- val_loss: 1.3278 - val_precision: 0.8256
Epoch 16/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5810 - f1_score: 0.5774 -
loss: 1.4736 - precision: 0.7826
Epoch 16: val_loss did not improve from 1.31837

657/657 ————— **18s** 27ms/step - accuracy: 0.5810 - f1_score: 0.5774
 - loss: 1.4735 - precision: 0.7827 - val_accuracy: 0.6279 - val_f1_score: 0.6252
 - val_loss: 1.3233 - val_precision: 0.8178

Epoch 17/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5787 - f1_score: 0.5747 -
 loss: 1.4747 - precision: 0.7778

Epoch 17: val_loss did not improve from 1.31837

657/657 ————— **18s** 27ms/step - accuracy: 0.5787 - f1_score: 0.5747
 - loss: 1.4746 - precision: 0.7778 - val_accuracy: 0.6257 - val_f1_score: 0.6243
 - val_loss: 1.3202 - val_precision: 0.8237

Epoch 18/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5799 - f1_score: 0.5768 -
 loss: 1.4708 - precision: 0.7801

Epoch 18: val_loss did not improve from 1.31837

657/657 ————— **18s** 27ms/step - accuracy: 0.5799 - f1_score: 0.5768
 - loss: 1.4707 - precision: 0.7801 - val_accuracy: 0.6254 - val_f1_score: 0.6230
 - val_loss: 1.3201 - val_precision: 0.8170

Epoch 19/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5817 - f1_score: 0.5780 -
 loss: 1.4541 - precision: 0.7815

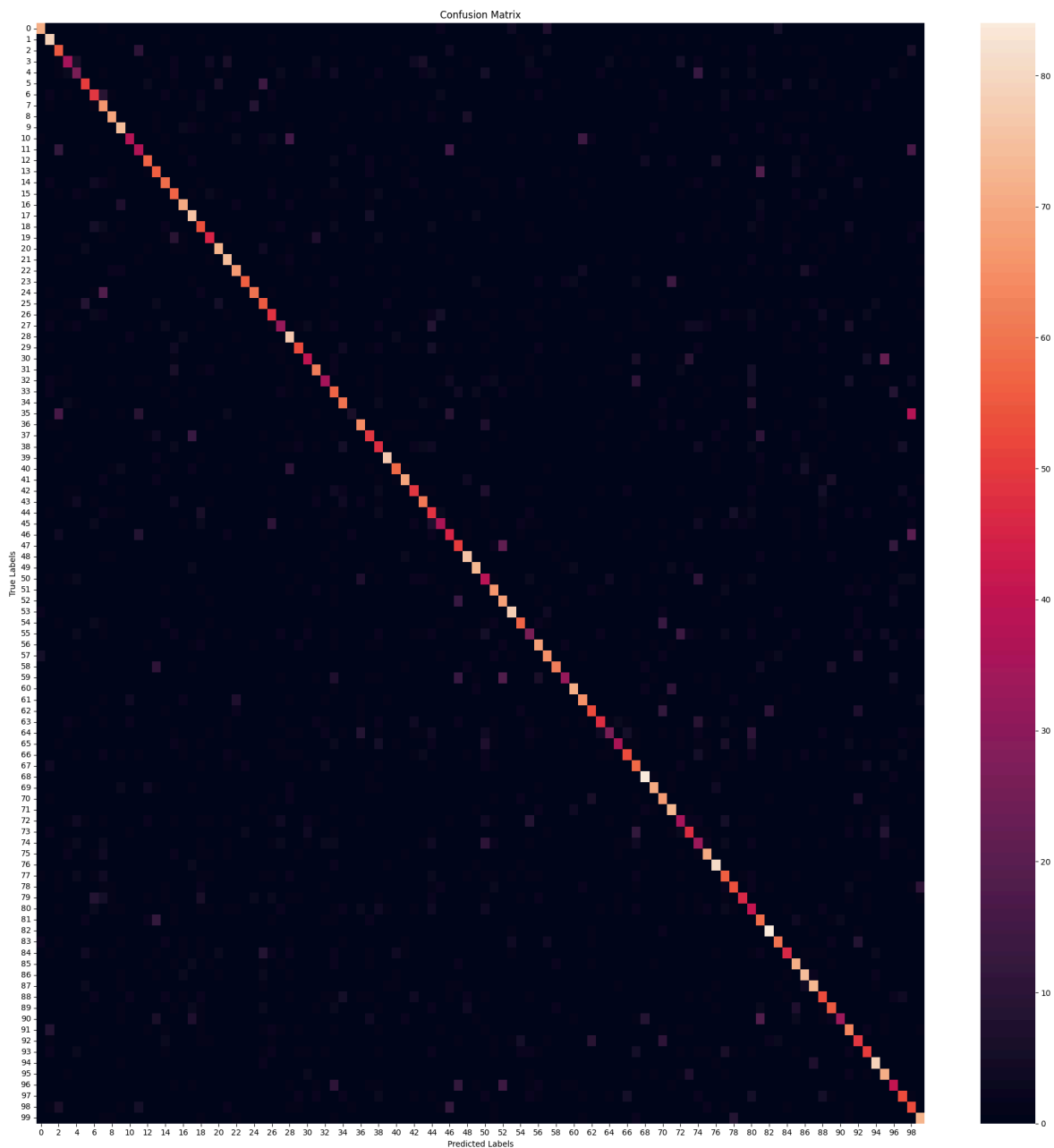
Epoch 19: val_loss did not improve from 1.31837

657/657 ————— **18s** 27ms/step - accuracy: 0.5817 - f1_score: 0.5780
 - loss: 1.4541 - precision: 0.7815 - val_accuracy: 0.6191 - val_f1_score: 0.6169
 - val_loss: 1.3234 - val_precision: 0.8199



141/141  **11s** 38ms/step - accuracy: 0.6154 - f1_score: 0.6043
- loss: 1.3345 - precision: 0.8197
Test Accuracy: 62.72%
Test Loss: 1.3070381879806519
Test Precision: 82.84%
Test F1 Scores (Per Class): [82.558136 76.69902 55.83756 46.66666 33.33333
60.975605
54.44444 59.192818 78.85714 76.68394 54.676254 45.398766
67.85713 58.63874 67.04545 58.461536 77.34807 71.698105
55.6701 57.324833 80. 76.68394 75.706215 73.20261
73.61964 57.591618 52.747242 42.10526 68.749985 65.8385
52.631577 68.508286 48.611107 50.220257 66.666664 11.8811865
70.329666 57.64705 49.999992 87.999985 69.87952 78.85714
57.64705 69.71428 45.71428 47.999996 47.872334 52.083332
78.12499 80.21977 38.23529 69.47368 63.25581 85.24589
66.66666 33.33333 70.10309 72.222206 77.49999 48.48484
80.681816 70.65217 63.473045 61.437904 39.999992 52.173912
66.25 53.95348 85.71428 80.92485 62.616817 73.36682
40.697666 56.28742 33.673462 77.77777 81.81817 54.36893
63.905323 63.013695 36.93693 52.44444 87.04663 70.58823
56.44171 69.607834 75.5102 75.64766 54.92227 62.499992
50.769226 63.636356 53.551907 57.471264 84.946236 66.35513
45.30386 62.352936 43.2653 76.59574]
Average Test F1 Scores:62.31

282/282  **18s** 38ms/step



	precision	recall	f1-score	support
beaver	0.87	0.79	0.83	90
dolphin	0.68	0.88	0.77	90
otter	0.51	0.61	0.56	90
seal	0.58	0.39	0.47	90
whale	0.39	0.29	0.33	90
aquarium fish	0.68	0.56	0.61	90
flatfish	0.54	0.54	0.54	90
ray	0.50	0.73	0.59	90
shark	0.81	0.77	0.79	90
trout	0.72	0.82	0.77	90
orchids	0.78	0.42	0.55	90
poppies	0.51	0.41	0.45	90
roses	0.73	0.63	0.68	90
sunflowers	0.55	0.62	0.58	90
tulips	0.69	0.66	0.67	90
bottles	0.54	0.63	0.58	90
bowls	0.77	0.78	0.77	90
cans	0.62	0.84	0.72	90
cups	0.52	0.60	0.56	90
plates	0.67	0.50	0.57	90
apples	0.80	0.80	0.80	90
mushrooms	0.72	0.82	0.77	90
oranges	0.77	0.74	0.76	90
pears	0.89	0.62	0.73	90
sweet peppers	0.82	0.67	0.74	90
clock	0.54	0.61	0.58	90
computer keyboard	0.52	0.53	0.53	90
lamp	0.52	0.36	0.42	90
telephone	0.57	0.86	0.69	90
television	0.75	0.59	0.66	90
bed	0.65	0.44	0.53	90
chair	0.68	0.69	0.69	90
couch	0.65	0.39	0.49	90
table	0.42	0.63	0.50	90
wardrobe	0.67	0.67	0.67	90
bee	0.55	0.07	0.12	90
beetle	0.70	0.71	0.70	90
butterfly	0.61	0.54	0.58	90
caterpillar	0.47	0.52	0.49	90
cockroach	0.91	0.86	0.88	90
bear	0.76	0.64	0.70	90
leopard	0.81	0.77	0.79	90
lion	0.61	0.54	0.58	90
tiger	0.71	0.68	0.69	90
wolf	0.40	0.53	0.46	90
bridge	0.60	0.40	0.48	90
castle	0.46	0.50	0.48	90
house	0.50	0.56	0.52	90
road	0.74	0.84	0.79	90
skyscraper	0.79	0.81	0.80	90
cloud	0.34	0.43	0.38	90
forest	0.66	0.73	0.69	90
mountain	0.54	0.76	0.63	90
plain	0.84	0.87	0.85	90
sea	0.70	0.63	0.67	90
camel	0.39	0.29	0.33	90
cattle	0.65	0.76	0.70	90
chimpanzee	0.72	0.72	0.72	90

elephant	0.89	0.69	0.78	90
kangaroo	0.76	0.36	0.48	90
fox	0.83	0.79	0.81	90
porcupine	0.69	0.72	0.71	90
possum	0.69	0.59	0.63	90
raccoon	0.75	0.52	0.61	90
skunk	0.65	0.29	0.40	90
crab	0.75	0.40	0.52	90
lobster	0.76	0.59	0.66	90
snail	0.46	0.64	0.54	90
spider	0.79	0.93	0.86	90
worm	0.84	0.78	0.81	90
baby	0.54	0.74	0.63	90
boy	0.67	0.81	0.73	90
girl	0.43	0.39	0.41	90
man	0.61	0.52	0.56	90
woman	0.31	0.37	0.34	90
crocodile	0.78	0.78	0.78	90
dinosaur	0.76	0.90	0.82	90
lizard	0.48	0.62	0.54	90
snake	0.68	0.60	0.64	90
turtle	0.82	0.51	0.63	90
hamster	0.31	0.46	0.37	90
mouse	0.44	0.66	0.52	90
rabbit	0.82	0.93	0.87	90
shrew	0.75	0.67	0.71	90
squirrel	0.63	0.51	0.56	90
maple	0.62	0.79	0.70	90
oak	0.70	0.82	0.76	90
palm	0.71	0.81	0.76	90
pine	0.51	0.59	0.55	90
willow	0.65	0.61	0.63	90
bicycle	0.82	0.37	0.51	90
bus	0.58	0.70	0.64	90
motorcycle	0.53	0.54	0.54	90
pickup truck	0.60	0.56	0.57	90
train	0.82	0.88	0.85	90
lawn-mower	0.57	0.79	0.66	90
rocket	0.45	0.46	0.45	90
streetcar	0.66	0.59	0.62	90
tank	0.34	0.59	0.43	90
tractor	0.73	0.80	0.77	90
accuracy			0.63	9000
macro avg	0.64	0.63	0.62	9000
weighted avg	0.64	0.63	0.62	9000

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1314/1314 ————— **0s** 26ms/step - accuracy: 0.5976 - f1_score: 0.5935
- loss: 1.3996 - precision: 0.7933
Epoch 1: val_loss improved from inf to 1.26219, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— **56s** 34ms/step - accuracy: 0.5976 - f1_score: 0.5935 - loss: 1.3996 - precision: 0.7933 - val_accuracy: 0.6444 - val_f1_score: 0.6417 - val_loss: 1.2622 - val_precision: 0.8278
Epoch 2/15
1313/1314 ————— **0s** 23ms/step - accuracy: 0.6212 - f1_score: 0.6178
- loss: 1.3049 - precision: 0.8064
Epoch 2: val_loss improved from 1.26219 to 1.25024, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6212 - f1_score: 0.6178 - loss: 1.3049 - precision: 0.8064 - val_accuracy: 0.6487 - val_f1_score: 0.6462 - val_loss: 1.2502 - val_precision: 0.8260
Epoch 3/15
1313/1314 ————— **0s** 23ms/step - accuracy: 0.6300 - f1_score: 0.6268
- loss: 1.2728 - precision: 0.8168
Epoch 3: val_loss improved from 1.25024 to 1.24453, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6300 - f1_score: 0.6268 - loss: 1.2728 - precision: 0.8168 - val_accuracy: 0.6503 - val_f1_score: 0.6478 - val_loss: 1.2445 - val_precision: 0.8260
Epoch 4/15
1314/1314 ————— **0s** 23ms/step - accuracy: 0.6359 - f1_score: 0.6329
- loss: 1.2550 - precision: 0.8188
Epoch 4: val_loss improved from 1.24453 to 1.23971, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6359 - f1_score: 0.6329 - loss: 1.2550 - precision: 0.8188 - val_accuracy: 0.6506 - val_f1_score: 0.6478 - val_loss: 1.2397 - val_precision: 0.8288
Epoch 5/15
1312/1314 ————— **0s** 23ms/step - accuracy: 0.6399 - f1_score: 0.6371
- loss: 1.2381 - precision: 0.8195
Epoch 5: val_loss improved from 1.23971 to 1.23538, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6399 - f1_score: 0.6371 - loss: 1.2381 - precision: 0.8195 - val_accuracy: 0.6500 - val_f1_score: 0.6474 - val_loss: 1.2354 - val_precision: 0.8271
Epoch 6/15
1314/1314 ————— **0s** 23ms/step - accuracy: 0.6457 - f1_score: 0.6424
- loss: 1.2256 - precision: 0.8241
Epoch 6: val_loss improved from 1.23538 to 1.23183, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6457 - f1_score: 0.6424 - loss: 1.2256 - precision: 0.8241 - val_accuracy: 0.6529 - val_f1_score: 0.6503 - val_loss: 1.2318 - val_precision: 0.8272
Epoch 7/15
1312/1314 ————— **0s** 23ms/step - accuracy: 0.6471 - f1_score: 0.6441
- loss: 1.2128 - precision: 0.8254
Epoch 7: val_loss improved from 1.23183 to 1.22846, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6471 - f1_score: 0.6441 - loss: 1.2129 - precision: 0.8254 - val_accuracy: 0.6530 - val_f1_score: 0.6507 - val_loss: 1.2285 - val_precision: 0.8284
Epoch 8/15
1313/1314 ————— **0s** 23ms/step - accuracy: 0.6477 - f1_score: 0.6449
- loss: 1.2066 - precision: 0.8280
Epoch 8: val_loss improved from 1.22846 to 1.22599, saving model to best_model_fine_tune_v5.keras

ne_tune_v5.keras

1314/1314 ————— **34s** 26ms/step - accuracy: 0.6477 - f1_score: 0.6449 - loss: 1.2066 - precision: 0.8280 - val_accuracy: 0.6546 - val_f1_score: 0.6523 - val_loss: 1.2260 - val_precision: 0.8280

Epoch 9/15

1314/1314 ————— **0s** 23ms/step - accuracy: 0.6535 - f1_score: 0.6504 - loss: 1.1891 - precision: 0.8274

Epoch 9: val_loss improved from 1.22599 to 1.22324, saving model to best_model_fine_tune_v5.keras

1314/1314 ————— **34s** 26ms/step - accuracy: 0.6535 - f1_score: 0.6504 - loss: 1.1891 - precision: 0.8274 - val_accuracy: 0.6550 - val_f1_score: 0.6530 - val_loss: 1.2232 - val_precision: 0.8291

Epoch 10/15

1314/1314 ————— **0s** 23ms/step - accuracy: 0.6568 - f1_score: 0.6542 - loss: 1.1824 - precision: 0.8329

Epoch 10: val_loss improved from 1.22324 to 1.22081, saving model to best_model_fine_tune_v5.keras

1314/1314 ————— **34s** 26ms/step - accuracy: 0.6568 - f1_score: 0.6542 - loss: 1.1824 - precision: 0.8329 - val_accuracy: 0.6558 - val_f1_score: 0.6535 - val_loss: 1.2208 - val_precision: 0.8289

Epoch 11/15

1313/1314 ————— **0s** 23ms/step - accuracy: 0.6581 - f1_score: 0.6553 - loss: 1.1848 - precision: 0.8362

Epoch 11: val_loss improved from 1.22081 to 1.21878, saving model to best_model_fine_tune_v5.keras

1314/1314 ————— **34s** 26ms/step - accuracy: 0.6581 - f1_score: 0.6553 - loss: 1.1848 - precision: 0.8362 - val_accuracy: 0.6560 - val_f1_score: 0.6537 - val_loss: 1.2188 - val_precision: 0.8304

Epoch 12/15

1313/1314 ————— **0s** 23ms/step - accuracy: 0.6589 - f1_score: 0.6562 - loss: 1.1730 - precision: 0.8374

Epoch 12: val_loss improved from 1.21878 to 1.21637, saving model to best_model_fine_tune_v5.keras

1314/1314 ————— **34s** 26ms/step - accuracy: 0.6589 - f1_score: 0.6562 - loss: 1.1730 - precision: 0.8374 - val_accuracy: 0.6567 - val_f1_score: 0.6545 - val_loss: 1.2164 - val_precision: 0.8308

Epoch 13/15

1313/1314 ————— **0s** 23ms/step - accuracy: 0.6619 - f1_score: 0.6592 - loss: 1.1669 - precision: 0.8391

Epoch 13: val_loss improved from 1.21637 to 1.21423, saving model to best_model_fine_tune_v5.keras

1314/1314 ————— **35s** 26ms/step - accuracy: 0.6619 - f1_score: 0.6592 - loss: 1.1669 - precision: 0.8391 - val_accuracy: 0.6567 - val_f1_score: 0.6547 - val_loss: 1.2142 - val_precision: 0.8306

Epoch 14/15

1313/1314 ————— **0s** 23ms/step - accuracy: 0.6617 - f1_score: 0.6589 - loss: 1.1597 - precision: 0.8417

Epoch 14: val_loss improved from 1.21423 to 1.21295, saving model to best_model_fine_tune_v5.keras

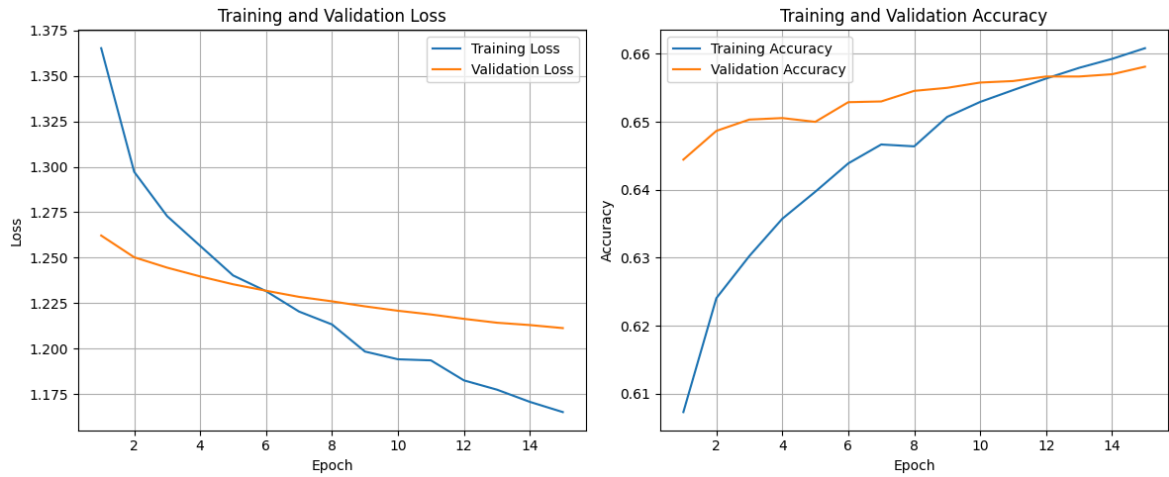
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6617 - f1_score: 0.6589 - loss: 1.1598 - precision: 0.8417 - val_accuracy: 0.6570 - val_f1_score: 0.6549 - val_loss: 1.2129 - val_precision: 0.8320

Epoch 15/15

1314/1314 ————— **0s** 23ms/step - accuracy: 0.6648 - f1_score: 0.6618 - loss: 1.1521 - precision: 0.8396

Epoch 15: val_loss improved from 1.21295 to 1.21128, saving model to best_model_fine_tune_v5.keras

1314/1314 ————— **35s** 26ms/step - accuracy: 0.6648 - f1_score: 0.6618 - loss: 1.1521 - precision: 0.8396 - val_accuracy: 0.6581 - val_f1_score: 0.6559 - val_loss: 1.2113 - val_precision: 0.8334



141/141 ————— **11s** 38ms/step - accuracy: 0.6467 - f1_score: 0.6385

- loss: 1.2265 - precision: 0.8277

Test Accuracy: 65.97%

Test Loss: 1.1988836526870728

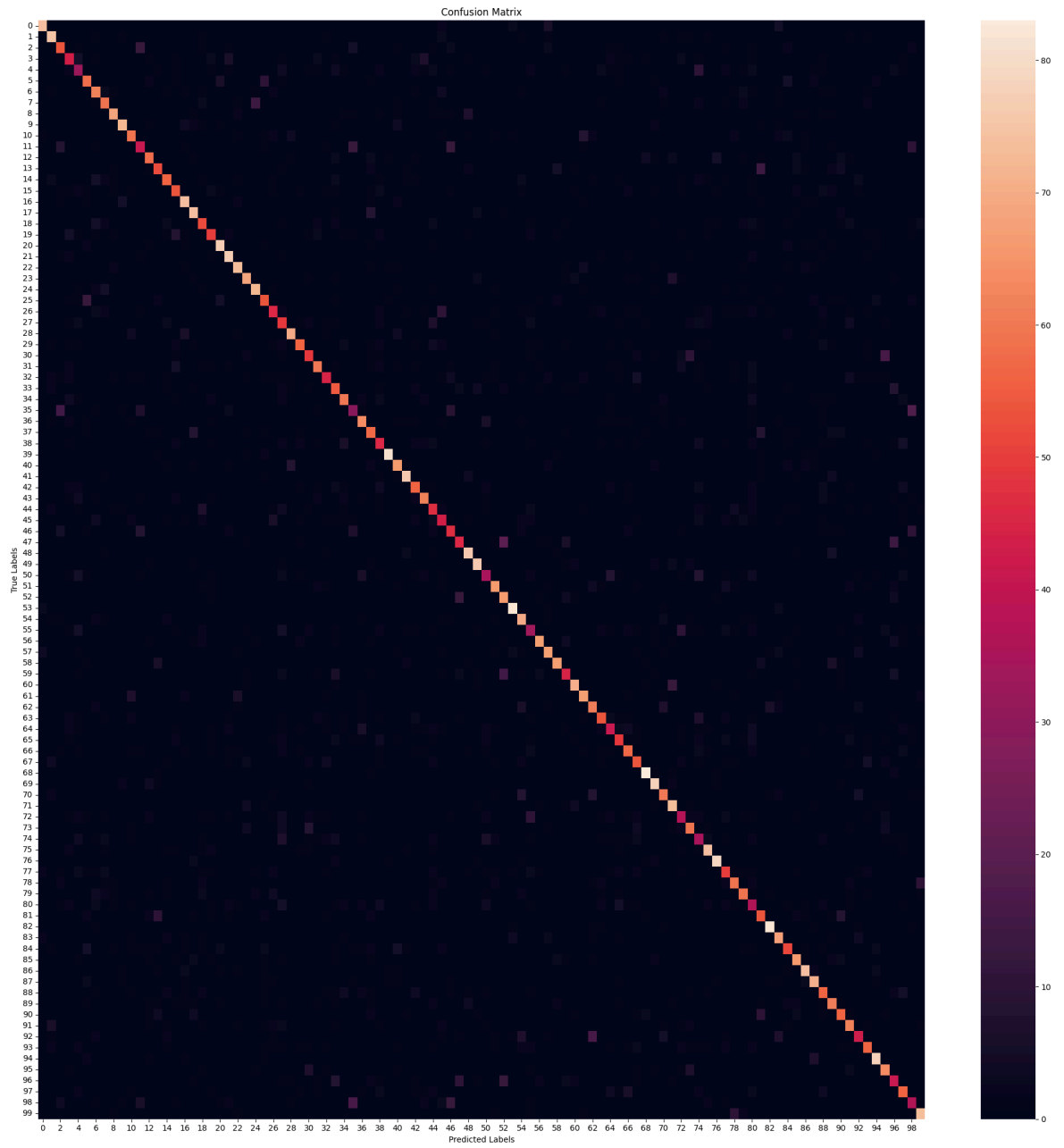
Test Precision: 83.75%

Test F1 Scores (Per Class): [84.090904 77.31957 57.754005 50.574715 37.714283 63.04347 63.2653

66.29213	79.768776	82.285706	67.42856	47.953213	67.44185	60.1156
65.11627	61.363636	77.08333	75.89743	57.45856	59.523804	82.7957
78.97436	79.347824	77.09496	79.12087	61.363636	55.421684	46.889946
79.09604	69.135796	57.309933	68.965515	56.96202	54.45544	68.181816
35.15151	69.94535	61.62162	50.	91.52542	70.967735	84.44444
62.569828	71.67629	49.47368	49.438198	50.53763	55.089813	79.58114
82.608696	44.02515	75.428566	63.768112	88.17203	68.965515	40.23668
78.82352	74.03315	77.906975	55.214714	79.775276	75.28088	65.24063
63.905323	49.999992	58.33332	64.04494	59.999996	88.770035	81.24999
67.415726	74.87179	44.31137	64.480865	37.23404	79.14438	87.29281
56.818176	66.666664	68.208084	36.734688	56.842102	89.61748	71.57894
60.355026	72.52747	80.87432	77.173904	60.540535	65.24063	62.222218
67.02702	53.749996	65.49707	83.87096	68.0851	45.81005	61.87845
41.142853	78.68852					

Average Test F1 Scores:65.80

282/282 ————— **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.86	0.82	0.84	90
dolphin	0.72	0.83	0.77	90
otter	0.56	0.60	0.58	90
seal	0.52	0.49	0.51	90
whale	0.39	0.37	0.38	90
aquarium fish	0.62	0.64	0.63	90
flatfish	0.58	0.69	0.63	90
ray	0.67	0.66	0.66	90
shark	0.83	0.77	0.80	90
trout	0.85	0.80	0.82	90
orchids	0.69	0.66	0.67	90
poppies	0.51	0.46	0.48	90
roses	0.71	0.64	0.67	90
sunflowers	0.63	0.58	0.60	90
tulips	0.68	0.62	0.65	90
bottles	0.63	0.60	0.61	90
bowls	0.73	0.82	0.77	90
cans	0.70	0.82	0.76	90
cups	0.57	0.58	0.57	90
plates	0.64	0.56	0.60	90
apples	0.80	0.86	0.83	90
mushrooms	0.73	0.86	0.79	90
oranges	0.78	0.81	0.79	90
pears	0.78	0.77	0.77	90
sweet peppers	0.78	0.80	0.79	90
clock	0.63	0.60	0.61	90
computer keyboard	0.61	0.51	0.55	90
lamp	0.41	0.54	0.47	90
telephone	0.80	0.78	0.79	90
television	0.78	0.62	0.69	90
bed	0.60	0.54	0.57	90
chair	0.71	0.67	0.69	90
couch	0.66	0.50	0.57	90
table	0.49	0.61	0.54	90
wardrobe	0.70	0.67	0.68	90
bee	0.39	0.32	0.35	90
beetle	0.69	0.71	0.70	90
butterfly	0.60	0.63	0.62	90
caterpillar	0.49	0.51	0.50	90
cockroach	0.93	0.90	0.92	90
bear	0.69	0.73	0.71	90
leopard	0.84	0.84	0.84	90
lion	0.63	0.62	0.63	90
tiger	0.75	0.69	0.72	90
wolf	0.47	0.52	0.49	90
bridge	0.50	0.49	0.49	90
castle	0.49	0.52	0.51	90
house	0.60	0.51	0.55	90
road	0.75	0.84	0.80	90
skyscraper	0.81	0.84	0.83	90
cloud	0.50	0.39	0.44	90
forest	0.77	0.73	0.75	90
mountain	0.56	0.73	0.64	90
plain	0.85	0.91	0.88	90
sea	0.62	0.78	0.69	90
camel	0.43	0.38	0.40	90
cattle	0.84	0.74	0.79	90
chimpanzee	0.74	0.74	0.74	90

elephant	0.82	0.74	0.78	90
kangaroo	0.62	0.50	0.55	90
fox	0.81	0.79	0.80	90
porcupine	0.76	0.74	0.75	90
possum	0.63	0.68	0.65	90
raccoon	0.69	0.60	0.64	90
skunk	0.55	0.46	0.50	90
crab	0.63	0.54	0.58	90
lobster	0.65	0.63	0.64	90
snail	0.61	0.60	0.60	90
spider	0.86	0.92	0.89	90
worm	0.76	0.87	0.81	90
baby	0.68	0.67	0.67	90
boy	0.70	0.81	0.75	90
girl	0.48	0.41	0.44	90
man	0.63	0.66	0.64	90
woman	0.36	0.39	0.37	90
crocodile	0.76	0.82	0.79	90
dinosaur	0.87	0.88	0.87	90
lizard	0.58	0.56	0.57	90
snake	0.69	0.66	0.67	90
turtle	0.71	0.66	0.68	90
hamster	0.34	0.40	0.37	90
mouse	0.54	0.60	0.57	90
rabbit	0.88	0.91	0.90	90
shrew	0.68	0.76	0.72	90
squirrel	0.65	0.57	0.60	90
maple	0.72	0.73	0.73	90
oak	0.80	0.82	0.81	90
palm	0.76	0.79	0.77	90
pine	0.59	0.62	0.61	90
willow	0.63	0.68	0.65	90
bicycle	0.62	0.62	0.62	90
bus	0.65	0.69	0.67	90
motorcycle	0.61	0.48	0.54	90
pickup truck	0.69	0.62	0.65	90
train	0.81	0.87	0.84	90
lawn-mower	0.65	0.71	0.68	90
rocket	0.46	0.46	0.46	90
streetcar	0.62	0.62	0.62	90
tank	0.42	0.40	0.41	90
tractor	0.78	0.81	0.79	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

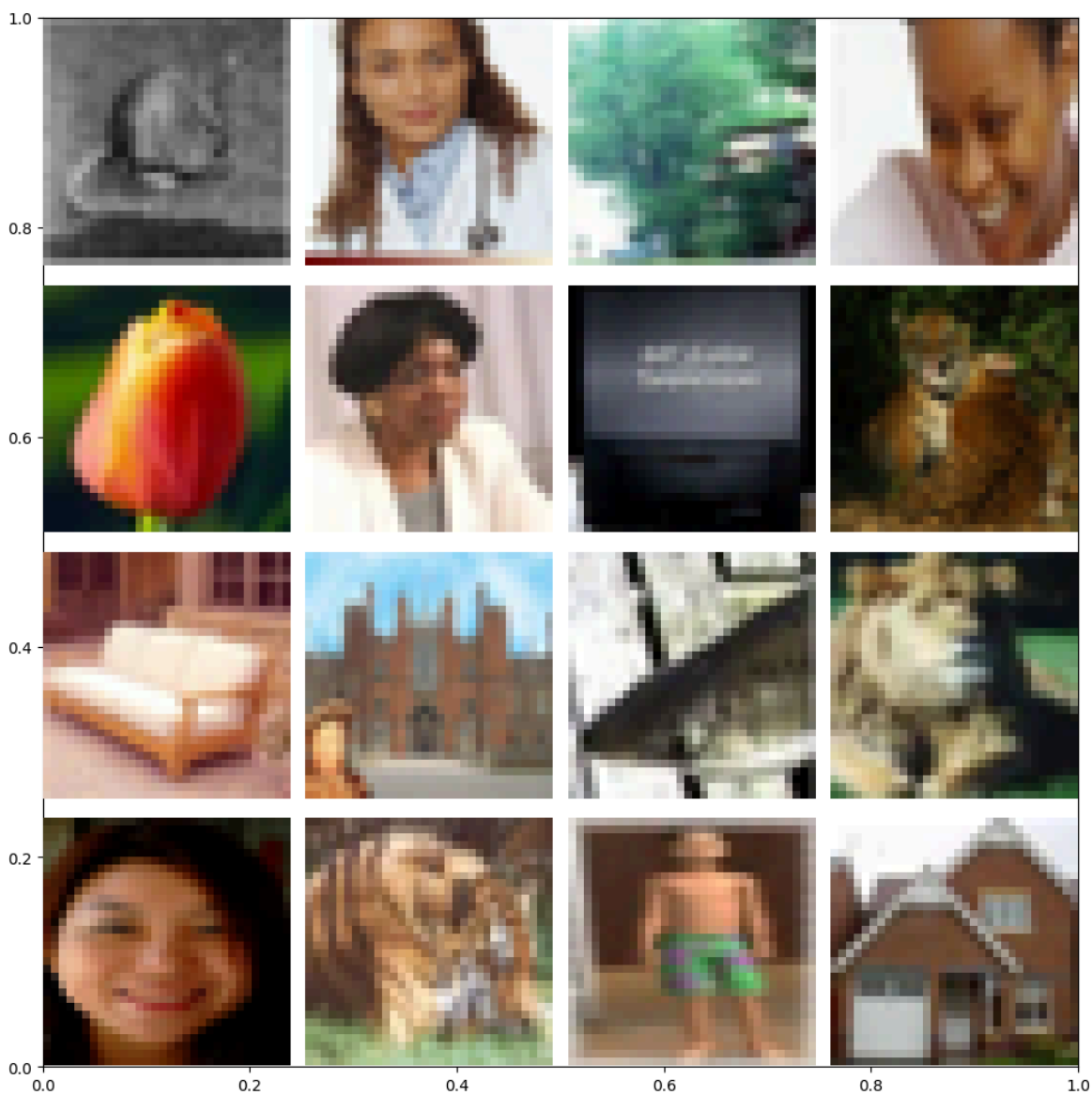
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

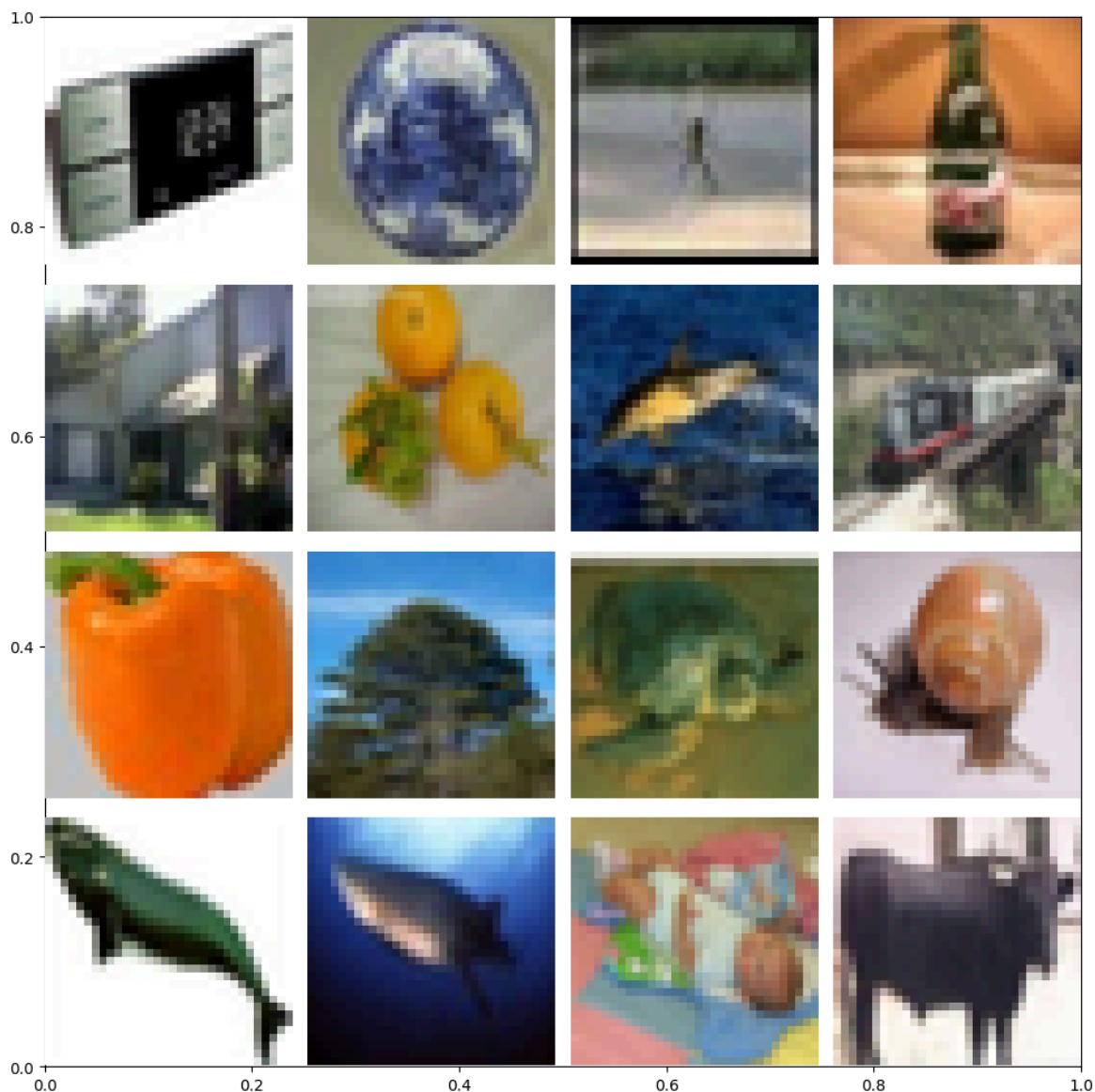
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[12903].shape: (1,)  
imgs[12903].shape: (32, 32, 3)
```

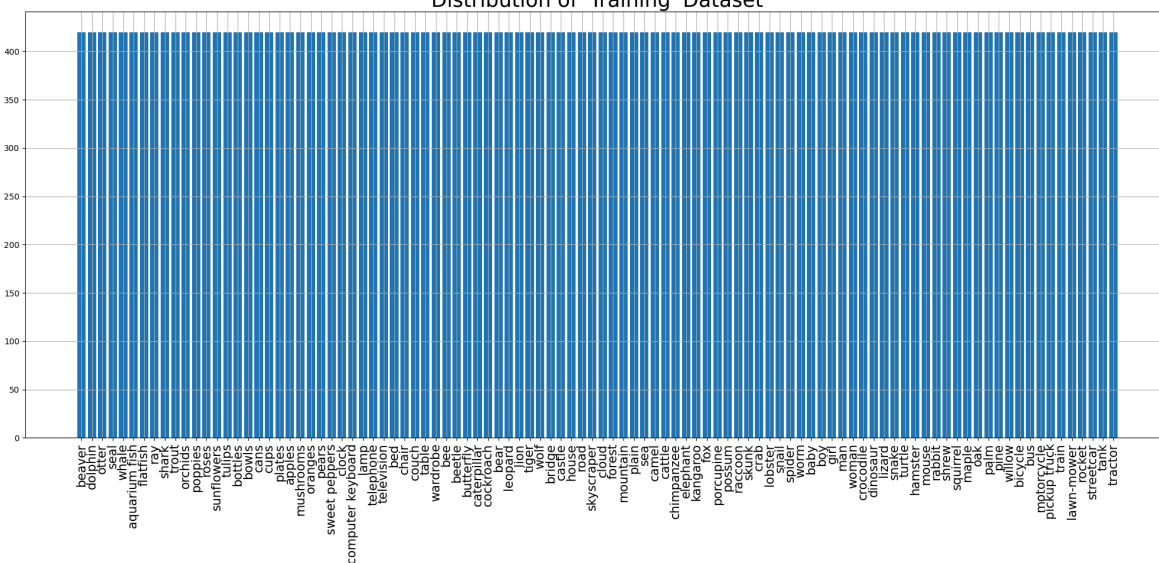


```
labels[33654].shape: (1,)  
imgs[33654].shape: (32, 32, 3)
```

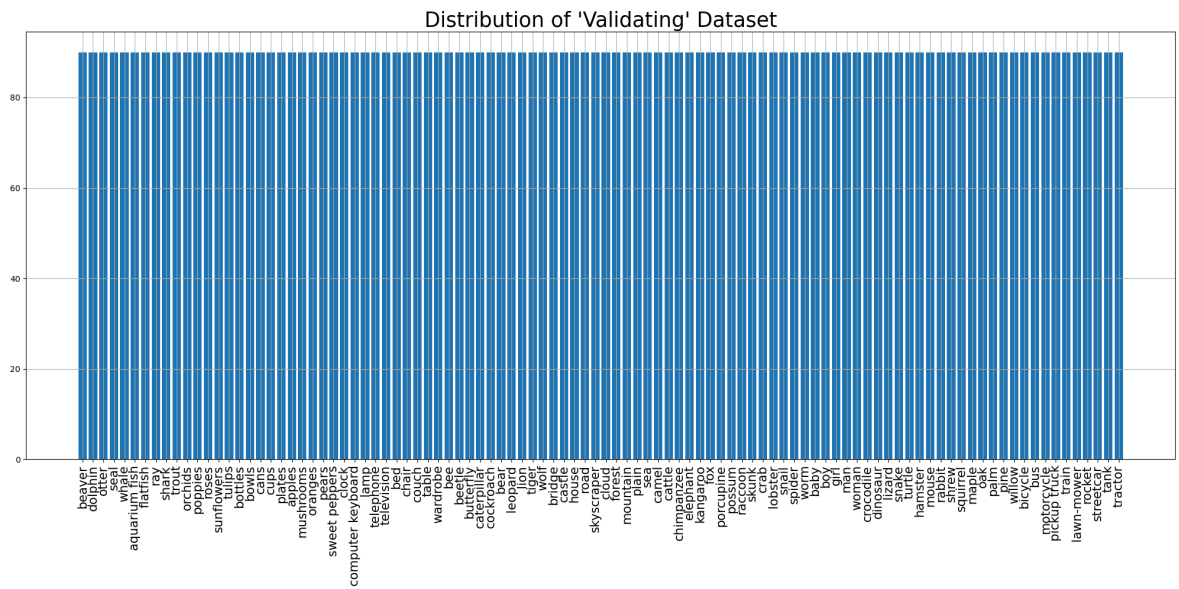



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

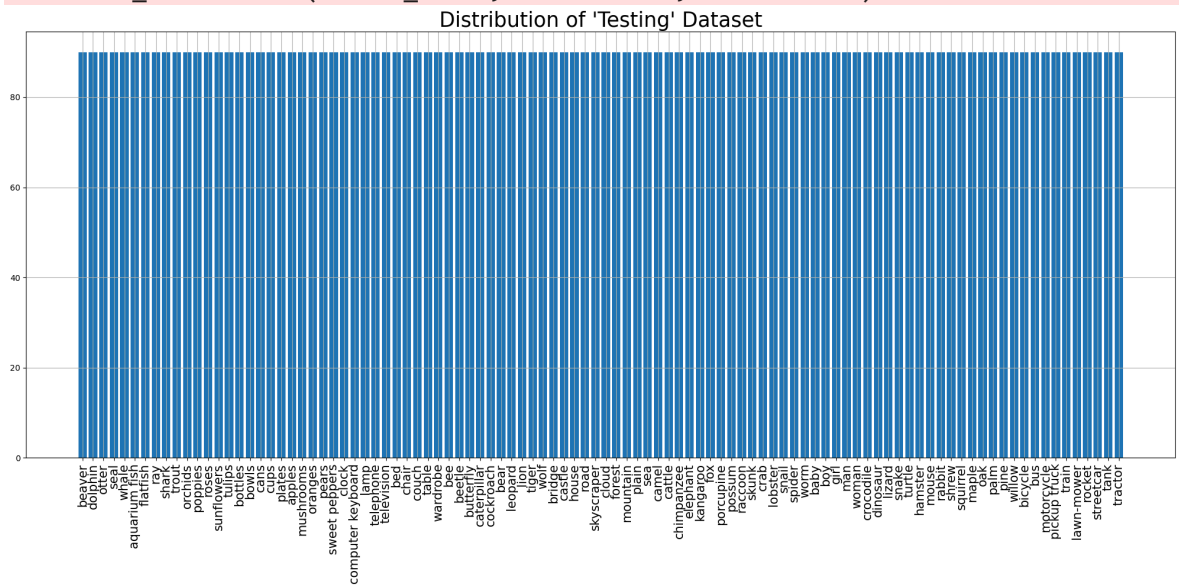
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657  **0s** 29ms/step - accuracy: 0.2628 - f1_score: 0.2572 - loss: 3.2066 - precision: 0.6431
Epoch 1: val_loss improved from inf to 1.59391, saving model to best_model_v5.keras
657/657  **40s** 45ms/step - accuracy: 0.2629 - f1_score: 0.2573 - loss: 3.2056 - precision: 0.6431 - val_accuracy: 0.5712 - val_f1_score: 0.5640 - val_loss: 1.5939 - val_precision: 0.8365
Epoch 2/25
656/657  **0s** 23ms/step - accuracy: 0.4781 - f1_score: 0.4729 - loss: 1.9402 - precision: 0.7285
Epoch 2: val_loss improved from 1.59391 to 1.48568, saving model to best_model_v5.keras
657/657  **19s** 29ms/step - accuracy: 0.4781 - f1_score: 0.4730 - loss: 1.9400 - precision: 0.7285 - val_accuracy: 0.5873 - val_f1_score: 0.5850 - val_loss: 1.4857 - val_precision: 0.8260
Epoch 3/25
655/657  **0s** 23ms/step - accuracy: 0.5091 - f1_score: 0.5038 - loss: 1.7838 - precision: 0.7445
Epoch 3: val_loss improved from 1.48568 to 1.42025, saving model to best_model_v5.keras
657/657  **19s** 29ms/step - accuracy: 0.5091 - f1_score: 0.5039 - loss: 1.7836 - precision: 0.7445 - val_accuracy: 0.5991 - val_f1_score: 0.5935 - val_loss: 1.4202 - val_precision: 0.8345
Epoch 4/25
655/657  **0s** 23ms/step - accuracy: 0.5281 - f1_score: 0.5236 - loss: 1.7022 - precision: 0.7502
Epoch 4: val_loss improved from 1.42025 to 1.38532, saving model to best_model_v5.keras
657/657  **19s** 29ms/step - accuracy: 0.5282 - f1_score: 0.5237 - loss: 1.7021 - precision: 0.7503 - val_accuracy: 0.6126 - val_f1_score: 0.6090 - val_loss: 1.3853 - val_precision: 0.8215
Epoch 5/25
657/657  **0s** 23ms/step - accuracy: 0.5364 - f1_score: 0.5323 - loss: 1.6554 - precision: 0.7630
Epoch 5: val_loss improved from 1.38532 to 1.37420, saving model to best_model_v5.keras
657/657  **19s** 29ms/step - accuracy: 0.5364 - f1_score: 0.5324 - loss: 1.6554 - precision: 0.7631 - val_accuracy: 0.6107 - val_f1_score: 0.6057 - val_loss: 1.3742 - val_precision: 0.8255
Epoch 6/25
655/657  **0s** 23ms/step - accuracy: 0.5477 - f1_score: 0.5436 - loss: 1.6095 - precision: 0.7649
Epoch 6: val_loss improved from 1.37420 to 1.34991, saving model to best_model_v5.keras
657/657  **19s** 29ms/step - accuracy: 0.5477 - f1_score: 0.5437 - loss: 1.6094 - precision: 0.7650 - val_accuracy: 0.6198 - val_f1_score: 0.6153 - val_loss: 1.3499 - val_precision: 0.8315
Epoch 7/25
655/657  **0s** 23ms/step - accuracy: 0.5557 - f1_score: 0.5518 - loss: 1.5825 - precision: 0.7709
Epoch 7: val_loss did not improve from 1.34991
657/657  **18s** 27ms/step - accuracy: 0.5557 - f1_score: 0.5518 - loss: 1.5824 - precision: 0.7709 - val_accuracy: 0.6203 - val_f1_score: 0.6160 - val_loss: 1.3605 - val_precision: 0.8315
Epoch 8/25
657/657  **0s** 23ms/step - accuracy: 0.5598 - f1_score: 0.5555 - loss: 1.5654 - precision: 0.7687
Epoch 8: val_loss improved from 1.34991 to 1.34460, saving model to best_model_v5.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5598 - f1_score: 0.5555
- loss: 1.5654 - precision: 0.7687 - val_accuracy: 0.6198 - val_f1_score: 0.6162
- val_loss: 1.3446 - val_precision: 0.8301
Epoch 9/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5566 - f1_score: 0.5520 -
loss: 1.5398 - precision: 0.7666
Epoch 9: val_loss did not improve from 1.34460

657/657 ————— **18s** 27ms/step - accuracy: 0.5566 - f1_score: 0.5521
- loss: 1.5397 - precision: 0.7666 - val_accuracy: 0.6196 - val_f1_score: 0.6162
- val_loss: 1.3536 - val_precision: 0.8276
Epoch 10/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5646 - f1_score: 0.5605 -
loss: 1.5312 - precision: 0.7746
Epoch 10: val_loss did not improve from 1.34460

657/657 ————— **18s** 28ms/step - accuracy: 0.5647 - f1_score: 0.5606
- loss: 1.5311 - precision: 0.7746 - val_accuracy: 0.6256 - val_f1_score: 0.6210
- val_loss: 1.3448 - val_precision: 0.8283
Epoch 11/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5632 - f1_score: 0.5600 -
loss: 1.5145 - precision: 0.7682
Epoch 11: val_loss improved from 1.34460 to 1.33719, saving model to best_model_v5.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5632 - f1_score: 0.5601
- loss: 1.5145 - precision: 0.7682 - val_accuracy: 0.6222 - val_f1_score: 0.6177
- val_loss: 1.3372 - val_precision: 0.8235
Epoch 12/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5731 - f1_score: 0.5696 -
loss: 1.5022 - precision: 0.7817
Epoch 12: val_loss improved from 1.33719 to 1.32147, saving model to best_model_v5.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5731 - f1_score: 0.5696
- loss: 1.5021 - precision: 0.7817 - val_accuracy: 0.6273 - val_f1_score: 0.6243
- val_loss: 1.3215 - val_precision: 0.8291
Epoch 13/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5735 - f1_score: 0.5691 -
loss: 1.5060 - precision: 0.7733
Epoch 13: val_loss did not improve from 1.32147

657/657 ————— **18s** 27ms/step - accuracy: 0.5735 - f1_score: 0.5691
- loss: 1.5060 - precision: 0.7733 - val_accuracy: 0.6193 - val_f1_score: 0.6179
- val_loss: 1.3351 - val_precision: 0.8092
Epoch 14/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5739 - f1_score: 0.5701 -
loss: 1.4874 - precision: 0.7750
Epoch 14: val_loss improved from 1.32147 to 1.31570, saving model to best_model_v5.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5739 - f1_score: 0.5702
- loss: 1.4873 - precision: 0.7750 - val_accuracy: 0.6316 - val_f1_score: 0.6273
- val_loss: 1.3157 - val_precision: 0.8283
Epoch 15/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5732 - f1_score: 0.5696 -
loss: 1.4847 - precision: 0.7750
Epoch 15: val_loss did not improve from 1.31570

657/657 ————— **18s** 27ms/step - accuracy: 0.5733 - f1_score: 0.5697
- loss: 1.4846 - precision: 0.7750 - val_accuracy: 0.6193 - val_f1_score: 0.6152
- val_loss: 1.3241 - val_precision: 0.8167
Epoch 16/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5774 - f1_score: 0.5740 -
loss: 1.4838 - precision: 0.7780
Epoch 16: val_loss improved from 1.31570 to 1.31563, saving model to best_model_v5.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5774 - f1_score: 0.5740
- loss: 1.4837 - precision: 0.7780 - val_accuracy: 0.6289 - val_f1_score: 0.6254
- val_loss: 1.3156 - val_precision: 0.8116
Epoch 17/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5760 - f1_score: 0.5729 -
loss: 1.4752 - precision: 0.7713
Epoch 17: val_loss did not improve from 1.31563

657/657 ————— **18s** 27ms/step - accuracy: 0.5760 - f1_score: 0.5729
- loss: 1.4751 - precision: 0.7713 - val_accuracy: 0.6264 - val_f1_score: 0.6233
- val_loss: 1.3267 - val_precision: 0.8196
Epoch 18/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5833 - f1_score: 0.5797 -
loss: 1.4688 - precision: 0.7762
Epoch 18: val_loss improved from 1.31563 to 1.30426, saving model to best_model_v5.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5834 - f1_score: 0.5797
- loss: 1.4688 - precision: 0.7762 - val_accuracy: 0.6308 - val_f1_score: 0.6286
- val_loss: 1.3043 - val_precision: 0.8269
Epoch 19/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5801 - f1_score: 0.5764 -
loss: 1.4497 - precision: 0.7779
Epoch 19: val_loss did not improve from 1.30426

657/657 ————— **18s** 27ms/step - accuracy: 0.5801 - f1_score: 0.5764
- loss: 1.4496 - precision: 0.7779 - val_accuracy: 0.6224 - val_f1_score: 0.6189
- val_loss: 1.3159 - val_precision: 0.8101
Epoch 20/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5827 - f1_score: 0.5790 -
loss: 1.4593 - precision: 0.7809
Epoch 20: val_loss did not improve from 1.30426

657/657 ————— **18s** 27ms/step - accuracy: 0.5827 - f1_score: 0.5790
- loss: 1.4593 - precision: 0.7809 - val_accuracy: 0.6256 - val_f1_score: 0.6214
- val_loss: 1.3114 - val_precision: 0.8134
Epoch 21/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5866 - f1_score: 0.5832 -
loss: 1.4416 - precision: 0.7781
Epoch 21: val_loss improved from 1.30426 to 1.30074, saving model to best_model_v5.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5866 - f1_score: 0.5832
- loss: 1.4416 - precision: 0.7781 - val_accuracy: 0.6319 - val_f1_score: 0.6299
- val_loss: 1.3007 - val_precision: 0.8226
Epoch 22/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5834 - f1_score: 0.5799 -
loss: 1.4441 - precision: 0.7739
Epoch 22: val_loss did not improve from 1.30074

657/657 ————— **18s** 27ms/step - accuracy: 0.5834 - f1_score: 0.5799
- loss: 1.4440 - precision: 0.7739 - val_accuracy: 0.6289 - val_f1_score: 0.6265
- val_loss: 1.3059 - val_precision: 0.8210
Epoch 23/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5830 - f1_score: 0.5796 -
loss: 1.4430 - precision: 0.7802
Epoch 23: val_loss did not improve from 1.30074

657/657 ————— **18s** 27ms/step - accuracy: 0.5830 - f1_score: 0.5796
- loss: 1.4430 - precision: 0.7803 - val_accuracy: 0.6201 - val_f1_score: 0.6167
- val_loss: 1.3165 - val_precision: 0.8139
Epoch 24/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5829 - f1_score: 0.5791 -
loss: 1.4510 - precision: 0.7791
Epoch 24: val_loss did not improve from 1.30074

657/657 ————— **18s** 27ms/step - accuracy: 0.5829 - f1_score: 0.5791
- loss: 1.4509 - precision: 0.7791 - val_accuracy: 0.6264 - val_f1_score: 0.6240

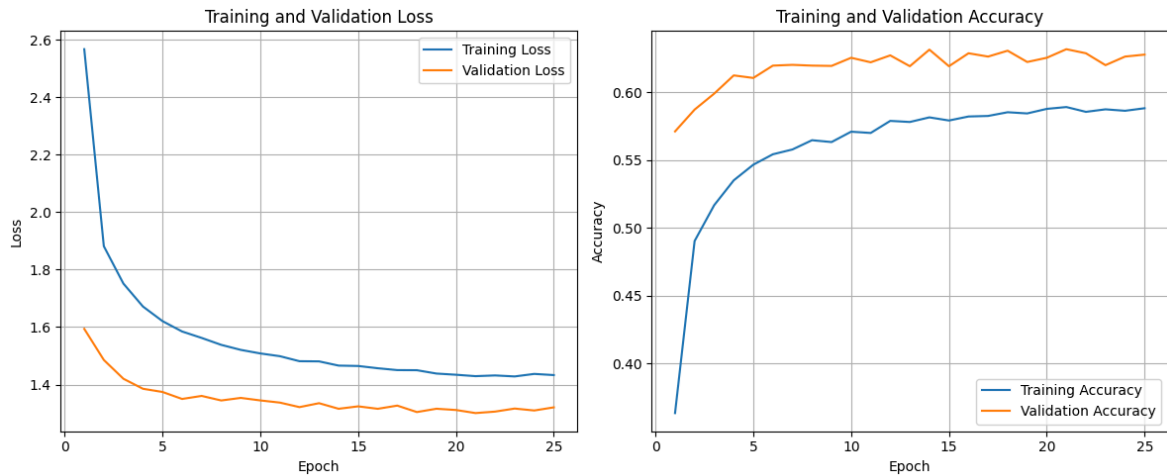
- val_loss: 1.3097 - val_precision: 0.8212

Epoch 25/25

655/657 ————— 0s 23ms/step - accuracy: 0.5840 - f1_score: 0.5805 -
loss: 1.4518 - precision: 0.7767

Epoch 25: val_loss did not improve from 1.30074

657/657 ————— 18s 27ms/step - accuracy: 0.5841 - f1_score: 0.5805
- loss: 1.4517 - precision: 0.7767 - val_accuracy: 0.6279 - val_f1_score: 0.6253
- val_loss: 1.3207 - val_precision: 0.8205



141/141 ————— 11s 38ms/step - accuracy: 0.6178 - f1_score: 0.6114
- loss: 1.3178 - precision: 0.8194

Test Accuracy: 62.94%

Test Loss: 1.2941255569458008

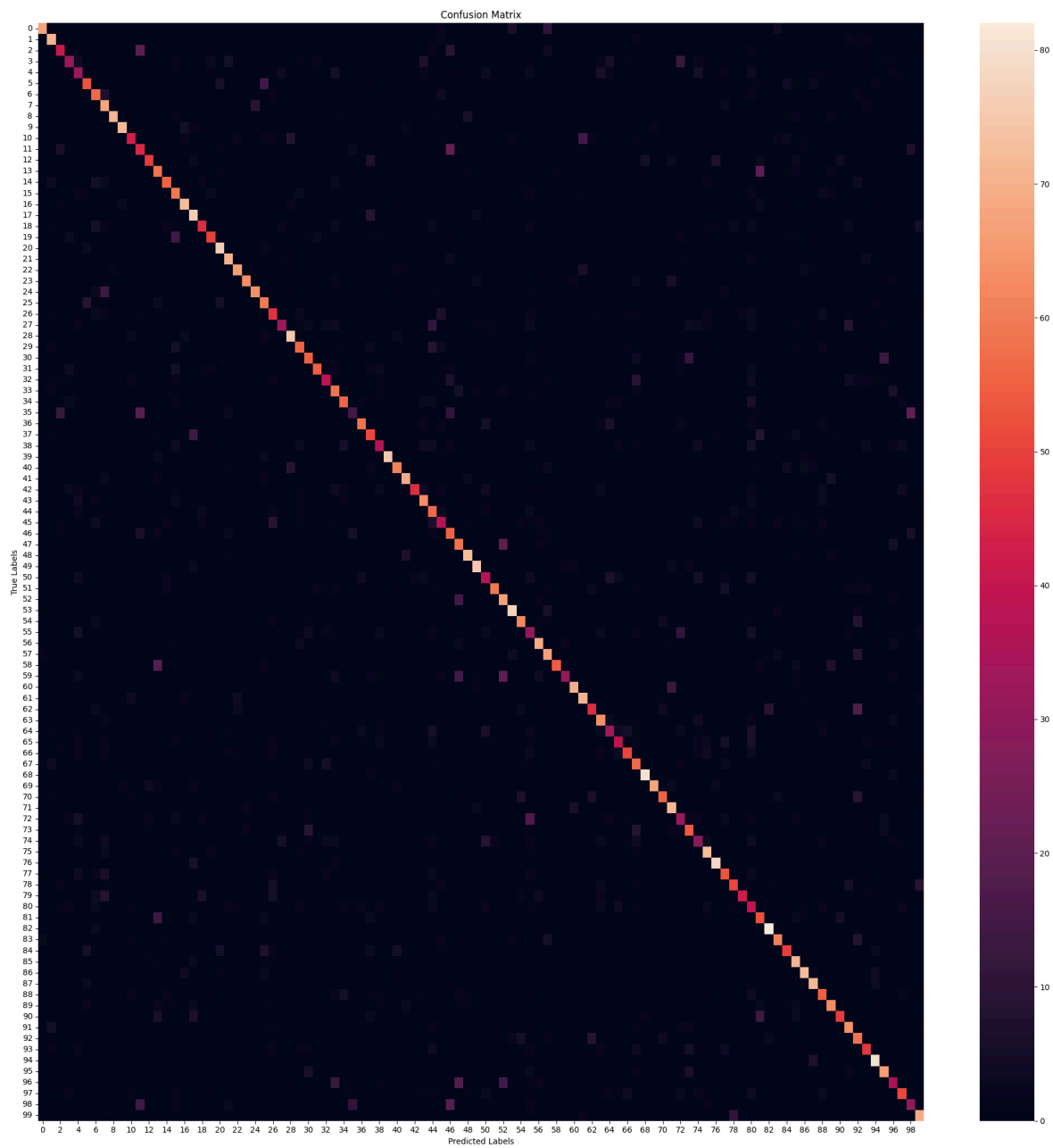
Test Precision: 82.65%

Test F1 Scores (Per Class): [81.9277 73.57513 50.61728 41.059597 34.44444 6
0.919537 57.575756

61.26125	80.681816	83.72092	57.718117	45.685272	62.420376	56.310677
65.11627	55.660374	78.07486	69.72476	56.097557	62.5	80.42328
76.34408	76.300575	78.75	76.190475	57.416267	49.735443	44.295296
75.75757	64.739876	59.7826	70.44025	49.681526	52.25224	66.666664
22.900757	69.87952	53.403137	48.684204	89.82036	69.71428	76.24309
62.162163	67.74193	49.78165	42.52873	48.484848	54.46009	81.56424
82.41757	42.937847	71.51515	61.187206	83.69564	69.27374	30.769226
70.76922	68.0203	73.469376	46.969692	80.	73.19587	57.861633
64.3216	42.857136	49.681526	64.55696	56.43564	87.43169	83.95061
66.27218	72.72727	36.363632	58.378376	36.024837	72.63681	84.7826
56.989246	62.962955	59.154922	33.61344	50.48543	87.23404	70.520226
59.036137	71.71716	79.347824	74.99999	61.111103	68.852455	62.820507
67.0157	52.777767	55.49132	82.05128	71.65774	42.424236	62.576683
37.26708	75.40983					

Average Test F1 Scores:62.85

282/282 ————— 11s 28ms/step



	precision	recall	f1-score	support
beaver	0.89	0.76	0.82	90
dolphin	0.69	0.79	0.74	90
otter	0.57	0.46	0.51	90
seal	0.51	0.34	0.41	90
whale	0.35	0.36	0.35	90
aquarium fish	0.63	0.59	0.61	90
flatfish	0.53	0.63	0.58	90
ray	0.52	0.76	0.61	90
shark	0.83	0.79	0.81	90
trout	0.88	0.80	0.84	90
orchids	0.73	0.48	0.58	90
poppies	0.42	0.50	0.46	90
roses	0.73	0.54	0.62	90
sunflowers	0.50	0.64	0.56	90
tulips	0.68	0.62	0.65	90
bottles	0.48	0.66	0.55	90
bowls	0.75	0.81	0.78	90
cans	0.59	0.84	0.70	90
cups	0.62	0.51	0.56	90
plates	0.71	0.56	0.62	90
apples	0.77	0.84	0.80	90
mushrooms	0.74	0.79	0.76	90
oranges	0.80	0.73	0.76	90
pears	0.90	0.70	0.79	90
sweet peppers	0.82	0.71	0.76	90
clock	0.50	0.67	0.57	90
computer keyboard	0.47	0.52	0.50	90
lamp	0.55	0.36	0.43	90
telephone	0.69	0.83	0.76	90
television	0.67	0.62	0.65	90
bed	0.59	0.61	0.60	90
chair	0.81	0.61	0.70	90
couch	0.58	0.43	0.50	90
table	0.44	0.64	0.52	90
wardrobe	0.73	0.62	0.67	90
bee	0.37	0.17	0.23	90
beetle	0.76	0.64	0.70	90
butterfly	0.50	0.57	0.53	90
caterpillar	0.59	0.41	0.48	90
cockroach	0.97	0.83	0.90	90
bear	0.72	0.68	0.70	90
leopard	0.76	0.77	0.76	90
lion	0.79	0.51	0.62	90
tiger	0.66	0.70	0.68	90
wolf	0.41	0.63	0.50	90
bridge	0.44	0.41	0.43	90
castle	0.40	0.62	0.48	90
house	0.47	0.64	0.54	90
road	0.82	0.81	0.82	90
skyscraper	0.82	0.83	0.82	90
cloud	0.44	0.42	0.43	90
forest	0.80	0.66	0.72	90
mountain	0.52	0.74	0.61	90
plain	0.82	0.86	0.84	90
sea	0.70	0.69	0.69	90
camel	0.28	0.33	0.31	90
cattle	0.66	0.77	0.71	90
chimpanzee	0.63	0.74	0.68	90

elephant	0.95	0.60	0.73	90
kangaroo	0.74	0.34	0.47	90
fox	0.82	0.78	0.80	90
porcupine	0.68	0.79	0.73	90
possum	0.67	0.51	0.58	90
raccoon	0.59	0.71	0.64	90
skunk	0.51	0.37	0.43	90
crab	0.58	0.43	0.50	90
lobster	0.75	0.57	0.65	90
snail	0.51	0.63	0.56	90
spider	0.86	0.89	0.87	90
worm	0.94	0.76	0.84	90
baby	0.71	0.62	0.66	90
boy	0.67	0.80	0.73	90
girl	0.37	0.36	0.36	90
man	0.57	0.60	0.58	90
woman	0.41	0.32	0.36	90
crocodile	0.66	0.81	0.73	90
dinosaur	0.83	0.87	0.85	90
lizard	0.55	0.59	0.57	90
snake	0.71	0.57	0.63	90
turtle	0.81	0.47	0.59	90
hamster	0.27	0.44	0.34	90
mouse	0.45	0.58	0.50	90
rabbit	0.84	0.91	0.87	90
shrew	0.73	0.68	0.71	90
squirrel	0.64	0.54	0.59	90
maple	0.66	0.79	0.72	90
oak	0.78	0.81	0.79	90
palm	0.71	0.80	0.75	90
pine	0.61	0.61	0.61	90
willow	0.68	0.70	0.69	90
bicycle	0.74	0.54	0.63	90
bus	0.63	0.71	0.67	90
motorcycle	0.46	0.64	0.53	90
pickup truck	0.58	0.53	0.55	90
train	0.76	0.89	0.82	90
lawn-mower	0.69	0.74	0.72	90
rocket	0.47	0.39	0.42	90
streetcar	0.70	0.57	0.63	90
tank	0.42	0.33	0.37	90
tractor	0.74	0.77	0.75	90
accuracy			0.63	9000
macro avg	0.64	0.63	0.63	9000
weighted avg	0.64	0.63	0.63	9000

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)



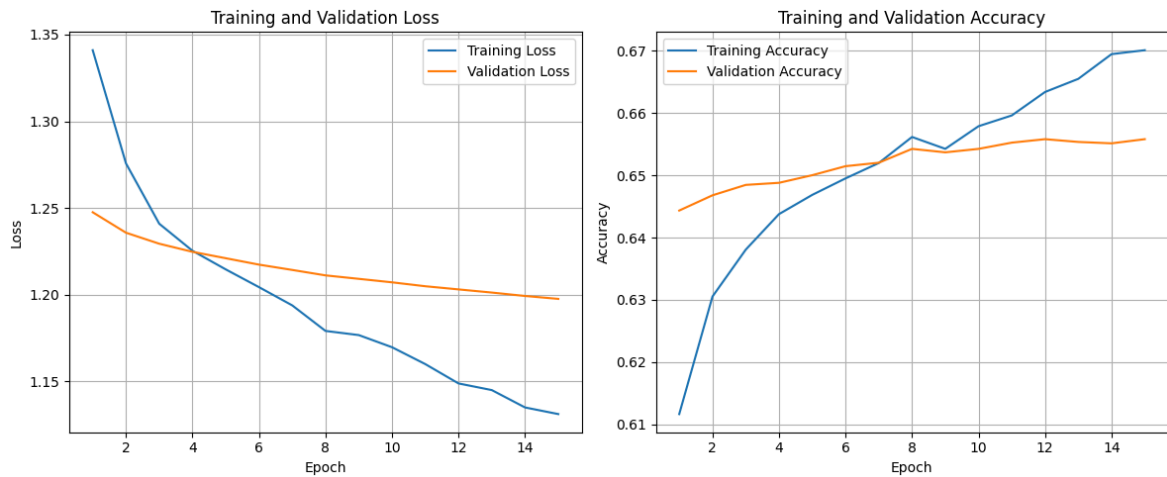
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1314/1314 ————— 0s 26ms/step - accuracy: 0.6025 - f1_score: 0.5997
- loss: 1.3766 - precision: 0.7882
Epoch 1: val_loss improved from inf to 1.24752, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.6025 - f1_score: 0.5997
- loss: 1.3766 - precision: 0.7882 - val_accuracy: 0.6443 - val_f1_score: 0.6422
- val_loss: 1.2475 - val_precision: 0.8249
Epoch 2/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6294 - f1_score: 0.6262
- loss: 1.2783 - precision: 0.8105
Epoch 2: val_loss improved from 1.24752 to 1.23572, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6294 - f1_score: 0.6262
- loss: 1.2783 - precision: 0.8105 - val_accuracy: 0.6468 - val_f1_score: 0.6448
- val_loss: 1.2357 - val_precision: 0.8272
Epoch 3/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.6398 - f1_score: 0.6369
- loss: 1.2374 - precision: 0.8190
Epoch 3: val_loss improved from 1.23572 to 1.22937, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6398 - f1_score: 0.6369
- loss: 1.2374 - precision: 0.8190 - val_accuracy: 0.6484 - val_f1_score: 0.6465
- val_loss: 1.2294 - val_precision: 0.8283
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6431 - f1_score: 0.6402
- loss: 1.2200 - precision: 0.8215
Epoch 4: val_loss improved from 1.22937 to 1.22469, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6431 - f1_score: 0.6402
- loss: 1.2200 - precision: 0.8215 - val_accuracy: 0.6488 - val_f1_score: 0.6468
- val_loss: 1.2247 - val_precision: 0.8298
Epoch 5/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6441 - f1_score: 0.6409
- loss: 1.2124 - precision: 0.8261
Epoch 5: val_loss improved from 1.22469 to 1.22099, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6441 - f1_score: 0.6409
- loss: 1.2124 - precision: 0.8261 - val_accuracy: 0.6500 - val_f1_score: 0.6482
- val_loss: 1.2210 - val_precision: 0.8301
Epoch 6/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6512 - f1_score: 0.6483
- loss: 1.1964 - precision: 0.8251
Epoch 6: val_loss improved from 1.22099 to 1.21730, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6512 - f1_score: 0.6483
- loss: 1.1964 - precision: 0.8251 - val_accuracy: 0.6514 - val_f1_score: 0.6497
- val_loss: 1.2173 - val_precision: 0.8312
Epoch 7/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6517 - f1_score: 0.6491
- loss: 1.1877 - precision: 0.8326
Epoch 7: val_loss improved from 1.21730 to 1.21424, saving model to best_model_fine_tune_v5.keras
1314/1314 ————— 34s 26ms/step - accuracy: 0.6517 - f1_score: 0.6491
- loss: 1.1877 - precision: 0.8326 - val_accuracy: 0.6520 - val_f1_score: 0.6503
- val_loss: 1.2142 - val_precision: 0.8312
Epoch 8/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6581 - f1_score: 0.6554
- loss: 1.1673 - precision: 0.8318
Epoch 8: val_loss improved from 1.21424 to 1.21110, saving model to best_model_fine_tune_v5.keras

```
ne_tune_v5.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6581 - f1_score: 0.655
4 - loss: 1.1673 - precision: 0.8318 - val_accuracy: 0.6542 - val_f1_score: 0.652
6 - val_loss: 1.2111 - val_precision: 0.8309
Epoch 9/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6552 - f1_score: 0.6526
- loss: 1.1648 - precision: 0.8354
Epoch 9: val_loss improved from 1.21110 to 1.20910, saving model to best_model_fi
ne_tune_v5.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6552 - f1_score: 0.652
5 - loss: 1.1648 - precision: 0.8354 - val_accuracy: 0.6537 - val_f1_score: 0.652
0 - val_loss: 1.2091 - val_precision: 0.8315
Epoch 10/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6605 - f1_score: 0.6578
- loss: 1.1615 - precision: 0.8351
Epoch 10: val_loss improved from 1.20910 to 1.20709, saving model to best_model_f
ine_tune_v5.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6605 - f1_score: 0.657
8 - loss: 1.1615 - precision: 0.8351 - val_accuracy: 0.6542 - val_f1_score: 0.652
4 - val_loss: 1.2071 - val_precision: 0.8321
Epoch 11/15
1313/1314 ————— 0s 23ms/step - accuracy: 0.6626 - f1_score: 0.6598
- loss: 1.1476 - precision: 0.8382
Epoch 11: val_loss improved from 1.20709 to 1.20484, saving model to best_model_f
ine_tune_v5.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6626 - f1_score: 0.659
8 - loss: 1.1477 - precision: 0.8382 - val_accuracy: 0.6552 - val_f1_score: 0.653
5 - val_loss: 1.2048 - val_precision: 0.8316
Epoch 12/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6653 - f1_score: 0.6625
- loss: 1.1361 - precision: 0.8406
Epoch 12: val_loss improved from 1.20484 to 1.20301, saving model to best_model_f
ine_tune_v5.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6653 - f1_score: 0.662
5 - loss: 1.1361 - precision: 0.8406 - val_accuracy: 0.6558 - val_f1_score: 0.654
0 - val_loss: 1.2030 - val_precision: 0.8316
Epoch 13/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.6670 - f1_score: 0.6642
- loss: 1.1356 - precision: 0.8413
Epoch 13: val_loss improved from 1.20301 to 1.20119, saving model to best_model_f
ine_tune_v5.keras
1314/1314 ————— 34s 26ms/step - accuracy: 0.6670 - f1_score: 0.664
2 - loss: 1.1356 - precision: 0.8413 - val_accuracy: 0.6553 - val_f1_score: 0.653
5 - val_loss: 1.2012 - val_precision: 0.8321
Epoch 14/15
1313/1314 ————— 0s 23ms/step - accuracy: 0.6724 - f1_score: 0.6695
- loss: 1.1215 - precision: 0.8432
Epoch 14: val_loss improved from 1.20119 to 1.19922, saving model to best_model_f
ine_tune_v5.keras
1314/1314 ————— 34s 26ms/step - accuracy: 0.6724 - f1_score: 0.669
5 - loss: 1.1216 - precision: 0.8431 - val_accuracy: 0.6551 - val_f1_score: 0.653
4 - val_loss: 1.1992 - val_precision: 0.8320
Epoch 15/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6736 - f1_score: 0.6711
- loss: 1.1143 - precision: 0.8459
Epoch 15: val_loss improved from 1.19922 to 1.19754, saving model to best_model_f
ine_tune_v5.keras
1314/1314 ————— 34s 26ms/step - accuracy: 0.6736 - f1_score: 0.671
1 - loss: 1.1143 - precision: 0.8459 - val_accuracy: 0.6558 - val_f1_score: 0.654
2 - val_loss: 1.1975 - val_precision: 0.8323
```



141/141 ————— **11s** 36ms/step - accuracy: 0.6449 - f1_score: 0.6377

- loss: 1.2114 - precision: 0.8257

Test Accuracy: 65.90%

Test Loss: 1.186752438545227

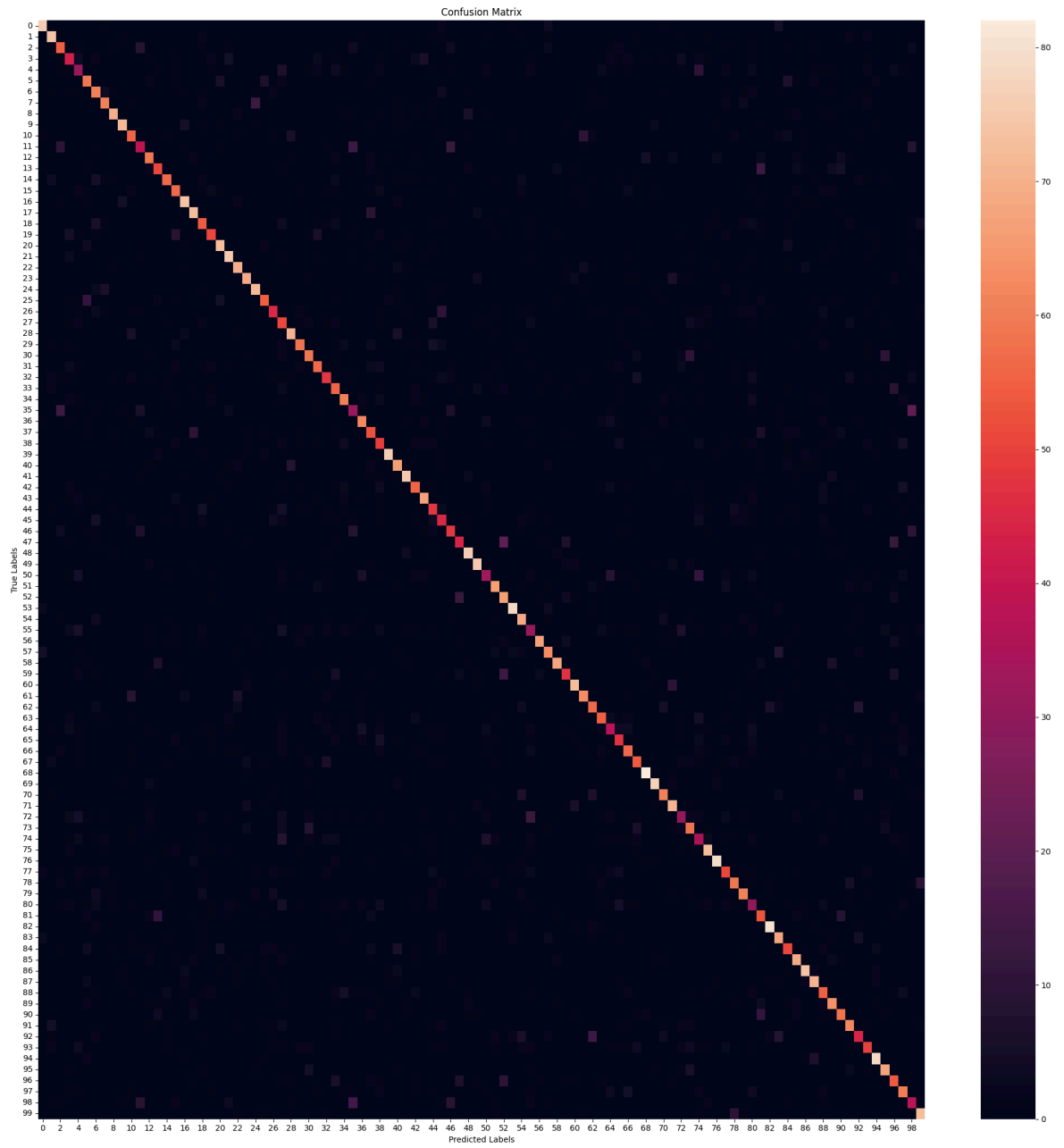
Test Precision: 83.50%

Test F1 Scores (Per Class): [83.9779 77.89473 59.139782 48.863632 35.838146 63.101597 59.803917

69.71428 81.39534 83.236984 62.222218 47.852753 65.19337 59.30232
68.263466 62.983418 77.89473 77.24867 58.064507 62.195118 79.347824
77.7202 76.08696 76.66666 78.68852 65.868256 56.96202 45.739902
78.453026 69.461075 65.19337 66.66666 56.80473 56.852787 68.15642
36.47058 69.66292 56.521736 51.578945 88.372086 68.74999 81.96721
63.636356 76.13636 51.891888 49.450542 53.631275 55.6962 81.28342
81.96721 41.50943 73.33333 64.390236 87.15083 70.05075 36.47058
81.707306 70.39106 80.473366 58.024685 81.11111 71.99999 62.63736
64.70587 46.060596 54.02298 62.63736 61.363636 88.17203 84.61538
68.926544 74.6114 38.461533 61.702126 37.634407 78.918915 87.29281
58.959538 67.42856 69.31818 32.78688 54.639168 87.91208 70.10309
58.285706 72.34043 81.31868 77.59562 58.82352 68.0851 64.80447
67.75956 53.012043 58.479523 84.7826 73.224045 54.82233 64.86485
41.988945 78.07486]

Average Test F1 Scores:65.78

282/282 ————— **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.84	0.84	0.84	90
dolphin	0.74	0.82	0.78	90
otter	0.57	0.61	0.59	90
seal	0.50	0.48	0.49	90
whale	0.37	0.34	0.36	90
aquarium fish	0.61	0.66	0.63	90
flatfish	0.54	0.68	0.60	90
ray	0.72	0.68	0.70	90
shark	0.85	0.78	0.81	90
trout	0.87	0.80	0.83	90
orchids	0.62	0.62	0.62	90
poppies	0.53	0.43	0.48	90
roses	0.65	0.66	0.65	90
sunflowers	0.63	0.57	0.60	90
tulips	0.74	0.63	0.68	90
bottles	0.63	0.63	0.63	90
bowls	0.74	0.82	0.78	90
cans	0.74	0.81	0.77	90
cups	0.56	0.60	0.58	90
plates	0.69	0.57	0.62	90
apples	0.78	0.81	0.79	90
mushrooms	0.73	0.83	0.78	90
oranges	0.74	0.78	0.76	90
pears	0.77	0.77	0.77	90
sweet peppers	0.77	0.80	0.79	90
clock	0.71	0.61	0.66	90
computer keyboard	0.66	0.50	0.57	90
lamp	0.38	0.57	0.46	90
telephone	0.78	0.79	0.78	90
television	0.75	0.64	0.69	90
bed	0.66	0.66	0.66	90
chair	0.70	0.63	0.67	90
couch	0.61	0.53	0.57	90
table	0.52	0.62	0.57	90
wardrobe	0.68	0.68	0.68	90
bee	0.39	0.34	0.36	90
beetle	0.70	0.69	0.70	90
butterfly	0.55	0.58	0.57	90
caterpillar	0.49	0.54	0.52	90
cockroach	0.93	0.84	0.88	90
bear	0.65	0.73	0.69	90
leopard	0.81	0.83	0.82	90
lion	0.65	0.62	0.64	90
tiger	0.78	0.74	0.76	90
wolf	0.51	0.53	0.52	90
bridge	0.49	0.50	0.49	90
castle	0.54	0.53	0.54	90
house	0.65	0.49	0.56	90
road	0.78	0.84	0.81	90
skyscraper	0.81	0.83	0.82	90
cloud	0.48	0.37	0.42	90
forest	0.73	0.73	0.73	90
mountain	0.57	0.73	0.64	90
plain	0.88	0.87	0.87	90
sea	0.64	0.77	0.70	90
camel	0.39	0.34	0.36	90
cattle	0.91	0.74	0.82	90
chimpanzee	0.71	0.70	0.70	90

elephant	0.86	0.76	0.80	90
kangaroo	0.65	0.52	0.58	90
fox	0.80	0.81	0.81	90
porcupine	0.74	0.70	0.72	90
possum	0.62	0.63	0.63	90
raccoon	0.69	0.61	0.65	90
skunk	0.51	0.42	0.46	90
crab	0.56	0.52	0.54	90
lobster	0.62	0.63	0.63	90
snail	0.63	0.60	0.61	90
spider	0.85	0.91	0.88	90
worm	0.84	0.86	0.85	90
baby	0.70	0.68	0.69	90
boy	0.70	0.79	0.74	90
girl	0.45	0.33	0.38	90
man	0.59	0.64	0.62	90
woman	0.37	0.39	0.38	90
crocodile	0.77	0.81	0.79	90
dinosaur	0.87	0.88	0.87	90
lizard	0.61	0.57	0.59	90
snake	0.69	0.66	0.67	90
turtle	0.71	0.68	0.69	90
hamster	0.32	0.33	0.33	90
mouse	0.51	0.59	0.55	90
rabbit	0.87	0.89	0.88	90
shrew	0.65	0.76	0.70	90
squirrel	0.60	0.57	0.58	90
maple	0.70	0.77	0.73	90
oak	0.80	0.82	0.81	90
palm	0.76	0.79	0.78	90
pine	0.57	0.61	0.59	90
willow	0.65	0.71	0.68	90
bicycle	0.65	0.64	0.65	90
bus	0.67	0.69	0.68	90
motorcycle	0.58	0.49	0.53	90
pickup truck	0.62	0.56	0.58	90
train	0.83	0.87	0.85	90
lawn-mower	0.72	0.76	0.74	90
rocket	0.50	0.60	0.55	90
streetcar	0.63	0.67	0.65	90
tank	0.42	0.42	0.42	90
tractor	0.75	0.81	0.78	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

Dropout(0.7), weight_decay(3e-1), l2(3e-1) - no epoch stop

```
In [ ]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

#             .map(augment_dataset, num_parallel_calls=tf.data.experimental.
#             .batch(batch_size)
#             .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.7), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)

#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)

#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                            histogram_freq=1, # Logs histograms for weights/acc
#                            write_graph=True, # Logs graph of model
#                            write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v1.keras', # Path where the best model is saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v1.keras', # Path where the best model is saved
                                       monitor='val_loss', # What to monitor (validation loss)
                                       save_best_only=True, # Only save model when val loss is best
                                       mode='min', # Looking to minimize the validation loss
                                       verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                   batch_size=batch_size, callbacks=[checkpoint], verbose=1)

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']

```

```

val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v1.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

```

```

model.summary()

#### <<<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[checkpoint_f

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v1.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores: {np.average(test_f1_scores.numpy()*100):.2f}")

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))

```

```

sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

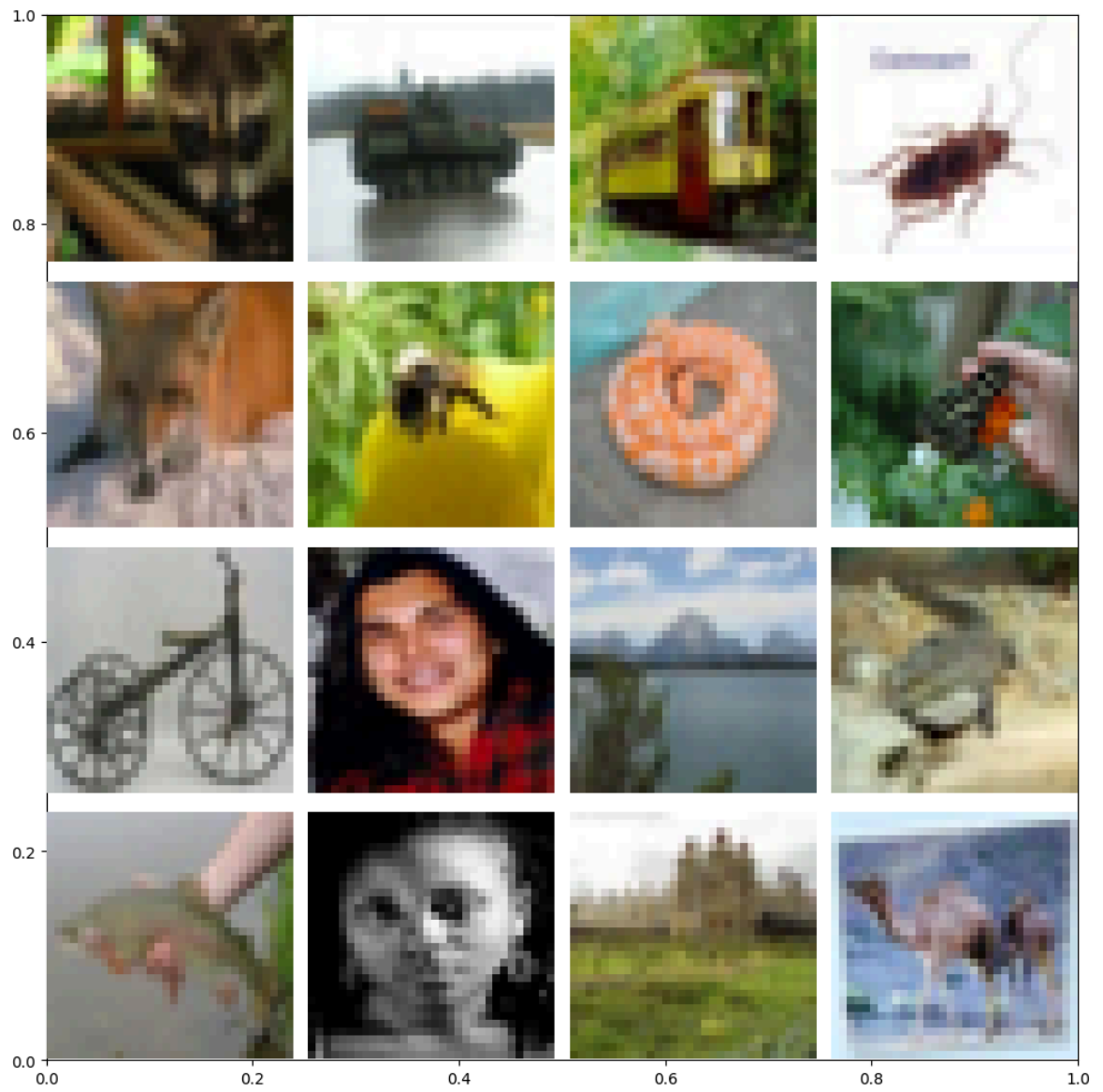
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>
169001437/169001437  2s 0us/step
X_temp.shape: (8000, 32, 32, 3)

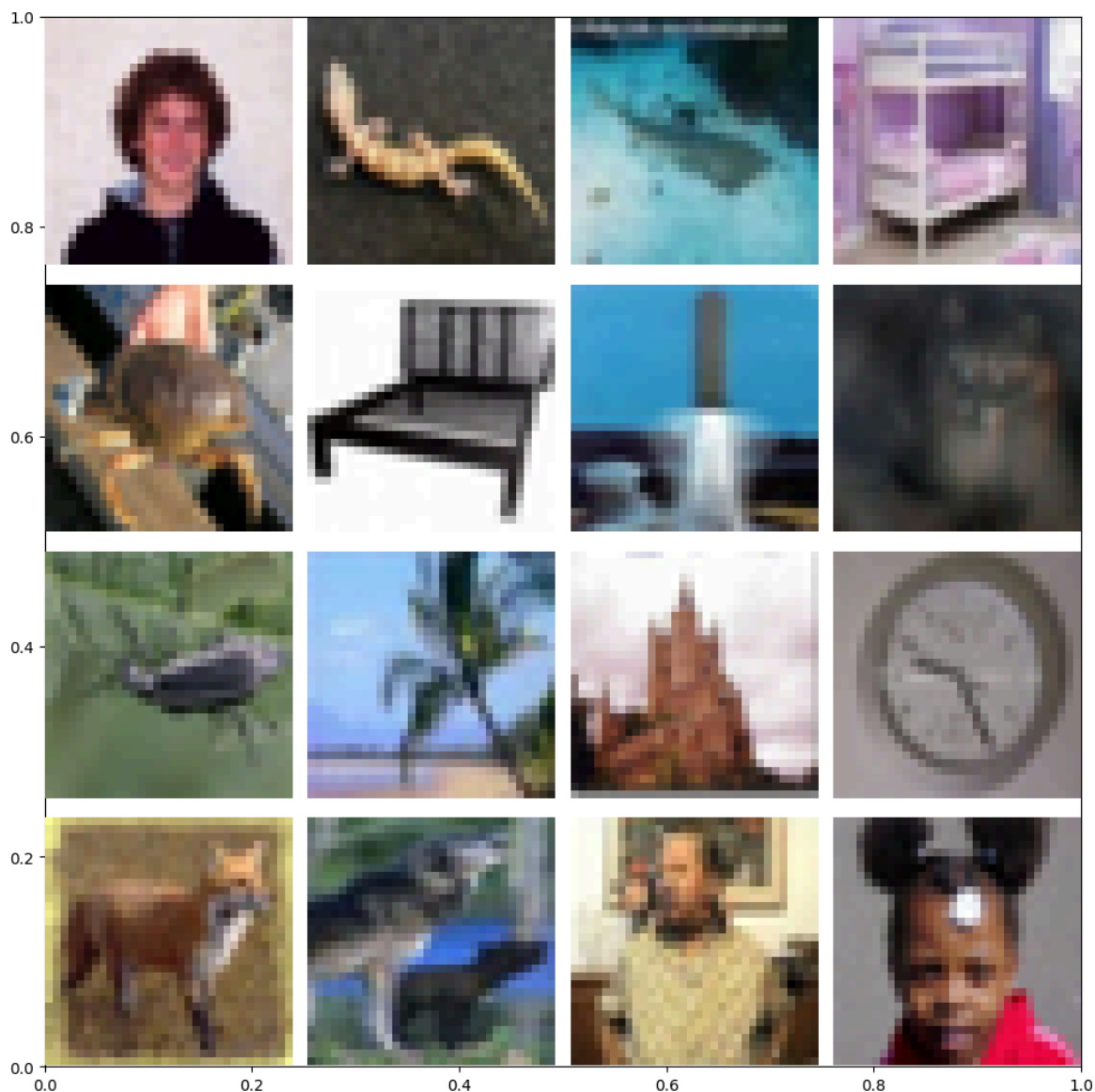
X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)

labels[38883].shape: (1,)
imgs[38883].shape: (32, 32, 3)

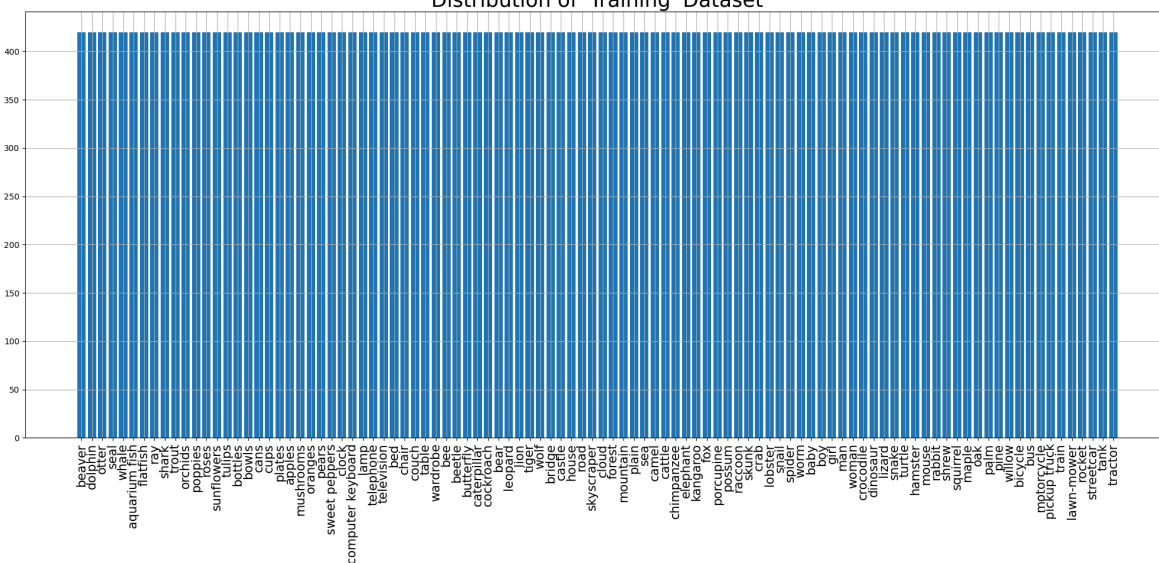


```
labels[32789].shape: (1,)  
imgs[32789].shape: (32, 32, 3)
```

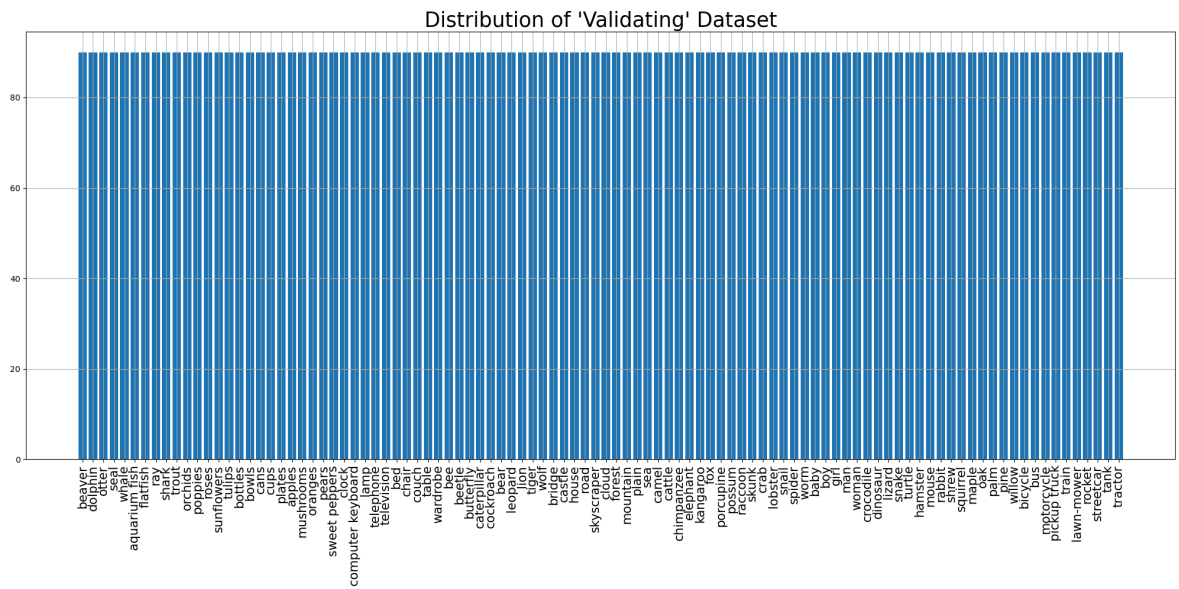



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

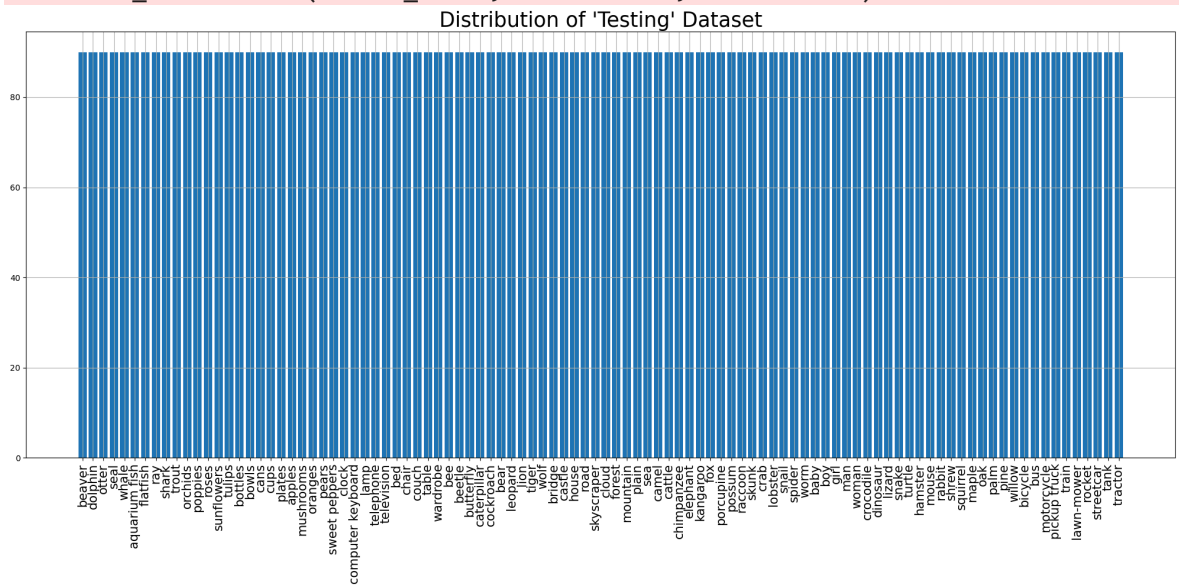
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5
94668760/94668760 ————— 0s 0us/step
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)

◀  ▶

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 33ms/step - accuracy: 0.2606 - f1_score: 0.2534 - loss: 3.2134 - precision: 0.6384
Epoch 1: val_loss improved from inf to 1.57112, saving model to best_model_v1.keras as
657/657 ————— 55s 57ms/step - accuracy: 0.2608 - f1_score: 0.2535 - loss: 3.2124 - precision: 0.6385 - val_accuracy: 0.5742 - val_f1_score: 0.5670 - val_loss: 1.5711 - val_precision: 0.8258
Epoch 2/25
657/657 ————— 0s 23ms/step - accuracy: 0.4759 - f1_score: 0.4707 - loss: 1.9459 - precision: 0.7372
Epoch 2: val_loss improved from 1.57112 to 1.45509, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4759 - f1_score: 0.4707 - loss: 1.9458 - precision: 0.7372 - val_accuracy: 0.5949 - val_f1_score: 0.5904 - val_loss: 1.4551 - val_precision: 0.8257
Epoch 3/25
657/657 ————— 0s 23ms/step - accuracy: 0.5132 - f1_score: 0.5085 - loss: 1.7775 - precision: 0.7505
Epoch 3: val_loss improved from 1.45509 to 1.41289, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5132 - f1_score: 0.5085 - loss: 1.7774 - precision: 0.7505 - val_accuracy: 0.6020 - val_f1_score: 0.5985 - val_loss: 1.4129 - val_precision: 0.8259
Epoch 4/25
656/657 ————— 0s 23ms/step - accuracy: 0.5319 - f1_score: 0.5270 - loss: 1.6929 - precision: 0.7591
Epoch 4: val_loss improved from 1.41289 to 1.39306, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5319 - f1_score: 0.5270 - loss: 1.6928 - precision: 0.7591 - val_accuracy: 0.6111 - val_f1_score: 0.6063 - val_loss: 1.3931 - val_precision: 0.8207
Epoch 5/25
655/657 ————— 0s 23ms/step - accuracy: 0.5373 - f1_score: 0.5328 - loss: 1.6556 - precision: 0.7550
Epoch 5: val_loss improved from 1.39306 to 1.36755, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5374 - f1_score: 0.5329 - loss: 1.6555 - precision: 0.7550 - val_accuracy: 0.6163 - val_f1_score: 0.6125 - val_loss: 1.3675 - val_precision: 0.8282
Epoch 6/25
657/657 ————— 0s 23ms/step - accuracy: 0.5459 - f1_score: 0.5423 - loss: 1.6131 - precision: 0.7644
Epoch 6: val_loss improved from 1.36755 to 1.35125, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5459 - f1_score: 0.5423 - loss: 1.6131 - precision: 0.7644 - val_accuracy: 0.6189 - val_f1_score: 0.6144 - val_loss: 1.3513 - val_precision: 0.8310
Epoch 7/25
656/657 ————— 0s 23ms/step - accuracy: 0.5564 - f1_score: 0.5523 - loss: 1.5858 - precision: 0.7657
Epoch 7: val_loss improved from 1.35125 to 1.34850, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5564 - f1_score: 0.5524 - loss: 1.5858 - precision: 0.7657 - val_accuracy: 0.6203 - val_f1_score: 0.6180 - val_loss: 1.3485 - val_precision: 0.8247
Epoch 8/25
655/657 ————— 0s 23ms/step - accuracy: 0.5566 - f1_score: 0.5526 - loss: 1.5636 - precision: 0.7698
Epoch 8: val_loss improved from 1.34850 to 1.34546, saving model to best_model_v

1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5566 - f1_score: 0.5526
- loss: 1.5635 - precision: 0.7698 - val_accuracy: 0.6210 - val_f1_score: 0.6191
- val_loss: 1.3455 - val_precision: 0.8206
Epoch 9/25
657/657 ————— 0s 23ms/step - accuracy: 0.5606 - f1_score: 0.5571 -
loss: 1.5575 - precision: 0.7660
Epoch 9: val_loss did not improve from 1.34546
657/657 ————— 18s 28ms/step - accuracy: 0.5606 - f1_score: 0.5571
- loss: 1.5574 - precision: 0.7660 - val_accuracy: 0.6176 - val_f1_score: 0.6129
- val_loss: 1.3489 - val_precision: 0.8228
Epoch 10/25
655/657 ————— 0s 23ms/step - accuracy: 0.5636 - f1_score: 0.5596 -
loss: 1.5326 - precision: 0.7724
Epoch 10: val_loss improved from 1.34546 to 1.33232, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5636 - f1_score: 0.5596
- loss: 1.5325 - precision: 0.7724 - val_accuracy: 0.6223 - val_f1_score: 0.6198
- val_loss: 1.3323 - val_precision: 0.8252
Epoch 11/25
657/657 ————— 0s 23ms/step - accuracy: 0.5664 - f1_score: 0.5629 -
loss: 1.5136 - precision: 0.7755
Epoch 11: val_loss improved from 1.33232 to 1.32744, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5664 - f1_score: 0.5629
- loss: 1.5136 - precision: 0.7755 - val_accuracy: 0.6220 - val_f1_score: 0.6193
- val_loss: 1.3274 - val_precision: 0.8161
Epoch 12/25
657/657 ————— 0s 23ms/step - accuracy: 0.5681 - f1_score: 0.5645 -
loss: 1.5077 - precision: 0.7750
Epoch 12: val_loss did not improve from 1.32744
657/657 ————— 18s 28ms/step - accuracy: 0.5681 - f1_score: 0.5646
- loss: 1.5077 - precision: 0.7750 - val_accuracy: 0.6281 - val_f1_score: 0.6262
- val_loss: 1.3353 - val_precision: 0.8304
Epoch 13/25
655/657 ————— 0s 23ms/step - accuracy: 0.5762 - f1_score: 0.5722 -
loss: 1.4930 - precision: 0.7745
Epoch 13: val_loss did not improve from 1.32744
657/657 ————— 18s 28ms/step - accuracy: 0.5762 - f1_score: 0.5723
- loss: 1.4930 - precision: 0.7745 - val_accuracy: 0.6287 - val_f1_score: 0.6269
- val_loss: 1.3280 - val_precision: 0.8247
Epoch 14/25
657/657 ————— 0s 23ms/step - accuracy: 0.5760 - f1_score: 0.5727 -
loss: 1.4803 - precision: 0.7804
Epoch 14: val_loss improved from 1.32744 to 1.30686, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5760 - f1_score: 0.5727
- loss: 1.4803 - precision: 0.7804 - val_accuracy: 0.6330 - val_f1_score: 0.6310
- val_loss: 1.3069 - val_precision: 0.8292
Epoch 15/25
655/657 ————— 0s 23ms/step - accuracy: 0.5748 - f1_score: 0.5713 -
loss: 1.4906 - precision: 0.7738
Epoch 15: val_loss did not improve from 1.30686
657/657 ————— 18s 28ms/step - accuracy: 0.5748 - f1_score: 0.5713
- loss: 1.4904 - precision: 0.7738 - val_accuracy: 0.6278 - val_f1_score: 0.6248
- val_loss: 1.3233 - val_precision: 0.8148
Epoch 16/25
655/657 ————— 0s 23ms/step - accuracy: 0.5790 - f1_score: 0.5752 -
loss: 1.4771 - precision: 0.7798
Epoch 16: val_loss did not improve from 1.30686

657/657 ————— **18s** 28ms/step - accuracy: 0.5790 - f1_score: 0.5752
- loss: 1.4769 - precision: 0.7798 - val_accuracy: 0.6269 - val_f1_score: 0.6214
- val_loss: 1.3199 - val_precision: 0.8195
Epoch 17/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5819 - f1_score: 0.5774 -
loss: 1.4650 - precision: 0.7771
Epoch 17: val_loss did not improve from 1.30686

657/657 ————— **18s** 28ms/step - accuracy: 0.5819 - f1_score: 0.5774
- loss: 1.4649 - precision: 0.7771 - val_accuracy: 0.6253 - val_f1_score: 0.6234
- val_loss: 1.3140 - val_precision: 0.8290
Epoch 18/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5782 - f1_score: 0.5752 -
loss: 1.4650 - precision: 0.7815
Epoch 18: val_loss did not improve from 1.30686

657/657 ————— **18s** 28ms/step - accuracy: 0.5782 - f1_score: 0.5752
- loss: 1.4649 - precision: 0.7815 - val_accuracy: 0.6263 - val_f1_score: 0.6232
- val_loss: 1.3171 - val_precision: 0.8156
Epoch 19/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5796 - f1_score: 0.5756 -
loss: 1.4681 - precision: 0.7770
Epoch 19: val_loss did not improve from 1.30686

657/657 ————— **18s** 28ms/step - accuracy: 0.5796 - f1_score: 0.5756
- loss: 1.4680 - precision: 0.7770 - val_accuracy: 0.6251 - val_f1_score: 0.6218
- val_loss: 1.3172 - val_precision: 0.8247
Epoch 20/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5777 - f1_score: 0.5739 -
loss: 1.4688 - precision: 0.7755
Epoch 20: val_loss did not improve from 1.30686

657/657 ————— **18s** 28ms/step - accuracy: 0.5777 - f1_score: 0.5739
- loss: 1.4688 - precision: 0.7755 - val_accuracy: 0.6219 - val_f1_score: 0.6200
- val_loss: 1.3128 - val_precision: 0.8197
Epoch 21/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5828 - f1_score: 0.5792 -
loss: 1.4562 - precision: 0.7747
Epoch 21: val_loss did not improve from 1.30686

657/657 ————— **18s** 28ms/step - accuracy: 0.5828 - f1_score: 0.5792
- loss: 1.4562 - precision: 0.7747 - val_accuracy: 0.6301 - val_f1_score: 0.6260
- val_loss: 1.3075 - val_precision: 0.8207
Epoch 22/25

656/657 ————— **0s** 23ms/step - accuracy: 0.5793 - f1_score: 0.5755 -
loss: 1.4509 - precision: 0.7787
Epoch 22: val_loss did not improve from 1.30686

657/657 ————— **18s** 28ms/step - accuracy: 0.5793 - f1_score: 0.5755
- loss: 1.4508 - precision: 0.7787 - val_accuracy: 0.6260 - val_f1_score: 0.6246
- val_loss: 1.3148 - val_precision: 0.8196
Epoch 23/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5808 - f1_score: 0.5771 -
loss: 1.4560 - precision: 0.7766
Epoch 23: val_loss did not improve from 1.30686

657/657 ————— **19s** 28ms/step - accuracy: 0.5808 - f1_score: 0.5771
- loss: 1.4559 - precision: 0.7766 - val_accuracy: 0.6286 - val_f1_score: 0.6247
- val_loss: 1.3091 - val_precision: 0.8206
Epoch 24/25

656/657 ————— **0s** 23ms/step - accuracy: 0.5842 - f1_score: 0.5803 -
loss: 1.4438 - precision: 0.7825
Epoch 24: val_loss improved from 1.30686 to 1.29402, saving model to best_model_v1.keras

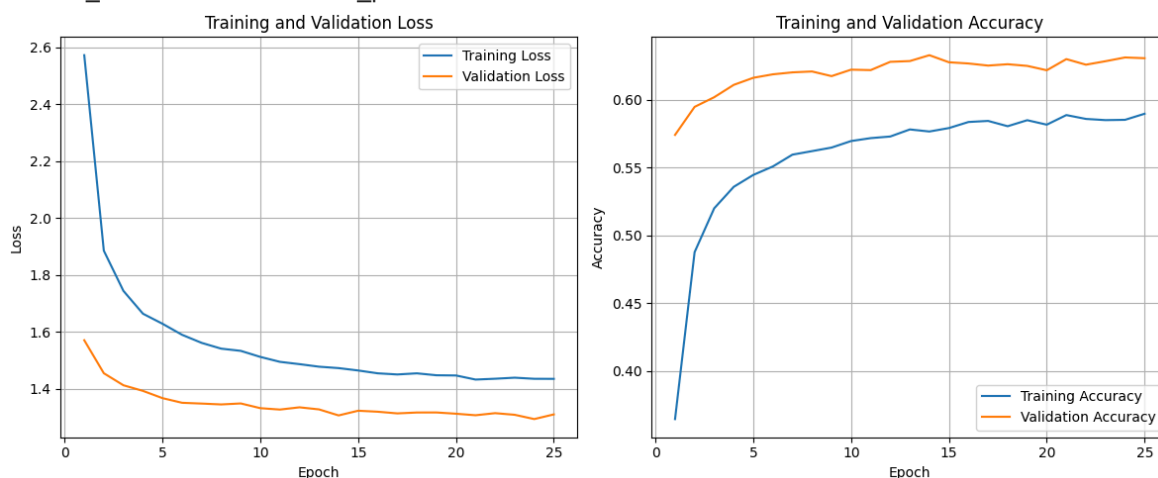
657/657 ————— **20s** 30ms/step - accuracy: 0.5842 - f1_score: 0.5803
- loss: 1.4438 - precision: 0.7825 - val_accuracy: 0.6313 - val_f1_score: 0.6293
- val_loss: 1.2940 - val_precision: 0.8200

Epoch 25/25

656/657 ————— 0s 23ms/step - accuracy: 0.5827 - f1_score: 0.5793 -
loss: 1.4621 - precision: 0.7774

Epoch 25: val_loss did not improve from 1.29402

657/657 ————— 18s 28ms/step - accuracy: 0.5828 - f1_score: 0.5793
- loss: 1.4620 - precision: 0.7774 - val_accuracy: 0.6308 - val_f1_score: 0.6286
- val_loss: 1.3104 - val_precision: 0.8248



141/141 ————— 11s 40ms/step - accuracy: 0.6087 - f1_score: 0.6023
- loss: 1.3120 - precision: 0.8133

Test Accuracy: 62.49%

Test Loss: 1.2902852296829224

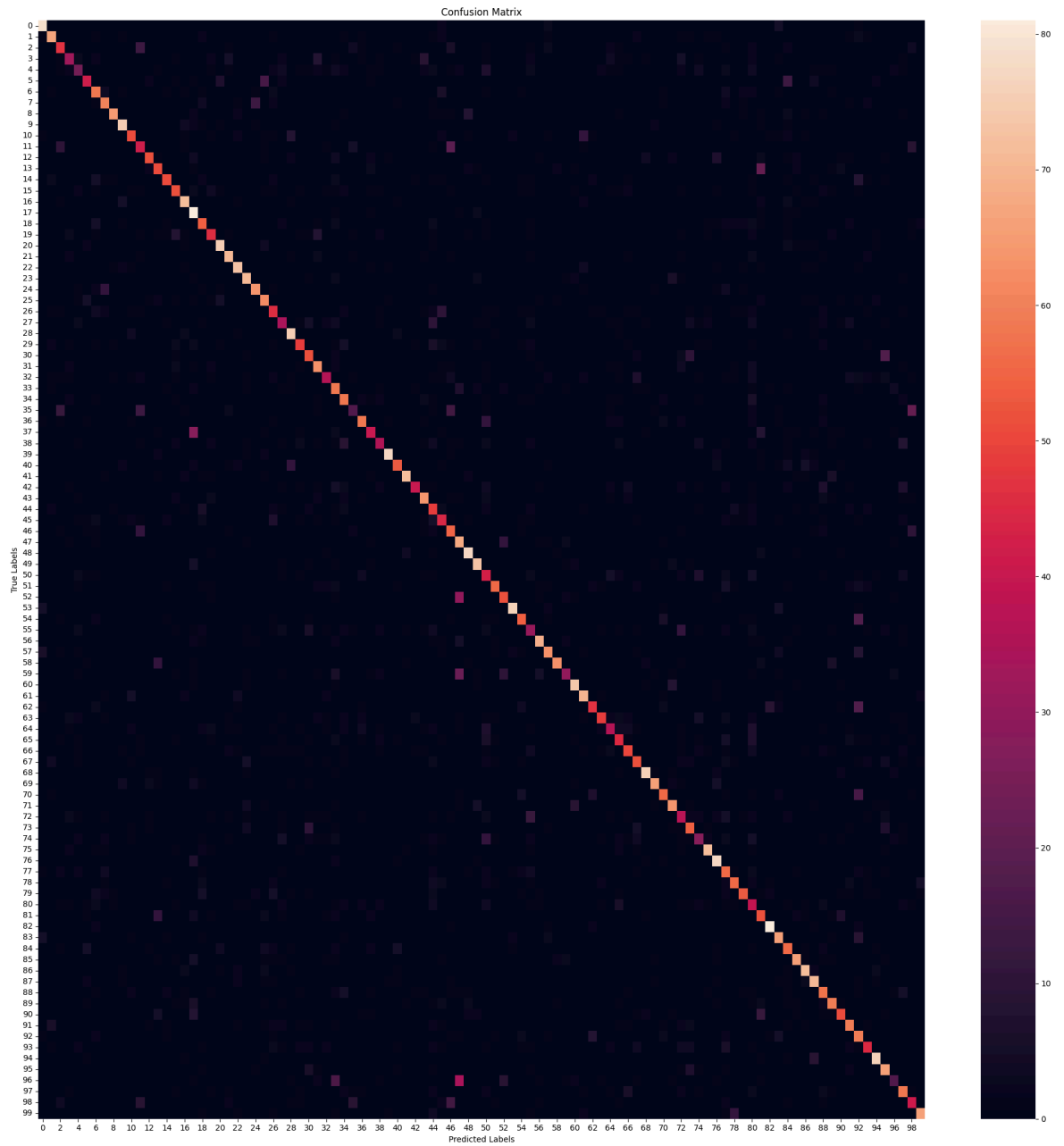
Test Precision: 82.16%

Test F1 Scores (Per Class): [82.7225 74.444435 53.107338 44.295296 32.432426 5
4.193542 59.18367

65.93406	80.24691	81.08108	61.818176	44.680847	67.09677	56.66666
65.384605	59.770107	77.00535	60.22304	57.14285	55.75757	80.64516
77.59562	78.49462	75.93583	73.988434	63.2653	50.27322	41.46341
76.14213	61.25	55.91397	65.62499	49.315063	48.33333	59.18367
25.757568	67.83625	52.980125	46.979862	89.017334	65.8385	84.70587
57.55395	70.71823	43.1718	48.087425	50.458706	50.936325	81.05263
79.347824	38.91402	70.88607	59.770107	87.356316	63.529408	35.02824
78.40909	71.99999	78.75	44.776115	80.43478	74.46808	58.749992
64.052284	45.85987	51.42857	58.823524	58.426964	83.9779	79.51807
65.88235	71.1111	39.999996	59.340656	36.129025	81.81818	76.23762
51.6129	65.11627	67.948715	33.33333	49.523804	87.09677	69.84126
55.721386	75.86206	77.83783	73.95833	58.76288	67.415726	62.576683
65.55555	46.82539	56.603767	83.9779	65.346535	27.199995	59.793808
44.565212	79.04191					

Average Test F1 Scores:62.46

282/282 ————— 18s 38ms/step



	precision	recall	f1-score	support
beaver	0.78	0.88	0.83	90
dolphin	0.74	0.74	0.74	90
otter	0.54	0.52	0.53	90
seal	0.56	0.37	0.44	90
whale	0.41	0.27	0.32	90
aquarium fish	0.65	0.47	0.54	90
flatfish	0.55	0.64	0.59	90
ray	0.65	0.67	0.66	90
shark	0.90	0.72	0.80	90
trout	0.79	0.83	0.81	90
orchids	0.68	0.57	0.62	90
poppies	0.43	0.47	0.45	90
roses	0.80	0.58	0.67	90
sunflowers	0.57	0.57	0.57	90
tulips	0.77	0.57	0.65	90
bottles	0.62	0.58	0.60	90
bowls	0.74	0.80	0.77	90
cans	0.45	0.90	0.60	90
cups	0.55	0.60	0.57	90
plates	0.61	0.51	0.55	90
apples	0.78	0.83	0.81	90
mushrooms	0.76	0.79	0.78	90
oranges	0.76	0.81	0.78	90
pears	0.73	0.79	0.76	90
sweet peppers	0.77	0.71	0.74	90
clock	0.58	0.69	0.63	90
computer keyboard	0.49	0.51	0.50	90
lamp	0.47	0.38	0.42	90
telephone	0.70	0.83	0.76	90
television	0.70	0.54	0.61	90
bed	0.54	0.58	0.56	90
chair	0.62	0.70	0.66	90
couch	0.64	0.40	0.49	90
table	0.39	0.64	0.48	90
wardrobe	0.55	0.64	0.59	90
bee	0.40	0.19	0.26	90
beetle	0.72	0.64	0.68	90
butterfly	0.66	0.44	0.53	90
caterpillar	0.60	0.39	0.47	90
cockroach	0.93	0.86	0.89	90
bear	0.75	0.59	0.66	90
leopard	0.90	0.80	0.85	90
lion	0.82	0.44	0.58	90
tiger	0.70	0.71	0.71	90
wolf	0.36	0.54	0.43	90
bridge	0.47	0.49	0.48	90
castle	0.43	0.61	0.50	90
house	0.38	0.76	0.51	90
road	0.77	0.86	0.81	90
skyscraper	0.78	0.81	0.79	90
cloud	0.33	0.48	0.39	90
forest	0.82	0.62	0.71	90
mountain	0.62	0.58	0.60	90
plain	0.90	0.84	0.87	90
sea	0.68	0.60	0.64	90
camel	0.36	0.34	0.35	90
cattle	0.80	0.77	0.78	90
chimpanzee	0.74	0.70	0.72	90

elephant	0.90	0.70	0.79	90
kangaroo	0.68	0.33	0.45	90
fox	0.79	0.82	0.80	90
porcupine	0.71	0.78	0.74	90
possum	0.67	0.52	0.59	90
raccoon	0.78	0.54	0.64	90
skunk	0.54	0.40	0.46	90
crab	0.53	0.50	0.51	90
lobster	0.62	0.56	0.59	90
snail	0.59	0.58	0.58	90
spider	0.84	0.84	0.84	90
worm	0.87	0.73	0.80	90
baby	0.70	0.62	0.66	90
boy	0.71	0.71	0.71	90
girl	0.39	0.41	0.40	90
man	0.59	0.60	0.59	90
woman	0.43	0.31	0.36	90
crocodile	0.85	0.80	0.82	90
dinosaur	0.69	0.86	0.76	90
lizard	0.44	0.62	0.52	90
snake	0.68	0.62	0.65	90
turtle	0.79	0.59	0.68	90
hamster	0.27	0.43	0.33	90
mouse	0.43	0.58	0.50	90
rabbit	0.84	0.90	0.87	90
shrew	0.67	0.73	0.70	90
squirrel	0.50	0.62	0.56	90
maple	0.79	0.73	0.76	90
oak	0.76	0.80	0.78	90
palm	0.70	0.79	0.74	90
pine	0.55	0.63	0.59	90
willow	0.68	0.66	0.67	90
bicycle	0.69	0.57	0.62	90
bus	0.66	0.66	0.66	90
motorcycle	0.36	0.66	0.47	90
pickup truck	0.65	0.50	0.57	90
train	0.84	0.84	0.84	90
lawn-mower	0.59	0.73	0.65	90
rocket	0.49	0.19	0.27	90
streetcar	0.56	0.64	0.60	90
tank	0.44	0.46	0.45	90
tractor	0.86	0.73	0.79	90
accuracy			0.62	9000
macro avg	0.64	0.62	0.62	9000
weighted avg	0.64	0.62	0.62	9000

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 26ms/step - accuracy: 0.6025 - f1_score: 0.6000
- loss: 1.3751 - precision: 0.7880
Epoch 1: val_loss improved from inf to 1.24262, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.6025 - f1_score: 0.6000
0 - loss: 1.3750 - precision: 0.7880 - val_accuracy: 0.6449 - val_f1_score: 0.6429
9 - val_loss: 1.2426 - val_precision: 0.8299
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6328 - f1_score: 0.6297
- loss: 1.2672 - precision: 0.8109
Epoch 2: val_loss improved from 1.24262 to 1.23139, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6328 - f1_score: 0.6297
7 - loss: 1.2672 - precision: 0.8109 - val_accuracy: 0.6468 - val_f1_score: 0.6449
9 - val_loss: 1.2314 - val_precision: 0.8271
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6430 - f1_score: 0.6401
- loss: 1.2324 - precision: 0.8175
Epoch 3: val_loss improved from 1.23139 to 1.22625, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6430 - f1_score: 0.6401
1 - loss: 1.2324 - precision: 0.8175 - val_accuracy: 0.6478 - val_f1_score: 0.6459
9 - val_loss: 1.2263 - val_precision: 0.8257
Epoch 4/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6469 - f1_score: 0.6441
- loss: 1.2071 - precision: 0.8237
Epoch 4: val_loss improved from 1.22625 to 1.22112, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6469 - f1_score: 0.6441
1 - loss: 1.2071 - precision: 0.8237 - val_accuracy: 0.6486 - val_f1_score: 0.6466
6 - val_loss: 1.2211 - val_precision: 0.8246
Epoch 5/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.6483 - f1_score: 0.6457
- loss: 1.1945 - precision: 0.8242
Epoch 5: val_loss improved from 1.22112 to 1.21782, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6483 - f1_score: 0.6457
7 - loss: 1.1945 - precision: 0.8242 - val_accuracy: 0.6483 - val_f1_score: 0.6462
2 - val_loss: 1.2178 - val_precision: 0.8257
Epoch 6/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6552 - f1_score: 0.6520
- loss: 1.1844 - precision: 0.8268
Epoch 6: val_loss improved from 1.21782 to 1.21428, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6552 - f1_score: 0.6520
0 - loss: 1.1844 - precision: 0.8268 - val_accuracy: 0.6503 - val_f1_score: 0.6486
6 - val_loss: 1.2143 - val_precision: 0.8259
Epoch 7/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.6567 - f1_score: 0.6539
- loss: 1.1667 - precision: 0.8271
Epoch 7: val_loss improved from 1.21428 to 1.21090, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.6567 - f1_score: 0.6539
9 - loss: 1.1667 - precision: 0.8271 - val_accuracy: 0.6511 - val_f1_score: 0.6494
4 - val_loss: 1.2109 - val_precision: 0.8249
Epoch 8/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.6604 - f1_score: 0.6576
- loss: 1.1571 - precision: 0.8283
Epoch 8: val_loss improved from 1.21090 to 1.20792, saving model to best_model_fine_tune_v1.keras

ne_tune_v1.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.6604 - f1_score: 0.6576 - loss: 1.1571 - precision: 0.8283 - val_accuracy: 0.6519 - val_f1_score: 0.6502 - val_loss: 1.2079 - val_precision: 0.8266

Epoch 9/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.6641 - f1_score: 0.6614 - loss: 1.1482 - precision: 0.8333

Epoch 9: val_loss improved from 1.20792 to 1.20448, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.6641 - f1_score: 0.6614 - loss: 1.1482 - precision: 0.8333 - val_accuracy: 0.6538 - val_f1_score: 0.6520 - val_loss: 1.2045 - val_precision: 0.8269

Epoch 10/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.6634 - f1_score: 0.6607 - loss: 1.1441 - precision: 0.8327

Epoch 10: val_loss improved from 1.20448 to 1.20251, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.6634 - f1_score: 0.6607 - loss: 1.1441 - precision: 0.8327 - val_accuracy: 0.6538 - val_f1_score: 0.6519 - val_loss: 1.2025 - val_precision: 0.8259

Epoch 11/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.6676 - f1_score: 0.6650 - loss: 1.1272 - precision: 0.8347

Epoch 11: val_loss improved from 1.20251 to 1.20013, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.6676 - f1_score: 0.6650 - loss: 1.1272 - precision: 0.8347 - val_accuracy: 0.6543 - val_f1_score: 0.6526 - val_loss: 1.2001 - val_precision: 0.8259

Epoch 12/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.6712 - f1_score: 0.6685 - loss: 1.1214 - precision: 0.8364

Epoch 12: val_loss improved from 1.20013 to 1.19825, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.6712 - f1_score: 0.6685 - loss: 1.1215 - precision: 0.8364 - val_accuracy: 0.6549 - val_f1_score: 0.6532 - val_loss: 1.1982 - val_precision: 0.8250

Epoch 13/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.6739 - f1_score: 0.6714 - loss: 1.1101 - precision: 0.8377

Epoch 13: val_loss improved from 1.19825 to 1.19634, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.6739 - f1_score: 0.6714 - loss: 1.1101 - precision: 0.8377 - val_accuracy: 0.6560 - val_f1_score: 0.6543 - val_loss: 1.1963 - val_precision: 0.8245

Epoch 14/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.6741 - f1_score: 0.6718 - loss: 1.1042 - precision: 0.8374

Epoch 14: val_loss improved from 1.19634 to 1.19380, saving model to best_model_fine_tune_v1.keras

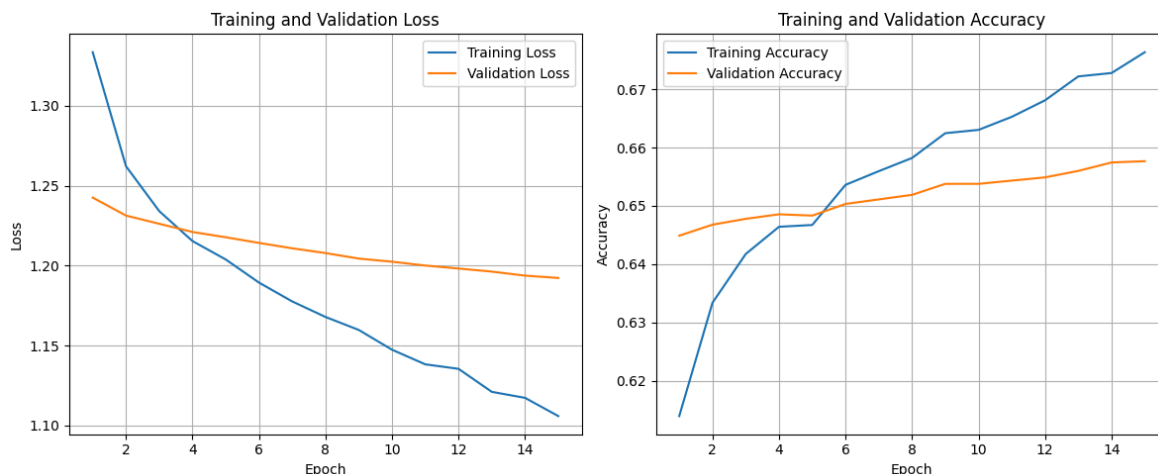
1314/1314 ————— 35s 26ms/step - accuracy: 0.6741 - f1_score: 0.6718 - loss: 1.1043 - precision: 0.8374 - val_accuracy: 0.6574 - val_f1_score: 0.6558 - val_loss: 1.1938 - val_precision: 0.8266

Epoch 15/15

1314/1314 ————— 0s 23ms/step - accuracy: 0.6795 - f1_score: 0.6768 - loss: 1.0937 - precision: 0.8404

Epoch 15: val_loss improved from 1.19380 to 1.19236, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.6795 - f1_score: 0.6768 - loss: 1.0937 - precision: 0.8404 - val_accuracy: 0.6577 - val_f1_score: 0.6560 - val_loss: 1.1924 - val_precision: 0.8270



141/141 ————— **11s** 40ms/step - accuracy: 0.6447 - f1_score: 0.6372

- loss: 1.2095 - precision: 0.8175

Test Accuracy: 65.59%

Test Loss: 1.1851799488067627

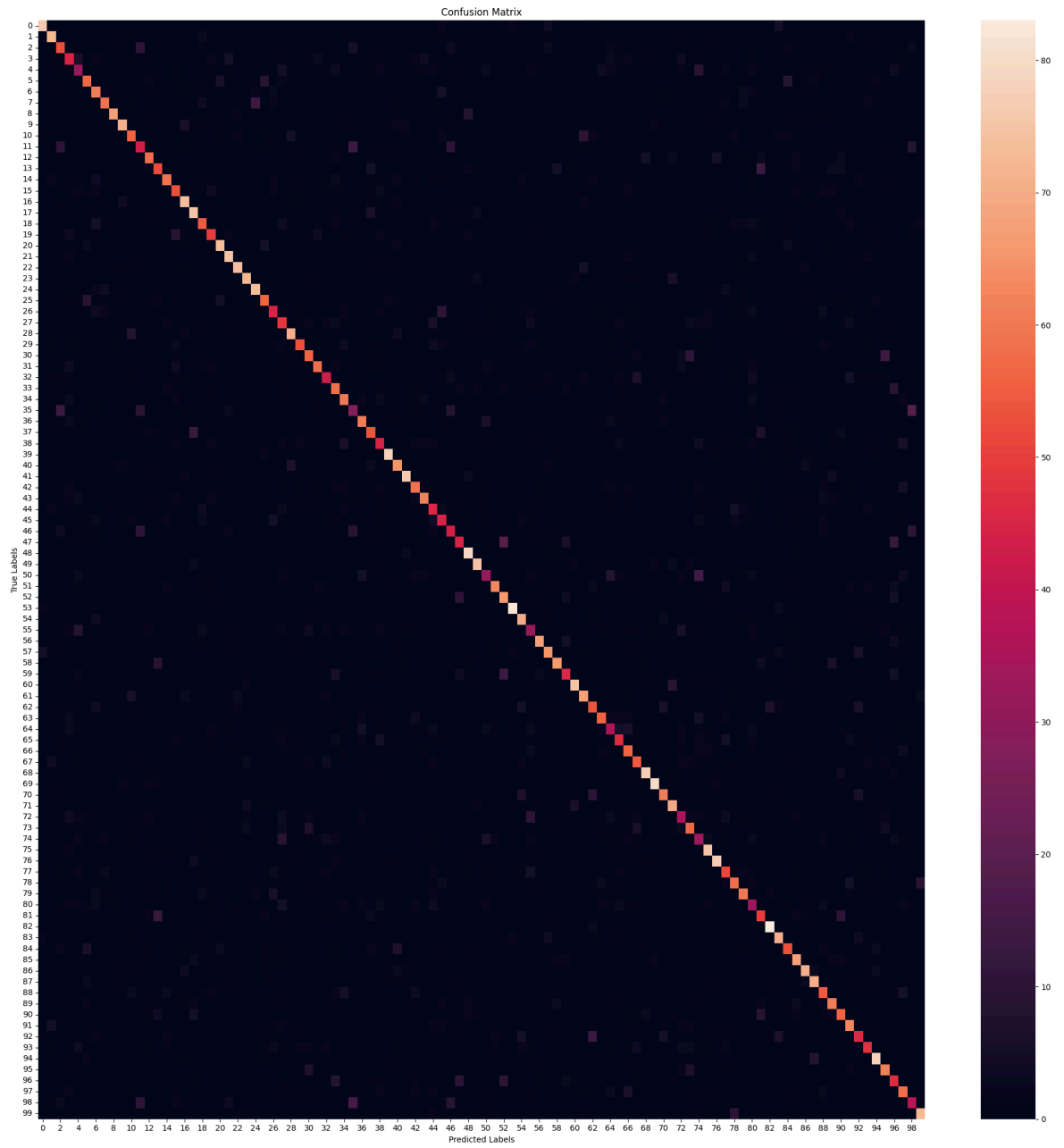
Test Precision: 82.54%

Test F1 Scores (Per Class): [84.91619 78.02197 57.297295 49.999992 37.209297 62.63736 61.616154

69.41176 81.437126 79.775276 63.636356 50.28571 65.16853 58.757053
67.816086 57.92349 75.89743 74.25742 59.7826 58.823524 80.21977
78.53403 81.31868 79.55801 79.347824 62.92134 50.56179 46.666664
80.681816 66.25 62.569828 69.461075 52.439022 56.038643 64.51612
32.94117 68.926544 59.999996 51.42857 89.65517 67.70833 84.44444
63.783775 73.37277 48.167538 50.847454 49.42528 54.54545 81.86528
83.33333 40.506325 72.41379 66. 90.60773 71.06599 37.869816
80.473366 72.625694 78.1065 56.097557 83.798874 74.31693 58.378376
66.27218 43.636356 54.97075 59.893044 61.111103 85.55555 85.10638
69.31818 74.074066 40.935665 62.63736 36.17021 79.787224 84.61538
58.757053 64.80447 68.965515 35.483864 53.475933 89.72973 74.73684
62.352936 73.513504 79.775276 78.02197 60.773476 66.304344 61.538452
66.31578 55.089813 59.25926 84.32432 67.75956 49.47368 62.702698
42.696625 78.68852]

Average Test F1 Scores:65.47

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.85	0.84	0.85	90
dolphin	0.77	0.80	0.79	90
otter	0.56	0.59	0.57	90
seal	0.50	0.50	0.50	90
whale	0.39	0.36	0.37	90
aquarium fish	0.62	0.63	0.63	90
flatfish	0.56	0.68	0.62	90
ray	0.74	0.66	0.69	90
shark	0.88	0.76	0.81	90
trout	0.81	0.79	0.80	90
orchids	0.65	0.62	0.64	90
poppies	0.52	0.49	0.51	90
roses	0.66	0.64	0.65	90
sunflowers	0.60	0.58	0.59	90
tulips	0.70	0.66	0.68	90
bottles	0.57	0.59	0.58	90
bowls	0.70	0.82	0.76	90
cans	0.67	0.83	0.74	90
cups	0.59	0.61	0.60	90
plates	0.62	0.56	0.59	90
apples	0.79	0.81	0.80	90
mushrooms	0.74	0.83	0.79	90
oranges	0.80	0.82	0.81	90
pears	0.79	0.80	0.80	90
sweet peppers	0.78	0.81	0.79	90
clock	0.64	0.62	0.63	90
computer keyboard	0.51	0.50	0.51	90
lamp	0.41	0.54	0.47	90
telephone	0.83	0.79	0.81	90
television	0.76	0.59	0.66	90
bed	0.63	0.62	0.63	90
chair	0.75	0.64	0.69	90
couch	0.58	0.48	0.52	90
table	0.50	0.64	0.56	90
wardrobe	0.62	0.67	0.65	90
bee	0.35	0.31	0.33	90
beetle	0.71	0.68	0.69	90
butterfly	0.60	0.60	0.60	90
caterpillar	0.53	0.50	0.51	90
cockroach	0.93	0.87	0.90	90
bear	0.64	0.72	0.68	90
leopard	0.84	0.84	0.84	90
lion	0.62	0.66	0.64	90
tiger	0.78	0.69	0.73	90
wolf	0.46	0.51	0.48	90
bridge	0.52	0.50	0.51	90
castle	0.52	0.49	0.50	90
house	0.60	0.50	0.55	90
road	0.77	0.88	0.82	90
skyscraper	0.83	0.83	0.83	90
cloud	0.47	0.36	0.41	90
forest	0.75	0.70	0.72	90
mountain	0.60	0.73	0.66	90
plain	0.90	0.91	0.91	90
sea	0.65	0.78	0.71	90
camel	0.40	0.36	0.38	90
cattle	0.86	0.76	0.80	90
chimpanzee	0.73	0.72	0.73	90

elephant	0.84	0.73	0.78	90
kangaroo	0.62	0.51	0.56	90
fox	0.84	0.83	0.84	90
porcupine	0.73	0.76	0.74	90
possum	0.57	0.60	0.58	90
raccoon	0.71	0.62	0.66	90
skunk	0.48	0.40	0.44	90
crab	0.58	0.52	0.55	90
lobster	0.58	0.62	0.60	90
snail	0.61	0.61	0.61	90
spider	0.86	0.86	0.86	90
worm	0.82	0.89	0.85	90
baby	0.71	0.68	0.69	90
boy	0.71	0.78	0.74	90
girl	0.43	0.39	0.41	90
man	0.62	0.63	0.63	90
woman	0.35	0.38	0.36	90
crocodile	0.77	0.83	0.80	90
dinosaur	0.84	0.86	0.85	90
lizard	0.59	0.57	0.58	90
snake	0.65	0.64	0.65	90
turtle	0.71	0.67	0.69	90
hamster	0.34	0.37	0.35	90
mouse	0.52	0.56	0.53	90
rabbit	0.87	0.92	0.90	90
shrew	0.71	0.79	0.75	90
squirrel	0.66	0.59	0.62	90
maple	0.72	0.76	0.74	90
oak	0.81	0.79	0.80	90
palm	0.77	0.79	0.78	90
pine	0.60	0.61	0.61	90
willow	0.65	0.68	0.66	90
bicycle	0.61	0.62	0.62	90
bus	0.64	0.70	0.67	90
motorcycle	0.60	0.51	0.55	90
pickup truck	0.67	0.53	0.59	90
train	0.82	0.87	0.84	90
lawn-mower	0.67	0.69	0.68	90
rocket	0.47	0.52	0.49	90
streetcar	0.61	0.64	0.63	90
tank	0.43	0.42	0.43	90
tractor	0.77	0.80	0.79	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.65	9000
weighted avg	0.66	0.66	0.65	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

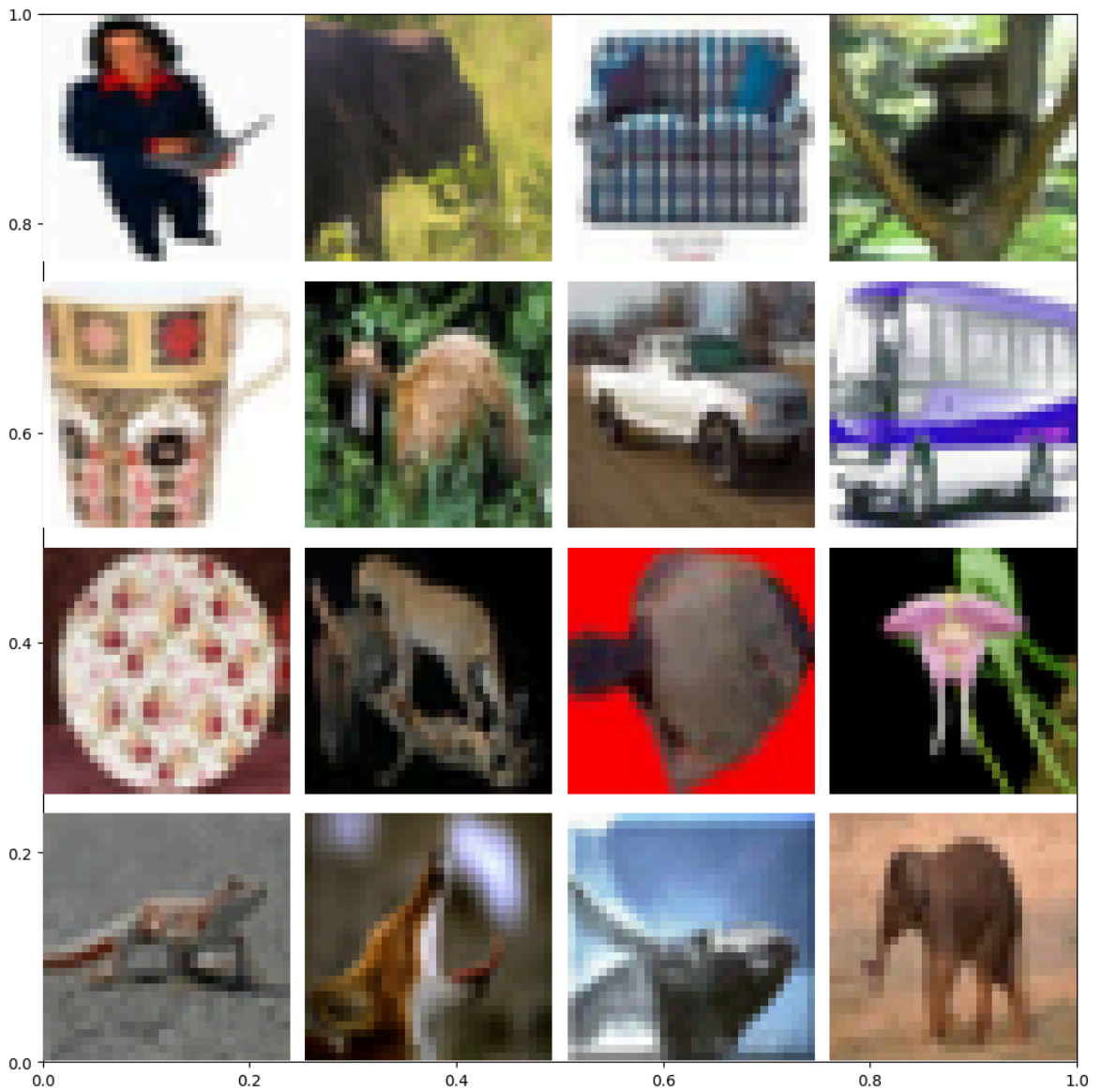
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

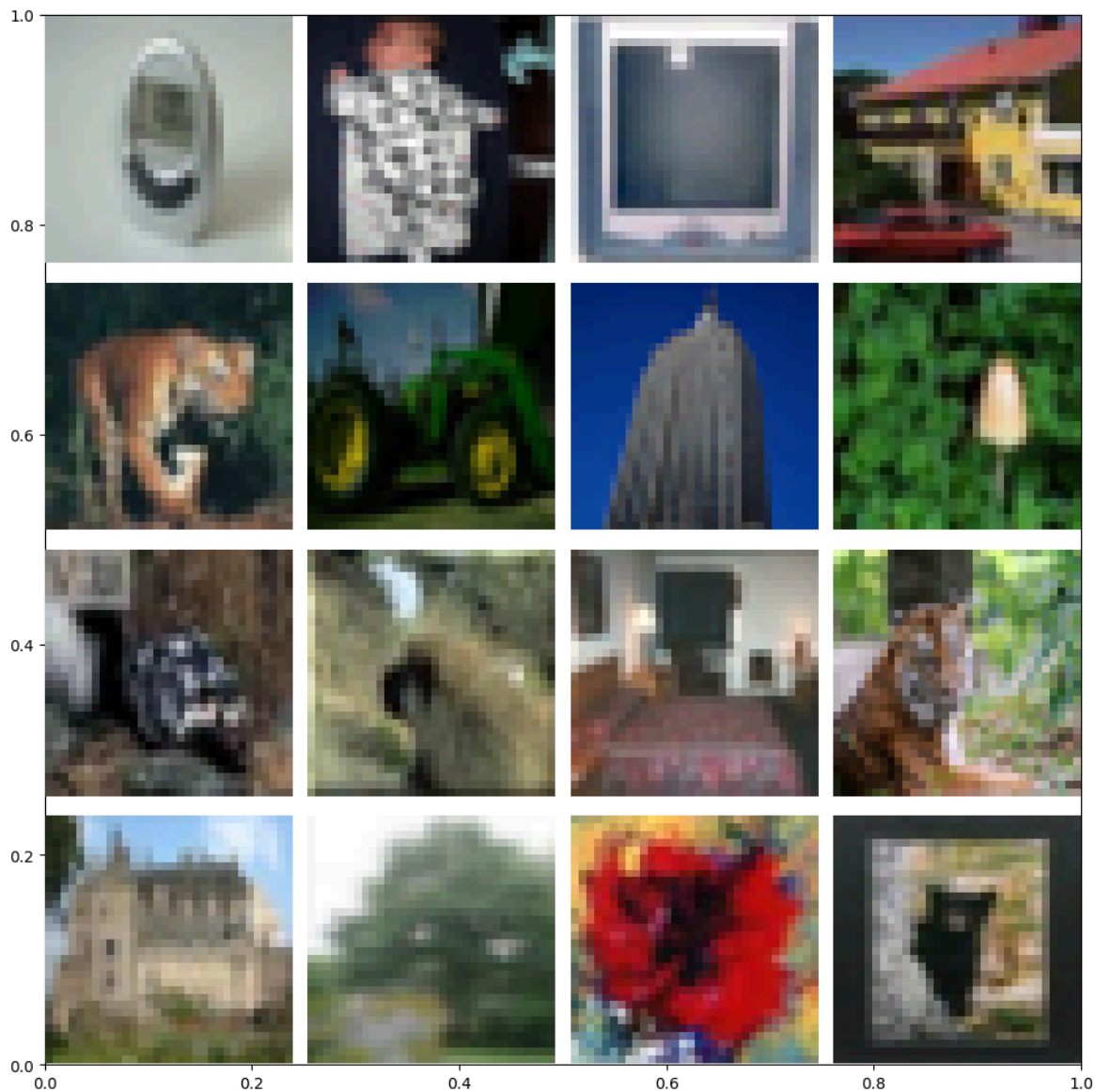
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[8132].shape: (1,)  
imgs[8132].shape: (32, 32, 3)
```

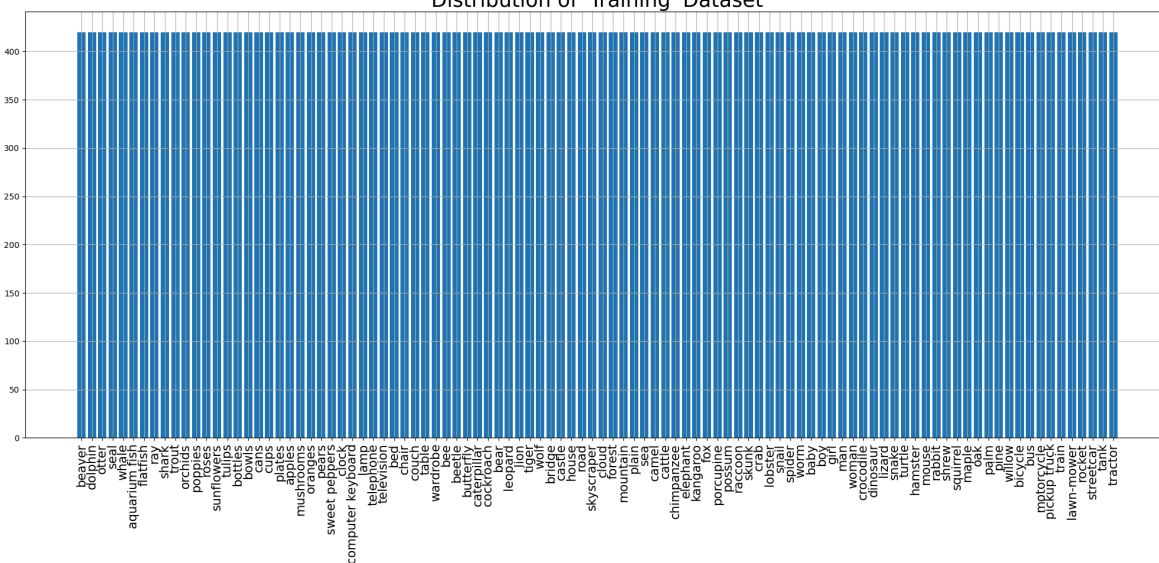


```
labels[32398].shape: (1,)  
imgs[32398].shape: (32, 32, 3)
```

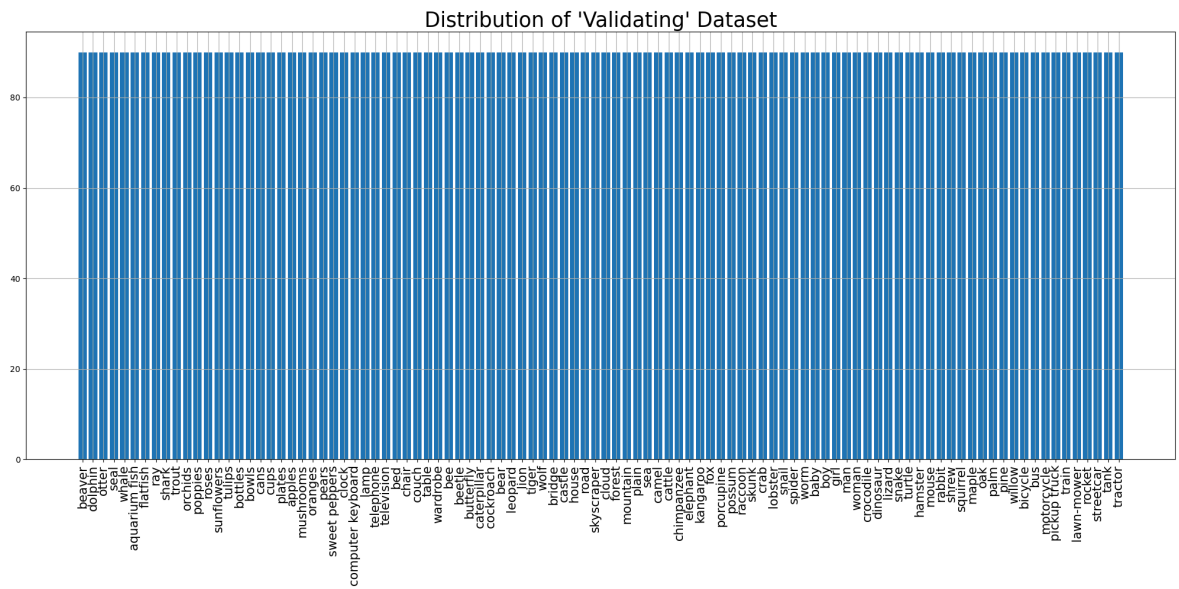


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

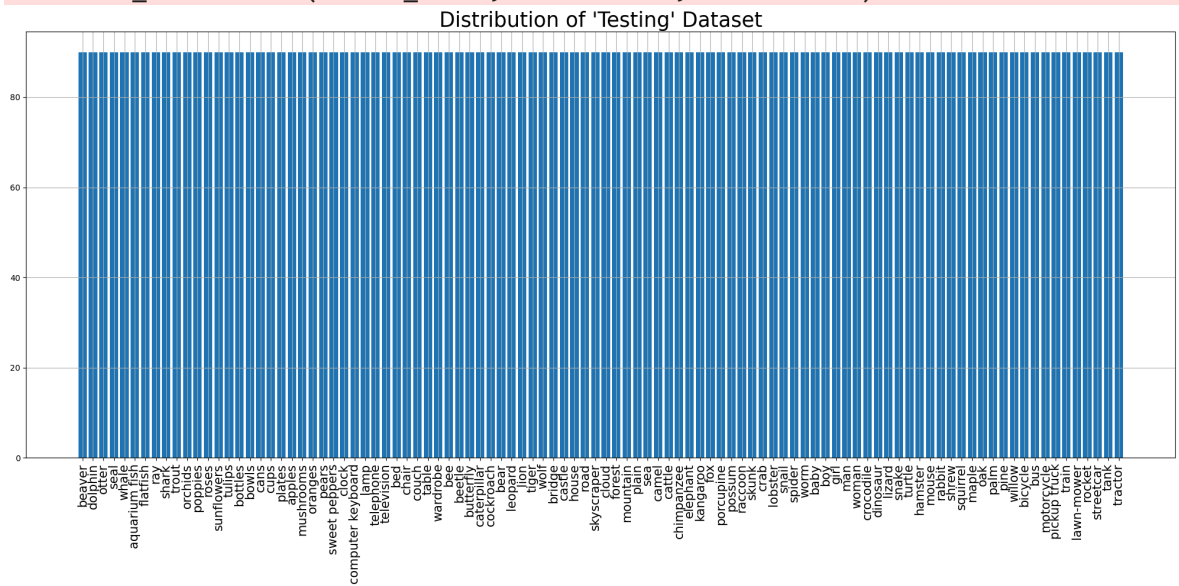
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)

◀  ▶

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 28ms/step - accuracy: 0.2670 - f1_score: 0.2610 - loss: 3.1818 - precision: 0.6370
Epoch 1: val_loss improved from inf to 1.57274, saving model to best_model_v1.keras
657/657 ————— 39s 44ms/step - accuracy: 0.2671 - f1_score: 0.2612 - loss: 3.1809 - precision: 0.6371 - val_accuracy: 0.5770 - val_f1_score: 0.5702 - val_loss: 1.5727 - val_precision: 0.8365
Epoch 2/25
656/657 ————— 0s 23ms/step - accuracy: 0.4699 - f1_score: 0.4650 - loss: 1.9528 - precision: 0.7291
Epoch 2: val_loss improved from 1.57274 to 1.46138, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4699 - f1_score: 0.4650 - loss: 1.9526 - precision: 0.7291 - val_accuracy: 0.5926 - val_f1_score: 0.5887 - val_loss: 1.4614 - val_precision: 0.8163
Epoch 3/25
655/657 ————— 0s 23ms/step - accuracy: 0.5089 - f1_score: 0.5043 - loss: 1.7857 - precision: 0.7465
Epoch 3: val_loss improved from 1.46138 to 1.41969, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5090 - f1_score: 0.5044 - loss: 1.7856 - precision: 0.7465 - val_accuracy: 0.6010 - val_f1_score: 0.5976 - val_loss: 1.4197 - val_precision: 0.8313
Epoch 4/25
656/657 ————— 0s 23ms/step - accuracy: 0.5275 - f1_score: 0.5237 - loss: 1.6989 - precision: 0.7609
Epoch 4: val_loss improved from 1.41969 to 1.37061, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5275 - f1_score: 0.5237 - loss: 1.6989 - precision: 0.7609 - val_accuracy: 0.6176 - val_f1_score: 0.6144 - val_loss: 1.3706 - val_precision: 0.8314
Epoch 5/25
657/657 ————— 0s 23ms/step - accuracy: 0.5363 - f1_score: 0.5324 - loss: 1.6501 - precision: 0.7607
Epoch 5: val_loss did not improve from 1.37061
657/657 ————— 18s 28ms/step - accuracy: 0.5363 - f1_score: 0.5324 - loss: 1.6501 - precision: 0.7607 - val_accuracy: 0.6086 - val_f1_score: 0.6033 - val_loss: 1.3894 - val_precision: 0.8272
Epoch 6/25
657/657 ————— 0s 23ms/step - accuracy: 0.5480 - f1_score: 0.5443 - loss: 1.6041 - precision: 0.7663
Epoch 6: val_loss did not improve from 1.37061
657/657 ————— 18s 28ms/step - accuracy: 0.5480 - f1_score: 0.5443 - loss: 1.6041 - precision: 0.7663 - val_accuracy: 0.6068 - val_f1_score: 0.6033 - val_loss: 1.3761 - val_precision: 0.8153
Epoch 7/25
655/657 ————— 0s 23ms/step - accuracy: 0.5482 - f1_score: 0.5431 - loss: 1.5920 - precision: 0.7590
Epoch 7: val_loss improved from 1.37061 to 1.34146, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5482 - f1_score: 0.5432 - loss: 1.5919 - precision: 0.7591 - val_accuracy: 0.6140 - val_f1_score: 0.6090 - val_loss: 1.3415 - val_precision: 0.8095
Epoch 8/25
655/657 ————— 0s 23ms/step - accuracy: 0.5611 - f1_score: 0.5571 - loss: 1.5602 - precision: 0.7790
Epoch 8: val_loss improved from 1.34146 to 1.33606, saving model to best_model_v1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5611 - f1_score: 0.5571

- loss: 1.5601 - precision: 0.7790 - val_accuracy: 0.6179 - val_f1_score: 0.6128
- val_loss: 1.3361 - val_precision: 0.8159
Epoch 9/25
655/657 ————— 0s 23ms/step - accuracy: 0.5603 - f1_score: 0.5557 -
loss: 1.5471 - precision: 0.7710
Epoch 9: val_loss did not improve from 1.33606
657/657 ————— 18s 28ms/step - accuracy: 0.5603 - f1_score: 0.5557
- loss: 1.5470 - precision: 0.7710 - val_accuracy: 0.6199 - val_f1_score: 0.6160
- val_loss: 1.3398 - val_precision: 0.8162
Epoch 10/25
655/657 ————— 0s 23ms/step - accuracy: 0.5640 - f1_score: 0.5612 -
loss: 1.5348 - precision: 0.7728
Epoch 10: val_loss improved from 1.33606 to 1.33506, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5640 - f1_score: 0.5612
- loss: 1.5347 - precision: 0.7728 - val_accuracy: 0.6251 - val_f1_score: 0.6231
- val_loss: 1.3351 - val_precision: 0.8256
Epoch 11/25
657/657 ————— 0s 23ms/step - accuracy: 0.5685 - f1_score: 0.5643 -
loss: 1.5125 - precision: 0.7748
Epoch 11: val_loss improved from 1.33506 to 1.33383, saving model to best_model_v
1.keras
657/657 ————— 19s 30ms/step - accuracy: 0.5685 - f1_score: 0.5643
- loss: 1.5125 - precision: 0.7748 - val_accuracy: 0.6226 - val_f1_score: 0.6194
- val_loss: 1.3338 - val_precision: 0.8300
Epoch 12/25
657/657 ————— 0s 23ms/step - accuracy: 0.5685 - f1_score: 0.5649 -
loss: 1.5180 - precision: 0.7733
Epoch 12: val_loss improved from 1.33383 to 1.33104, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5685 - f1_score: 0.5649
- loss: 1.5180 - precision: 0.7733 - val_accuracy: 0.6260 - val_f1_score: 0.6230
- val_loss: 1.3310 - val_precision: 0.8269
Epoch 13/25
655/657 ————— 0s 23ms/step - accuracy: 0.5699 - f1_score: 0.5664 -
loss: 1.5115 - precision: 0.7759
Epoch 13: val_loss improved from 1.33104 to 1.32774, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5699 - f1_score: 0.5664
- loss: 1.5114 - precision: 0.7759 - val_accuracy: 0.6199 - val_f1_score: 0.6170
- val_loss: 1.3277 - val_precision: 0.8190
Epoch 14/25
657/657 ————— 0s 23ms/step - accuracy: 0.5680 - f1_score: 0.5642 -
loss: 1.5067 - precision: 0.7749
Epoch 14: val_loss improved from 1.32774 to 1.31286, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5680 - f1_score: 0.5642
- loss: 1.5067 - precision: 0.7749 - val_accuracy: 0.6283 - val_f1_score: 0.6255
- val_loss: 1.3129 - val_precision: 0.8205
Epoch 15/25
657/657 ————— 0s 23ms/step - accuracy: 0.5758 - f1_score: 0.5723 -
loss: 1.4808 - precision: 0.7758
Epoch 15: val_loss did not improve from 1.31286
657/657 ————— 18s 28ms/step - accuracy: 0.5758 - f1_score: 0.5723
- loss: 1.4808 - precision: 0.7758 - val_accuracy: 0.6306 - val_f1_score: 0.6251
- val_loss: 1.3183 - val_precision: 0.8216
Epoch 16/25
656/657 ————— 0s 23ms/step - accuracy: 0.5779 - f1_score: 0.5744 -
loss: 1.4754 - precision: 0.7730
Epoch 16: val_loss improved from 1.31286 to 1.31117, saving model to best_model_v

1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5779 - f1_score: 0.5744
- loss: 1.4753 - precision: 0.7731 - val_accuracy: 0.6280 - val_f1_score: 0.6235
- val_loss: 1.3112 - val_precision: 0.8148
Epoch 17/25
655/657 ————— 0s 23ms/step - accuracy: 0.5762 - f1_score: 0.5727 -
loss: 1.4782 - precision: 0.7714
Epoch 17: val_loss improved from 1.31117 to 1.31090, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5762 - f1_score: 0.5727
- loss: 1.4782 - precision: 0.7714 - val_accuracy: 0.6310 - val_f1_score: 0.6275
- val_loss: 1.3109 - val_precision: 0.8215
Epoch 18/25
655/657 ————— 0s 23ms/step - accuracy: 0.5815 - f1_score: 0.5774 -
loss: 1.4679 - precision: 0.7811
Epoch 18: val_loss did not improve from 1.31090
657/657 ————— 18s 28ms/step - accuracy: 0.5815 - f1_score: 0.5774
- loss: 1.4678 - precision: 0.7811 - val_accuracy: 0.6323 - val_f1_score: 0.6268
- val_loss: 1.3146 - val_precision: 0.8225
Epoch 19/25
655/657 ————— 0s 23ms/step - accuracy: 0.5820 - f1_score: 0.5788 -
loss: 1.4591 - precision: 0.7779
Epoch 19: val_loss improved from 1.31090 to 1.30540, saving model to best_model_v
1.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5820 - f1_score: 0.5789
- loss: 1.4591 - precision: 0.7779 - val_accuracy: 0.6317 - val_f1_score: 0.6265
- val_loss: 1.3054 - val_precision: 0.8177
Epoch 20/25
657/657 ————— 0s 23ms/step - accuracy: 0.5826 - f1_score: 0.5788 -
loss: 1.4578 - precision: 0.7812
Epoch 20: val_loss did not improve from 1.30540
657/657 ————— 18s 28ms/step - accuracy: 0.5826 - f1_score: 0.5788
- loss: 1.4578 - precision: 0.7812 - val_accuracy: 0.6224 - val_f1_score: 0.6184
- val_loss: 1.3180 - val_precision: 0.8196
Epoch 21/25
655/657 ————— 0s 23ms/step - accuracy: 0.5785 - f1_score: 0.5754 -
loss: 1.4647 - precision: 0.7754
Epoch 21: val_loss did not improve from 1.30540
657/657 ————— 18s 28ms/step - accuracy: 0.5786 - f1_score: 0.5754
- loss: 1.4646 - precision: 0.7754 - val_accuracy: 0.6308 - val_f1_score: 0.6257
- val_loss: 1.3151 - val_precision: 0.8179
Epoch 22/25
657/657 ————— 0s 23ms/step - accuracy: 0.5870 - f1_score: 0.5837 -
loss: 1.4506 - precision: 0.7804
Epoch 22: val_loss did not improve from 1.30540
657/657 ————— 18s 28ms/step - accuracy: 0.5870 - f1_score: 0.5838
- loss: 1.4505 - precision: 0.7804 - val_accuracy: 0.6292 - val_f1_score: 0.6263
- val_loss: 1.3090 - val_precision: 0.8270
Epoch 23/25
656/657 ————— 0s 23ms/step - accuracy: 0.5855 - f1_score: 0.5819 -
loss: 1.4457 - precision: 0.7813
Epoch 23: val_loss did not improve from 1.30540
657/657 ————— 18s 28ms/step - accuracy: 0.5855 - f1_score: 0.5819
- loss: 1.4456 - precision: 0.7813 - val_accuracy: 0.6242 - val_f1_score: 0.6208
- val_loss: 1.3228 - val_precision: 0.8186
Epoch 24/25
657/657 ————— 0s 23ms/step - accuracy: 0.5859 - f1_score: 0.5819 -
loss: 1.4468 - precision: 0.7788
Epoch 24: val_loss improved from 1.30540 to 1.30255, saving model to best_model_v
1.keras

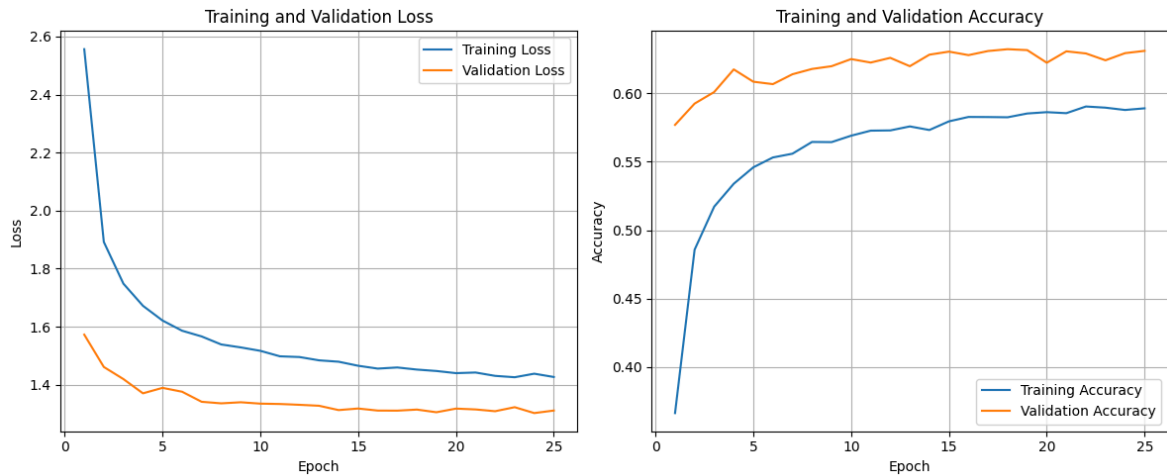
657/657 ————— **19s** 29ms/step - accuracy: 0.5859 - f1_score: 0.5819
 - loss: 1.4468 - precision: 0.7788 - val_accuracy: 0.6294 - val_f1_score: 0.6280
 - val_loss: 1.3026 - val_precision: 0.8274

Epoch 25/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5841 - f1_score: 0.5807 -
 loss: 1.4393 - precision: 0.7799

Epoch 25: val_loss did not improve from 1.30255

657/657 ————— **18s** 28ms/step - accuracy: 0.5841 - f1_score: 0.5807
 - loss: 1.4392 - precision: 0.7800 - val_accuracy: 0.6311 - val_f1_score: 0.6277
 - val_loss: 1.3113 - val_precision: 0.8260



141/141 ————— **11s** 38ms/step - accuracy: 0.6098 - f1_score: 0.6025
 - loss: 1.3234 - precision: 0.8148

Test Accuracy: 62.62%

Test Loss: 1.2982264757156372

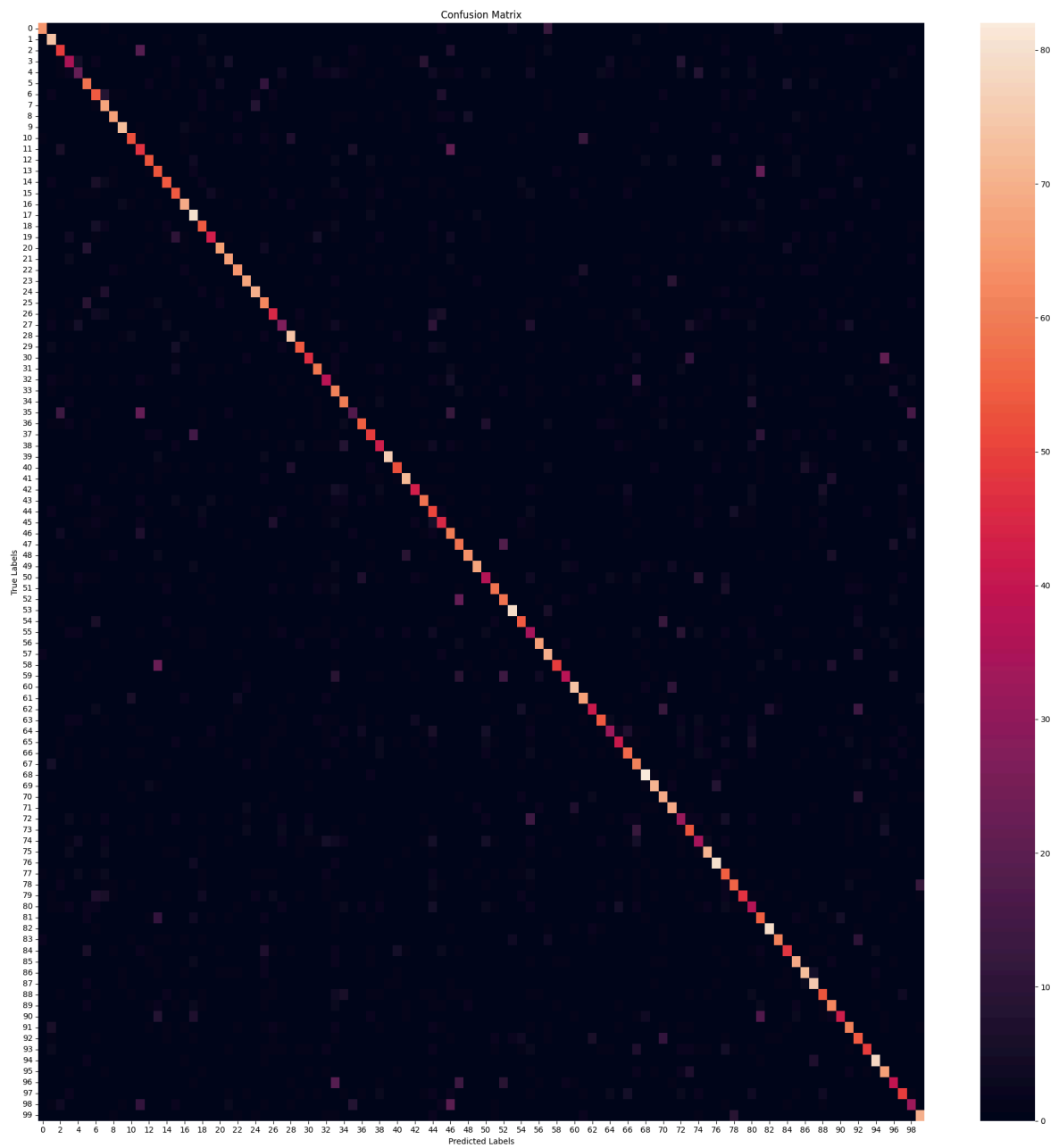
Test Precision: 82.57%

Test F1 Scores (Per Class): [80.74533 73.26733 54.44444 42.94478 30.34482 6
 0.73298 55.10204

66.01941 77.906975 83.42856 65. 43.92523 64.63414 50.961536
 63.905323 58.69565 76.66666 71.42857 59.016384 56.57894 78.57143
 74.03315 76.02339 77.27273 80. 59.903378 52.941174 36.11111
 78.30688 67.49999 61.33333 66.666664 46.060596 46.212112 62.5
 24.999996 65.088745 59.036137 49.122803 88.888885 65.82278 78.68852
 56.953632 68.63905 44.34782 50.847454 49.586773 55.50239 77.10842
 79.53216 43.786976 69.87952 60.73298 86.81318 63.905323 33.999996
 78.362564 68.965515 67.586205 52.777775 80.43478 73.224045 55.405403
 65.85365 44.44444 54.304623 61.956512 53.744488 83.248726 79.775276
 65.09434 72.251305 36.2573 59.886997 37.988823 78.453026 79.20792
 52.631577 65.11627 62.251644 36.274506 48.888885 87.77777 72.61904
 58.536575 71.13402 77.24867 74.62686 58.88888 65.60846 57.534237
 66.304344 52.427177 58.682625 83.422455 66.666664 46.78362 57.64705
 37.125744 76.50272]

Average Test F1 Scores:62.56

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.92	0.72	0.81	90
dolphin	0.66	0.82	0.73	90
otter	0.54	0.54	0.54	90
seal	0.48	0.39	0.43	90
whale	0.40	0.24	0.30	90
aquarium fish	0.57	0.64	0.61	90
flatfish	0.51	0.60	0.55	90
ray	0.59	0.76	0.66	90
shark	0.82	0.74	0.78	90
trout	0.86	0.81	0.83	90
orchids	0.74	0.58	0.65	90
poppies	0.38	0.52	0.44	90
roses	0.72	0.59	0.65	90
sunflowers	0.45	0.59	0.51	90
tulips	0.68	0.60	0.64	90
bottles	0.57	0.60	0.59	90
bowls	0.77	0.77	0.77	90
cans	0.60	0.89	0.71	90
cups	0.58	0.60	0.59	90
plates	0.69	0.48	0.57	90
apples	0.85	0.73	0.79	90
mushrooms	0.74	0.74	0.74	90
oranges	0.80	0.72	0.76	90
pears	0.79	0.76	0.77	90
sweet peppers	0.82	0.78	0.80	90
clock	0.53	0.69	0.60	90
computer keyboard	0.56	0.50	0.53	90
lamp	0.48	0.29	0.36	90
telephone	0.75	0.82	0.78	90
television	0.76	0.60	0.67	90
bed	0.77	0.51	0.61	90
chair	0.69	0.66	0.67	90
couch	0.51	0.42	0.46	90
table	0.35	0.68	0.46	90
wardrobe	0.59	0.67	0.62	90
bee	0.44	0.19	0.26	90
beetle	0.70	0.61	0.65	90
butterfly	0.64	0.54	0.59	90
caterpillar	0.52	0.47	0.49	90
cockroach	0.94	0.84	0.89	90
bear	0.76	0.58	0.66	90
leopard	0.77	0.80	0.79	90
lion	0.70	0.48	0.57	90
tiger	0.73	0.64	0.69	90
wolf	0.36	0.57	0.44	90
bridge	0.52	0.50	0.51	90
castle	0.40	0.67	0.50	90
house	0.49	0.64	0.56	90
road	0.84	0.71	0.77	90
skyscraper	0.84	0.76	0.80	90
cloud	0.47	0.41	0.44	90
forest	0.76	0.64	0.70	90
mountain	0.57	0.64	0.61	90
plain	0.86	0.88	0.87	90
sea	0.68	0.60	0.64	90
camel	0.31	0.38	0.34	90
cattle	0.83	0.74	0.78	90
chimpanzee	0.62	0.78	0.69	90

elephant	0.89	0.54	0.68	90
kangaroo	0.70	0.42	0.53	90
fox	0.78	0.82	0.80	90
porcupine	0.72	0.74	0.73	90
possum	0.71	0.46	0.55	90
raccoon	0.73	0.60	0.66	90
skunk	0.59	0.36	0.44	90
crab	0.67	0.46	0.54	90
lobster	0.61	0.63	0.62	90
snail	0.45	0.68	0.54	90
spider	0.77	0.91	0.84	90
worm	0.81	0.79	0.80	90
baby	0.57	0.77	0.65	90
boy	0.68	0.77	0.72	90
girl	0.38	0.34	0.36	90
man	0.61	0.59	0.60	90
woman	0.38	0.38	0.38	90
crocodile	0.78	0.79	0.78	90
dinosaur	0.71	0.89	0.79	90
lizard	0.46	0.61	0.53	90
snake	0.68	0.62	0.65	90
turtle	0.77	0.52	0.62	90
hamster	0.32	0.41	0.36	90
mouse	0.41	0.61	0.49	90
rabbit	0.88	0.88	0.88	90
shrew	0.78	0.68	0.73	90
squirrel	0.65	0.53	0.59	90
maple	0.66	0.77	0.71	90
oak	0.74	0.81	0.77	90
palm	0.68	0.83	0.75	90
pine	0.59	0.59	0.59	90
willow	0.63	0.69	0.66	90
bicycle	0.75	0.47	0.58	90
bus	0.65	0.68	0.66	90
motorcycle	0.47	0.60	0.52	90
pickup truck	0.64	0.54	0.59	90
train	0.80	0.87	0.83	90
lawn-mower	0.60	0.74	0.67	90
rocket	0.49	0.44	0.47	90
streetcar	0.61	0.54	0.58	90
tank	0.42	0.36	0.38	90
tractor	0.75	0.78	0.77	90
accuracy			0.63	9000
macro avg	0.64	0.63	0.63	9000
weighted avg	0.64	0.63	0.63	9000

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 26ms/step - accuracy: 0.6056 - f1_score: 0.6033
- loss: 1.3577 - precision: 0.7981
Epoch 1: val_loss improved from inf to 1.24511, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.6056 - f1_score: 0.6033 - loss: 1.3577 - precision: 0.7981 - val_accuracy: 0.6429 - val_f1_score: 0.6407 - val_loss: 1.2451 - val_precision: 0.8317
Epoch 2/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6307 - f1_score: 0.6280
- loss: 1.2629 - precision: 0.8105
Epoch 2: val_loss improved from 1.24511 to 1.23371, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6307 - f1_score: 0.6280 - loss: 1.2629 - precision: 0.8105 - val_accuracy: 0.6457 - val_f1_score: 0.6434 - val_loss: 1.2337 - val_precision: 0.8281
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6399 - f1_score: 0.6370
- loss: 1.2301 - precision: 0.8164
Epoch 3: val_loss improved from 1.23371 to 1.22723, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6399 - f1_score: 0.6370 - loss: 1.2301 - precision: 0.8164 - val_accuracy: 0.6466 - val_f1_score: 0.6441 - val_loss: 1.2272 - val_precision: 0.8263
Epoch 4/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6486 - f1_score: 0.6458
- loss: 1.2057 - precision: 0.8232
Epoch 4: val_loss improved from 1.22723 to 1.22245, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6486 - f1_score: 0.6458 - loss: 1.2057 - precision: 0.8232 - val_accuracy: 0.6469 - val_f1_score: 0.6443 - val_loss: 1.2224 - val_precision: 0.8257
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6466 - f1_score: 0.6437
- loss: 1.1983 - precision: 0.8183
Epoch 5: val_loss improved from 1.22245 to 1.21824, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6466 - f1_score: 0.6437 - loss: 1.1984 - precision: 0.8183 - val_accuracy: 0.6488 - val_f1_score: 0.6464 - val_loss: 1.2182 - val_precision: 0.8262
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6526 - f1_score: 0.6499
- loss: 1.1861 - precision: 0.8235
Epoch 6: val_loss improved from 1.21824 to 1.21446, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6526 - f1_score: 0.6499 - loss: 1.1862 - precision: 0.8235 - val_accuracy: 0.6507 - val_f1_score: 0.6483 - val_loss: 1.2145 - val_precision: 0.8262
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6564 - f1_score: 0.6535
- loss: 1.1698 - precision: 0.8266
Epoch 7: val_loss improved from 1.21446 to 1.21133, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6564 - f1_score: 0.6535 - loss: 1.1699 - precision: 0.8266 - val_accuracy: 0.6532 - val_f1_score: 0.6510 - val_loss: 1.2113 - val_precision: 0.8256
Epoch 8/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6601 - f1_score: 0.6572
- loss: 1.1580 - precision: 0.8306
Epoch 8: val_loss improved from 1.21133 to 1.20848, saving model to best_model_fine_tune_v1.keras

```
ne_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6601 - f1_score: 0.657
2 - loss: 1.1580 - precision: 0.8306 - val_accuracy: 0.6527 - val_f1_score: 0.650
3 - val_loss: 1.2085 - val_precision: 0.8258
Epoch 9/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6631 - f1_score: 0.6601
- loss: 1.1461 - precision: 0.8331
Epoch 9: val_loss improved from 1.20848 to 1.20580, saving model to best_model_fi
ne_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6631 - f1_score: 0.660
1 - loss: 1.1461 - precision: 0.8331 - val_accuracy: 0.6526 - val_f1_score: 0.650
3 - val_loss: 1.2058 - val_precision: 0.8246
Epoch 10/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6670 - f1_score: 0.6641
- loss: 1.1328 - precision: 0.8364
Epoch 10: val_loss improved from 1.20580 to 1.20273, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6669 - f1_score: 0.664
1 - loss: 1.1329 - precision: 0.8364 - val_accuracy: 0.6524 - val_f1_score: 0.650
3 - val_loss: 1.2027 - val_precision: 0.8261
Epoch 11/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6682 - f1_score: 0.6657
- loss: 1.1320 - precision: 0.8353
Epoch 11: val_loss improved from 1.20273 to 1.20069, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6682 - f1_score: 0.665
7 - loss: 1.1320 - precision: 0.8352 - val_accuracy: 0.6540 - val_f1_score: 0.651
9 - val_loss: 1.2007 - val_precision: 0.8255
Epoch 12/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6732 - f1_score: 0.6707
- loss: 1.1149 - precision: 0.8377
Epoch 12: val_loss improved from 1.20069 to 1.19825, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6732 - f1_score: 0.670
7 - loss: 1.1149 - precision: 0.8377 - val_accuracy: 0.6539 - val_f1_score: 0.651
7 - val_loss: 1.1983 - val_precision: 0.8251
Epoch 13/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6744 - f1_score: 0.6718
- loss: 1.1060 - precision: 0.8390
Epoch 13: val_loss improved from 1.19825 to 1.19655, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6744 - f1_score: 0.671
8 - loss: 1.1060 - precision: 0.8390 - val_accuracy: 0.6544 - val_f1_score: 0.652
0 - val_loss: 1.1965 - val_precision: 0.8236
Epoch 14/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6777 - f1_score: 0.6750
- loss: 1.0988 - precision: 0.8422
Epoch 14: val_loss improved from 1.19655 to 1.19496, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6776 - f1_score: 0.675
0 - loss: 1.0988 - precision: 0.8422 - val_accuracy: 0.6560 - val_f1_score: 0.653
6 - val_loss: 1.1950 - val_precision: 0.8255
Epoch 15/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6786 - f1_score: 0.6759
- loss: 1.0955 - precision: 0.8444
Epoch 15: val_loss improved from 1.19496 to 1.19360, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6786 - f1_score: 0.675
9 - loss: 1.0956 - precision: 0.8444 - val_accuracy: 0.6571 - val_f1_score: 0.654
5 - val_loss: 1.1936 - val_precision: 0.8255
```



141/141 ————— **11s 40ms/step** - accuracy: 0.6465 - f1_score: 0.6383

- loss: 1.2077 - precision: 0.8153

Test Accuracy: 65.76%

Test Loss: 1.1848493814468384

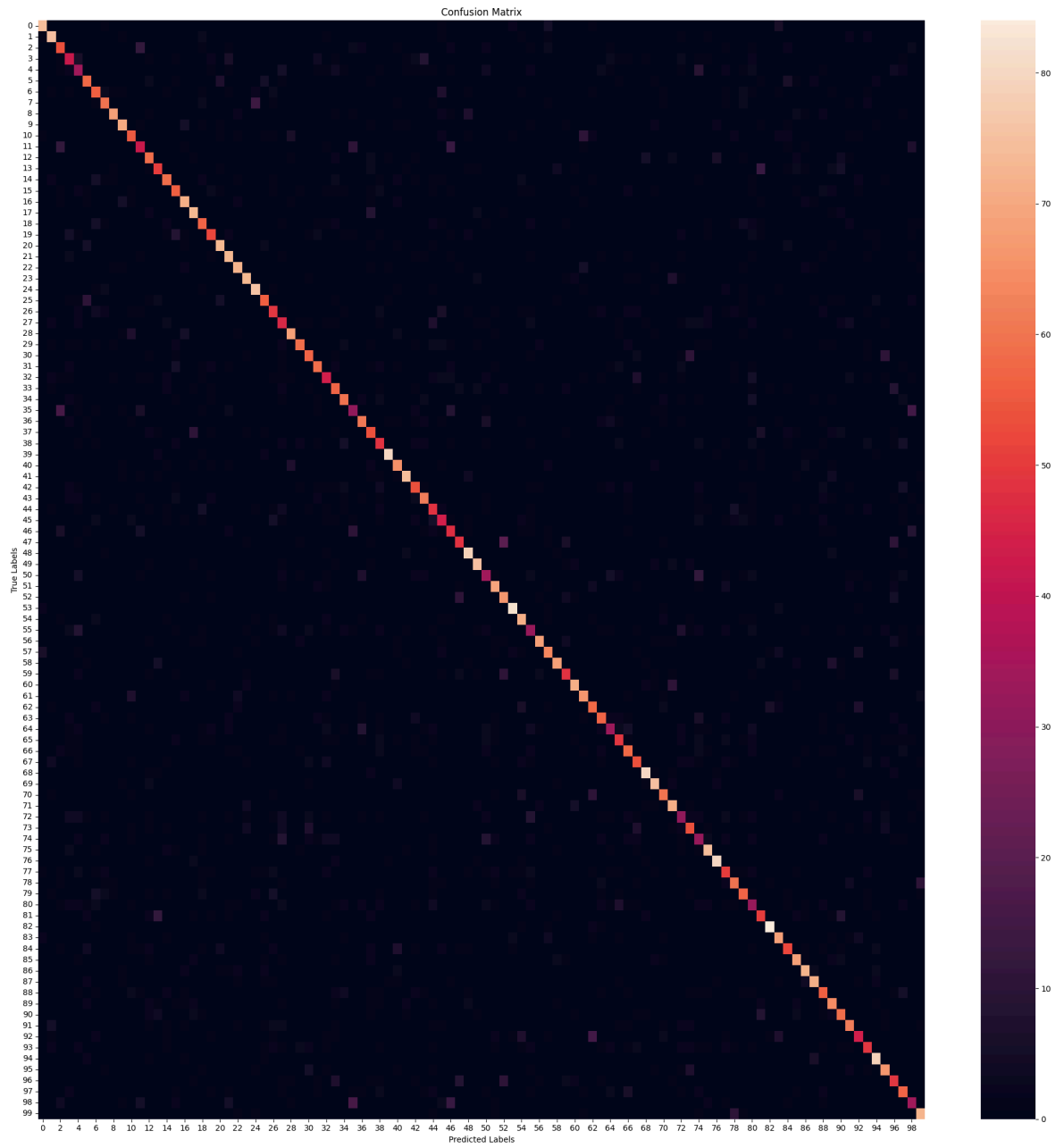
Test Precision: 82.63%

Test F1 Scores (Per Class): [83.615814 78.947365 55.6701 47.999996 37.158463 63.387978 59.574467

69.36415 81.437126 79.54544 62.14689 49.704136 63.440853 58.823524
67.42856 62.222218 76.34408 76.43979 62.29508 62.275444 79.347824
74.111664 78.68852 81.11111 79.99999 66.27218 55.05617 46.46464
76.83616 72.39264 64.04494 69.005844 51.461983 57.575756 67.796616
37.349392 68.15642 59.459454 51.891888 90.90909 68.783066 83.615814
61.363636 69.66292 51.06383 49.438198 49.999992 57.83132 81.025635
81.96721 40.23668 76.24309 65.346535 89.61748 70.99999 37.647053
79.99999 70.786514 79.53216 58.895695 80.22598 73.74301 61.702126
64.77273 43.137253 57.64705 63.04347 60.67415 85.56149 80.64516
69.76743 73.95833 38.2716 59.999996 34.224594 79.569885 87.43169
56.818176 66.666664 64.40677 34.594585 55.865913 91.304344 71.20418
62.650597 73.513504 78.68852 79.329605 62.222218 66.666664 64.51612
68.13186 53.65853 57.64705 84.04254 71.35134 51.851852 60.96256
40.71856 77.83783]

Average Test F1 Scores:65.59

282/282 ————— **11s 28ms/step**



	precision	recall	f1-score	support
beaver	0.85	0.82	0.84	90
dolphin	0.75	0.83	0.79	90
otter	0.52	0.60	0.56	90
seal	0.49	0.47	0.48	90
whale	0.37	0.38	0.37	90
aquarium fish	0.62	0.64	0.63	90
flatfish	0.57	0.62	0.60	90
ray	0.72	0.67	0.69	90
shark	0.88	0.76	0.81	90
trout	0.81	0.78	0.80	90
orchids	0.63	0.61	0.62	90
poppies	0.53	0.47	0.50	90
roses	0.61	0.66	0.63	90
sunflowers	0.62	0.56	0.59	90
tulips	0.69	0.66	0.67	90
bottles	0.62	0.62	0.62	90
bowls	0.74	0.79	0.76	90
cans	0.72	0.81	0.76	90
cups	0.61	0.63	0.62	90
plates	0.68	0.58	0.62	90
apples	0.78	0.81	0.79	90
mushrooms	0.68	0.81	0.74	90
oranges	0.77	0.80	0.79	90
pears	0.81	0.81	0.81	90
sweet peppers	0.76	0.84	0.80	90
clock	0.71	0.62	0.66	90
computer keyboard	0.56	0.54	0.55	90
lamp	0.43	0.51	0.46	90
telephone	0.78	0.76	0.77	90
television	0.81	0.66	0.72	90
bed	0.65	0.63	0.64	90
chair	0.73	0.66	0.69	90
couch	0.54	0.49	0.51	90
table	0.53	0.63	0.58	90
wardrobe	0.69	0.67	0.68	90
bee	0.41	0.34	0.37	90
beetle	0.69	0.68	0.68	90
butterfly	0.57	0.60	0.59	90
caterpillar	0.51	0.53	0.52	90
cockroach	0.93	0.89	0.91	90
bear	0.66	0.72	0.69	90
leopard	0.85	0.82	0.84	90
lion	0.63	0.60	0.61	90
tiger	0.70	0.69	0.70	90
wolf	0.49	0.53	0.51	90
bridge	0.50	0.49	0.49	90
castle	0.48	0.52	0.50	90
house	0.63	0.53	0.58	90
road	0.75	0.88	0.81	90
skyscraper	0.81	0.83	0.82	90
cloud	0.43	0.38	0.40	90
forest	0.76	0.77	0.76	90
mountain	0.59	0.73	0.65	90
plain	0.88	0.91	0.90	90
sea	0.65	0.79	0.71	90
camel	0.40	0.36	0.38	90
cattle	0.85	0.76	0.80	90
chimpanzee	0.72	0.70	0.71	90

elephant	0.84	0.76	0.80	90
kangaroo	0.66	0.53	0.59	90
fox	0.82	0.79	0.80	90
porcupine	0.74	0.73	0.74	90
possum	0.59	0.64	0.62	90
raccoon	0.66	0.63	0.65	90
skunk	0.52	0.37	0.43	90
crab	0.61	0.54	0.58	90
lobster	0.62	0.64	0.63	90
snail	0.61	0.60	0.61	90
spider	0.82	0.89	0.86	90
worm	0.78	0.83	0.81	90
baby	0.73	0.67	0.70	90
boy	0.70	0.79	0.74	90
girl	0.43	0.34	0.38	90
man	0.60	0.60	0.60	90
woman	0.33	0.36	0.34	90
crocodile	0.77	0.82	0.80	90
dinosaur	0.86	0.89	0.87	90
lizard	0.58	0.56	0.57	90
snake	0.67	0.67	0.67	90
turtle	0.66	0.63	0.64	90
hamster	0.34	0.36	0.35	90
mouse	0.56	0.56	0.56	90
rabbit	0.89	0.93	0.91	90
shrew	0.67	0.76	0.71	90
squirrel	0.68	0.58	0.63	90
maple	0.72	0.76	0.74	90
oak	0.77	0.80	0.79	90
palm	0.80	0.79	0.79	90
pine	0.62	0.62	0.62	90
willow	0.63	0.71	0.67	90
bicycle	0.62	0.66	0.64	90
bus	0.67	0.69	0.68	90
motorcycle	0.59	0.49	0.54	90
pickup truck	0.61	0.54	0.58	90
train	0.81	0.88	0.84	90
lawn-mower	0.69	0.73	0.71	90
rocket	0.49	0.54	0.52	90
streetcar	0.59	0.63	0.61	90
tank	0.44	0.38	0.41	90
tractor	0.76	0.80	0.78	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

With higher mu in training

```
In [ ]: for repeat_2_times in range(2):
        ##### <<<<<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")
```

```

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphins', 'otter', 'seal', 'whale', 'aquarium fish', 'f
'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                    .map(augment_dataset, num_parallel_calls=tf.data.experimen
#                    .batch(batch_size))

```

```

# .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.7), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2

```

```

        print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
        print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
        print(sample[1].shape) # Should be (batch_size, 100)
    print(f"Model input shape: {model.input_shape}")
    print(f"Model output shape: {model.output_shape}")
    sample = next(iter(test_dataset.as_numpy_iterator()))
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
    print(sample[0].shape, sample[1].shape) # Should match model input and output
    print("\n")
    #for x, y in test_dataset.take(1):
    #    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
    #for x_batch, y_batch in test_dataset.take(1):
    #    test_loss, test_acc = model.evaluate(x_batch, y_batch)
    #    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

    # Compile the model
    #tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
    model.compile(optimizer=optimizers.Adam(learning_rate=5e-3, weight_decay=3e-5),
                  loss='categorical_crossentropy',
                  metrics=['accuracy', 'precision', 'f1_score'])

    model.summary()

    ##### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
    early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                                   patience=5, # Num. epochs with no improvement
                                   restore_best_weights=True)

    #reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
    #                               factor=0.1, # Reduce lr by a factor
    #                               patience=3, # Num epochs w/ no improvement
    #                               min_lr=1e-6, # Min lr
    #                               verbose=1)

    #tensorboard = TensorBoard(log_dir='./logs', # Logs directory
    #                           histogram_freq=1, # Logs histograms for weights/acc
    #                           write_graph=True, # Logs graph of model
    #                           write_images=True) # Log images like weight histograms

    checkpoint = ModelCheckpoint('best_model_v2.keras', # Path where the best model is saved
                                monitor='val_loss', # What to monitor (validation loss)
                                save_best_only=True, # Only save model when val loss is best
                                mode='min', # Looking to minimize the validation loss
                                verbose=1) # Print out message when saving model

    checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v2.keras', # Path where the best model is saved
                                           monitor='val_loss', # What to monitor (validation loss)
                                           save_best_only=True, # Only save model when val loss is best
                                           mode='min', # Looking to minimize the validation loss
                                           verbose=1) # Print out message when saving model

    #cvs_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

    # Train the model
    history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                       batch_size=batch_size, callbacks=[early_stopping, checkpoint])

    ##### <<<<<<<<<Plot Training & Validation Error>>>>>>>>

    # Extract loss and accuracy
    epochs = range(1, len(history.history['loss'])+1)
    train_loss = history.history['loss']
    val_loss = history.history['val_loss']
    train_acc = history.history['accuracy']

```

```

val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v2.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

```

```

#### <<<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v2.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')

```



```
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

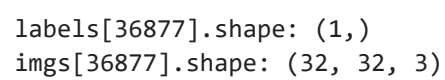
y_val.shape: (9000, 1)

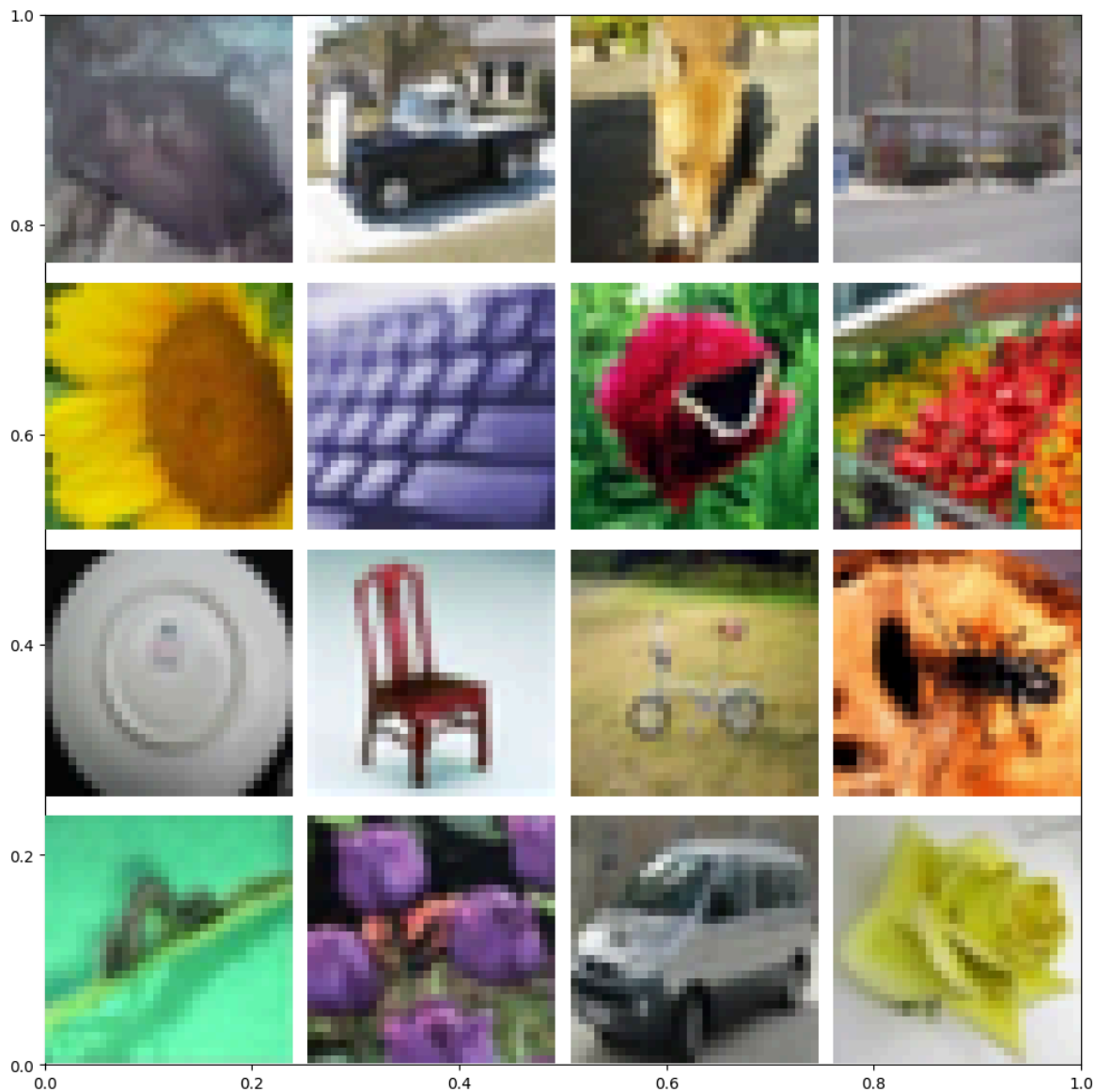
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

labels[24032].shape: (1,)

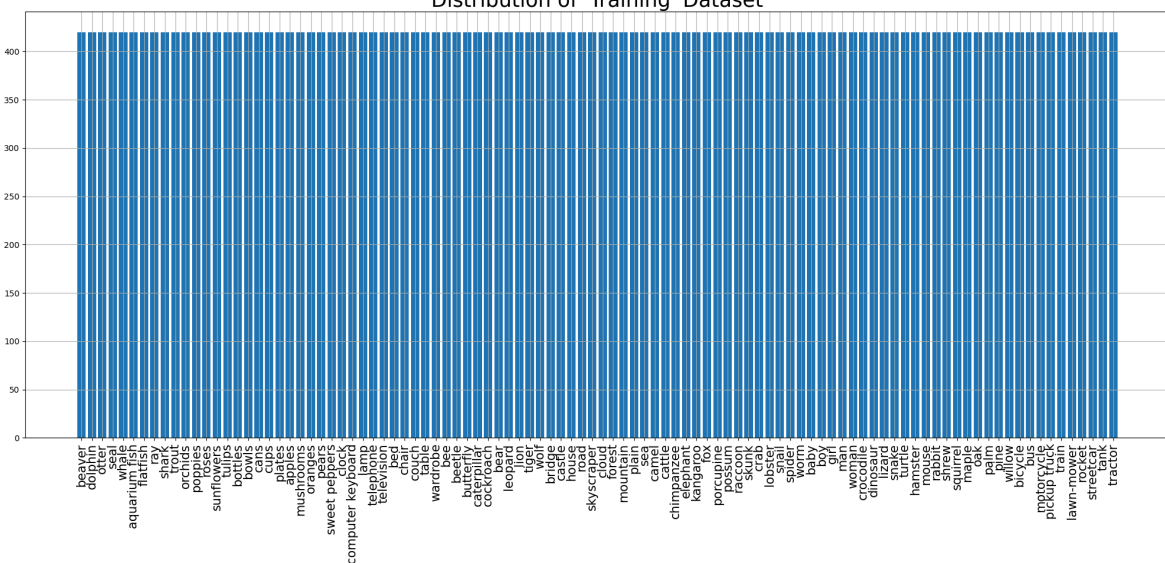
imgs[24032].shape: (32, 32, 3)



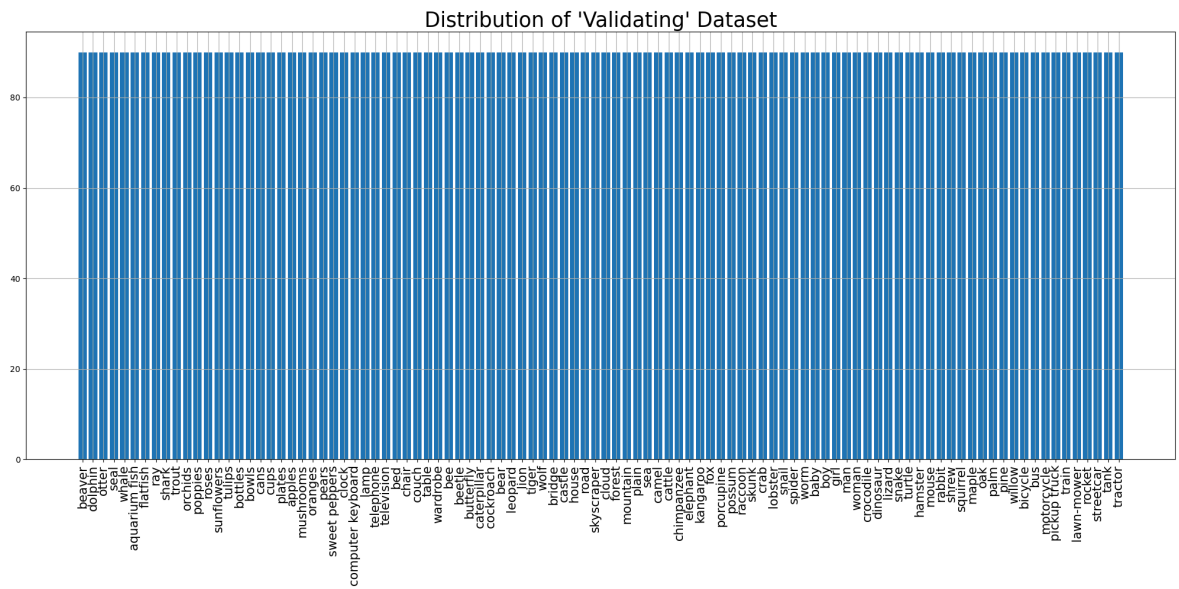


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

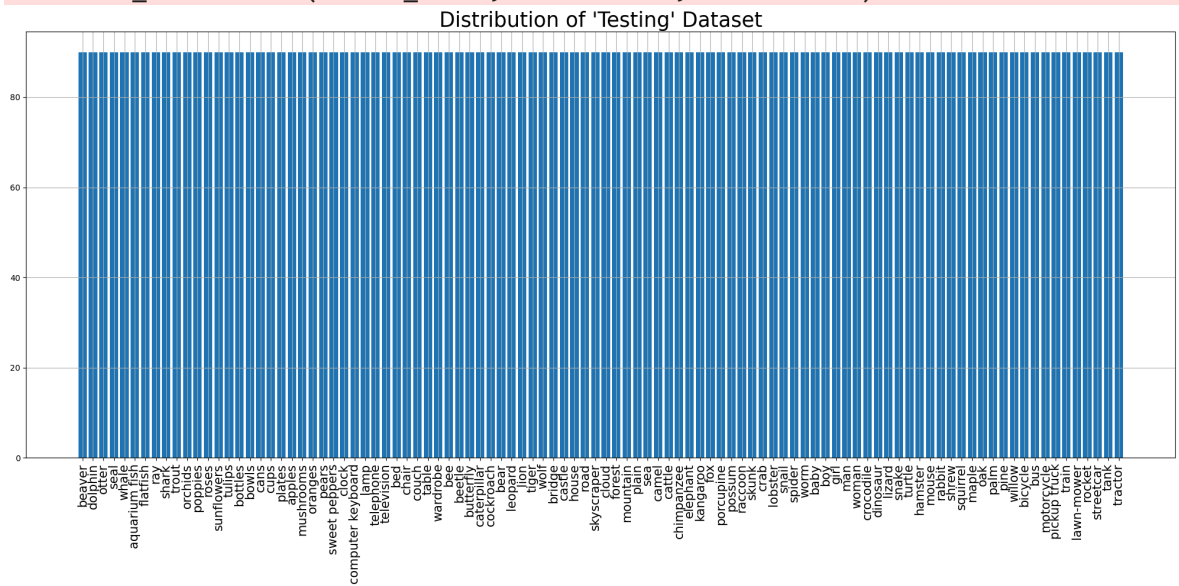
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25

657/657 ————— **0s** 28ms/step - accuracy: 0.1638 - f1_score: 0.1535 - loss: 3.7451 - precision: 0.4742

Epoch 1: val_loss improved from inf to 2.26985, saving model to best_model_v2.keras

657/657 ————— **39s** 44ms/step - accuracy: 0.1638 - f1_score: 0.1535 - loss: 3.7445 - precision: 0.4743 - val_accuracy: 0.3981 - val_f1_score: 0.3721 - val_loss: 2.2698 - val_precision: 0.7767

Epoch 2/25

655/657 ————— **0s** 24ms/step - accuracy: 0.2278 - f1_score: 0.2227 - loss: 3.1281 - precision: 0.5456

Epoch 2: val_loss improved from 2.26985 to 2.21026, saving model to best_model_v2.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.2278 - f1_score: 0.2227 - loss: 3.1279 - precision: 0.5456 - val_accuracy: 0.4014 - val_f1_score: 0.3785 - val_loss: 2.2103 - val_precision: 0.7719

Epoch 3/25

657/657 ————— **0s** 23ms/step - accuracy: 0.2364 - f1_score: 0.2328 - loss: 3.0702 - precision: 0.5480

Epoch 3: val_loss improved from 2.21026 to 2.18609, saving model to best_model_v2.keras

657/657 ————— **19s** 30ms/step - accuracy: 0.2364 - f1_score: 0.2328 - loss: 3.0701 - precision: 0.5480 - val_accuracy: 0.4180 - val_f1_score: 0.3905 - val_loss: 2.1861 - val_precision: 0.7948

Epoch 4/25

657/657 ————— **0s** 24ms/step - accuracy: 0.2424 - f1_score: 0.2367 - loss: 3.0196 - precision: 0.5769

Epoch 4: val_loss improved from 2.18609 to 2.10141, saving model to best_model_v2.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.2424 - f1_score: 0.2367 - loss: 3.0196 - precision: 0.5769 - val_accuracy: 0.4374 - val_f1_score: 0.4190 - val_loss: 2.1014 - val_precision: 0.8065

Epoch 5/25

656/657 ————— **0s** 23ms/step - accuracy: 0.2401 - f1_score: 0.2359 - loss: 3.0220 - precision: 0.5545

Epoch 5: val_loss did not improve from 2.10141

657/657 ————— **19s** 28ms/step - accuracy: 0.2401 - f1_score: 0.2359 - loss: 3.0219 - precision: 0.5545 - val_accuracy: 0.4136 - val_f1_score: 0.3875 - val_loss: 2.1998 - val_precision: 0.7680

Epoch 6/25

655/657 ————— **0s** 23ms/step - accuracy: 0.2450 - f1_score: 0.2398 - loss: 2.9981 - precision: 0.5541

Epoch 6: val_loss did not improve from 2.10141

657/657 ————— **18s** 28ms/step - accuracy: 0.2450 - f1_score: 0.2398 - loss: 2.9980 - precision: 0.5541 - val_accuracy: 0.4180 - val_f1_score: 0.3925 - val_loss: 2.1745 - val_precision: 0.8195

Epoch 7/25

657/657 ————— **0s** 23ms/step - accuracy: 0.2527 - f1_score: 0.2474 - loss: 2.9622 - precision: 0.5685

Epoch 7: val_loss did not improve from 2.10141

657/657 ————— **19s** 28ms/step - accuracy: 0.2527 - f1_score: 0.2475 - loss: 2.9622 - precision: 0.5685 - val_accuracy: 0.4399 - val_f1_score: 0.4194 - val_loss: 2.1177 - val_precision: 0.8281

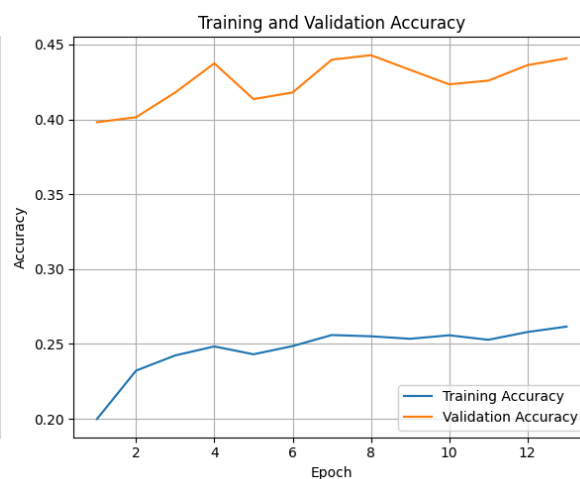
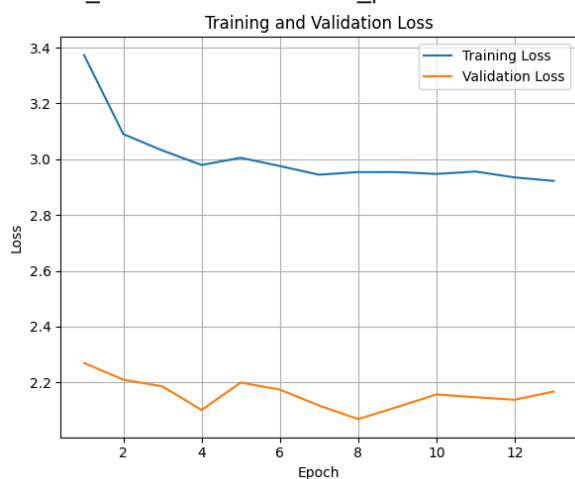
Epoch 8/25

656/657 ————— **0s** 23ms/step - accuracy: 0.2534 - f1_score: 0.2485 - loss: 2.9597 - precision: 0.5813

Epoch 8: val_loss improved from 2.10141 to 2.06887, saving model to best_model_v2.keras

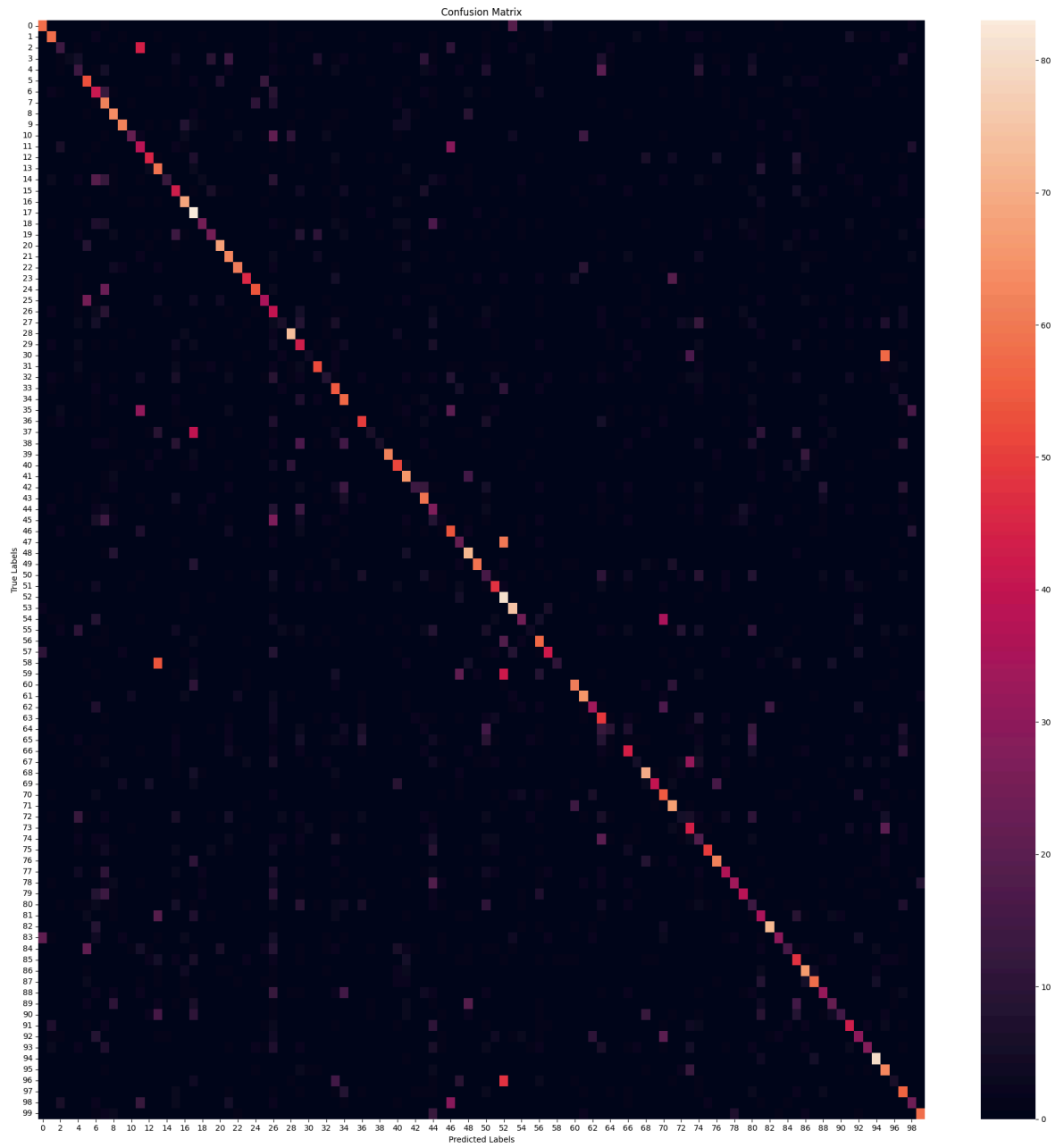
657/657 ————— **19s** 30ms/step - accuracy: 0.2534 - f1_score: 0.2485 - loss: 2.9597 - precision: 0.5813 - val_accuracy: 0.4429 - val_f1_score: 0.4212

- val_loss: 2.0689 - val_precision: 0.7927
 Epoch 9/25
657/657 ————— 0s 23ms/step - accuracy: 0.2507 - f1_score: 0.2442 -
 loss: 2.9597 - precision: 0.5684
 Epoch 9: val_loss did not improve from 2.06887
657/657 ————— 19s 28ms/step - accuracy: 0.2507 - f1_score: 0.2442
 - loss: 2.9597 - precision: 0.5684 - val_accuracy: 0.4331 - val_f1_score: 0.4113
 - val_loss: 2.1123 - val_precision: 0.7866
 Epoch 10/25
655/657 ————— 0s 23ms/step - accuracy: 0.2514 - f1_score: 0.2451 -
 loss: 2.9759 - precision: 0.5754
 Epoch 10: val_loss did not improve from 2.06887
657/657 ————— 19s 28ms/step - accuracy: 0.2514 - f1_score: 0.2451
 - loss: 2.9757 - precision: 0.5754 - val_accuracy: 0.4234 - val_f1_score: 0.4047
 - val_loss: 2.1572 - val_precision: 0.7830
 Epoch 11/25
656/657 ————— 0s 24ms/step - accuracy: 0.2515 - f1_score: 0.2466 -
 loss: 2.9793 - precision: 0.5631
 Epoch 11: val_loss did not improve from 2.06887
657/657 ————— 19s 28ms/step - accuracy: 0.2515 - f1_score: 0.2466
 - loss: 2.9792 - precision: 0.5631 - val_accuracy: 0.4259 - val_f1_score: 0.4048
 - val_loss: 2.1472 - val_precision: 0.8002
 Epoch 12/25
656/657 ————— 0s 24ms/step - accuracy: 0.2530 - f1_score: 0.2470 -
 loss: 2.9452 - precision: 0.5750
 Epoch 12: val_loss did not improve from 2.06887
657/657 ————— 19s 28ms/step - accuracy: 0.2530 - f1_score: 0.2470
 - loss: 2.9452 - precision: 0.5750 - val_accuracy: 0.4362 - val_f1_score: 0.4184
 - val_loss: 2.1381 - val_precision: 0.8148
 Epoch 13/25
657/657 ————— 0s 24ms/step - accuracy: 0.2576 - f1_score: 0.2514 -
 loss: 2.9452 - precision: 0.5733
 Epoch 13: val_loss did not improve from 2.06887
657/657 ————— 19s 28ms/step - accuracy: 0.2576 - f1_score: 0.2514
 - loss: 2.9451 - precision: 0.5733 - val_accuracy: 0.4408 - val_f1_score: 0.4195
 - val_loss: 2.1679 - val_precision: 0.8143



141/141  **11s** 40ms/step - accuracy: 0.4325 - f1_score: 0.4098
- loss: 2.0858 - precision: 0.7896
Test Accuracy: 44.48%
Test Loss: 2.0535354614257812
Test Precision: 79.45%
Test F1 Scores (Per Class): [64.088394 64.088394 20.740736 6.4516125 15.20467
6 49.76958
32.031246 41.750835 60.09389 75.60975 36.06557 31.45161
58.064507 43.54243 22.857141 37.391296 71.87499 51.234562
37.037033 34.722214 77.27273 58.604645 74.84662 61.33333
67.08074 47.61904 20.304565 10.909089 74.87179 33.46303
7.8431354 58.757053 13.67521 43.478256 47.3029 0.
54.64481 11.42857 11.999998 80. 52.577312 57.522118
25.454544 53.88127 19.852938 3.53982 44.081627 25.882345
62.33766 74.68353 15.094334 53.631275 44.999996 71.77033
34.04255 7.6190457 53.773582 54.54545 21.568624 0.
68.53932 69.148926 46.258495 36.16236 13.793102 0.
47.567562 9.090906 68.29268 59.259254 45.081963 63.507103
7.936504 38.766514 15.929201 66.66667 68.508286 45.283012
49.275356 49.350643 11.9266 34.31372 80. 45.45454
24.806196 40.85106 63.46153 67.44185 38.787876 33.58778
27.272722 47.77777 37.735844 40.287766 82.90154 44.839855
10.169488 43.57976 30.573242 72.5]
Average Test F1 Scores:42.33

282/282  **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.64	0.64	0.64	90
dolphin	0.64	0.64	0.64	90
otter	0.31	0.16	0.21	90
seal	1.00	0.03	0.06	90
whale	0.16	0.14	0.15	90
aquarium fish	0.42	0.59	0.49	90
flatfish	0.25	0.46	0.32	90
ray	0.30	0.69	0.42	90
shark	0.52	0.71	0.60	90
trout	0.84	0.69	0.76	90
orchids	0.69	0.24	0.36	90
poppies	0.24	0.42	0.31	90
roses	0.69	0.50	0.58	90
sunflowers	0.33	0.66	0.44	90
tulips	0.80	0.13	0.23	90
bottles	0.30	0.48	0.37	90
bowls	0.68	0.77	0.72	90
cans	0.35	0.92	0.51	90
cups	0.56	0.28	0.37	90
plates	0.47	0.28	0.35	90
apples	0.78	0.76	0.77	90
mushrooms	0.50	0.70	0.59	90
oranges	0.84	0.68	0.75	90
pears	0.77	0.51	0.61	90
sweet peppers	0.76	0.60	0.67	90
clock	0.61	0.39	0.48	90
computer keyboard	0.13	0.44	0.20	90
lamp	0.30	0.07	0.11	90
telephone	0.70	0.81	0.75	90
television	0.26	0.48	0.33	90
bed	0.33	0.04	0.08	90
chair	0.60	0.58	0.59	90
couch	0.30	0.09	0.14	90
table	0.34	0.61	0.44	90
wardrobe	0.38	0.63	0.47	90
bee	0.00	0.00	0.00	90
beetle	0.54	0.56	0.55	90
butterfly	0.40	0.07	0.11	90
caterpillar	0.60	0.07	0.12	90
cockroach	0.95	0.69	0.80	90
bear	0.49	0.57	0.53	90
leopard	0.48	0.72	0.58	90
lion	0.70	0.16	0.25	90
tiger	0.46	0.66	0.54	90
wolf	0.15	0.30	0.20	90
bridge	0.09	0.02	0.04	90
castle	0.35	0.60	0.44	90
house	0.27	0.24	0.26	90
road	0.51	0.80	0.62	90
skyscraper	0.87	0.66	0.75	90
cloud	0.13	0.18	0.15	90
forest	0.54	0.53	0.54	90
mountain	0.30	0.90	0.45	90
plain	0.63	0.83	0.72	90
sea	0.47	0.27	0.34	90
camel	0.27	0.04	0.08	90
cattle	0.47	0.63	0.54	90
chimpanzee	0.66	0.47	0.55	90

elephant	0.92	0.12	0.22	90
kangaroo	0.00	0.00	0.00	90
fox	0.69	0.68	0.69	90
porcupine	0.66	0.72	0.69	90
possum	0.60	0.38	0.46	90
raccoon	0.27	0.54	0.36	90
skunk	0.33	0.10	0.15	90
crab	0.00	0.00	0.00	90
lobster	0.46	0.49	0.48	90
snail	0.25	0.06	0.09	90
spider	0.61	0.78	0.68	90
worm	0.89	0.44	0.59	90
baby	0.36	0.61	0.45	90
boy	0.55	0.74	0.64	90
girl	0.14	0.06	0.08	90
man	0.32	0.49	0.39	90
woman	0.13	0.20	0.16	90
crocodile	0.83	0.56	0.67	90
dinosaur	0.68	0.69	0.69	90
lizard	0.52	0.40	0.45	90
snake	0.71	0.38	0.49	90
turtle	0.59	0.42	0.49	90
hamster	0.10	0.14	0.12	90
mouse	0.31	0.39	0.34	90
rabbit	0.80	0.80	0.80	90
shrew	0.71	0.33	0.45	90
squirrel	0.41	0.18	0.25	90
maple	0.33	0.53	0.41	90
oak	0.56	0.73	0.63	90
palm	0.71	0.64	0.67	90
pine	0.43	0.36	0.39	90
willow	0.54	0.24	0.34	90
bicycle	0.75	0.17	0.27	90
bus	0.48	0.48	0.48	90
motorcycle	0.43	0.33	0.38	90
pickup truck	0.57	0.31	0.40	90
train	0.78	0.89	0.83	90
lawn-mower	0.33	0.70	0.45	90
rocket	0.21	0.07	0.10	90
streetcar	0.34	0.62	0.44	90
tank	0.35	0.27	0.30	90
tractor	0.83	0.64	0.72	90
accuracy			0.44	9000
macro avg	0.49	0.44	0.42	9000
weighted avg	0.49	0.44	0.42	9000

```

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 2048)
dense_4 (Dense)	(None, 512)
dropout_2 (Dropout)	(None, 512)
dense_5 (Dense)	(None, 100)



Total params: 24,665,188 (94.09 MB)
Trainable params: 1,100,388 (4.20 MB)
Non-trainable params: 23,564,800 (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 27ms/step - accuracy: 0.2694 - f1_score: 0.2567
- loss: 2.8631 - precision: 0.5769
Epoch 1: val_loss improved from inf to 2.00090, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 57s 35ms/step - accuracy: 0.2694 - f1_score: 0.2567
- loss: 2.8630 - precision: 0.5770 - val_accuracy: 0.4692 - val_f1_score: 0.4514
- val_loss: 2.0009 - val_precision: 0.8332
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.2898 - f1_score: 0.2785
- loss: 2.7299 - precision: 0.6147
Epoch 2: val_loss improved from 2.00090 to 1.94934, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.2898 - f1_score: 0.2785
- loss: 2.7299 - precision: 0.6147 - val_accuracy: 0.4900 - val_f1_score: 0.4765
- val_loss: 1.9493 - val_precision: 0.8398
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3070 - f1_score: 0.2973
- loss: 2.6542 - precision: 0.6405
Epoch 3: val_loss improved from 1.94934 to 1.90862, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.3070 - f1_score: 0.2973
- loss: 2.6541 - precision: 0.6405 - val_accuracy: 0.5042 - val_f1_score: 0.4940
- val_loss: 1.9086 - val_precision: 0.8472
Epoch 4/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3155 - f1_score: 0.3086
- loss: 2.5951 - precision: 0.6527
Epoch 4: val_loss improved from 1.90862 to 1.87739, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.3155 - f1_score: 0.3086
- loss: 2.5951 - precision: 0.6527 - val_accuracy: 0.5122 - val_f1_score: 0.5048
- val_loss: 1.8774 - val_precision: 0.8527
Epoch 5/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.3259 - f1_score: 0.3202
- loss: 2.5423 - precision: 0.6654
Epoch 5: val_loss improved from 1.87739 to 1.85263, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.3260 - f1_score: 0.3202
- loss: 2.5423 - precision: 0.6654 - val_accuracy: 0.5147 - val_f1_score: 0.5076
- val_loss: 1.8526 - val_precision: 0.8555
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3331 - f1_score: 0.3279
- loss: 2.5122 - precision: 0.6737
Epoch 6: val_loss improved from 1.85263 to 1.83256, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.3331 - f1_score: 0.3280
- loss: 2.5122 - precision: 0.6737 - val_accuracy: 0.5203 - val_f1_score: 0.5138
- val_loss: 1.8326 - val_precision: 0.8543
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3416 - f1_score: 0.3367
- loss: 2.4756 - precision: 0.6920
Epoch 7: val_loss improved from 1.83256 to 1.81589, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.3416 - f1_score: 0.3367
- loss: 2.4756 - precision: 0.6920 - val_accuracy: 0.5233 - val_f1_score: 0.5168
- val_loss: 1.8159 - val_precision: 0.8534
Epoch 8/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3487 - f1_score: 0.3433
- loss: 2.4424 - precision: 0.6915
Epoch 8: val_loss improved from 1.81589 to 1.80046, saving model to best_model_fine_tune_v2.keras

ne_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.3487 - f1_score: 0.3433 - loss: 2.4424 - precision: 0.6915 - val_accuracy: 0.5272 - val_f1_score: 0.5209 - val_loss: 1.8005 - val_precision: 0.8515

Epoch 9/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.3527 - f1_score: 0.3473 - loss: 2.4193 - precision: 0.6973

Epoch 9: val_loss improved from 1.80046 to 1.78836, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.3527 - f1_score: 0.3473 - loss: 2.4193 - precision: 0.6973 - val_accuracy: 0.5289 - val_f1_score: 0.5227 - val_loss: 1.7884 - val_precision: 0.8529

Epoch 10/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.3587 - f1_score: 0.3535 - loss: 2.4022 - precision: 0.7025

Epoch 10: val_loss improved from 1.78836 to 1.77638, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.3587 - f1_score: 0.3535 - loss: 2.4022 - precision: 0.7025 - val_accuracy: 0.5328 - val_f1_score: 0.5272 - val_loss: 1.7764 - val_precision: 0.8518

Epoch 11/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.3615 - f1_score: 0.3573 - loss: 2.3795 - precision: 0.7089

Epoch 11: val_loss improved from 1.77638 to 1.76537, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.3615 - f1_score: 0.3573 - loss: 2.3795 - precision: 0.7089 - val_accuracy: 0.5346 - val_f1_score: 0.5290 - val_loss: 1.7654 - val_precision: 0.8533

Epoch 12/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.3728 - f1_score: 0.3682 - loss: 2.3367 - precision: 0.7205

Epoch 12: val_loss improved from 1.76537 to 1.75307, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.3728 - f1_score: 0.3682 - loss: 2.3367 - precision: 0.7205 - val_accuracy: 0.5368 - val_f1_score: 0.5315 - val_loss: 1.7531 - val_precision: 0.8522

Epoch 13/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.3715 - f1_score: 0.3672 - loss: 2.3368 - precision: 0.7184

Epoch 13: val_loss improved from 1.75307 to 1.74469, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.3715 - f1_score: 0.3672 - loss: 2.3367 - precision: 0.7184 - val_accuracy: 0.5384 - val_f1_score: 0.5332 - val_loss: 1.7447 - val_precision: 0.8514

Epoch 14/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.3769 - f1_score: 0.3726 - loss: 2.3081 - precision: 0.7256

Epoch 14: val_loss improved from 1.74469 to 1.73615, saving model to best_model_fine_tune_v2.keras

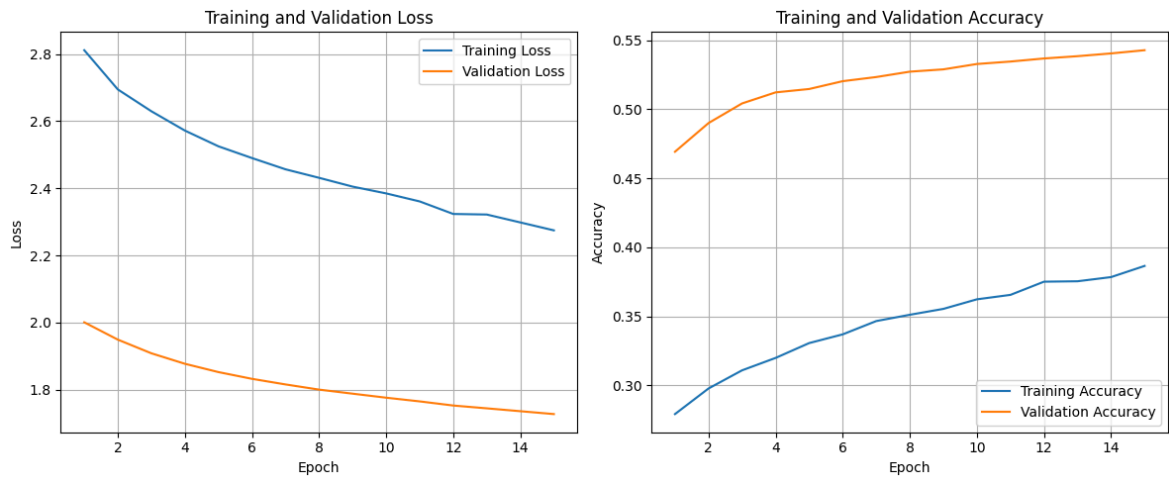
1314/1314 ————— 36s 27ms/step - accuracy: 0.3769 - f1_score: 0.3726 - loss: 2.3081 - precision: 0.7256 - val_accuracy: 0.5404 - val_f1_score: 0.5350 - val_loss: 1.7362 - val_precision: 0.8530

Epoch 15/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.3831 - f1_score: 0.3791 - loss: 2.2834 - precision: 0.7319

Epoch 15: val_loss improved from 1.73615 to 1.72762, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.3831 - f1_score: 0.3791 - loss: 2.2833 - precision: 0.7319 - val_accuracy: 0.5428 - val_f1_score: 0.5376 - val_loss: 1.7276 - val_precision: 0.8528



141/141 ————— **11s** 38ms/step - accuracy: 0.5311 - f1_score: 0.5211

- loss: 1.7391 - precision: 0.8380

Test Accuracy: 54.43%

Test Loss: 1.709373950958252

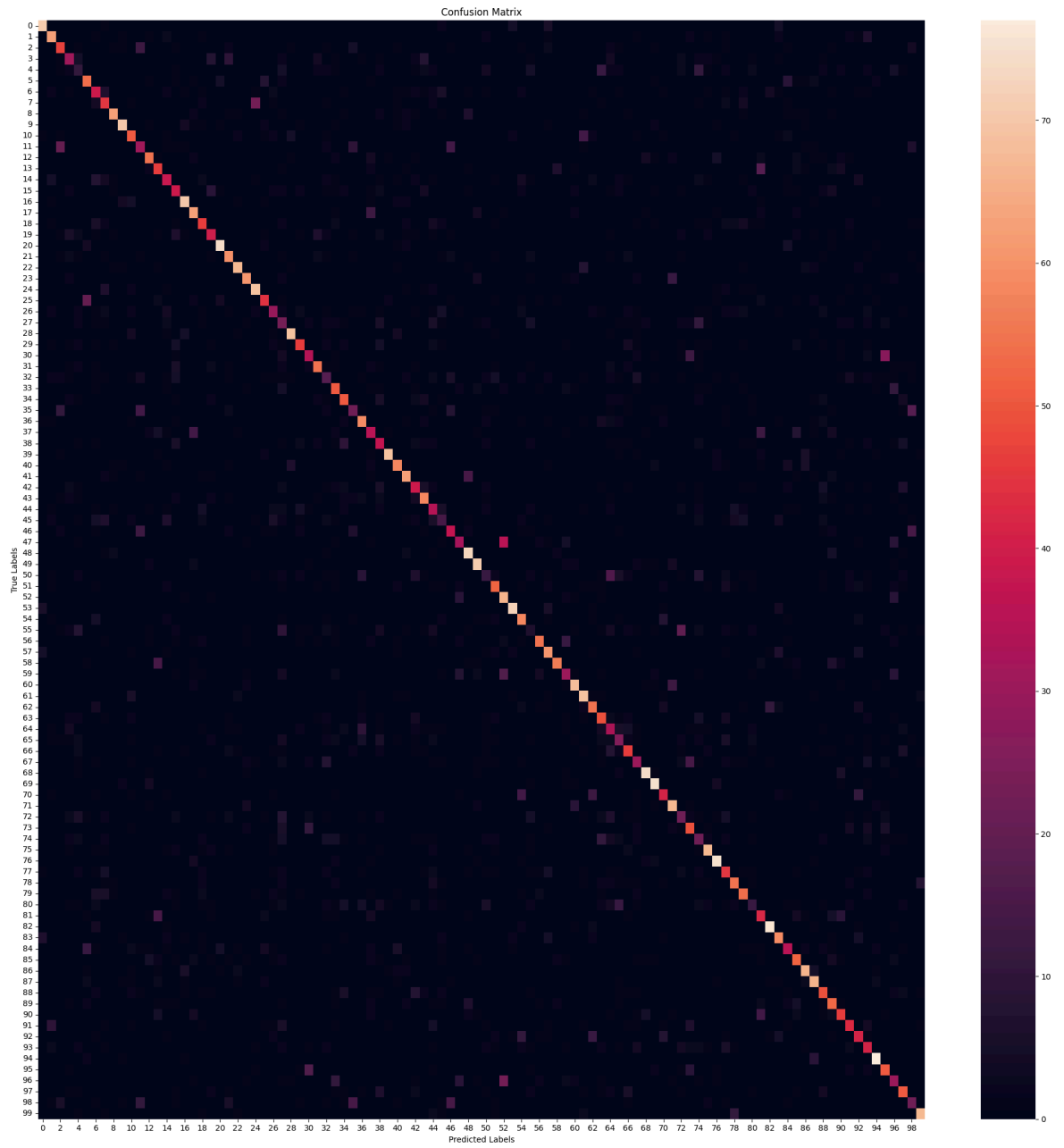
Test Precision: 85.64%

Test F1 Scores (Per Class): [78.02197 65.968575 50.267376 37.349392 14.814812 5
6.544495 40.816322

46.63212	75.90361	80.459755	56.66666	34.44444	64.70587	48.167538
44.44444	42.62294	76.08696	64.61538	52.272724	47.33727	81.767944
63.157898	77.01149	71.345024	71.87499	53.65853	37.41935	20.814476
78.40909	56.44171	41.212116	65.088745	21.301771	50.74626	55.13513
28.025469	61.13989	40.697666	36.999996	83.13252	64.80447	69.23077
42.162155	61.702126	34.343426	20.134222	41.807903	41.02564	70.1923
75.93583	16.783215	60.1156	55.696194	81.81818	56.862743	10.144923
67.90123	63.21243	69.99999	38.216557	73.513504	69.346725	58.510624
49.999992	32.195114	30.232555	50.549446	36.58536	81.31868	78.53403
50.306744	69.791664	26.3473	50.51546	23.157888	73.224045	79.787224
46.874992	61.797745	61.01694	18.320606	42.424236	82.16216	63.101597
43.636356	55.91397	71.73912	69.10995	53.551907	54.35897	53.48836
50.887566	47.12643	43.386234	83.69564	52.849735	32.78688	55.737698
27.160486	76.57142					

Average Test F1 Scores:53.97

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.77	0.79	0.78	90
dolphin	0.62	0.70	0.66	90
otter	0.48	0.52	0.50	90
seal	0.41	0.34	0.37	90
whale	0.15	0.12	0.14	90
aquarium fish	0.53	0.60	0.57	90
flatfish	0.37	0.43	0.40	90
ray	0.43	0.50	0.46	90
shark	0.82	0.70	0.75	90
trout	0.83	0.78	0.80	90
orchids	0.57	0.57	0.57	90
poppies	0.34	0.34	0.34	90
roses	0.69	0.61	0.65	90
sunflowers	0.46	0.51	0.48	90
tulips	0.47	0.42	0.44	90
bottles	0.42	0.43	0.43	90
bowls	0.74	0.78	0.76	90
cans	0.60	0.70	0.65	90
cups	0.53	0.51	0.52	90
plates	0.51	0.44	0.47	90
apples	0.81	0.82	0.82	90
mushrooms	0.60	0.67	0.63	90
oranges	0.80	0.74	0.77	90
pears	0.75	0.68	0.71	90
sweet peppers	0.68	0.77	0.72	90
clock	0.59	0.49	0.54	90
computer keyboard	0.45	0.32	0.37	90
lamp	0.18	0.26	0.21	90
telephone	0.80	0.77	0.78	90
television	0.63	0.51	0.56	90
bed	0.45	0.38	0.41	90
chair	0.70	0.61	0.65	90
couch	0.23	0.20	0.21	90
table	0.46	0.57	0.51	90
wardrobe	0.54	0.57	0.55	90
bee	0.33	0.24	0.28	90
beetle	0.57	0.66	0.61	90
butterfly	0.43	0.39	0.41	90
caterpillar	0.34	0.41	0.37	90
cockroach	0.91	0.77	0.83	90
bear	0.65	0.64	0.65	90
leopard	0.68	0.70	0.69	90
lion	0.41	0.43	0.42	90
tiger	0.59	0.64	0.62	90
wolf	0.31	0.38	0.34	90
bridge	0.25	0.17	0.20	90
castle	0.43	0.41	0.42	90
house	0.48	0.36	0.41	90
road	0.62	0.81	0.71	90
skyscraper	0.73	0.79	0.76	90
cloud	0.23	0.13	0.17	90
forest	0.63	0.58	0.60	90
mountain	0.45	0.73	0.56	90
plain	0.84	0.80	0.82	90
sea	0.51	0.64	0.57	90
camel	0.15	0.08	0.10	90
cattle	0.76	0.61	0.68	90
chimpanzee	0.59	0.68	0.63	90

elephant	0.80	0.62	0.70	90
kangaroo	0.45	0.33	0.38	90
fox	0.72	0.76	0.74	90
porcupine	0.63	0.77	0.69	90
possum	0.56	0.61	0.59	90
raccoon	0.46	0.54	0.50	90
skunk	0.29	0.37	0.32	90
crab	0.32	0.29	0.30	90
lobster	0.50	0.51	0.51	90
snail	0.41	0.33	0.37	90
spider	0.80	0.82	0.81	90
worm	0.74	0.83	0.79	90
baby	0.56	0.46	0.50	90
boy	0.66	0.74	0.70	90
girl	0.29	0.24	0.26	90
man	0.47	0.54	0.51	90
woman	0.22	0.24	0.23	90
crocodile	0.72	0.74	0.73	90
dinosaur	0.77	0.83	0.80	90
lizard	0.44	0.50	0.47	90
snake	0.62	0.61	0.62	90
turtle	0.62	0.60	0.61	90
hamster	0.29	0.13	0.18	90
mouse	0.39	0.47	0.42	90
rabbit	0.80	0.84	0.82	90
shrew	0.61	0.66	0.63	90
squirrel	0.48	0.40	0.44	90
maple	0.54	0.58	0.56	90
oak	0.70	0.73	0.72	90
palm	0.65	0.73	0.69	90
pine	0.53	0.54	0.54	90
willow	0.50	0.59	0.54	90
bicycle	0.56	0.51	0.53	90
bus	0.54	0.48	0.51	90
motorcycle	0.49	0.46	0.47	90
pickup truck	0.41	0.46	0.43	90
train	0.82	0.86	0.84	90
lawn-mower	0.50	0.57	0.53	90
rocket	0.32	0.33	0.33	90
streetcar	0.55	0.57	0.56	90
tank	0.31	0.24	0.27	90
tractor	0.79	0.74	0.77	90
accuracy			0.54	9000
macro avg	0.54	0.54	0.54	9000
weighted avg	0.54	0.54	0.54	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

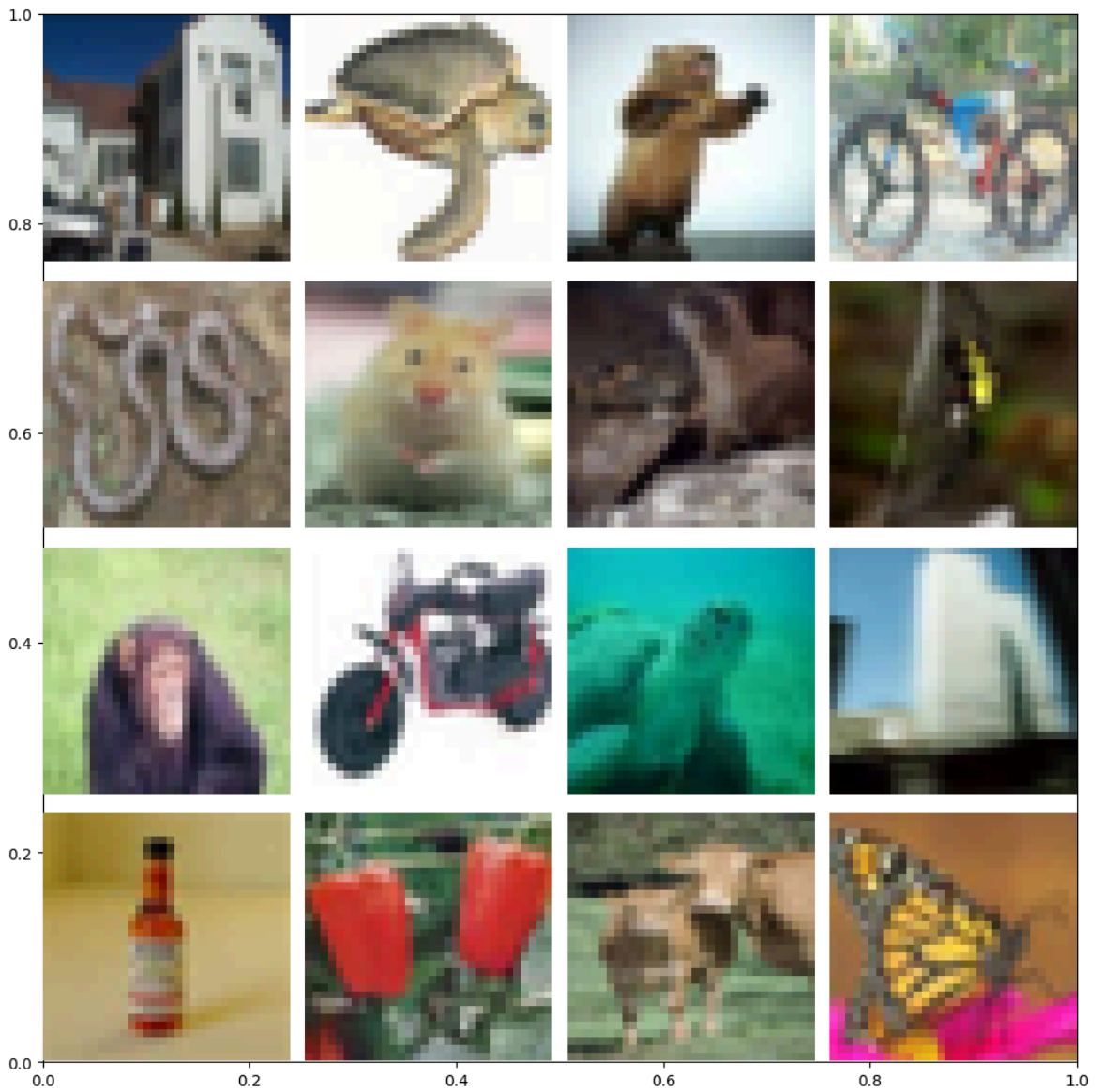
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

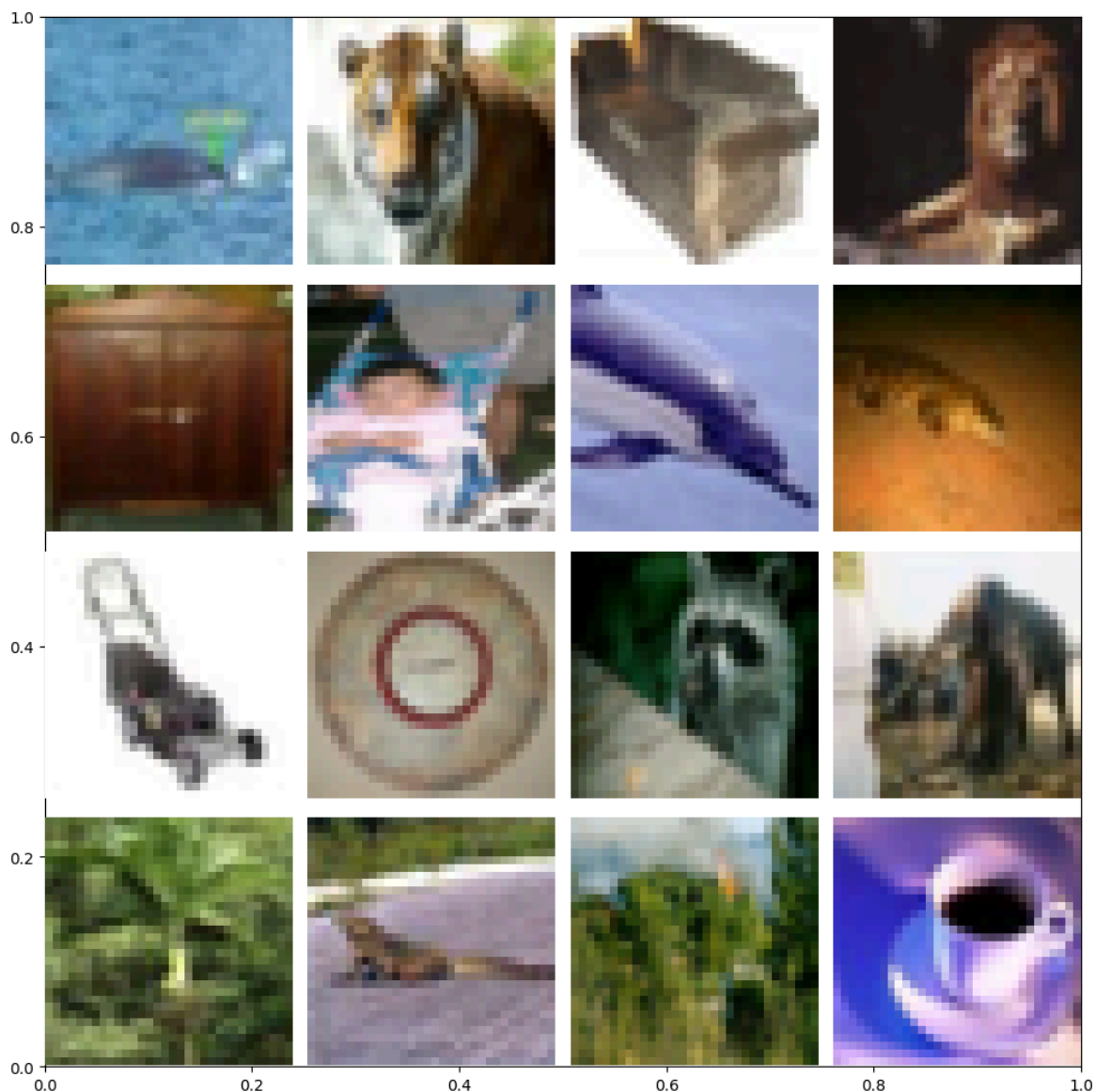
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[29870].shape: (1,)  
imgs[29870].shape: (32, 32, 3)
```

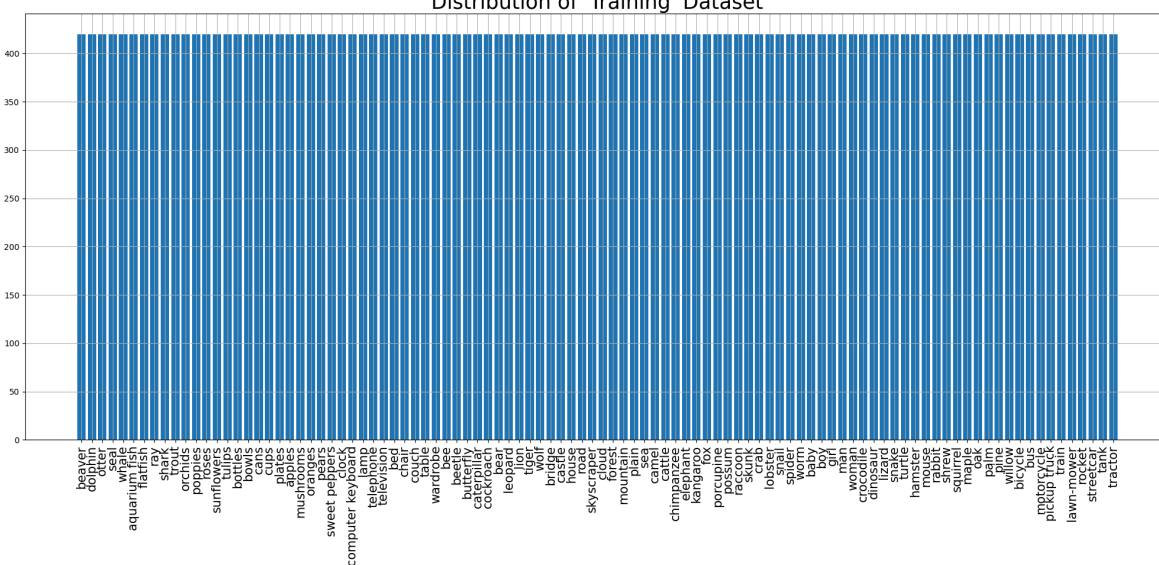


```
labels[22606].shape: (1,)  
imgs[22606].shape: (32, 32, 3)
```

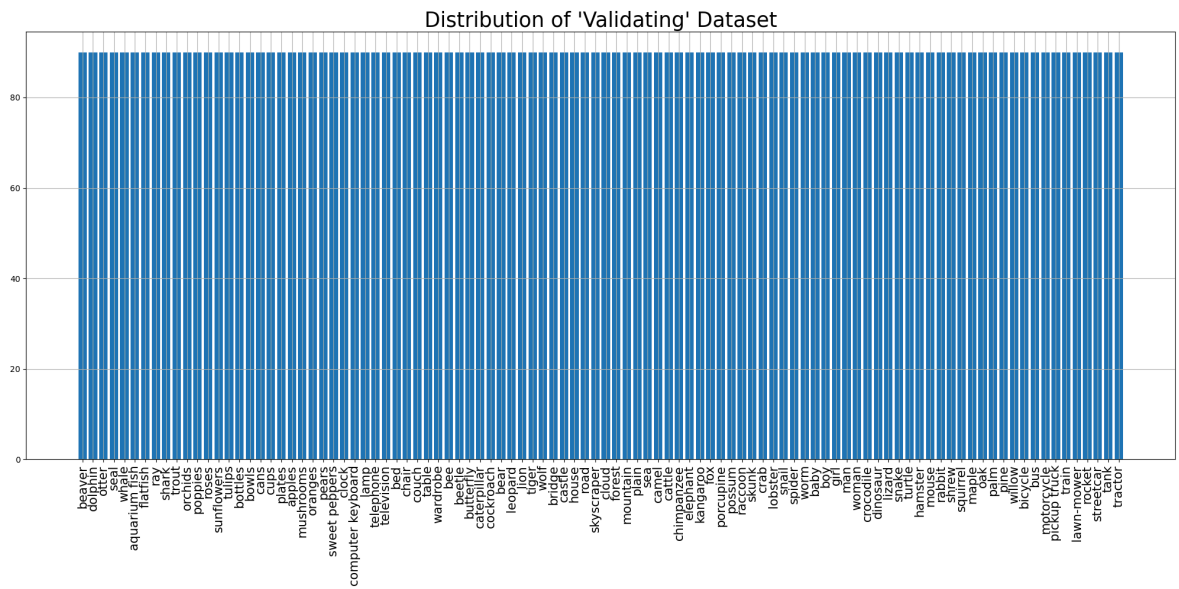


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

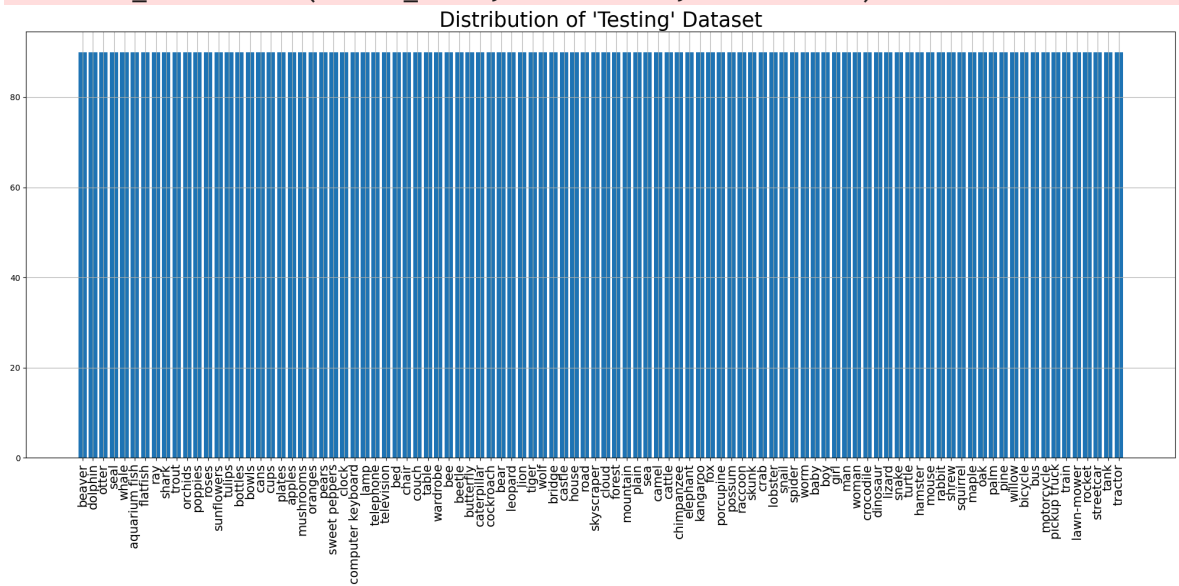
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25

657/657  **0s** 29ms/step - accuracy: 0.1723 - f1_score: 0.1627 - loss: 3.6943 - precision: 0.4927

Epoch 1: val_loss improved from inf to 2.27831, saving model to best_model_v2.keras as

657/657  **40s** 45ms/step - accuracy: 0.1724 - f1_score: 0.1627 - loss: 3.6937 - precision: 0.4928 - val_accuracy: 0.3947 - val_f1_score: 0.3738 - val_loss: 2.2783 - val_precision: 0.8055

Epoch 2/25

657/657  **0s** 24ms/step - accuracy: 0.2291 - f1_score: 0.2235 - loss: 3.0986 - precision: 0.5567

Epoch 2: val_loss improved from 2.27831 to 2.19225, saving model to best_model_v2.keras

657/657  **20s** 30ms/step - accuracy: 0.2291 - f1_score: 0.2235 - loss: 3.0986 - precision: 0.5567 - val_accuracy: 0.4187 - val_f1_score: 0.3984 - val_loss: 2.1923 - val_precision: 0.7982

Epoch 3/25

656/657  **0s** 24ms/step - accuracy: 0.2382 - f1_score: 0.2330 - loss: 3.0394 - precision: 0.5637

Epoch 3: val_loss improved from 2.19225 to 2.18379, saving model to best_model_v2.keras

657/657  **20s** 30ms/step - accuracy: 0.2382 - f1_score: 0.2330 - loss: 3.0394 - precision: 0.5637 - val_accuracy: 0.4302 - val_f1_score: 0.4038 - val_loss: 2.1838 - val_precision: 0.7976

Epoch 4/25

656/657  **0s** 24ms/step - accuracy: 0.2492 - f1_score: 0.2442 - loss: 2.9846 - precision: 0.5735

Epoch 4: val_loss improved from 2.18379 to 2.18322, saving model to best_model_v2.keras

657/657  **20s** 30ms/step - accuracy: 0.2492 - f1_score: 0.2442 - loss: 2.9846 - precision: 0.5735 - val_accuracy: 0.4214 - val_f1_score: 0.3919 - val_loss: 2.1832 - val_precision: 0.7876

Epoch 5/25

657/657  **0s** 24ms/step - accuracy: 0.2444 - f1_score: 0.2387 - loss: 2.9933 - precision: 0.5715

Epoch 5: val_loss improved from 2.18322 to 2.13875, saving model to best_model_v2.keras

657/657  **20s** 30ms/step - accuracy: 0.2445 - f1_score: 0.2387 - loss: 2.9932 - precision: 0.5715 - val_accuracy: 0.4310 - val_f1_score: 0.4037 - val_loss: 2.1387 - val_precision: 0.8095

Epoch 6/25

656/657  **0s** 24ms/step - accuracy: 0.2479 - f1_score: 0.2422 - loss: 2.9871 - precision: 0.5774

Epoch 6: val_loss did not improve from 2.13875

657/657  **19s** 29ms/step - accuracy: 0.2479 - f1_score: 0.2422 - loss: 2.9870 - precision: 0.5774 - val_accuracy: 0.4147 - val_f1_score: 0.3903 - val_loss: 2.1541 - val_precision: 0.7652

Epoch 7/25

657/657  **0s** 24ms/step - accuracy: 0.2462 - f1_score: 0.2406 - loss: 3.0022 - precision: 0.5583

Epoch 7: val_loss improved from 2.13875 to 2.11894, saving model to best_model_v2.keras

657/657  **20s** 30ms/step - accuracy: 0.2462 - f1_score: 0.2406 - loss: 3.0022 - precision: 0.5583 - val_accuracy: 0.4271 - val_f1_score: 0.3993 - val_loss: 2.1189 - val_precision: 0.7483

Epoch 8/25

657/657  **0s** 24ms/step - accuracy: 0.2492 - f1_score: 0.2431 - loss: 2.9900 - precision: 0.5560

Epoch 8: val_loss did not improve from 2.11894

657/657  **19s** 29ms/step - accuracy: 0.2492 - f1_score: 0.2431

- loss: 2.9899 - precision: 0.5560 - val_accuracy: 0.4316 - val_f1_score: 0.4008
- val_loss: 2.1888 - val_precision: 0.7969
Epoch 9/25
655/657 ————— 0s 24ms/step - accuracy: 0.2512 - f1_score: 0.2452 -
loss: 2.9741 - precision: 0.5632
Epoch 9: val_loss did not improve from 2.11894
657/657 ————— 19s 29ms/step - accuracy: 0.2513 - f1_score: 0.2452
- loss: 2.9740 - precision: 0.5633 - val_accuracy: 0.4311 - val_f1_score: 0.4093
- val_loss: 2.1253 - val_precision: 0.8079
Epoch 10/25
657/657 ————— 0s 24ms/step - accuracy: 0.2542 - f1_score: 0.2488 -
loss: 2.9656 - precision: 0.5658
Epoch 10: val_loss improved from 2.11894 to 2.11567, saving model to best_model_v
2.keras
657/657 ————— 20s 30ms/step - accuracy: 0.2542 - f1_score: 0.2488
- loss: 2.9655 - precision: 0.5658 - val_accuracy: 0.4438 - val_f1_score: 0.4189
- val_loss: 2.1157 - val_precision: 0.8042
Epoch 11/25
655/657 ————— 0s 24ms/step - accuracy: 0.2499 - f1_score: 0.2440 -
loss: 2.9754 - precision: 0.5569
Epoch 11: val_loss improved from 2.11567 to 2.10599, saving model to best_model_v
2.keras
657/657 ————— 20s 30ms/step - accuracy: 0.2499 - f1_score: 0.2441
- loss: 2.9753 - precision: 0.5570 - val_accuracy: 0.4341 - val_f1_score: 0.4076
- val_loss: 2.1060 - val_precision: 0.8025
Epoch 12/25
655/657 ————— 0s 24ms/step - accuracy: 0.2517 - f1_score: 0.2457 -
loss: 2.9793 - precision: 0.5683
Epoch 12: val_loss did not improve from 2.10599
657/657 ————— 19s 29ms/step - accuracy: 0.2517 - f1_score: 0.2457
- loss: 2.9791 - precision: 0.5682 - val_accuracy: 0.4288 - val_f1_score: 0.4016
- val_loss: 2.1606 - val_precision: 0.8053
Epoch 13/25
657/657 ————— 0s 24ms/step - accuracy: 0.2528 - f1_score: 0.2458 -
loss: 2.9588 - precision: 0.5684
Epoch 13: val_loss improved from 2.10599 to 2.05439, saving model to best_model_v
2.keras
657/657 ————— 20s 30ms/step - accuracy: 0.2528 - f1_score: 0.2458
- loss: 2.9588 - precision: 0.5684 - val_accuracy: 0.4396 - val_f1_score: 0.4174
- val_loss: 2.0544 - val_precision: 0.7875
Epoch 14/25
655/657 ————— 0s 24ms/step - accuracy: 0.2558 - f1_score: 0.2486 -
loss: 2.9522 - precision: 0.5725
Epoch 14: val_loss did not improve from 2.05439
657/657 ————— 19s 29ms/step - accuracy: 0.2558 - f1_score: 0.2487
- loss: 2.9521 - precision: 0.5724 - val_accuracy: 0.4317 - val_f1_score: 0.4015
- val_loss: 2.1353 - val_precision: 0.8053
Epoch 15/25
657/657 ————— 0s 24ms/step - accuracy: 0.2531 - f1_score: 0.2473 -
loss: 2.9429 - precision: 0.5674
Epoch 15: val_loss did not improve from 2.05439
657/657 ————— 19s 29ms/step - accuracy: 0.2531 - f1_score: 0.2473
- loss: 2.9429 - precision: 0.5674 - val_accuracy: 0.4401 - val_f1_score: 0.4167
- val_loss: 2.1315 - val_precision: 0.8241
Epoch 16/25
655/657 ————— 0s 24ms/step - accuracy: 0.2587 - f1_score: 0.2527 -
loss: 2.9356 - precision: 0.5648
Epoch 16: val_loss did not improve from 2.05439
657/657 ————— 19s 29ms/step - accuracy: 0.2588 - f1_score: 0.2527
- loss: 2.9355 - precision: 0.5649 - val_accuracy: 0.4386 - val_f1_score: 0.4180

- val_loss: 2.1643 - val_precision: 0.8170

Epoch 17/25

656/657 ————— 0s 24ms/step - accuracy: 0.2612 - f1_score: 0.2547 -
loss: 2.9329 - precision: 0.5895

Epoch 17: val_loss did not improve from 2.05439

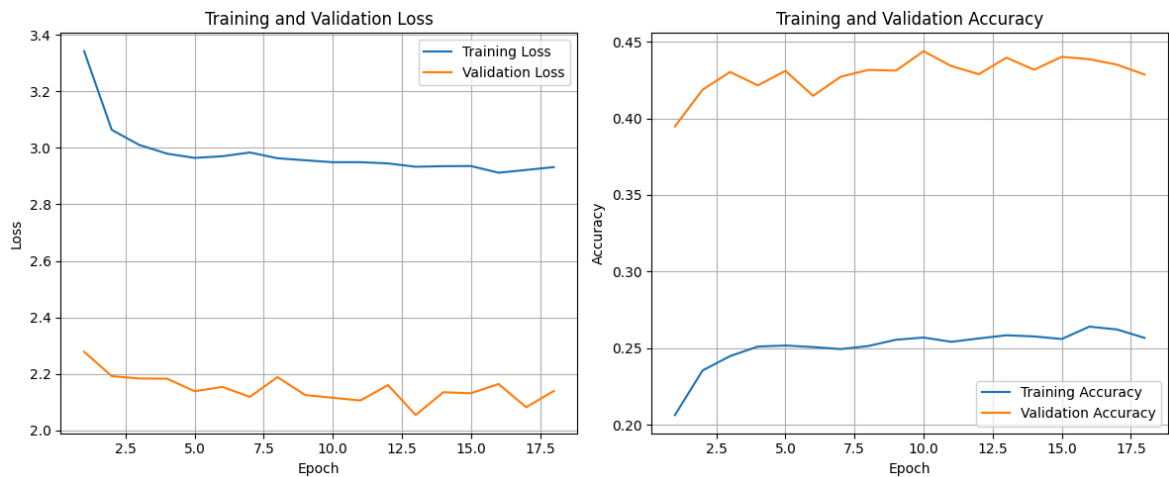
657/657 ————— 19s 29ms/step - accuracy: 0.2612 - f1_score: 0.2547
- loss: 2.9328 - precision: 0.5895 - val_accuracy: 0.4350 - val_f1_score: 0.4177
- val_loss: 2.0823 - val_precision: 0.8157

Epoch 18/25

656/657 ————— 0s 24ms/step - accuracy: 0.2513 - f1_score: 0.2463 -
loss: 2.9652 - precision: 0.5649

Epoch 18: val_loss did not improve from 2.05439

657/657 ————— 19s 28ms/step - accuracy: 0.2513 - f1_score: 0.2463
- loss: 2.9651 - precision: 0.5649 - val_accuracy: 0.4286 - val_f1_score: 0.4014
- val_loss: 2.1395 - val_precision: 0.7841



141/141 ————— 12s 38ms/step - accuracy: 0.4309 - f1_score: 0.4084
- loss: 2.0882 - precision: 0.7825

Test Accuracy: 44.07%

Test Loss: 2.0559418201446533

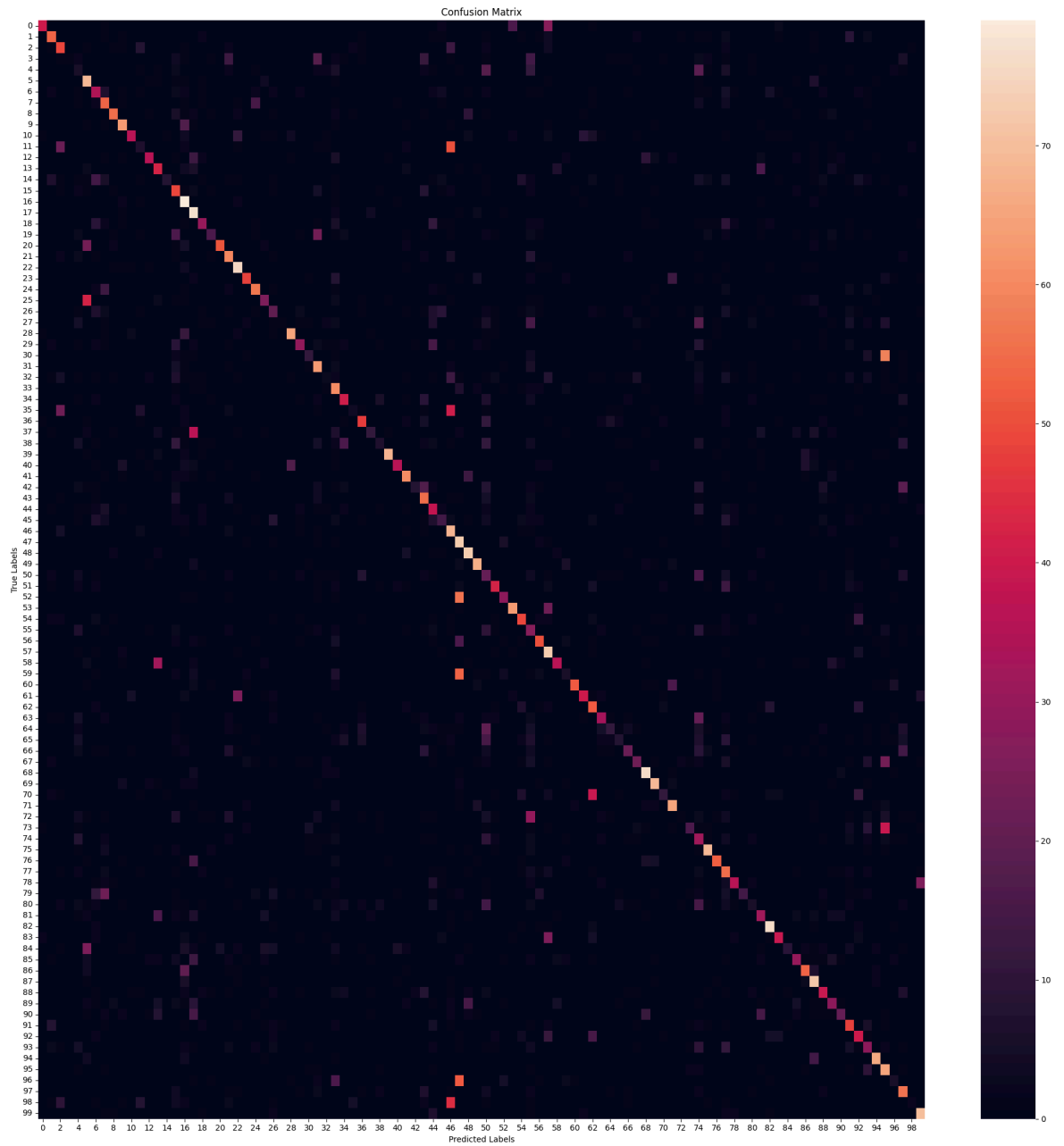
Test Precision: 78.51%

Test F1 Scores (Per Class): [58.82352 59.340656 44.954124 2.197802 6.55737
2 47.91666

32.22748	45.569614	68.322975	68.47825	51.798557	10.447758
52.857136	39.62264	15.999998	36.029408	51.298695	47.239258
39.999996	26.666664	69.86301	54.298634	67.26457	62.745094
63.687145	34.96503	24.691353	2.1739125	72.222206	41.726616
18.803413	50.806446	5.405403	41.78082	46.857132	7.8431354
54.02298	19.819817	12.068964	83.95061	52.173912	66.66666
15.686272	45.267487	32.90043	20.28985	37.883003	38.726784
67.90697	72.82608	14.237283	53.947365	40.287766	71.99999
55.05617	18.181814	57.471264	48.993282	49.999992	9.836062
71.23287	55.555546	44.255314	44.897953	18.803413	13.5593195
32.835815	33.57664	72.38095	74.59459	21.05263	67.01031
2.083332	27.826082	19.047615	74.999985	68.83116	36.66666
49.999992	26.923075	9.836062	36.158188	76.76767	53.06122
17.30769	41.095886	62.7907	62.882088	45.0867	34.14634
33.33333	45.497627	41.450768	29.47368	78.1065	40.498436
10.169488	45.52845	4.1666656	69.30692		

Average Test F1 Scores:42.04

282/282 ————— 11s 28ms/step



	precision	recall	f1-score	support
beaver	0.87	0.44	0.59	90
dolphin	0.59	0.60	0.59	90
otter	0.38	0.54	0.45	90
seal	1.00	0.01	0.02	90
whale	0.06	0.07	0.07	90
aquarium fish	0.35	0.77	0.48	90
flatfish	0.28	0.38	0.32	90
ray	0.37	0.60	0.46	90
shark	0.77	0.61	0.68	90
trout	0.67	0.70	0.68	90
orchids	0.73	0.40	0.52	90
poppies	0.16	0.08	0.10	90
roses	0.74	0.41	0.53	90
sunflowers	0.34	0.47	0.40	90
tulips	0.80	0.09	0.16	90
bottles	0.27	0.54	0.36	90
bowls	0.36	0.88	0.51	90
cans	0.33	0.86	0.47	90
cups	0.50	0.33	0.40	90
plates	0.53	0.18	0.27	90
apples	0.91	0.57	0.70	90
mushrooms	0.46	0.67	0.54	90
oranges	0.56	0.83	0.67	90
pears	0.76	0.53	0.63	90
sweet peppers	0.64	0.63	0.64	90
clock	0.47	0.28	0.35	90
computer keyboard	0.28	0.22	0.25	90
lamp	0.50	0.01	0.02	90
telephone	0.72	0.72	0.72	90
television	0.59	0.32	0.42	90
bed	0.41	0.12	0.19	90
chair	0.40	0.70	0.51	90
couch	0.14	0.03	0.05	90
table	0.30	0.68	0.42	90
wardrobe	0.48	0.46	0.47	90
bee	0.33	0.04	0.08	90
beetle	0.56	0.52	0.54	90
butterfly	0.52	0.12	0.20	90
caterpillar	0.27	0.08	0.12	90
cockroach	0.94	0.76	0.84	90
bear	0.75	0.40	0.52	90
leopard	0.66	0.68	0.67	90
lion	0.67	0.09	0.16	90
tiger	0.36	0.61	0.45	90
wolf	0.27	0.42	0.33	90
bridge	0.29	0.16	0.20	90
castle	0.25	0.76	0.38	90
house	0.25	0.81	0.39	90
road	0.58	0.81	0.68	90
skyscraper	0.71	0.74	0.73	90
cloud	0.10	0.23	0.14	90
forest	0.67	0.47	0.55	90
mountain	0.57	0.31	0.40	90
plain	0.74	0.70	0.72	90
sea	0.56	0.54	0.55	90
camel	0.13	0.30	0.18	90
cattle	0.60	0.56	0.57	90
chimpanzee	0.35	0.81	0.49	90

elephant	0.67	0.40	0.50	90
kangaroo	0.19	0.07	0.10	90
fox	0.93	0.58	0.71	90
porcupine	0.74	0.44	0.56	90
possum	0.36	0.58	0.44	90
raccoon	0.58	0.37	0.45	90
skunk	0.41	0.12	0.19	90
crab	0.29	0.09	0.14	90
lobster	0.50	0.24	0.33	90
snail	0.49	0.26	0.34	90
spider	0.63	0.84	0.72	90
worm	0.73	0.77	0.75	90
baby	0.48	0.12	0.19	90
boy	0.62	0.72	0.67	90
girl	0.14	0.01	0.02	90
man	0.64	0.18	0.28	90
woman	0.13	0.36	0.19	90
crocodile	0.73	0.77	0.75	90
dinosaur	0.83	0.59	0.69	90
lizard	0.26	0.61	0.37	90
snake	0.67	0.40	0.50	90
turtle	1.00	0.16	0.27	90
hamster	0.19	0.07	0.10	90
mouse	0.37	0.36	0.36	90
rabbit	0.70	0.84	0.77	90
shrew	0.68	0.43	0.53	90
squirrel	0.64	0.10	0.17	90
maple	0.54	0.33	0.41	90
oak	0.66	0.60	0.63	90
palm	0.51	0.80	0.63	90
pine	0.48	0.43	0.45	90
willow	0.38	0.31	0.34	90
bicycle	0.58	0.23	0.33	90
bus	0.40	0.53	0.45	90
motorcycle	0.39	0.44	0.42	90
pickup truck	0.28	0.31	0.29	90
train	0.84	0.73	0.78	90
lawn-mower	0.28	0.72	0.40	90
rocket	0.21	0.07	0.10	90
streetcar	0.36	0.62	0.46	90
tank	0.33	0.02	0.04	90
tractor	0.62	0.78	0.69	90
accuracy			0.44	9000
macro avg	0.50	0.44	0.42	9000
weighted avg	0.50	0.44	0.42	9000

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)



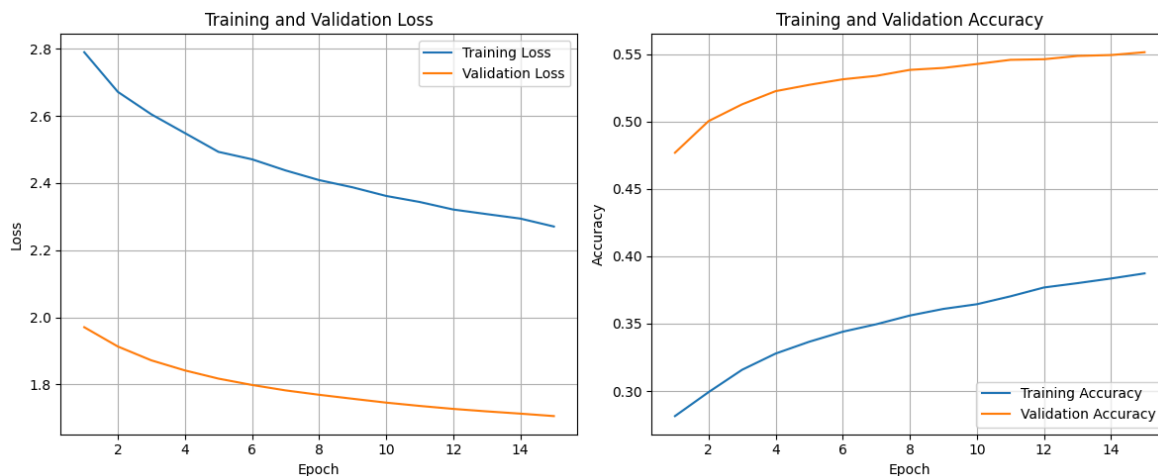
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 26ms/step - accuracy: 0.2722 - f1_score: 0.2587
- loss: 2.8456 - precision: 0.5697
Epoch 1: val_loss improved from inf to 1.97060, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.2722 - f1_score: 0.2588 - loss: 2.8455 - precision: 0.5697 - val_accuracy: 0.4768 - val_f1_score: 0.4602 - val_loss: 1.9706 - val_precision: 0.8371
Epoch 2/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.2941 - f1_score: 0.2830 - loss: 2.7073 - precision: 0.6172
Epoch 2: val_loss improved from 1.97060 to 1.91324, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.2942 - f1_score: 0.2830 - loss: 2.7072 - precision: 0.6172 - val_accuracy: 0.5002 - val_f1_score: 0.4873 - val_loss: 1.9132 - val_precision: 0.8552
Epoch 3/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3104 - f1_score: 0.3012 - loss: 2.6390 - precision: 0.6443
Epoch 3: val_loss improved from 1.91324 to 1.87187, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3104 - f1_score: 0.3012 - loss: 2.6390 - precision: 0.6443 - val_accuracy: 0.5127 - val_f1_score: 0.5025 - val_loss: 1.8719 - val_precision: 0.8618
Epoch 4/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3247 - f1_score: 0.3174 - loss: 2.5659 - precision: 0.6635
Epoch 4: val_loss improved from 1.87187 to 1.84195, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3247 - f1_score: 0.3174 - loss: 2.5659 - precision: 0.6635 - val_accuracy: 0.5224 - val_f1_score: 0.5136 - val_loss: 1.8419 - val_precision: 0.8602
Epoch 5/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3336 - f1_score: 0.3267 - loss: 2.5119 - precision: 0.6717
Epoch 5: val_loss improved from 1.84195 to 1.81751, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3336 - f1_score: 0.3267 - loss: 2.5119 - precision: 0.6717 - val_accuracy: 0.5271 - val_f1_score: 0.5194 - val_loss: 1.8175 - val_precision: 0.8596
Epoch 6/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3406 - f1_score: 0.3345 - loss: 2.4913 - precision: 0.6809
Epoch 6: val_loss improved from 1.81751 to 1.79861, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3406 - f1_score: 0.3345 - loss: 2.4913 - precision: 0.6809 - val_accuracy: 0.5312 - val_f1_score: 0.5235 - val_loss: 1.7986 - val_precision: 0.8581
Epoch 7/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.3440 - f1_score: 0.3373 - loss: 2.4607 - precision: 0.6804
Epoch 7: val_loss improved from 1.79861 to 1.78242, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3440 - f1_score: 0.3373 - loss: 2.4606 - precision: 0.6804 - val_accuracy: 0.5338 - val_f1_score: 0.5268 - val_loss: 1.7824 - val_precision: 0.8561
Epoch 8/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3516 - f1_score: 0.3455 - loss: 2.4303 - precision: 0.6855
Epoch 8: val_loss improved from 1.78242 to 1.76936, saving model to best_model_fine_tune_v2.keras

```
ne_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3516 - f1_score: 0.345
5 - loss: 2.4303 - precision: 0.6855 - val_accuracy: 0.5382 - val_f1_score: 0.531
8 - val_loss: 1.7694 - val_precision: 0.8564
Epoch 9/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.3570 - f1_score: 0.3509
- loss: 2.4091 - precision: 0.6972
Epoch 9: val_loss improved from 1.76936 to 1.75749, saving model to best_model_fi
ne_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3570 - f1_score: 0.350
9 - loss: 2.4091 - precision: 0.6972 - val_accuracy: 0.5397 - val_f1_score: 0.533
8 - val_loss: 1.7575 - val_precision: 0.8563
Epoch 10/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3604 - f1_score: 0.3552
- loss: 2.3765 - precision: 0.6992
Epoch 10: val_loss improved from 1.75749 to 1.74598, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3604 - f1_score: 0.355
2 - loss: 2.3765 - precision: 0.6992 - val_accuracy: 0.5426 - val_f1_score: 0.537
2 - val_loss: 1.7460 - val_precision: 0.8557
Epoch 11/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3673 - f1_score: 0.3621
- loss: 2.3618 - precision: 0.7070
Epoch 11: val_loss improved from 1.74598 to 1.73616, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3673 - f1_score: 0.362
1 - loss: 2.3618 - precision: 0.7070 - val_accuracy: 0.5457 - val_f1_score: 0.540
3 - val_loss: 1.7362 - val_precision: 0.8550
Epoch 12/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3758 - f1_score: 0.3709
- loss: 2.3326 - precision: 0.7112
Epoch 12: val_loss improved from 1.73616 to 1.72729, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3758 - f1_score: 0.370
9 - loss: 2.3326 - precision: 0.7112 - val_accuracy: 0.5461 - val_f1_score: 0.541
0 - val_loss: 1.7273 - val_precision: 0.8543
Epoch 13/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3761 - f1_score: 0.3704
- loss: 2.3236 - precision: 0.7146
Epoch 13: val_loss improved from 1.72729 to 1.71990, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3761 - f1_score: 0.370
4 - loss: 2.3236 - precision: 0.7146 - val_accuracy: 0.5486 - val_f1_score: 0.544
0 - val_loss: 1.7199 - val_precision: 0.8529
Epoch 14/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3819 - f1_score: 0.3766
- loss: 2.3060 - precision: 0.7216
Epoch 14: val_loss improved from 1.71990 to 1.71319, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3819 - f1_score: 0.376
6 - loss: 2.3060 - precision: 0.7216 - val_accuracy: 0.5492 - val_f1_score: 0.544
7 - val_loss: 1.7132 - val_precision: 0.8515
Epoch 15/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.3853 - f1_score: 0.3798
- loss: 2.2811 - precision: 0.7225
Epoch 15: val_loss improved from 1.71319 to 1.70591, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3853 - f1_score: 0.379
8 - loss: 2.2811 - precision: 0.7225 - val_accuracy: 0.5513 - val_f1_score: 0.546
9 - val_loss: 1.7059 - val_precision: 0.8523
```



141/141 ————— **11s** 38ms/step - accuracy: 0.5333 - f1_score: 0.5235

- loss: 1.7176 - precision: 0.8483

Test Accuracy: 54.56%

Test Loss: 1.691511631011963

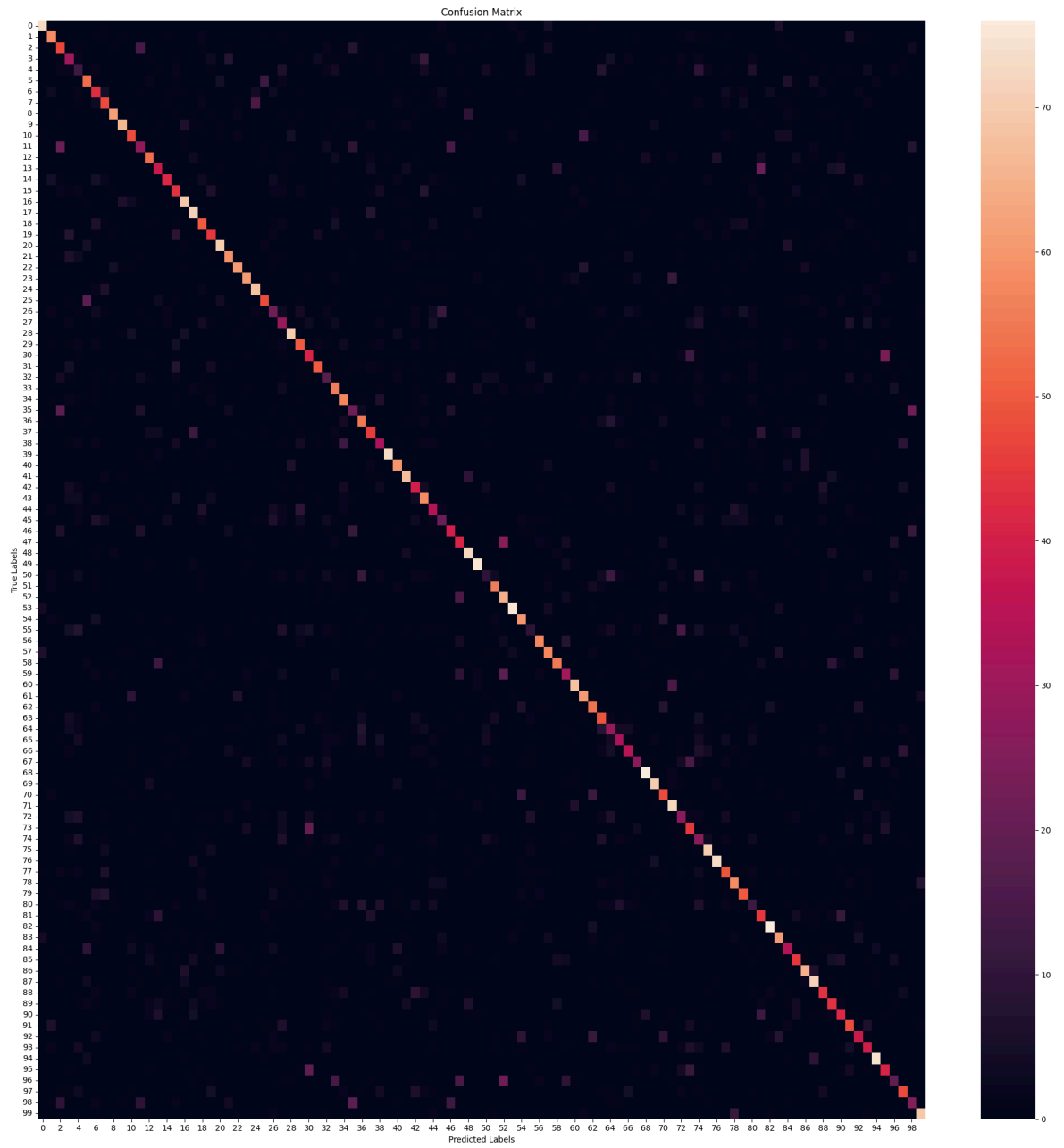
Test Precision: 86.01%

Test F1 Scores (Per Class): [80.66298 63.387978 43.92523 32.978718 13.953484 5
5.789463 41.545887

52.747242 74.251495 76.74419 55.49132 33.532932 60.227264 44.705875
54.54545 46.808506 72.251305 70.64676 51.813465 49.999992 75.268814
65.92178 71.345024 71.345024 73.91304 57.48503 27.160486 29.591831
79.09604 53.191483 41.62436 64.51612 21.118008 51.851845 57.575756
26.993858 55.83756 52.873566 36.571426 87.95179 63.87434 73.62636
43.243237 57.71144 38.63636 27.09677 45.45454 45.81005 72.3618
81.52173 13.636359 62.92134 57.522118 85.22727 59.999996 14.492748
68.67469 61.956512 65.454544 38.509315 76.40449 65.217384 56.544495
51.578945 31.182789 40.764324 43.478252 36.734684 83.060104 72.916664
52.173912 70.93597 32.72727 46.15384 23.636358 74.8663 81.11111
50.7772 62.702698 58.139526 16.774189 45.45454 84.44444 63.87434
38.857136 51.72413 69.89246 72.82051 50.28571 49.999992 46.92737
50.793648 44.18604 42.62294 81.31868 45.81005 25.165558 54.545456
29.940113 77.52808]

Average Test F1 Scores:54.09

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.80	0.81	0.81	90
dolphin	0.62	0.64	0.63	90
otter	0.38	0.52	0.44	90
seal	0.32	0.34	0.33	90
whale	0.15	0.13	0.14	90
aquarium fish	0.53	0.59	0.56	90
flatfish	0.37	0.48	0.42	90
ray	0.52	0.53	0.53	90
shark	0.81	0.69	0.74	90
trout	0.81	0.73	0.77	90
orchids	0.58	0.53	0.55	90
poppies	0.36	0.31	0.34	90
roses	0.62	0.59	0.60	90
sunflowers	0.47	0.42	0.45	90
tulips	0.66	0.47	0.55	90
bottles	0.45	0.49	0.47	90
bowls	0.68	0.77	0.72	90
cans	0.64	0.79	0.71	90
cups	0.49	0.56	0.52	90
plates	0.50	0.50	0.50	90
apples	0.73	0.78	0.75	90
mushrooms	0.66	0.66	0.66	90
oranges	0.76	0.68	0.72	90
pears	0.75	0.68	0.71	90
sweet peppers	0.72	0.76	0.74	90
clock	0.62	0.53	0.57	90
computer keyboard	0.31	0.24	0.27	90
lamp	0.27	0.32	0.30	90
telephone	0.80	0.78	0.79	90
television	0.51	0.56	0.53	90
bed	0.38	0.46	0.42	90
chair	0.77	0.56	0.65	90
couch	0.24	0.19	0.21	90
table	0.44	0.62	0.52	90
wardrobe	0.53	0.63	0.58	90
bee	0.30	0.24	0.27	90
beetle	0.51	0.61	0.56	90
butterfly	0.55	0.51	0.53	90
caterpillar	0.38	0.36	0.37	90
cockroach	0.96	0.81	0.88	90
bear	0.60	0.67	0.63	90
leopard	0.73	0.74	0.74	90
lion	0.42	0.44	0.43	90
tiger	0.52	0.64	0.58	90
wolf	0.40	0.38	0.39	90
bridge	0.32	0.23	0.27	90
castle	0.47	0.44	0.45	90
house	0.46	0.46	0.46	90
road	0.66	0.80	0.72	90
skyscraper	0.80	0.83	0.82	90
cloud	0.21	0.10	0.14	90
forest	0.64	0.62	0.63	90
mountain	0.48	0.72	0.58	90
plain	0.87	0.83	0.85	90
sea	0.55	0.67	0.60	90
camel	0.21	0.11	0.14	90
cattle	0.75	0.63	0.69	90
chimpanzee	0.61	0.63	0.62	90

elephant	0.72	0.61	0.66	90
kangaroo	0.44	0.34	0.39	90
fox	0.77	0.76	0.76	90
porcupine	0.64	0.68	0.66	90
possum	0.53	0.60	0.57	90
raccoon	0.49	0.54	0.52	90
skunk	0.30	0.32	0.31	90
crab	0.48	0.36	0.41	90
lobster	0.49	0.39	0.43	90
snail	0.47	0.30	0.37	90
spider	0.82	0.84	0.83	90
worm	0.69	0.78	0.73	90
baby	0.51	0.53	0.52	90
boy	0.64	0.80	0.71	90
girl	0.36	0.30	0.33	90
man	0.43	0.50	0.46	90
woman	0.20	0.29	0.24	90
crocodile	0.72	0.78	0.75	90
dinosaur	0.81	0.81	0.81	90
lizard	0.48	0.54	0.51	90
snake	0.61	0.64	0.63	90
turtle	0.61	0.56	0.58	90
hamster	0.20	0.14	0.17	90
mouse	0.42	0.50	0.45	90
rabbit	0.84	0.84	0.84	90
shrew	0.60	0.68	0.64	90
squirrel	0.40	0.38	0.39	90
maple	0.54	0.50	0.52	90
oak	0.68	0.72	0.70	90
palm	0.67	0.79	0.72	90
pine	0.52	0.49	0.50	90
willow	0.51	0.49	0.50	90
bicycle	0.47	0.47	0.47	90
bus	0.48	0.53	0.51	90
motorcycle	0.46	0.42	0.44	90
pickup truck	0.42	0.43	0.43	90
train	0.80	0.82	0.81	90
lawn-mower	0.46	0.46	0.46	90
rocket	0.31	0.21	0.25	90
streetcar	0.56	0.53	0.55	90
tank	0.32	0.28	0.30	90
tractor	0.78	0.77	0.78	90
accuracy			0.55	9000
macro avg	0.54	0.55	0.54	9000
weighted avg	0.54	0.55	0.54	9000

Higher mu in fine_tuning

```
In [ ]: for repeat_2_times in range(2):
        ##### <<<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")
```

```

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphins', 'otter', 'seal', 'whale', 'aquarium fish', 'f
'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                    .map(augment_dataset, num_parallel_calls=tf.data.experimen
#                    .batch(batch_size))

```

```

# .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.7), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2

```

```

        print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
        print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
        print(sample[1].shape) # Should be (batch_size, 100)
    print(f"Model input shape: {model.input_shape}")
    print(f"Model output shape: {model.output_shape}")
    sample = next(iter(test_dataset.as_numpy_iterator()))
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
    print(sample[0].shape, sample[1].shape) # Should match model input and output
    print("\n")
    #for x, y in test_dataset.take(1):
    #    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
    #for x_batch, y_batch in test_dataset.take(1):
    #    test_loss, test_acc = model.evaluate(x_batch, y_batch)
    #    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

    # Compile the model
    #tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
    model.compile(optimizer=optimizers.Adam(learning_rate=5e-3, weight_decay=3e-5),
                  loss='categorical_crossentropy',
                  metrics=['accuracy', 'precision', 'f1_score'])

    model.summary()

    #### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
    early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                                   patience=5, # Num. epochs with no improvement
                                   restore_best_weights=True)

    #reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
    #                               factor=0.1, # Reduce lr by a factor
    #                               patience=3, # Num epochs w/ no improvement
    #                               min_lr=1e-6, # Min lr
    #                               verbose=1)

    #tensorboard = TensorBoard(log_dir='./logs', # Logs directory
    #                           histogram_freq=1, # Logs histograms for weights/acc
    #                           write_graph=True, # Logs graph of model
    #                           write_images=True) # Log images like weight histograms

    checkpoint = ModelCheckpoint('best_model_v3.keras', # Path where the best model is saved
                                monitor='val_loss', # What to monitor (validation loss)
                                save_best_only=True, # Only save model when val loss is best
                                mode='min', # Looking to minimize the validation loss
                                verbose=1) # Print out message when saving model

    checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v3.keras', # Path where the best model is saved
                                            monitor='val_loss', # What to monitor (validation loss)
                                            save_best_only=True, # Only save model when val loss is best
                                            mode='min', # Looking to minimize the validation loss
                                            verbose=1) # Print out message when saving model

    #cvs_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

    # Train the model
    history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                        batch_size=batch_size, callbacks=[early_stopping, checkpoint])

    #### <<<<<<<<Plot Training & Validation Error>>>>>>>>

    # Extract loss and accuracy
    epochs = range(1, len(history.history['loss'])+1)
    train_loss = history.history['loss']
    val_loss = history.history['val_loss']
    train_acc = history.history['accuracy']

```

```

val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v3.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>

# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-4),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

```

```

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v3.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')

```



```
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

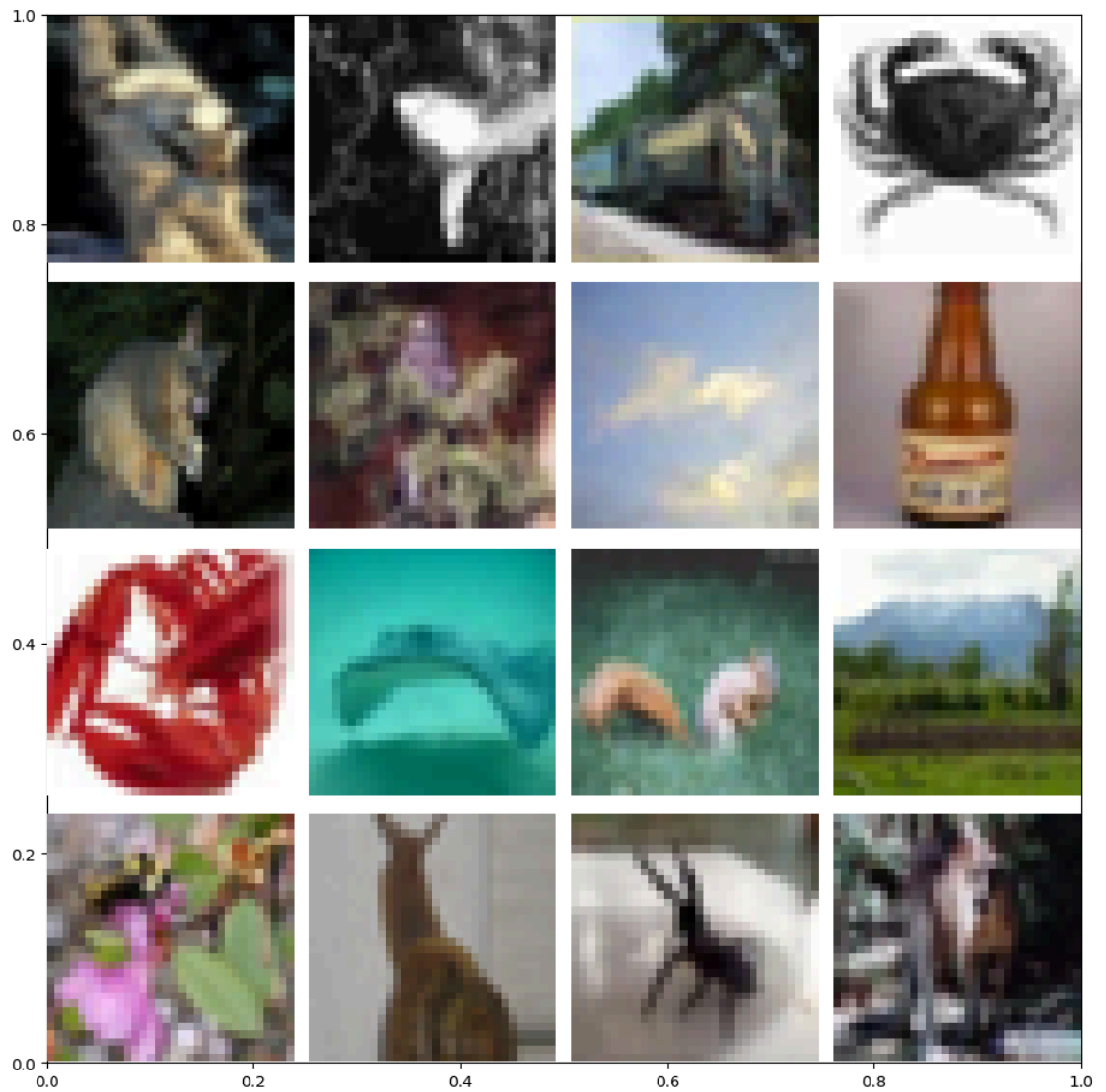
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

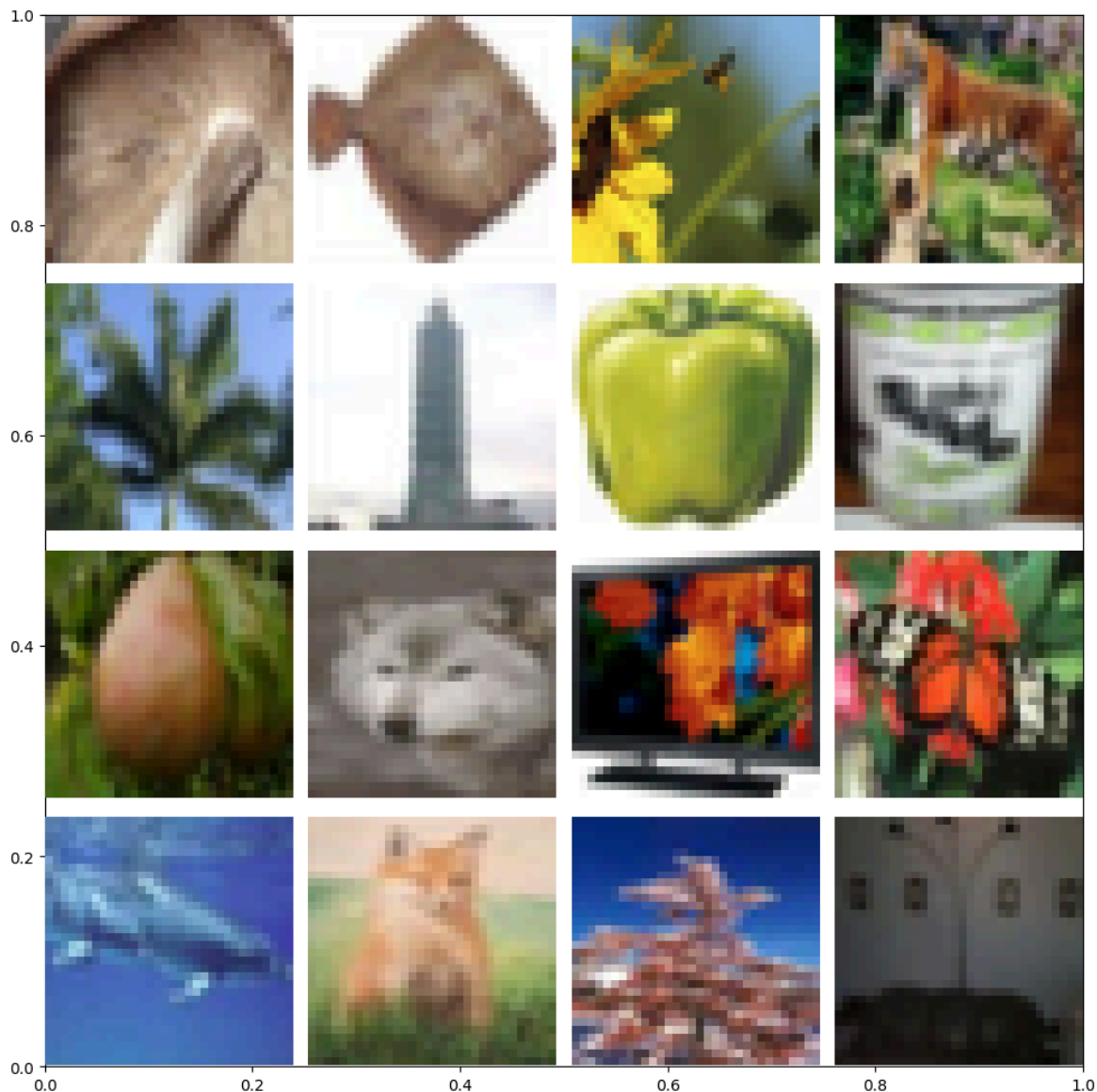
y_test.shape: (9000, 1)

labels[1161].shape: (1,)

imgs[1161].shape: (32, 32, 3)

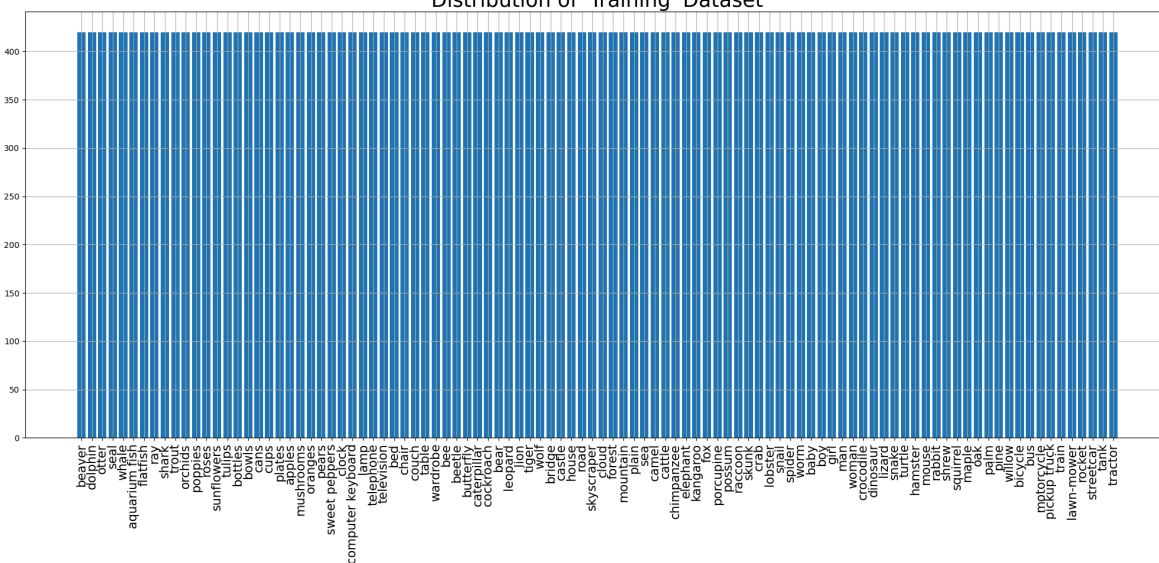


labels[22011].shape: (1,)
imgs[22011].shape: (32, 32, 3)

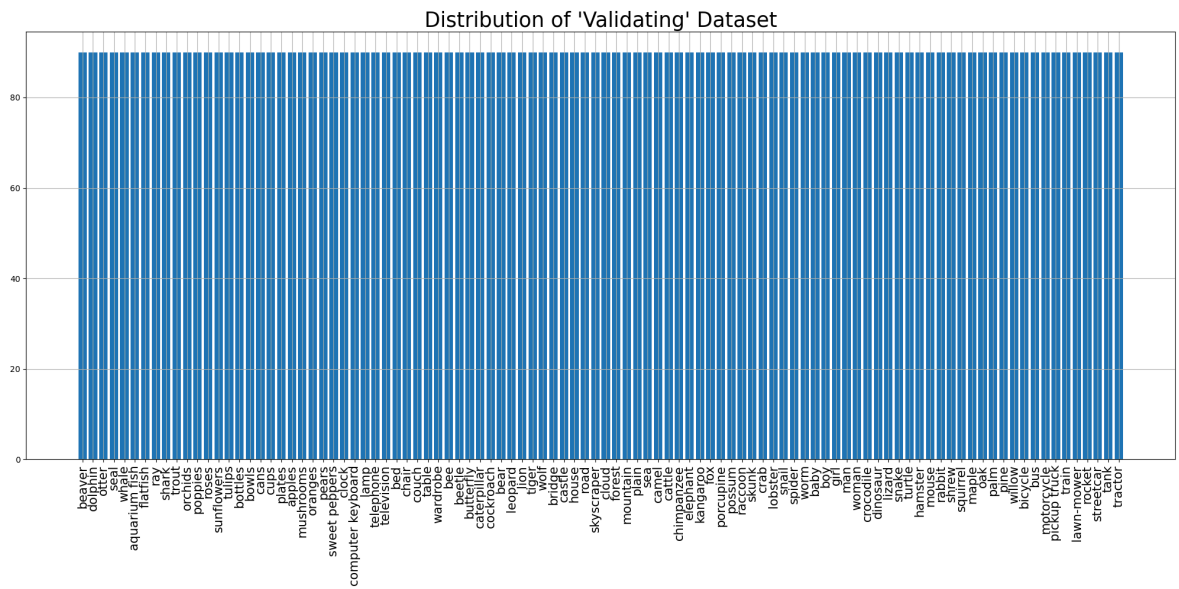


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

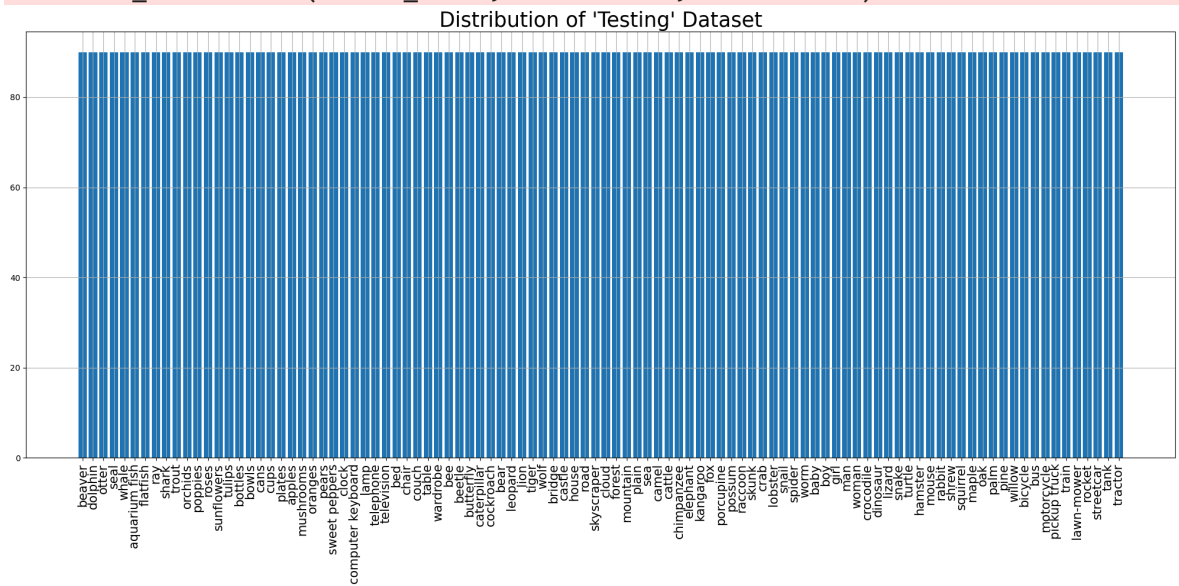
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_4"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None , 2048)
dense_8 (Dense)	(None , 512)
dropout_4 (Dropout)	(None , 512)
dense_9 (Dense)	(None , 100)

◀  ▶

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25

657/657 ————— **0s** 28ms/step - accuracy: 0.1787 - f1_score: 0.1685 - loss: 3.6655 - precision: 0.5004

Epoch 1: val_loss improved from inf to 2.29197, saving model to best_model_v3.keras

657/657 ————— **40s** 45ms/step - accuracy: 0.1788 - f1_score: 0.1686 - loss: 3.6649 - precision: 0.5005 - val_accuracy: 0.3976 - val_f1_score: 0.3750 - val_loss: 2.2920 - val_precision: 0.7953

Epoch 2/25

656/657 ————— **0s** 23ms/step - accuracy: 0.2312 - f1_score: 0.2255 - loss: 3.0981 - precision: 0.5449

Epoch 2: val_loss improved from 2.29197 to 2.20332, saving model to best_model_v3.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.2312 - f1_score: 0.2255 - loss: 3.0980 - precision: 0.5449 - val_accuracy: 0.4081 - val_f1_score: 0.3794 - val_loss: 2.2033 - val_precision: 0.7730

Epoch 3/25

656/657 ————— **0s** 23ms/step - accuracy: 0.2422 - f1_score: 0.2372 - loss: 3.0366 - precision: 0.5549

Epoch 3: val_loss improved from 2.20332 to 2.16808, saving model to best_model_v3.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.2422 - f1_score: 0.2373 - loss: 3.0364 - precision: 0.5549 - val_accuracy: 0.4268 - val_f1_score: 0.4037 - val_loss: 2.1681 - val_precision: 0.7935

Epoch 4/25

655/657 ————— **0s** 23ms/step - accuracy: 0.2463 - f1_score: 0.2422 - loss: 3.0132 - precision: 0.5550

Epoch 4: val_loss did not improve from 2.16808

657/657 ————— **18s** 28ms/step - accuracy: 0.2463 - f1_score: 0.2422 - loss: 3.0130 - precision: 0.5551 - val_accuracy: 0.4090 - val_f1_score: 0.3865 - val_loss: 2.2140 - val_precision: 0.7735

Epoch 5/25

656/657 ————— **0s** 23ms/step - accuracy: 0.2475 - f1_score: 0.2425 - loss: 3.0054 - precision: 0.5604

Epoch 5: val_loss improved from 2.16808 to 2.09667, saving model to best_model_v3.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.2475 - f1_score: 0.2425 - loss: 3.0053 - precision: 0.5604 - val_accuracy: 0.4371 - val_f1_score: 0.4052 - val_loss: 2.0967 - val_precision: 0.7881

Epoch 6/25

655/657 ————— **0s** 23ms/step - accuracy: 0.2576 - f1_score: 0.2518 - loss: 2.9577 - precision: 0.5720

Epoch 6: val_loss did not improve from 2.09667

657/657 ————— **18s** 28ms/step - accuracy: 0.2576 - f1_score: 0.2518 - loss: 2.9576 - precision: 0.5720 - val_accuracy: 0.4327 - val_f1_score: 0.4088 - val_loss: 2.1403 - val_precision: 0.7864

Epoch 7/25

657/657 ————— **0s** 23ms/step - accuracy: 0.2475 - f1_score: 0.2414 - loss: 2.9683 - precision: 0.5579

Epoch 7: val_loss did not improve from 2.09667

657/657 ————— **18s** 28ms/step - accuracy: 0.2475 - f1_score: 0.2414 - loss: 2.9683 - precision: 0.5579 - val_accuracy: 0.4319 - val_f1_score: 0.4100 - val_loss: 2.1336 - val_precision: 0.7721

Epoch 8/25

656/657 ————— **0s** 23ms/step - accuracy: 0.2496 - f1_score: 0.2451 - loss: 2.9847 - precision: 0.5616

Epoch 8: val_loss did not improve from 2.09667

657/657 ————— **18s** 28ms/step - accuracy: 0.2496 - f1_score: 0.2451 - loss: 2.9845 - precision: 0.5616 - val_accuracy: 0.4286 - val_f1_score: 0.4012 - val_loss: 2.0995 - val_precision: 0.7719

Epoch 9/25

655/657 ————— 0s 23ms/step - accuracy: 0.2568 - f1_score: 0.2505 -
loss: 2.9571 - precision: 0.5678

Epoch 9: val_loss did not improve from 2.09667

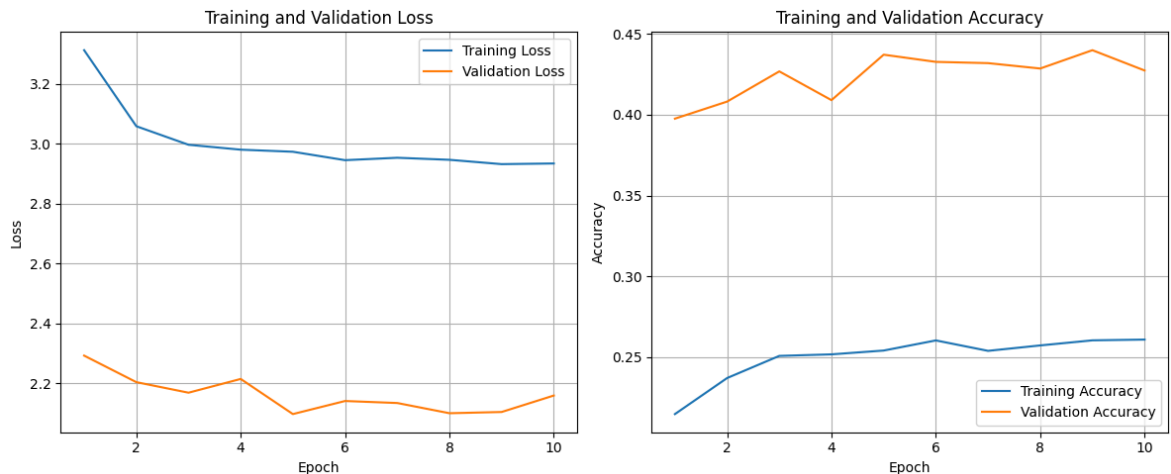
657/657 ————— 18s 28ms/step - accuracy: 0.2569 - f1_score: 0.2506
- loss: 2.9570 - precision: 0.5678 - val_accuracy: 0.4399 - val_f1_score: 0.4125
- val_loss: 2.1036 - val_precision: 0.7900

Epoch 10/25

657/657 ————— 0s 23ms/step - accuracy: 0.2604 - f1_score: 0.2558 -
loss: 2.9424 - precision: 0.5681

Epoch 10: val_loss did not improve from 2.09667

657/657 ————— 18s 28ms/step - accuracy: 0.2604 - f1_score: 0.2558
- loss: 2.9424 - precision: 0.5681 - val_accuracy: 0.4273 - val_f1_score: 0.4010
- val_loss: 2.1586 - val_precision: 0.8096



141/141 ————— 11s 38ms/step - accuracy: 0.4300 - f1_score: 0.3960
- loss: 2.1113 - precision: 0.7709

Test Accuracy: 43.76%

Test Loss: 2.084737539291382

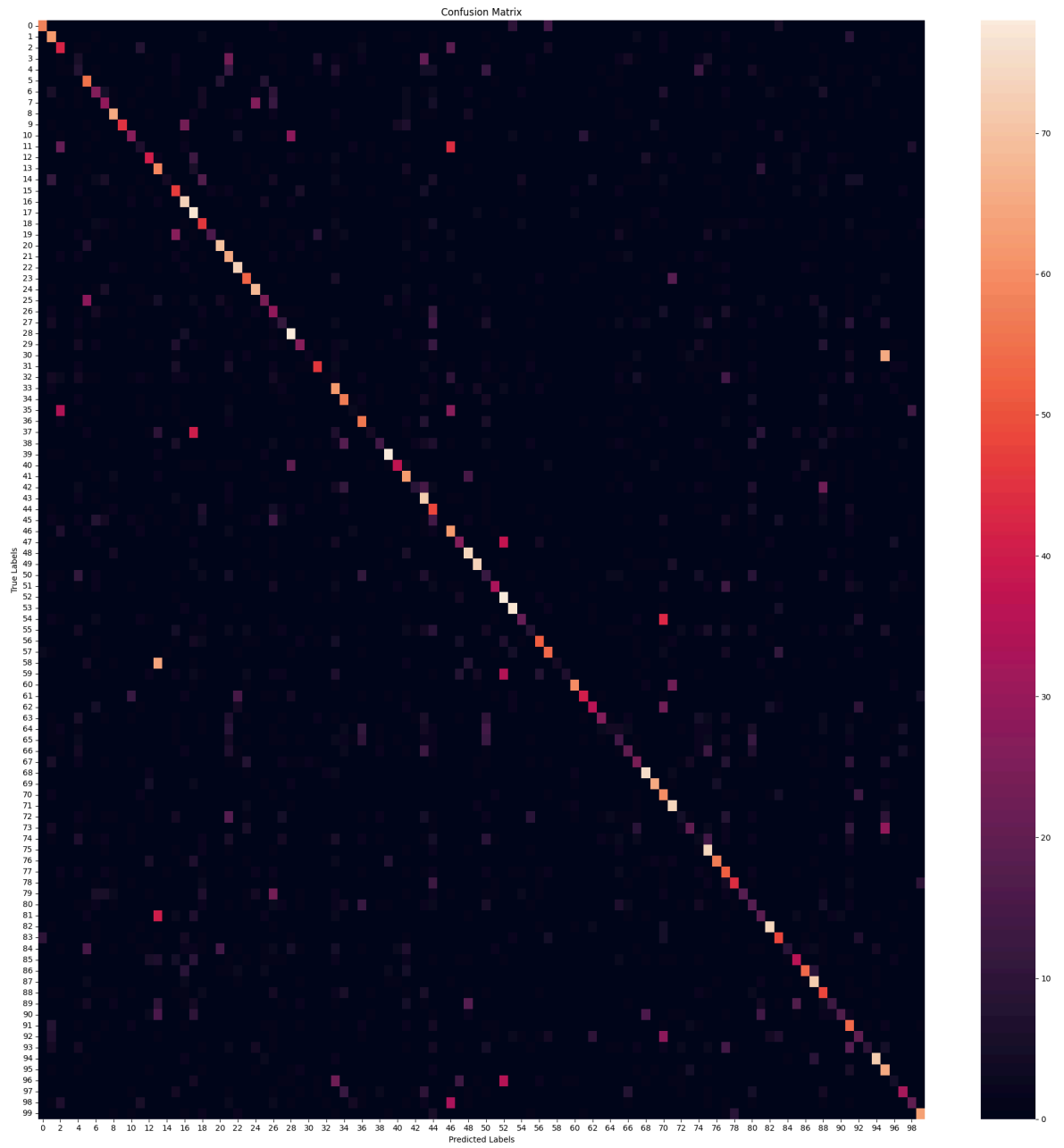
Test Precision: 78.77%

Test F1 Scores (Per Class): [72.04968 53.218872 37.16814 0. 8.37695
8 48.888885

29.378525	36.249996	68.783066	66.17647	35.52631	10.85271
48.235287	37.223972	8.333332	40.34334	61.603367	51.851845
41.628952	29.629627	74.074066	44.3686	71.56862	56.382973
66.666664	32.653057	23.107565	15.714282	63.93443	36.486485
4.1237106	54.761906	2.083332	42.176865	52.293568	5.660375
52.33644	7.8431354	23.72881	81.675385	52.112667	51.666664
16.822426	47.682114	31.683163	0.	41.33333	33.766228
66.07142	79.347824	11.059903	46.80851	53.7931	78.97436
32.307686	12.903222	61.176468	59.016384	8.510637	13.999998
72.289154	56.55172	47.999996	40.944878	7.339447	17.499996
25.974024	30.188673	72.46376	76.02339	39.34426	66.07142
8.620687	31.49606	3.7499952	58.964134	69.090904	37.818176
56.774185	32.142857	17.224876	23.170727	80.43478	50.261772
17.30769	43.37349	64.28571	69.90291	38.095234	21.138208
29.059824	37.24138	24.390238	18.348621	79.55801	43.04635
8.264459	42.3841	29.197077	73.2558	]	

Average Test F1 Scores:40.57

282/282 ————— 11s 28ms/step



	precision	recall	f1-score	support
beaver	0.82	0.64	0.72	90
dolphin	0.43	0.69	0.53	90
otter	0.31	0.47	0.37	90
seal	0.00	0.00	0.00	90
whale	0.08	0.09	0.08	90
aquarium fish	0.41	0.61	0.49	90
flatfish	0.30	0.29	0.29	90
ray	0.41	0.32	0.36	90
shark	0.66	0.72	0.69	90
trout	0.98	0.50	0.66	90
orchids	0.44	0.30	0.36	90
poppies	0.18	0.08	0.11	90
roses	0.51	0.46	0.48	90
sunflowers	0.26	0.66	0.37	90
tulips	0.67	0.04	0.08	90
bottles	0.33	0.52	0.40	90
bowls	0.50	0.81	0.62	90
cans	0.37	0.86	0.52	90
cups	0.35	0.51	0.42	90
plates	0.89	0.18	0.30	90
apples	0.71	0.78	0.74	90
mushrooms	0.32	0.72	0.44	90
oranges	0.64	0.81	0.72	90
pears	0.54	0.59	0.56	90
sweet peppers	0.60	0.76	0.67	90
clock	0.42	0.27	0.33	90
computer keyboard	0.18	0.32	0.23	90
lamp	0.22	0.12	0.16	90
telephone	0.51	0.87	0.64	90
television	0.47	0.30	0.36	90
bed	0.33	0.02	0.04	90
chair	0.59	0.51	0.55	90
couch	0.17	0.01	0.02	90
table	0.30	0.69	0.42	90
wardrobe	0.45	0.63	0.52	90
bee	0.19	0.03	0.06	90
beetle	0.45	0.62	0.52	90
butterfly	0.33	0.04	0.08	90
caterpillar	0.50	0.16	0.24	90
cockroach	0.77	0.87	0.82	90
bear	0.71	0.41	0.52	90
leopard	0.41	0.69	0.52	90
lion	0.53	0.10	0.17	90
tiger	0.34	0.80	0.48	90
wolf	0.23	0.53	0.32	90
bridge	0.00	0.00	0.00	90
castle	0.30	0.69	0.41	90
house	0.41	0.29	0.34	90
road	0.55	0.82	0.66	90
skyscraper	0.78	0.81	0.79	90
cloud	0.09	0.13	0.11	90
forest	0.65	0.37	0.47	90
mountain	0.39	0.87	0.54	90
plain	0.73	0.86	0.79	90
sea	0.53	0.23	0.32	90
camel	0.24	0.09	0.13	90
cattle	0.65	0.58	0.61	90
chimpanzee	0.58	0.60	0.59	90

elephant	1.00	0.04	0.09	90
kangaroo	0.70	0.08	0.14	90
fox	0.79	0.67	0.72	90
porcupine	0.75	0.46	0.57	90
possum	0.60	0.40	0.48	90
raccoon	0.70	0.29	0.41	90
skunk	0.21	0.04	0.07	90
crab	0.20	0.16	0.17	90
lobster	0.31	0.22	0.26	90
snail	0.35	0.27	0.30	90
spider	0.64	0.83	0.72	90
worm	0.80	0.72	0.76	90
baby	0.28	0.67	0.39	90
boy	0.55	0.82	0.66	90
girl	0.19	0.06	0.09	90
man	0.54	0.22	0.31	90
woman	0.04	0.03	0.04	90
crocodile	0.46	0.82	0.59	90
dinosaur	0.76	0.63	0.69	90
lizard	0.28	0.58	0.38	90
snake	0.68	0.49	0.57	90
turtle	0.82	0.20	0.32	90
hamster	0.15	0.20	0.17	90
mouse	0.26	0.21	0.23	90
rabbit	0.79	0.82	0.80	90
shrew	0.48	0.53	0.50	90
squirrel	0.64	0.10	0.17	90
maple	0.47	0.40	0.43	90
oak	0.69	0.60	0.64	90
palm	0.62	0.80	0.70	90
pine	0.30	0.53	0.38	90
willow	0.39	0.14	0.21	90
bicycle	0.63	0.19	0.29	90
bus	0.27	0.60	0.37	90
motorcycle	0.27	0.22	0.24	90
pickup truck	0.53	0.11	0.18	90
train	0.79	0.80	0.80	90
lawn-mower	0.31	0.72	0.43	90
rocket	0.16	0.06	0.08	90
streetcar	0.52	0.36	0.42	90
tank	0.43	0.22	0.29	90
tractor	0.77	0.70	0.73	90
accuracy			0.44	9000
macro avg	0.47	0.44	0.41	9000
weighted avg	0.47	0.44	0.41	9000

```

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Model: "sequential_4"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7, 7, 2048)
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None , 2048)
dense_8 (Dense)	(None , 512)
dropout_4 (Dropout)	(None , 512)
dense_9 (Dense)	(None , 100)



Total params: 24,665,188 (94.09 MB)

Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 27ms/step - accuracy: 0.3046 - f1_score: 0.2923
- loss: 2.6824 - precision: 0.6369
Epoch 1: val_loss improved from inf to 1.76315, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.3046 - f1_score: 0.2923 - loss: 2.6821 - precision: 0.6370 - val_accuracy: 0.5337 - val_f1_score: 0.5258 - val_loss: 1.7632 - val_precision: 0.8461
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3772 - f1_score: 0.3723 - loss: 2.3286 - precision: 0.7009
Epoch 2: val_loss improved from 1.76315 to 1.67602, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3772 - f1_score: 0.3723 - loss: 2.3286 - precision: 0.7009 - val_accuracy: 0.5499 - val_f1_score: 0.5450 - val_loss: 1.6760 - val_precision: 0.8562
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4126 - f1_score: 0.4090 - loss: 2.1624 - precision: 0.7380
Epoch 3: val_loss improved from 1.67602 to 1.62342, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4126 - f1_score: 0.4090 - loss: 2.1623 - precision: 0.7380 - val_accuracy: 0.5649 - val_f1_score: 0.5601 - val_loss: 1.6234 - val_precision: 0.8496
Epoch 4/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4359 - f1_score: 0.4319 - loss: 2.0878 - precision: 0.7552
Epoch 4: val_loss improved from 1.62342 to 1.58228, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4359 - f1_score: 0.4320 - loss: 2.0877 - precision: 0.7552 - val_accuracy: 0.5723 - val_f1_score: 0.5677 - val_loss: 1.5823 - val_precision: 0.8496
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4551 - f1_score: 0.4508 - loss: 2.0008 - precision: 0.7629
Epoch 5: val_loss improved from 1.58228 to 1.54817, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4551 - f1_score: 0.4508 - loss: 2.0007 - precision: 0.7629 - val_accuracy: 0.5798 - val_f1_score: 0.5756 - val_loss: 1.5482 - val_precision: 0.8425
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4697 - f1_score: 0.4656 - loss: 1.9465 - precision: 0.7710
Epoch 6: val_loss improved from 1.54817 to 1.52328, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4697 - f1_score: 0.4656 - loss: 1.9465 - precision: 0.7710 - val_accuracy: 0.5848 - val_f1_score: 0.5806 - val_loss: 1.5233 - val_precision: 0.8447
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4797 - f1_score: 0.4758 - loss: 1.9010 - precision: 0.7770
Epoch 7: val_loss improved from 1.52328 to 1.50204, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4797 - f1_score: 0.4759 - loss: 1.9010 - precision: 0.7770 - val_accuracy: 0.5911 - val_f1_score: 0.5869 - val_loss: 1.5020 - val_precision: 0.8420
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.4914 - f1_score: 0.4877 - loss: 1.8562 - precision: 0.7862
Epoch 8: val_loss improved from 1.50204 to 1.48544, saving model to best_model_fine_tune_v3.keras

ne_tune_v3.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.4914 - f1_score: 0.4877 - loss: 1.8562 - precision: 0.7862 - val_accuracy: 0.5944 - val_f1_score: 0.5910 - val_loss: 1.4854 - val_precision: 0.8453

Epoch 9/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.4981 - f1_score: 0.4941 - loss: 1.8174 - precision: 0.7847

Epoch 9: val_loss improved from 1.48544 to 1.46868, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.4981 - f1_score: 0.4941 - loss: 1.8174 - precision: 0.7847 - val_accuracy: 0.5996 - val_f1_score: 0.5957 - val_loss: 1.4687 - val_precision: 0.8432

Epoch 10/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.5050 - f1_score: 0.5006 - loss: 1.7845 - precision: 0.7925

Epoch 10: val_loss improved from 1.46868 to 1.45457, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.5050 - f1_score: 0.5006 - loss: 1.7845 - precision: 0.7925 - val_accuracy: 0.6013 - val_f1_score: 0.5977 - val_loss: 1.4546 - val_precision: 0.8408

Epoch 11/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.5149 - f1_score: 0.5113 - loss: 1.7597 - precision: 0.7890

Epoch 11: val_loss improved from 1.45457 to 1.43877, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.5149 - f1_score: 0.5113 - loss: 1.7597 - precision: 0.7890 - val_accuracy: 0.6034 - val_f1_score: 0.5996 - val_loss: 1.4388 - val_precision: 0.8396

Epoch 12/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.5228 - f1_score: 0.5185 - loss: 1.7234 - precision: 0.7960

Epoch 12: val_loss improved from 1.43877 to 1.42542, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.5228 - f1_score: 0.5185 - loss: 1.7234 - precision: 0.7960 - val_accuracy: 0.6094 - val_f1_score: 0.6058 - val_loss: 1.4254 - val_precision: 0.8390

Epoch 13/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.5256 - f1_score: 0.5216 - loss: 1.7038 - precision: 0.7976

Epoch 13: val_loss improved from 1.42542 to 1.41423, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.5256 - f1_score: 0.5216 - loss: 1.7038 - precision: 0.7976 - val_accuracy: 0.6099 - val_f1_score: 0.6066 - val_loss: 1.4142 - val_precision: 0.8414

Epoch 14/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.5326 - f1_score: 0.5287 - loss: 1.6698 - precision: 0.7988

Epoch 14: val_loss improved from 1.41423 to 1.40653, saving model to best_model_fine_tune_v3.keras

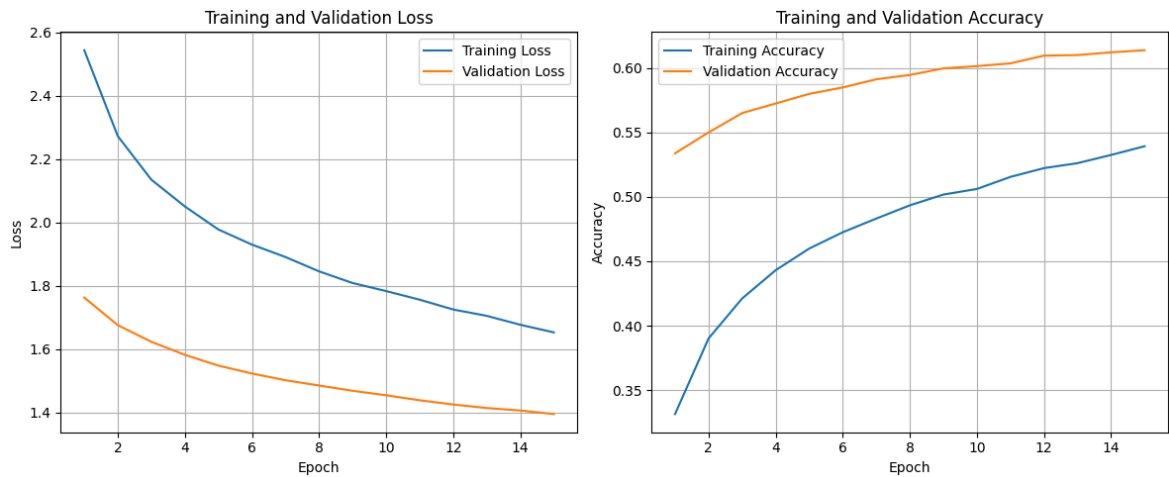
1314/1314 ————— 35s 27ms/step - accuracy: 0.5326 - f1_score: 0.5287 - loss: 1.6699 - precision: 0.7988 - val_accuracy: 0.6120 - val_f1_score: 0.6086 - val_loss: 1.4065 - val_precision: 0.8384

Epoch 15/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.5407 - f1_score: 0.5372 - loss: 1.6473 - precision: 0.7991

Epoch 15: val_loss improved from 1.40653 to 1.39516, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.5407 - f1_score: 0.5372 - loss: 1.6473 - precision: 0.7991 - val_accuracy: 0.6137 - val_f1_score: 0.6106 - val_loss: 1.3952 - val_precision: 0.8357



141/141 ————— **11s** 38ms/step - accuracy: 0.6105 - f1_score: 0.6015

- loss: 1.4062 - precision: 0.8348

Test Accuracy: 62.13%

Test Loss: 1.3784271478652954

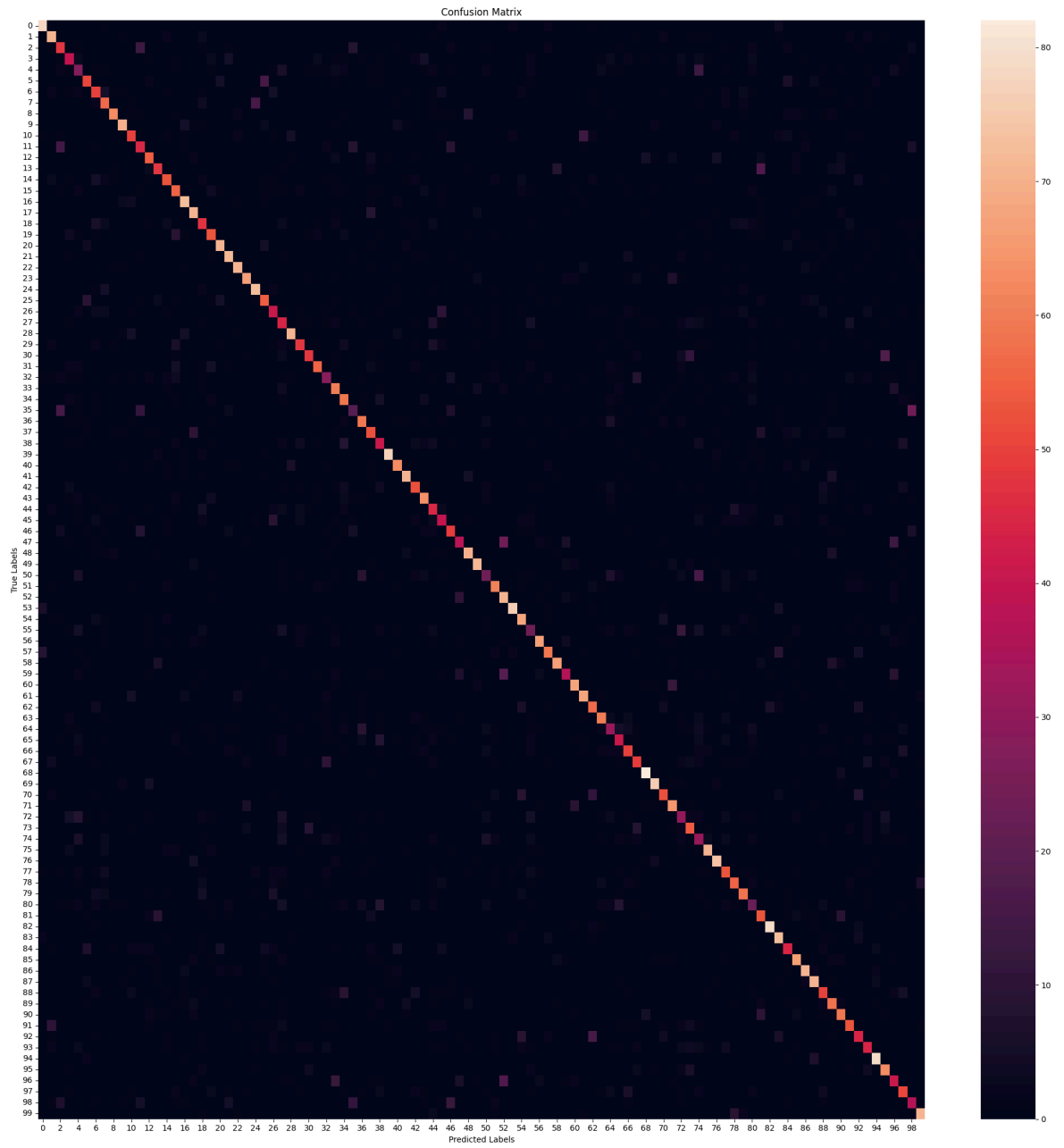
Test Precision: 84.36%

Test F1 Scores (Per Class): [83.422455 72.82051 50.261772 48.19277 29.999996 5
8.959538 52.356018

59.999992 76.36363 80.459755 59.171593 50.279324 61.797745 58.536575
61.9883 58.947365 76.041664 73.19587 48.958324 59.21787 78.88888
75.39267 79.329605 77.906975 77.65957 59.7826 46.32768 41.90476
80.681816 58.895695 59.25926 67.878784 35.443035 58.536583 59.89847
26.206892 63.101597 56.830597 45.55555 89.534874 69.99999 79.775276
59.428566 69.565216 46.63212 46.242767 53.84615 48.71794 75.675674
80. 32.432426 68.53932 63.39285 86.85714 66.666664 29.81366
76.57142 69.82249 77.906975 49.00662 76.40449 74.31693 60.31745
68.965515 37.125744 48.520706 56.179775 55.05617 86.31578 82.608696
61.90476 69.51871 34.090904 58.69565 30.243896 76.756744 81.31868
56.684486 63.276833 64.80447 27.544903 54.639168 88.76403 70.87379
53.333324 69.47368 76.756744 77.173904 56.497173 59.487175 63.440853
60.571426 50.86705 50.86705 85.405396 65.284966 43.850266 57.627117
42.52873 79.329605]

Average Test F1 Scores:61.88

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.80	0.87	0.83	90
dolphin	0.68	0.79	0.73	90
otter	0.48	0.53	0.50	90
seal	0.53	0.44	0.48	90
whale	0.30	0.30	0.30	90
aquarium fish	0.61	0.57	0.59	90
flatfish	0.50	0.56	0.52	90
ray	0.57	0.63	0.60	90
shark	0.84	0.70	0.76	90
trout	0.83	0.78	0.80	90
orchids	0.63	0.56	0.59	90
poppies	0.51	0.50	0.50	90
roses	0.62	0.61	0.62	90
sunflowers	0.65	0.53	0.59	90
tulips	0.65	0.59	0.62	90
bottles	0.56	0.62	0.59	90
bowls	0.72	0.81	0.76	90
cans	0.68	0.79	0.73	90
cups	0.46	0.52	0.49	90
plates	0.60	0.59	0.59	90
apples	0.79	0.79	0.79	90
mushrooms	0.71	0.80	0.75	90
oranges	0.80	0.79	0.79	90
pears	0.82	0.74	0.78	90
sweet peppers	0.74	0.81	0.78	90
clock	0.59	0.61	0.60	90
computer keyboard	0.47	0.46	0.46	90
lamp	0.37	0.49	0.42	90
telephone	0.83	0.79	0.81	90
television	0.66	0.53	0.59	90
bed	0.67	0.53	0.59	90
chair	0.75	0.62	0.68	90
couch	0.41	0.31	0.35	90
table	0.52	0.67	0.59	90
wardrobe	0.55	0.66	0.60	90
bee	0.35	0.21	0.26	90
beetle	0.61	0.66	0.63	90
butterfly	0.56	0.58	0.57	90
caterpillar	0.46	0.46	0.46	90
cockroach	0.94	0.86	0.90	90
bear	0.70	0.70	0.70	90
leopard	0.81	0.79	0.80	90
lion	0.61	0.58	0.59	90
tiger	0.68	0.71	0.70	90
wolf	0.44	0.50	0.47	90
bridge	0.48	0.44	0.46	90
castle	0.53	0.54	0.54	90
house	0.58	0.42	0.49	90
road	0.74	0.78	0.76	90
skyscraper	0.80	0.80	0.80	90
cloud	0.41	0.27	0.32	90
forest	0.69	0.68	0.69	90
mountain	0.53	0.79	0.63	90
plain	0.89	0.84	0.87	90
sea	0.60	0.76	0.67	90
camel	0.34	0.27	0.30	90
cattle	0.79	0.74	0.77	90
chimpanzee	0.75	0.66	0.70	90

elephant	0.82	0.74	0.78	90
kangaroo	0.61	0.41	0.49	90
fox	0.77	0.76	0.76	90
porcupine	0.73	0.76	0.74	90
possum	0.58	0.63	0.60	90
raccoon	0.71	0.67	0.69	90
skunk	0.40	0.34	0.37	90
crab	0.52	0.46	0.49	90
lobster	0.57	0.56	0.56	90
snail	0.56	0.54	0.55	90
spider	0.82	0.91	0.86	90
worm	0.81	0.84	0.83	90
baby	0.67	0.58	0.62	90
boy	0.67	0.72	0.70	90
girl	0.35	0.33	0.34	90
man	0.57	0.60	0.59	90
woman	0.27	0.34	0.30	90
crocodile	0.75	0.79	0.77	90
dinosaur	0.80	0.82	0.81	90
lizard	0.55	0.59	0.57	90
snake	0.64	0.62	0.63	90
turtle	0.65	0.64	0.65	90
hamster	0.30	0.26	0.28	90
mouse	0.51	0.59	0.55	90
rabbit	0.90	0.88	0.89	90
shrew	0.63	0.81	0.71	90
squirrel	0.59	0.49	0.53	90
maple	0.66	0.73	0.69	90
oak	0.75	0.79	0.77	90
palm	0.76	0.79	0.77	90
pine	0.57	0.56	0.56	90
willow	0.55	0.64	0.59	90
bicycle	0.61	0.66	0.63	90
bus	0.62	0.59	0.61	90
motorcycle	0.53	0.49	0.51	90
pickup truck	0.53	0.49	0.51	90
train	0.83	0.88	0.85	90
lawn-mower	0.61	0.70	0.65	90
rocket	0.42	0.46	0.44	90
streetcar	0.59	0.57	0.58	90
tank	0.44	0.41	0.43	90
tractor	0.80	0.79	0.79	90
accuracy			0.62	9000
macro avg	0.62	0.62	0.62	9000
weighted avg	0.62	0.62	0.62	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

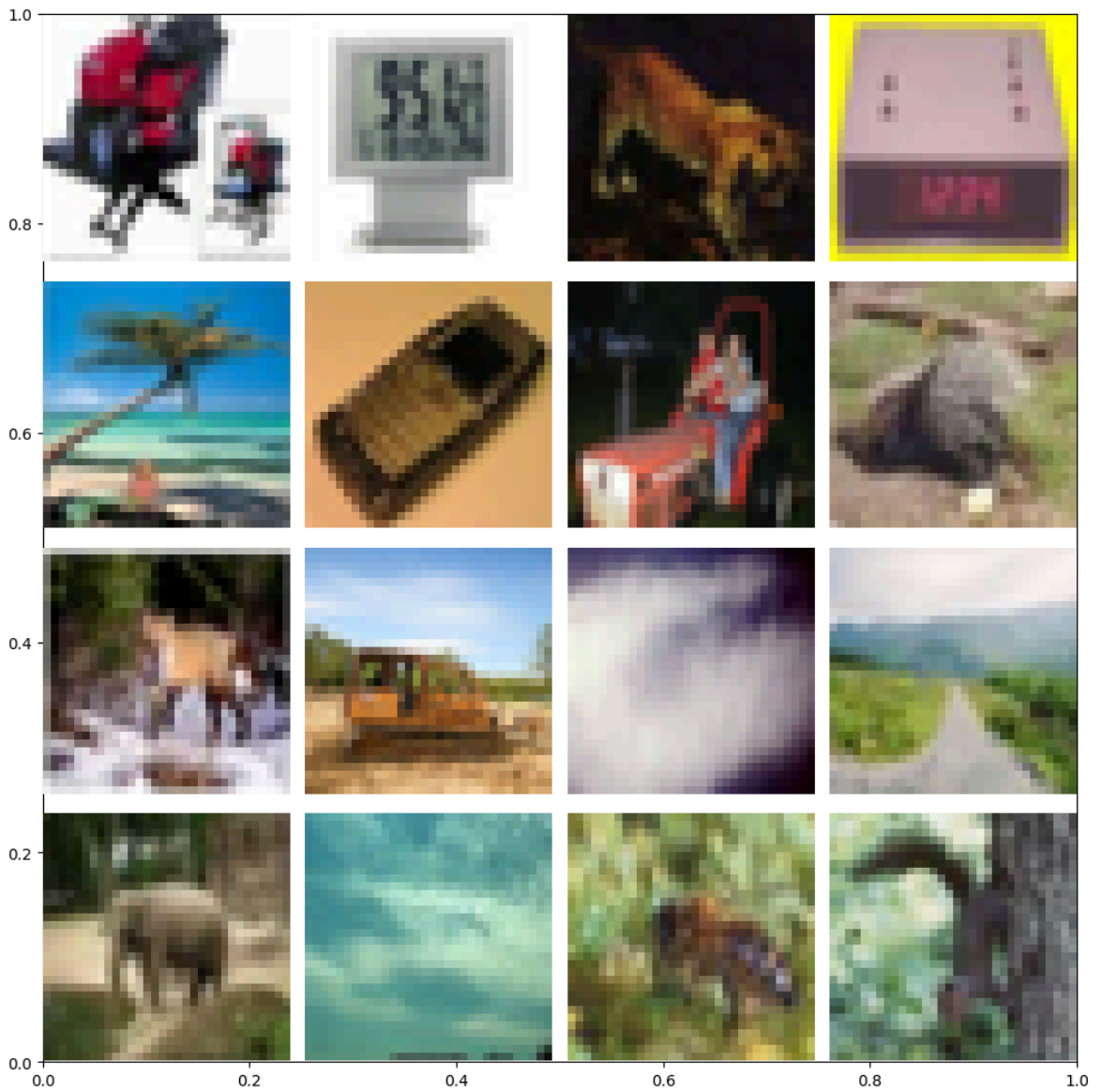
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

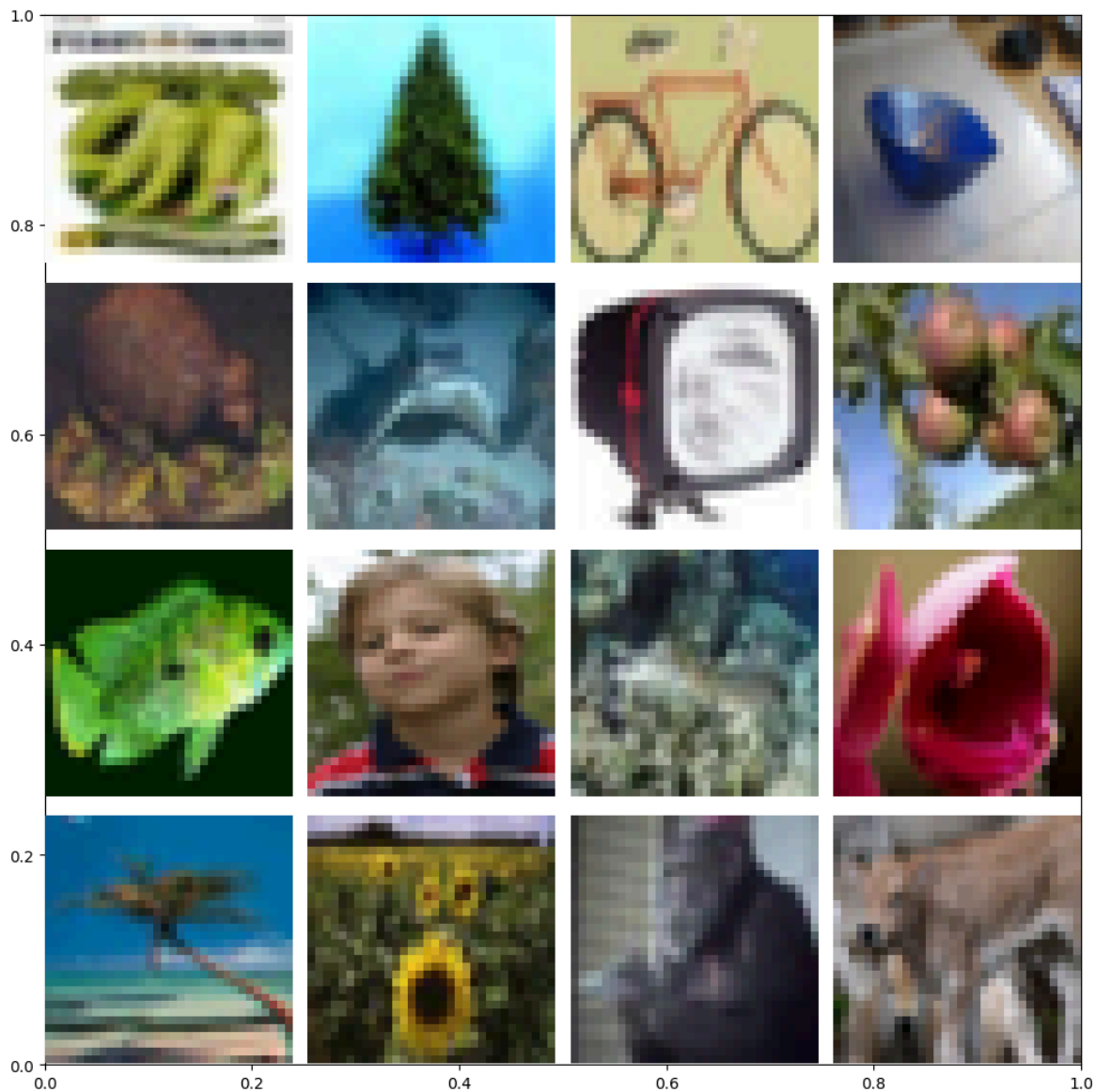
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[39087].shape: (1,)  
imgs[39087].shape: (32, 32, 3)
```

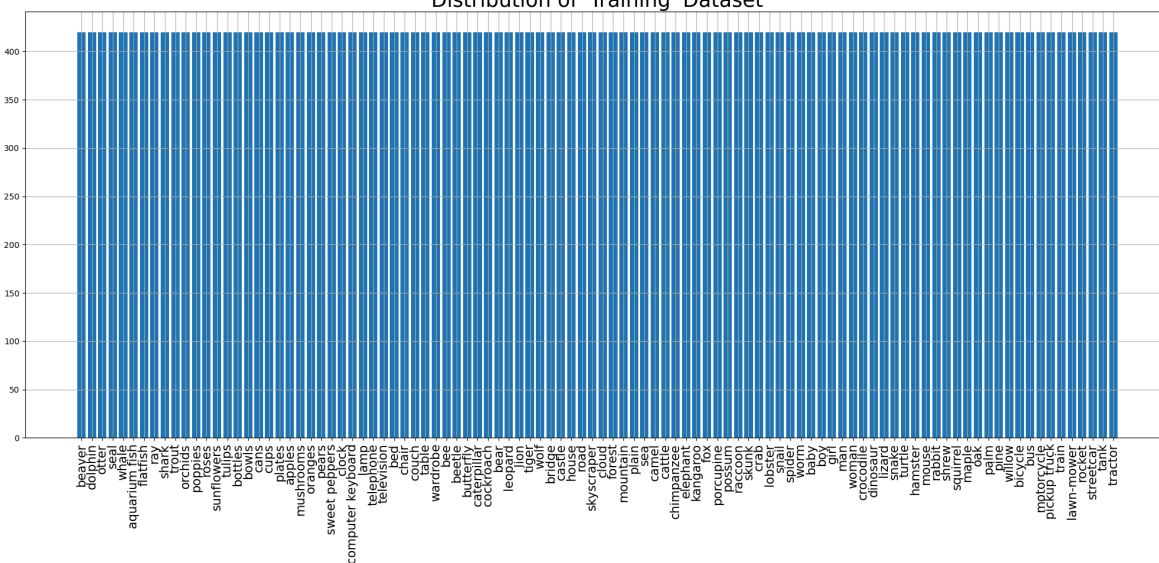


```
labels[24201].shape: (1,)  
imgs[24201].shape: (32, 32, 3)
```

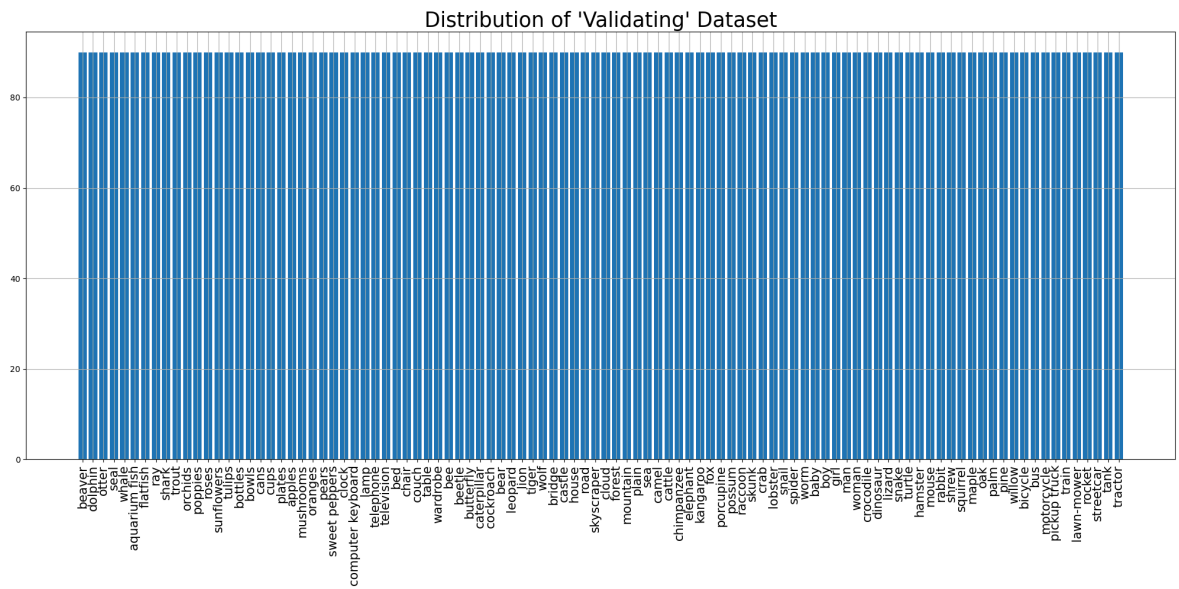


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

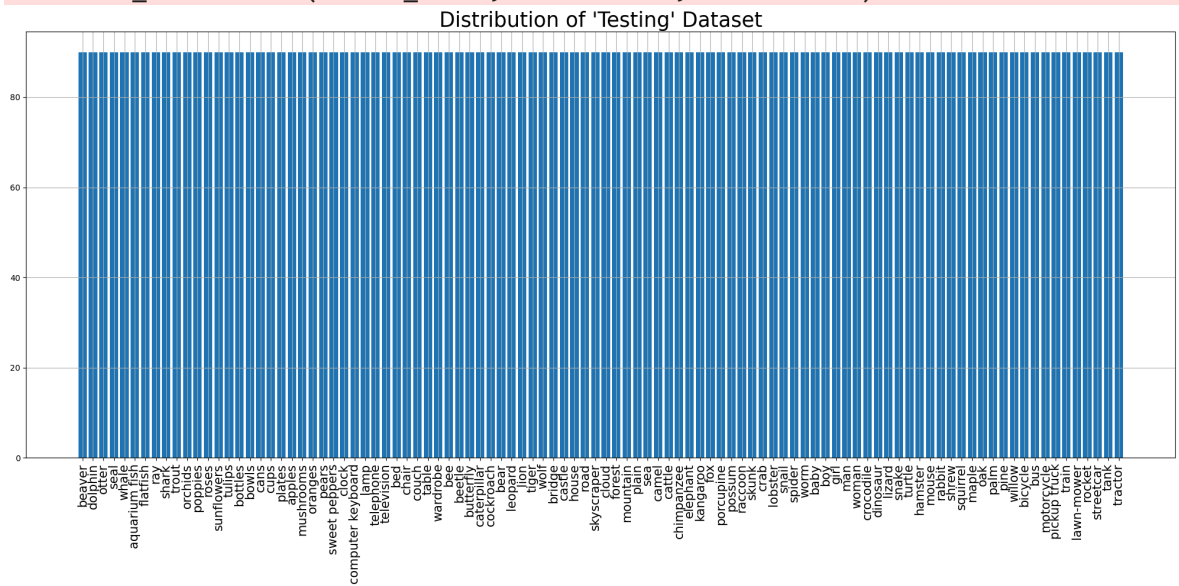
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_5"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None , 2048)
dense_10 (Dense)	(None , 512)
dropout_5 (Dropout)	(None , 512)
dense_11 (Dense)	(None , 100)

◀  ▶

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— **0s** 28ms/step - accuracy: 0.1700 - f1_score: 0.1595 - loss: 3.7160 - precision: 0.4983
Epoch 1: val_loss improved from inf to 2.31241, saving model to best_model_v3.keras as
657/657 ————— **41s** 45ms/step - accuracy: 0.1701 - f1_score: 0.1595 - loss: 3.7154 - precision: 0.4984 - val_accuracy: 0.3893 - val_f1_score: 0.3632 - val_loss: 2.3124 - val_precision: 0.7890
Epoch 2/25
656/657 ————— **0s** 24ms/step - accuracy: 0.2317 - f1_score: 0.2267 - loss: 3.0886 - precision: 0.5455
Epoch 2: val_loss improved from 2.31241 to 2.14306, saving model to best_model_v3.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.2317 - f1_score: 0.2268 - loss: 3.0886 - precision: 0.5456 - val_accuracy: 0.4148 - val_f1_score: 0.3900 - val_loss: 2.1431 - val_precision: 0.7925
Epoch 3/25
656/657 ————— **0s** 23ms/step - accuracy: 0.2353 - f1_score: 0.2309 - loss: 3.0607 - precision: 0.5557
Epoch 3: val_loss did not improve from 2.14306
657/657 ————— **19s** 28ms/step - accuracy: 0.2354 - f1_score: 0.2309 - loss: 3.0606 - precision: 0.5557 - val_accuracy: 0.4143 - val_f1_score: 0.3896 - val_loss: 2.1696 - val_precision: 0.7940
Epoch 4/25
657/657 ————— **0s** 23ms/step - accuracy: 0.2434 - f1_score: 0.2388 - loss: 3.0364 - precision: 0.5537
Epoch 4: val_loss did not improve from 2.14306
657/657 ————— **19s** 28ms/step - accuracy: 0.2434 - f1_score: 0.2388 - loss: 3.0364 - precision: 0.5537 - val_accuracy: 0.4122 - val_f1_score: 0.3902 - val_loss: 2.2114 - val_precision: 0.7839
Epoch 5/25
656/657 ————— **0s** 23ms/step - accuracy: 0.2510 - f1_score: 0.2449 - loss: 2.9770 - precision: 0.5736
Epoch 5: val_loss improved from 2.14306 to 2.11157, saving model to best_model_v3.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.2510 - f1_score: 0.2449 - loss: 2.9770 - precision: 0.5736 - val_accuracy: 0.4324 - val_f1_score: 0.4070 - val_loss: 2.1116 - val_precision: 0.7887
Epoch 6/25
655/657 ————— **0s** 23ms/step - accuracy: 0.2503 - f1_score: 0.2443 - loss: 2.9681 - precision: 0.5650
Epoch 6: val_loss did not improve from 2.11157
657/657 ————— **18s** 28ms/step - accuracy: 0.2503 - f1_score: 0.2444 - loss: 2.9680 - precision: 0.5650 - val_accuracy: 0.4277 - val_f1_score: 0.3996 - val_loss: 2.1478 - val_precision: 0.7869
Epoch 7/25
655/657 ————— **0s** 23ms/step - accuracy: 0.2478 - f1_score: 0.2424 - loss: 2.9969 - precision: 0.5616
Epoch 7: val_loss did not improve from 2.11157
657/657 ————— **18s** 28ms/step - accuracy: 0.2478 - f1_score: 0.2425 - loss: 2.9968 - precision: 0.5617 - val_accuracy: 0.4293 - val_f1_score: 0.4109 - val_loss: 2.1878 - val_precision: 0.7996
Epoch 8/25
657/657 ————— **0s** 23ms/step - accuracy: 0.2534 - f1_score: 0.2483 - loss: 2.9732 - precision: 0.5635
Epoch 8: val_loss did not improve from 2.11157
657/657 ————— **18s** 28ms/step - accuracy: 0.2534 - f1_score: 0.2483 - loss: 2.9731 - precision: 0.5635 - val_accuracy: 0.4219 - val_f1_score: 0.3972 - val_loss: 2.1881 - val_precision: 0.8124
Epoch 9/25

655/657 — 0s 23ms/step - accuracy: 0.2542 - f1_score: 0.2492 - loss: 2.9563 - precision: 0.5719
 Epoch 9: val_loss improved from 2.11157 to 2.09481, saving model to best_model_v3.keras

657/657 — 19s 30ms/step - accuracy: 0.2542 - f1_score: 0.2492 - loss: 2.9563 - precision: 0.5719 - val_accuracy: 0.4408 - val_f1_score: 0.4193 - val_loss: 2.0948 - val_precision: 0.8015
 Epoch 10/25

656/657 — 0s 23ms/step - accuracy: 0.2571 - f1_score: 0.2521 - loss: 2.9519 - precision: 0.5774
 Epoch 10: val_loss did not improve from 2.09481

657/657 — 18s 28ms/step - accuracy: 0.2571 - f1_score: 0.2521 - loss: 2.9519 - precision: 0.5774 - val_accuracy: 0.4400 - val_f1_score: 0.4186 - val_loss: 2.1372 - val_precision: 0.8082
 Epoch 11/25

655/657 — 0s 23ms/step - accuracy: 0.2504 - f1_score: 0.2450 - loss: 2.9571 - precision: 0.5646
 Epoch 11: val_loss did not improve from 2.09481

657/657 — 18s 28ms/step - accuracy: 0.2504 - f1_score: 0.2450 - loss: 2.9571 - precision: 0.5646 - val_accuracy: 0.4357 - val_f1_score: 0.4099 - val_loss: 2.1317 - val_precision: 0.7783
 Epoch 12/25

655/657 — 0s 23ms/step - accuracy: 0.2525 - f1_score: 0.2461 - loss: 2.9690 - precision: 0.5686
 Epoch 12: val_loss did not improve from 2.09481

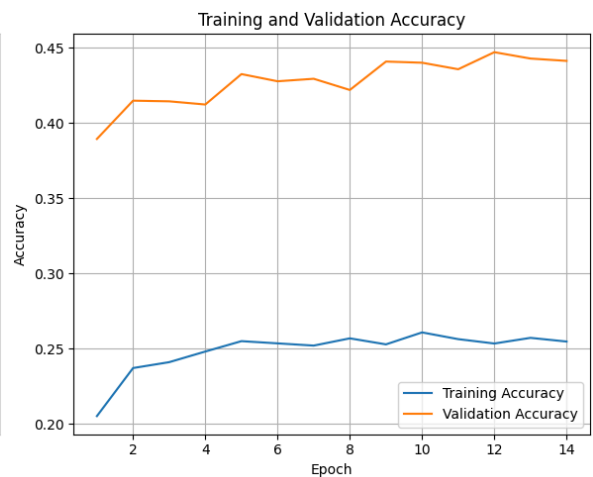
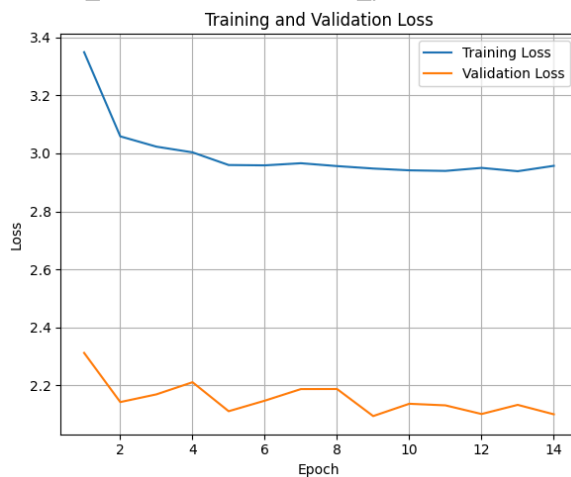
657/657 — 18s 28ms/step - accuracy: 0.2525 - f1_score: 0.2461 - loss: 2.9689 - precision: 0.5686 - val_accuracy: 0.4470 - val_f1_score: 0.4173 - val_loss: 2.1019 - val_precision: 0.8241
 Epoch 13/25

656/657 — 0s 23ms/step - accuracy: 0.2554 - f1_score: 0.2489 - loss: 2.9466 - precision: 0.5756
 Epoch 13: val_loss did not improve from 2.09481

657/657 — 19s 28ms/step - accuracy: 0.2554 - f1_score: 0.2489 - loss: 2.9466 - precision: 0.5756 - val_accuracy: 0.4428 - val_f1_score: 0.4246 - val_loss: 2.1335 - val_precision: 0.8251
 Epoch 14/25

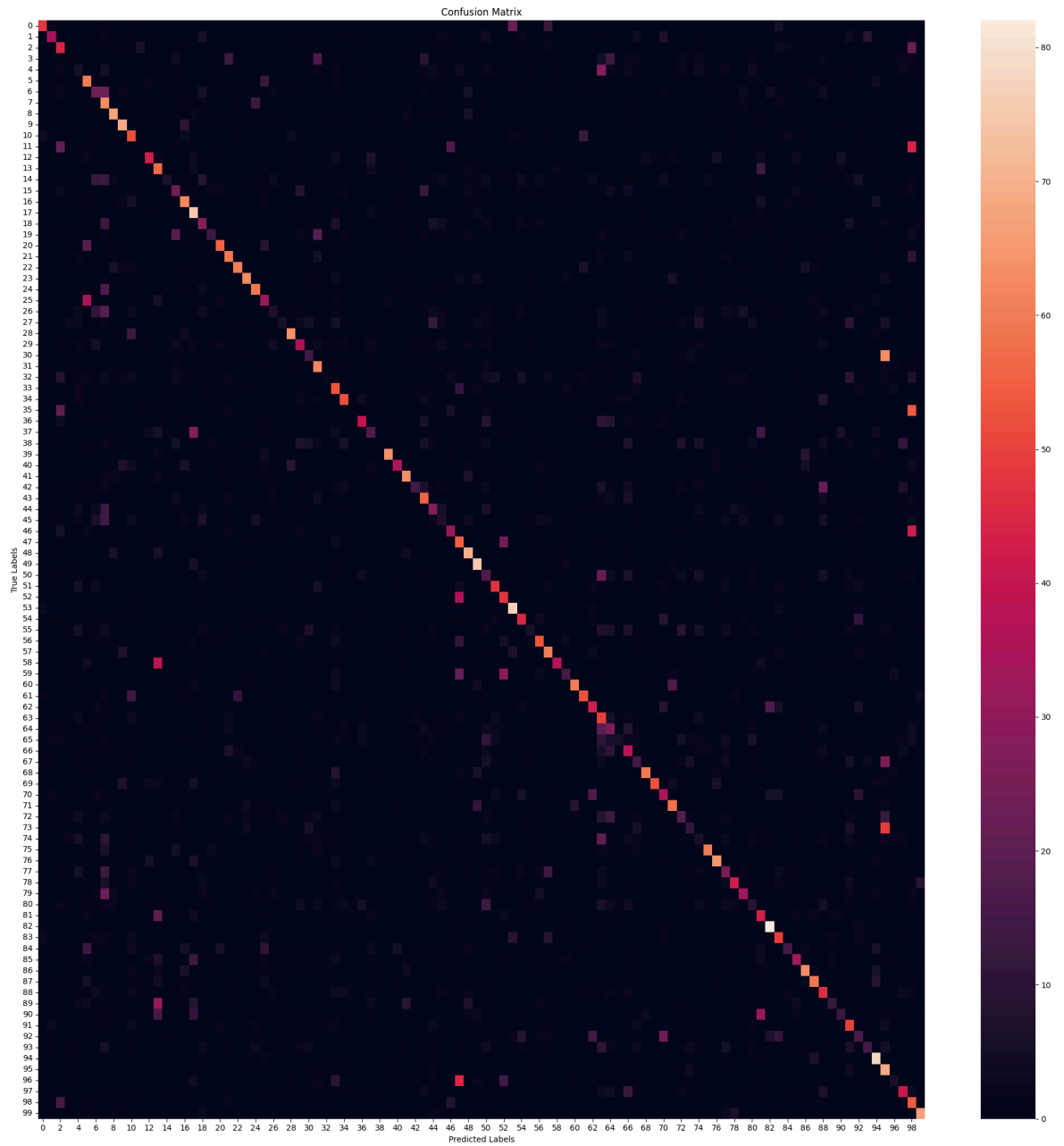
655/657 — 0s 23ms/step - accuracy: 0.2555 - f1_score: 0.2504 - loss: 2.9615 - precision: 0.5719
 Epoch 14: val_loss did not improve from 2.09481

657/657 — 18s 28ms/step - accuracy: 0.2555 - f1_score: 0.2504 - loss: 2.9615 - precision: 0.5719 - val_accuracy: 0.4412 - val_f1_score: 0.4211 - val_loss: 2.1009 - val_precision: 0.8181



141/141 ————— **11s** 38ms/step - accuracy: 0.4256 - f1_score: 0.4006
- loss: 2.1303 - precision: 0.8027
Test Accuracy: 43.81%
Test Loss: 2.0926787853240967
Test Precision: 80.87%
Test F1 Scores (Per Class): [60.75949 46.896545 37.60683 1.9230747 7.49999
5 46.874996
26.519335 33.15789 71.20418 70.40816 46.017693 1.9801961
55.84415 38.38383 12.244897 27.745659 60.19417 54.34782
30.107521 23.140493 67.46988 58.706455 70.17543 71.590904
64.480865 34.224594 11.570245 9.345792 73.56321 39.548016
20.689651 51.452274 3.999998 41.89723 59.770107 0.
51.612896 23.611105 0. 79.99999 51.798557 66.31578
26.086954 53.080566 30.85714 9.999995 38.2716 36.912746
64.22018 71.090034 15.81395 54.651157 43.243237 72.30046
47.872334 11.009172 58.56353 58.536583 55.223877 26.016256
72.72727 61.176468 47.77777 29.761902 24.413141 11.650482
35.023033 18.749996 72.8395 73.10344 36.923073 63.387978
22.360243 17.741932 7.453411 73.17073 73.44632 28.735628
55.6962 43.137253 12.328763 37.885456 76.9953 51.041664
24.193542 43.999992 66.666664 65.92178 40.174667 21.238937
18.897635 45.45454 19.277103 25.599995 69.955154 39.541542
10.344825 48.80952 28.19843 74.86033]
Average Test F1 Scores:41.69

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.71	0.53	0.61	90
dolphin	0.62	0.38	0.47	90
otter	0.31	0.49	0.38	90
seal	0.07	0.01	0.02	90
whale	0.08	0.07	0.07	90
aquarium fish	0.36	0.67	0.47	90
flatfish	0.26	0.27	0.27	90
ray	0.22	0.70	0.33	90
shark	0.67	0.76	0.71	90
trout	0.65	0.77	0.70	90
orchids	0.38	0.58	0.46	90
poppies	0.09	0.01	0.02	90
roses	0.67	0.48	0.56	90
sunflowers	0.28	0.63	0.38	90
tulips	0.75	0.07	0.12	90
bottles	0.29	0.27	0.28	90
bowls	0.53	0.69	0.60	90
cans	0.40	0.83	0.54	90
cups	0.29	0.31	0.30	90
plates	0.45	0.16	0.23	90
apples	0.74	0.62	0.67	90
mushrooms	0.54	0.66	0.59	90
oranges	0.74	0.67	0.70	90
pears	0.73	0.70	0.72	90
sweet peppers	0.63	0.66	0.64	90
clock	0.33	0.36	0.34	90
computer keyboard	0.23	0.08	0.12	90
lamp	0.29	0.06	0.09	90
telephone	0.76	0.71	0.74	90
television	0.40	0.39	0.40	90
bed	0.27	0.17	0.21	90
chair	0.41	0.69	0.51	90
couch	0.20	0.02	0.04	90
table	0.33	0.59	0.42	90
wardrobe	0.62	0.58	0.60	90
bee	0.00	0.00	0.00	90
beetle	0.62	0.44	0.52	90
butterfly	0.31	0.19	0.23	90
caterpillar	0.00	0.00	0.00	90
cockroach	0.91	0.71	0.80	90
bear	0.73	0.40	0.52	90
leopard	0.63	0.70	0.66	90
lion	0.60	0.17	0.26	90
tiger	0.46	0.62	0.53	90
wolf	0.32	0.30	0.31	90
bridge	0.14	0.08	0.10	90
castle	0.43	0.34	0.38	90
house	0.26	0.61	0.37	90
road	0.55	0.78	0.64	90
skyscraper	0.62	0.83	0.71	90
cloud	0.14	0.19	0.16	90
forest	0.57	0.52	0.55	90
mountain	0.36	0.53	0.43	90
plain	0.63	0.86	0.72	90
sea	0.46	0.50	0.48	90
camel	0.32	0.07	0.11	90
cattle	0.58	0.59	0.59	90
chimpanzee	0.52	0.67	0.59	90

elephant	0.84	0.41	0.55	90
kangaroo	0.48	0.18	0.26	90
fox	0.80	0.67	0.73	90
porcupine	0.65	0.58	0.61	90
possum	0.48	0.48	0.48	90
raccoon	0.20	0.56	0.30	90
skunk	0.21	0.29	0.24	90
crab	0.46	0.07	0.12	90
lobster	0.30	0.42	0.35	90
snail	0.21	0.17	0.19	90
spider	0.82	0.66	0.73	90
worm	0.96	0.59	0.73	90
baby	0.34	0.40	0.37	90
boy	0.62	0.64	0.63	90
girl	0.25	0.20	0.22	90
man	0.31	0.12	0.18	90
woman	0.09	0.07	0.07	90
crocodile	0.81	0.67	0.73	90
dinosaur	0.75	0.72	0.73	90
lizard	0.30	0.28	0.29	90
snake	0.64	0.48	0.55	90
turtle	0.52	0.37	0.43	90
hamster	0.16	0.10	0.12	90
mouse	0.31	0.48	0.38	90
rabbit	0.67	0.91	0.77	90
shrew	0.48	0.54	0.51	90
squirrel	0.44	0.17	0.24	90
maple	0.55	0.37	0.44	90
oak	0.64	0.70	0.67	90
palm	0.66	0.66	0.66	90
pine	0.33	0.51	0.40	90
willow	0.52	0.13	0.21	90
bicycle	0.32	0.13	0.19	90
bus	0.38	0.56	0.45	90
motorcycle	0.21	0.18	0.19	90
pickup truck	0.46	0.18	0.26	90
train	0.59	0.87	0.70	90
lawn-mower	0.27	0.77	0.40	90
rocket	0.23	0.07	0.10	90
streetcar	0.53	0.46	0.49	90
tank	0.18	0.60	0.28	90
tractor	0.75	0.74	0.75	90
accuracy			0.44	9000
macro avg	0.45	0.44	0.42	9000
weighted avg	0.45	0.44	0.42	9000

```

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Model: "sequential_5"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7, 7, 2048)
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None , 2048)
dense_10 (Dense)	(None , 512)
dropout_5 (Dropout)	(None , 512)
dense_11 (Dense)	(None , 100)

**Total params:** 24,665,188 (94.09 MB)**Trainable params:** 1,100,388 (4.20 MB)**Non-trainable params:** 23,564,800 (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 27ms/step - accuracy: 0.3058 - f1_score: 0.2961
- loss: 2.6726 - precision: 0.6326
Epoch 1: val_loss improved from inf to 1.76027, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.3058 - f1_score: 0.2962 - loss: 2.6723 - precision: 0.6327 - val_accuracy: 0.5324 - val_f1_score: 0.5247 - val_loss: 1.7603 - val_precision: 0.8406
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.3722 - f1_score: 0.3650 - loss: 2.3404 - precision: 0.7028
Epoch 2: val_loss improved from 1.76027 to 1.68629, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.3722 - f1_score: 0.3650 - loss: 2.3404 - precision: 0.7028 - val_accuracy: 0.5477 - val_f1_score: 0.5432 - val_loss: 1.6863 - val_precision: 0.8452
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4036 - f1_score: 0.3977 - loss: 2.2227 - precision: 0.7366
Epoch 3: val_loss improved from 1.68629 to 1.64100, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.4036 - f1_score: 0.3977 - loss: 2.2226 - precision: 0.7366 - val_accuracy: 0.5578 - val_f1_score: 0.5536 - val_loss: 1.6410 - val_precision: 0.8504
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.4243 - f1_score: 0.4186 - loss: 2.1194 - precision: 0.7547
Epoch 4: val_loss improved from 1.64100 to 1.60892, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.4243 - f1_score: 0.4186 - loss: 2.1193 - precision: 0.7547 - val_accuracy: 0.5637 - val_f1_score: 0.5593 - val_loss: 1.6089 - val_precision: 0.8512
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4422 - f1_score: 0.4370 - loss: 2.0474 - precision: 0.7675
Epoch 5: val_loss improved from 1.60892 to 1.58085, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.4422 - f1_score: 0.4370 - loss: 2.0474 - precision: 0.7675 - val_accuracy: 0.5727 - val_f1_score: 0.5685 - val_loss: 1.5809 - val_precision: 0.8542
Epoch 6/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.4503 - f1_score: 0.4452 - loss: 2.0126 - precision: 0.7700
Epoch 6: val_loss improved from 1.58085 to 1.55865, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.4503 - f1_score: 0.4452 - loss: 2.0126 - precision: 0.7700 - val_accuracy: 0.5764 - val_f1_score: 0.5721 - val_loss: 1.5586 - val_precision: 0.8497
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4598 - f1_score: 0.4548 - loss: 1.9647 - precision: 0.7765
Epoch 7: val_loss improved from 1.55865 to 1.54183, saving model to best_model_fine_tune_v3.keras
1314/1314 ————— 37s 28ms/step - accuracy: 0.4598 - f1_score: 0.4548 - loss: 1.9647 - precision: 0.7765 - val_accuracy: 0.5819 - val_f1_score: 0.5772 - val_loss: 1.5418 - val_precision: 0.8477
Epoch 8/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.4628 - f1_score: 0.4578 - loss: 1.9413 - precision: 0.7739
Epoch 8: val_loss improved from 1.54183 to 1.52327, saving model to best_model_fine_tune_v3.keras

ne_tune_v3.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.4628 - f1_score: 0.4578 - loss: 1.9413 - precision: 0.7739 - val_accuracy: 0.5846 - val_f1_score: 0.5801 - val_loss: 1.5233 - val_precision: 0.8472

Epoch 9/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.4758 - f1_score: 0.4708 - loss: 1.9075 - precision: 0.7790

Epoch 9: val_loss improved from 1.52327 to 1.50816, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.4758 - f1_score: 0.4708 - loss: 1.9075 - precision: 0.7790 - val_accuracy: 0.5884 - val_f1_score: 0.5842 - val_loss: 1.5082 - val_precision: 0.8483

Epoch 10/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.4814 - f1_score: 0.4767 - loss: 1.8732 - precision: 0.7812

Epoch 10: val_loss improved from 1.50816 to 1.49645, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.4814 - f1_score: 0.4767 - loss: 1.8732 - precision: 0.7812 - val_accuracy: 0.5897 - val_f1_score: 0.5855 - val_loss: 1.4965 - val_precision: 0.8477

Epoch 11/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.4853 - f1_score: 0.4805 - loss: 1.8540 - precision: 0.7874

Epoch 11: val_loss improved from 1.49645 to 1.48527, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.4853 - f1_score: 0.4805 - loss: 1.8540 - precision: 0.7874 - val_accuracy: 0.5926 - val_f1_score: 0.5887 - val_loss: 1.4853 - val_precision: 0.8456

Epoch 12/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.4942 - f1_score: 0.4898 - loss: 1.8259 - precision: 0.7922

Epoch 12: val_loss improved from 1.48527 to 1.47242, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.4942 - f1_score: 0.4898 - loss: 1.8259 - precision: 0.7922 - val_accuracy: 0.5954 - val_f1_score: 0.5920 - val_loss: 1.4724 - val_precision: 0.8468

Epoch 13/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.4971 - f1_score: 0.4925 - loss: 1.8015 - precision: 0.7916

Epoch 13: val_loss improved from 1.47242 to 1.46252, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.4971 - f1_score: 0.4925 - loss: 1.8015 - precision: 0.7916 - val_accuracy: 0.5952 - val_f1_score: 0.5920 - val_loss: 1.4625 - val_precision: 0.8443

Epoch 14/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.5018 - f1_score: 0.4978 - loss: 1.7843 - precision: 0.7948

Epoch 14: val_loss improved from 1.46252 to 1.45271, saving model to best_model_fine_tune_v3.keras

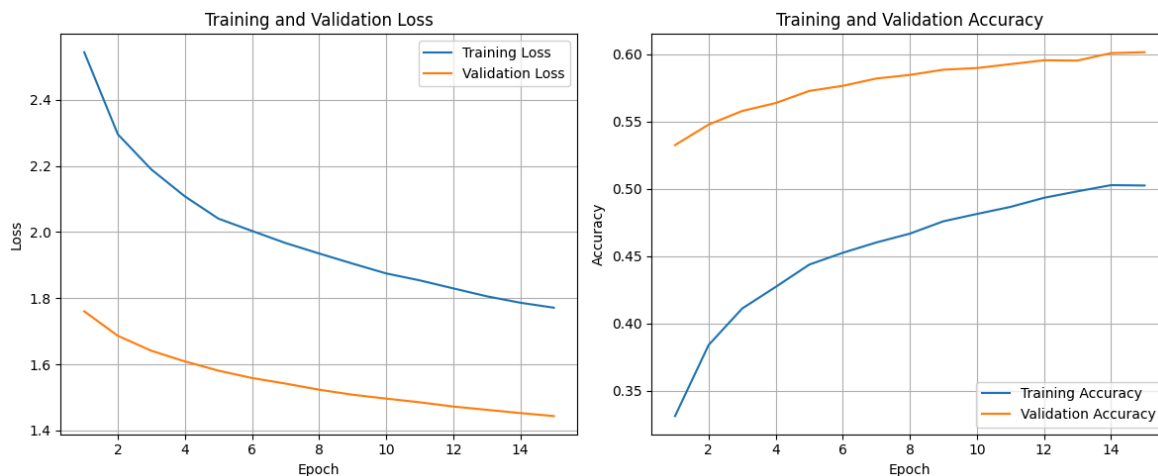
1314/1314 ————— 36s 27ms/step - accuracy: 0.5018 - f1_score: 0.4978 - loss: 1.7843 - precision: 0.7948 - val_accuracy: 0.6008 - val_f1_score: 0.5981 - val_loss: 1.4527 - val_precision: 0.8474

Epoch 15/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.5017 - f1_score: 0.4975 - loss: 1.7671 - precision: 0.7942

Epoch 15: val_loss improved from 1.45271 to 1.44357, saving model to best_model_fine_tune_v3.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.5017 - f1_score: 0.4975 - loss: 1.7671 - precision: 0.7942 - val_accuracy: 0.6014 - val_f1_score: 0.5986 - val_loss: 1.4436 - val_precision: 0.8463



141/141 ————— **11s** 38ms/step - accuracy: 0.5898 - f1_score: 0.5821

- loss: 1.4549 - precision: 0.8379

Test Accuracy: 60.64%

Test Loss: 1.4258472919464111

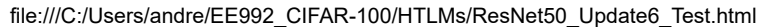
Test Precision: 85.31%

Test F1 Scores (Per Class): [82.16216 70.99999 53.20197 41.9753 26.530605 6 0.109283 54.82233

61.702126 80.23952 82.41757 62.92134 42.391296 66.666664 52.380943
64.59627 54.545456 76.08696 72.251305 55.49738 53.714287 80.22598
72.727264 76.300575 77.71428 75.93583 60.714275 48.148144 38.095238
80.681816 59.090908 47.999996 63.905323 39.024387 54.46009 63.157898
19.512192 64.2857 54.545444 45.74468 88.75739 63.101597 79.55801
54.64481 69.61325 47.05882 36.363632 53.801163 44.02515 78.75647
81.52173 28.38709 69.23077 61.53846 85.549126 67.92453 31.847128
75.86206 66.29213 74.556206 49.056595 75.28088 72.04301 60.63829
62.499992 37.349392 38.961033 55.865913 53.179188 83.060104 83.9779
60.355026 69.51871 33.33333 55.91397 29.850739 78.49462 85.22727
55.681812 62.499992 60.67415 23.37662 52.972965 87.00564 68.39378
52.808983 66.66666 78.453026 76.24309 53.763435 58.63874 60.21505
57.627117 45.71428 54.23729 86.33879 56.852787 33.898304 56.140347
31.90184 79.329605]

Average Test F1 Scores:60.40

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.80	0.84	0.82	90
dolphin	0.65	0.79	0.71	90
otter	0.48	0.60	0.53	90
seal	0.47	0.38	0.42	90
whale	0.25	0.29	0.27	90
aquarium fish	0.59	0.61	0.60	90
flatfish	0.50	0.60	0.55	90
ray	0.59	0.64	0.62	90
shark	0.87	0.74	0.80	90
trout	0.82	0.83	0.82	90
orchids	0.64	0.62	0.63	90
poppies	0.41	0.43	0.42	90
roses	0.69	0.64	0.67	90
sunflowers	0.56	0.49	0.52	90
tulips	0.73	0.58	0.65	90
bottles	0.56	0.53	0.55	90
bowls	0.74	0.78	0.76	90
cans	0.68	0.77	0.72	90
cups	0.52	0.59	0.55	90
plates	0.55	0.52	0.54	90
apples	0.82	0.79	0.80	90
mushrooms	0.70	0.76	0.73	90
oranges	0.80	0.73	0.76	90
pears	0.80	0.76	0.78	90
sweet peppers	0.73	0.79	0.76	90
clock	0.65	0.57	0.61	90
computer keyboard	0.54	0.43	0.48	90
lamp	0.31	0.49	0.38	90
telephone	0.83	0.79	0.81	90
television	0.60	0.58	0.59	90
bed	0.60	0.40	0.48	90
chair	0.68	0.60	0.64	90
couch	0.43	0.36	0.39	90
table	0.47	0.64	0.54	90
wardrobe	0.60	0.67	0.63	90
bee	0.22	0.18	0.20	90
beetle	0.59	0.70	0.64	90
butterfly	0.53	0.57	0.55	90
caterpillar	0.44	0.48	0.46	90
cockroach	0.95	0.83	0.89	90
bear	0.61	0.66	0.63	90
leopard	0.79	0.80	0.80	90
lion	0.54	0.56	0.55	90
tiger	0.69	0.70	0.70	90
wolf	0.45	0.49	0.47	90
bridge	0.37	0.36	0.36	90
castle	0.57	0.51	0.54	90
house	0.51	0.39	0.44	90
road	0.74	0.84	0.79	90
skyscraper	0.80	0.83	0.82	90
cloud	0.34	0.24	0.28	90
forest	0.68	0.70	0.69	90
mountain	0.50	0.80	0.62	90
plain	0.89	0.82	0.86	90
sea	0.59	0.80	0.68	90
camel	0.37	0.28	0.32	90
cattle	0.79	0.73	0.76	90
chimpanzee	0.68	0.66	0.67	90

elephant	0.80	0.70	0.75	90
kangaroo	0.57	0.43	0.49	90
fox	0.76	0.74	0.75	90
porcupine	0.70	0.74	0.72	90
possum	0.58	0.63	0.61	90
raccoon	0.64	0.61	0.62	90
skunk	0.41	0.34	0.37	90
crab	0.47	0.33	0.39	90
lobster	0.56	0.56	0.56	90
snail	0.55	0.51	0.53	90
spider	0.82	0.84	0.83	90
worm	0.84	0.84	0.84	90
baby	0.65	0.57	0.60	90
boy	0.67	0.72	0.70	90
girl	0.38	0.30	0.33	90
man	0.54	0.58	0.56	90
woman	0.27	0.33	0.30	90
crocodile	0.76	0.81	0.78	90
dinosaur	0.87	0.83	0.85	90
lizard	0.56	0.54	0.55	90
snake	0.64	0.61	0.62	90
turtle	0.61	0.60	0.61	90
hamster	0.28	0.20	0.23	90
mouse	0.52	0.54	0.53	90
rabbit	0.89	0.86	0.87	90
shrew	0.64	0.73	0.68	90
squirrel	0.53	0.52	0.53	90
maple	0.65	0.69	0.67	90
oak	0.78	0.79	0.78	90
palm	0.76	0.77	0.76	90
pine	0.52	0.56	0.54	90
willow	0.55	0.62	0.59	90
bicycle	0.58	0.62	0.60	90
bus	0.59	0.57	0.58	90
motorcycle	0.47	0.44	0.46	90
pickup truck	0.55	0.53	0.54	90
train	0.85	0.88	0.86	90
lawn-mower	0.52	0.62	0.57	90
rocket	0.34	0.33	0.34	90
streetcar	0.59	0.53	0.56	90
tank	0.36	0.29	0.32	90
tractor	0.80	0.79	0.79	90
accuracy			0.61	9000
macro avg	0.61	0.61	0.60	9000
weighted avg	0.61	0.61	0.60	9000

RMSprop

```
In [ ]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")
```

```

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                    .map(augment_dataset, num_parallel_calls=tf.data.experimen
#                    .batch(batch_size))

```

```

# .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100), dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100), dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.7), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2

```

```

        print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
        print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
        print(sample[1].shape) # Should be (batch_size, 100)
    print(f"Model input shape: {model.input_shape}")
    print(f"Model output shape: {model.output_shape}")
    sample = next(iter(test_dataset.as_numpy_iterator()))
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
    print(sample[0].shape, sample[1].shape) # Should match model input and output
    print("\n")
    #for x, y in test_dataset.take(1):
    #    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
    #for x_batch, y_batch in test_dataset.take(1):
    #    test_loss, test_acc = model.evaluate(x_batch, y_batch)
    #    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
model.compile(optimizer=optimizers.RMSprop(learning_rate=1e-3, momentum=0.1,
        loss='categorical_crossentropy',
        metrics=['accuracy', 'precision', 'f1_score']))

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
        patience=5, # Num. epochs with no improvement
        restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#    factor=0.1, # Reduce lr by a factor
#    patience=3, # Num epochs w/ no improvement
#    min_lr=1e-6, # Min lr
#    verbose=1)
#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#    histogram_freq=1, # Logs histograms for weights/acc
#    write_graph=True, # Logs graph of model
#    write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v4.keras', # Path where the best model is saved
        monitor='val_loss', # What to monitor (validation loss)
        save_best_only=True, # Only save model when val loss is best
        mode='min', # Looking to minimize the validation loss
        verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v4.keras', # Path where the best model is saved
        monitor='val_loss', # What to monitor (validation loss)
        save_best_only=True, # Only save model when val loss is best
        mode='min', # Looking to minimize the validation loss
        verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
        batch_size=batch_size, callbacks=[early_stopping, checkpoint])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']

```

```

val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v4.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>

# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-4),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

```

```

#### <<<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v4.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')

```

```
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

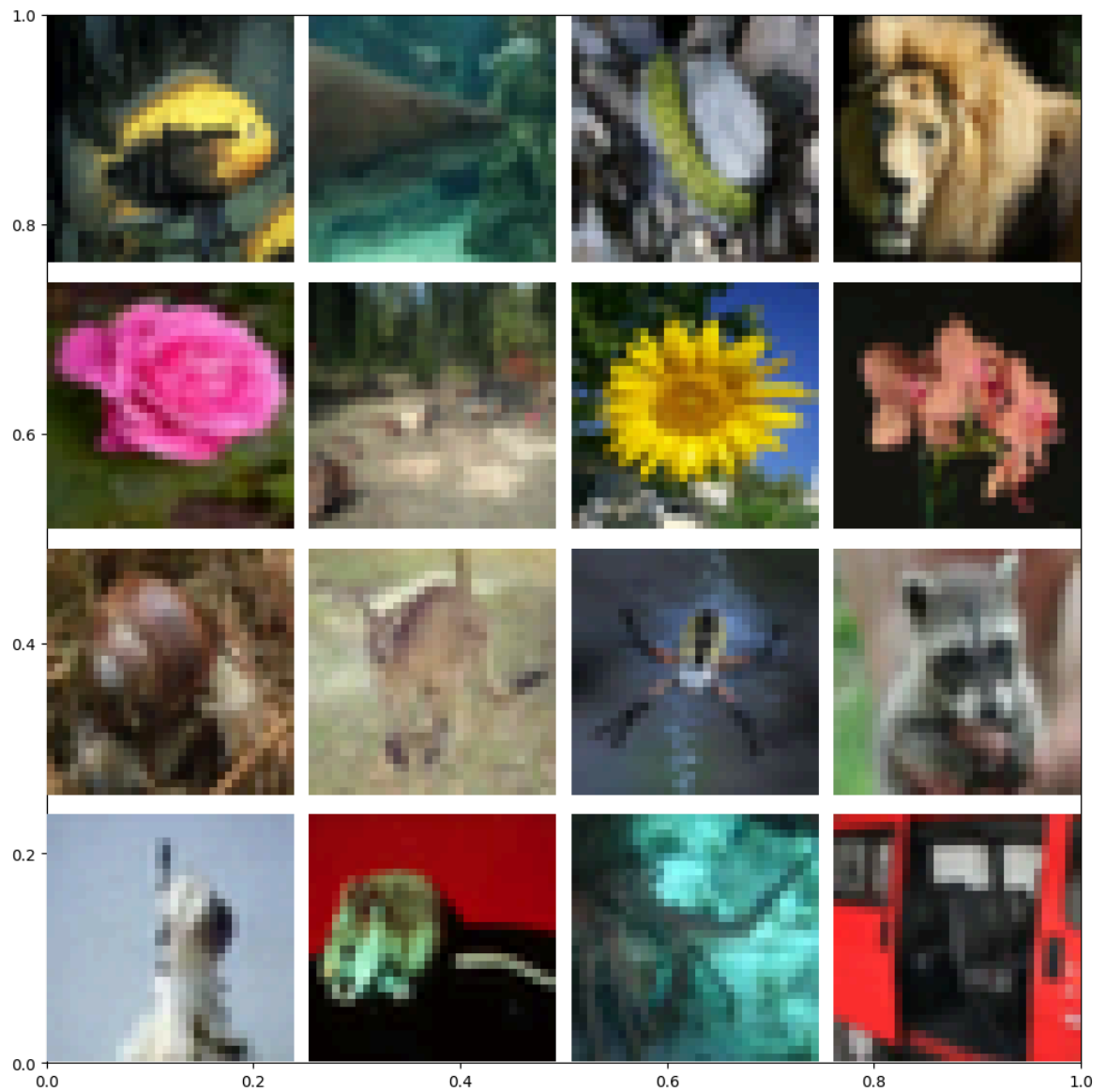
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

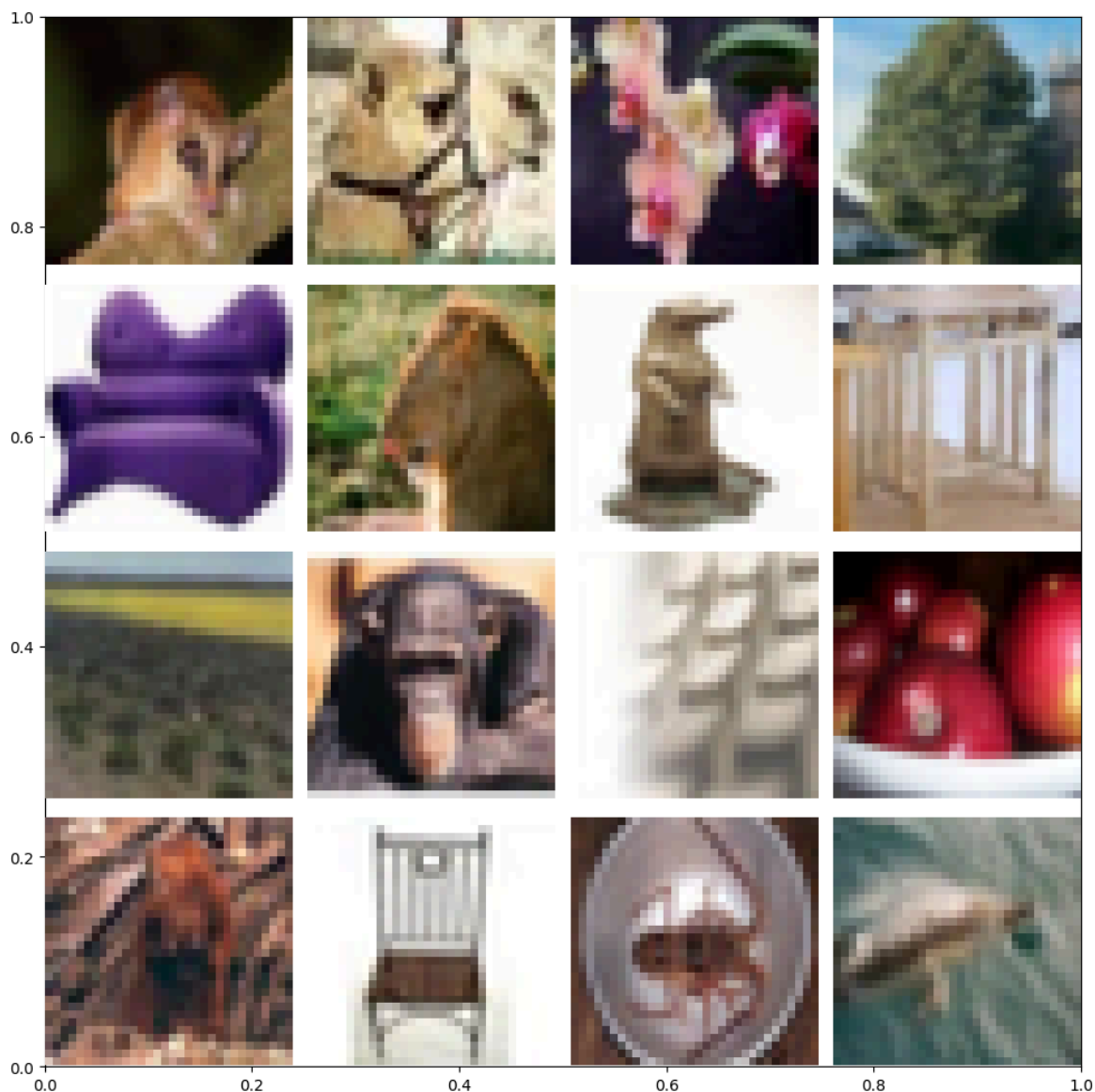
y_test.shape: (9000, 1)

labels[1834].shape: (1,)

imgs[1834].shape: (32, 32, 3)

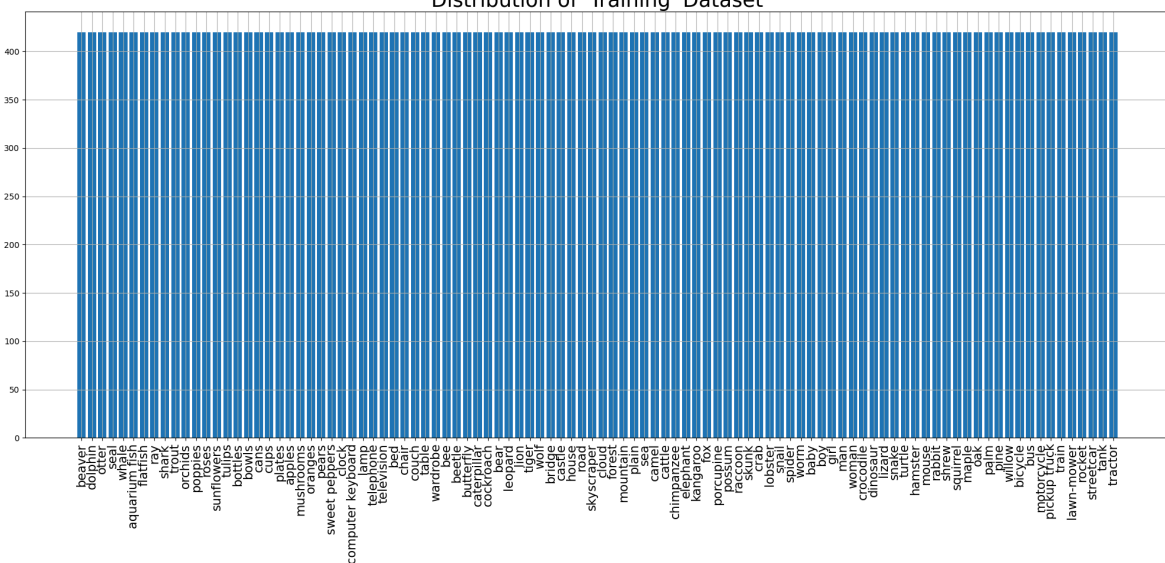


```
labels[16620].shape: (1,)  
imgs[16620].shape: (32, 32, 3)
```

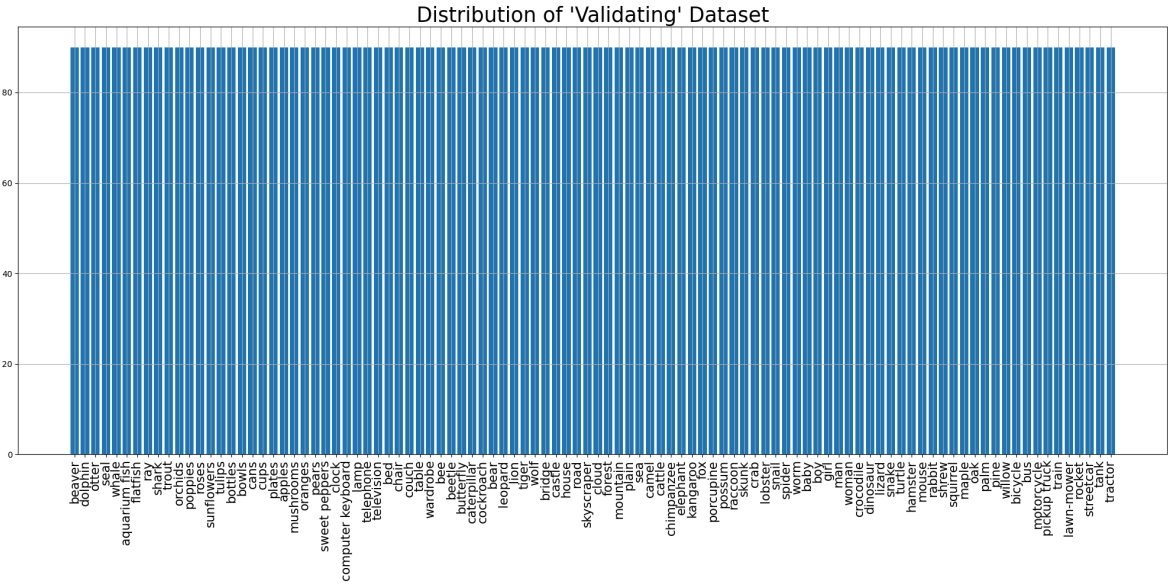


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

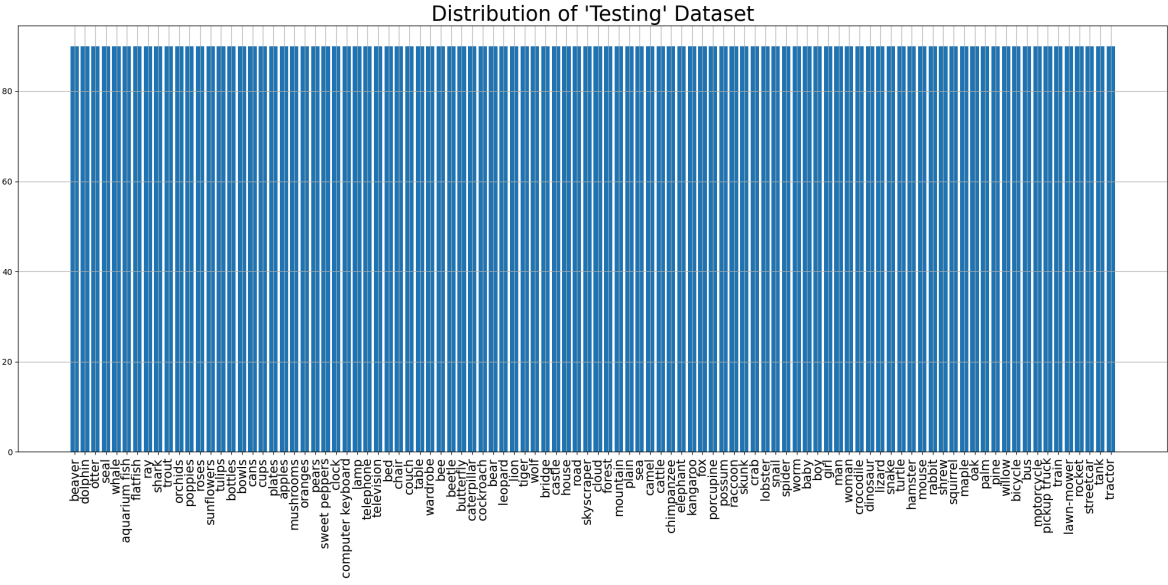
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_8"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_8 (GlobalAveragePooling2D)	(None , 2048)
dense_16 (Dense)	(None , 512)
dropout_8 (Dropout)	(None , 512)
dense_17 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

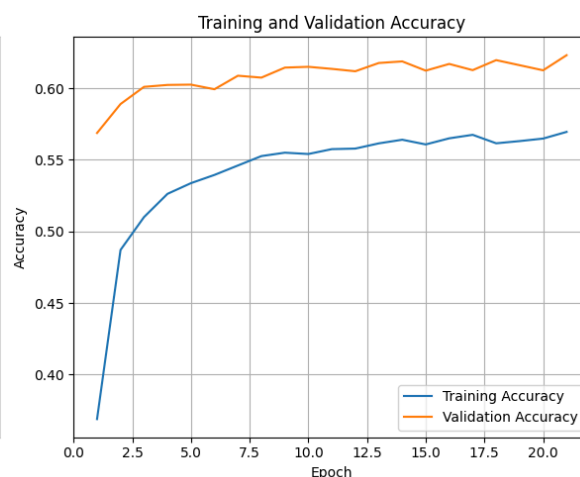
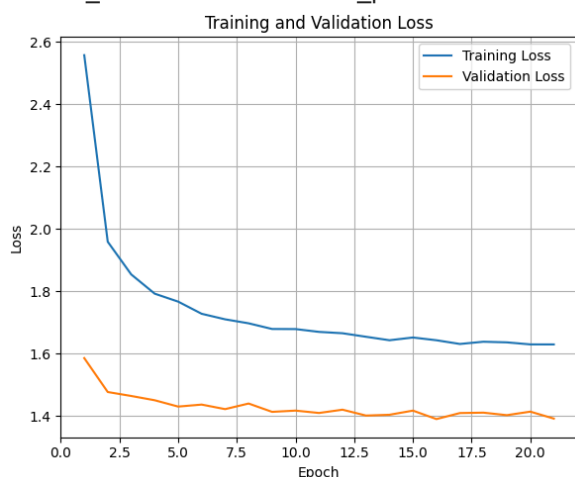
Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 29ms/step - accuracy: 0.2762 - f1_score: 0.2685 - loss: 3.1422 - precision: 0.6164
Epoch 1: val_loss improved from inf to 1.58542, saving model to best_model_v4.keras
657/657 ————— 40s 45ms/step - accuracy: 0.2764 - f1_score: 0.2687 - loss: 3.1413 - precision: 0.6165 - val_accuracy: 0.5688 - val_f1_score: 0.5619 - val_loss: 1.5854 - val_precision: 0.7932
Epoch 2/25
655/657 ————— 0s 24ms/step - accuracy: 0.4750 - f1_score: 0.4679 - loss: 2.0000 - precision: 0.6944
Epoch 2: val_loss improved from 1.58542 to 1.47654, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.4751 - f1_score: 0.4680 - loss: 1.9998 - precision: 0.6944 - val_accuracy: 0.5890 - val_f1_score: 0.5841 - val_loss: 1.4765 - val_precision: 0.8033
Epoch 3/25
655/657 ————— 0s 23ms/step - accuracy: 0.5017 - f1_score: 0.4966 - loss: 1.8824 - precision: 0.7076
Epoch 3: val_loss improved from 1.47654 to 1.46385, saving model to best_model_v4.keras
657/657 ————— 19s 30ms/step - accuracy: 0.5018 - f1_score: 0.4967 - loss: 1.8823 - precision: 0.7077 - val_accuracy: 0.6010 - val_f1_score: 0.5978 - val_loss: 1.4638 - val_precision: 0.7869
Epoch 4/25
655/657 ————— 0s 24ms/step - accuracy: 0.5192 - f1_score: 0.5139 - loss: 1.8167 - precision: 0.7164
Epoch 4: val_loss improved from 1.46385 to 1.45013, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5192 - f1_score: 0.5139 - loss: 1.8166 - precision: 0.7164 - val_accuracy: 0.6023 - val_f1_score: 0.5967 - val_loss: 1.4501 - val_precision: 0.7895
Epoch 5/25
656/657 ————— 0s 24ms/step - accuracy: 0.5285 - f1_score: 0.5240 - loss: 1.7925 - precision: 0.7225
Epoch 5: val_loss improved from 1.45013 to 1.42984, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5285 - f1_score: 0.5240 - loss: 1.7924 - precision: 0.7225 - val_accuracy: 0.6026 - val_f1_score: 0.5975 - val_loss: 1.4298 - val_precision: 0.8003
Epoch 6/25
657/657 ————— 0s 24ms/step - accuracy: 0.5325 - f1_score: 0.5280 - loss: 1.7493 - precision: 0.7210
Epoch 6: val_loss did not improve from 1.42984
657/657 ————— 19s 28ms/step - accuracy: 0.5325 - f1_score: 0.5280 - loss: 1.7493 - precision: 0.7210 - val_accuracy: 0.5993 - val_f1_score: 0.5970 - val_loss: 1.4362 - val_precision: 0.7816
Epoch 7/25
655/657 ————— 0s 24ms/step - accuracy: 0.5405 - f1_score: 0.5368 - loss: 1.7357 - precision: 0.7218
Epoch 7: val_loss improved from 1.42984 to 1.42164, saving model to best_model_v4.keras
657/657 ————— 20s 31ms/step - accuracy: 0.5405 - f1_score: 0.5369 - loss: 1.7356 - precision: 0.7218 - val_accuracy: 0.6088 - val_f1_score: 0.6063 - val_loss: 1.4216 - val_precision: 0.7840
Epoch 8/25
657/657 ————— 0s 24ms/step - accuracy: 0.5481 - f1_score: 0.5443 - loss: 1.7255 - precision: 0.7278
Epoch 8: val_loss did not improve from 1.42164
657/657 ————— 19s 28ms/step - accuracy: 0.5481 - f1_score: 0.5443

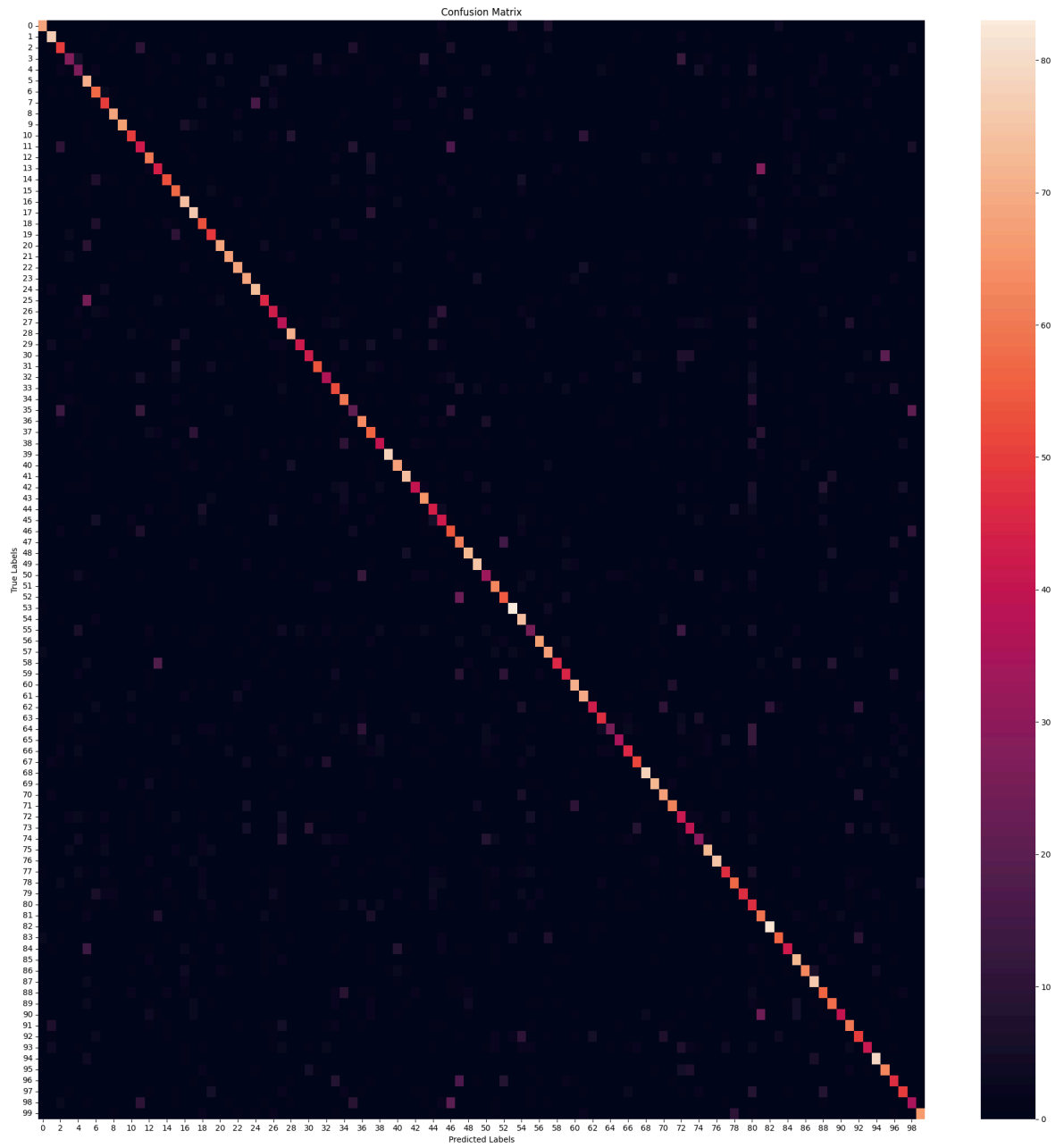
- loss: 1.7255 - precision: 0.7278 - val_accuracy: 0.6074 - val_f1_score: 0.6021
- val_loss: 1.4396 - val_precision: 0.7857
Epoch 9/25
655/657 ————— 0s 24ms/step - accuracy: 0.5481 - f1_score: 0.5435 -
loss: 1.7099 - precision: 0.7276
Epoch 9: val_loss improved from 1.42164 to 1.41289, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5481 - f1_score: 0.5436
- loss: 1.7098 - precision: 0.7276 - val_accuracy: 0.6144 - val_f1_score: 0.6113
- val_loss: 1.4129 - val_precision: 0.7820
Epoch 10/25
657/657 ————— 0s 23ms/step - accuracy: 0.5490 - f1_score: 0.5454 -
loss: 1.7024 - precision: 0.7321
Epoch 10: val_loss did not improve from 1.41289
657/657 ————— 19s 28ms/step - accuracy: 0.5490 - f1_score: 0.5454
- loss: 1.7024 - precision: 0.7321 - val_accuracy: 0.6150 - val_f1_score: 0.6130
- val_loss: 1.4169 - val_precision: 0.7817
Epoch 11/25
655/657 ————— 0s 24ms/step - accuracy: 0.5552 - f1_score: 0.5520 -
loss: 1.6793 - precision: 0.7278
Epoch 11: val_loss improved from 1.41289 to 1.40928, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5552 - f1_score: 0.5520
- loss: 1.6793 - precision: 0.7278 - val_accuracy: 0.6136 - val_f1_score: 0.6145
- val_loss: 1.4093 - val_precision: 0.7781
Epoch 12/25
657/657 ————— 0s 23ms/step - accuracy: 0.5514 - f1_score: 0.5481 -
loss: 1.6850 - precision: 0.7290
Epoch 12: val_loss did not improve from 1.40928
657/657 ————— 19s 28ms/step - accuracy: 0.5514 - f1_score: 0.5481
- loss: 1.6849 - precision: 0.7290 - val_accuracy: 0.6119 - val_f1_score: 0.6086
- val_loss: 1.4198 - val_precision: 0.7812
Epoch 13/25
656/657 ————— 0s 23ms/step - accuracy: 0.5586 - f1_score: 0.5558 -
loss: 1.6696 - precision: 0.7336
Epoch 13: val_loss improved from 1.40928 to 1.40098, saving model to best_model_v4.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5586 - f1_score: 0.5558
- loss: 1.6695 - precision: 0.7337 - val_accuracy: 0.6177 - val_f1_score: 0.6146
- val_loss: 1.4010 - val_precision: 0.7943
Epoch 14/25
656/657 ————— 0s 23ms/step - accuracy: 0.5621 - f1_score: 0.5584 -
loss: 1.6552 - precision: 0.7408
Epoch 14: val_loss did not improve from 1.40098
657/657 ————— 19s 28ms/step - accuracy: 0.5621 - f1_score: 0.5584
- loss: 1.6552 - precision: 0.7408 - val_accuracy: 0.6188 - val_f1_score: 0.6174
- val_loss: 1.4036 - val_precision: 0.7893
Epoch 15/25
656/657 ————— 0s 23ms/step - accuracy: 0.5584 - f1_score: 0.5543 -
loss: 1.6740 - precision: 0.7388
Epoch 15: val_loss did not improve from 1.40098
657/657 ————— 18s 28ms/step - accuracy: 0.5584 - f1_score: 0.5543
- loss: 1.6739 - precision: 0.7388 - val_accuracy: 0.6123 - val_f1_score: 0.6075
- val_loss: 1.4169 - val_precision: 0.7872
Epoch 16/25
657/657 ————— 0s 23ms/step - accuracy: 0.5620 - f1_score: 0.5590 -
loss: 1.6542 - precision: 0.7414
Epoch 16: val_loss improved from 1.40098 to 1.38972, saving model to best_model_v4.keras
657/657 ————— 19s 30ms/step - accuracy: 0.5621 - f1_score: 0.5590

- loss: 1.6542 - precision: 0.7414 - val_accuracy: 0.6170 - val_f1_score: 0.6147
 - val_loss: 1.3897 - val_precision: 0.7913
 Epoch 17/25
657/657 ————— 0s 23ms/step - accuracy: 0.5638 - f1_score: 0.5597 -
 loss: 1.6474 - precision: 0.7386
 Epoch 17: val_loss did not improve from 1.38972
657/657 ————— 18s 28ms/step - accuracy: 0.5638 - f1_score: 0.5597
 - loss: 1.6473 - precision: 0.7386 - val_accuracy: 0.6127 - val_f1_score: 0.6105
 - val_loss: 1.4091 - val_precision: 0.7812
 Epoch 18/25
655/657 ————— 0s 24ms/step - accuracy: 0.5565 - f1_score: 0.5530 -
 loss: 1.6602 - precision: 0.7356
 Epoch 18: val_loss did not improve from 1.38972
657/657 ————— 19s 28ms/step - accuracy: 0.5566 - f1_score: 0.5530
 - loss: 1.6601 - precision: 0.7356 - val_accuracy: 0.6197 - val_f1_score: 0.6181
 - val_loss: 1.4103 - val_precision: 0.7856
 Epoch 19/25
655/657 ————— 0s 24ms/step - accuracy: 0.5589 - f1_score: 0.5551 -
 loss: 1.6532 - precision: 0.7334
 Epoch 19: val_loss did not improve from 1.38972
657/657 ————— 19s 29ms/step - accuracy: 0.5590 - f1_score: 0.5551
 - loss: 1.6531 - precision: 0.7334 - val_accuracy: 0.6161 - val_f1_score: 0.6165
 - val_loss: 1.4023 - val_precision: 0.7833
 Epoch 20/25
657/657 ————— 0s 24ms/step - accuracy: 0.5604 - f1_score: 0.5563 -
 loss: 1.6536 - precision: 0.7403
 Epoch 20: val_loss did not improve from 1.38972
657/657 ————— 19s 29ms/step - accuracy: 0.5604 - f1_score: 0.5563
 - loss: 1.6535 - precision: 0.7403 - val_accuracy: 0.6126 - val_f1_score: 0.6099
 - val_loss: 1.4136 - val_precision: 0.7843
 Epoch 21/25
656/657 ————— 0s 24ms/step - accuracy: 0.5645 - f1_score: 0.5607 -
 loss: 1.6485 - precision: 0.7384
 Epoch 21: val_loss did not improve from 1.38972
657/657 ————— 19s 28ms/step - accuracy: 0.5645 - f1_score: 0.5608
 - loss: 1.6485 - precision: 0.7384 - val_accuracy: 0.6231 - val_f1_score: 0.6216
 - val_loss: 1.3913 - val_precision: 0.7802



141/141  **11s** 41ms/step - accuracy: 0.6112 - f1_score: 0.6034
- loss: 1.4088 - precision: 0.7826
Test Accuracy: 62.06%
Test Loss: 1.3918565511703491
Test Precision: 78.94%
Test F1 Scores (Per Class): [82.63472 78.57142 53.191483 38.88888 33.12883 5
7.72357 58.163254
60.975605 79.768776 83.75 60.975605 45.4054 62.827217 51.764698
66.66666 56.15763 75.12689 75.62189 57.297295 54.748596 79.31035
76.83616 78.40909 75.40983 77.65957 57.142853 48.044685 38.38383
77.59562 59.154922 56.209152 62.7907 45.283012 51.74129 57.971012
27.027023 63.054188 53.58851 52.631577 90.69767 66.666664 81.767944
54.421764 70.65217 48.087425 45.901634 49.76958 55.45454 76.59574
80.64516 39.053246 68.1081 60.109283 86.01036 66.07142 31.446535
77.456635 66.666664 65.24823 56.603767 77.59562 75.675674 58.503395
59.872604 39.682537 50.724636 58.59872 59.64912 86.18784 79.12087
70.83333 72.51461 37.96296 50.980385 36.708855 77.00535 82.872925
54.651157 66.66666 63.94557 31.438122 51.08225 85.86387 69.565216
51.219505 68.57142 76.36363 77.948715 52.830185 60.73298 55.17241
63.157898 54.64481 52.90322 81.025635 67.02127 49.214657 60.24096
39.548016 79.99999]
Average Test F1 Scores:61.97

282/282  **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.90	0.77	0.83	90
dolphin	0.72	0.86	0.78	90
otter	0.51	0.56	0.53	90
seal	0.52	0.31	0.39	90
whale	0.37	0.30	0.33	90
aquarium fish	0.46	0.79	0.58	90
flatfish	0.54	0.63	0.58	90
ray	0.68	0.56	0.61	90
shark	0.83	0.77	0.80	90
trout	0.96	0.74	0.84	90
orchids	0.68	0.56	0.61	90
poppies	0.44	0.47	0.45	90
roses	0.59	0.67	0.63	90
sunflowers	0.55	0.49	0.52	90
tulips	0.77	0.59	0.67	90
bottles	0.50	0.63	0.56	90
bowls	0.69	0.82	0.75	90
cans	0.68	0.84	0.76	90
cups	0.56	0.59	0.57	90
plates	0.55	0.54	0.55	90
apples	0.82	0.77	0.79	90
mushrooms	0.78	0.76	0.77	90
oranges	0.80	0.77	0.78	90
pears	0.74	0.77	0.75	90
sweet peppers	0.74	0.81	0.78	90
clock	0.65	0.51	0.57	90
computer keyboard	0.48	0.48	0.48	90
lamp	0.35	0.42	0.38	90
telephone	0.76	0.79	0.78	90
television	0.81	0.47	0.59	90
bed	0.68	0.48	0.56	90
chair	0.66	0.60	0.63	90
couch	0.52	0.40	0.45	90
table	0.47	0.58	0.52	90
wardrobe	0.51	0.67	0.58	90
bee	0.36	0.23	0.28	90
beetle	0.57	0.71	0.63	90
butterfly	0.47	0.62	0.54	90
caterpillar	0.65	0.44	0.53	90
cockroach	0.95	0.87	0.91	90
bear	0.60	0.76	0.67	90
leopard	0.81	0.82	0.82	90
lion	0.70	0.44	0.54	90
tiger	0.69	0.72	0.71	90
wolf	0.48	0.49	0.48	90
bridge	0.45	0.47	0.46	90
castle	0.43	0.60	0.50	90
house	0.47	0.68	0.55	90
road	0.73	0.80	0.77	90
skyscraper	0.78	0.83	0.81	90
cloud	0.42	0.37	0.39	90
forest	0.66	0.70	0.68	90
mountain	0.59	0.61	0.60	90
plain	0.81	0.92	0.86	90
sea	0.55	0.82	0.66	90
camel	0.36	0.28	0.31	90
cattle	0.81	0.74	0.77	90
chimpanzee	0.60	0.76	0.67	90

elephant	0.90	0.51	0.65	90
kangaroo	0.65	0.50	0.57	90
fox	0.76	0.79	0.78	90
porcupine	0.74	0.78	0.76	90
possum	0.75	0.48	0.59	90
raccoon	0.70	0.52	0.60	90
skunk	0.69	0.28	0.40	90
crab	0.73	0.39	0.51	90
lobster	0.69	0.51	0.59	90
snail	0.63	0.57	0.60	90
spider	0.86	0.87	0.86	90
worm	0.78	0.80	0.79	90
baby	0.67	0.76	0.71	90
boy	0.77	0.69	0.73	90
girl	0.33	0.46	0.38	90
man	0.62	0.43	0.51	90
woman	0.43	0.32	0.37	90
crocodile	0.74	0.80	0.77	90
dinosaur	0.82	0.83	0.83	90
lizard	0.57	0.52	0.55	90
snake	0.70	0.63	0.67	90
turtle	0.82	0.52	0.64	90
hamster	0.22	0.52	0.31	90
mouse	0.42	0.66	0.51	90
rabbit	0.81	0.91	0.86	90
shrew	0.79	0.62	0.70	90
squirrel	0.57	0.47	0.51	90
maple	0.60	0.80	0.69	90
oak	0.84	0.70	0.76	90
palm	0.72	0.84	0.78	90
pine	0.46	0.62	0.53	90
willow	0.57	0.64	0.61	90
bicycle	0.73	0.44	0.55	90
bus	0.60	0.67	0.63	90
motorcycle	0.54	0.56	0.55	90
pickup truck	0.63	0.46	0.53	90
train	0.75	0.88	0.81	90
lawn-mower	0.64	0.70	0.67	90
rocket	0.46	0.52	0.49	90
streetcar	0.66	0.56	0.60	90
tank	0.41	0.39	0.40	90
tractor	0.88	0.73	0.80	90
accuracy			0.62	9000
macro avg	0.64	0.62	0.62	9000
weighted avg	0.64	0.62	0.62	9000

Model: "sequential_8"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_8 (GlobalAveragePooling2D)	(None , 2048)
dense_16 (Dense)	(None , 512)
dropout_8 (Dropout)	(None , 512)
dense_17 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 27ms/step - accuracy: 0.6100 - f1_score: 0.6077
- loss: 1.4310 - precision: 0.7855
Epoch 1: val_loss improved from inf to 1.28182, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 56s 35ms/step - accuracy: 0.6100 - f1_score: 0.6077 - loss: 1.4309 - precision: 0.7855 - val_accuracy: 0.6499 - val_f1_score: 0.6481 - val_loss: 1.2818 - val_precision: 0.8170
Epoch 2/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6432 - f1_score: 0.6413
- loss: 1.2689 - precision: 0.8175
Epoch 2: val_loss improved from 1.28182 to 1.25551, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6432 - f1_score: 0.6413 - loss: 1.2689 - precision: 0.8175 - val_accuracy: 0.6542 - val_f1_score: 0.6526 - val_loss: 1.2555 - val_precision: 0.8148
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6607 - f1_score: 0.6593
- loss: 1.1983 - precision: 0.8259
Epoch 3: val_loss improved from 1.25551 to 1.23485, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6607 - f1_score: 0.6593 - loss: 1.1983 - precision: 0.8259 - val_accuracy: 0.6588 - val_f1_score: 0.6571 - val_loss: 1.2349 - val_precision: 0.8164
Epoch 4/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6732 - f1_score: 0.6715
- loss: 1.1375 - precision: 0.8314
Epoch 4: val_loss improved from 1.23485 to 1.22245, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6732 - f1_score: 0.6715 - loss: 1.1375 - precision: 0.8314 - val_accuracy: 0.6614 - val_f1_score: 0.6601 - val_loss: 1.2225 - val_precision: 0.8166
Epoch 5/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6833 - f1_score: 0.6817
- loss: 1.0859 - precision: 0.8395
Epoch 5: val_loss improved from 1.22245 to 1.21038, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6833 - f1_score: 0.6817 - loss: 1.0859 - precision: 0.8395 - val_accuracy: 0.6591 - val_f1_score: 0.6579 - val_loss: 1.2104 - val_precision: 0.8147
Epoch 6/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6940 - f1_score: 0.6925
- loss: 1.0506 - precision: 0.8468
Epoch 6: val_loss improved from 1.21038 to 1.20554, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6939 - f1_score: 0.6925 - loss: 1.0507 - precision: 0.8468 - val_accuracy: 0.6616 - val_f1_score: 0.6603 - val_loss: 1.2055 - val_precision: 0.8164
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7047 - f1_score: 0.7031
- loss: 1.0142 - precision: 0.8505
Epoch 7: val_loss improved from 1.20554 to 1.19861, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7047 - f1_score: 0.7031 - loss: 1.0142 - precision: 0.8505 - val_accuracy: 0.6653 - val_f1_score: 0.6637 - val_loss: 1.1986 - val_precision: 0.8151
Epoch 8/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7087 - f1_score: 0.7073
- loss: 0.9909 - precision: 0.8543
Epoch 8: val_loss improved from 1.19861 to 1.19184, saving model to best_model_fine_tune_v4.keras

```
ne_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7087 - f1_score: 0.707
3 - loss: 0.9909 - precision: 0.8543 - val_accuracy: 0.6648 - val_f1_score: 0.663
7 - val_loss: 1.1918 - val_precision: 0.8172
Epoch 9/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7191 - f1_score: 0.7179
- loss: 0.9566 - precision: 0.8611
Epoch 9: val_loss improved from 1.19184 to 1.18829, saving model to best_model_fi
ne_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7191 - f1_score: 0.717
9 - loss: 0.9567 - precision: 0.8611 - val_accuracy: 0.6662 - val_f1_score: 0.665
5 - val_loss: 1.1883 - val_precision: 0.8164
Epoch 10/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7256 - f1_score: 0.7243
- loss: 0.9317 - precision: 0.8615
Epoch 10: val_loss improved from 1.18829 to 1.18112, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7255 - f1_score: 0.724
3 - loss: 0.9318 - precision: 0.8615 - val_accuracy: 0.6663 - val_f1_score: 0.664
9 - val_loss: 1.1811 - val_precision: 0.8192
Epoch 11/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7291 - f1_score: 0.7275
- loss: 0.9082 - precision: 0.8663
Epoch 11: val_loss improved from 1.18112 to 1.18020, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7291 - f1_score: 0.727
5 - loss: 0.9082 - precision: 0.8663 - val_accuracy: 0.6677 - val_f1_score: 0.666
5 - val_loss: 1.1802 - val_precision: 0.8167
Epoch 12/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7325 - f1_score: 0.7310
- loss: 0.8915 - precision: 0.8659
Epoch 12: val_loss improved from 1.18020 to 1.18009, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7324 - f1_score: 0.730
9 - loss: 0.8916 - precision: 0.8659 - val_accuracy: 0.6653 - val_f1_score: 0.664
6 - val_loss: 1.1801 - val_precision: 0.8134
Epoch 13/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7436 - f1_score: 0.7424
- loss: 0.8633 - precision: 0.8750
Epoch 13: val_loss improved from 1.18009 to 1.17463, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7435 - f1_score: 0.742
4 - loss: 0.8634 - precision: 0.8750 - val_accuracy: 0.6700 - val_f1_score: 0.668
6 - val_loss: 1.1746 - val_precision: 0.8152
Epoch 14/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7497 - f1_score: 0.7486
- loss: 0.8429 - precision: 0.8737
Epoch 14: val_loss improved from 1.17463 to 1.17062, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7497 - f1_score: 0.748
6 - loss: 0.8430 - precision: 0.8737 - val_accuracy: 0.6717 - val_f1_score: 0.670
5 - val_loss: 1.1706 - val_precision: 0.8143
Epoch 15/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7524 - f1_score: 0.7511
- loss: 0.8237 - precision: 0.8746
Epoch 15: val_loss did not improve from 1.17062
1314/1314 ————— 35s 26ms/step - accuracy: 0.7523 - f1_score: 0.751
1 - loss: 0.8238 - precision: 0.8746 - val_accuracy: 0.6690 - val_f1_score: 0.667
5 - val_loss: 1.1738 - val_precision: 0.8106
```



141/141 ————— **11s** 38ms/step - accuracy: 0.6661 - f1_score: 0.6584

- loss: 1.1890 - precision: 0.8043

Test Accuracy: 67.43%

Test Loss: 1.1645960807800293

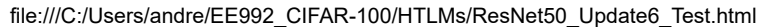
Test Precision: 81.45%

Test F1 Scores (Per Class): [85.39326 79.787224 60.396034 48.128338 36.90476 69.005844 65.24063

68.852455	80.23255	83.236984	64.77273	50.279324	67.0391	61.6279
69.87952	62.222218	75.89743	77.83783	62.569828	58.757053	84.153
79.55801	80.64516	80.89887	82.7957	70.05649	56.818176	48.2412
80.22598	70.731705	64.28571	74.117645	50.306744	59.487175	68.852455
33.734936	71.91011	58.378376	56.140347	90.80459	69.84126	84.91619
63.5838	73.33333	48.543686	53.191483	48.235287	61.72839	84.61538
83.9779	44.808735	77.09496	68.04123	90.60773	75.62189	37.349392
82.75862	73.40425	81.21211	59.550552	80.681816	74.444435	66.29834
70.23808	44.318172	53.012043	62.365585	64.36781	87.91208	85.86956
75.14451	76.43979	42.62294	61.458332	38.297867	81.91489	86.18784
60.227264	67.03297	67.796616	38.50267	57.303364	91.489365	73.79678
62.14689	74.19354	85.057465	80.22598	63.5838	66.321236	62.5641
70.39106	56.28742	61.349686	84.491974	71.957664	55.208332	60.439552
44.318172	79.99999					

Average Test F1 Scores:67.39

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.86	0.84	0.85	90
dolphin	0.77	0.83	0.80	90
otter	0.54	0.68	0.60	90
seal	0.46	0.50	0.48	90
whale	0.40	0.34	0.37	90
aquarium fish	0.73	0.66	0.69	90
flatfish	0.63	0.68	0.65	90
ray	0.68	0.70	0.69	90
shark	0.84	0.77	0.80	90
trout	0.87	0.80	0.83	90
orchids	0.66	0.63	0.65	90
poppies	0.51	0.50	0.50	90
roses	0.67	0.67	0.67	90
sunflowers	0.65	0.59	0.62	90
tulips	0.76	0.64	0.70	90
bottles	0.62	0.62	0.62	90
bowls	0.71	0.83	0.77	90
cans	0.76	0.80	0.78	90
cups	0.63	0.62	0.63	90
plates	0.60	0.58	0.59	90
apples	0.83	0.86	0.84	90
mushrooms	0.79	0.80	0.80	90
oranges	0.78	0.83	0.81	90
pears	0.82	0.80	0.81	90
sweet peppers	0.80	0.86	0.83	90
clock	0.71	0.69	0.70	90
computer keyboard	0.58	0.56	0.57	90
lamp	0.44	0.53	0.48	90
telephone	0.83	0.79	0.81	90
television	0.78	0.64	0.71	90
bed	0.68	0.60	0.64	90
chair	0.79	0.70	0.74	90
couch	0.56	0.46	0.50	90
table	0.55	0.64	0.59	90
wardrobe	0.68	0.70	0.69	90
bee	0.37	0.31	0.34	90
beetle	0.73	0.71	0.72	90
butterfly	0.57	0.60	0.58	90
caterpillar	0.59	0.53	0.56	90
cockroach	0.94	0.88	0.91	90
bear	0.67	0.73	0.70	90
leopard	0.85	0.84	0.85	90
lion	0.66	0.61	0.64	90
tiger	0.73	0.73	0.73	90
wolf	0.43	0.56	0.49	90
bridge	0.51	0.54	0.52	90
castle	0.51	0.46	0.48	90
house	0.69	0.56	0.62	90
road	0.84	0.86	0.85	90
skyscraper	0.84	0.84	0.84	90
cloud	0.44	0.46	0.45	90
forest	0.78	0.77	0.77	90
mountain	0.63	0.73	0.68	90
plain	0.90	0.91	0.91	90
sea	0.68	0.84	0.76	90
camel	0.41	0.34	0.38	90
cattle	0.86	0.80	0.83	90
chimpanzee	0.70	0.77	0.73	90

elephant	0.89	0.74	0.81	90
kangaroo	0.60	0.58	0.59	90
fox	0.83	0.79	0.81	90
porcupine	0.74	0.74	0.74	90
possum	0.66	0.67	0.66	90
raccoon	0.76	0.66	0.70	90
skunk	0.45	0.43	0.44	90
crab	0.58	0.49	0.53	90
lobster	0.60	0.64	0.62	90
snail	0.67	0.62	0.64	90
spider	0.87	0.89	0.88	90
worm	0.84	0.88	0.86	90
baby	0.78	0.72	0.75	90
boy	0.72	0.81	0.76	90
girl	0.42	0.43	0.43	90
man	0.58	0.66	0.61	90
woman	0.37	0.40	0.38	90
crocodile	0.79	0.86	0.82	90
dinosaur	0.86	0.87	0.86	90
lizard	0.62	0.59	0.60	90
snake	0.66	0.68	0.67	90
turtle	0.68	0.67	0.67	90
hamster	0.37	0.40	0.39	90
mouse	0.58	0.57	0.57	90
rabbit	0.88	0.96	0.91	90
shrew	0.71	0.77	0.74	90
squirrel	0.63	0.61	0.62	90
maple	0.72	0.77	0.74	90
oak	0.88	0.82	0.85	90
palm	0.82	0.79	0.80	90
pine	0.66	0.61	0.64	90
willow	0.62	0.71	0.66	90
bicycle	0.58	0.68	0.63	90
bus	0.71	0.70	0.70	90
motorcycle	0.61	0.52	0.56	90
pickup truck	0.68	0.56	0.61	90
train	0.81	0.88	0.84	90
lawn-mower	0.69	0.76	0.72	90
rocket	0.52	0.59	0.55	90
streetcar	0.60	0.61	0.60	90
tank	0.45	0.43	0.44	90
tractor	0.85	0.76	0.80	90
accuracy			0.67	9000
macro avg	0.68	0.67	0.67	9000
weighted avg	0.68	0.67	0.67	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

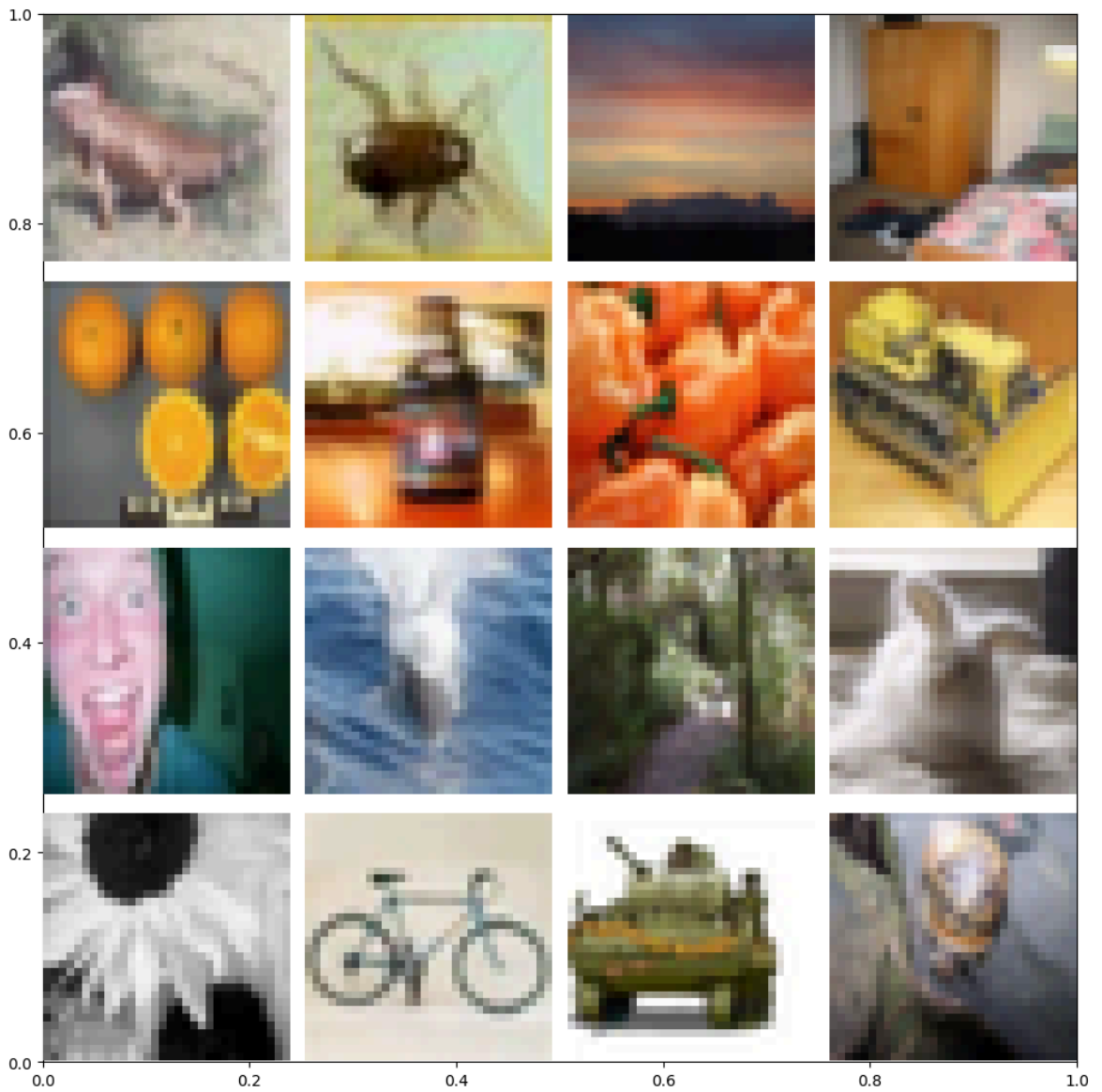
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

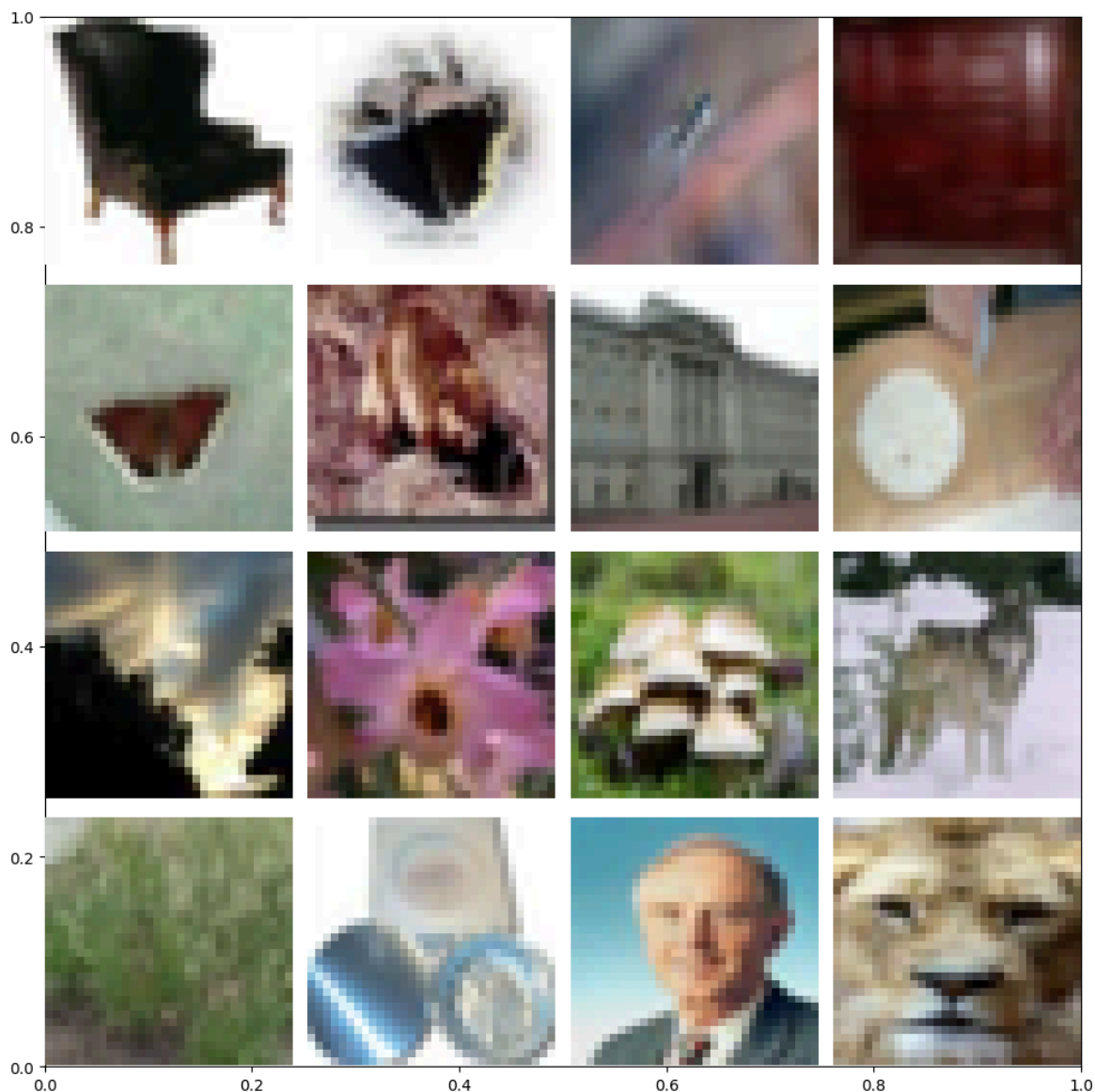
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[24968].shape: (1,)  
imgs[24968].shape: (32, 32, 3)
```

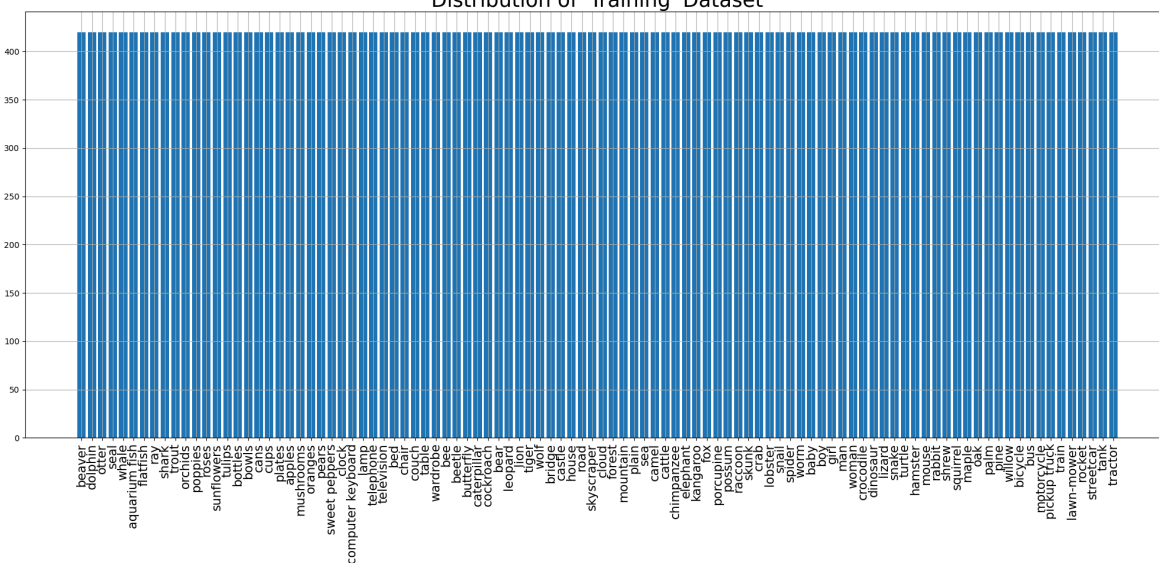


```
labels[12320].shape: (1,)  
imgs[12320].shape: (32, 32, 3)
```

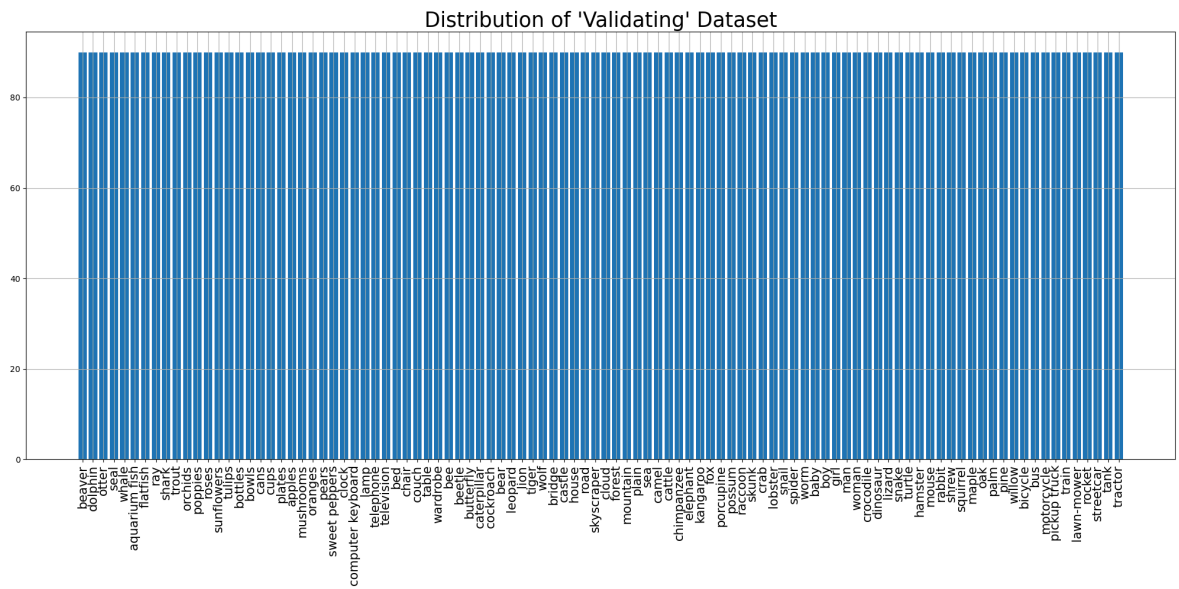


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

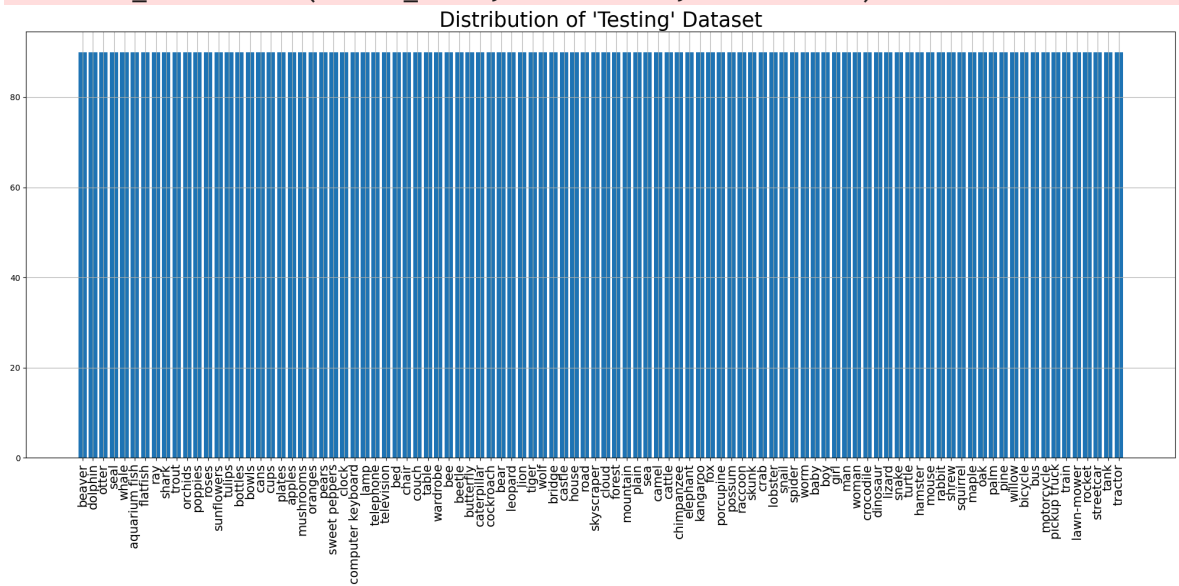
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_9"

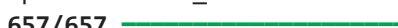
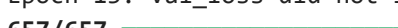
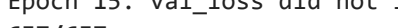
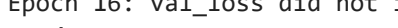
Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_9 (GlobalAveragePooling2D)	(None , 2048)
dense_18 (Dense)	(None , 512)
dropout_9 (Dropout)	(None , 512)
dense_19 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— **0s** 28ms/step - accuracy: 0.2721 - f1_score: 0.2655 - loss: 3.1557 - precision: 0.6142
Epoch 1: val_loss improved from inf to 1.56750, saving model to best_model_v4.keras
657/657 ————— **39s** 44ms/step - accuracy: 0.2723 - f1_score: 0.2656 - loss: 3.1548 - precision: 0.6143 - val_accuracy: 0.5660 - val_f1_score: 0.5601 - val_loss: 1.5675 - val_precision: 0.7931
Epoch 2/25
656/657 ————— **0s** 24ms/step - accuracy: 0.4725 - f1_score: 0.4667 - loss: 2.0085 - precision: 0.6934
Epoch 2: val_loss improved from 1.56750 to 1.48507, saving model to best_model_v4.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.4725 - f1_score: 0.4667 - loss: 2.0083 - precision: 0.6934 - val_accuracy: 0.5909 - val_f1_score: 0.5878 - val_loss: 1.4851 - val_precision: 0.7985
Epoch 3/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5036 - f1_score: 0.4986 - loss: 1.8881 - precision: 0.7156
Epoch 3: val_loss improved from 1.48507 to 1.45347, saving model to best_model_v4.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5037 - f1_score: 0.4986 - loss: 1.8880 - precision: 0.7156 - val_accuracy: 0.5983 - val_f1_score: 0.5995 - val_loss: 1.4535 - val_precision: 0.7921
Epoch 4/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5220 - f1_score: 0.5175 - loss: 1.8204 - precision: 0.7173
Epoch 4: val_loss did not improve from 1.45347
657/657 ————— **19s** 28ms/step - accuracy: 0.5220 - f1_score: 0.5176 - loss: 1.8204 - precision: 0.7173 - val_accuracy: 0.6001 - val_f1_score: 0.5963 - val_loss: 1.4574 - val_precision: 0.7878
Epoch 5/25
656/657 ————— **0s** 23ms/step - accuracy: 0.5285 - f1_score: 0.5240 - loss: 1.7856 - precision: 0.7188
Epoch 5: val_loss improved from 1.45347 to 1.43406, saving model to best_model_v4.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5285 - f1_score: 0.5240 - loss: 1.7855 - precision: 0.7188 - val_accuracy: 0.6123 - val_f1_score: 0.6071 - val_loss: 1.4341 - val_precision: 0.7835
Epoch 6/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5289 - f1_score: 0.5245 - loss: 1.7669 - precision: 0.7201
Epoch 6: val_loss improved from 1.43406 to 1.41831, saving model to best_model_v4.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5290 - f1_score: 0.5245 - loss: 1.7668 - precision: 0.7201 - val_accuracy: 0.6176 - val_f1_score: 0.6151 - val_loss: 1.4183 - val_precision: 0.7875
Epoch 7/25
656/657 ————— **0s** 23ms/step - accuracy: 0.5362 - f1_score: 0.5315 - loss: 1.7454 - precision: 0.7288
Epoch 7: val_loss did not improve from 1.41831
657/657 ————— **18s** 28ms/step - accuracy: 0.5362 - f1_score: 0.5315 - loss: 1.7453 - precision: 0.7288 - val_accuracy: 0.6061 - val_f1_score: 0.6018 - val_loss: 1.4373 - val_precision: 0.7796
Epoch 8/25
657/657 ————— **0s** 23ms/step - accuracy: 0.5428 - f1_score: 0.5384 - loss: 1.7201 - precision: 0.7267
Epoch 8: val_loss improved from 1.41831 to 1.41213, saving model to best_model_v4.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5428 - f1_score: 0.5384

- loss: 1.7201 - precision: 0.7267 - val_accuracy: 0.6132 - val_f1_score: 0.6101
- val_loss: 1.4121 - val_precision: 0.7936
Epoch 9/25
656/657  0s 23ms/step - accuracy: 0.5452 - f1_score: 0.5410 -
loss: 1.7075 - precision: 0.7312
Epoch 9: val_loss improved from 1.41213 to 1.39927, saving model to best_model_v4.keras
657/657  19s 30ms/step - accuracy: 0.5452 - f1_score: 0.5410
- loss: 1.7074 - precision: 0.7312 - val_accuracy: 0.6143 - val_f1_score: 0.6113
- val_loss: 1.3993 - val_precision: 0.7924
Epoch 10/25
655/657  0s 23ms/step - accuracy: 0.5480 - f1_score: 0.5441 -
loss: 1.7008 - precision: 0.7344
Epoch 10: val_loss did not improve from 1.39927
657/657  18s 28ms/step - accuracy: 0.5480 - f1_score: 0.5442
- loss: 1.7007 - precision: 0.7345 - val_accuracy: 0.6120 - val_f1_score: 0.6086
- val_loss: 1.4151 - val_precision: 0.7969
Epoch 11/25
655/657  0s 23ms/step - accuracy: 0.5508 - f1_score: 0.5466 -
loss: 1.6942 - precision: 0.7327
Epoch 11: val_loss did not improve from 1.39927
657/657  18s 28ms/step - accuracy: 0.5508 - f1_score: 0.5466
- loss: 1.6942 - precision: 0.7328 - val_accuracy: 0.6084 - val_f1_score: 0.6055
- val_loss: 1.4151 - val_precision: 0.7904
Epoch 12/25
655/657  0s 23ms/step - accuracy: 0.5555 - f1_score: 0.5515 -
loss: 1.6815 - precision: 0.7399
Epoch 12: val_loss did not improve from 1.39927
657/657  18s 28ms/step - accuracy: 0.5555 - f1_score: 0.5516
- loss: 1.6815 - precision: 0.7399 - val_accuracy: 0.6209 - val_f1_score: 0.6170
- val_loss: 1.4051 - val_precision: 0.7925
Epoch 13/25
657/657  0s 23ms/step - accuracy: 0.5555 - f1_score: 0.5519 -
loss: 1.6693 - precision: 0.7350
Epoch 13: val_loss did not improve from 1.39927
657/657  18s 28ms/step - accuracy: 0.5555 - f1_score: 0.5519
- loss: 1.6693 - precision: 0.7350 - val_accuracy: 0.6157 - val_f1_score: 0.6143
- val_loss: 1.4007 - val_precision: 0.7865
Epoch 14/25
656/657  0s 23ms/step - accuracy: 0.5544 - f1_score: 0.5506 -
loss: 1.6711 - precision: 0.7324
Epoch 14: val_loss improved from 1.39927 to 1.39693, saving model to best_model_v4.keras
657/657  19s 30ms/step - accuracy: 0.5544 - f1_score: 0.5507
- loss: 1.6711 - precision: 0.7324 - val_accuracy: 0.6174 - val_f1_score: 0.6140
- val_loss: 1.3969 - val_precision: 0.7956
Epoch 15/25
656/657  0s 23ms/step - accuracy: 0.5600 - f1_score: 0.5562 -
loss: 1.6575 - precision: 0.7373
Epoch 15: val_loss did not improve from 1.39693
657/657  18s 28ms/step - accuracy: 0.5600 - f1_score: 0.5562
- loss: 1.6575 - precision: 0.7373 - val_accuracy: 0.6221 - val_f1_score: 0.6164
- val_loss: 1.4002 - val_precision: 0.7867
Epoch 16/25
656/657  0s 23ms/step - accuracy: 0.5567 - f1_score: 0.5531 -
loss: 1.6783 - precision: 0.7357
Epoch 16: val_loss did not improve from 1.39693
657/657  19s 28ms/step - accuracy: 0.5567 - f1_score: 0.5531
- loss: 1.6783 - precision: 0.7357 - val_accuracy: 0.6127 - val_f1_score: 0.6081
- val_loss: 1.4142 - val_precision: 0.7760

Epoch 17/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5572 - f1_score: 0.5536 - loss: 1.6683 - precision: 0.7392

Epoch 17: val_loss did not improve from 1.39693

657/657 ————— **18s** 28ms/step - accuracy: 0.5572 - f1_score: 0.5536 - loss: 1.6683 - precision: 0.7392 - val_accuracy: 0.6130 - val_f1_score: 0.6086 - val_loss: 1.4030 - val_precision: 0.7941

Epoch 18/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5571 - f1_score: 0.5532 - loss: 1.6569 - precision: 0.7361

Epoch 18: val_loss did not improve from 1.39693

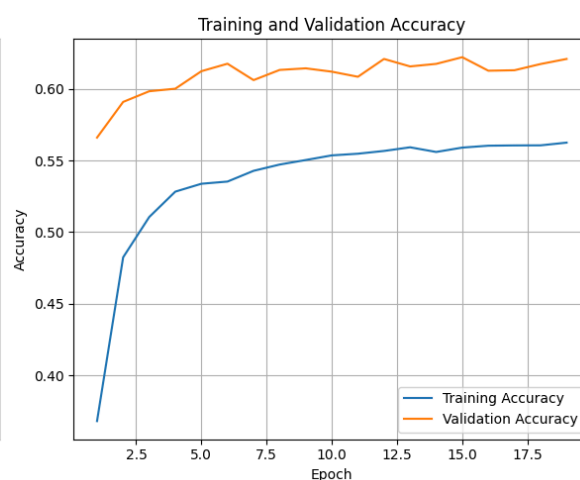
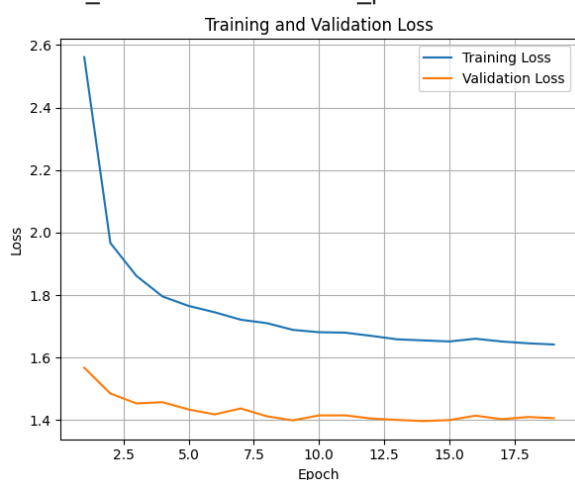
657/657 ————— **18s** 28ms/step - accuracy: 0.5572 - f1_score: 0.5532 - loss: 1.6568 - precision: 0.7361 - val_accuracy: 0.6173 - val_f1_score: 0.6140 - val_loss: 1.4099 - val_precision: 0.7898

Epoch 19/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5606 - f1_score: 0.5571 - loss: 1.6539 - precision: 0.7367

Epoch 19: val_loss did not improve from 1.39693

657/657 ————— **18s** 28ms/step - accuracy: 0.5606 - f1_score: 0.5572 - loss: 1.6538 - precision: 0.7367 - val_accuracy: 0.6209 - val_f1_score: 0.6174 - val_loss: 1.4063 - val_precision: 0.7856



141/141 ————— **11s** 38ms/step - accuracy: 0.6057 - f1_score: 0.5967 - loss: 1.4325 - precision: 0.7854

Test Accuracy: 61.70%

Test Loss: 1.408251404762268

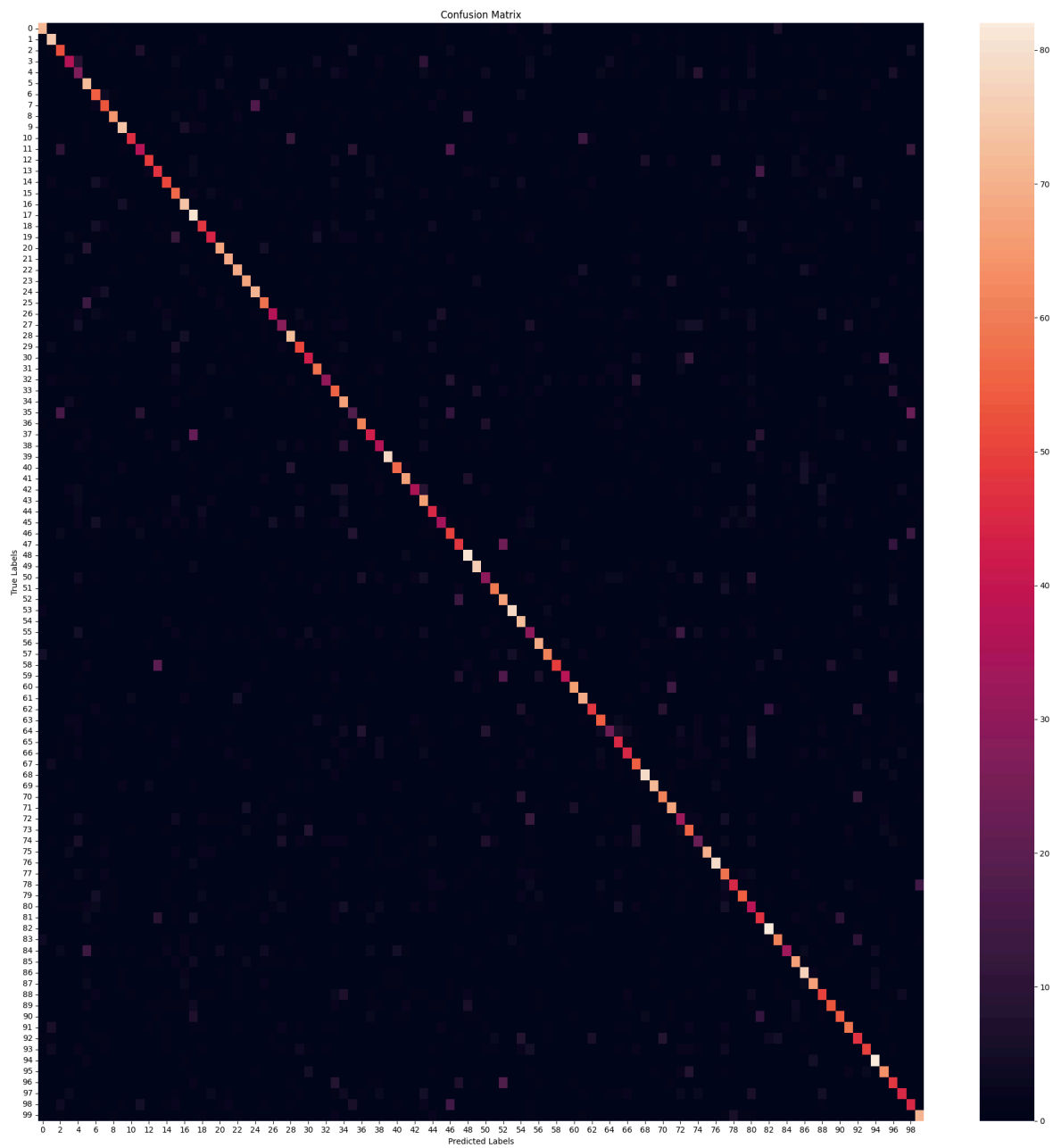
Test Precision: 79.50%

Test F1 Scores (Per Class): [81.81818 77.31957 54.736835 46.249996 28.260866 64.60175 57.591618

61.271667	76.92307	82.68156	61.33333	48.05194	62.33766	54.02298
59.999992	52.054794	74.371864	67.21992	54.545456	57.14285	76.83616
75.40983	76.24309	76.40449	75.93583	63.73626	49.66443	35.89743
75.64766	58.620686	52.499992	67.05202	43.165462	51.37614	58.771927
25.531908	69.31818	52.760727	43.529408	89.65517	65.89595	78.362564
51.470585	58.928566	52.023117	47.55244	49.261078	54.857136	76.777245
79.166664	36.249996	67.42856	60.36036	86.666664	66.666664	31.693987
73.79678	68.53932	69.50355	53.521126	76.74419	71.50259	63.576157
62.499992	36.923073	50.847454	58.064507	56.122444	83.15788	82.75862
65.94594	69.74358	35.35911	60.869556	28.048775	82.84023	82.291664
51.55555	59.602642	66.26506	30.522083	49.999992	84.53608	69.71428
48.17518	69.791664	71.962616	75.706215	51.282047	65.03067	61.714287
66.29213	48.958324	59.880234	81.592026	66.666664	44.85981	56.603767
45.45454	74.34555					

Average Test F1 Scores:61.37

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.84	0.80	0.82	90
dolphin	0.72	0.83	0.77	90
otter	0.52	0.58	0.55	90
seal	0.54	0.41	0.47	90
whale	0.27	0.29	0.28	90
aquarium fish	0.54	0.81	0.65	90
flatfish	0.54	0.61	0.58	90
ray	0.64	0.59	0.61	90
shark	0.82	0.72	0.77	90
trout	0.83	0.82	0.83	90
orchids	0.77	0.51	0.61	90
poppies	0.58	0.41	0.48	90
roses	0.75	0.54	0.63	90
sunflowers	0.56	0.52	0.54	90
tulips	0.64	0.57	0.60	90
bottles	0.44	0.63	0.52	90
bowls	0.68	0.82	0.74	90
cans	0.54	0.90	0.67	90
cups	0.56	0.53	0.55	90
plates	0.69	0.49	0.57	90
apples	0.78	0.76	0.77	90
mushrooms	0.74	0.77	0.75	90
oranges	0.76	0.77	0.76	90
pears	0.77	0.76	0.76	90
sweet peppers	0.73	0.79	0.76	90
clock	0.63	0.64	0.64	90
computer keyboard	0.63	0.41	0.50	90
lamp	0.42	0.31	0.36	90
telephone	0.71	0.81	0.76	90
television	0.61	0.57	0.59	90
bed	0.60	0.47	0.53	90
chair	0.70	0.64	0.67	90
couch	0.61	0.33	0.43	90
table	0.44	0.62	0.51	90
wardrobe	0.49	0.74	0.59	90
bee	0.35	0.20	0.26	90
beetle	0.71	0.68	0.69	90
butterfly	0.59	0.48	0.53	90
caterpillar	0.46	0.41	0.44	90
cockroach	0.93	0.87	0.90	90
bear	0.69	0.63	0.66	90
leopard	0.83	0.74	0.78	90
lion	0.76	0.39	0.51	90
tiger	0.49	0.73	0.59	90
wolf	0.54	0.50	0.52	90
bridge	0.64	0.38	0.48	90
castle	0.44	0.56	0.49	90
house	0.56	0.53	0.55	90
road	0.67	0.90	0.77	90
skyscraper	0.75	0.84	0.79	90
cloud	0.41	0.32	0.36	90
forest	0.69	0.66	0.67	90
mountain	0.51	0.74	0.61	90
plain	0.87	0.87	0.87	90
sea	0.57	0.81	0.67	90
camel	0.31	0.32	0.32	90
cattle	0.72	0.77	0.74	90
chimpanzee	0.69	0.68	0.69	90

elephant	0.96	0.54	0.70	90
kangaroo	0.73	0.42	0.54	90
fox	0.80	0.73	0.77	90
porcupine	0.67	0.77	0.72	90
possum	0.79	0.53	0.64	90
raccoon	0.64	0.61	0.62	90
skunk	0.60	0.27	0.37	90
crab	0.52	0.50	0.51	90
lobster	0.69	0.50	0.58	90
snail	0.52	0.61	0.56	90
spider	0.79	0.88	0.83	90
worm	0.86	0.80	0.83	90
baby	0.64	0.68	0.66	90
boy	0.65	0.76	0.70	90
girl	0.35	0.36	0.35	90
man	0.60	0.62	0.61	90
woman	0.31	0.26	0.28	90
crocodile	0.89	0.78	0.83	90
dinosaur	0.77	0.88	0.82	90
lizard	0.43	0.64	0.52	90
snake	0.74	0.50	0.60	90
turtle	0.72	0.61	0.66	90
hamster	0.24	0.42	0.31	90
mouse	0.48	0.52	0.50	90
rabbit	0.79	0.91	0.85	90
shrew	0.72	0.68	0.70	90
squirrel	0.70	0.37	0.48	90
maple	0.66	0.74	0.70	90
oak	0.62	0.86	0.72	90
palm	0.77	0.74	0.76	90
pine	0.48	0.56	0.51	90
willow	0.73	0.59	0.65	90
bicycle	0.64	0.60	0.62	90
bus	0.67	0.66	0.66	90
motorcycle	0.46	0.52	0.49	90
pickup truck	0.65	0.56	0.60	90
train	0.74	0.91	0.82	90
lawn-mower	0.63	0.71	0.67	90
rocket	0.38	0.53	0.45	90
streetcar	0.65	0.50	0.57	90
tank	0.42	0.50	0.45	90
tractor	0.70	0.79	0.74	90
accuracy			0.62	9000
macro avg	0.63	0.62	0.61	9000
weighted avg	0.63	0.62	0.61	9000

Model: "sequential_9"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_9 (GlobalAveragePooling2D)	(None , 2048)
dense_18 (Dense)	(None , 512)
dropout_9 (Dropout)	(None , 512)
dense_19 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 27ms/step - accuracy: 0.6054 - f1_score: 0.6028
- loss: 1.4478 - precision: 0.7844
Epoch 1: val_loss improved from inf to 1.29269, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.6054 - f1_score: 0.6028 - loss: 1.4478 - precision: 0.7844 - val_accuracy: 0.6467 - val_f1_score: 0.6458 - val_loss: 1.2927 - val_precision: 0.8163
Epoch 2/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6391 - f1_score: 0.6375 - loss: 1.2912 - precision: 0.8131
Epoch 2: val_loss improved from 1.29269 to 1.26516, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6391 - f1_score: 0.6375 - loss: 1.2912 - precision: 0.8131 - val_accuracy: 0.6500 - val_f1_score: 0.6488 - val_loss: 1.2652 - val_precision: 0.8139
Epoch 3/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6563 - f1_score: 0.6548 - loss: 1.2154 - precision: 0.8221
Epoch 3: val_loss improved from 1.26516 to 1.24737, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6563 - f1_score: 0.6548 - loss: 1.2154 - precision: 0.8221 - val_accuracy: 0.6529 - val_f1_score: 0.6512 - val_loss: 1.2474 - val_precision: 0.8183
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6679 - f1_score: 0.6664 - loss: 1.1519 - precision: 0.8312
Epoch 4: val_loss improved from 1.24737 to 1.23324, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6679 - f1_score: 0.6664 - loss: 1.1519 - precision: 0.8312 - val_accuracy: 0.6547 - val_f1_score: 0.6541 - val_loss: 1.2332 - val_precision: 0.8190
Epoch 5/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6816 - f1_score: 0.6804 - loss: 1.1078 - precision: 0.8417
Epoch 5: val_loss improved from 1.23324 to 1.22118, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6816 - f1_score: 0.6804 - loss: 1.1078 - precision: 0.8417 - val_accuracy: 0.6567 - val_f1_score: 0.6553 - val_loss: 1.2212 - val_precision: 0.8189
Epoch 6/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6888 - f1_score: 0.6876 - loss: 1.0697 - precision: 0.8447
Epoch 6: val_loss improved from 1.22118 to 1.21481, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6888 - f1_score: 0.6875 - loss: 1.0697 - precision: 0.8447 - val_accuracy: 0.6584 - val_f1_score: 0.6570 - val_loss: 1.2148 - val_precision: 0.8194
Epoch 7/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6974 - f1_score: 0.6958 - loss: 1.0366 - precision: 0.8488
Epoch 7: val_loss improved from 1.21481 to 1.20674, saving model to best_model_fine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6973 - f1_score: 0.6958 - loss: 1.0367 - precision: 0.8488 - val_accuracy: 0.6626 - val_f1_score: 0.6614 - val_loss: 1.2067 - val_precision: 0.8159
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7052 - f1_score: 0.7040 - loss: 1.0028 - precision: 0.8534
Epoch 8: val_loss improved from 1.20674 to 1.20080, saving model to best_model_fine_tune_v4.keras

```
ne_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7052 - f1_score: 0.704
0 - loss: 1.0029 - precision: 0.8534 - val_accuracy: 0.6618 - val_f1_score: 0.660
9 - val_loss: 1.2008 - val_precision: 0.8180
Epoch 9/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7106 - f1_score: 0.7094
- loss: 0.9803 - precision: 0.8574
Epoch 9: val_loss improved from 1.20080 to 1.19544, saving model to best_model_fi
ne_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7106 - f1_score: 0.709
4 - loss: 0.9803 - precision: 0.8574 - val_accuracy: 0.6629 - val_f1_score: 0.661
5 - val_loss: 1.1954 - val_precision: 0.8171
Epoch 10/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7191 - f1_score: 0.7178
- loss: 0.9529 - precision: 0.8620
Epoch 10: val_loss improved from 1.19544 to 1.19161, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7191 - f1_score: 0.717
8 - loss: 0.9529 - precision: 0.8620 - val_accuracy: 0.6657 - val_f1_score: 0.665
0 - val_loss: 1.1916 - val_precision: 0.8189
Epoch 11/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7235 - f1_score: 0.7223
- loss: 0.9251 - precision: 0.8598
Epoch 11: val_loss improved from 1.19161 to 1.18565, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7235 - f1_score: 0.722
3 - loss: 0.9252 - precision: 0.8598 - val_accuracy: 0.6691 - val_f1_score: 0.667
9 - val_loss: 1.1857 - val_precision: 0.8172
Epoch 12/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7288 - f1_score: 0.7276
- loss: 0.9126 - precision: 0.8670
Epoch 12: val_loss did not improve from 1.18565
1314/1314 ————— 34s 26ms/step - accuracy: 0.7287 - f1_score: 0.727
6 - loss: 0.9127 - precision: 0.8670 - val_accuracy: 0.6646 - val_f1_score: 0.663
3 - val_loss: 1.1880 - val_precision: 0.8176
Epoch 13/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7369 - f1_score: 0.7356
- loss: 0.8820 - precision: 0.8707
Epoch 13: val_loss improved from 1.18565 to 1.18232, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7369 - f1_score: 0.735
6 - loss: 0.8821 - precision: 0.8707 - val_accuracy: 0.6696 - val_f1_score: 0.668
2 - val_loss: 1.1823 - val_precision: 0.8145
Epoch 14/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7411 - f1_score: 0.7398
- loss: 0.8655 - precision: 0.8719
Epoch 14: val_loss improved from 1.18232 to 1.18117, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7410 - f1_score: 0.739
8 - loss: 0.8655 - precision: 0.8719 - val_accuracy: 0.6688 - val_f1_score: 0.667
6 - val_loss: 1.1812 - val_precision: 0.8150
Epoch 15/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7470 - f1_score: 0.7457
- loss: 0.8429 - precision: 0.8759
Epoch 15: val_loss improved from 1.18117 to 1.17799, saving model to best_model_f
ine_tune_v4.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7470 - f1_score: 0.745
6 - loss: 0.8430 - precision: 0.8759 - val_accuracy: 0.6694 - val_f1_score: 0.668
4 - val_loss: 1.1780 - val_precision: 0.8115
```



141/141 ————— 11s 38ms/step - accuracy: 0.6652 - f1_score: 0.6569

- loss: 1.1887 - precision: 0.8022

Test Accuracy: 67.52%

Test Loss: 1.169106364250183

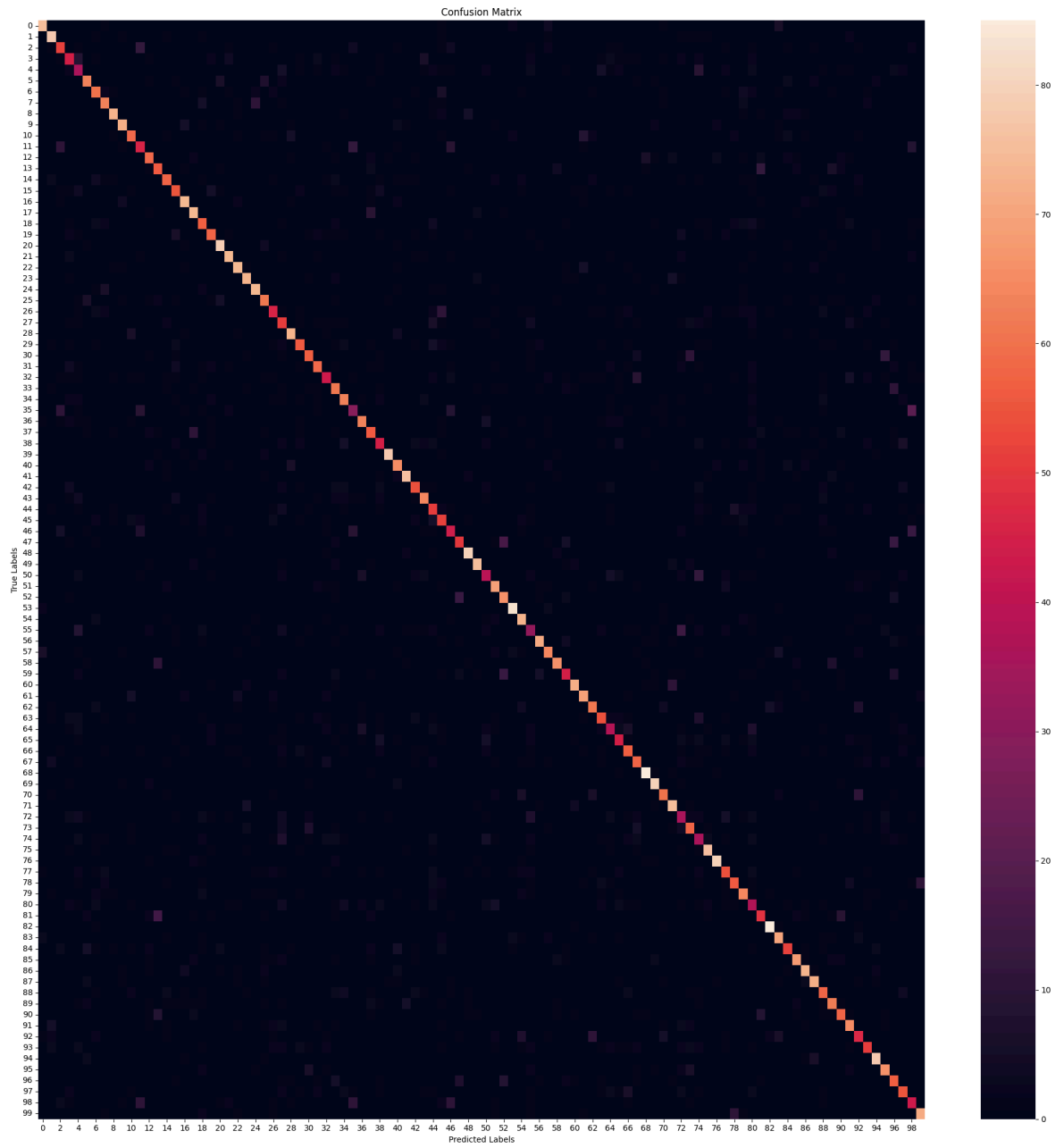
Test Precision: 81.05%

Test F1 Scores (Per Class): [83.798874 80.82901 57.142853 50.847454 37.69633 69.99999 64.51612

71.18643	84.39305	82.285706	67.816086	51.396645	67.816086	61.202175
67.83625	62.14689	79.14438	77.89473	59.374992	66.666664	84.04254
81.31868	80.87432	81.11111	81.31868	68.15642	55.75757	47.846886
81.11111	65.49707	66.27907	69.82249	53.749996	59.803917	67.37968
36.94267	72.41379	62.92134	52.023117	90.69767	68.783066	85.39326
63.529408	72.72727	50.74626	53.60824	50.279324	60.60605	86.48647
84.74575	47.27272	73.40425	67.34693	89.72973	73.73737	36.686382
81.60919	72.31638	77.64706	56.603767	82.285706	76.66666	68.926544
62.427734	46.060596	53.571423	63.33333	62.702698	90.42553	84.656075
70.17543	78.53403	38.50267	63.387978	37.305695	82.16216	89.385475
61.01694	65.49707	68.0851	37.755096	56.32183	91.891884	73.68421
63.030296	77.27273	80.66298	79.55801	63.387978	68.13186	65.90909
69.148926	54.335255	61.818176	85.24589	71.73912	56.999992	63.218384
46.23655	79.329605]					

Average Test F1 Scores:67.47

282/282 ————— 13s 34ms/step



	precision	recall	f1-score	support
beaver	0.84	0.83	0.84	90
dolphin	0.76	0.87	0.81	90
otter	0.57	0.58	0.57	90
seal	0.52	0.50	0.51	90
whale	0.36	0.40	0.38	90
aquarium fish	0.70	0.70	0.70	90
flatfish	0.62	0.67	0.65	90
ray	0.72	0.70	0.71	90
shark	0.88	0.81	0.84	90
trout	0.85	0.80	0.82	90
orchids	0.70	0.66	0.68	90
poppies	0.51	0.51	0.51	90
roses	0.70	0.66	0.68	90
sunflowers	0.60	0.62	0.61	90
tulips	0.72	0.64	0.68	90
bottles	0.63	0.61	0.62	90
bowls	0.76	0.82	0.79	90
cans	0.74	0.82	0.78	90
cups	0.56	0.63	0.59	90
plates	0.69	0.64	0.67	90
apples	0.81	0.88	0.84	90
mushrooms	0.80	0.82	0.81	90
oranges	0.80	0.82	0.81	90
pears	0.81	0.81	0.81	90
sweet peppers	0.80	0.82	0.81	90
clock	0.69	0.68	0.68	90
computer keyboard	0.61	0.51	0.56	90
lamp	0.42	0.57	0.49	90
telephone	0.81	0.81	0.81	90
television	0.69	0.62	0.65	90
bed	0.70	0.63	0.66	90
chair	0.75	0.66	0.70	90
couch	0.61	0.48	0.54	90
table	0.54	0.68	0.60	90
wardrobe	0.65	0.70	0.67	90
bee	0.43	0.32	0.37	90
beetle	0.75	0.70	0.72	90
butterfly	0.64	0.62	0.63	90
caterpillar	0.54	0.50	0.52	90
cockroach	0.95	0.87	0.91	90
bear	0.66	0.72	0.69	90
leopard	0.86	0.84	0.85	90
lion	0.68	0.60	0.64	90
tiger	0.74	0.71	0.73	90
wolf	0.46	0.57	0.51	90
bridge	0.50	0.58	0.54	90
castle	0.50	0.49	0.49	90
house	0.67	0.56	0.61	90
road	0.84	0.89	0.86	90
skyscraper	0.86	0.83	0.85	90
cloud	0.52	0.43	0.47	90
forest	0.70	0.77	0.73	90
mountain	0.62	0.73	0.67	90
plain	0.87	0.92	0.90	90
sea	0.68	0.81	0.74	90
camel	0.39	0.34	0.37	90
cattle	0.85	0.79	0.82	90
chimpanzee	0.74	0.71	0.72	90

elephant	0.82	0.73	0.78	90
kangaroo	0.65	0.50	0.57	90
fox	0.85	0.80	0.82	90
porcupine	0.77	0.77	0.77	90
possum	0.70	0.68	0.69	90
raccoon	0.65	0.60	0.62	90
skunk	0.51	0.42	0.46	90
crab	0.58	0.50	0.54	90
lobster	0.63	0.63	0.63	90
snail	0.61	0.64	0.63	90
spider	0.87	0.94	0.90	90
worm	0.81	0.89	0.85	90
baby	0.74	0.67	0.70	90
boy	0.74	0.83	0.79	90
girl	0.37	0.40	0.39	90
man	0.62	0.64	0.63	90
woman	0.35	0.40	0.37	90
crocodile	0.80	0.84	0.82	90
dinosaur	0.90	0.89	0.89	90
lizard	0.62	0.60	0.61	90
snake	0.69	0.62	0.65	90
turtle	0.65	0.71	0.68	90
hamster	0.35	0.41	0.38	90
mouse	0.58	0.54	0.56	90
rabbit	0.89	0.94	0.92	90
shrew	0.70	0.78	0.74	90
squirrel	0.69	0.58	0.63	90
maple	0.79	0.76	0.77	90
oak	0.80	0.81	0.81	90
palm	0.79	0.80	0.80	90
pine	0.62	0.64	0.63	90
willow	0.67	0.69	0.68	90
bicycle	0.67	0.64	0.66	90
bus	0.67	0.72	0.70	90
motorcycle	0.57	0.52	0.54	90
pickup truck	0.68	0.57	0.62	90
train	0.84	0.87	0.85	90
lawn-mower	0.70	0.73	0.72	90
rocket	0.52	0.63	0.57	90
streetcar	0.65	0.61	0.63	90
tank	0.44	0.48	0.46	90
tractor	0.80	0.79	0.79	90
accuracy			0.68	9000
macro avg	0.68	0.68	0.67	9000
weighted avg	0.68	0.68	0.67	9000

Dropout(0.6), weight_decay(3e-1), l2(3e-1) - no epoch stop for either part

```
In [ ]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

#             .map(augment_dataset, num_parallel_calls=tf.data.experimental.
#             .batch(batch_size)
#             .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'>
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)

#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)

#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/acc
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v7.keras', # Path where the best model is saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v7.keras', # Path where the best model is saved
                                       monitor='val_loss', # What to monitor (validation loss)
                                       save_best_only=True, # Only save model when val loss is best
                                       mode='min', # Looking to minimize the validation loss
                                       verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                   batch_size=batch_size, callbacks=[early_stopping], verbose=1)

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']

```

```

val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v7.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

```

```

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[checkpoint_f

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v7.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))

```



```
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

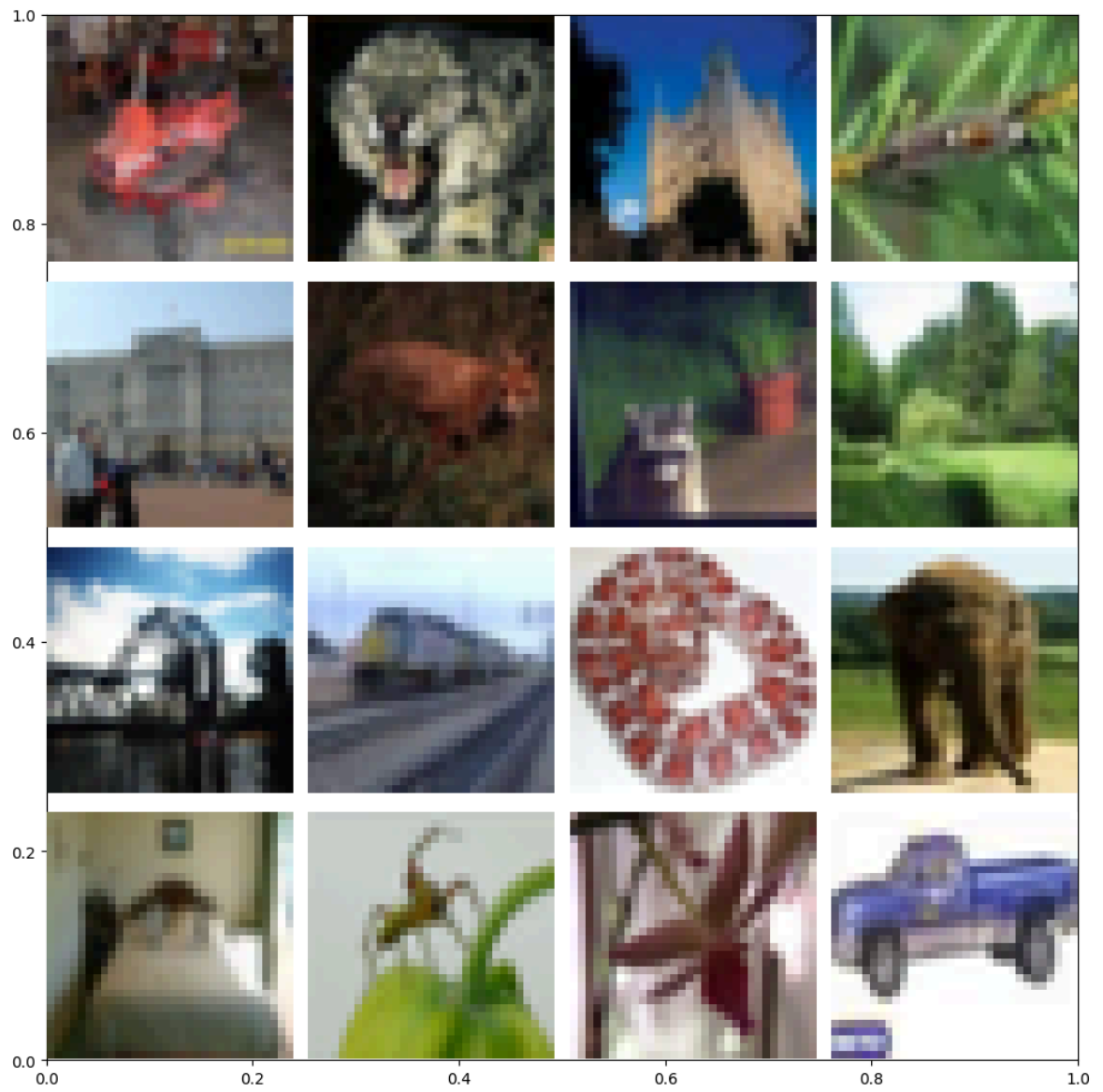
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

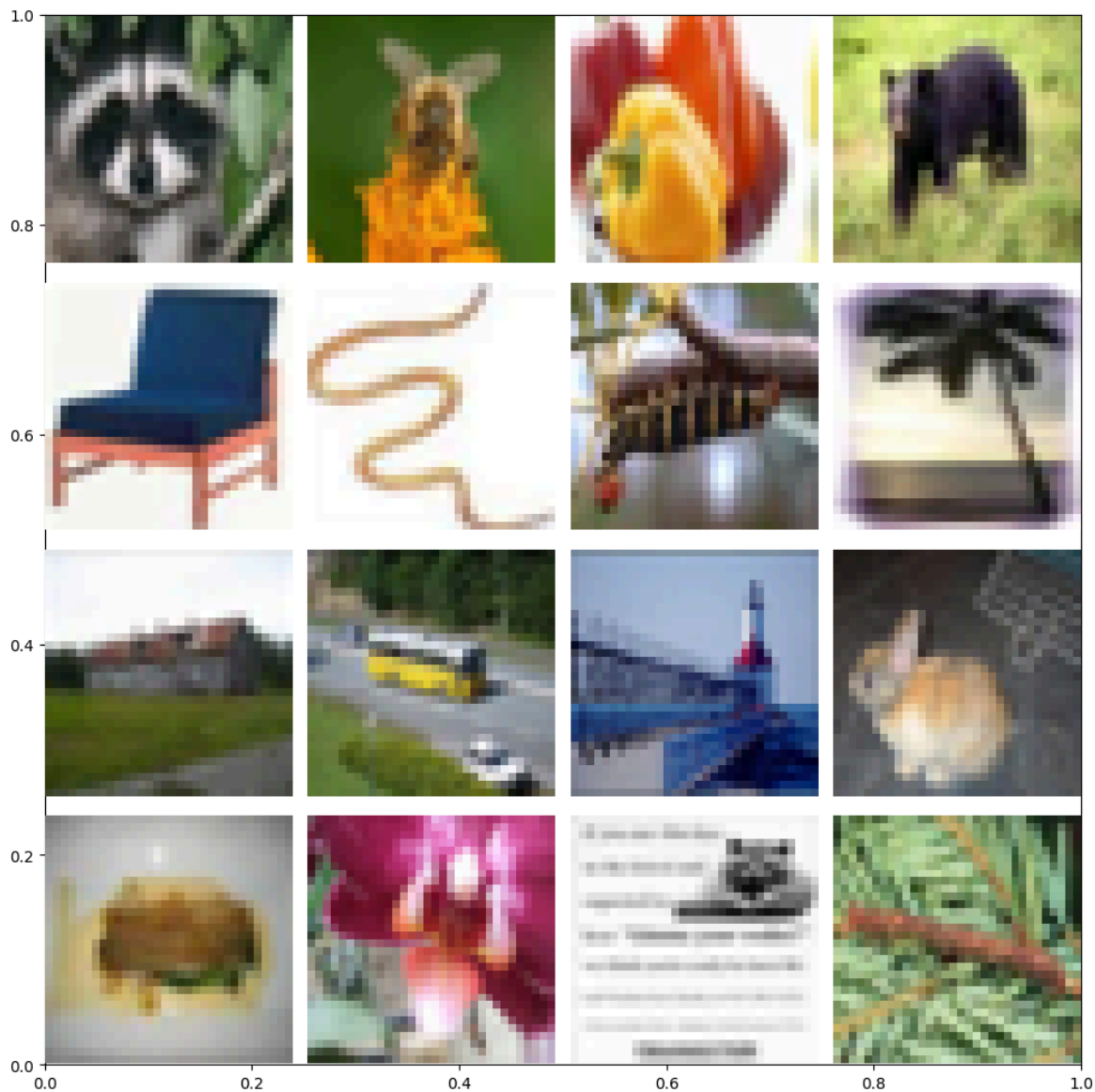
labels[5309].shape: (1,)

imgs[5309].shape: (32, 32, 3)



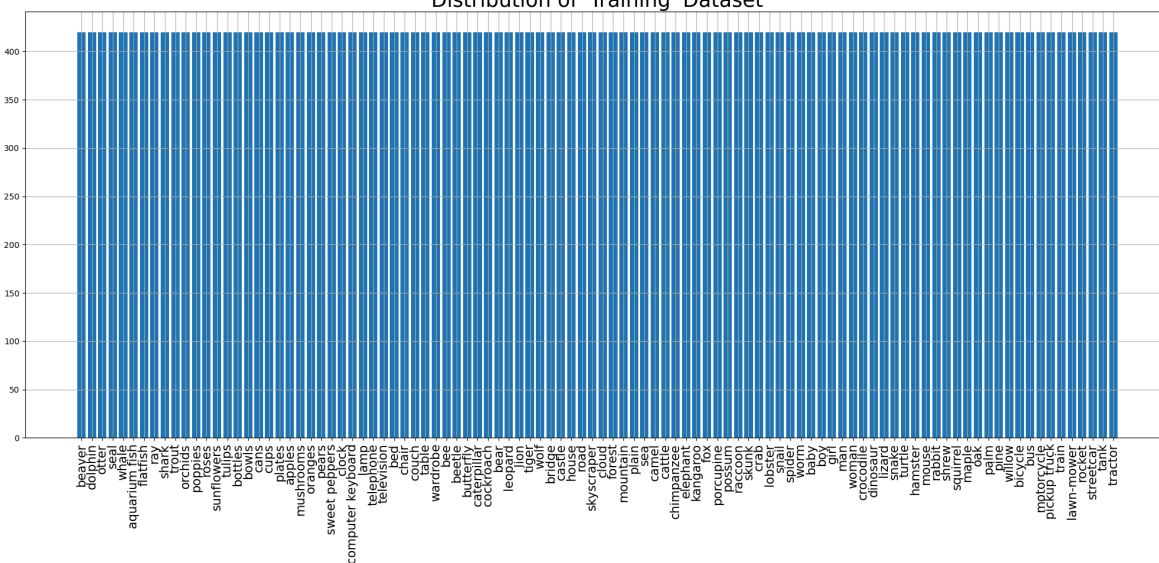
labels[41291].shape: (1,)

imgs[41291].shape: (32, 32, 3)

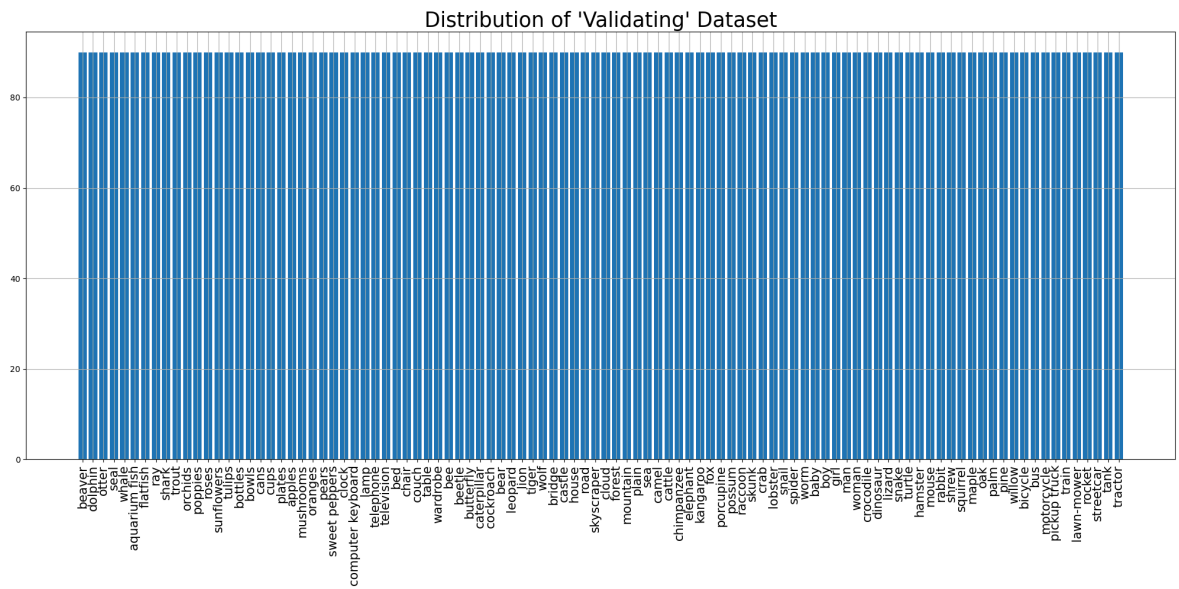


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

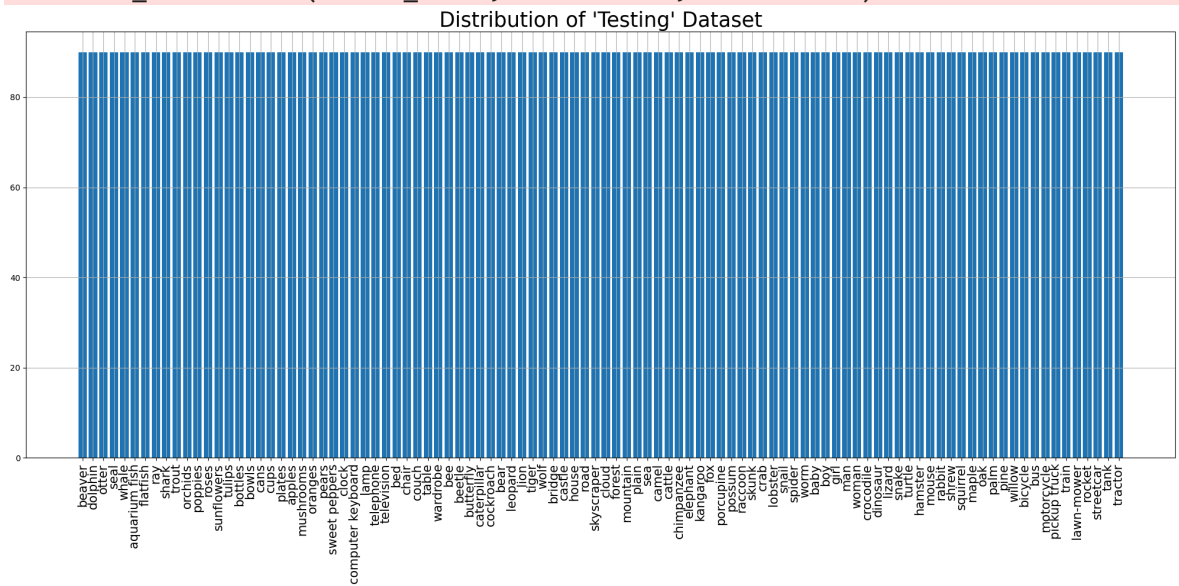
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_10"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_10 (GlobalAveragePooling2D)	(None , 2048)
dense_20 (Dense)	(None , 512)
dropout_10 (Dropout)	(None , 512)
dense_21 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25

657/657  **0s** 30ms/step - accuracy: 0.3193 - f1_score: 0.3137 - loss: 2.8899 - precision: 0.6782

Epoch 1: val_loss improved from inf to 1.50222, saving model to best_model_v7.keras

657/657  **40s** 45ms/step - accuracy: 0.3195 - f1_score: 0.3139 - loss: 2.8890 - precision: 0.6782 - val_accuracy: 0.5852 - val_f1_score: 0.5807 - val_loss: 1.5022 - val_precision: 0.8202

Epoch 2/25

655/657  **0s** 24ms/step - accuracy: 0.5289 - f1_score: 0.5245 - loss: 1.7260 - precision: 0.7525

Epoch 2: val_loss improved from 1.50222 to 1.43288, saving model to best_model_v7.keras

657/657  **20s** 30ms/step - accuracy: 0.5289 - f1_score: 0.5246 - loss: 1.7258 - precision: 0.7525 - val_accuracy: 0.5978 - val_f1_score: 0.5952 - val_loss: 1.4329 - val_precision: 0.8212

Epoch 3/25

656/657  **0s** 24ms/step - accuracy: 0.5571 - f1_score: 0.5532 - loss: 1.5765 - precision: 0.7712

Epoch 3: val_loss improved from 1.43288 to 1.36391, saving model to best_model_v7.keras

657/657  **20s** 30ms/step - accuracy: 0.5572 - f1_score: 0.5532 - loss: 1.5764 - precision: 0.7712 - val_accuracy: 0.6174 - val_f1_score: 0.6133 - val_loss: 1.3639 - val_precision: 0.8215

Epoch 4/25

656/657  **0s** 24ms/step - accuracy: 0.5787 - f1_score: 0.5752 - loss: 1.4800 - precision: 0.7801

Epoch 4: val_loss improved from 1.36391 to 1.34170, saving model to best_model_v7.keras

657/657  **21s** 31ms/step - accuracy: 0.5787 - f1_score: 0.5752 - loss: 1.4799 - precision: 0.7802 - val_accuracy: 0.6183 - val_f1_score: 0.6149 - val_loss: 1.3417 - val_precision: 0.8172

Epoch 5/25

656/657  **0s** 24ms/step - accuracy: 0.5898 - f1_score: 0.5864 - loss: 1.4302 - precision: 0.7855

Epoch 5: val_loss improved from 1.34170 to 1.32215, saving model to best_model_v7.keras

657/657  **20s** 30ms/step - accuracy: 0.5898 - f1_score: 0.5865 - loss: 1.4301 - precision: 0.7855 - val_accuracy: 0.6243 - val_f1_score: 0.6213 - val_loss: 1.3222 - val_precision: 0.8204

Epoch 6/25

657/657  **0s** 24ms/step - accuracy: 0.6047 - f1_score: 0.6015 - loss: 1.3813 - precision: 0.7887

Epoch 6: val_loss did not improve from 1.32215

657/657  **19s** 28ms/step - accuracy: 0.6047 - f1_score: 0.6015 - loss: 1.3813 - precision: 0.7887 - val_accuracy: 0.6272 - val_f1_score: 0.6227 - val_loss: 1.3247 - val_precision: 0.8175

Epoch 7/25

656/657  **0s** 24ms/step - accuracy: 0.6061 - f1_score: 0.6029 - loss: 1.3610 - precision: 0.7925

Epoch 7: val_loss improved from 1.32215 to 1.31176, saving model to best_model_v7.keras

657/657  **20s** 30ms/step - accuracy: 0.6061 - f1_score: 0.6029 - loss: 1.3609 - precision: 0.7925 - val_accuracy: 0.6319 - val_f1_score: 0.6293 - val_loss: 1.3118 - val_precision: 0.8128

Epoch 8/25

655/657  **0s** 24ms/step - accuracy: 0.6102 - f1_score: 0.6076 - loss: 1.3341 - precision: 0.7954

Epoch 8: val_loss improved from 1.31176 to 1.30541, saving model to best_model_v7.keras

657/657 ————— **19s** 30ms/step - accuracy: 0.6102 - f1_score: 0.6077
- loss: 1.3340 - precision: 0.7955 - val_accuracy: 0.6276 - val_f1_score: 0.6249
- val_loss: 1.3054 - val_precision: 0.8106
Epoch 9/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6220 - f1_score: 0.6184 -
loss: 1.3034 - precision: 0.8007
Epoch 9: val_loss improved from 1.30541 to 1.30121, saving model to best_model_v
7.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6221 - f1_score: 0.6184
- loss: 1.3034 - precision: 0.8008 - val_accuracy: 0.6309 - val_f1_score: 0.6290
- val_loss: 1.3012 - val_precision: 0.8037
Epoch 10/25

657/657 ————— **0s** 24ms/step - accuracy: 0.6227 - f1_score: 0.6194 -
loss: 1.2879 - precision: 0.7996
Epoch 10: val_loss improved from 1.30121 to 1.29586, saving model to best_model_v
7.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6227 - f1_score: 0.6194
- loss: 1.2879 - precision: 0.7996 - val_accuracy: 0.6320 - val_f1_score: 0.6282
- val_loss: 1.2959 - val_precision: 0.8064
Epoch 11/25

657/657 ————— **0s** 24ms/step - accuracy: 0.6255 - f1_score: 0.6223 -
loss: 1.2678 - precision: 0.8005
Epoch 11: val_loss improved from 1.29586 to 1.29486, saving model to best_model_v
7.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6255 - f1_score: 0.6223
- loss: 1.2678 - precision: 0.8005 - val_accuracy: 0.6309 - val_f1_score: 0.6284
- val_loss: 1.2949 - val_precision: 0.8020
Epoch 12/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6286 - f1_score: 0.6254 -
loss: 1.2619 - precision: 0.8035
Epoch 12: val_loss improved from 1.29486 to 1.28623, saving model to best_model_v
7.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6286 - f1_score: 0.6254
- loss: 1.2618 - precision: 0.8035 - val_accuracy: 0.6349 - val_f1_score: 0.6308
- val_loss: 1.2862 - val_precision: 0.8108
Epoch 13/25

656/657 ————— **0s** 25ms/step - accuracy: 0.6279 - f1_score: 0.6249 -
loss: 1.2496 - precision: 0.7978
Epoch 13: val_loss improved from 1.28623 to 1.28401, saving model to best_model_v
7.keras

657/657 ————— **20s** 31ms/step - accuracy: 0.6279 - f1_score: 0.6249
- loss: 1.2496 - precision: 0.7978 - val_accuracy: 0.6351 - val_f1_score: 0.6339
- val_loss: 1.2840 - val_precision: 0.8137
Epoch 14/25

655/657 ————— **0s** 25ms/step - accuracy: 0.6409 - f1_score: 0.6379 -
loss: 1.2311 - precision: 0.8143
Epoch 14: val_loss did not improve from 1.28401

657/657 ————— **19s** 29ms/step - accuracy: 0.6410 - f1_score: 0.6379
- loss: 1.2310 - precision: 0.8143 - val_accuracy: 0.6350 - val_f1_score: 0.6327
- val_loss: 1.2888 - val_precision: 0.8062
Epoch 15/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6416 - f1_score: 0.6388 -
loss: 1.2265 - precision: 0.8073
Epoch 15: val_loss did not improve from 1.28401

657/657 ————— **19s** 29ms/step - accuracy: 0.6416 - f1_score: 0.6388
- loss: 1.2265 - precision: 0.8073 - val_accuracy: 0.6319 - val_f1_score: 0.6287
- val_loss: 1.2976 - val_precision: 0.8042
Epoch 16/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6402 - f1_score: 0.6375 -
loss: 1.2298 - precision: 0.8055

Epoch 16: val_loss improved from 1.28401 to 1.28399, saving model to best_model_v7.keras

657/657  **20s** 30ms/step - accuracy: 0.6402 - f1_score: 0.6375
- loss: 1.2298 - precision: 0.8055 - val_accuracy: 0.6313 - val_f1_score: 0.6280
- val_loss: 1.2840 - val_precision: 0.8100

Epoch 17/25

656/657  **0s** 24ms/step - accuracy: 0.6373 - f1_score: 0.6345 -
loss: 1.2230 - precision: 0.8038

Epoch 17: val_loss did not improve from 1.28399

657/657  **19s** 29ms/step - accuracy: 0.6374 - f1_score: 0.6345
- loss: 1.2229 - precision: 0.8038 - val_accuracy: 0.6306 - val_f1_score: 0.6274
- val_loss: 1.2882 - val_precision: 0.8079

Epoch 18/25

655/657  **0s** 24ms/step - accuracy: 0.6387 - f1_score: 0.6358 -
loss: 1.2213 - precision: 0.8075

Epoch 18: val_loss improved from 1.28399 to 1.27947, saving model to best_model_v7.keras

657/657  **20s** 30ms/step - accuracy: 0.6387 - f1_score: 0.6358
- loss: 1.2212 - precision: 0.8075 - val_accuracy: 0.6316 - val_f1_score: 0.6294
- val_loss: 1.2795 - val_precision: 0.8034

Epoch 19/25

655/657  **0s** 24ms/step - accuracy: 0.6447 - f1_score: 0.6413 -
loss: 1.2115 - precision: 0.8097

Epoch 19: val_loss did not improve from 1.27947

657/657  **19s** 29ms/step - accuracy: 0.6447 - f1_score: 0.6413
- loss: 1.2113 - precision: 0.8097 - val_accuracy: 0.6328 - val_f1_score: 0.6302
- val_loss: 1.2845 - val_precision: 0.8054

Epoch 20/25

655/657  **0s** 25ms/step - accuracy: 0.6407 - f1_score: 0.6376 -
loss: 1.2070 - precision: 0.8048

Epoch 20: val_loss did not improve from 1.27947

657/657  **19s** 29ms/step - accuracy: 0.6407 - f1_score: 0.6376
- loss: 1.2070 - precision: 0.8048 - val_accuracy: 0.6340 - val_f1_score: 0.6310
- val_loss: 1.2822 - val_precision: 0.8041

Epoch 21/25

656/657  **0s** 24ms/step - accuracy: 0.6468 - f1_score: 0.6436 -
loss: 1.1974 - precision: 0.8117

Epoch 21: val_loss improved from 1.27947 to 1.27562, saving model to best_model_v7.keras

657/657  **19s** 30ms/step - accuracy: 0.6468 - f1_score: 0.6436
- loss: 1.1973 - precision: 0.8117 - val_accuracy: 0.6341 - val_f1_score: 0.6313
- val_loss: 1.2756 - val_precision: 0.8052

Epoch 22/25

656/657  **0s** 24ms/step - accuracy: 0.6481 - f1_score: 0.6456 -
loss: 1.1904 - precision: 0.8037

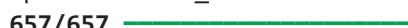
Epoch 22: val_loss did not improve from 1.27562

657/657  **19s** 29ms/step - accuracy: 0.6481 - f1_score: 0.6456
- loss: 1.1903 - precision: 0.8037 - val_accuracy: 0.6311 - val_f1_score: 0.6278
- val_loss: 1.2850 - val_precision: 0.7969

Epoch 23/25

655/657  **0s** 24ms/step - accuracy: 0.6472 - f1_score: 0.6447 -
loss: 1.1880 - precision: 0.8086

Epoch 23: val_loss did not improve from 1.27562

657/657  **19s** 28ms/step - accuracy: 0.6473 - f1_score: 0.6448
- loss: 1.1879 - precision: 0.8086 - val_accuracy: 0.6369 - val_f1_score: 0.6348
- val_loss: 1.2767 - val_precision: 0.8008

Epoch 24/25

656/657  **0s** 23ms/step - accuracy: 0.6448 - f1_score: 0.6423 -
loss: 1.1997 - precision: 0.8107

Epoch 24: val_loss did not improve from 1.27562

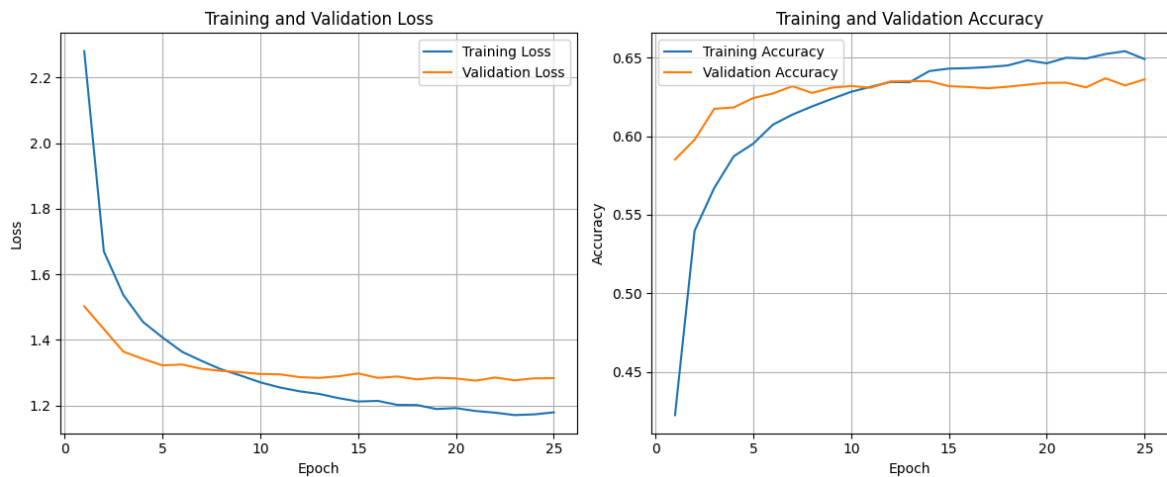
657/657 ————— **19s** 28ms/step - accuracy: 0.6448 - f1_score: 0.6424
- loss: 1.1996 - precision: 0.8107 - val_accuracy: 0.6323 - val_f1_score: 0.6299
- val_loss: 1.2826 - val_precision: 0.7986

Epoch 25/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6442 - f1_score: 0.6412 -
loss: 1.1964 - precision: 0.8076

Epoch 25: val_loss did not improve from 1.27562

657/657 ————— **19s** 28ms/step - accuracy: 0.6442 - f1_score: 0.6413
- loss: 1.1963 - precision: 0.8076 - val_accuracy: 0.6362 - val_f1_score: 0.6342
- val_loss: 1.2831 - val_precision: 0.8033



141/141 ————— **11s** 38ms/step - accuracy: 0.6202 - f1_score: 0.6106
- loss: 1.3066 - precision: 0.7933

Test Accuracy: 63.22%

Test Loss: 1.2761101722717285

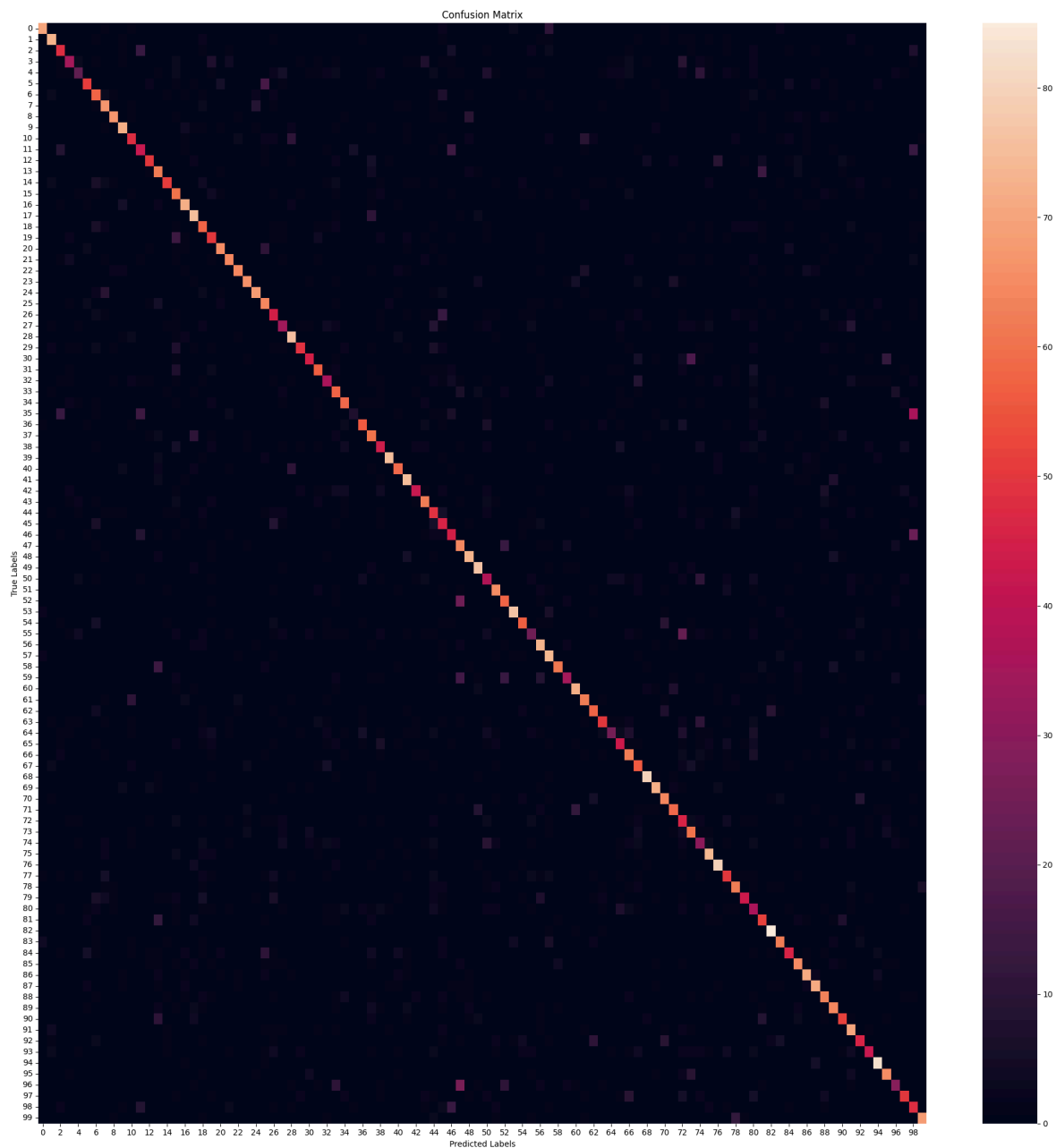
Test Precision: 80.61%

Test F1 Scores (Per Class): [82.352936 77.41935 54.857136 47.9452 31.57894 6
2.962955 55.50239

68.0203	79.761894	78.68852	57.83132	47.191006	65.359474	56.363632
66.666664	53.097343	77.41935	72.11538	57.425735	54.34782	76.92307
73.56321	77.10842	77.64706	79.069756	61.24402	47.61904	43.589737
76.76767	65.33333	59.602642	66.66666	45.56962	56.310677	66.29213
13.592232	68.29269	59.113293	50.86705	87.71929	68.604645	77.7202
58.333324	68.13186	47.572807	44.444443	47.872334	53.33333	78.07486
78.172585	39.57219	72.625694	61.702126	87.00564	68.263466	33.56643
70.1923	67.8899	76.72956	50.724636	78.260864	69.27374	66.666664
63.694263	39.370075	50.29239	59.903378	56.852787	86.95651	80.89887
67.36841	68.604645	39.48497	60.60605	32.96703	76.28865	83.33333
57.80347	66.66666	58.108105	33.96226	55.319138	86.73469	71.67629
55.75757	70.65217	78.88888	78.88888	62.376225	67.72487	61.53846
65.11627	55.75757	55.629128	86.01036	70.27026	39.455776	62.1118
41.20171	79.99999					

Average Test F1 Scores:62.93

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.88	0.78	0.82	90
dolphin	0.75	0.81	0.78	90
otter	0.56	0.53	0.55	90
seal	0.62	0.39	0.48	90
whale	0.49	0.23	0.32	90
aquarium fish	0.71	0.57	0.63	90
flatfish	0.49	0.64	0.56	90
ray	0.63	0.74	0.68	90
shark	0.86	0.74	0.80	90
trout	0.77	0.80	0.79	90
orchids	0.63	0.53	0.58	90
poppies	0.48	0.47	0.47	90
roses	0.79	0.56	0.65	90
sunflowers	0.48	0.69	0.56	90
tulips	0.81	0.57	0.67	90
bottles	0.44	0.67	0.53	90
bowls	0.75	0.80	0.77	90
cans	0.64	0.83	0.72	90
cups	0.52	0.64	0.57	90
plates	0.53	0.56	0.54	90
apples	0.82	0.72	0.77	90
mushrooms	0.76	0.71	0.74	90
oranges	0.84	0.71	0.77	90
pears	0.82	0.73	0.78	90
sweet peppers	0.83	0.76	0.79	90
clock	0.54	0.71	0.61	90
computer keyboard	0.45	0.50	0.48	90
lamp	0.52	0.38	0.44	90
telephone	0.70	0.84	0.77	90
television	0.82	0.54	0.65	90
bed	0.74	0.50	0.60	90
chair	0.70	0.63	0.67	90
couch	0.53	0.40	0.46	90
table	0.50	0.64	0.56	90
wardrobe	0.67	0.66	0.66	90
bee	0.54	0.08	0.14	90
beetle	0.76	0.62	0.68	90
butterfly	0.53	0.67	0.59	90
caterpillar	0.53	0.49	0.51	90
cockroach	0.93	0.83	0.88	90
bear	0.72	0.66	0.69	90
leopard	0.73	0.83	0.78	90
lion	0.78	0.47	0.58	90
tiger	0.67	0.69	0.68	90
wolf	0.42	0.54	0.48	90
bridge	0.39	0.51	0.44	90
castle	0.46	0.50	0.48	90
house	0.43	0.71	0.53	90
road	0.75	0.81	0.78	90
skyscraper	0.72	0.86	0.78	90
cloud	0.38	0.41	0.40	90
forest	0.73	0.72	0.73	90
mountain	0.59	0.64	0.62	90
plain	0.89	0.86	0.87	90
sea	0.74	0.63	0.68	90
camel	0.45	0.27	0.34	90
cattle	0.62	0.81	0.70	90
chimpanzee	0.58	0.82	0.68	90

elephant	0.88	0.68	0.77	90
kangaroo	0.73	0.39	0.51	90
fox	0.77	0.80	0.78	90
porcupine	0.70	0.69	0.69	90
possum	0.69	0.64	0.67	90
raccoon	0.75	0.56	0.64	90
skunk	0.68	0.28	0.39	90
crab	0.53	0.48	0.50	90
lobster	0.53	0.69	0.60	90
snail	0.53	0.62	0.57	90
spider	0.85	0.89	0.87	90
worm	0.82	0.80	0.81	90
baby	0.64	0.71	0.67	90
boy	0.72	0.66	0.69	90
girl	0.32	0.51	0.39	90
man	0.56	0.67	0.61	90
woman	0.33	0.33	0.33	90
crocodile	0.71	0.82	0.76	90
dinosaur	0.78	0.89	0.83	90
lizard	0.60	0.56	0.58	90
snake	0.65	0.69	0.67	90
turtle	0.74	0.48	0.58	90
hamster	0.30	0.40	0.34	90
mouse	0.53	0.58	0.55	90
rabbit	0.80	0.94	0.87	90
shrew	0.75	0.69	0.72	90
squirrel	0.61	0.51	0.56	90
maple	0.69	0.72	0.71	90
oak	0.79	0.79	0.79	90
palm	0.79	0.79	0.79	90
pine	0.56	0.70	0.62	90
willow	0.65	0.71	0.68	90
bicycle	0.66	0.58	0.62	90
bus	0.56	0.78	0.65	90
motorcycle	0.61	0.51	0.56	90
pickup truck	0.69	0.47	0.56	90
train	0.81	0.92	0.86	90
lawn-mower	0.68	0.72	0.70	90
rocket	0.51	0.32	0.39	90
streetcar	0.70	0.56	0.62	90
tank	0.34	0.53	0.41	90
tractor	0.85	0.76	0.80	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential_10"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_10 (GlobalAveragePooling2D)	(None , 2048)
dense_20 (Dense)	(None , 512)
dropout_10 (Dropout)	(None , 512)
dense_21 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1312/1314 ————— 0s 26ms/step - accuracy: 0.6690 - f1_score: 0.6663
- loss: 1.1108 - precision: 0.8211
Epoch 1: val_loss improved from inf to 1.20359, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.6690 - f1_score: 0.6663 - loss: 1.1108 - precision: 0.8211 - val_accuracy: 0.6561 - val_f1_score: 0.6541 - val_loss: 1.2036 - val_precision: 0.8169
Epoch 2/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.6973 - f1_score: 0.6953
- loss: 1.0162 - precision: 0.8463
Epoch 2: val_loss improved from 1.20359 to 1.19178, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6973 - f1_score: 0.6953 - loss: 1.0162 - precision: 0.8463 - val_accuracy: 0.6574 - val_f1_score: 0.6554 - val_loss: 1.1918 - val_precision: 0.8192
Epoch 3/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7042 - f1_score: 0.7022
- loss: 0.9842 - precision: 0.8467
Epoch 3: val_loss improved from 1.19178 to 1.18480, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7042 - f1_score: 0.7022 - loss: 0.9843 - precision: 0.8467 - val_accuracy: 0.6607 - val_f1_score: 0.6588 - val_loss: 1.1848 - val_precision: 0.8204
Epoch 4/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7079 - f1_score: 0.7056
- loss: 0.9693 - precision: 0.8482
Epoch 4: val_loss improved from 1.18480 to 1.17976, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.7079 - f1_score: 0.7056 - loss: 0.9693 - precision: 0.8482 - val_accuracy: 0.6638 - val_f1_score: 0.6622 - val_loss: 1.1798 - val_precision: 0.8211
Epoch 5/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7149 - f1_score: 0.7128
- loss: 0.9535 - precision: 0.8543
Epoch 5: val_loss improved from 1.17976 to 1.17619, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7149 - f1_score: 0.7128 - loss: 0.9535 - precision: 0.8543 - val_accuracy: 0.6634 - val_f1_score: 0.6619 - val_loss: 1.1762 - val_precision: 0.8193
Epoch 6/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7169 - f1_score: 0.7147
- loss: 0.9430 - precision: 0.8539
Epoch 6: val_loss improved from 1.17619 to 1.17298, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7169 - f1_score: 0.7147 - loss: 0.9430 - precision: 0.8539 - val_accuracy: 0.6638 - val_f1_score: 0.6623 - val_loss: 1.1730 - val_precision: 0.8206
Epoch 7/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7232 - f1_score: 0.7213
- loss: 0.9289 - precision: 0.8573
Epoch 7: val_loss improved from 1.17298 to 1.17007, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7232 - f1_score: 0.7213 - loss: 0.9289 - precision: 0.8573 - val_accuracy: 0.6647 - val_f1_score: 0.6631 - val_loss: 1.1701 - val_precision: 0.8205
Epoch 8/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7268 - f1_score: 0.7252
- loss: 0.9195 - precision: 0.8614
Epoch 8: val_loss improved from 1.17007 to 1.16732, saving model to best_model_fine_tune_v7.keras

```
ne_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7268 - f1_score: 0.725
2 - loss: 0.9196 - precision: 0.8614 - val_accuracy: 0.6666 - val_f1_score: 0.664
9 - val_loss: 1.1673 - val_precision: 0.8215
Epoch 9/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7267 - f1_score: 0.7245
- loss: 0.9051 - precision: 0.8625
Epoch 9: val_loss improved from 1.16732 to 1.16560, saving model to best_model_fi
ne_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7267 - f1_score: 0.724
5 - loss: 0.9051 - precision: 0.8625 - val_accuracy: 0.6666 - val_f1_score: 0.664
9 - val_loss: 1.1656 - val_precision: 0.8216
Epoch 10/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7321 - f1_score: 0.7300
- loss: 0.8931 - precision: 0.8657
Epoch 10: val_loss improved from 1.16560 to 1.16361, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7321 - f1_score: 0.730
0 - loss: 0.8931 - precision: 0.8657 - val_accuracy: 0.6679 - val_f1_score: 0.666
3 - val_loss: 1.1636 - val_precision: 0.8205
Epoch 11/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7326 - f1_score: 0.7307
- loss: 0.8897 - precision: 0.8630
Epoch 11: val_loss improved from 1.16361 to 1.16117, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7326 - f1_score: 0.730
7 - loss: 0.8898 - precision: 0.8630 - val_accuracy: 0.6674 - val_f1_score: 0.665
9 - val_loss: 1.1612 - val_precision: 0.8194
Epoch 12/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7367 - f1_score: 0.7349
- loss: 0.8777 - precision: 0.8677
Epoch 12: val_loss improved from 1.16117 to 1.15936, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7367 - f1_score: 0.734
9 - loss: 0.8777 - precision: 0.8677 - val_accuracy: 0.6673 - val_f1_score: 0.665
9 - val_loss: 1.1594 - val_precision: 0.8215
Epoch 13/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7382 - f1_score: 0.7362
- loss: 0.8776 - precision: 0.8662
Epoch 13: val_loss improved from 1.15936 to 1.15757, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7382 - f1_score: 0.736
2 - loss: 0.8776 - precision: 0.8662 - val_accuracy: 0.6666 - val_f1_score: 0.665
0 - val_loss: 1.1576 - val_precision: 0.8212
Epoch 14/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7423 - f1_score: 0.7402
- loss: 0.8625 - precision: 0.8724
Epoch 14: val_loss improved from 1.15757 to 1.15644, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7423 - f1_score: 0.740
2 - loss: 0.8625 - precision: 0.8724 - val_accuracy: 0.6673 - val_f1_score: 0.665
7 - val_loss: 1.1564 - val_precision: 0.8197
Epoch 15/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7410 - f1_score: 0.7389
- loss: 0.8605 - precision: 0.8725
Epoch 15: val_loss improved from 1.15644 to 1.15552, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7410 - f1_score: 0.738
9 - loss: 0.8605 - precision: 0.8725 - val_accuracy: 0.6682 - val_f1_score: 0.666
7 - val_loss: 1.1555 - val_precision: 0.8204
```

Epoch 16/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7459 - f1_score: 0.7440
- loss: 0.8479 - precision: 0.8718
Epoch 16: val_loss improved from 1.15552 to 1.15429, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7459 - f1_score: 0.7440 - loss: 0.8479 - precision: 0.8718 - val_accuracy: 0.6677 - val_f1_score: 0.6662 - val_loss: 1.1543 - val_precision: 0.8202
Epoch 17/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7461 - f1_score: 0.7443 - loss: 0.8475 - precision: 0.8738
Epoch 17: val_loss improved from 1.15429 to 1.15196, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7461 - f1_score: 0.7443 - loss: 0.8476 - precision: 0.8738 - val_accuracy: 0.6683 - val_f1_score: 0.6666 - val_loss: 1.1520 - val_precision: 0.8206
Epoch 18/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7493 - f1_score: 0.7471 - loss: 0.8360 - precision: 0.8763
Epoch 18: val_loss improved from 1.15196 to 1.15079, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7493 - f1_score: 0.7471 - loss: 0.8361 - precision: 0.8763 - val_accuracy: 0.6683 - val_f1_score: 0.6668 - val_loss: 1.1508 - val_precision: 0.8208
Epoch 19/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7470 - f1_score: 0.7450 - loss: 0.8363 - precision: 0.8749
Epoch 19: val_loss improved from 1.15079 to 1.15020, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7469 - f1_score: 0.7450 - loss: 0.8364 - precision: 0.8749 - val_accuracy: 0.6692 - val_f1_score: 0.6675 - val_loss: 1.1502 - val_precision: 0.8203
Epoch 20/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7531 - f1_score: 0.7510 - loss: 0.8276 - precision: 0.8785
Epoch 20: val_loss improved from 1.15020 to 1.14928, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7530 - f1_score: 0.7509 - loss: 0.8276 - precision: 0.8785 - val_accuracy: 0.6694 - val_f1_score: 0.6677 - val_loss: 1.1493 - val_precision: 0.8205
Epoch 21/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7560 - f1_score: 0.7540 - loss: 0.8192 - precision: 0.8780
Epoch 21: val_loss improved from 1.14928 to 1.14840, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7560 - f1_score: 0.7540 - loss: 0.8192 - precision: 0.8780 - val_accuracy: 0.6698 - val_f1_score: 0.6683 - val_loss: 1.1484 - val_precision: 0.8197
Epoch 22/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7582 - f1_score: 0.7564 - loss: 0.8110 - precision: 0.8818
Epoch 22: val_loss improved from 1.14840 to 1.14731, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7582 - f1_score: 0.7564 - loss: 0.8110 - precision: 0.8818 - val_accuracy: 0.6701 - val_f1_score: 0.6686 - val_loss: 1.1473 - val_precision: 0.8186
Epoch 23/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7603 - f1_score: 0.7584 - loss: 0.8055 - precision: 0.8818
Epoch 23: val_loss improved from 1.14731 to 1.14682, saving model to best_model_fine_tune_v7.keras

ine_tune_v7.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.7603 - f1_score: 0.7583 - loss: 0.8056 - precision: 0.8818 - val_accuracy: 0.6704 - val_f1_score: 0.6688 - val_loss: 1.1468 - val_precision: 0.8216

Epoch 24/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.7606 - f1_score: 0.7587 - loss: 0.8059 - precision: 0.8823

Epoch 24: val_loss improved from 1.14682 to 1.14573, saving model to best_model_fine_tune_v7.keras

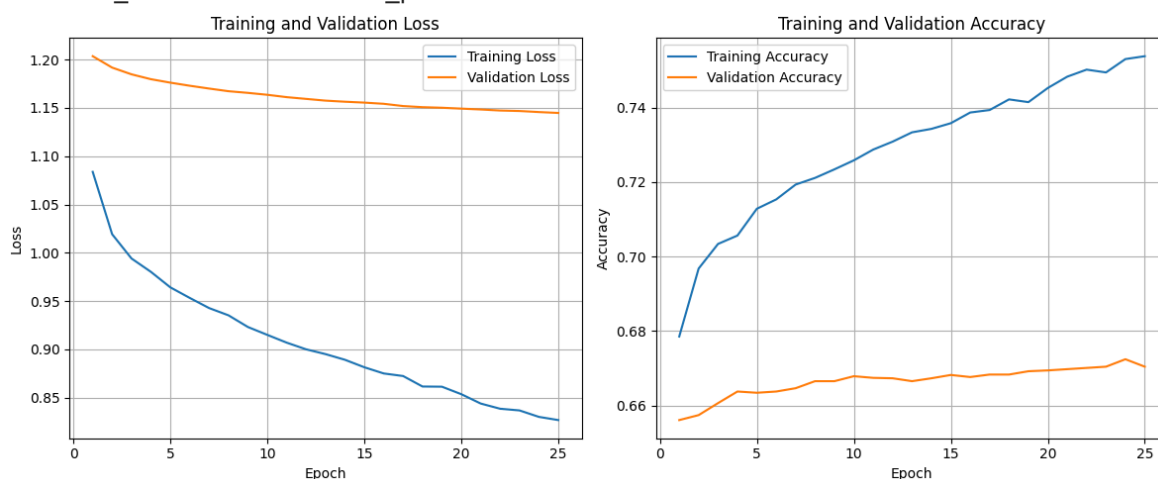
1314/1314 ————— **35s** 27ms/step - accuracy: 0.7605 - f1_score: 0.7587 - loss: 0.8059 - precision: 0.8823 - val_accuracy: 0.6724 - val_f1_score: 0.6707 - val_loss: 1.1457 - val_precision: 0.8184

Epoch 25/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.7639 - f1_score: 0.7619 - loss: 0.7945 - precision: 0.8843

Epoch 25: val_loss improved from 1.14573 to 1.14481, saving model to best_model_fine_tune_v7.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.7638 - f1_score: 0.7619 - loss: 0.7945 - precision: 0.8843 - val_accuracy: 0.6704 - val_f1_score: 0.6690 - val_loss: 1.1448 - val_precision: 0.8201



141/141 ————— **11s** 38ms/step - accuracy: 0.6587 - f1_score: 0.6504 - loss: 1.1744 - precision: 0.8107

Test Accuracy: 67.04%

Test Loss: 1.145362138748169

Test Precision: 81.91%

Test F1 Scores (Per Class): [84.26965 78.723404 54.081623 54.545456 34.28571 65.92178 63.783775

72.41379 79.28994 83.04093 62.92134 49.10179 65.89595 64.
67.42856 63.636356 75.62189 74.6114 62.702698 60.355026 81.08108
81.481476 78.918915 78.65168 82.35293 64.739876 53.107338 50.7317
79.78141 66.26506 61.363636 71.428566 53.333324 61.855663 68.852455
34.682076 72.222206 61.87845 53.33333 90.28571 72.04301 82.7957
65.92178 69.23077 47.959175 52.272724 51.136356 60.818714 83.69564
83.51647 44.31137 73.74301 64.55026 89.99999 69.74358 39.080452
81.87134 76.08696 79.04191 59.036137 80.89887 71.26436 67.72487
62.14689 46.913574 59.340656 65.94594 63.33333 86.02149 84.491974
67.77777 75.39267 43.749992 62.983418 38.20224 81.96721 86.33879
62.06896 66.666664 64.44444 38.88888 57.92349 89.99999 72.251305
62.427734 73.79678 82.48587 78.918915 65.217384 68.0851 63.82978
68.1081 55.421684 62.195118 87.29281 71.20418 58.031082 65.93406
40.909084 78.88888]

Average Test F1 Scores:66.89

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.85	0.83	0.84	90
dolphin	0.76	0.82	0.79	90
otter	0.50	0.59	0.54	90
seal	0.56	0.53	0.55	90
whale	0.35	0.33	0.34	90
aquarium fish	0.66	0.66	0.66	90
flatfish	0.62	0.66	0.64	90
ray	0.75	0.70	0.72	90
shark	0.85	0.74	0.79	90
trout	0.88	0.79	0.83	90
orchids	0.64	0.62	0.63	90
poppies	0.53	0.46	0.49	90
roses	0.69	0.63	0.66	90
sunflowers	0.66	0.62	0.64	90
tulips	0.69	0.66	0.67	90
bottles	0.65	0.62	0.64	90
bowls	0.68	0.84	0.76	90
cans	0.70	0.80	0.75	90
cups	0.61	0.64	0.63	90
plates	0.65	0.57	0.60	90
apples	0.79	0.83	0.81	90
mushrooms	0.78	0.86	0.81	90
oranges	0.77	0.81	0.79	90
pears	0.80	0.78	0.79	90
sweet peppers	0.79	0.86	0.82	90
clock	0.67	0.62	0.65	90
computer keyboard	0.54	0.52	0.53	90
lamp	0.45	0.58	0.51	90
telephone	0.78	0.81	0.80	90
television	0.72	0.61	0.66	90
bed	0.63	0.60	0.61	90
chair	0.77	0.67	0.71	90
couch	0.59	0.49	0.53	90
table	0.58	0.67	0.62	90
wardrobe	0.68	0.70	0.69	90
bee	0.36	0.33	0.35	90
beetle	0.72	0.72	0.72	90
butterfly	0.62	0.62	0.62	90
caterpillar	0.53	0.53	0.53	90
cockroach	0.93	0.88	0.90	90
bear	0.70	0.74	0.72	90
leopard	0.80	0.86	0.83	90
lion	0.66	0.66	0.66	90
tiger	0.68	0.70	0.69	90
wolf	0.44	0.52	0.48	90
bridge	0.53	0.51	0.52	90
castle	0.52	0.50	0.51	90
house	0.63	0.58	0.60	90
road	0.82	0.86	0.84	90
skyscraper	0.83	0.84	0.84	90
cloud	0.48	0.41	0.44	90
forest	0.74	0.74	0.74	90
mountain	0.62	0.68	0.65	90
plain	0.90	0.90	0.90	90
sea	0.65	0.76	0.70	90
camel	0.40	0.38	0.39	90
cattle	0.86	0.78	0.82	90
chimpanzee	0.74	0.78	0.76	90

elephant	0.86	0.73	0.79	90
kangaroo	0.64	0.54	0.59	90
fox	0.82	0.80	0.81	90
porcupine	0.74	0.69	0.71	90
possum	0.65	0.71	0.68	90
raccoon	0.63	0.61	0.62	90
skunk	0.53	0.42	0.47	90
crab	0.59	0.60	0.59	90
lobster	0.64	0.68	0.66	90
snail	0.63	0.63	0.63	90
spider	0.83	0.89	0.86	90
worm	0.81	0.88	0.84	90
baby	0.68	0.68	0.68	90
boy	0.71	0.80	0.75	90
girl	0.50	0.39	0.44	90
man	0.63	0.63	0.63	90
woman	0.39	0.38	0.38	90
crocodile	0.81	0.83	0.82	90
dinosaur	0.85	0.88	0.86	90
lizard	0.64	0.60	0.62	90
snake	0.67	0.67	0.67	90
turtle	0.64	0.64	0.64	90
hamster	0.39	0.39	0.39	90
mouse	0.57	0.59	0.58	90
rabbit	0.90	0.90	0.90	90
shrew	0.68	0.77	0.72	90
squirrel	0.65	0.60	0.62	90
maple	0.71	0.77	0.74	90
oak	0.84	0.81	0.82	90
palm	0.77	0.81	0.79	90
pine	0.64	0.67	0.65	90
willow	0.65	0.71	0.68	90
bicycle	0.61	0.67	0.64	90
bus	0.66	0.70	0.68	90
motorcycle	0.61	0.51	0.55	90
pickup truck	0.69	0.57	0.62	90
train	0.87	0.88	0.87	90
lawn-mower	0.67	0.76	0.71	90
rocket	0.54	0.62	0.58	90
streetcar	0.65	0.67	0.66	90
tank	0.42	0.40	0.41	90
tractor	0.79	0.79	0.79	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.67	9000
weighted avg	0.67	0.67	0.67	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

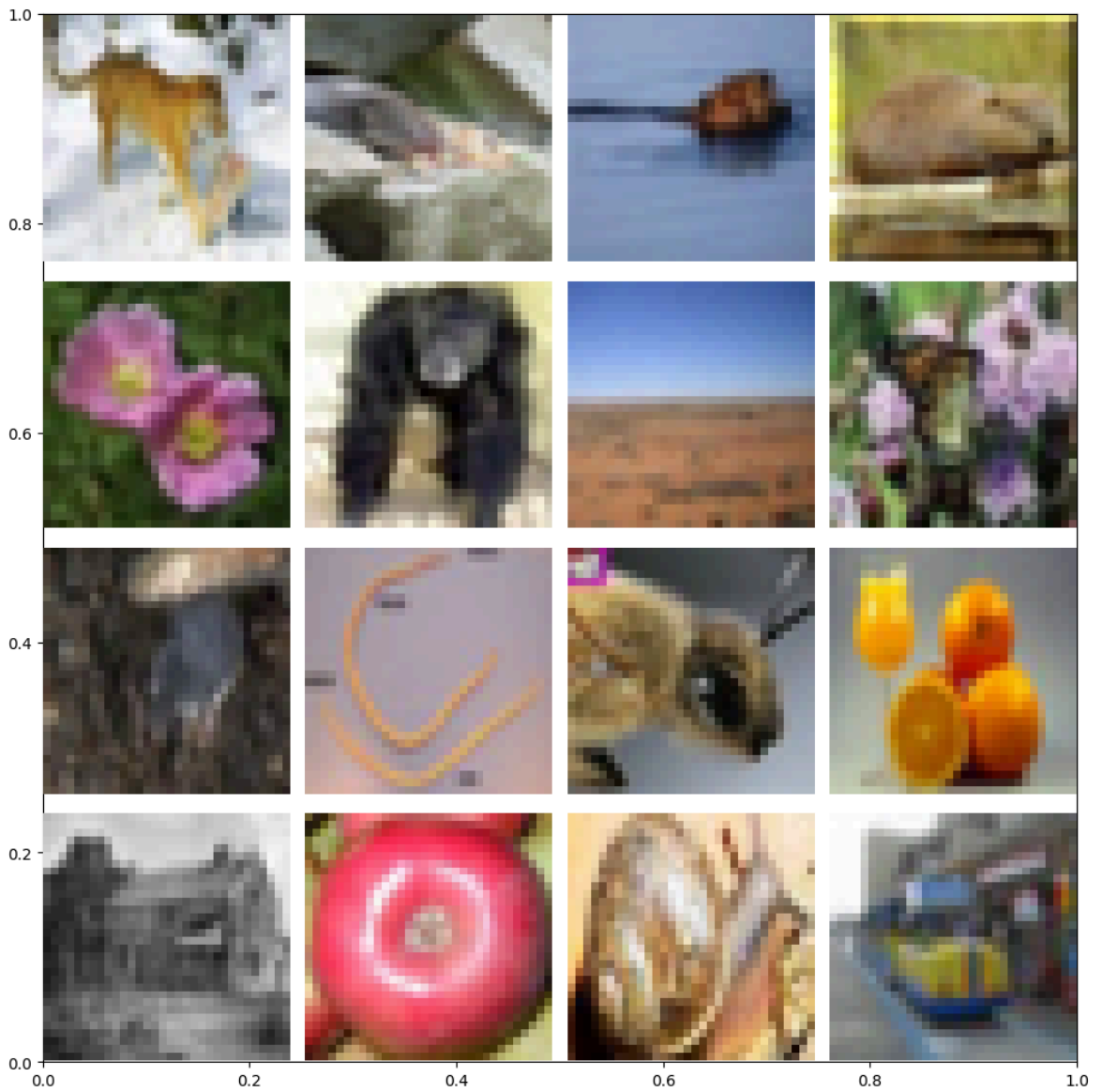
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

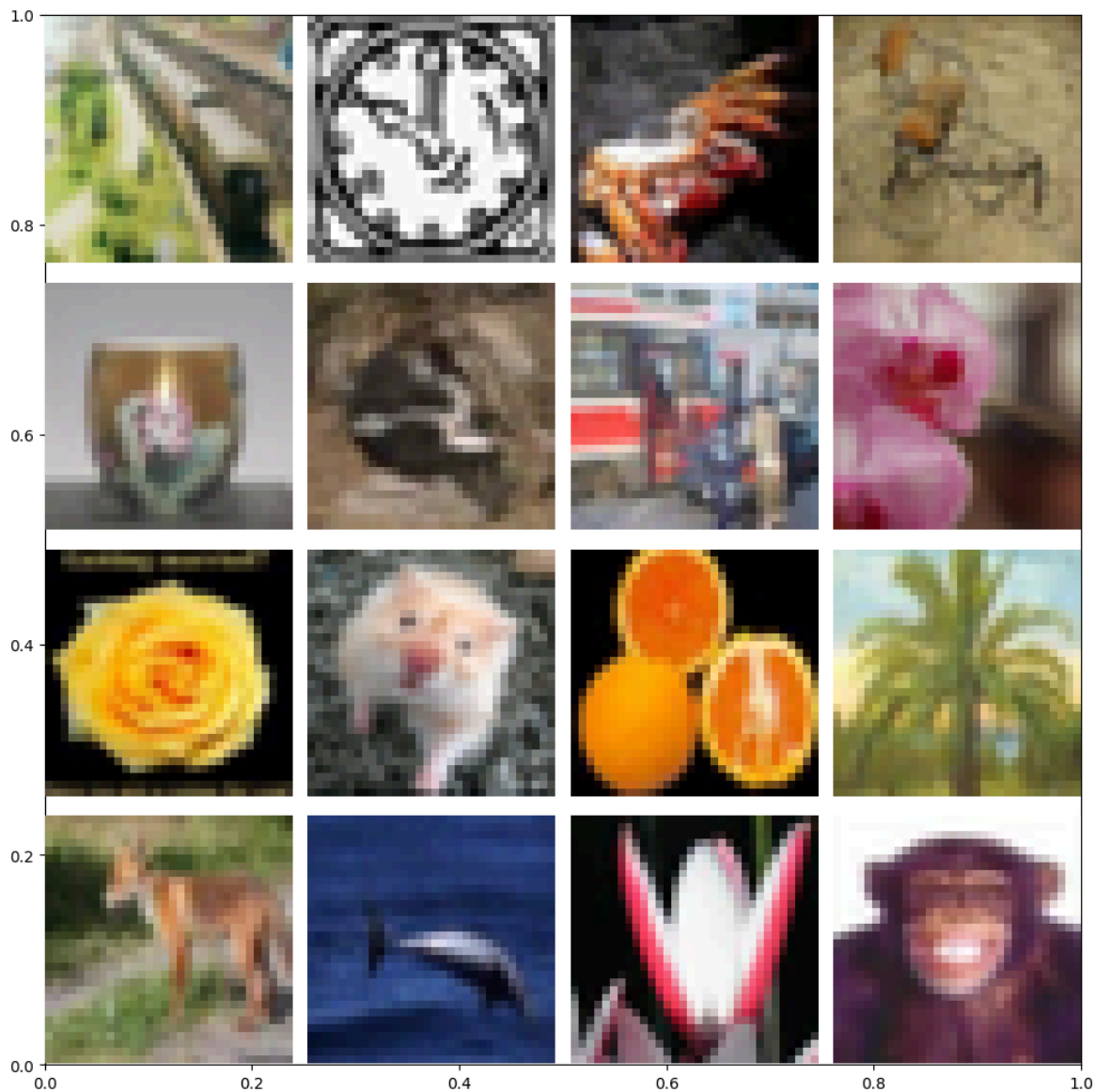
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[36785].shape: (1,)  
imgs[36785].shape: (32, 32, 3)
```

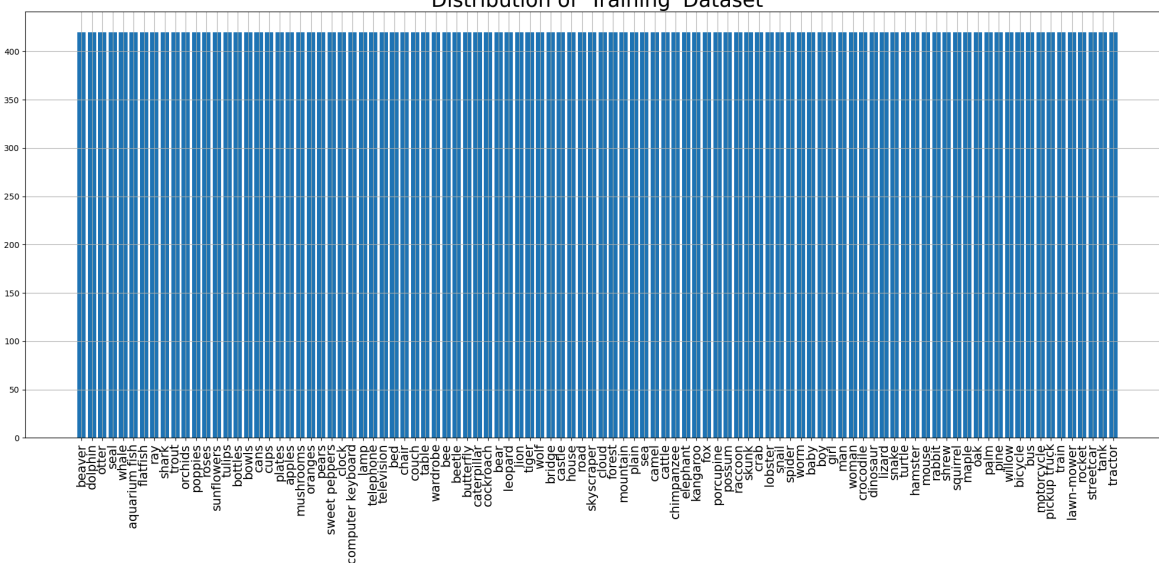


```
labels[24577].shape: (1,)  
imgs[24577].shape: (32, 32, 3)
```

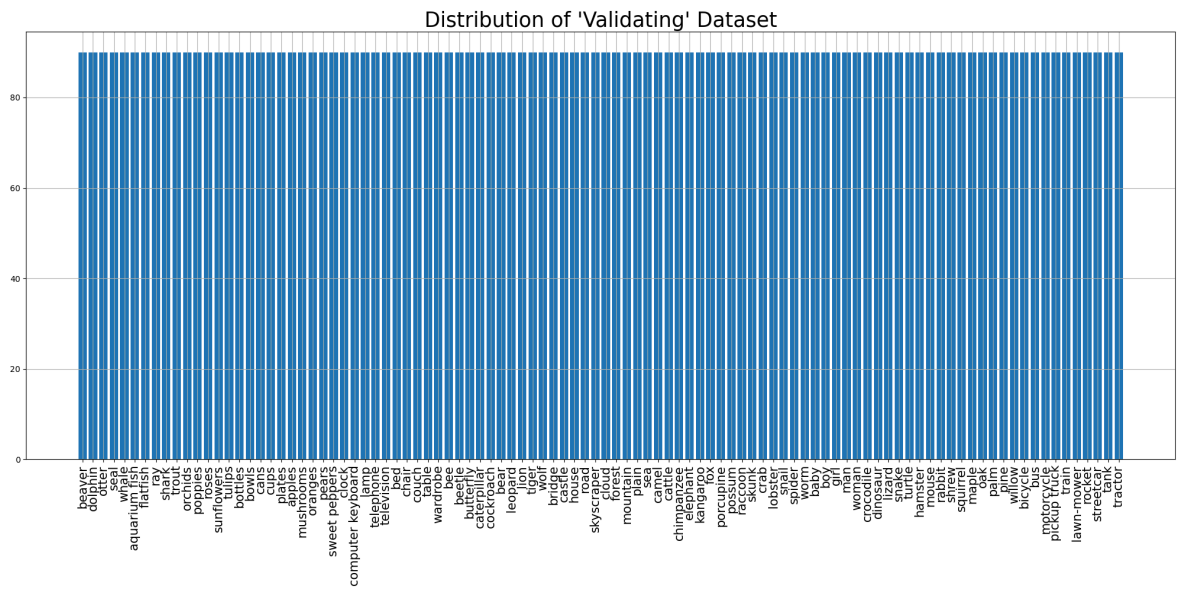


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

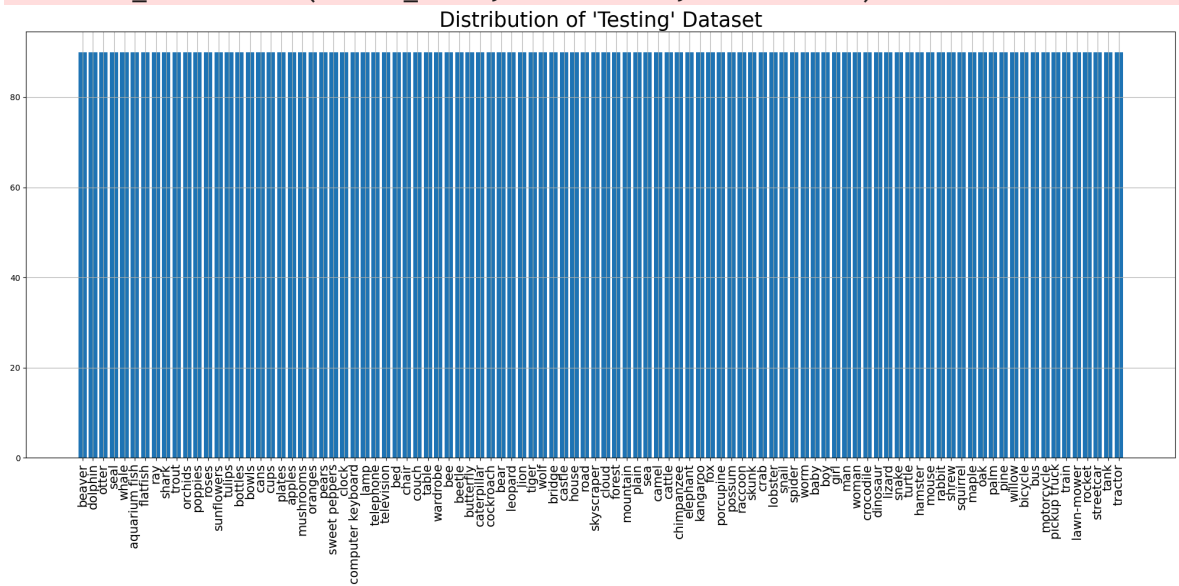
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_11"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_11 (GlobalAveragePooling2D)	(None , 2048)
dense_22 (Dense)	(None , 512)
dropout_11 (Dropout)	(None , 512)
dense_23 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 29ms/step - accuracy: 0.3187 - f1_score: 0.3128 - loss: 2.8955 - precision: 0.6710
Epoch 1: val_loss improved from inf to 1.51045, saving model to best_model_v7.keras as
657/657 ————— 39s 44ms/step - accuracy: 0.3188 - f1_score: 0.3130 - loss: 2.8946 - precision: 0.6711 - val_accuracy: 0.5842 - val_f1_score: 0.5805 - val_loss: 1.5104 - val_precision: 0.8176
Epoch 2/25
656/657 ————— 0s 23ms/step - accuracy: 0.5230 - f1_score: 0.5185 - loss: 1.7401 - precision: 0.7593
Epoch 2: val_loss improved from 1.51045 to 1.41946, saving model to best_model_v7.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5230 - f1_score: 0.5186 - loss: 1.7400 - precision: 0.7593 - val_accuracy: 0.5988 - val_f1_score: 0.5927 - val_loss: 1.4195 - val_precision: 0.8075
Epoch 3/25
657/657 ————— 0s 23ms/step - accuracy: 0.5596 - f1_score: 0.5557 - loss: 1.5634 - precision: 0.7717
Epoch 3: val_loss improved from 1.41946 to 1.36287, saving model to best_model_v7.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5596 - f1_score: 0.5557 - loss: 1.5633 - precision: 0.7717 - val_accuracy: 0.6166 - val_f1_score: 0.6129 - val_loss: 1.3629 - val_precision: 0.8226
Epoch 4/25
657/657 ————— 0s 23ms/step - accuracy: 0.5791 - f1_score: 0.5759 - loss: 1.4813 - precision: 0.7853
Epoch 4: val_loss improved from 1.36287 to 1.34354, saving model to best_model_v7.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5791 - f1_score: 0.5759 - loss: 1.4813 - precision: 0.7853 - val_accuracy: 0.6184 - val_f1_score: 0.6156 - val_loss: 1.3435 - val_precision: 0.8199
Epoch 5/25
656/657 ————— 0s 23ms/step - accuracy: 0.5910 - f1_score: 0.5872 - loss: 1.4323 - precision: 0.7889
Epoch 5: val_loss improved from 1.34354 to 1.32785, saving model to best_model_v7.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5910 - f1_score: 0.5872 - loss: 1.4323 - precision: 0.7889 - val_accuracy: 0.6209 - val_f1_score: 0.6169 - val_loss: 1.3279 - val_precision: 0.8150
Epoch 6/25
657/657 ————— 0s 23ms/step - accuracy: 0.5979 - f1_score: 0.5943 - loss: 1.4058 - precision: 0.7906
Epoch 6: val_loss improved from 1.32785 to 1.32033, saving model to best_model_v7.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5979 - f1_score: 0.5943 - loss: 1.4057 - precision: 0.7906 - val_accuracy: 0.6237 - val_f1_score: 0.6233 - val_loss: 1.3203 - val_precision: 0.8160
Epoch 7/25
655/657 ————— 0s 23ms/step - accuracy: 0.6067 - f1_score: 0.6033 - loss: 1.3647 - precision: 0.7913
Epoch 7: val_loss improved from 1.32033 to 1.30506, saving model to best_model_v7.keras
657/657 ————— 19s 29ms/step - accuracy: 0.6067 - f1_score: 0.6033 - loss: 1.3647 - precision: 0.7913 - val_accuracy: 0.6303 - val_f1_score: 0.6293 - val_loss: 1.3051 - val_precision: 0.8124
Epoch 8/25
655/657 ————— 0s 23ms/step - accuracy: 0.6124 - f1_score: 0.6092 - loss: 1.3390 - precision: 0.7972
Epoch 8: val_loss did not improve from 1.30506

657/657 ————— **18s** 28ms/step - accuracy: 0.6124 - f1_score: 0.6092
- loss: 1.3389 - precision: 0.7972 - val_accuracy: 0.6272 - val_f1_score: 0.6264
- val_loss: 1.3080 - val_precision: 0.8169
Epoch 9/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6153 - f1_score: 0.6118 -
loss: 1.3175 - precision: 0.7949
Epoch 9: val_loss improved from 1.30506 to 1.30222, saving model to best_model_v
7.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.6153 - f1_score: 0.6118
- loss: 1.3174 - precision: 0.7949 - val_accuracy: 0.6270 - val_f1_score: 0.6245
- val_loss: 1.3022 - val_precision: 0.8044
Epoch 10/25
657/657 ————— **0s** 23ms/step - accuracy: 0.6251 - f1_score: 0.6213 -
loss: 1.2936 - precision: 0.8023
Epoch 10: val_loss did not improve from 1.30222
657/657 ————— **18s** 28ms/step - accuracy: 0.6251 - f1_score: 0.6213
- loss: 1.2936 - precision: 0.8023 - val_accuracy: 0.6271 - val_f1_score: 0.6247
- val_loss: 1.3102 - val_precision: 0.8093
Epoch 11/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6243 - f1_score: 0.6213 -
loss: 1.2852 - precision: 0.8007
Epoch 11: val_loss improved from 1.30222 to 1.29168, saving model to best_model_v
7.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.6243 - f1_score: 0.6213
- loss: 1.2851 - precision: 0.8007 - val_accuracy: 0.6337 - val_f1_score: 0.6311
- val_loss: 1.2917 - val_precision: 0.8137
Epoch 12/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6254 - f1_score: 0.6219 -
loss: 1.2708 - precision: 0.8022
Epoch 12: val_loss improved from 1.29168 to 1.27840, saving model to best_model_v
7.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.6254 - f1_score: 0.6219
- loss: 1.2708 - precision: 0.8022 - val_accuracy: 0.6384 - val_f1_score: 0.6371
- val_loss: 1.2784 - val_precision: 0.8106
Epoch 13/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6315 - f1_score: 0.6284 -
loss: 1.2544 - precision: 0.8054
Epoch 13: val_loss did not improve from 1.27840
657/657 ————— **18s** 28ms/step - accuracy: 0.6315 - f1_score: 0.6285
- loss: 1.2544 - precision: 0.8054 - val_accuracy: 0.6329 - val_f1_score: 0.6317
- val_loss: 1.2818 - val_precision: 0.8154
Epoch 14/25
655/657 ————— **0s** 23ms/step - accuracy: 0.6336 - f1_score: 0.6300 -
loss: 1.2476 - precision: 0.8067
Epoch 14: val_loss did not improve from 1.27840
657/657 ————— **18s** 28ms/step - accuracy: 0.6336 - f1_score: 0.6300
- loss: 1.2475 - precision: 0.8068 - val_accuracy: 0.6303 - val_f1_score: 0.6276
- val_loss: 1.2926 - val_precision: 0.8089
Epoch 15/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6353 - f1_score: 0.6319 -
loss: 1.2338 - precision: 0.8090
Epoch 15: val_loss did not improve from 1.27840
657/657 ————— **18s** 28ms/step - accuracy: 0.6353 - f1_score: 0.6319
- loss: 1.2338 - precision: 0.8090 - val_accuracy: 0.6247 - val_f1_score: 0.6216
- val_loss: 1.3038 - val_precision: 0.8080
Epoch 16/25
657/657 ————— **0s** 23ms/step - accuracy: 0.6366 - f1_score: 0.6335 -
loss: 1.2367 - precision: 0.8069
Epoch 16: val_loss did not improve from 1.27840
657/657 ————— **18s** 28ms/step - accuracy: 0.6366 - f1_score: 0.6335

- loss: 1.2367 - precision: 0.8069 - val_accuracy: 0.6306 - val_f1_score: 0.6278
- val_loss: 1.2923 - val_precision: 0.8041
Epoch 17/25
655/657 ————— 0s 23ms/step - accuracy: 0.6390 - f1_score: 0.6360 -
loss: 1.2241 - precision: 0.8044
Epoch 17: val_loss did not improve from 1.27840
657/657 ————— 19s 28ms/step - accuracy: 0.6390 - f1_score: 0.6360
- loss: 1.2240 - precision: 0.8044 - val_accuracy: 0.6348 - val_f1_score: 0.6324
- val_loss: 1.2817 - val_precision: 0.8098
Epoch 18/25
656/657 ————— 0s 23ms/step - accuracy: 0.6389 - f1_score: 0.6365 -
loss: 1.2240 - precision: 0.8068
Epoch 18: val_loss improved from 1.27840 to 1.26951, saving model to best_model_v
7.keras
657/657 ————— 19s 30ms/step - accuracy: 0.6389 - f1_score: 0.6365
- loss: 1.2240 - precision: 0.8068 - val_accuracy: 0.6350 - val_f1_score: 0.6323
- val_loss: 1.2695 - val_precision: 0.8039
Epoch 19/25
656/657 ————— 0s 23ms/step - accuracy: 0.6401 - f1_score: 0.6371 -
loss: 1.2145 - precision: 0.8044
Epoch 19: val_loss did not improve from 1.26951
657/657 ————— 18s 28ms/step - accuracy: 0.6401 - f1_score: 0.6371
- loss: 1.2144 - precision: 0.8045 - val_accuracy: 0.6307 - val_f1_score: 0.6300
- val_loss: 1.2945 - val_precision: 0.7973
Epoch 20/25
655/657 ————— 0s 23ms/step - accuracy: 0.6452 - f1_score: 0.6422 -
loss: 1.2103 - precision: 0.8086
Epoch 20: val_loss improved from 1.26951 to 1.26808, saving model to best_model_v
7.keras
657/657 ————— 19s 29ms/step - accuracy: 0.6452 - f1_score: 0.6422
- loss: 1.2103 - precision: 0.8086 - val_accuracy: 0.6411 - val_f1_score: 0.6401
- val_loss: 1.2681 - val_precision: 0.8114
Epoch 21/25
656/657 ————— 0s 23ms/step - accuracy: 0.6418 - f1_score: 0.6384 -
loss: 1.2082 - precision: 0.8083
Epoch 21: val_loss did not improve from 1.26808
657/657 ————— 19s 28ms/step - accuracy: 0.6418 - f1_score: 0.6384
- loss: 1.2081 - precision: 0.8083 - val_accuracy: 0.6361 - val_f1_score: 0.6334
- val_loss: 1.2714 - val_precision: 0.7962
Epoch 22/25
656/657 ————— 0s 23ms/step - accuracy: 0.6509 - f1_score: 0.6477 -
loss: 1.1950 - precision: 0.8104
Epoch 22: val_loss improved from 1.26808 to 1.26419, saving model to best_model_v
7.keras
657/657 ————— 19s 29ms/step - accuracy: 0.6509 - f1_score: 0.6477
- loss: 1.1950 - precision: 0.8104 - val_accuracy: 0.6412 - val_f1_score: 0.6403
- val_loss: 1.2642 - val_precision: 0.8121
Epoch 23/25
655/657 ————— 0s 24ms/step - accuracy: 0.6439 - f1_score: 0.6409 -
loss: 1.2001 - precision: 0.8119
Epoch 23: val_loss did not improve from 1.26419
657/657 ————— 19s 28ms/step - accuracy: 0.6439 - f1_score: 0.6409
- loss: 1.2000 - precision: 0.8119 - val_accuracy: 0.6388 - val_f1_score: 0.6371
- val_loss: 1.2743 - val_precision: 0.8079
Epoch 24/25
655/657 ————— 0s 23ms/step - accuracy: 0.6463 - f1_score: 0.6430 -
loss: 1.1979 - precision: 0.8090
Epoch 24: val_loss did not improve from 1.26419
657/657 ————— 19s 28ms/step - accuracy: 0.6463 - f1_score: 0.6430
- loss: 1.1978 - precision: 0.8090 - val_accuracy: 0.6363 - val_f1_score: 0.6325

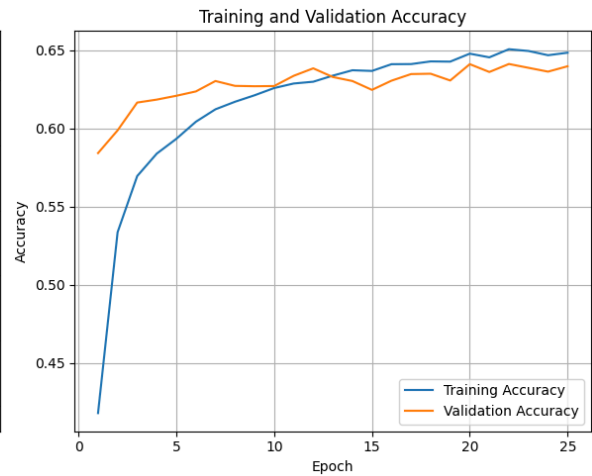
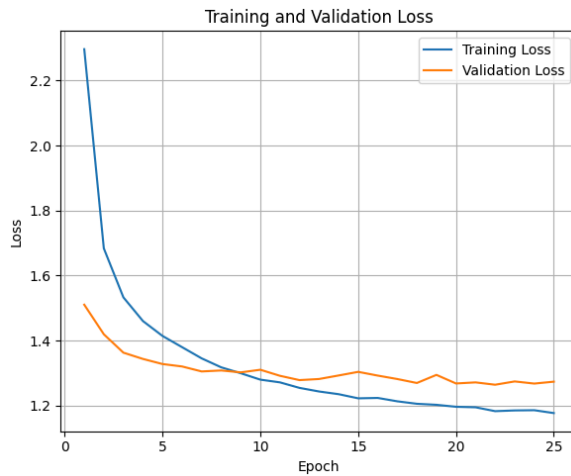
- val_loss: 1.2677 - val_precision: 0.8041

Epoch 25/25

656/657 ————— 0s 24ms/step - accuracy: 0.6443 - f1_score: 0.6418 -
loss: 1.1963 - precision: 0.8081

Epoch 25: val_loss did not improve from 1.26419

657/657 ————— 19s 28ms/step - accuracy: 0.6443 - f1_score: 0.6418
- loss: 1.1963 - precision: 0.8081 - val_accuracy: 0.6398 - val_f1_score: 0.6373
- val_loss: 1.2736 - val_precision: 0.8078



141/141 ————— 11s 38ms/step - accuracy: 0.6237 - f1_score: 0.6161
- loss: 1.2869 - precision: 0.8027

Test Accuracy: 63.68%

Test Loss: 1.2638587951660156

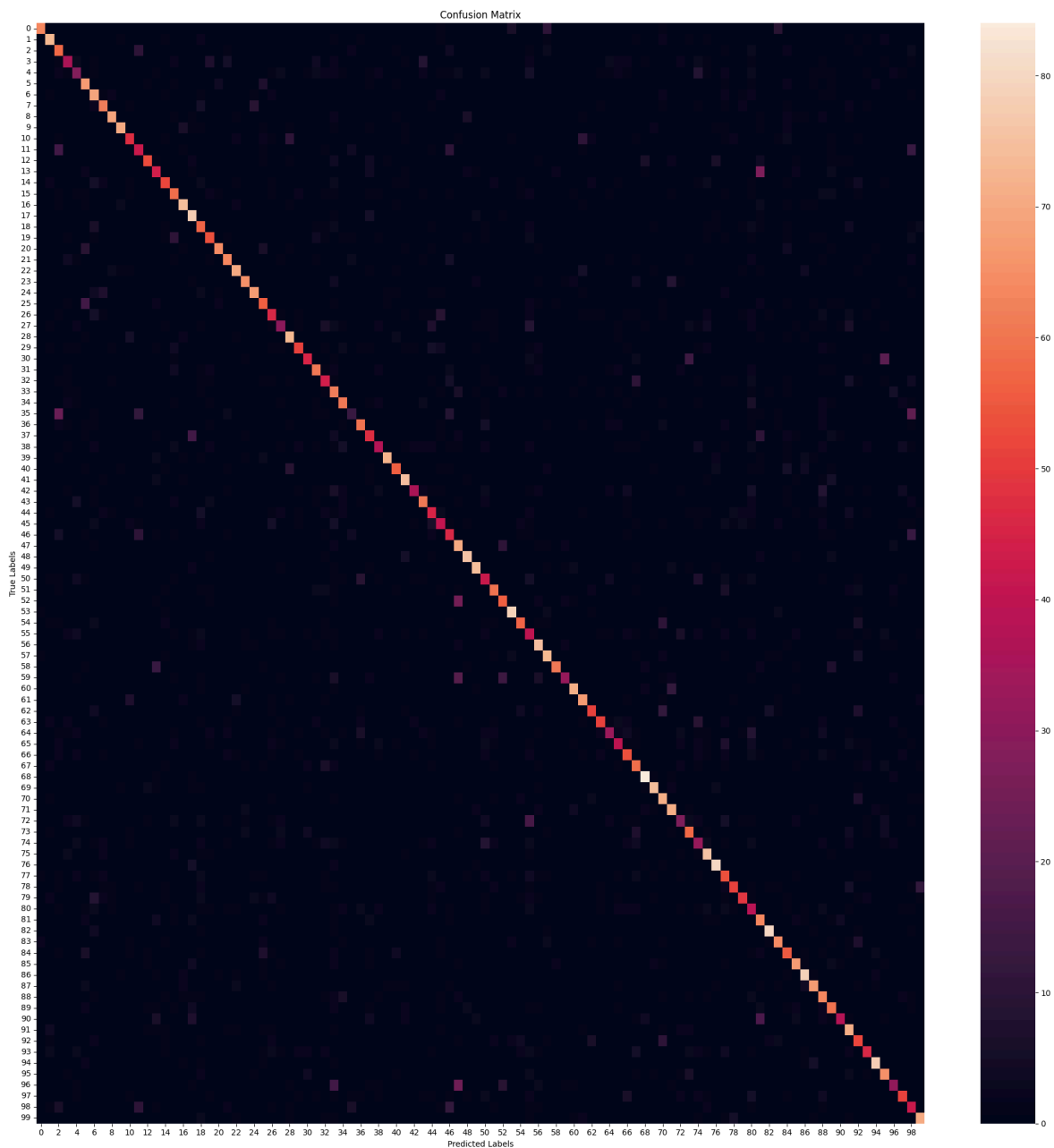
Test Precision: 81.13%

Test F1 Scores (Per Class): [79.74683 77.65957 52.727272 47.05882 34.999992 6
4.11483 62.443436

70.05649	81.17647	83.33333	58.749992	47.191006	69.73684	49.438198
64.197525	60.82474	80.43478	70.90909	54.54545	62.427734	78.5276
71.18643	77.34807	76.92307	77.456635	59.574467	52.571426	41.379307
76.041664	65.80644	59.999996	68.53932	49.438198	48.062004	67.40331
20.96774	68.181816	54.335255	49.350643	86.74699	68.29269	79.787224
53.73134	68.926544	49.999992	49.10179	47.66839	55.555553	83.060104
82.22221	44.808735	70.17543	61.538452	85.86956	68.67469	36.697243
74.25742	75.64766	78.70967	48.854958	80.681816	72.04301	64.15094
66.666664	45.83333	51.282047	60.33519	58.12807	85.71428	83.615814
65.74073	73.19587	37.499992	62.702698	35.555546	75.99999	83.76962
52.427177	65.80644	63.636356	35.616432	53.389828	87.91208	71.50838
58.82352	74.03315	81.025635	77.01149	57.27272	61.53846	55.072464
64.84018	53.06122	58.59872	86.666664	64.999985	43.35664	63.354034
40.191383	76.92307					

Average Test F1 Scores:63.55

282/282 ————— 11s 28ms/step



	precision	recall	f1-score	support
beaver	0.93	0.70	0.80	90
dolphin	0.74	0.81	0.78	90
otter	0.45	0.64	0.53	90
seal	0.57	0.40	0.47	90
whale	0.40	0.31	0.35	90
aquarium fish	0.56	0.74	0.64	90
flatfish	0.53	0.77	0.62	90
ray	0.71	0.69	0.70	90
shark	0.86	0.77	0.81	90
trout	0.90	0.78	0.83	90
orchids	0.67	0.52	0.59	90
poppies	0.48	0.47	0.47	90
roses	0.85	0.59	0.70	90
sunflowers	0.51	0.49	0.50	90
tulips	0.72	0.58	0.64	90
bottles	0.57	0.66	0.61	90
bowls	0.79	0.82	0.80	90
cans	0.60	0.87	0.71	90
cups	0.48	0.63	0.55	90
plates	0.65	0.60	0.62	90
apples	0.88	0.71	0.79	90
mushrooms	0.72	0.70	0.71	90
oranges	0.77	0.78	0.77	90
pears	0.82	0.72	0.77	90
sweet peppers	0.81	0.74	0.77	90
clock	0.57	0.62	0.60	90
computer keyboard	0.54	0.51	0.53	90
lamp	0.55	0.33	0.41	90
telephone	0.72	0.81	0.76	90
television	0.78	0.57	0.66	90
bed	0.75	0.50	0.60	90
chair	0.69	0.68	0.69	90
couch	0.50	0.49	0.49	90
table	0.37	0.69	0.48	90
wardrobe	0.67	0.68	0.67	90
bee	0.38	0.14	0.21	90
beetle	0.70	0.67	0.68	90
butterfly	0.57	0.52	0.54	90
caterpillar	0.59	0.42	0.49	90
cockroach	0.95	0.80	0.87	90
bear	0.76	0.62	0.68	90
leopard	0.77	0.83	0.80	90
lion	0.82	0.40	0.54	90
tiger	0.70	0.68	0.69	90
wolf	0.50	0.50	0.50	90
bridge	0.53	0.46	0.49	90
castle	0.45	0.51	0.48	90
house	0.43	0.78	0.56	90
road	0.82	0.84	0.83	90
skyscraper	0.82	0.82	0.82	90
cloud	0.45	0.47	0.46	90
forest	0.74	0.67	0.70	90
mountain	0.61	0.62	0.62	90
plain	0.84	0.88	0.86	90
sea	0.75	0.63	0.69	90
camel	0.31	0.44	0.37	90
cattle	0.67	0.83	0.74	90
chimpanzee	0.71	0.81	0.76	90

elephant	0.92	0.68	0.78	90
kangaroo	0.78	0.36	0.49	90
fox	0.83	0.79	0.81	90
porcupine	0.70	0.74	0.72	90
possum	0.74	0.57	0.64	90
raccoon	0.81	0.57	0.67	90
skunk	0.61	0.37	0.46	90
crab	0.61	0.44	0.51	90
lobster	0.61	0.60	0.60	90
snail	0.52	0.66	0.58	90
spider	0.79	0.93	0.86	90
worm	0.85	0.82	0.84	90
baby	0.56	0.79	0.66	90
boy	0.68	0.79	0.73	90
girl	0.50	0.30	0.38	90
man	0.61	0.64	0.63	90
woman	0.36	0.36	0.36	90
crocodile	0.69	0.84	0.76	90
dinosaur	0.79	0.89	0.84	90
lizard	0.47	0.60	0.52	90
snake	0.78	0.57	0.66	90
turtle	0.77	0.54	0.64	90
hamster	0.30	0.43	0.36	90
mouse	0.43	0.70	0.53	90
rabbit	0.87	0.89	0.88	90
shrew	0.72	0.71	0.72	90
squirrel	0.57	0.61	0.59	90
maple	0.74	0.74	0.74	90
oak	0.75	0.88	0.81	90
palm	0.80	0.74	0.77	90
pine	0.48	0.70	0.57	90
willow	0.57	0.67	0.62	90
bicycle	0.79	0.42	0.55	90
bus	0.55	0.79	0.65	90
motorcycle	0.49	0.58	0.53	90
pickup truck	0.69	0.51	0.59	90
train	0.87	0.87	0.87	90
lawn-mower	0.59	0.72	0.65	90
rocket	0.58	0.34	0.43	90
streetcar	0.72	0.57	0.63	90
tank	0.35	0.47	0.40	90
tractor	0.76	0.78	0.77	90
accuracy			0.64	9000
macro avg	0.66	0.64	0.64	9000
weighted avg	0.66	0.64	0.64	9000

Model: "sequential_11"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_11 (GlobalAveragePooling2D)	(None , 2048)
dense_22 (Dense)	(None , 512)
dropout_11 (Dropout)	(None , 512)
dense_23 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1314/1314 ————— 0s 27ms/step - accuracy: 0.6679 - f1_score: 0.6658
- loss: 1.1179 - precision: 0.8272
Epoch 1: val_loss improved from inf to 1.20007, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 56s 35ms/step - accuracy: 0.6679 - f1_score: 0.6658 - loss: 1.1179 - precision: 0.8272 - val_accuracy: 0.6573 - val_f1_score: 0.6561 - val_loss: 1.2001 - val_precision: 0.8215
Epoch 2/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.6946 - f1_score: 0.6925 - loss: 1.0229 - precision: 0.8426
Epoch 2: val_loss improved from 1.20007 to 1.18937, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6946 - f1_score: 0.6925 - loss: 1.0229 - precision: 0.8426 - val_accuracy: 0.6577 - val_f1_score: 0.6562 - val_loss: 1.1894 - val_precision: 0.8205
Epoch 3/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7040 - f1_score: 0.7017 - loss: 0.9905 - precision: 0.8485
Epoch 3: val_loss improved from 1.18937 to 1.18375, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7040 - f1_score: 0.7016 - loss: 0.9905 - precision: 0.8485 - val_accuracy: 0.6586 - val_f1_score: 0.6570 - val_loss: 1.1837 - val_precision: 0.8188
Epoch 4/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7094 - f1_score: 0.7073 - loss: 0.9736 - precision: 0.8512
Epoch 4: val_loss improved from 1.18375 to 1.17899, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7094 - f1_score: 0.7073 - loss: 0.9736 - precision: 0.8512 - val_accuracy: 0.6613 - val_f1_score: 0.6597 - val_loss: 1.1790 - val_precision: 0.8199
Epoch 5/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7142 - f1_score: 0.7120 - loss: 0.9618 - precision: 0.8540
Epoch 5: val_loss improved from 1.17899 to 1.17500, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7142 - f1_score: 0.7120 - loss: 0.9618 - precision: 0.8540 - val_accuracy: 0.6617 - val_f1_score: 0.6603 - val_loss: 1.1750 - val_precision: 0.8204
Epoch 6/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7161 - f1_score: 0.7141 - loss: 0.9470 - precision: 0.8573
Epoch 6: val_loss improved from 1.17500 to 1.17098, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7161 - f1_score: 0.7141 - loss: 0.9471 - precision: 0.8573 - val_accuracy: 0.6608 - val_f1_score: 0.6592 - val_loss: 1.1710 - val_precision: 0.8191
Epoch 7/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7204 - f1_score: 0.7182 - loss: 0.9348 - precision: 0.8577
Epoch 7: val_loss improved from 1.17098 to 1.16921, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7204 - f1_score: 0.7182 - loss: 0.9348 - precision: 0.8577 - val_accuracy: 0.6619 - val_f1_score: 0.6604 - val_loss: 1.1692 - val_precision: 0.8182
Epoch 8/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7240 - f1_score: 0.7220 - loss: 0.9247 - precision: 0.8619
Epoch 8: val_loss improved from 1.16921 to 1.16640, saving model to best_model_fine_tune_v7.keras

```
ne_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7239 - f1_score: 0.722
0 - loss: 0.9247 - precision: 0.8619 - val_accuracy: 0.6622 - val_f1_score: 0.660
6 - val_loss: 1.1664 - val_precision: 0.8170
Epoch 9/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7244 - f1_score: 0.7222
- loss: 0.9162 - precision: 0.8607
Epoch 9: val_loss improved from 1.16640 to 1.16350, saving model to best_model_fi
ne_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7244 - f1_score: 0.722
2 - loss: 0.9162 - precision: 0.8607 - val_accuracy: 0.6637 - val_f1_score: 0.662
3 - val_loss: 1.1635 - val_precision: 0.8197
Epoch 10/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7289 - f1_score: 0.7270
- loss: 0.9071 - precision: 0.8628
Epoch 10: val_loss improved from 1.16350 to 1.16108, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7289 - f1_score: 0.727
0 - loss: 0.9071 - precision: 0.8628 - val_accuracy: 0.6658 - val_f1_score: 0.664
4 - val_loss: 1.1611 - val_precision: 0.8201
Epoch 11/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7318 - f1_score: 0.7298
- loss: 0.8950 - precision: 0.8650
Epoch 11: val_loss improved from 1.16108 to 1.15920, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7318 - f1_score: 0.729
8 - loss: 0.8950 - precision: 0.8650 - val_accuracy: 0.6652 - val_f1_score: 0.663
7 - val_loss: 1.1592 - val_precision: 0.8190
Epoch 12/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7329 - f1_score: 0.7310
- loss: 0.8911 - precision: 0.8686
Epoch 12: val_loss improved from 1.15920 to 1.15763, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7328 - f1_score: 0.731
0 - loss: 0.8911 - precision: 0.8686 - val_accuracy: 0.6659 - val_f1_score: 0.664
6 - val_loss: 1.1576 - val_precision: 0.8204
Epoch 13/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7373 - f1_score: 0.7353
- loss: 0.8829 - precision: 0.8706
Epoch 13: val_loss improved from 1.15763 to 1.15517, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7373 - f1_score: 0.735
3 - loss: 0.8830 - precision: 0.8706 - val_accuracy: 0.6663 - val_f1_score: 0.664
9 - val_loss: 1.1552 - val_precision: 0.8198
Epoch 14/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7389 - f1_score: 0.7369
- loss: 0.8726 - precision: 0.8717
Epoch 14: val_loss improved from 1.15517 to 1.15449, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7389 - f1_score: 0.736
9 - loss: 0.8726 - precision: 0.8717 - val_accuracy: 0.6658 - val_f1_score: 0.664
5 - val_loss: 1.1545 - val_precision: 0.8194
Epoch 15/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7392 - f1_score: 0.7372
- loss: 0.8703 - precision: 0.8713
Epoch 15: val_loss improved from 1.15449 to 1.15278, saving model to best_model_f
ine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7392 - f1_score: 0.737
2 - loss: 0.8703 - precision: 0.8713 - val_accuracy: 0.6654 - val_f1_score: 0.664
1 - val_loss: 1.1528 - val_precision: 0.8193
```

Epoch 16/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7409 - f1_score: 0.7387
- loss: 0.8599 - precision: 0.8738
Epoch 16: val_loss improved from 1.15278 to 1.15156, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7409 - f1_score: 0.7387 - loss: 0.8599 - precision: 0.8738 - val_accuracy: 0.6669 - val_f1_score: 0.6656 - val_loss: 1.1516 - val_precision: 0.8171
Epoch 17/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7437 - f1_score: 0.7418
- loss: 0.8509 - precision: 0.8729
Epoch 17: val_loss improved from 1.15156 to 1.14991, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7437 - f1_score: 0.7418 - loss: 0.8509 - precision: 0.8729 - val_accuracy: 0.6680 - val_f1_score: 0.6666 - val_loss: 1.1499 - val_precision: 0.8196
Epoch 18/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7457 - f1_score: 0.7438
- loss: 0.8498 - precision: 0.8760
Epoch 18: val_loss improved from 1.14991 to 1.14941, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7457 - f1_score: 0.7438 - loss: 0.8498 - precision: 0.8760 - val_accuracy: 0.6676 - val_f1_score: 0.6662 - val_loss: 1.1494 - val_precision: 0.8192
Epoch 19/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.7483 - f1_score: 0.7463
- loss: 0.8422 - precision: 0.8771
Epoch 19: val_loss improved from 1.14941 to 1.14838, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7482 - f1_score: 0.7463 - loss: 0.8422 - precision: 0.8770 - val_accuracy: 0.6678 - val_f1_score: 0.6664 - val_loss: 1.1484 - val_precision: 0.8190
Epoch 20/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7494 - f1_score: 0.7475
- loss: 0.8344 - precision: 0.8765
Epoch 20: val_loss improved from 1.14838 to 1.14726, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7493 - f1_score: 0.7475 - loss: 0.8344 - precision: 0.8765 - val_accuracy: 0.6682 - val_f1_score: 0.6668 - val_loss: 1.1473 - val_precision: 0.8192
Epoch 21/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.7537 - f1_score: 0.7518
- loss: 0.8243 - precision: 0.8801
Epoch 21: val_loss improved from 1.14726 to 1.14557, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7537 - f1_score: 0.7518 - loss: 0.8243 - precision: 0.8801 - val_accuracy: 0.6679 - val_f1_score: 0.6666 - val_loss: 1.1456 - val_precision: 0.8200
Epoch 22/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7523 - f1_score: 0.7504
- loss: 0.8249 - precision: 0.8795
Epoch 22: val_loss improved from 1.14557 to 1.14472, saving model to best_model_fine_tune_v7.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7522 - f1_score: 0.7504 - loss: 0.8250 - precision: 0.8795 - val_accuracy: 0.6686 - val_f1_score: 0.6671 - val_loss: 1.1447 - val_precision: 0.8182
Epoch 23/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.7559 - f1_score: 0.7541
- loss: 0.8179 - precision: 0.8778
Epoch 23: val_loss improved from 1.14472 to 1.14337, saving model to best_model_fine_tune_v7.keras

ine_tune_v7.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.7558 - f1_score: 0.7541 - loss: 0.8179 - precision: 0.8778 - val_accuracy: 0.6699 - val_f1_score: 0.6684 - val_loss: 1.1434 - val_precision: 0.8188

Epoch 24/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.7586 - f1_score: 0.7570 - loss: 0.8151 - precision: 0.8812

Epoch 24: val_loss improved from 1.14337 to 1.14314, saving model to best_model_fine_tune_v7.keras

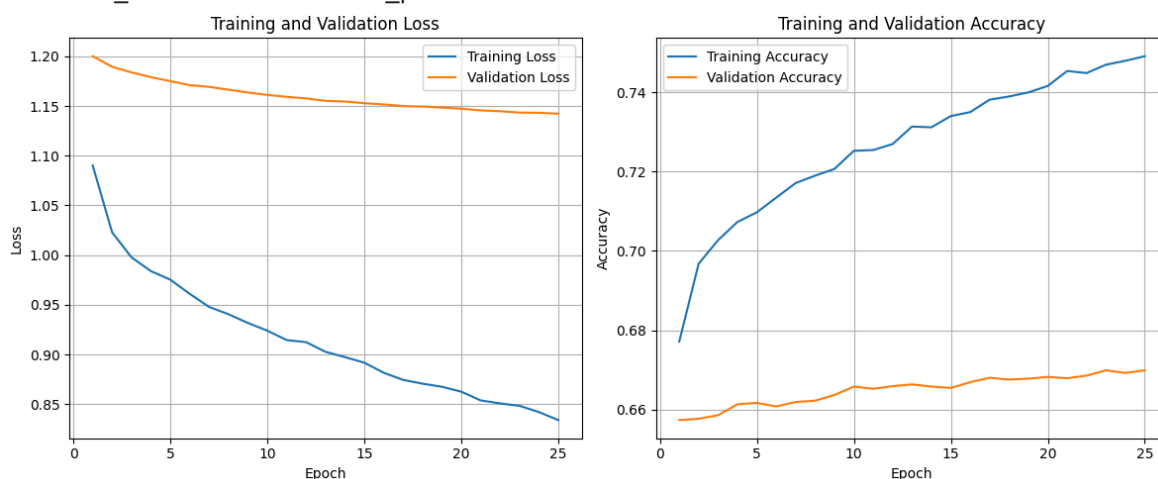
1314/1314 ————— **36s** 27ms/step - accuracy: 0.7586 - f1_score: 0.7570 - loss: 0.8151 - precision: 0.8812 - val_accuracy: 0.6692 - val_f1_score: 0.6678 - val_loss: 1.1431 - val_precision: 0.8201

Epoch 25/25

1313/1314 ————— **0s** 24ms/step - accuracy: 0.7585 - f1_score: 0.7566 - loss: 0.8014 - precision: 0.8839

Epoch 25: val_loss improved from 1.14314 to 1.14227, saving model to best_model_fine_tune_v7.keras

1314/1314 ————— **35s** 27ms/step - accuracy: 0.7585 - f1_score: 0.7566 - loss: 0.8015 - precision: 0.8839 - val_accuracy: 0.6699 - val_f1_score: 0.6684 - val_loss: 1.1423 - val_precision: 0.8196



141/141 ————— **11s** 37ms/step - accuracy: 0.6620 - f1_score: 0.6533 - loss: 1.1620 - precision: 0.8062

Test Accuracy: 67.11%

Test Loss: 1.1412248611450195

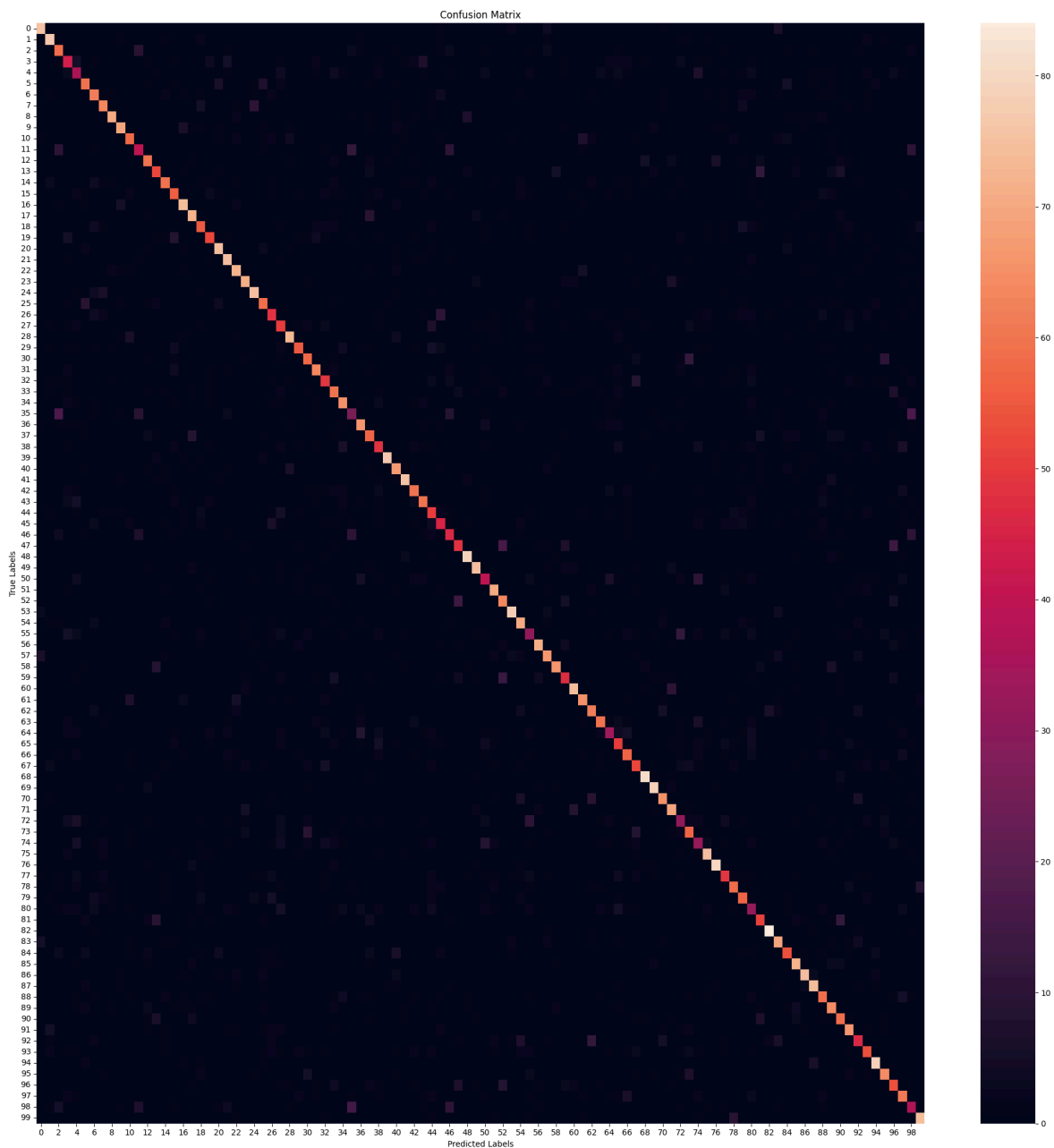
Test Precision: 81.53%

Test F1 Scores (Per Class): [83.060104 80.628265 59.595955 48.88888 39.779003 66.666664 63.589745

71.18643 81.87134 78.85714 66.666664 49.079746 67.415726 59.770107
69.76743 62.222218 79.569885 76.59574 60.773476 62.650597 80.64516
77.31957 77.83783 79.329605 81.08108 65.55555 54.857136 51.282047
80.66298 68.322975 64.80447 72.83237 56.64739 61.53846 70.65217
32.098763 71.73912 61.62162 54.545456 90.58823 70.526306 84.91619
67.796616 69.36415 51.282047 52.631577 51.111107 57.142853 85.405396
82.41757 47.058823 77.34807 64.61538 87.15083 71.428566 37.349392
81.14285 72.92817 78.57143 55.95237 82.22221 74.28571 66.66666
71.428566 44.736843 53.763435 62.222218 58.100555 87.56756 85.405396
73.863625 72.251305 38.036804 61.290314 37.209297 80.64516 88.39778
56.97674 67.42856 63.687145 34.972675 57.627117 91.304344 72.63157
63.529408 77.00535 83.798874 78.723404 63.783775 65.30612 61.13989
70.967735 55.421684 62.427734 86.33879 70.71823 56.842102 63.541664
40.437157 80.87432]

Average Test F1 Scores:66.92

282/282 ————— **13s** 35ms/step



	precision	recall	f1-score	support
beaver	0.82	0.84	0.83	90
dolphin	0.76	0.86	0.81	90
otter	0.55	0.66	0.60	90
seal	0.49	0.49	0.49	90
whale	0.40	0.40	0.40	90
aquarium fish	0.67	0.67	0.67	90
flatfish	0.59	0.69	0.64	90
ray	0.72	0.70	0.71	90
shark	0.86	0.78	0.82	90
trout	0.81	0.77	0.79	90
orchids	0.69	0.64	0.67	90
poppies	0.55	0.44	0.49	90
roses	0.68	0.67	0.67	90
sunflowers	0.62	0.58	0.60	90
tulips	0.73	0.67	0.70	90
bottles	0.62	0.62	0.62	90
bowls	0.77	0.82	0.80	90
cans	0.73	0.80	0.77	90
cups	0.60	0.61	0.61	90
plates	0.68	0.58	0.63	90
apples	0.78	0.83	0.81	90
mushrooms	0.72	0.83	0.77	90
oranges	0.76	0.80	0.78	90
pears	0.80	0.79	0.79	90
sweet peppers	0.79	0.83	0.81	90
clock	0.66	0.66	0.66	90
computer keyboard	0.56	0.53	0.55	90
lamp	0.48	0.56	0.51	90
telephone	0.80	0.81	0.81	90
television	0.77	0.61	0.68	90
bed	0.65	0.64	0.65	90
chair	0.76	0.70	0.73	90
couch	0.59	0.54	0.57	90
table	0.57	0.67	0.62	90
wardrobe	0.69	0.72	0.71	90
bee	0.36	0.29	0.32	90
beetle	0.70	0.73	0.72	90
butterfly	0.60	0.63	0.62	90
caterpillar	0.56	0.53	0.55	90
cockroach	0.96	0.86	0.91	90
bear	0.66	0.74	0.70	90
leopard	0.85	0.84	0.85	90
lion	0.69	0.67	0.68	90
tiger	0.72	0.67	0.69	90
wolf	0.48	0.56	0.51	90
bridge	0.56	0.50	0.53	90
castle	0.51	0.51	0.51	90
house	0.62	0.53	0.57	90
road	0.83	0.88	0.85	90
skyscraper	0.82	0.83	0.82	90
cloud	0.50	0.44	0.47	90
forest	0.77	0.78	0.77	90
mountain	0.60	0.70	0.65	90
plain	0.88	0.87	0.87	90
sea	0.66	0.78	0.71	90
camel	0.41	0.34	0.37	90
cattle	0.84	0.79	0.81	90
chimpanzee	0.73	0.73	0.73	90

elephant	0.85	0.73	0.79	90
kangaroo	0.60	0.52	0.56	90
fox	0.82	0.82	0.82	90
porcupine	0.76	0.72	0.74	90
possum	0.65	0.69	0.67	90
raccoon	0.78	0.67	0.72	90
skunk	0.55	0.38	0.45	90
crab	0.52	0.56	0.54	90
lobster	0.63	0.63	0.63	90
snail	0.58	0.58	0.58	90
spider	0.85	0.90	0.88	90
worm	0.83	0.88	0.85	90
baby	0.76	0.72	0.74	90
boy	0.68	0.77	0.72	90
girl	0.42	0.34	0.38	90
man	0.59	0.63	0.61	90
woman	0.39	0.36	0.37	90
crocodile	0.78	0.83	0.81	90
dinosaur	0.88	0.89	0.88	90
lizard	0.60	0.54	0.57	90
snake	0.69	0.66	0.67	90
turtle	0.64	0.63	0.64	90
hamster	0.34	0.36	0.35	90
mouse	0.59	0.57	0.58	90
rabbit	0.89	0.93	0.91	90
shrew	0.69	0.77	0.73	90
squirrel	0.68	0.60	0.64	90
maple	0.74	0.80	0.77	90
oak	0.84	0.83	0.84	90
palm	0.76	0.82	0.79	90
pine	0.62	0.66	0.64	90
willow	0.60	0.71	0.65	90
bicycle	0.57	0.66	0.61	90
bus	0.69	0.73	0.71	90
motorcycle	0.61	0.51	0.55	90
pickup truck	0.65	0.60	0.62	90
train	0.85	0.88	0.86	90
lawn-mower	0.70	0.71	0.71	90
rocket	0.54	0.60	0.57	90
streetcar	0.60	0.68	0.64	90
tank	0.40	0.41	0.40	90
tractor	0.80	0.82	0.81	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.67	9000
weighted avg	0.67	0.67	0.67	9000

**RMSProp, Dropout(0.6),
weight_decay(3e-1), l2(3e-1) - no epoch
stop for either part (may need more
momentum later)**

```
In [ ]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>
        # Load CIFAR-100 dataset
```

```

(X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

# Split (8000) of training data into temporary set
X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_size=0.1)
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp, y_temp, test_size=0.5)
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5)

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & accuracy
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'fish',
                 'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', 'bottle',
                 'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock',
                 'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardrobe',
                 'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wolf',
                 'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea',
                 'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 'squirrel',
                 'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile',
                 'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple',
                 'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mower',
                 'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100), dtype=tf.float32))
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE))

```



```

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                      .map(augment_dataset, num_parallel_calls=tf.data.experimental.
#                      .batch(batch_size)
#                      .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<Pre-trained model>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes

```

```

])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'>
    print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.RMSprop(learning_rate=1e-3, momentum=0.1,
    loss='categorical_crossentropy',
    metrics=['accuracy', 'precision', 'f1_score']))

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
    patience=5, # Num. epochs with no improvement
    restore_best_weights=True)

#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#    factor=0.1, # Reduce lr by a factor
#    patience=3, # Num epochs w/ no improvement
#    min_lr=1e-6, # Min lr
#    verbose=1)
#tensorboard = TensorBoard(log_dir='./Logs', # Logs directory
#    histogram_freq=1, # Logs histograms for weights/acc
#    write_graph=True, # Logs graph of model
#    write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v8.keras', # Path where the best model is saved
    monitor='val_loss', # What to monitor (validation loss)
    save_best_only=True, # Only save model when val loss is best
    mode='min', # Looking to minimize the validation loss
    verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v8.keras', # Path where the best model is saved
    monitor='val_loss', # What to monitor (validation loss)
    save_best_only=True, # Only save model when val loss is best
    mode='min', # Looking to minimize the validation loss
    verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
    batch_size=batch_size, callbacks=[checkpoint], verbose=1)

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

```

```

# Extract Loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v8.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features

```

```

model.compile(optimizer=optimizers.Adam(learning_rate=1e-4),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset,
                              batch_size=batch_size, callbacks=[checkpoint_f

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v8.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

```

```

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

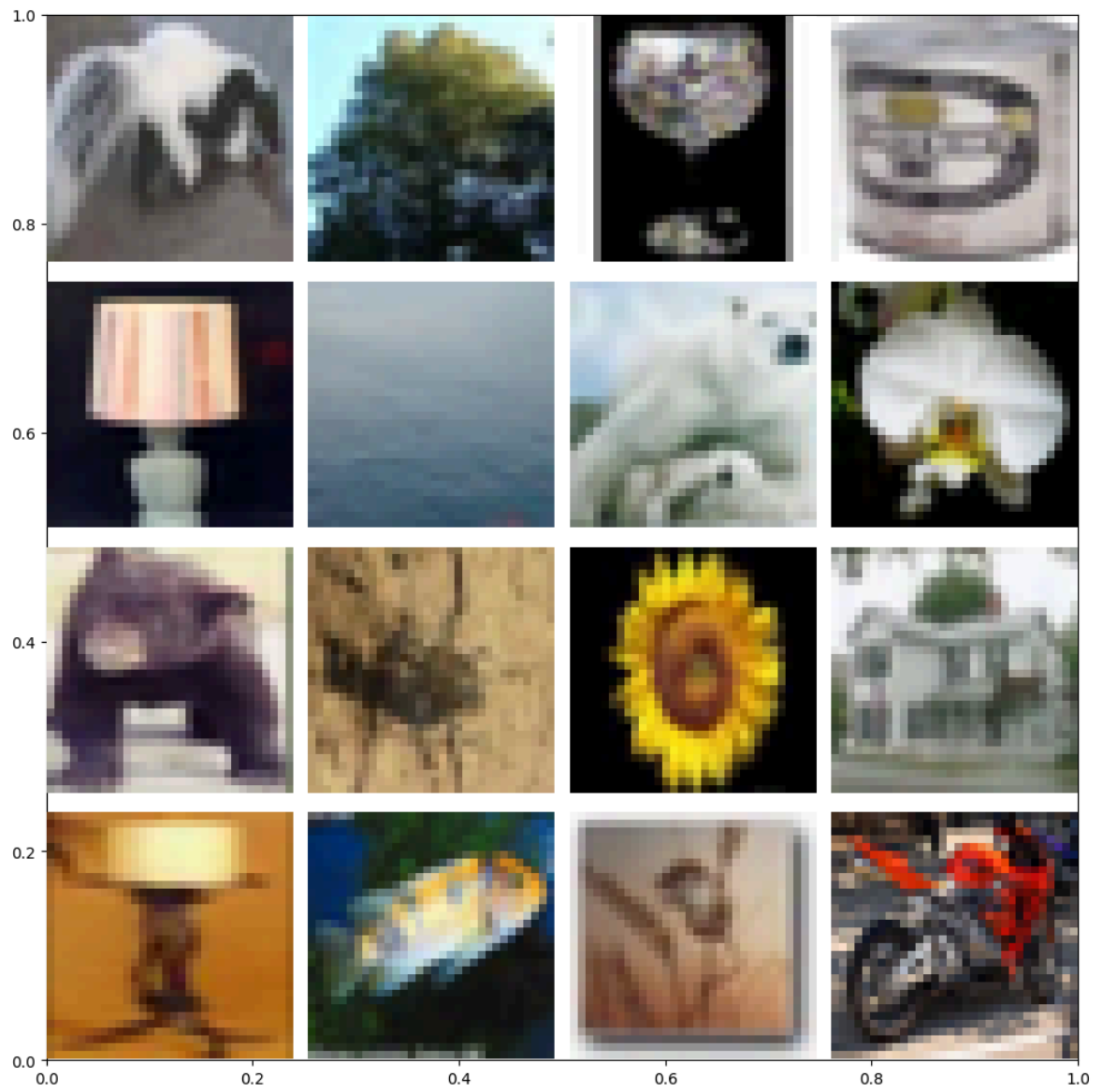
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

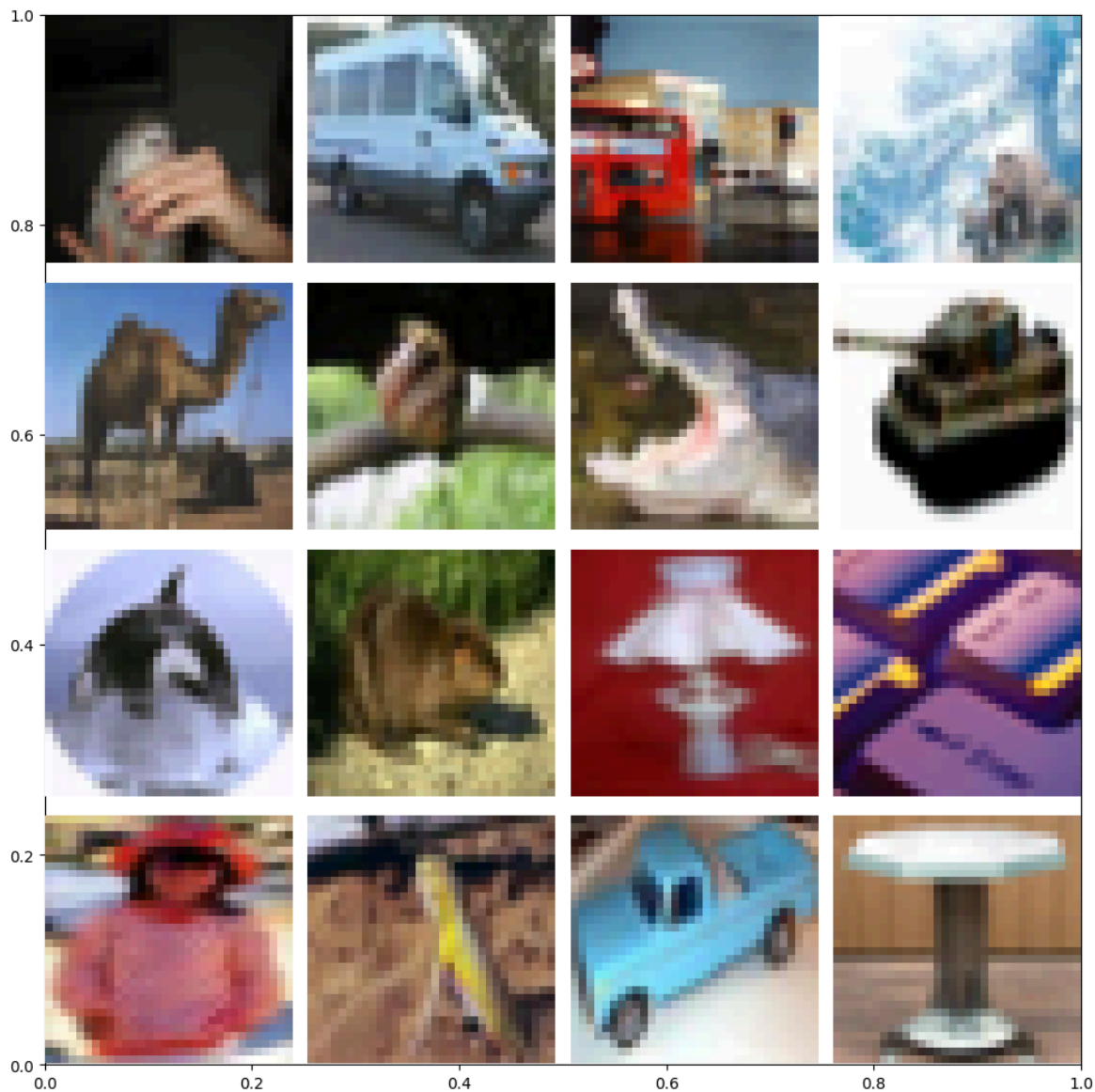
y_test.shape: (9000, 1)

labels[40462].shape: (1,)

imgs[40462].shape: (32, 32, 3)

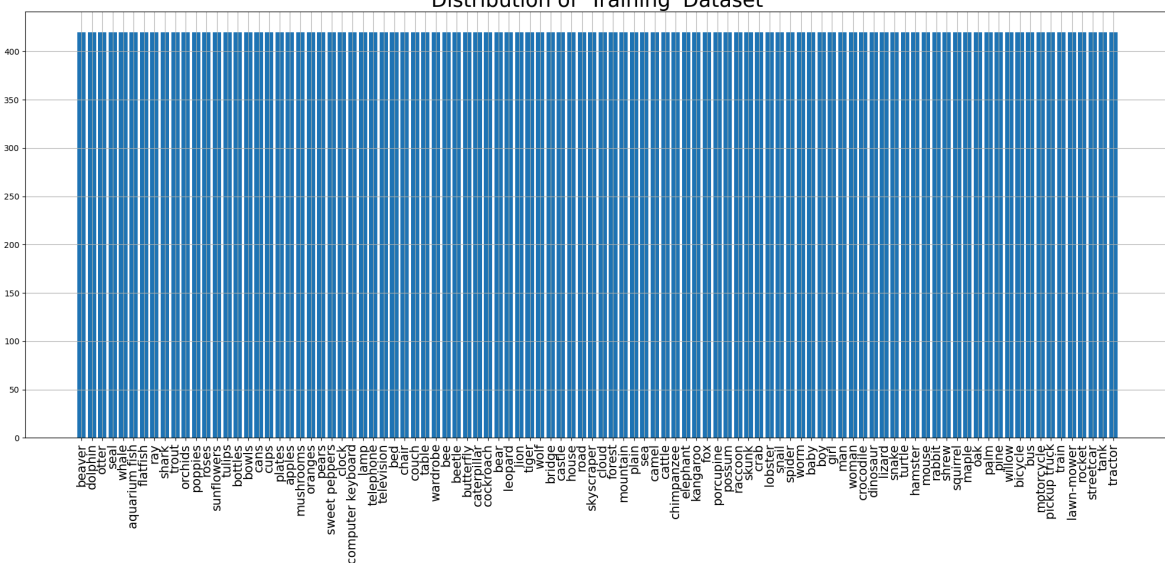


```
labels[1062].shape: (1,)  
imgs[1062].shape: (32, 32, 3)
```

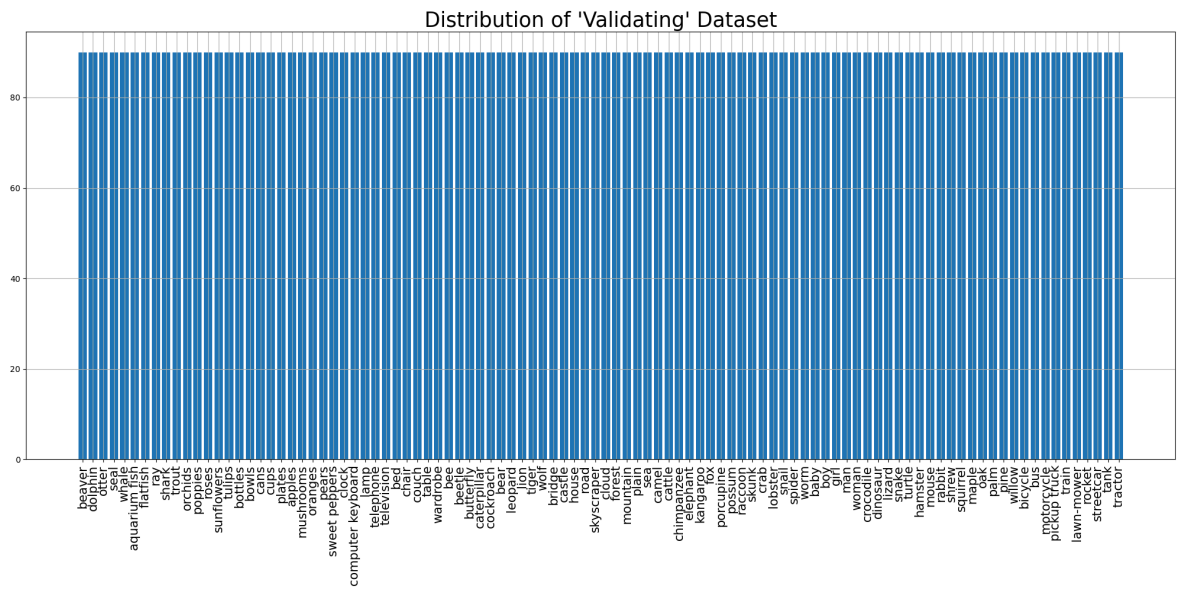


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

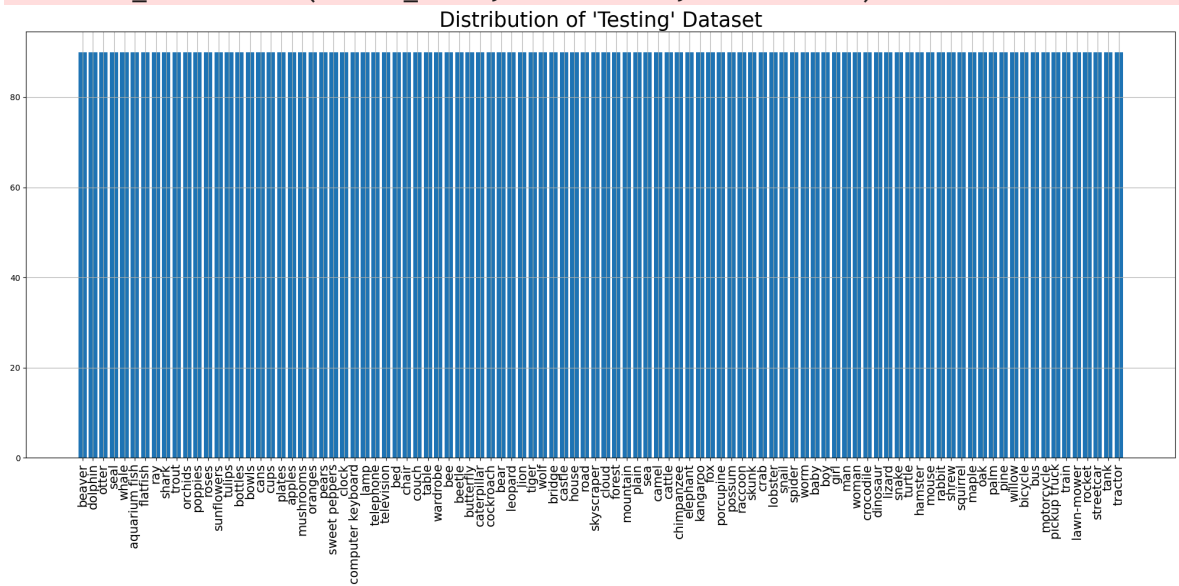
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_12"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_12 (GlobalAveragePooling2D)	(None , 2048)
dense_24 (Dense)	(None , 512)
dropout_12 (Dropout)	(None , 512)
dense_25 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 29ms/step - accuracy: 0.3221 - f1_score: 0.3164 - loss: 2.8499 - precision: 0.6632
Epoch 1: val_loss improved from inf to 1.51041, saving model to best_model_v8.keras
657/657 ————— 40s 45ms/step - accuracy: 0.3223 - f1_score: 0.3165 - loss: 2.8491 - precision: 0.6632 - val_accuracy: 0.5802 - val_f1_score: 0.5767 - val_loss: 1.5104 - val_precision: 0.7802
Epoch 2/25
655/657 ————— 0s 24ms/step - accuracy: 0.5223 - f1_score: 0.5184 - loss: 1.7880 - precision: 0.7200
Epoch 2: val_loss improved from 1.51041 to 1.42714, saving model to best_model_v8.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5223 - f1_score: 0.5184 - loss: 1.7878 - precision: 0.7200 - val_accuracy: 0.5980 - val_f1_score: 0.5936 - val_loss: 1.4271 - val_precision: 0.7848
Epoch 3/25
657/657 ————— 0s 24ms/step - accuracy: 0.5530 - f1_score: 0.5487 - loss: 1.6450 - precision: 0.7375
Epoch 3: val_loss did not improve from 1.42714
657/657 ————— 19s 29ms/step - accuracy: 0.5530 - f1_score: 0.5487 - loss: 1.6450 - precision: 0.7375 - val_accuracy: 0.6016 - val_f1_score: 0.5952 - val_loss: 1.4352 - val_precision: 0.7683
Epoch 4/25
655/657 ————— 0s 24ms/step - accuracy: 0.5674 - f1_score: 0.5642 - loss: 1.5783 - precision: 0.7444
Epoch 4: val_loss improved from 1.42714 to 1.37101, saving model to best_model_v8.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5675 - f1_score: 0.5642 - loss: 1.5782 - precision: 0.7444 - val_accuracy: 0.6179 - val_f1_score: 0.6153 - val_loss: 1.3710 - val_precision: 0.7870
Epoch 5/25
655/657 ————— 0s 24ms/step - accuracy: 0.5835 - f1_score: 0.5802 - loss: 1.5384 - precision: 0.7514
Epoch 5: val_loss did not improve from 1.37101
657/657 ————— 19s 29ms/step - accuracy: 0.5836 - f1_score: 0.5802 - loss: 1.5383 - precision: 0.7514 - val_accuracy: 0.6111 - val_f1_score: 0.6085 - val_loss: 1.3905 - val_precision: 0.7828
Epoch 6/25
655/657 ————— 0s 24ms/step - accuracy: 0.5896 - f1_score: 0.5861 - loss: 1.5012 - precision: 0.7564
Epoch 6: val_loss did not improve from 1.37101
657/657 ————— 19s 29ms/step - accuracy: 0.5896 - f1_score: 0.5861 - loss: 1.5011 - precision: 0.7564 - val_accuracy: 0.6214 - val_f1_score: 0.6197 - val_loss: 1.3784 - val_precision: 0.7850
Epoch 7/25
656/657 ————— 0s 24ms/step - accuracy: 0.5989 - f1_score: 0.5954 - loss: 1.4734 - precision: 0.7543
Epoch 7: val_loss did not improve from 1.37101
657/657 ————— 19s 29ms/step - accuracy: 0.5989 - f1_score: 0.5954 - loss: 1.4734 - precision: 0.7543 - val_accuracy: 0.6234 - val_f1_score: 0.6195 - val_loss: 1.3807 - val_precision: 0.7773
Epoch 8/25
655/657 ————— 0s 24ms/step - accuracy: 0.6031 - f1_score: 0.5997 - loss: 1.4449 - precision: 0.7566
Epoch 8: val_loss did not improve from 1.37101
657/657 ————— 19s 29ms/step - accuracy: 0.6032 - f1_score: 0.5998 - loss: 1.4448 - precision: 0.7566 - val_accuracy: 0.6188 - val_f1_score: 0.6182 - val_loss: 1.3860 - val_precision: 0.7742
Epoch 9/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6054 - f1_score: 0.6026 - loss: 1.4267 - precision: 0.7613
Epoch 9: val_loss did not improve from 1.37101

657/657 ————— **19s** 29ms/step - accuracy: 0.6054 - f1_score: 0.6026 - loss: 1.4266 - precision: 0.7614 - val_accuracy: 0.6161 - val_f1_score: 0.6144 - val_loss: 1.3963 - val_precision: 0.7717
Epoch 10/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6022 - f1_score: 0.5991 - loss: 1.4211 - precision: 0.7581
Epoch 10: val_loss did not improve from 1.37101

657/657 ————— **19s** 29ms/step - accuracy: 0.6022 - f1_score: 0.5991 - loss: 1.4211 - precision: 0.7581 - val_accuracy: 0.6138 - val_f1_score: 0.6105 - val_loss: 1.3935 - val_precision: 0.7787
Epoch 11/25

657/657 ————— **0s** 24ms/step - accuracy: 0.6158 - f1_score: 0.6127 - loss: 1.3976 - precision: 0.7666
Epoch 11: val_loss did not improve from 1.37101

657/657 ————— **19s** 29ms/step - accuracy: 0.6158 - f1_score: 0.6127 - loss: 1.3976 - precision: 0.7666 - val_accuracy: 0.6196 - val_f1_score: 0.6176 - val_loss: 1.3857 - val_precision: 0.7730
Epoch 12/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6123 - f1_score: 0.6095 - loss: 1.3948 - precision: 0.7660
Epoch 12: val_loss improved from 1.37101 to 1.37037, saving model to best_model_v8.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6123 - f1_score: 0.6095 - loss: 1.3947 - precision: 0.7661 - val_accuracy: 0.6236 - val_f1_score: 0.6223 - val_loss: 1.3704 - val_precision: 0.7723
Epoch 13/25

657/657 ————— **0s** 24ms/step - accuracy: 0.6131 - f1_score: 0.6103 - loss: 1.3974 - precision: 0.7645
Epoch 13: val_loss did not improve from 1.37037

657/657 ————— **19s** 29ms/step - accuracy: 0.6131 - f1_score: 0.6103 - loss: 1.3974 - precision: 0.7645 - val_accuracy: 0.6211 - val_f1_score: 0.6185 - val_loss: 1.3900 - val_precision: 0.7675
Epoch 14/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6191 - f1_score: 0.6162 - loss: 1.3846 - precision: 0.7646
Epoch 14: val_loss did not improve from 1.37037

657/657 ————— **19s** 29ms/step - accuracy: 0.6192 - f1_score: 0.6162 - loss: 1.3845 - precision: 0.7647 - val_accuracy: 0.6239 - val_f1_score: 0.6237 - val_loss: 1.3752 - val_precision: 0.7748
Epoch 15/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6153 - f1_score: 0.6127 - loss: 1.3761 - precision: 0.7682
Epoch 15: val_loss did not improve from 1.37037

657/657 ————— **19s** 29ms/step - accuracy: 0.6153 - f1_score: 0.6127 - loss: 1.3760 - precision: 0.7682 - val_accuracy: 0.6236 - val_f1_score: 0.6211 - val_loss: 1.3798 - val_precision: 0.7704
Epoch 16/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6191 - f1_score: 0.6164 - loss: 1.3880 - precision: 0.7680
Epoch 16: val_loss did not improve from 1.37037

657/657 ————— **19s** 29ms/step - accuracy: 0.6191 - f1_score: 0.6164 - loss: 1.3879 - precision: 0.7680 - val_accuracy: 0.6247 - val_f1_score: 0.6224 - val_loss: 1.3808 - val_precision: 0.7687
Epoch 17/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6180 - f1_score: 0.6148 - loss: 1.3682 - precision: 0.7713
Epoch 17: val_loss did not improve from 1.37037

657/657 ————— **19s** 29ms/step - accuracy: 0.6180 - f1_score: 0.6148
- loss: 1.3681 - precision: 0.7713 - val_accuracy: 0.6286 - val_f1_score: 0.6279
- val_loss: 1.3747 - val_precision: 0.7788
Epoch 18/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6217 - f1_score: 0.6193 -
loss: 1.3595 - precision: 0.7618
Epoch 18: val_loss improved from 1.37037 to 1.36687, saving model to best_model_v
8.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6217 - f1_score: 0.6194
- loss: 1.3595 - precision: 0.7618 - val_accuracy: 0.6267 - val_f1_score: 0.6229
- val_loss: 1.3669 - val_precision: 0.7705
Epoch 19/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6165 - f1_score: 0.6133 -
loss: 1.3637 - precision: 0.7617
Epoch 19: val_loss did not improve from 1.36687

657/657 ————— **19s** 29ms/step - accuracy: 0.6165 - f1_score: 0.6133
- loss: 1.3636 - precision: 0.7618 - val_accuracy: 0.6207 - val_f1_score: 0.6220
- val_loss: 1.3812 - val_precision: 0.7777
Epoch 20/25

657/657 ————— **0s** 24ms/step - accuracy: 0.6222 - f1_score: 0.6195 -
loss: 1.3616 - precision: 0.7682
Epoch 20: val_loss improved from 1.36687 to 1.35954, saving model to best_model_v
8.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.6222 - f1_score: 0.6195
- loss: 1.3616 - precision: 0.7682 - val_accuracy: 0.6286 - val_f1_score: 0.6281
- val_loss: 1.3595 - val_precision: 0.7708
Epoch 21/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6229 - f1_score: 0.6200 -
loss: 1.3540 - precision: 0.7687
Epoch 21: val_loss did not improve from 1.35954

657/657 ————— **19s** 29ms/step - accuracy: 0.6230 - f1_score: 0.6200
- loss: 1.3539 - precision: 0.7687 - val_accuracy: 0.6300 - val_f1_score: 0.6303
- val_loss: 1.3639 - val_precision: 0.7749
Epoch 22/25

657/657 ————— **0s** 24ms/step - accuracy: 0.6228 - f1_score: 0.6202 -
loss: 1.3546 - precision: 0.7709
Epoch 22: val_loss did not improve from 1.35954

657/657 ————— **19s** 29ms/step - accuracy: 0.6228 - f1_score: 0.6202
- loss: 1.3546 - precision: 0.7709 - val_accuracy: 0.6222 - val_f1_score: 0.6202
- val_loss: 1.3794 - val_precision: 0.7738
Epoch 23/25

655/657 ————— **0s** 24ms/step - accuracy: 0.6283 - f1_score: 0.6253 -
loss: 1.3486 - precision: 0.7709
Epoch 23: val_loss did not improve from 1.35954

657/657 ————— **19s** 29ms/step - accuracy: 0.6283 - f1_score: 0.6253
- loss: 1.3486 - precision: 0.7709 - val_accuracy: 0.6261 - val_f1_score: 0.6253
- val_loss: 1.3767 - val_precision: 0.7731
Epoch 24/25

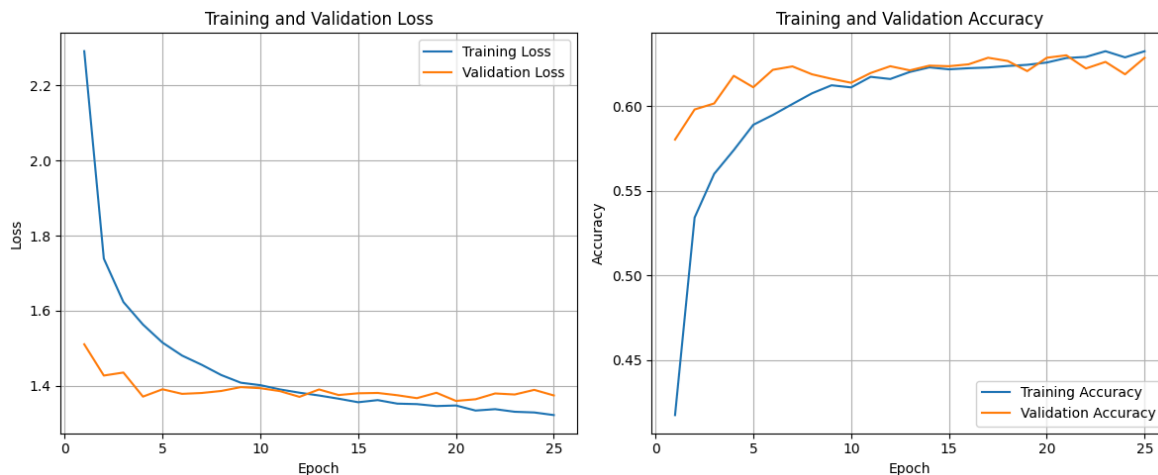
656/657 ————— **0s** 24ms/step - accuracy: 0.6280 - f1_score: 0.6257 -
loss: 1.3353 - precision: 0.7751
Epoch 24: val_loss did not improve from 1.35954

657/657 ————— **19s** 29ms/step - accuracy: 0.6280 - f1_score: 0.6257
- loss: 1.3353 - precision: 0.7751 - val_accuracy: 0.6188 - val_f1_score: 0.6193
- val_loss: 1.3890 - val_precision: 0.7596
Epoch 25/25

656/657 ————— **0s** 24ms/step - accuracy: 0.6267 - f1_score: 0.6242 -
loss: 1.3426 - precision: 0.7729
Epoch 25: val_loss did not improve from 1.35954

657/657 ————— **19s** 29ms/step - accuracy: 0.6267 - f1_score: 0.6242

- loss: 1.3426 - precision: 0.7729 - val_accuracy: 0.6286 - val_f1_score: 0.6242
 - val_loss: 1.3741 - val_precision: 0.7750



141/141 ————— **11s** 38ms/step - accuracy: 0.6201 - f1_score: 0.6135

- loss: 1.3762 - precision: 0.7667

Test Accuracy: 63.24%

Test Loss: 1.3494226932525635

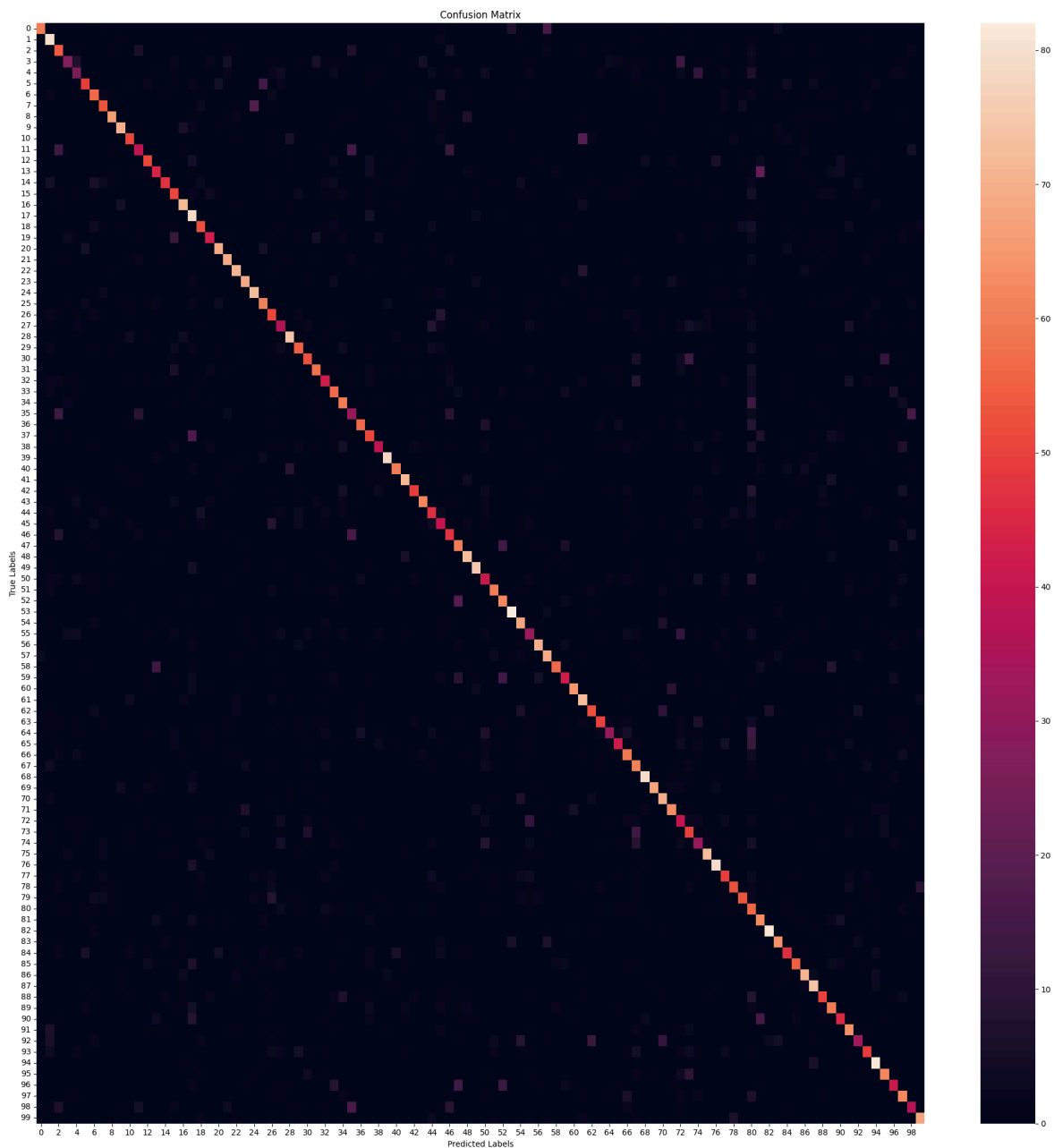
Test Precision: 77.75%

Test F1 Scores (Per Class): [77.41935 69.869 51.923065 39.999996 32.051277 60.49382 61.62162

63.473045	77.64706	79.09604	61.818176	49.056595	63.75	53.65853
62.66666	54.255314	76.59574	60.30534	60.1156	53.749996	77.9661
73.79678	78.02197	76.24309	77.24867	65.94594	49.751236	42.857136
78.30688	61.111103	62.650597	69.047615	48.837204	57.868015	62.827217
34.25414	70.731705	56.35359	52.702694	87.15083	71.856285	78.68852
63.636356	76.073616	49.46236	43.010746	49.735443	58.252422	77.24867
81.08108	44.086014	70.17543	62.311554	86.31578	66.99506	37.209297
77.77777	68.656715	73.07692	51.851852	78.313255	69.194305	63.030296
63.694263	45.58823	55.999992	61.7801	53.744488	85.405396	83.95061
69.30692	72.41379	39.40886	53.475933	36.363632	82.48587	81.86528
55.681812	65.	65.03067	33.33333	57.534237	88.52458	72.31638
57.499992	69.62025	75.53191	77.08333	61.349686	62.176155	56.96202
65.62499	47.482006	57.64705	81.40703	69.66292	49.10179	62.944157
44.31137	77.27273					

Average Test F1 Scores:63.30

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.92	0.67	0.77	90
dolphin	0.58	0.89	0.70	90
otter	0.46	0.60	0.52	90
seal	0.60	0.30	0.40	90
whale	0.38	0.28	0.32	90
aquarium fish	0.68	0.54	0.60	90
flatfish	0.60	0.63	0.62	90
ray	0.69	0.59	0.63	90
shark	0.82	0.73	0.78	90
trout	0.80	0.78	0.79	90
orchids	0.68	0.57	0.62	90
poppies	0.57	0.43	0.49	90
roses	0.73	0.57	0.64	90
sunflowers	0.59	0.49	0.54	90
tulips	0.78	0.52	0.63	90
bottles	0.53	0.57	0.55	90
bowls	0.73	0.80	0.77	90
cans	0.46	0.88	0.60	90
cups	0.63	0.58	0.60	90
plates	0.61	0.48	0.54	90
apples	0.79	0.77	0.78	90
mushrooms	0.71	0.77	0.74	90
oranges	0.77	0.79	0.78	90
pears	0.76	0.77	0.76	90
sweet peppers	0.74	0.81	0.77	90
clock	0.64	0.68	0.66	90
computer keyboard	0.46	0.57	0.50	90
lamp	0.46	0.40	0.43	90
telephone	0.75	0.82	0.78	90
television	0.61	0.61	0.61	90
bed	0.68	0.58	0.63	90
chair	0.74	0.64	0.69	90
couch	0.51	0.47	0.49	90
table	0.53	0.63	0.58	90
wardrobe	0.59	0.67	0.63	90
bee	0.34	0.34	0.34	90
beetle	0.78	0.63	0.70	90
butterfly	0.56	0.57	0.56	90
caterpillar	0.67	0.43	0.53	90
cockroach	0.88	0.87	0.87	90
bear	0.78	0.67	0.72	90
leopard	0.77	0.80	0.79	90
lion	0.77	0.54	0.64	90
tiger	0.85	0.69	0.76	90
wolf	0.48	0.51	0.50	90
bridge	0.42	0.44	0.43	90
castle	0.47	0.52	0.49	90
house	0.52	0.67	0.58	90
road	0.74	0.81	0.77	90
skyscraper	0.79	0.83	0.81	90
cloud	0.43	0.46	0.44	90
forest	0.74	0.67	0.70	90
mountain	0.57	0.69	0.62	90
plain	0.82	0.91	0.86	90
sea	0.60	0.76	0.67	90
camel	0.39	0.36	0.37	90
cattle	0.78	0.78	0.78	90
chimpanzee	0.62	0.77	0.69	90

elephant	0.86	0.63	0.73	90
kangaroo	0.58	0.47	0.52	90
fox	0.86	0.72	0.78	90
porcupine	0.60	0.81	0.69	90
possum	0.69	0.58	0.63	90
raccoon	0.75	0.56	0.64	90
skunk	0.67	0.34	0.46	90
crab	0.69	0.47	0.56	90
lobster	0.58	0.66	0.62	90
snail	0.45	0.68	0.54	90
spider	0.83	0.88	0.85	90
worm	0.94	0.76	0.84	90
baby	0.62	0.78	0.69	90
boy	0.75	0.70	0.72	90
girl	0.35	0.44	0.39	90
man	0.52	0.56	0.53	90
woman	0.37	0.36	0.36	90
crocodile	0.84	0.81	0.82	90
dinosaur	0.77	0.88	0.82	90
lizard	0.57	0.54	0.56	90
snake	0.74	0.58	0.65	90
turtle	0.73	0.59	0.65	90
hamster	0.23	0.62	0.33	90
mouse	0.49	0.70	0.58	90
rabbit	0.87	0.90	0.89	90
shrew	0.74	0.71	0.72	90
squirrel	0.66	0.51	0.57	90
maple	0.81	0.61	0.70	90
oak	0.72	0.79	0.76	90
palm	0.73	0.82	0.77	90
pine	0.68	0.56	0.61	90
willow	0.58	0.67	0.62	90
bicycle	0.66	0.50	0.57	90
bus	0.62	0.71	0.66	90
motorcycle	0.67	0.37	0.47	90
pickup truck	0.61	0.54	0.58	90
train	0.74	0.90	0.81	90
lawn-mower	0.70	0.69	0.70	90
rocket	0.53	0.46	0.49	90
streetcar	0.58	0.69	0.63	90
tank	0.48	0.41	0.44	90
tractor	0.79	0.76	0.77	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential_12"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_12 (GlobalAveragePooling2D)	(None , 2048)
dense_24 (Dense)	(None , 512)
dropout_12 (Dropout)	(None , 512)
dense_25 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 27ms/step - accuracy: 0.6721 - f1_score: 0.6708
- loss: 1.1538 - precision: 0.8139
Epoch 1: val_loss improved from inf to 1.23634, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 57s 36ms/step - accuracy: 0.6721 - f1_score: 0.6708 - loss: 1.1537 - precision: 0.8140 - val_accuracy: 0.6570 - val_f1_score: 0.6553 - val_loss: 1.2363 - val_precision: 0.8047
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7083 - f1_score: 0.7066 - loss: 0.9953 - precision: 0.8447
Epoch 2: val_loss improved from 1.23634 to 1.21582, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7083 - f1_score: 0.7066 - loss: 0.9954 - precision: 0.8447 - val_accuracy: 0.6608 - val_f1_score: 0.6602 - val_loss: 1.2158 - val_precision: 0.8051
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7247 - f1_score: 0.7232 - loss: 0.9283 - precision: 0.8570
Epoch 3: val_loss improved from 1.21582 to 1.20372, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7247 - f1_score: 0.7232 - loss: 0.9283 - precision: 0.8570 - val_accuracy: 0.6640 - val_f1_score: 0.6627 - val_loss: 1.2037 - val_precision: 0.8050
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7452 - f1_score: 0.7437 - loss: 0.8679 - precision: 0.8682
Epoch 4: val_loss improved from 1.20372 to 1.19842, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7452 - f1_score: 0.7437 - loss: 0.8679 - precision: 0.8682 - val_accuracy: 0.6643 - val_f1_score: 0.6629 - val_loss: 1.1984 - val_precision: 0.8040
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7532 - f1_score: 0.7520 - loss: 0.8263 - precision: 0.8712
Epoch 5: val_loss improved from 1.19842 to 1.18828, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7532 - f1_score: 0.7520 - loss: 0.8264 - precision: 0.8712 - val_accuracy: 0.6663 - val_f1_score: 0.6651 - val_loss: 1.1883 - val_precision: 0.8066
Epoch 6/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7649 - f1_score: 0.7634 - loss: 0.7894 - precision: 0.8795
Epoch 6: val_loss improved from 1.18828 to 1.18361, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7649 - f1_score: 0.7633 - loss: 0.7895 - precision: 0.8795 - val_accuracy: 0.6679 - val_f1_score: 0.6669 - val_loss: 1.1836 - val_precision: 0.8080
Epoch 7/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7721 - f1_score: 0.7709 - loss: 0.7580 - precision: 0.8840
Epoch 7: val_loss improved from 1.18361 to 1.17851, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7721 - f1_score: 0.7709 - loss: 0.7580 - precision: 0.8840 - val_accuracy: 0.6694 - val_f1_score: 0.6692 - val_loss: 1.1785 - val_precision: 0.8059
Epoch 8/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7815 - f1_score: 0.7803 - loss: 0.7272 - precision: 0.8903
Epoch 8: val_loss improved from 1.17851 to 1.17797, saving model to best_model_fine_tune_v8.keras

ne_tune_v8.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7815 - f1_score: 0.7802 - loss: 0.7273 - precision: 0.8903 - val_accuracy: 0.6676 - val_f1_score: 0.6661 - val_loss: 1.1780 - val_precision: 0.8046

Epoch 9/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7890 - f1_score: 0.7878 - loss: 0.6977 - precision: 0.8925

Epoch 9: val_loss improved from 1.17797 to 1.17433, saving model to best_model_fine_tune_v8.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7889 - f1_score: 0.7877 - loss: 0.6978 - precision: 0.8925 - val_accuracy: 0.6701 - val_f1_score: 0.6687 - val_loss: 1.1743 - val_precision: 0.8057

Epoch 10/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7952 - f1_score: 0.7942 - loss: 0.6765 - precision: 0.8982

Epoch 10: val_loss improved from 1.17433 to 1.17425, saving model to best_model_fine_tune_v8.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7952 - f1_score: 0.7941 - loss: 0.6766 - precision: 0.8982 - val_accuracy: 0.6700 - val_f1_score: 0.6689 - val_loss: 1.1743 - val_precision: 0.8063

Epoch 11/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.8006 - f1_score: 0.7994 - loss: 0.6587 - precision: 0.8999

Epoch 11: val_loss did not improve from 1.17425

1314/1314 ————— 35s 27ms/step - accuracy: 0.8006 - f1_score: 0.7994 - loss: 0.6587 - precision: 0.8999 - val_accuracy: 0.6720 - val_f1_score: 0.6709 - val_loss: 1.1764 - val_precision: 0.8069

Epoch 12/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.8080 - f1_score: 0.8070 - loss: 0.6340 - precision: 0.9017

Epoch 12: val_loss did not improve from 1.17425

1314/1314 ————— 35s 27ms/step - accuracy: 0.8080 - f1_score: 0.8070 - loss: 0.6340 - precision: 0.9016 - val_accuracy: 0.6723 - val_f1_score: 0.6712 - val_loss: 1.1753 - val_precision: 0.8021

Epoch 13/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.8137 - f1_score: 0.8127 - loss: 0.6147 - precision: 0.9057

Epoch 13: val_loss improved from 1.17425 to 1.16505, saving model to best_model_fine_tune_v8.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.8136 - f1_score: 0.8127 - loss: 0.6148 - precision: 0.9057 - val_accuracy: 0.6724 - val_f1_score: 0.6712 - val_loss: 1.1651 - val_precision: 0.8067

Epoch 14/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.8217 - f1_score: 0.8208 - loss: 0.5950 - precision: 0.9086

Epoch 14: val_loss did not improve from 1.16505

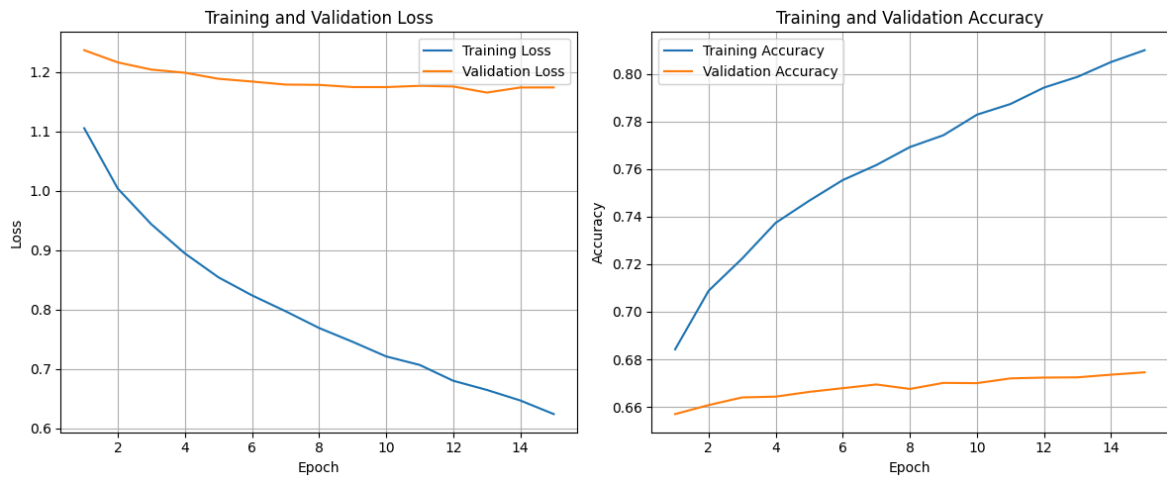
1314/1314 ————— 35s 27ms/step - accuracy: 0.8217 - f1_score: 0.8208 - loss: 0.5950 - precision: 0.9086 - val_accuracy: 0.6736 - val_f1_score: 0.6726 - val_loss: 1.1736 - val_precision: 0.7995

Epoch 15/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.8241 - f1_score: 0.8228 - loss: 0.5758 - precision: 0.9109

Epoch 15: val_loss did not improve from 1.16505

1314/1314 ————— 35s 27ms/step - accuracy: 0.8240 - f1_score: 0.8228 - loss: 0.5759 - precision: 0.9109 - val_accuracy: 0.6746 - val_f1_score: 0.6734 - val_loss: 1.1737 - val_precision: 0.8001



141/141 ————— **11s** 38ms/step - accuracy: 0.6687 - f1_score: 0.6603

- loss: 1.1836 - precision: 0.7927

Test Accuracy: 67.84%

Test Loss: 1.164182186126709

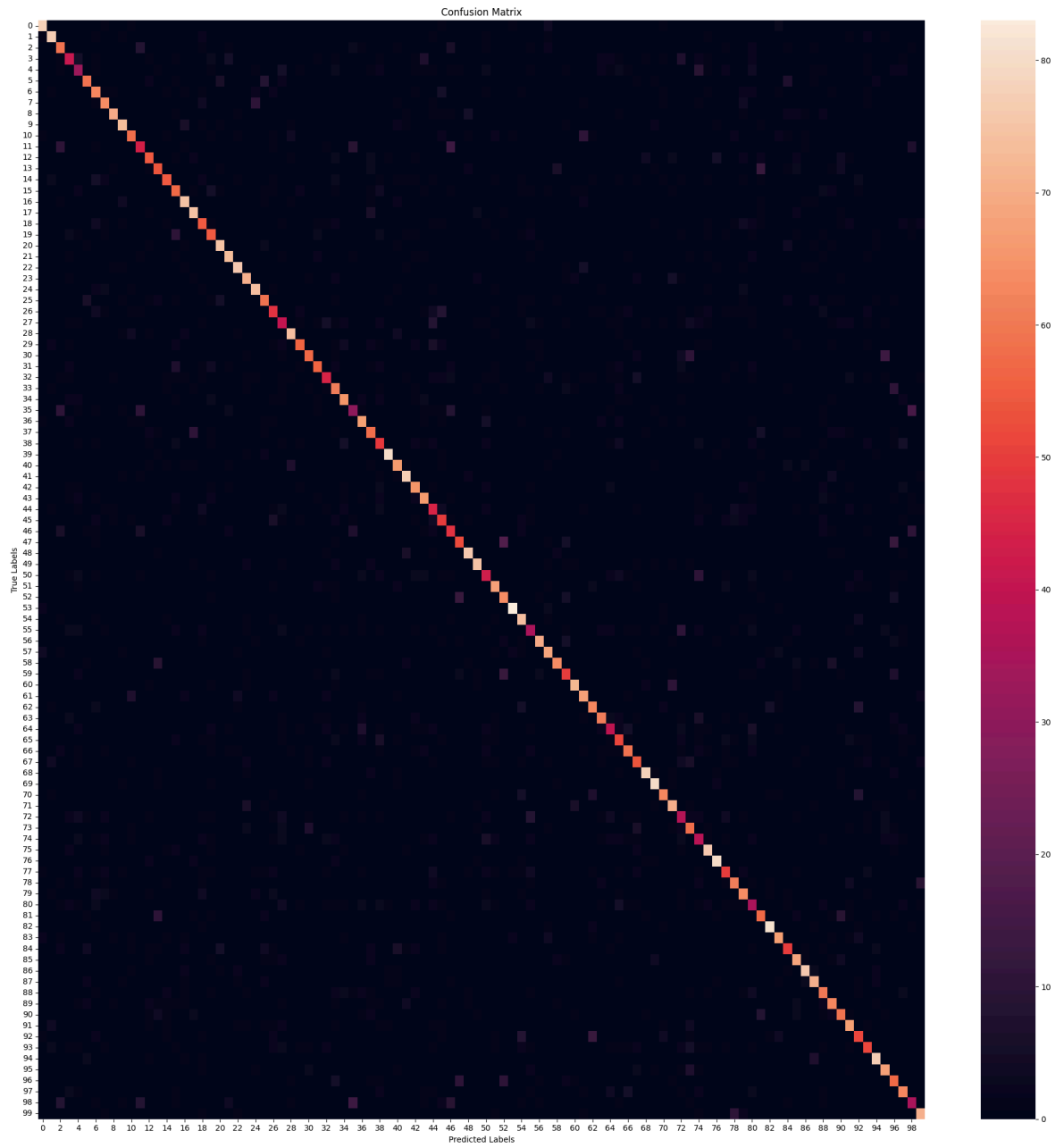
Test Precision: 80.37%

Test F1 Scores (Per Class): [86.18784 81.481476 59.89847 49.999992 39.52095 69.005844 63.316578

71.1111	82.84023	81.56424	68.23529	51.16278	65.088745	61.01694
67.48465	59.999992	78.947365	79.787224	60.109283	59.7826	81.08108
79.78141	83.51647	76.756744	80.43478	66.29213	55.172413	44.692734
81.31868	69.565216	69.090904	69.565216	55.555553	60.999996	72.625694
38.709675	71.276596	64.088394	52.127655	90.90909	71.35134	81.91489
70.27026	72.13114	47.368412	54.945045	50.526314	61.53846	83.9779
84.26965	49.142857	75.55555	64.646454	92.22221	73.26733	42.16867
81.87134	73.79678	76.190475	58.823524	82.285706	74.31693	68.852455
72.51461	48.447197	55.13513	68.208084	60.33519	86.18784	85.56149
74.117645	75.93583	40.437157	60.10362	40.86021	80.85106	86.95651
59.880234	68.53932	67.36841	38.043476	61.956512	89.502754	76.13636
58.823524	72.251305	84.44444	77.173904	65.57377	69.23077	64.13043
71.65774	56.35359	60.355026	84.153	72.34043	56.43564	68.852455
42.16867	78.88888					

Average Test F1 Scores:67.71

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.86	0.87	0.86	90
dolphin	0.78	0.86	0.81	90
otter	0.55	0.66	0.60	90
seal	0.55	0.46	0.50	90
whale	0.43	0.37	0.40	90
aquarium fish	0.73	0.66	0.69	90
flatfish	0.58	0.70	0.63	90
ray	0.71	0.71	0.71	90
shark	0.89	0.78	0.83	90
trout	0.82	0.81	0.82	90
orchids	0.72	0.64	0.68	90
poppies	0.54	0.49	0.51	90
roses	0.70	0.61	0.65	90
sunflowers	0.62	0.60	0.61	90
tulips	0.75	0.61	0.67	90
bottles	0.57	0.63	0.60	90
bowls	0.75	0.83	0.79	90
cans	0.77	0.83	0.80	90
cups	0.59	0.61	0.60	90
plates	0.59	0.61	0.60	90
apples	0.79	0.83	0.81	90
mushrooms	0.78	0.81	0.80	90
oranges	0.83	0.84	0.84	90
pears	0.75	0.79	0.77	90
sweet peppers	0.79	0.82	0.80	90
clock	0.67	0.66	0.66	90
computer keyboard	0.57	0.53	0.55	90
lamp	0.45	0.44	0.45	90
telephone	0.80	0.82	0.81	90
television	0.79	0.62	0.70	90
bed	0.76	0.63	0.69	90
chair	0.79	0.62	0.70	90
couch	0.62	0.50	0.56	90
table	0.55	0.68	0.61	90
wardrobe	0.73	0.72	0.73	90
bee	0.46	0.33	0.39	90
beetle	0.68	0.74	0.71	90
butterfly	0.64	0.64	0.64	90
caterpillar	0.50	0.54	0.52	90
cockroach	0.93	0.89	0.91	90
bear	0.69	0.73	0.71	90
leopard	0.79	0.86	0.82	90
lion	0.68	0.72	0.70	90
tiger	0.71	0.73	0.72	90
wolf	0.45	0.50	0.47	90
bridge	0.54	0.56	0.55	90
castle	0.48	0.53	0.51	90
house	0.66	0.58	0.62	90
road	0.84	0.84	0.84	90
skyscraper	0.85	0.83	0.84	90
cloud	0.51	0.48	0.49	90
forest	0.75	0.76	0.75	90
mountain	0.59	0.71	0.65	90
plain	0.92	0.92	0.92	90
sea	0.66	0.82	0.73	90
camel	0.46	0.39	0.42	90
cattle	0.86	0.78	0.82	90
chimpanzee	0.71	0.77	0.74	90

elephant	0.82	0.71	0.76	90
kangaroo	0.62	0.56	0.59	90
fox	0.85	0.80	0.82	90
porcupine	0.73	0.76	0.74	90
possum	0.68	0.70	0.69	90
raccoon	0.76	0.68	0.72	90
skunk	0.55	0.43	0.48	90
crab	0.54	0.57	0.55	90
lobster	0.71	0.66	0.68	90
snail	0.61	0.60	0.60	90
spider	0.86	0.87	0.86	90
worm	0.82	0.89	0.86	90
baby	0.79	0.70	0.74	90
boy	0.73	0.79	0.76	90
girl	0.40	0.41	0.40	90
man	0.56	0.64	0.60	90
woman	0.40	0.42	0.41	90
crocodile	0.78	0.84	0.81	90
dinosaur	0.85	0.89	0.87	90
lizard	0.65	0.56	0.60	90
snake	0.69	0.68	0.69	90
turtle	0.64	0.71	0.67	90
hamster	0.37	0.39	0.38	90
mouse	0.61	0.63	0.62	90
rabbit	0.89	0.90	0.90	90
shrew	0.78	0.74	0.76	90
squirrel	0.62	0.56	0.59	90
maple	0.68	0.77	0.72	90
oak	0.84	0.84	0.84	90
palm	0.76	0.79	0.77	90
pine	0.65	0.67	0.66	90
willow	0.68	0.70	0.69	90
bicycle	0.63	0.66	0.64	90
bus	0.69	0.74	0.72	90
motorcycle	0.56	0.57	0.56	90
pickup truck	0.65	0.57	0.60	90
train	0.83	0.86	0.84	90
lawn-mower	0.69	0.76	0.72	90
rocket	0.51	0.63	0.56	90
streetcar	0.68	0.70	0.69	90
tank	0.46	0.39	0.42	90
tractor	0.79	0.79	0.79	90
accuracy			0.68	9000
macro avg	0.68	0.68	0.68	9000
weighted avg	0.68	0.68	0.68	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

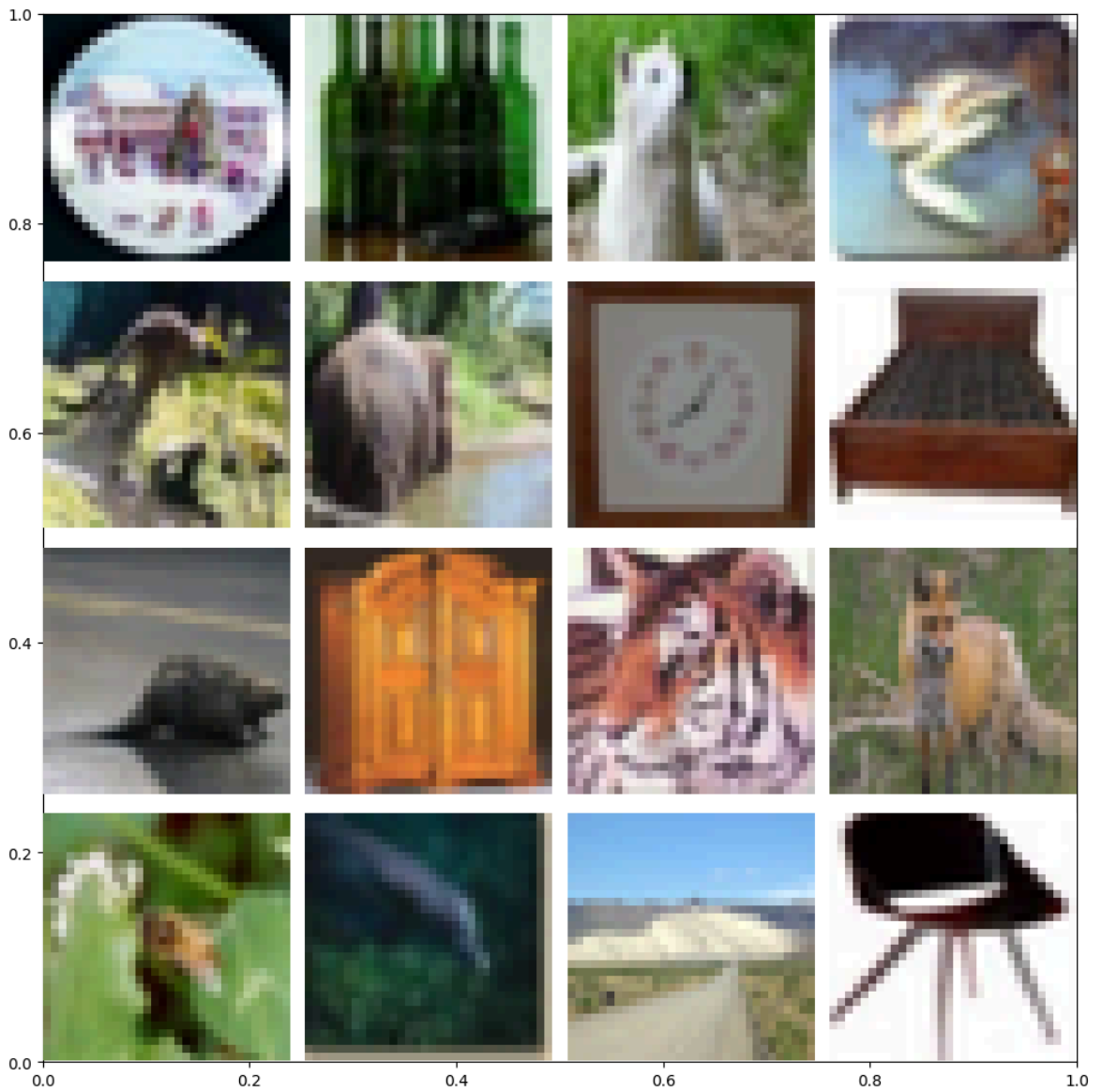
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

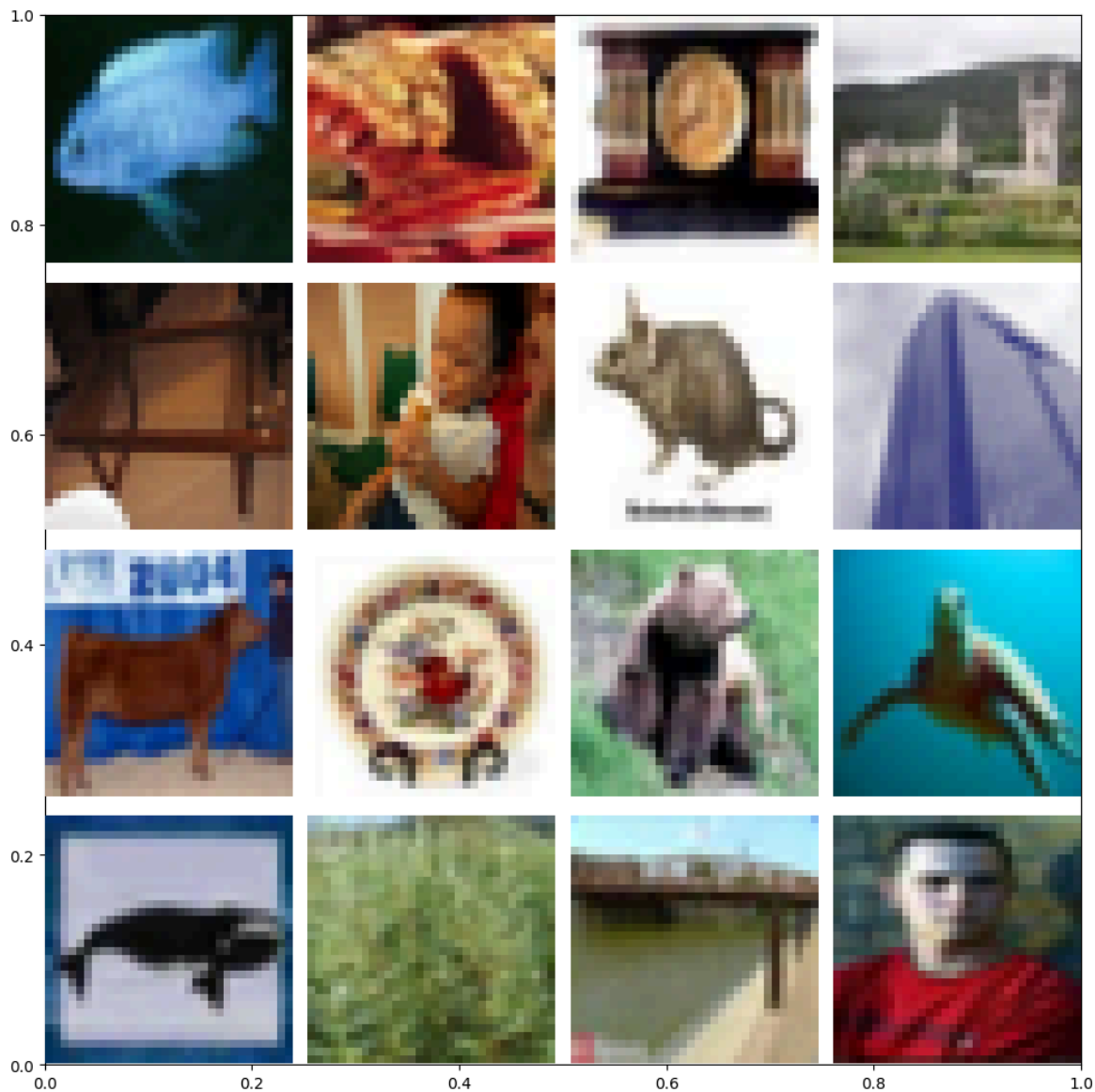
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[3124].shape: (1,)
imgs[3124].shape: (32, 32, 3)
```

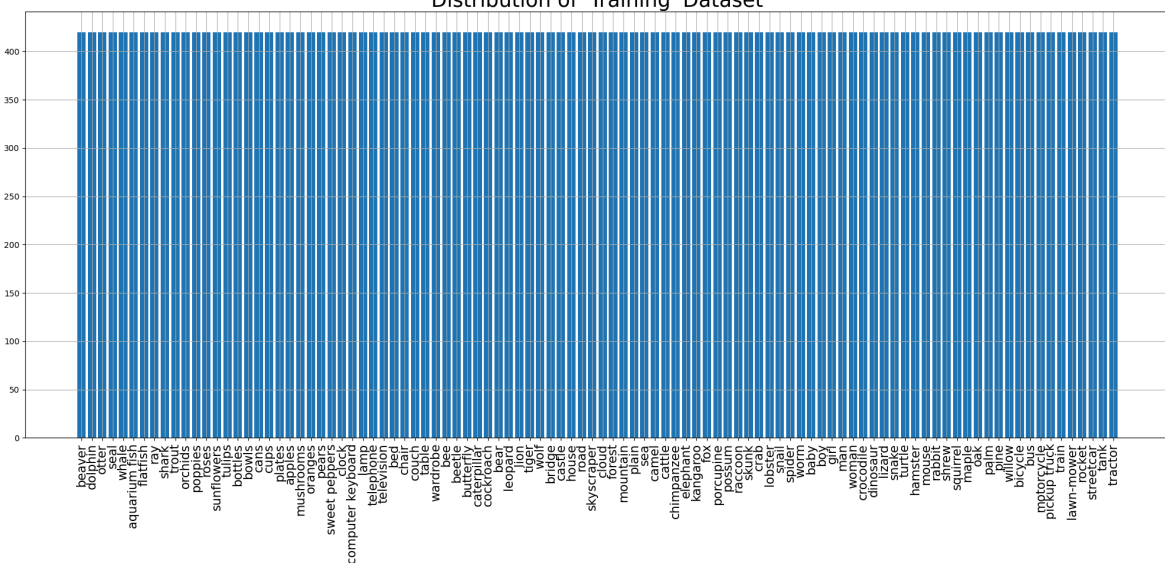


```
labels[36678].shape: (1,)
imgs[36678].shape: (32, 32, 3)
```

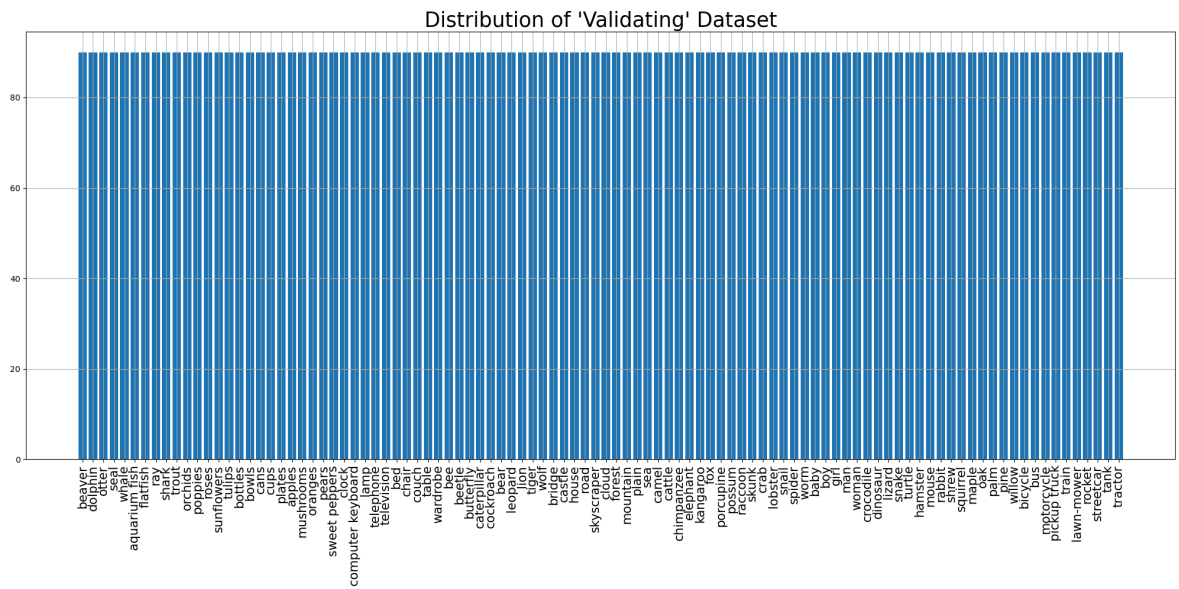



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

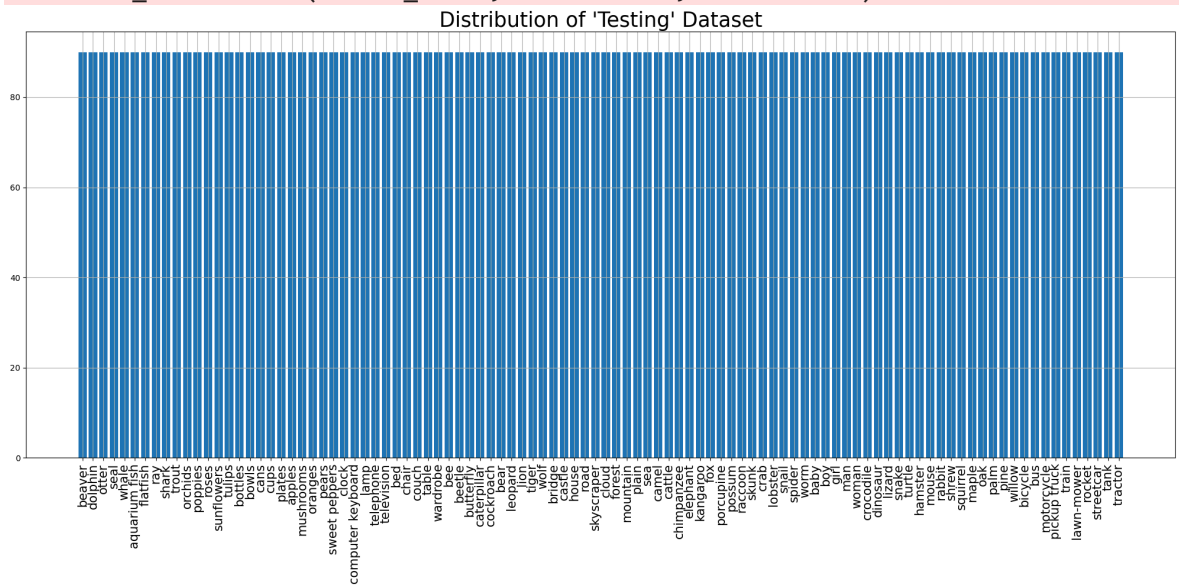
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_13"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_13 (GlobalAveragePooling2D)	(None , 2048)
dense_26 (Dense)	(None , 512)
dropout_13 (Dropout)	(None , 512)
dense_27 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

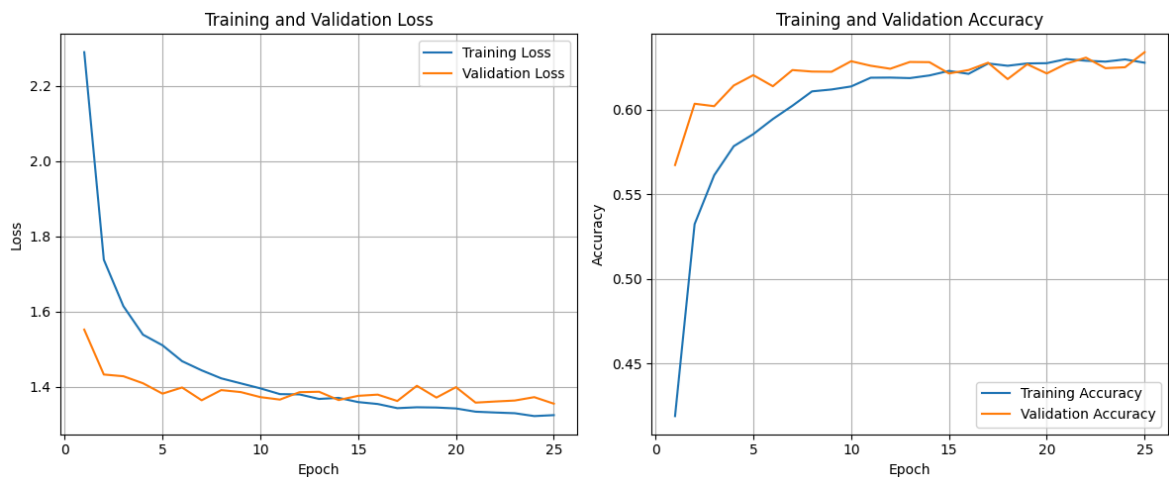
Epoch 1/25
657/657 ————— **0s** 29ms/step - accuracy: 0.3255 - f1_score: 0.3187 - loss: 2.8477 - precision: 0.6472
Epoch 1: val_loss improved from inf to 1.55163, saving model to best_model_v8.keras
657/657 ————— **39s** 44ms/step - accuracy: 0.3256 - f1_score: 0.3189 - loss: 2.8468 - precision: 0.6473 - val_accuracy: 0.5673 - val_f1_score: 0.5607 - val_loss: 1.5516 - val_precision: 0.7709
Epoch 2/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5203 - f1_score: 0.5155 - loss: 1.7829 - precision: 0.7196
Epoch 2: val_loss improved from 1.55163 to 1.43222, saving model to best_model_v8.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5203 - f1_score: 0.5155 - loss: 1.7828 - precision: 0.7196 - val_accuracy: 0.6036 - val_f1_score: 0.6026 - val_loss: 1.4322 - val_precision: 0.7823
Epoch 3/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5517 - f1_score: 0.5478 - loss: 1.6437 - precision: 0.7351
Epoch 3: val_loss improved from 1.43222 to 1.42763, saving model to best_model_v8.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5518 - f1_score: 0.5478 - loss: 1.6437 - precision: 0.7352 - val_accuracy: 0.6021 - val_f1_score: 0.5968 - val_loss: 1.4276 - val_precision: 0.7819
Epoch 4/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5749 - f1_score: 0.5707 - loss: 1.5674 - precision: 0.7514
Epoch 4: val_loss improved from 1.42763 to 1.40899, saving model to best_model_v8.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5749 - f1_score: 0.5707 - loss: 1.5673 - precision: 0.7514 - val_accuracy: 0.6143 - val_f1_score: 0.6116 - val_loss: 1.4090 - val_precision: 0.7704
Epoch 5/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5788 - f1_score: 0.5752 - loss: 1.5423 - precision: 0.7457
Epoch 5: val_loss improved from 1.40899 to 1.38128, saving model to best_model_v8.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5788 - f1_score: 0.5753 - loss: 1.5421 - precision: 0.7457 - val_accuracy: 0.6204 - val_f1_score: 0.6176 - val_loss: 1.3813 - val_precision: 0.7752
Epoch 6/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5894 - f1_score: 0.5857 - loss: 1.4909 - precision: 0.7494
Epoch 6: val_loss did not improve from 1.38128
657/657 ————— **19s** 28ms/step - accuracy: 0.5894 - f1_score: 0.5857 - loss: 1.4908 - precision: 0.7494 - val_accuracy: 0.6139 - val_f1_score: 0.6137 - val_loss: 1.3977 - val_precision: 0.7691
Epoch 7/25
655/657 ————— **0s** 23ms/step - accuracy: 0.5949 - f1_score: 0.5917 - loss: 1.4642 - precision: 0.7524
Epoch 7: val_loss improved from 1.38128 to 1.36363, saving model to best_model_v8.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.5949 - f1_score: 0.5917 - loss: 1.4642 - precision: 0.7525 - val_accuracy: 0.6234 - val_f1_score: 0.6221 - val_loss: 1.3636 - val_precision: 0.7784
Epoch 8/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6067 - f1_score: 0.6036 - loss: 1.4327 - precision: 0.7651
Epoch 8: val_loss did not improve from 1.36363
657/657 ————— **19s** 28ms/step - accuracy: 0.6067 - f1_score: 0.6036

- loss: 1.4326 - precision: 0.7651 - val_accuracy: 0.6226 - val_f1_score: 0.6184
- val_loss: 1.3907 - val_precision: 0.7680
Epoch 9/25
656/657 ————— 0s 23ms/step - accuracy: 0.6036 - f1_score: 0.6003 -
loss: 1.4330 - precision: 0.7561
Epoch 9: val_loss did not improve from 1.36363
657/657 ————— 19s 28ms/step - accuracy: 0.6036 - f1_score: 0.6003
- loss: 1.4329 - precision: 0.7561 - val_accuracy: 0.6224 - val_f1_score: 0.6206
- val_loss: 1.3854 - val_precision: 0.7733
Epoch 10/25
657/657 ————— 0s 23ms/step - accuracy: 0.6105 - f1_score: 0.6076 -
loss: 1.4161 - precision: 0.7634
Epoch 10: val_loss did not improve from 1.36363
657/657 ————— 19s 28ms/step - accuracy: 0.6105 - f1_score: 0.6076
- loss: 1.4161 - precision: 0.7634 - val_accuracy: 0.6287 - val_f1_score: 0.6257
- val_loss: 1.3721 - val_precision: 0.7725
Epoch 11/25
657/657 ————— 0s 23ms/step - accuracy: 0.6114 - f1_score: 0.6088 -
loss: 1.3990 - precision: 0.7620
Epoch 11: val_loss did not improve from 1.36363
657/657 ————— 19s 28ms/step - accuracy: 0.6114 - f1_score: 0.6088
- loss: 1.3990 - precision: 0.7620 - val_accuracy: 0.6260 - val_f1_score: 0.6230
- val_loss: 1.3654 - val_precision: 0.7756
Epoch 12/25
656/657 ————— 0s 24ms/step - accuracy: 0.6161 - f1_score: 0.6128 -
loss: 1.3831 - precision: 0.7655
Epoch 12: val_loss did not improve from 1.36363
657/657 ————— 19s 28ms/step - accuracy: 0.6161 - f1_score: 0.6128
- loss: 1.3830 - precision: 0.7655 - val_accuracy: 0.6242 - val_f1_score: 0.6241
- val_loss: 1.3852 - val_precision: 0.7646
Epoch 13/25
656/657 ————— 0s 24ms/step - accuracy: 0.6135 - f1_score: 0.6105 -
loss: 1.3888 - precision: 0.7656
Epoch 13: val_loss did not improve from 1.36363
657/657 ————— 19s 28ms/step - accuracy: 0.6136 - f1_score: 0.6105
- loss: 1.3887 - precision: 0.7656 - val_accuracy: 0.6282 - val_f1_score: 0.6243
- val_loss: 1.3863 - val_precision: 0.7645
Epoch 14/25
656/657 ————— 0s 23ms/step - accuracy: 0.6172 - f1_score: 0.6141 -
loss: 1.3872 - precision: 0.7663
Epoch 14: val_loss did not improve from 1.36363
657/657 ————— 19s 28ms/step - accuracy: 0.6172 - f1_score: 0.6141
- loss: 1.3871 - precision: 0.7663 - val_accuracy: 0.6281 - val_f1_score: 0.6264
- val_loss: 1.3642 - val_precision: 0.7826
Epoch 15/25
655/657 ————— 0s 23ms/step - accuracy: 0.6158 - f1_score: 0.6125 -
loss: 1.3781 - precision: 0.7643
Epoch 15: val_loss did not improve from 1.36363
657/657 ————— 19s 28ms/step - accuracy: 0.6158 - f1_score: 0.6125
- loss: 1.3780 - precision: 0.7643 - val_accuracy: 0.6214 - val_f1_score: 0.6193
- val_loss: 1.3755 - val_precision: 0.7664
Epoch 16/25
657/657 ————— 0s 24ms/step - accuracy: 0.6191 - f1_score: 0.6164 -
loss: 1.3615 - precision: 0.7673
Epoch 16: val_loss did not improve from 1.36363
657/657 ————— 19s 28ms/step - accuracy: 0.6191 - f1_score: 0.6164
- loss: 1.3615 - precision: 0.7673 - val_accuracy: 0.6234 - val_f1_score: 0.6212
- val_loss: 1.3787 - val_precision: 0.7742
Epoch 17/25
657/657 ————— 0s 23ms/step - accuracy: 0.6208 - f1_score: 0.6178 -

loss: 1.3658 - precision: 0.7667
Epoch 17: val_loss improved from 1.36363 to 1.36175, saving model to best_model_v8.keras
657/657  19s 29ms/step - accuracy: 0.6208 - f1_score: 0.6178
- loss: 1.3657 - precision: 0.7667 - val_accuracy: 0.6278 - val_f1_score: 0.6256
- val_loss: 1.3618 - val_precision: 0.7718
Epoch 18/25
656/657  0s 23ms/step - accuracy: 0.6231 - f1_score: 0.6200 -
loss: 1.3552 - precision: 0.7682
Epoch 18: val_loss did not improve from 1.36175
657/657  19s 28ms/step - accuracy: 0.6231 - f1_score: 0.6200
- loss: 1.3552 - precision: 0.7682 - val_accuracy: 0.6181 - val_f1_score: 0.6155
- val_loss: 1.4022 - val_precision: 0.7588
Epoch 19/25
656/657  0s 24ms/step - accuracy: 0.6236 - f1_score: 0.6205 -
loss: 1.3579 - precision: 0.7687
Epoch 19: val_loss did not improve from 1.36175
657/657  19s 28ms/step - accuracy: 0.6236 - f1_score: 0.6205
- loss: 1.3579 - precision: 0.7687 - val_accuracy: 0.6269 - val_f1_score: 0.6258
- val_loss: 1.3708 - val_precision: 0.7771
Epoch 20/25
655/657  0s 24ms/step - accuracy: 0.6246 - f1_score: 0.6222 -
loss: 1.3484 - precision: 0.7697
Epoch 20: val_loss did not improve from 1.36175
657/657  19s 28ms/step - accuracy: 0.6247 - f1_score: 0.6222
- loss: 1.3484 - precision: 0.7697 - val_accuracy: 0.6214 - val_f1_score: 0.6191
- val_loss: 1.3985 - val_precision: 0.7683
Epoch 21/25
657/657  0s 24ms/step - accuracy: 0.6303 - f1_score: 0.6275 -
loss: 1.3412 - precision: 0.7756
Epoch 21: val_loss improved from 1.36175 to 1.35739, saving model to best_model_v8.keras
657/657  19s 30ms/step - accuracy: 0.6303 - f1_score: 0.6275
- loss: 1.3412 - precision: 0.7756 - val_accuracy: 0.6272 - val_f1_score: 0.6276
- val_loss: 1.3574 - val_precision: 0.7732
Epoch 22/25
657/657  0s 24ms/step - accuracy: 0.6256 - f1_score: 0.6230 -
loss: 1.3460 - precision: 0.7689
Epoch 22: val_loss did not improve from 1.35739
657/657  19s 28ms/step - accuracy: 0.6256 - f1_score: 0.6230
- loss: 1.3460 - precision: 0.7689 - val_accuracy: 0.6308 - val_f1_score: 0.6275
- val_loss: 1.3604 - val_precision: 0.7817
Epoch 23/25
656/657  0s 23ms/step - accuracy: 0.6250 - f1_score: 0.6218 -
loss: 1.3426 - precision: 0.7723
Epoch 23: val_loss did not improve from 1.35739
657/657  19s 28ms/step - accuracy: 0.6250 - f1_score: 0.6218
- loss: 1.3426 - precision: 0.7723 - val_accuracy: 0.6246 - val_f1_score: 0.6232
- val_loss: 1.3629 - val_precision: 0.7773
Epoch 24/25
657/657  0s 24ms/step - accuracy: 0.6284 - f1_score: 0.6252 -
loss: 1.3328 - precision: 0.7698
Epoch 24: val_loss did not improve from 1.35739
657/657  19s 28ms/step - accuracy: 0.6284 - f1_score: 0.6253
- loss: 1.3327 - precision: 0.7698 - val_accuracy: 0.6251 - val_f1_score: 0.6238
- val_loss: 1.3719 - val_precision: 0.7675
Epoch 25/25
655/657  0s 24ms/step - accuracy: 0.6218 - f1_score: 0.6192 -
loss: 1.3451 - precision: 0.7642
Epoch 25: val_loss improved from 1.35739 to 1.35461, saving model to best_model_v

8.keras

657/657 ————— **19s** 30ms/step - accuracy: 0.6218 - f1_score: 0.6192
 - loss: 1.3450 - precision: 0.7642 - val_accuracy: 0.6340 - val_f1_score: 0.6302
 - val_loss: 1.3546 - val_precision: 0.7804



141/141 ————— **13s** 38ms/step - accuracy: 0.6249 - f1_score: 0.6154
 - loss: 1.3851 - precision: 0.7648

Test Accuracy: 63.44%

Test Loss: 1.3604594469070435

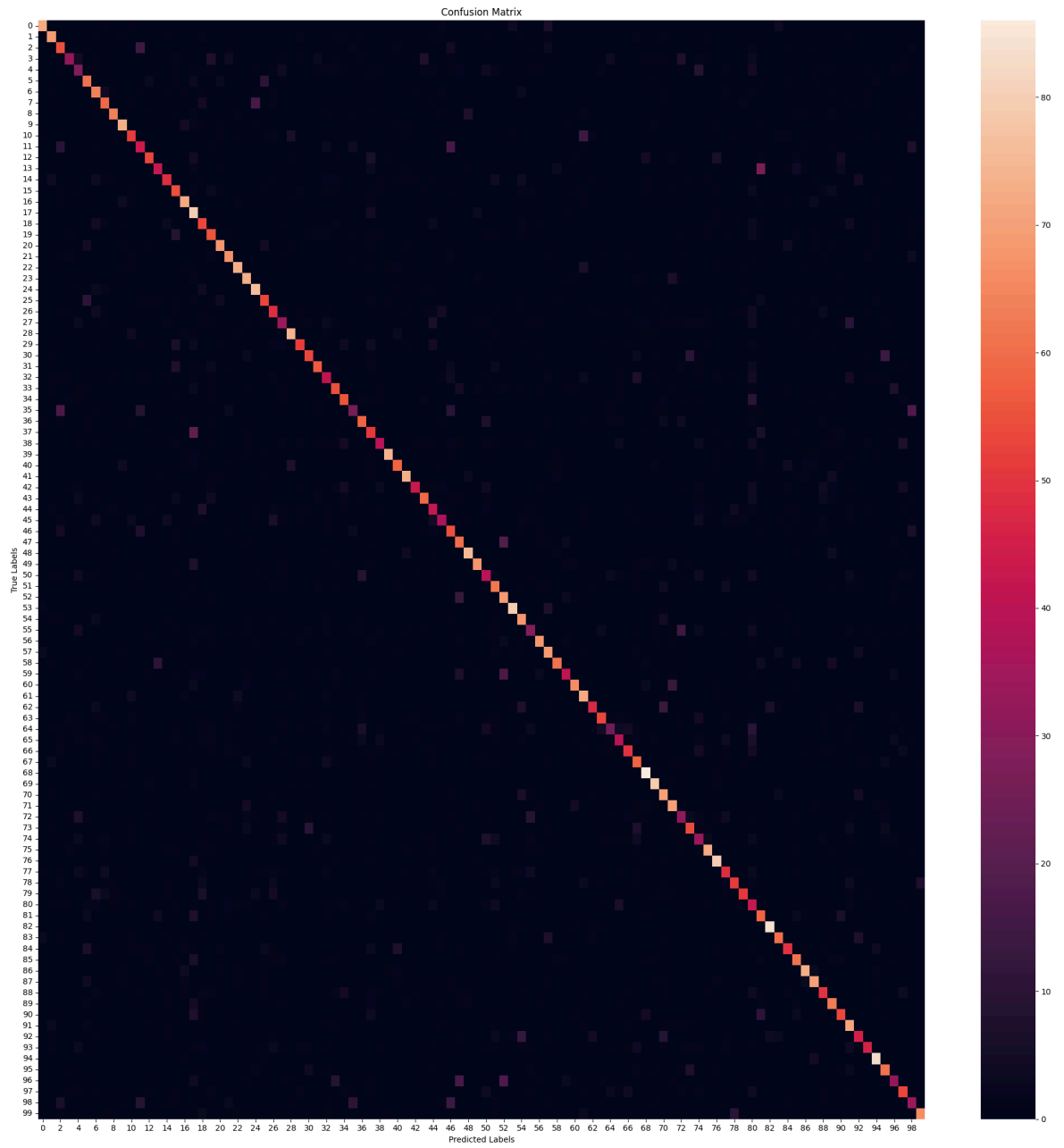
Test Precision: 77.45%

Test F1 Scores (Per Class): [83.236984 74.59459 53.65853 45.714283 32.94117 6
 2.244892 60.377346

66.666664	77.57575	80.66298	60.818714	48.88888	65.03067	55.26315
60.49382	56.701027	76.59574	55.36332	51.70731	57.435894	77.71428
73.224045	80.21977	76.041664	76.76767	60.571426	53.551907	40.476185
79.99999	64.197525	63.09523	69.090904	45.054943	60.109283	63.636356
36.619713	64.44444	49.999992	51.315792	87.57396	67.05202	81.11111
60.56338	69.005844	47.191006	46.753242	51.64319	58.252422	82.41757
79.99999	43.093918	62.626255	63.302742	86.33879	63.25581	37.908493
77.27273	70.05075	74.84662	54.666668	76.74419	70.243904	62.251644
65.43209	37.037033	50.632904	61.72839	59.18367	83.09178	81.44328
65.11627	71.428566	38.095238	62.7907	37.77777	80.89887	81.63265
56.64739	63.803684	67.54967	33.070858	53.703697	90.81081	71.00591
61.349686	66.304344	81.11111	75.268814	58.479523	65.968575	62.352936
65.437775	53.571423	61.33333	85.1282	67.40331	43.243237	60.919537
39.999996	78.57143					

Average Test F1 Scores:63.26

282/282 ————— **12s** 29ms/step



	precision	recall	f1-score	support
beaver	0.87	0.80	0.83	90
dolphin	0.73	0.77	0.75	90
otter	0.48	0.61	0.54	90
seal	0.64	0.36	0.46	90
whale	0.35	0.31	0.33	90
aquarium fish	0.58	0.68	0.62	90
flatfish	0.52	0.71	0.60	90
ray	0.68	0.66	0.67	90
shark	0.85	0.71	0.78	90
trout	0.80	0.81	0.81	90
orchids	0.64	0.58	0.61	90
poppies	0.49	0.49	0.49	90
roses	0.73	0.59	0.65	90
sunflowers	0.68	0.47	0.55	90
tulips	0.68	0.54	0.60	90
bottles	0.53	0.61	0.57	90
bowls	0.73	0.80	0.77	90
cans	0.40	0.89	0.56	90
cups	0.46	0.59	0.52	90
plates	0.53	0.62	0.57	90
apples	0.80	0.76	0.78	90
mushrooms	0.72	0.74	0.73	90
oranges	0.79	0.81	0.80	90
pears	0.72	0.81	0.76	90
sweet peppers	0.70	0.84	0.77	90
clock	0.62	0.59	0.61	90
computer keyboard	0.53	0.54	0.54	90
lamp	0.44	0.38	0.40	90
telephone	0.77	0.82	0.80	90
television	0.72	0.58	0.64	90
bed	0.68	0.59	0.63	90
chair	0.76	0.63	0.69	90
couch	0.45	0.46	0.45	90
table	0.59	0.61	0.60	90
wardrobe	0.65	0.62	0.64	90
bee	0.50	0.29	0.37	90
beetle	0.64	0.64	0.64	90
butterfly	0.46	0.57	0.51	90
caterpillar	0.63	0.43	0.51	90
cockroach	0.94	0.82	0.88	90
bear	0.71	0.64	0.67	90
leopard	0.81	0.81	0.81	90
lion	0.83	0.48	0.61	90
tiger	0.73	0.66	0.69	90
wolf	0.48	0.47	0.47	90
bridge	0.56	0.40	0.47	90
castle	0.45	0.61	0.52	90
house	0.52	0.67	0.58	90
road	0.82	0.83	0.82	90
skyscraper	0.85	0.76	0.80	90
cloud	0.43	0.43	0.43	90
forest	0.57	0.69	0.63	90
mountain	0.54	0.77	0.63	90
plain	0.85	0.88	0.86	90
sea	0.54	0.76	0.63	90
camel	0.46	0.32	0.38	90
cattle	0.79	0.76	0.77	90
chimpanzee	0.64	0.77	0.70	90

elephant	0.84	0.68	0.75	90
kangaroo	0.68	0.46	0.55	90
fox	0.80	0.73	0.77	90
porcupine	0.63	0.80	0.70	90
possum	0.77	0.52	0.62	90
raccoon	0.74	0.59	0.65	90
skunk	0.56	0.28	0.37	90
crab	0.59	0.44	0.51	90
lobster	0.69	0.56	0.62	90
snail	0.55	0.64	0.59	90
spider	0.74	0.96	0.83	90
worm	0.76	0.88	0.81	90
baby	0.56	0.78	0.65	90
boy	0.66	0.78	0.71	90
girl	0.41	0.36	0.38	90
man	0.66	0.60	0.63	90
woman	0.38	0.38	0.38	90
crocodile	0.82	0.80	0.81	90
dinosaur	0.75	0.89	0.82	90
lizard	0.59	0.54	0.57	90
snake	0.71	0.58	0.64	90
turtle	0.84	0.57	0.68	90
hamster	0.26	0.47	0.33	90
mouse	0.46	0.64	0.54	90
rabbit	0.88	0.93	0.91	90
shrew	0.76	0.67	0.71	90
squirrel	0.68	0.56	0.61	90
maple	0.65	0.68	0.66	90
oak	0.81	0.81	0.81	90
palm	0.73	0.78	0.75	90
pine	0.62	0.56	0.58	90
willow	0.62	0.70	0.66	90
bicycle	0.66	0.59	0.62	90
bus	0.56	0.79	0.65	90
motorcycle	0.58	0.50	0.54	90
pickup truck	0.77	0.51	0.61	90
train	0.79	0.92	0.85	90
lawn-mower	0.67	0.68	0.67	90
rocket	0.55	0.36	0.43	90
streetcar	0.63	0.59	0.61	90
tank	0.42	0.38	0.40	90
tractor	0.85	0.73	0.79	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential_13"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_13 (GlobalAveragePooling2D)	(None , 2048)
dense_26 (Dense)	(None , 512)
dropout_13 (Dropout)	(None , 512)
dense_27 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 27ms/step - accuracy: 0.6768 - f1_score: 0.6752
- loss: 1.1360 - precision: 0.8171
Epoch 1: val_loss improved from inf to 1.22872, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.6768 - f1_score: 0.6752 - loss: 1.1359 - precision: 0.8172 - val_accuracy: 0.6566 - val_f1_score: 0.6555 - val_loss: 1.2287 - val_precision: 0.8037
Epoch 2/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7157 - f1_score: 0.7142
- loss: 0.9740 - precision: 0.8482
Epoch 2: val_loss improved from 1.22872 to 1.21051, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7157 - f1_score: 0.7141 - loss: 0.9740 - precision: 0.8482 - val_accuracy: 0.6607 - val_f1_score: 0.6593 - val_loss: 1.2105 - val_precision: 0.8031
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7333 - f1_score: 0.7320
- loss: 0.9058 - precision: 0.8594
Epoch 3: val_loss improved from 1.21051 to 1.19453, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7333 - f1_score: 0.7320 - loss: 0.9058 - precision: 0.8594 - val_accuracy: 0.6637 - val_f1_score: 0.6623 - val_loss: 1.1945 - val_precision: 0.8091
Epoch 4/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7499 - f1_score: 0.7484
- loss: 0.8515 - precision: 0.8702
Epoch 4: val_loss improved from 1.19453 to 1.18443, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7499 - f1_score: 0.7484 - loss: 0.8516 - precision: 0.8702 - val_accuracy: 0.6684 - val_f1_score: 0.6671 - val_loss: 1.1844 - val_precision: 0.8090
Epoch 5/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7611 - f1_score: 0.7598
- loss: 0.8075 - precision: 0.8776
Epoch 5: val_loss improved from 1.18443 to 1.18122, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7611 - f1_score: 0.7598 - loss: 0.8075 - precision: 0.8776 - val_accuracy: 0.6654 - val_f1_score: 0.6639 - val_loss: 1.1812 - val_precision: 0.8080
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7693 - f1_score: 0.7679
- loss: 0.7784 - precision: 0.8821
Epoch 6: val_loss improved from 1.18122 to 1.17629, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7693 - f1_score: 0.7679 - loss: 0.7785 - precision: 0.8821 - val_accuracy: 0.6720 - val_f1_score: 0.6707 - val_loss: 1.1763 - val_precision: 0.8101
Epoch 7/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7778 - f1_score: 0.7763
- loss: 0.7418 - precision: 0.8862
Epoch 7: val_loss improved from 1.17629 to 1.17286, saving model to best_model_fine_tune_v8.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7778 - f1_score: 0.7763 - loss: 0.7418 - precision: 0.8862 - val_accuracy: 0.6709 - val_f1_score: 0.6697 - val_loss: 1.1729 - val_precision: 0.8059
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7857 - f1_score: 0.7845
- loss: 0.7122 - precision: 0.8896
Epoch 8: val_loss improved from 1.17286 to 1.17062, saving model to best_model_fine_tune_v8.keras

ne_tune_v8.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7857 - f1_score: 0.7845 - loss: 0.7123 - precision: 0.8896 - val_accuracy: 0.6726 - val_f1_score: 0.6717 - val_loss: 1.1706 - val_precision: 0.8054

Epoch 9/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.7908 - f1_score: 0.7897 - loss: 0.6906 - precision: 0.8931

Epoch 9: val_loss improved from 1.17062 to 1.16682, saving model to best_model_fine_tune_v8.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7908 - f1_score: 0.7897 - loss: 0.6906 - precision: 0.8931 - val_accuracy: 0.6723 - val_f1_score: 0.6708 - val_loss: 1.1668 - val_precision: 0.8073

Epoch 10/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7993 - f1_score: 0.7982 - loss: 0.6666 - precision: 0.8995

Epoch 10: val_loss improved from 1.16682 to 1.16565, saving model to best_model_fine_tune_v8.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7992 - f1_score: 0.7982 - loss: 0.6667 - precision: 0.8994 - val_accuracy: 0.6732 - val_f1_score: 0.6720 - val_loss: 1.1657 - val_precision: 0.8054

Epoch 11/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.8092 - f1_score: 0.8082 - loss: 0.6373 - precision: 0.9033

Epoch 11: val_loss did not improve from 1.16565

1314/1314 ————— 35s 27ms/step - accuracy: 0.8092 - f1_score: 0.8082 - loss: 0.6373 - precision: 0.9033 - val_accuracy: 0.6739 - val_f1_score: 0.6725 - val_loss: 1.1662 - val_precision: 0.8070

Epoch 12/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.8154 - f1_score: 0.8144 - loss: 0.6151 - precision: 0.9049

Epoch 12: val_loss did not improve from 1.16565

1314/1314 ————— 35s 27ms/step - accuracy: 0.8154 - f1_score: 0.8143 - loss: 0.6152 - precision: 0.9049 - val_accuracy: 0.6710 - val_f1_score: 0.6698 - val_loss: 1.1694 - val_precision: 0.8008

Epoch 13/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.8159 - f1_score: 0.8148 - loss: 0.6040 - precision: 0.9044

Epoch 13: val_loss did not improve from 1.16565

1314/1314 ————— 35s 27ms/step - accuracy: 0.8159 - f1_score: 0.8148 - loss: 0.6040 - precision: 0.9044 - val_accuracy: 0.6733 - val_f1_score: 0.6721 - val_loss: 1.1682 - val_precision: 0.8004

Epoch 14/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.8245 - f1_score: 0.8235 - loss: 0.5812 - precision: 0.9085

Epoch 14: val_loss did not improve from 1.16565

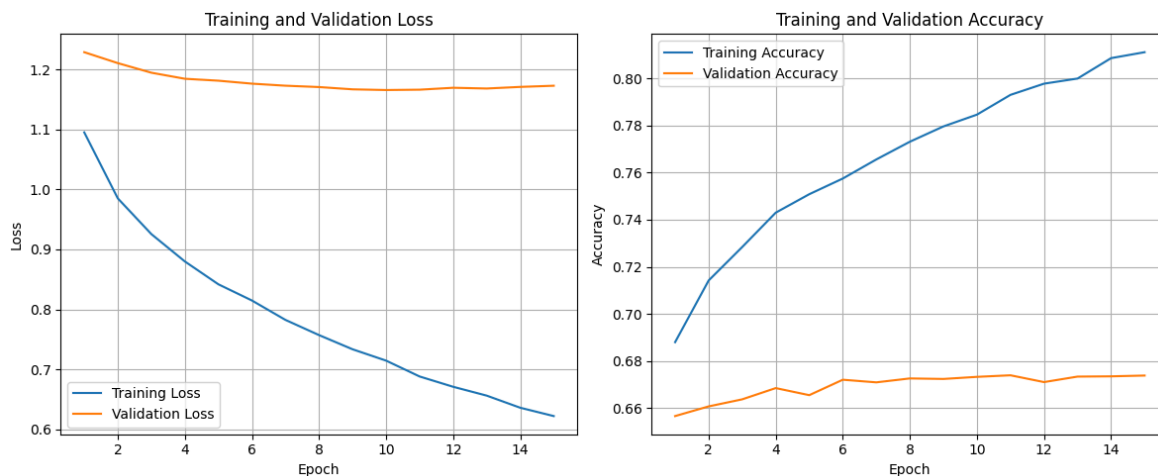
1314/1314 ————— 35s 26ms/step - accuracy: 0.8245 - f1_score: 0.8235 - loss: 0.5813 - precision: 0.9084 - val_accuracy: 0.6734 - val_f1_score: 0.6724 - val_loss: 1.1709 - val_precision: 0.7983

Epoch 15/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.8269 - f1_score: 0.8261 - loss: 0.5708 - precision: 0.9105

Epoch 15: val_loss did not improve from 1.16565

1314/1314 ————— 35s 26ms/step - accuracy: 0.8269 - f1_score: 0.8261 - loss: 0.5709 - precision: 0.9105 - val_accuracy: 0.6738 - val_f1_score: 0.6724 - val_loss: 1.1728 - val_precision: 0.7980



141/141 ————— 11s 38ms/step - accuracy: 0.6681 - f1_score: 0.6601

- loss: 1.1878 - precision: 0.7967

Test Accuracy: 68.21%

Test Loss: 1.1633059978485107

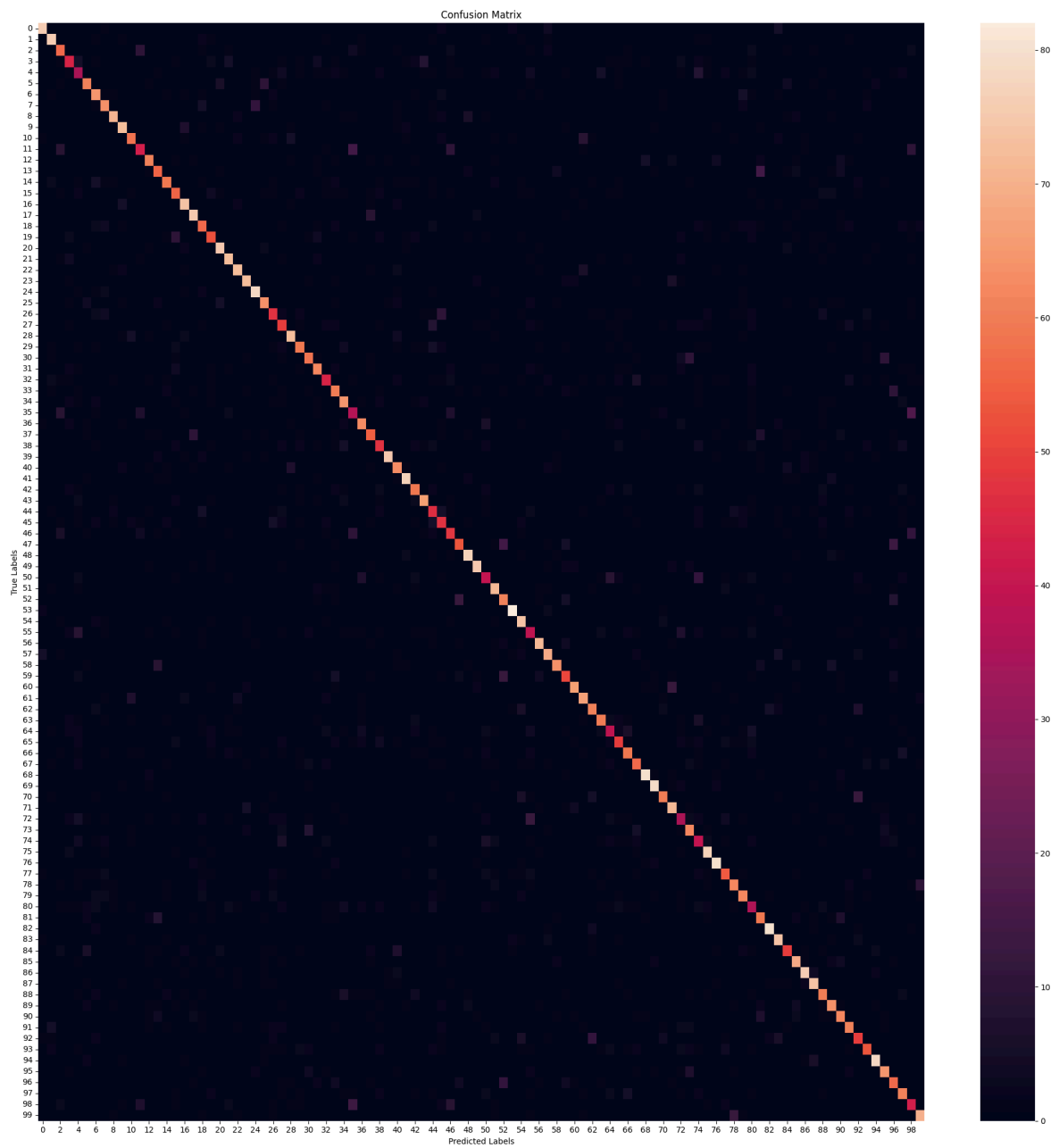
Test Precision: 80.62%

Test F1 Scores (Per Class): [85.39326 79.99999 60.63829 52.694607 38.251358 68.53932 65.989845

69.565216	81.60919	80.66298	68.63905	52.439022	70.93022	62.92134
69.82249	61.202175	77.89473	79.787224	60.31745	60.465115	82.41757
78.07486	80.66298	81.56424	83.422455	71.428566	54.651157	52.127655
80.66298	69.36415	67.44185	68.888885	54.65838	62.5	69.51871
41.57303	71.590904	64.32748	55.621304	88.75739	66.31578	84.61538
65.19337	76.300575	47.422672	51.366108	52.22218	63.855415	82.7957
84.26965	45.0867	75.39267	63.541664	90.60773	72.54902	44.318172
80.446915	75.40983	77.10842	58.620686	78.1065	74.03315	68.53932
69.36415	46.706585	57.309933	65.92178	65.517235	87.91208	86.81318
71.00591	75.789474	40.935665	65.957436	41.66666	81.91489	86.48647
62.7907	69.66292	68.888885	39.999996	61.13989	89.385475	75.89743
59.39393	74.19354	84.91619	78.07486	65.93406	70.329666	66.31015
68.53932	57.471264	63.855415	86.666664	71.03825	58.163254	67.40331
45.9893	78.65168					

Average Test F1 Scores:68.13

282/282 ————— 11s 28ms/step



	precision	recall	f1-score	support
beaver	0.86	0.84	0.85	90
dolphin	0.77	0.84	0.80	90
otter	0.58	0.63	0.61	90
seal	0.57	0.49	0.53	90
whale	0.38	0.39	0.38	90
aquarium fish	0.69	0.68	0.69	90
flatfish	0.61	0.72	0.66	90
ray	0.68	0.71	0.70	90
shark	0.85	0.79	0.82	90
trout	0.80	0.81	0.81	90
orchids	0.73	0.64	0.69	90
poppies	0.58	0.48	0.52	90
roses	0.74	0.68	0.71	90
sunflowers	0.64	0.62	0.63	90
tulips	0.75	0.66	0.70	90
bottles	0.60	0.62	0.61	90
bowls	0.74	0.82	0.78	90
cans	0.77	0.83	0.80	90
cups	0.58	0.63	0.60	90
plates	0.63	0.58	0.60	90
apples	0.81	0.83	0.82	90
mushrooms	0.75	0.81	0.78	90
oranges	0.80	0.81	0.81	90
pears	0.82	0.81	0.82	90
sweet peppers	0.80	0.87	0.83	90
clock	0.70	0.71	0.71	90
computer keyboard	0.57	0.52	0.55	90
lamp	0.50	0.54	0.52	90
telephone	0.80	0.81	0.81	90
television	0.72	0.66	0.69	90
bed	0.71	0.64	0.67	90
chair	0.69	0.69	0.69	90
couch	0.62	0.49	0.55	90
table	0.59	0.67	0.62	90
wardrobe	0.67	0.72	0.70	90
bee	0.43	0.41	0.42	90
beetle	0.73	0.70	0.72	90
butterfly	0.68	0.61	0.64	90
caterpillar	0.59	0.52	0.56	90
cockroach	0.95	0.83	0.89	90
bear	0.63	0.70	0.66	90
leopard	0.84	0.86	0.85	90
lion	0.65	0.66	0.65	90
tiger	0.80	0.73	0.76	90
wolf	0.44	0.51	0.47	90
bridge	0.51	0.52	0.51	90
castle	0.53	0.53	0.53	90
house	0.70	0.59	0.64	90
road	0.80	0.86	0.83	90
skyscraper	0.85	0.83	0.84	90
cloud	0.48	0.44	0.46	90
forest	0.71	0.80	0.75	90
mountain	0.60	0.68	0.64	90
plain	0.90	0.91	0.91	90
sea	0.65	0.82	0.73	90
camel	0.45	0.43	0.44	90
cattle	0.81	0.80	0.80	90
chimpanzee	0.74	0.77	0.75	90

elephant	0.84	0.71	0.77	90
kangaroo	0.61	0.57	0.59	90
fox	0.84	0.73	0.78	90
porcupine	0.74	0.74	0.74	90
possum	0.69	0.68	0.69	90
raccoon	0.72	0.67	0.69	90
skunk	0.51	0.43	0.47	90
crab	0.60	0.54	0.57	90
lobster	0.66	0.66	0.66	90
snail	0.68	0.63	0.66	90
spider	0.87	0.89	0.88	90
worm	0.86	0.88	0.87	90
baby	0.76	0.67	0.71	90
boy	0.72	0.80	0.76	90
girl	0.43	0.39	0.41	90
man	0.63	0.69	0.66	90
woman	0.39	0.44	0.41	90
crocodile	0.79	0.86	0.82	90
dinosaur	0.84	0.89	0.86	90
lizard	0.66	0.60	0.63	90
snake	0.70	0.69	0.70	90
turtle	0.69	0.69	0.69	90
hamster	0.40	0.40	0.40	90
mouse	0.57	0.66	0.61	90
rabbit	0.90	0.89	0.89	90
shrew	0.70	0.82	0.76	90
squirrel	0.65	0.54	0.59	90
maple	0.72	0.77	0.74	90
oak	0.85	0.84	0.85	90
palm	0.76	0.82	0.79	90
pine	0.65	0.67	0.66	90
willow	0.70	0.71	0.70	90
bicycle	0.64	0.69	0.66	90
bus	0.69	0.68	0.69	90
motorcycle	0.59	0.54	0.57	90
pickup truck	0.70	0.59	0.64	90
train	0.88	0.87	0.87	90
lawn-mower	0.70	0.72	0.71	90
rocket	0.54	0.63	0.58	90
streetcar	0.67	0.68	0.67	90
tank	0.44	0.48	0.46	90
tractor	0.80	0.78	0.79	90
accuracy			0.68	9000
macro avg	0.68	0.68	0.68	9000
weighted avg	0.68	0.68	0.68	9000

**RMSProp, Dropout(0.6),
weight_decay(3e-1), l2(3e-1),
momentum=0.2 - no epoch stop for
either part**

```
In [ ]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>
        # Load CIFAR-100 dataset
```

```

(X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

# Split (8000) of training data into temporary set
X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_size=0.1)
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp, y_temp, test_size=0.5)
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5)

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & accuracy
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'fish',
                 'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', 'bottle',
                 'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock',
                 'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardrobe',
                 'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wolf',
                 'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea',
                 'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 'squirrel',
                 'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile',
                 'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple',
                 'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mower',
                 'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100), dtype=tf.float32))
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE))

```

```

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
#                      .batch(batch_size)
#                      .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<Pre-trained model>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes

```

```

])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'>
    print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.RMSprop(learning_rate=1e-3, momentum=0.2,
    loss='categorical_crossentropy',
    metrics=['accuracy', 'precision', 'f1_score']))

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
    patience=5, # Num. epochs with no improvement
    restore_best_weights=True)

#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#    factor=0.1, # Reduce lr by a factor
#    patience=3, # Num epochs w/ no improvement
#    min_lr=1e-6, # Min lr
#    verbose=1)
#tensorboard = TensorBoard(log_dir='./Logs', # Logs directory
#    histogram_freq=1, # Logs histograms for weights/acc
#    write_graph=True, # Logs graph of model
#    write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v9.keras', # Path where the best model is saved
    monitor='val_loss', # What to monitor (validation loss)
    save_best_only=True, # Only save model when val loss is best
    mode='min', # Looking to minimize the validation loss
    verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v9.keras', # Path where the best model is saved
    monitor='val_loss', # What to monitor (validation loss)
    save_best_only=True, # Only save model when val loss is best
    mode='min', # Looking to minimize the validation loss
    verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
    batch_size=batch_size, callbacks=[early_stopping, checkpoint, checkpoint_fine_tune])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

```

```

# Extract Loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v9.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower Learning rate (prevents destroying learned features

```

```

model.compile(optimizer=optimizers.Adam(learning_rate=1e-4),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset,
                              batch_size=batch_size, callbacks=[checkpoint_f

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v9.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

```

```

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

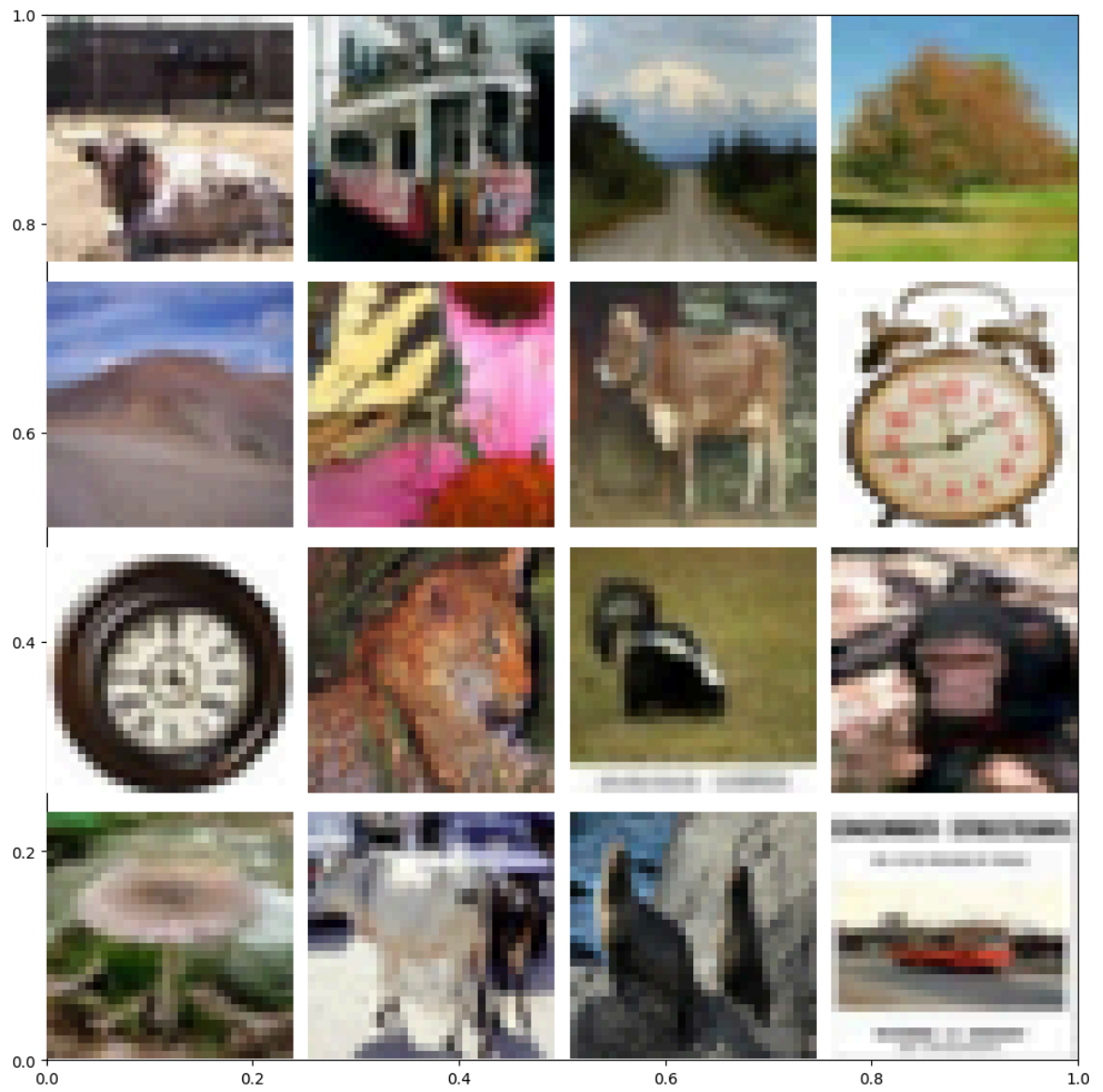
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

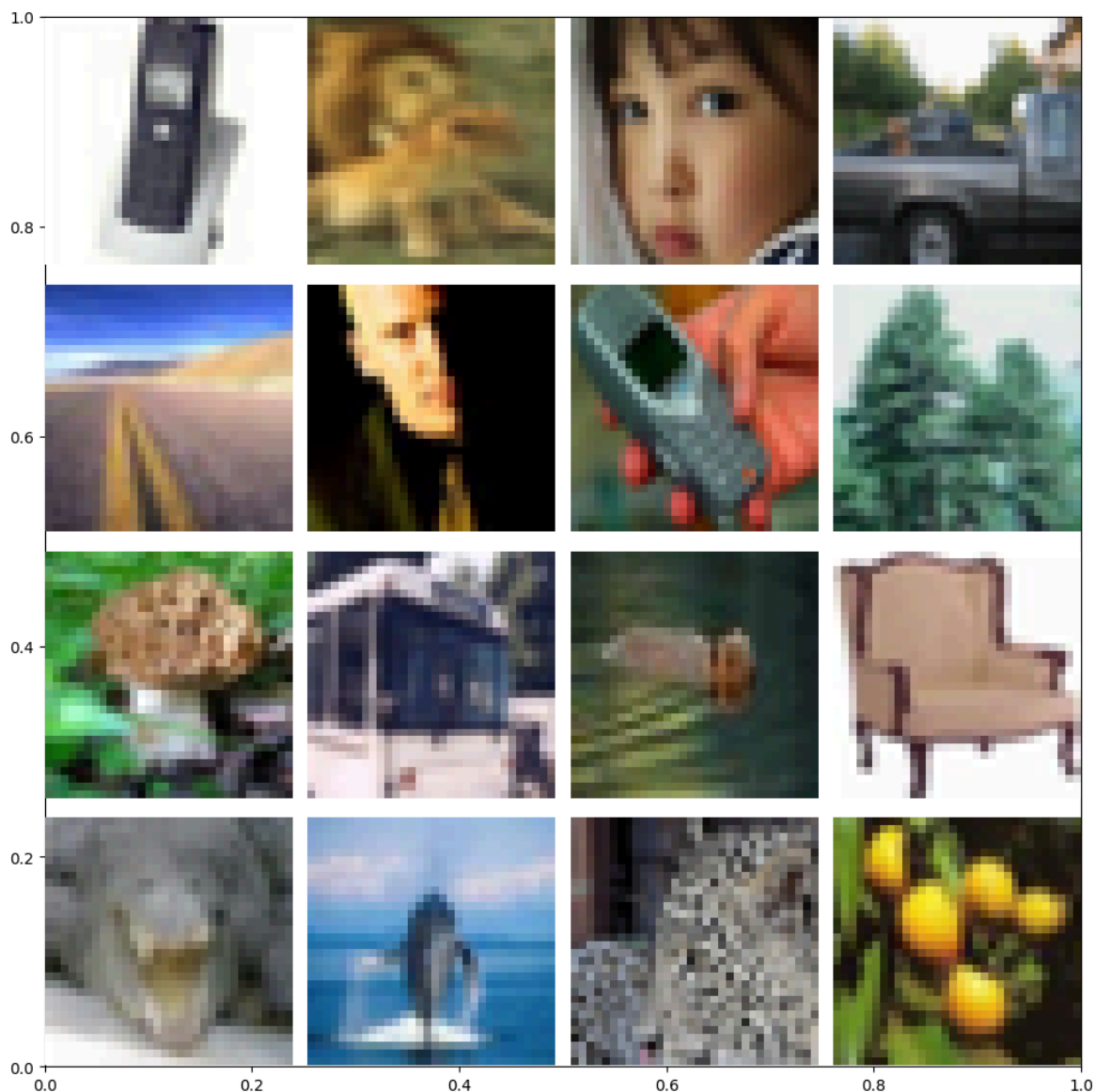
y_test.shape: (9000, 1)

labels[7581].shape: (1,)

imgs[7581].shape: (32, 32, 3)

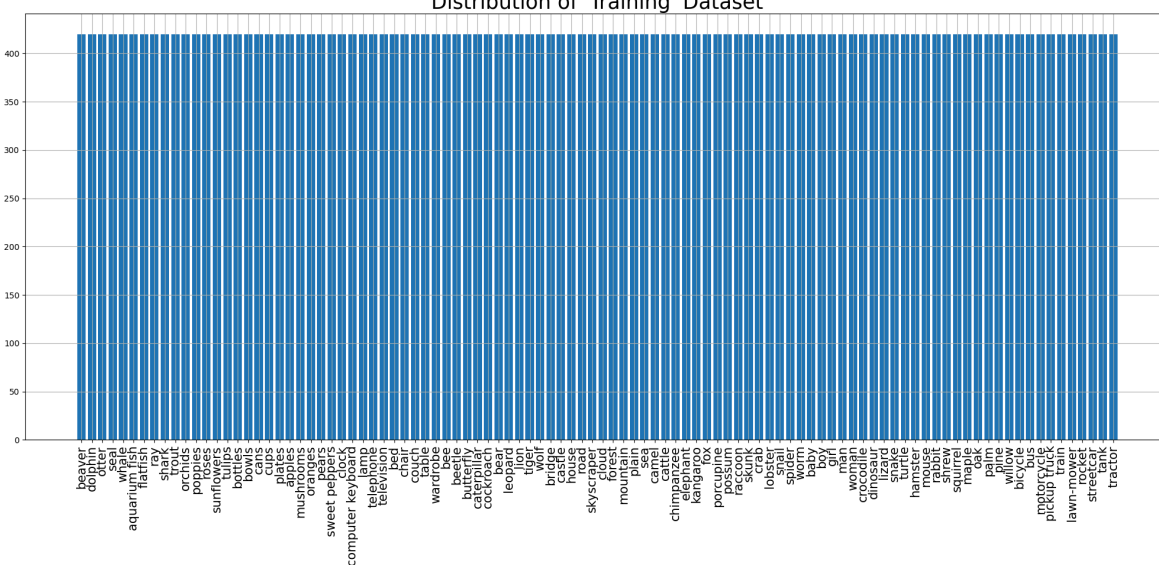


labels[23799].shape: (1,)
imgs[23799].shape: (32, 32, 3)

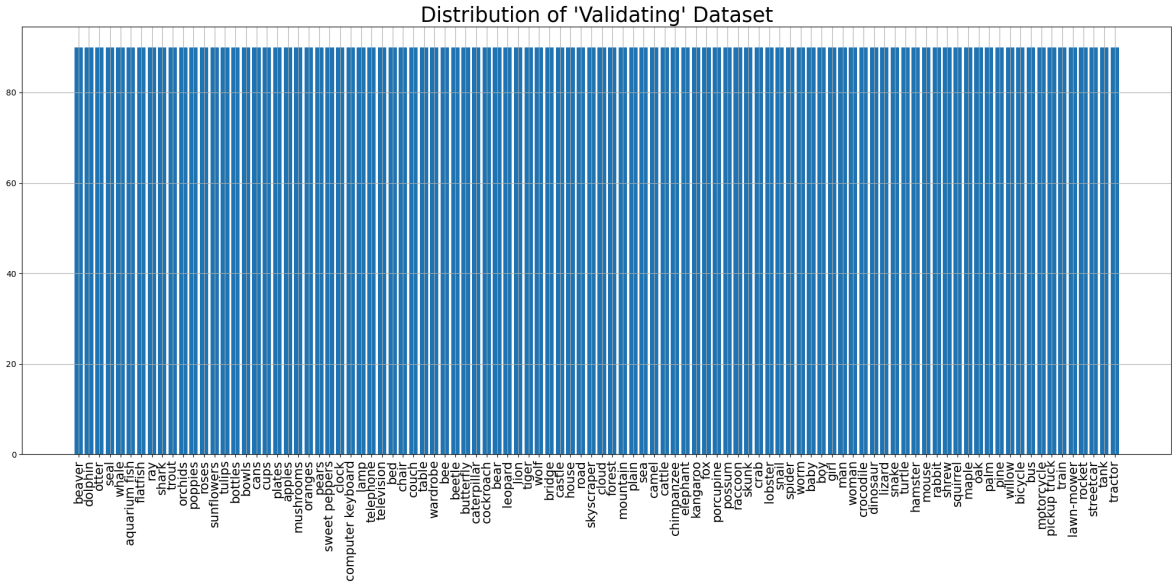


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels names, rotation=90, fontsize=15)
```

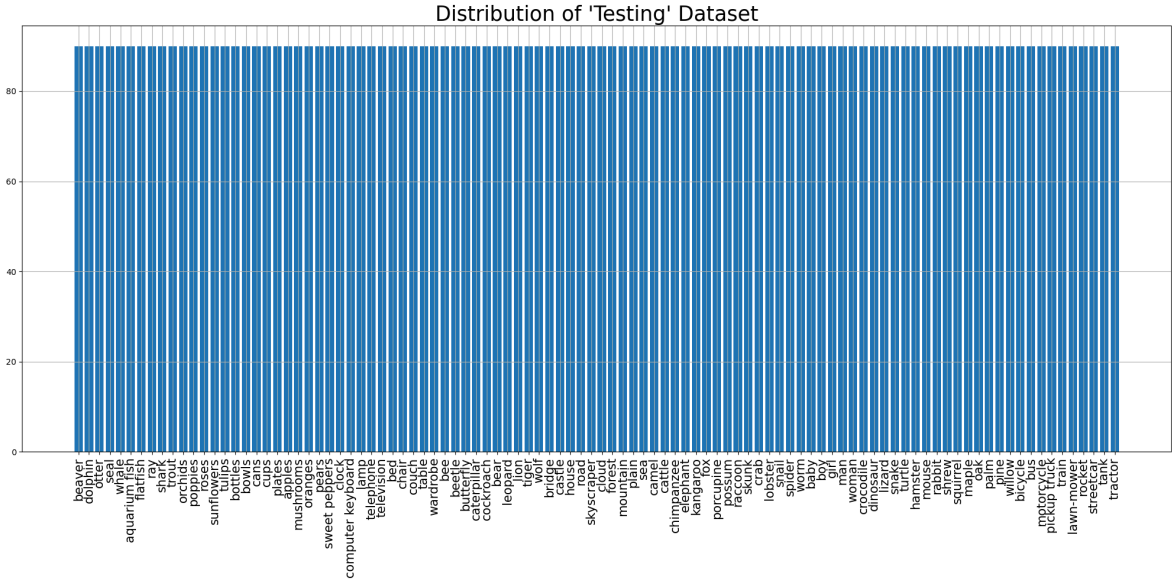
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_14"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_14 (GlobalAveragePooling2D)	(None , 2048)
dense_28 (Dense)	(None , 512)
dropout_14 (Dropout)	(None , 512)
dense_29 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25

657/657 ————— **0s** 29ms/step - accuracy: 0.3301 - f1_score: 0.3228 - loss: 2.8255 - precision: 0.6331

Epoch 1: val_loss improved from inf to 1.53193, saving model to best_model_v9.keras

657/657 ————— **42s** 44ms/step - accuracy: 0.3302 - f1_score: 0.3229 - loss: 2.8247 - precision: 0.6331 - val_accuracy: 0.5727 - val_f1_score: 0.5700 - val_loss: 1.5319 - val_precision: 0.7719

Epoch 2/25

656/657 ————— **0s** 24ms/step - accuracy: 0.5141 - f1_score: 0.5089 - loss: 1.8248 - precision: 0.7106

Epoch 2: val_loss improved from 1.53193 to 1.44819, saving model to best_model_v9.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.5142 - f1_score: 0.5089 - loss: 1.8247 - precision: 0.7106 - val_accuracy: 0.5954 - val_f1_score: 0.5933 - val_loss: 1.4482 - val_precision: 0.7805

Epoch 3/25

656/657 ————— **0s** 24ms/step - accuracy: 0.5446 - f1_score: 0.5405 - loss: 1.6946 - precision: 0.7232

Epoch 3: val_loss did not improve from 1.44819

657/657 ————— **19s** 29ms/step - accuracy: 0.5446 - f1_score: 0.5406 - loss: 1.6945 - precision: 0.7232 - val_accuracy: 0.5992 - val_f1_score: 0.5980 - val_loss: 1.4724 - val_precision: 0.7664

Epoch 4/25

655/657 ————— **0s** 24ms/step - accuracy: 0.5624 - f1_score: 0.5591 - loss: 1.6293 - precision: 0.7349

Epoch 4: val_loss improved from 1.44819 to 1.43201, saving model to best_model_v9.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.5624 - f1_score: 0.5591 - loss: 1.6292 - precision: 0.7349 - val_accuracy: 0.6059 - val_f1_score: 0.6061 - val_loss: 1.4320 - val_precision: 0.7675

Epoch 5/25

655/657 ————— **0s** 24ms/step - accuracy: 0.5730 - f1_score: 0.5694 - loss: 1.5735 - precision: 0.7356

Epoch 5: val_loss improved from 1.43201 to 1.40804, saving model to best_model_v9.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.5730 - f1_score: 0.5694 - loss: 1.5734 - precision: 0.7357 - val_accuracy: 0.6174 - val_f1_score: 0.6141 - val_loss: 1.4080 - val_precision: 0.7799

Epoch 6/25

656/657 ————— **0s** 24ms/step - accuracy: 0.5847 - f1_score: 0.5816 - loss: 1.5384 - precision: 0.7474

Epoch 6: val_loss did not improve from 1.40804

657/657 ————— **19s** 28ms/step - accuracy: 0.5848 - f1_score: 0.5816 - loss: 1.5383 - precision: 0.7474 - val_accuracy: 0.6168 - val_f1_score: 0.6159 - val_loss: 1.4139 - val_precision: 0.7708

Epoch 7/25

655/657 ————— **0s** 24ms/step - accuracy: 0.5855 - f1_score: 0.5825 - loss: 1.5366 - precision: 0.7463

Epoch 7: val_loss did not improve from 1.40804

657/657 ————— **19s** 28ms/step - accuracy: 0.5855 - f1_score: 0.5825 - loss: 1.5365 - precision: 0.7464 - val_accuracy: 0.6082 - val_f1_score: 0.6027 - val_loss: 1.4368 - val_precision: 0.7629

Epoch 8/25

655/657 ————— **0s** 24ms/step - accuracy: 0.5885 - f1_score: 0.5852 - loss: 1.4978 - precision: 0.7474

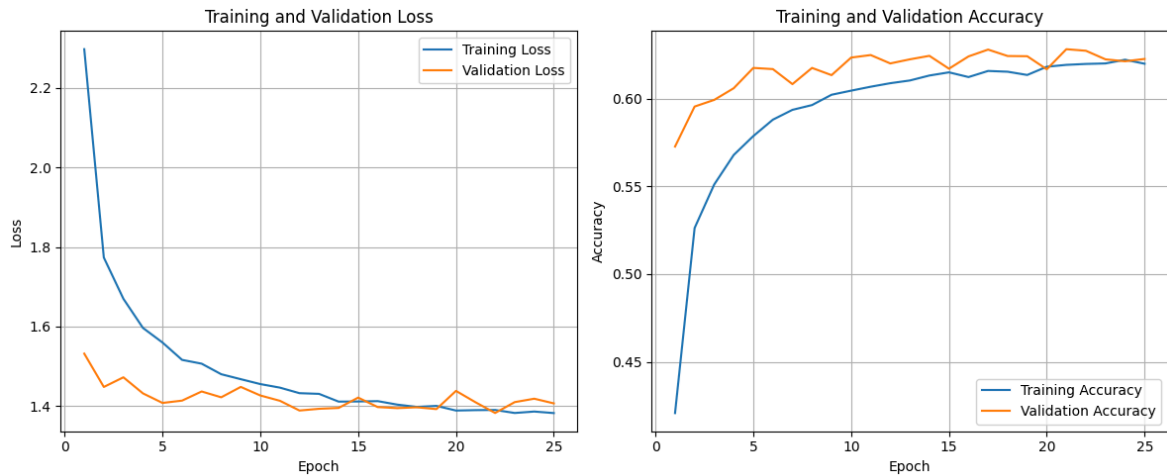
Epoch 8: val_loss did not improve from 1.40804

657/657 ————— **19s** 29ms/step - accuracy: 0.5886 - f1_score: 0.5852 - loss: 1.4977 - precision: 0.7474 - val_accuracy: 0.6174 - val_f1_score: 0.6165 - val_loss: 1.4222 - val_precision: 0.7660

Epoch 9/25
657/657 ————— 0s 24ms/step - accuracy: 0.5958 - f1_score: 0.5923 -
loss: 1.4893 - precision: 0.7518
Epoch 9: val_loss did not improve from 1.40804
657/657 ————— 19s 29ms/step - accuracy: 0.5958 - f1_score: 0.5923
- loss: 1.4893 - precision: 0.7518 - val_accuracy: 0.6133 - val_f1_score: 0.6093
- val_loss: 1.4483 - val_precision: 0.7505
Epoch 10/25
657/657 ————— 0s 24ms/step - accuracy: 0.6013 - f1_score: 0.5978 -
loss: 1.4684 - precision: 0.7548
Epoch 10: val_loss did not improve from 1.40804
657/657 ————— 19s 29ms/step - accuracy: 0.6013 - f1_score: 0.5978
- loss: 1.4684 - precision: 0.7548 - val_accuracy: 0.6233 - val_f1_score: 0.6181
- val_loss: 1.4268 - val_precision: 0.7644
Epoch 11/25
657/657 ————— 0s 24ms/step - accuracy: 0.6028 - f1_score: 0.6000 -
loss: 1.4594 - precision: 0.7546
Epoch 11: val_loss did not improve from 1.40804
657/657 ————— 19s 29ms/step - accuracy: 0.6028 - f1_score: 0.6000
- loss: 1.4594 - precision: 0.7546 - val_accuracy: 0.6248 - val_f1_score: 0.6193
- val_loss: 1.4134 - val_precision: 0.7685
Epoch 12/25
657/657 ————— 0s 24ms/step - accuracy: 0.6074 - f1_score: 0.6037 -
loss: 1.4416 - precision: 0.7635
Epoch 12: val_loss improved from 1.40804 to 1.38896, saving model to best_model_v
9.keras
657/657 ————— 20s 30ms/step - accuracy: 0.6074 - f1_score: 0.6037
- loss: 1.4416 - precision: 0.7635 - val_accuracy: 0.6200 - val_f1_score: 0.6147
- val_loss: 1.3890 - val_precision: 0.7699
Epoch 13/25
657/657 ————— 0s 24ms/step - accuracy: 0.6047 - f1_score: 0.6017 -
loss: 1.4559 - precision: 0.7563
Epoch 13: val_loss did not improve from 1.38896
657/657 ————— 19s 29ms/step - accuracy: 0.6047 - f1_score: 0.6017
- loss: 1.4559 - precision: 0.7563 - val_accuracy: 0.6223 - val_f1_score: 0.6201
- val_loss: 1.3933 - val_precision: 0.7686
Epoch 14/25
656/657 ————— 0s 24ms/step - accuracy: 0.6095 - f1_score: 0.6068 -
loss: 1.4237 - precision: 0.7568
Epoch 14: val_loss did not improve from 1.38896
657/657 ————— 19s 29ms/step - accuracy: 0.6095 - f1_score: 0.6069
- loss: 1.4237 - precision: 0.7568 - val_accuracy: 0.6243 - val_f1_score: 0.6214
- val_loss: 1.3954 - val_precision: 0.7772
Epoch 15/25
656/657 ————— 0s 24ms/step - accuracy: 0.6123 - f1_score: 0.6092 -
loss: 1.4223 - precision: 0.7630
Epoch 15: val_loss did not improve from 1.38896
657/657 ————— 19s 28ms/step - accuracy: 0.6123 - f1_score: 0.6092
- loss: 1.4223 - precision: 0.7630 - val_accuracy: 0.6170 - val_f1_score: 0.6139
- val_loss: 1.4214 - val_precision: 0.7588
Epoch 16/25
657/657 ————— 0s 24ms/step - accuracy: 0.6083 - f1_score: 0.6058 -
loss: 1.4266 - precision: 0.7595
Epoch 16: val_loss did not improve from 1.38896
657/657 ————— 19s 29ms/step - accuracy: 0.6083 - f1_score: 0.6058
- loss: 1.4266 - precision: 0.7595 - val_accuracy: 0.6240 - val_f1_score: 0.6194
- val_loss: 1.3976 - val_precision: 0.7691
Epoch 17/25
655/657 ————— 0s 24ms/step - accuracy: 0.6132 - f1_score: 0.6098 -
loss: 1.4216 - precision: 0.7557

Epoch 17: val_loss did not improve from 1.38896
657/657  **19s** 29ms/step - accuracy: 0.6132 - f1_score: 0.6098
- loss: 1.4215 - precision: 0.7557 - val_accuracy: 0.6279 - val_f1_score: 0.6245
- val_loss: 1.3949 - val_precision: 0.7704
Epoch 18/25
655/657  **0s** 24ms/step - accuracy: 0.6128 - f1_score: 0.6102 -
loss: 1.4120 - precision: 0.7610
Epoch 18: val_loss did not improve from 1.38896
657/657  **19s** 29ms/step - accuracy: 0.6128 - f1_score: 0.6102
- loss: 1.4120 - precision: 0.7610 - val_accuracy: 0.6242 - val_f1_score: 0.6233
- val_loss: 1.3971 - val_precision: 0.7714
Epoch 19/25
655/657  **0s** 24ms/step - accuracy: 0.6072 - f1_score: 0.6043 -
loss: 1.4183 - precision: 0.7566
Epoch 19: val_loss did not improve from 1.38896
657/657  **19s** 29ms/step - accuracy: 0.6072 - f1_score: 0.6043
- loss: 1.4182 - precision: 0.7566 - val_accuracy: 0.6241 - val_f1_score: 0.6217
- val_loss: 1.3925 - val_precision: 0.7697
Epoch 20/25
656/657  **0s** 24ms/step - accuracy: 0.6143 - f1_score: 0.6112 -
loss: 1.4061 - precision: 0.7608
Epoch 20: val_loss did not improve from 1.38896
657/657  **19s** 29ms/step - accuracy: 0.6143 - f1_score: 0.6112
- loss: 1.4060 - precision: 0.7608 - val_accuracy: 0.6167 - val_f1_score: 0.6153
- val_loss: 1.4383 - val_precision: 0.7636
Epoch 21/25
655/657  **0s** 24ms/step - accuracy: 0.6163 - f1_score: 0.6131 -
loss: 1.4028 - precision: 0.7588
Epoch 21: val_loss did not improve from 1.38896
657/657  **19s** 29ms/step - accuracy: 0.6163 - f1_score: 0.6132
- loss: 1.4028 - precision: 0.7588 - val_accuracy: 0.6281 - val_f1_score: 0.6258
- val_loss: 1.4096 - val_precision: 0.7632
Epoch 22/25
657/657  **0s** 24ms/step - accuracy: 0.6181 - f1_score: 0.6155 -
loss: 1.3991 - precision: 0.7643
Epoch 22: val_loss improved from 1.38896 to 1.38281, saving model to best_model_v
9.keras
657/657  **20s** 30ms/step - accuracy: 0.6181 - f1_score: 0.6155
- loss: 1.3991 - precision: 0.7643 - val_accuracy: 0.6272 - val_f1_score: 0.6263
- val_loss: 1.3828 - val_precision: 0.7757
Epoch 23/25
656/657  **0s** 24ms/step - accuracy: 0.6151 - f1_score: 0.6123 -
loss: 1.3965 - precision: 0.7619
Epoch 23: val_loss did not improve from 1.38281
657/657  **19s** 29ms/step - accuracy: 0.6151 - f1_score: 0.6123
- loss: 1.3964 - precision: 0.7619 - val_accuracy: 0.6223 - val_f1_score: 0.6232
- val_loss: 1.4101 - val_precision: 0.7639
Epoch 24/25
656/657  **0s** 24ms/step - accuracy: 0.6192 - f1_score: 0.6163 -
loss: 1.4063 - precision: 0.7590
Epoch 24: val_loss did not improve from 1.38281
657/657  **19s** 29ms/step - accuracy: 0.6192 - f1_score: 0.6163
- loss: 1.4062 - precision: 0.7590 - val_accuracy: 0.6212 - val_f1_score: 0.6191
- val_loss: 1.4187 - val_precision: 0.7659
Epoch 25/25
657/657  **0s** 24ms/step - accuracy: 0.6148 - f1_score: 0.6127 -
loss: 1.4049 - precision: 0.7632
Epoch 25: val_loss did not improve from 1.38281
657/657  **19s** 29ms/step - accuracy: 0.6148 - f1_score: 0.6127

- loss: 1.4049 - precision: 0.7633 - val_accuracy: 0.6226 - val_f1_score: 0.6184
- val_loss: 1.4069 - val_precision: 0.7722



141/141 ————— **11s** 38ms/step - accuracy: 0.6130 - f1_score: 0.6047

- loss: 1.4060 - precision: 0.7704

Test Accuracy: 62.46%

Test Loss: 1.3874173164367676

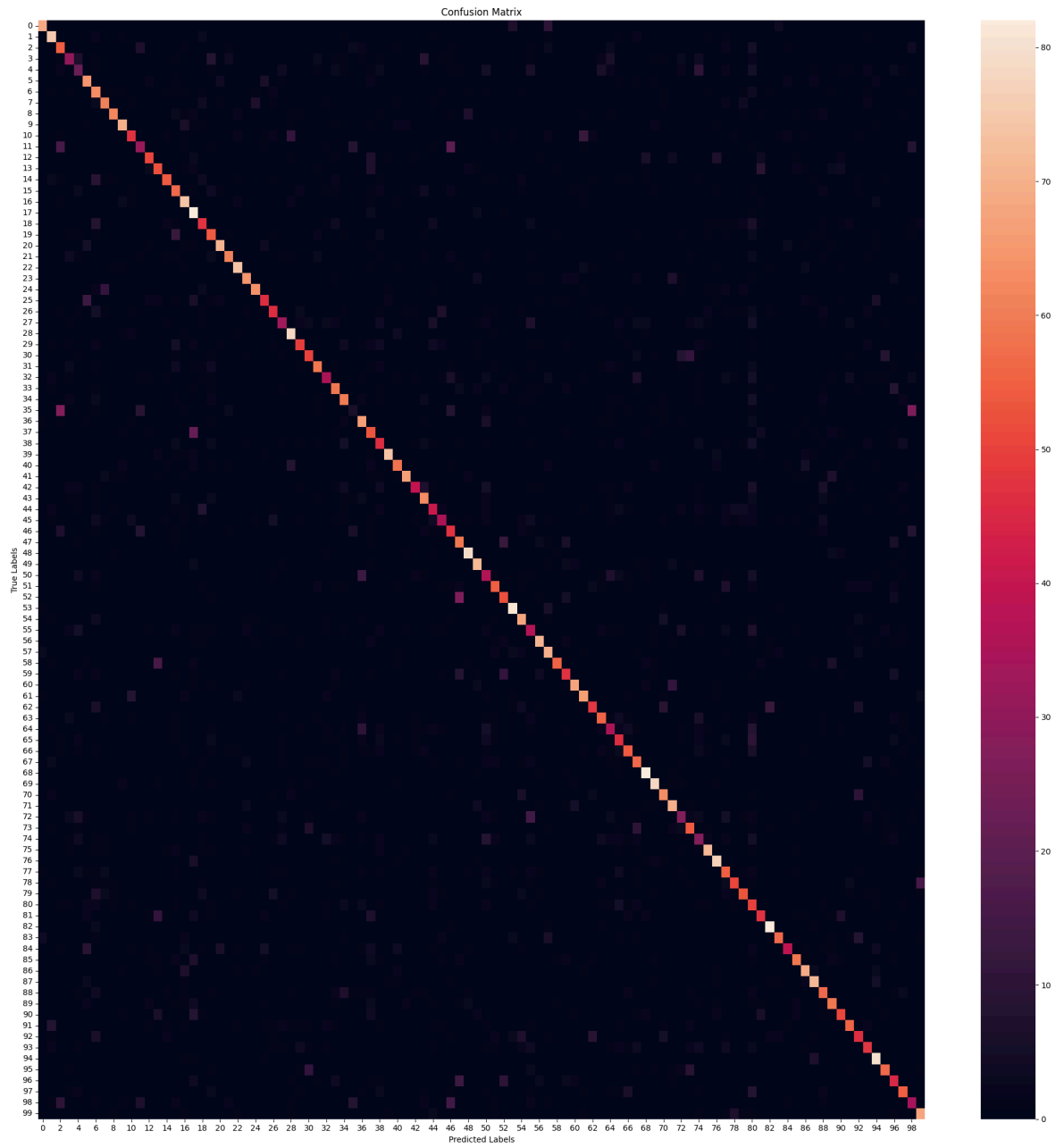
Test Precision: 78.19%

Test F1 Scores (Per Class): [81.17647 75.75757 50.691242 41.059597 28.048775 6
6.32652 51.428562

68.15642	76.543205	79.54544	58.385086	40.259735	64.15094	56.830597
68.35443	55.882347	72.195114	62.357407	51.933693	54.82233	80.681816
73.93939	81.767944	76.02339	77.10842	57.499992	56.44171	43.421047
76.84728	61.25	60.24096	70.23808	46.66666	62.433857	63.157898
9.756094	64.423065	51.456303	48.42105	87.058815	67.05202	81.21211
55.944054	65.284966	46.06741	46.979862	46.078426	56.19047	83.59788
77.00535	36.548218	65.088745	57.777775	84.37499	69.03552	37.563442
77.59562	68.99999	73.684204	55.621304	79.069756	71.35134	60.377354
64.	42.16867	50.27322	62.06896	58.461536	83.248726	82.53968
68.852455	72.916664	35.06493	61.01694	29.670324	80.89887	78.75647
54.99999	64.55696	64.63414	34.364254	56.28742	84.102554	71.25
55.71428	71.51515	75.40983	77.00535	60.96256	65.55555	61.818176
66.666664	52.873566	57.48503	78.431366	68.67469	48.648643	65.868256
37.988823	72.727264]					

Average Test F1 Scores:62.33

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.86	0.77	0.81	90
dolphin	0.69	0.83	0.76	90
otter	0.43	0.61	0.51	90
seal	0.50	0.33	0.40	90
whale	0.31	0.26	0.28	90
aquarium fish	0.61	0.72	0.66	90
flatfish	0.41	0.70	0.51	90
ray	0.69	0.68	0.68	90
shark	0.86	0.69	0.77	90
trout	0.81	0.78	0.80	90
orchids	0.66	0.52	0.58	90
poppies	0.48	0.34	0.40	90
roses	0.74	0.57	0.64	90
sunflowers	0.56	0.58	0.57	90
tulips	0.79	0.60	0.68	90
bottles	0.50	0.63	0.56	90
bowls	0.64	0.82	0.72	90
cans	0.47	0.91	0.62	90
cups	0.52	0.52	0.52	90
plates	0.50	0.60	0.55	90
apples	0.83	0.79	0.81	90
mushrooms	0.81	0.68	0.74	90
oranges	0.81	0.82	0.82	90
pears	0.80	0.72	0.76	90
sweet peppers	0.84	0.71	0.77	90
clock	0.66	0.51	0.57	90
computer keyboard	0.63	0.51	0.56	90
lamp	0.53	0.37	0.43	90
telephone	0.69	0.87	0.77	90
television	0.69	0.54	0.61	90
bed	0.66	0.56	0.60	90
chair	0.76	0.66	0.70	90
couch	0.58	0.39	0.47	90
table	0.60	0.66	0.62	90
wardrobe	0.60	0.67	0.63	90
bee	0.18	0.07	0.10	90
beetle	0.57	0.74	0.65	90
butterfly	0.46	0.59	0.51	90
caterpillar	0.46	0.51	0.49	90
cockroach	0.93	0.82	0.87	90
bear	0.70	0.64	0.67	90
leopard	0.89	0.74	0.81	90
lion	0.75	0.44	0.56	90
tiger	0.61	0.70	0.65	90
wolf	0.47	0.46	0.46	90
bridge	0.59	0.39	0.47	90
castle	0.41	0.52	0.46	90
house	0.49	0.66	0.56	90
road	0.80	0.88	0.84	90
skyscraper	0.74	0.80	0.77	90
cloud	0.34	0.40	0.37	90
forest	0.70	0.61	0.65	90
mountain	0.58	0.58	0.58	90
plain	0.79	0.90	0.84	90
sea	0.64	0.76	0.69	90
camel	0.35	0.41	0.38	90
cattle	0.76	0.79	0.78	90
chimpanzee	0.63	0.78	0.70	90

elephant	0.90	0.62	0.74	90
kangaroo	0.59	0.52	0.56	90
fox	0.83	0.76	0.79	90
porcupine	0.70	0.73	0.72	90
possum	0.70	0.53	0.60	90
raccoon	0.66	0.62	0.64	90
skunk	0.46	0.39	0.42	90
crab	0.49	0.51	0.50	90
lobster	0.64	0.60	0.62	90
snail	0.54	0.63	0.58	90
spider	0.77	0.91	0.83	90
worm	0.80	0.87	0.83	90
baby	0.68	0.70	0.69	90
boy	0.69	0.78	0.73	90
girl	0.42	0.30	0.35	90
man	0.62	0.60	0.61	90
woman	0.29	0.30	0.30	90
crocodile	0.82	0.80	0.81	90
dinosaur	0.74	0.86	0.79	90
lizard	0.50	0.61	0.55	90
snake	0.75	0.57	0.65	90
turtle	0.72	0.59	0.65	90
hamster	0.25	0.56	0.34	90
mouse	0.61	0.52	0.56	90
rabbit	0.78	0.91	0.84	90
shrew	0.81	0.63	0.71	90
squirrel	0.78	0.43	0.56	90
maple	0.79	0.66	0.72	90
oak	0.74	0.77	0.75	90
palm	0.74	0.80	0.77	90
pine	0.59	0.63	0.61	90
willow	0.66	0.66	0.66	90
bicycle	0.68	0.57	0.62	90
bus	0.72	0.62	0.67	90
motorcycle	0.55	0.51	0.53	90
pickup truck	0.62	0.53	0.57	90
train	0.70	0.89	0.78	90
lawn-mower	0.75	0.63	0.69	90
rocket	0.47	0.50	0.49	90
streetcar	0.71	0.61	0.66	90
tank	0.38	0.38	0.38	90
tractor	0.70	0.76	0.73	90
accuracy			0.62	9000
macro avg	0.64	0.62	0.62	9000
weighted avg	0.64	0.62	0.62	9000

Model: "sequential_14"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_14 (GlobalAveragePooling2D)	(None , 2048)
dense_28 (Dense)	(None , 512)
dropout_14 (Dropout)	(None , 512)
dense_29 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1314/1314 ————— 0s 27ms/step - accuracy: 0.6669 - f1_score: 0.6655
- loss: 1.1791 - precision: 0.8119
Epoch 1: val_loss improved from inf to 1.26479, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.6669 - f1_score: 0.6655 - loss: 1.1791 - precision: 0.8119 - val_accuracy: 0.6556 - val_f1_score: 0.6551 - val_loss: 1.2648 - val_precision: 0.7977
Epoch 2/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7074 - f1_score: 0.7059 - loss: 1.0123 - precision: 0.8401
Epoch 2: val_loss improved from 1.26479 to 1.23869, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7074 - f1_score: 0.7059 - loss: 1.0123 - precision: 0.8401 - val_accuracy: 0.6612 - val_f1_score: 0.6603 - val_loss: 1.2387 - val_precision: 0.8055
Epoch 3/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7263 - f1_score: 0.7249 - loss: 0.9451 - precision: 0.8577
Epoch 3: val_loss improved from 1.23869 to 1.22881, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7263 - f1_score: 0.7249 - loss: 0.9451 - precision: 0.8577 - val_accuracy: 0.6617 - val_f1_score: 0.6601 - val_loss: 1.2288 - val_precision: 0.8027
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7394 - f1_score: 0.7380 - loss: 0.8849 - precision: 0.8641
Epoch 4: val_loss improved from 1.22881 to 1.21371, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7394 - f1_score: 0.7380 - loss: 0.8849 - precision: 0.8641 - val_accuracy: 0.6652 - val_f1_score: 0.6642 - val_loss: 1.2137 - val_precision: 0.8066
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7476 - f1_score: 0.7465 - loss: 0.8457 - precision: 0.8720
Epoch 5: val_loss improved from 1.21371 to 1.21301, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7476 - f1_score: 0.7465 - loss: 0.8458 - precision: 0.8719 - val_accuracy: 0.6651 - val_f1_score: 0.6643 - val_loss: 1.2130 - val_precision: 0.8051
Epoch 6/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7596 - f1_score: 0.7586 - loss: 0.8065 - precision: 0.8777
Epoch 6: val_loss improved from 1.21301 to 1.20403, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7596 - f1_score: 0.7586 - loss: 0.8065 - precision: 0.8777 - val_accuracy: 0.6688 - val_f1_score: 0.6674 - val_loss: 1.2040 - val_precision: 0.8037
Epoch 7/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7680 - f1_score: 0.7667 - loss: 0.7735 - precision: 0.8827
Epoch 7: val_loss improved from 1.20403 to 1.20210, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7680 - f1_score: 0.7667 - loss: 0.7735 - precision: 0.8827 - val_accuracy: 0.6689 - val_f1_score: 0.6678 - val_loss: 1.2021 - val_precision: 0.8060
Epoch 8/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7766 - f1_score: 0.7753 - loss: 0.7432 - precision: 0.8855
Epoch 8: val_loss improved from 1.20210 to 1.19637, saving model to best_model_fine_tune_v9.keras

ne_tune_v9.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7765 - f1_score: 0.7753 - loss: 0.7433 - precision: 0.8855 - val_accuracy: 0.6699 - val_f1_score: 0.6689 - val_loss: 1.1964 - val_precision: 0.8067

Epoch 9/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.7837 - f1_score: 0.7825 - loss: 0.7173 - precision: 0.8893

Epoch 9: val_loss improved from 1.19637 to 1.19611, saving model to best_model_fine_tune_v9.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.7837 - f1_score: 0.7825 - loss: 0.7173 - precision: 0.8893 - val_accuracy: 0.6704 - val_f1_score: 0.6694 - val_loss: 1.1961 - val_precision: 0.8041

Epoch 10/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.7921 - f1_score: 0.7909 - loss: 0.6905 - precision: 0.8949

Epoch 10: val_loss improved from 1.19611 to 1.19436, saving model to best_model_fine_tune_v9.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7921 - f1_score: 0.7909 - loss: 0.6906 - precision: 0.8949 - val_accuracy: 0.6741 - val_f1_score: 0.6728 - val_loss: 1.1944 - val_precision: 0.8033

Epoch 11/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.8002 - f1_score: 0.7991 - loss: 0.6662 - precision: 0.8995

Epoch 11: val_loss did not improve from 1.19436

1314/1314 ————— 35s 26ms/step - accuracy: 0.8001 - f1_score: 0.7991 - loss: 0.6663 - precision: 0.8995 - val_accuracy: 0.6696 - val_f1_score: 0.6685 - val_loss: 1.1983 - val_precision: 0.8011

Epoch 12/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.8033 - f1_score: 0.8022 - loss: 0.6508 - precision: 0.9001

Epoch 12: val_loss did not improve from 1.19436

1314/1314 ————— 35s 26ms/step - accuracy: 0.8033 - f1_score: 0.8022 - loss: 0.6509 - precision: 0.9001 - val_accuracy: 0.6743 - val_f1_score: 0.6733 - val_loss: 1.1973 - val_precision: 0.7998

Epoch 13/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.8099 - f1_score: 0.8089 - loss: 0.6312 - precision: 0.9040

Epoch 13: val_loss did not improve from 1.19436

1314/1314 ————— 35s 26ms/step - accuracy: 0.8099 - f1_score: 0.8089 - loss: 0.6313 - precision: 0.9039 - val_accuracy: 0.6736 - val_f1_score: 0.6722 - val_loss: 1.1963 - val_precision: 0.8001

Epoch 14/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.8145 - f1_score: 0.8134 - loss: 0.6085 - precision: 0.9061

Epoch 14: val_loss did not improve from 1.19436

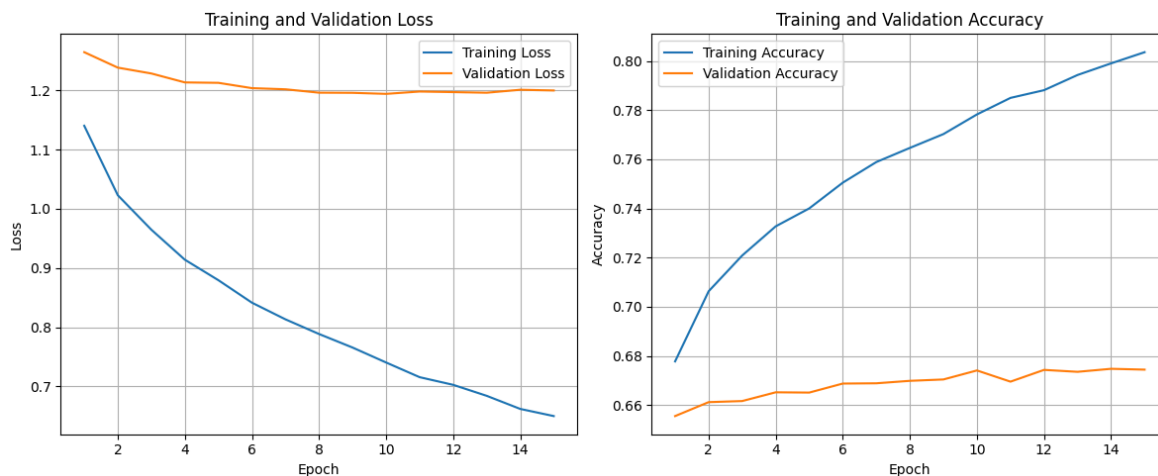
1314/1314 ————— 35s 26ms/step - accuracy: 0.8145 - f1_score: 0.8134 - loss: 0.6086 - precision: 0.9061 - val_accuracy: 0.6748 - val_f1_score: 0.6734 - val_loss: 1.2013 - val_precision: 0.7983

Epoch 15/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.8196 - f1_score: 0.8186 - loss: 0.5959 - precision: 0.9077

Epoch 15: val_loss did not improve from 1.19436

1314/1314 ————— 35s 26ms/step - accuracy: 0.8196 - f1_score: 0.8186 - loss: 0.5960 - precision: 0.9077 - val_accuracy: 0.6744 - val_f1_score: 0.6733 - val_loss: 1.2001 - val_precision: 0.7970



141/141 ————— 11s 38ms/step - accuracy: 0.6627 - f1_score: 0.6542

- loss: 1.2015 - precision: 0.7953

Test Accuracy: 67.62%

Test Loss: 1.180232286453247

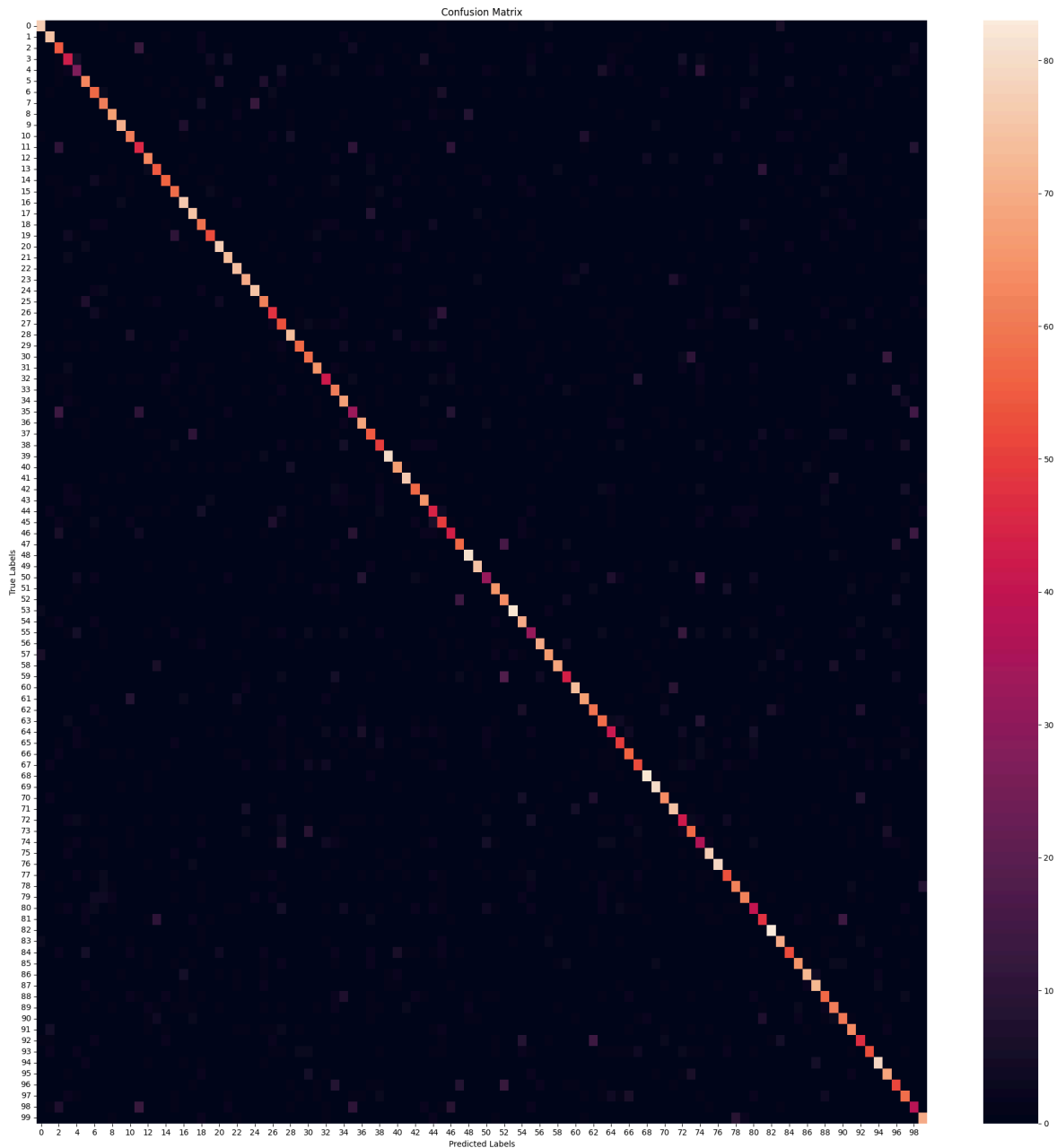
Test Precision: 80.66%

Test F1 Scores (Per Class): [84.61538 80.64516 54.726364 49.71098 33.96226 69.23077 61.290314

67.77777	78.82352	84.02366	68.15642	51.685387	71.18643	64.32748
69.565216	63.387978	78.75647	78.53403	63.101597	63.803684	82.35293
80.66298	80.87432	78.21229	82.41757	70.85714	54.23729	51.456303
81.56424	68.263466	64.088394	73.863625	52.1739	58.3732	67.66169
39.999996	73.79678	59.7826	55.865913	90.90909	70.89946	83.060104
67.05882	71.428566	48.128338	51.282047	49.71751	65.14285	86.17021
82.872925	41.290318	74.28571	63.054188	92.13482	70.35175	39.50617
80.92485	75.428566	82.142845	54.999996	82.02247	76.13636	64.83515
64.088394	48.235287	58.479523	63.276833	60.465115	89.61748	86.48647
73.56321	77.7202	46.66666	64.40677	36.548218	80.20833	86.81318
61.714287	72.51461	68.13186	42.553185	56.80473	90.71037	74.46808
64.59627	73.0337	81.35593	80.446915	63.33333	63.21243	62.5
68.8172	55.95237	64.63414	85.86956	70.40816	56.66666	60.10362
41.758232	77.77777					

Average Test F1 Scores:67.52

282/282 ————— 11s 28ms/step



	precision	recall	f1-score	support
beaver	0.84	0.86	0.85	90
dolphin	0.78	0.83	0.81	90
otter	0.50	0.61	0.55	90
seal	0.52	0.48	0.50	90
whale	0.39	0.30	0.34	90
aquarium fish	0.68	0.70	0.69	90
flatfish	0.59	0.63	0.61	90
ray	0.68	0.68	0.68	90
shark	0.84	0.74	0.79	90
trout	0.90	0.79	0.84	90
orchids	0.69	0.68	0.68	90
poppies	0.52	0.51	0.52	90
roses	0.72	0.70	0.71	90
sunflowers	0.68	0.61	0.64	90
tulips	0.79	0.62	0.70	90
bottles	0.62	0.64	0.63	90
bowls	0.74	0.84	0.79	90
cans	0.74	0.83	0.79	90
cups	0.61	0.66	0.63	90
plates	0.71	0.58	0.64	90
apples	0.79	0.86	0.82	90
mushrooms	0.80	0.81	0.81	90
oranges	0.80	0.82	0.81	90
pears	0.79	0.78	0.78	90
sweet peppers	0.82	0.83	0.82	90
clock	0.73	0.69	0.71	90
computer keyboard	0.55	0.53	0.54	90
lamp	0.46	0.59	0.51	90
telephone	0.82	0.81	0.82	90
television	0.74	0.63	0.68	90
bed	0.64	0.64	0.64	90
chair	0.76	0.72	0.74	90
couch	0.59	0.47	0.52	90
table	0.51	0.68	0.58	90
wardrobe	0.61	0.76	0.68	90
bee	0.44	0.37	0.40	90
beetle	0.71	0.77	0.74	90
butterfly	0.59	0.61	0.60	90
caterpillar	0.56	0.56	0.56	90
cockroach	0.93	0.89	0.91	90
bear	0.68	0.74	0.71	90
leopard	0.82	0.84	0.83	90
lion	0.71	0.63	0.67	90
tiger	0.71	0.72	0.71	90
wolf	0.46	0.50	0.48	90
bridge	0.47	0.56	0.51	90
castle	0.51	0.49	0.50	90
house	0.67	0.63	0.65	90
road	0.83	0.90	0.86	90
skyscraper	0.82	0.83	0.83	90
cloud	0.49	0.36	0.41	90
forest	0.76	0.72	0.74	90
mountain	0.57	0.71	0.63	90
plain	0.93	0.91	0.92	90
sea	0.64	0.78	0.70	90
camel	0.44	0.36	0.40	90
cattle	0.84	0.78	0.81	90
chimpanzee	0.78	0.73	0.75	90

elephant	0.88	0.77	0.82	90
kangaroo	0.63	0.49	0.55	90
fox	0.83	0.81	0.82	90
porcupine	0.78	0.74	0.76	90
possum	0.64	0.66	0.65	90
raccoon	0.64	0.64	0.64	90
skunk	0.51	0.46	0.48	90
crab	0.62	0.56	0.58	90
lobster	0.64	0.62	0.63	90
snail	0.63	0.58	0.60	90
spider	0.88	0.91	0.90	90
worm	0.84	0.89	0.86	90
baby	0.76	0.71	0.74	90
boy	0.73	0.83	0.78	90
girl	0.47	0.47	0.47	90
man	0.66	0.63	0.64	90
woman	0.34	0.40	0.37	90
crocodile	0.75	0.86	0.80	90
dinosaur	0.86	0.88	0.87	90
lizard	0.63	0.59	0.61	90
snake	0.77	0.69	0.73	90
turtle	0.67	0.69	0.68	90
hamster	0.41	0.44	0.43	90
mouse	0.61	0.53	0.57	90
rabbit	0.89	0.92	0.91	90
shrew	0.71	0.78	0.74	90
squirrel	0.73	0.58	0.65	90
maple	0.74	0.72	0.73	90
oak	0.83	0.80	0.81	90
palm	0.81	0.80	0.80	90
pine	0.63	0.63	0.63	90
willow	0.59	0.68	0.63	90
bicycle	0.59	0.67	0.62	90
bus	0.67	0.71	0.69	90
motorcycle	0.60	0.52	0.56	90
pickup truck	0.72	0.59	0.65	90
train	0.84	0.88	0.86	90
lawn-mower	0.65	0.77	0.70	90
rocket	0.57	0.57	0.57	90
streetcar	0.56	0.64	0.60	90
tank	0.41	0.42	0.42	90
tractor	0.78	0.78	0.78	90
accuracy			0.68	9000
macro avg	0.68	0.68	0.68	9000
weighted avg	0.68	0.68	0.68	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

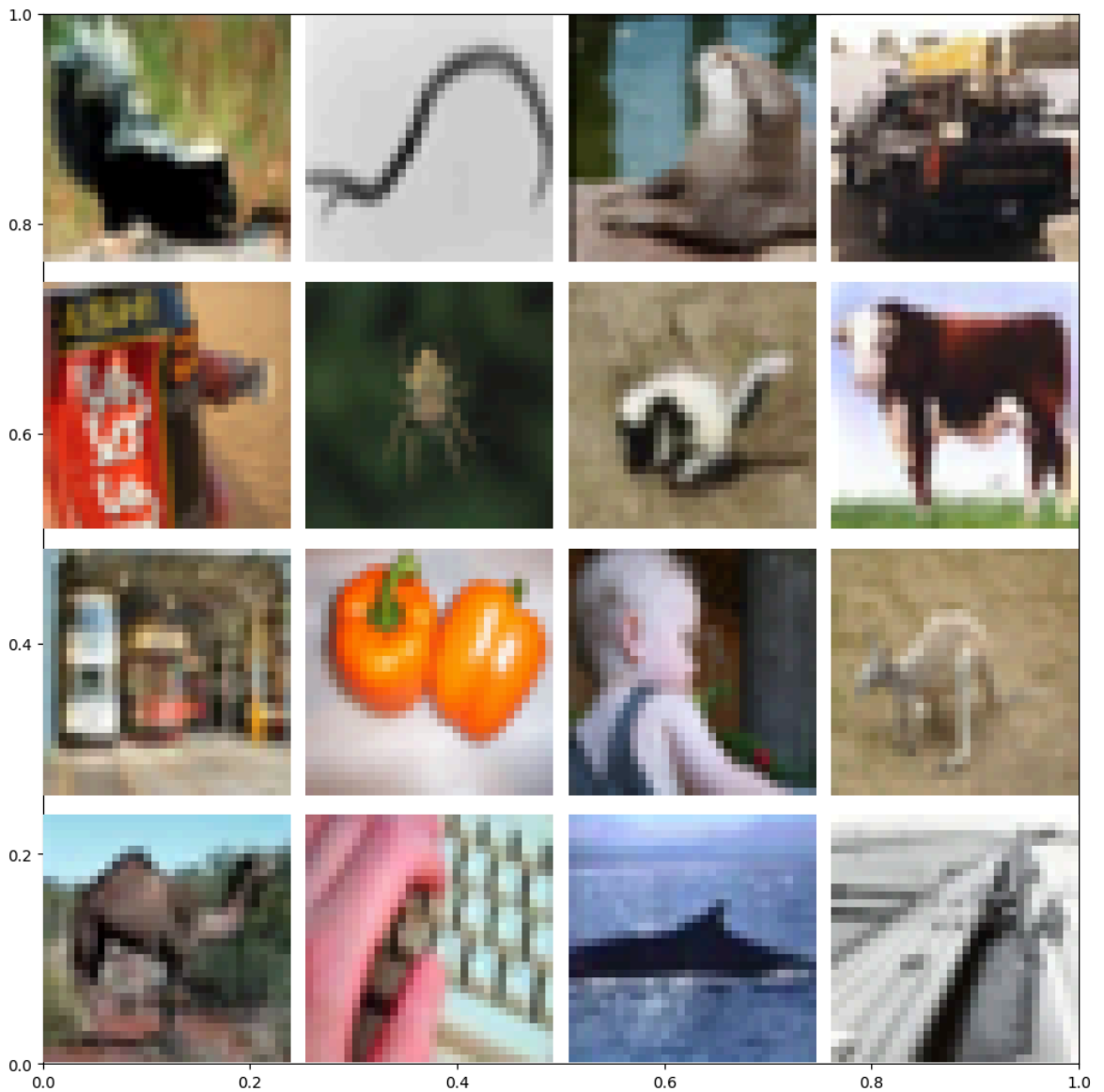
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

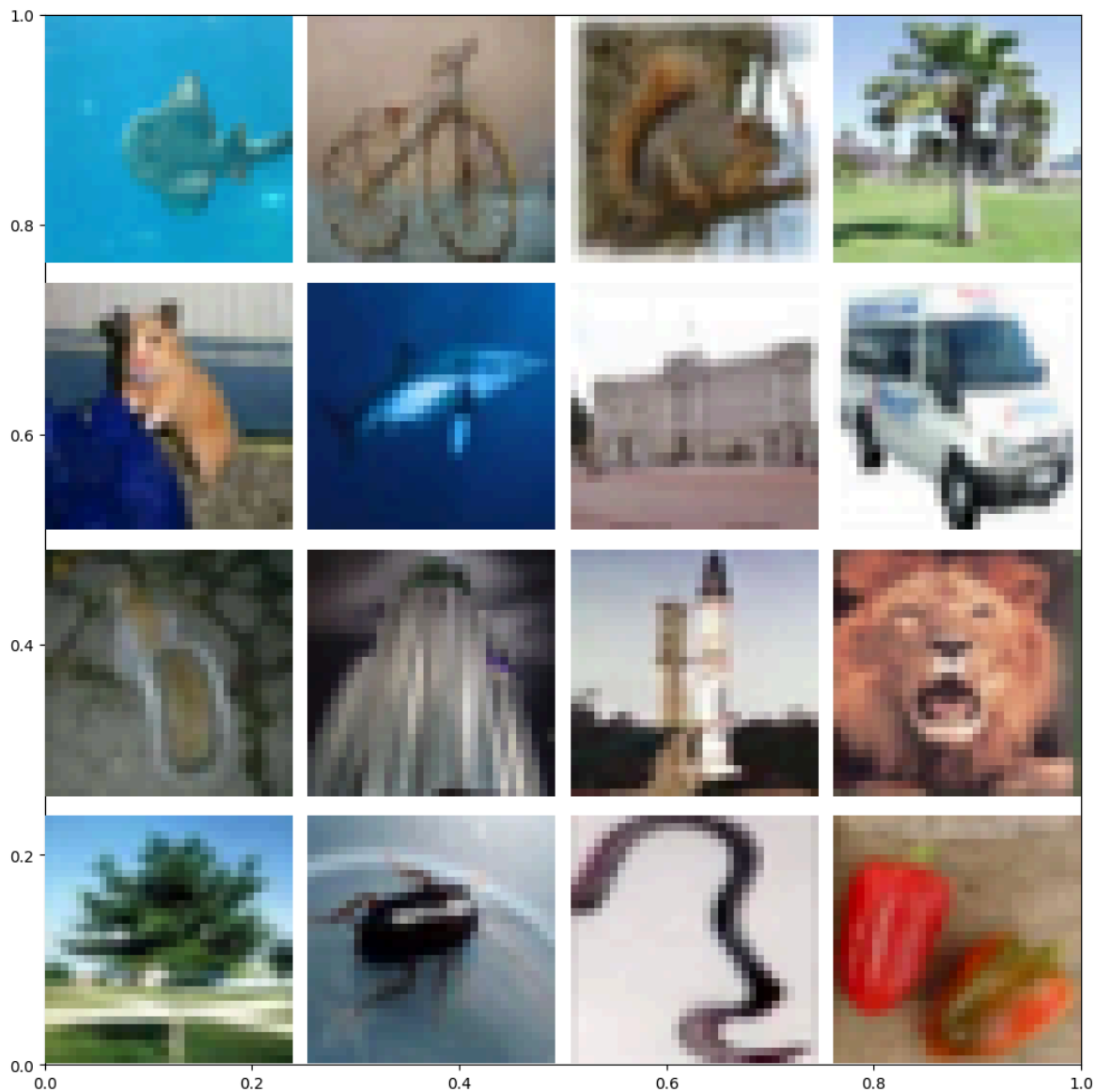
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[22009].shape: (1,)  
imgs[22009].shape: (32, 32, 3)
```

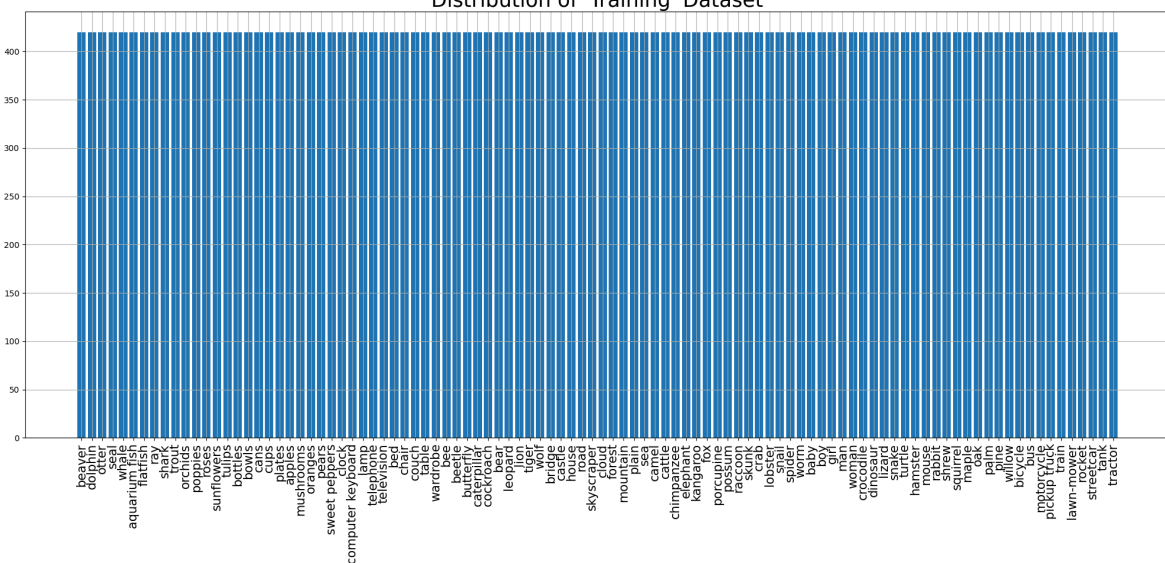


```
labels[13746].shape: (1,)  
imgs[13746].shape: (32, 32, 3)
```

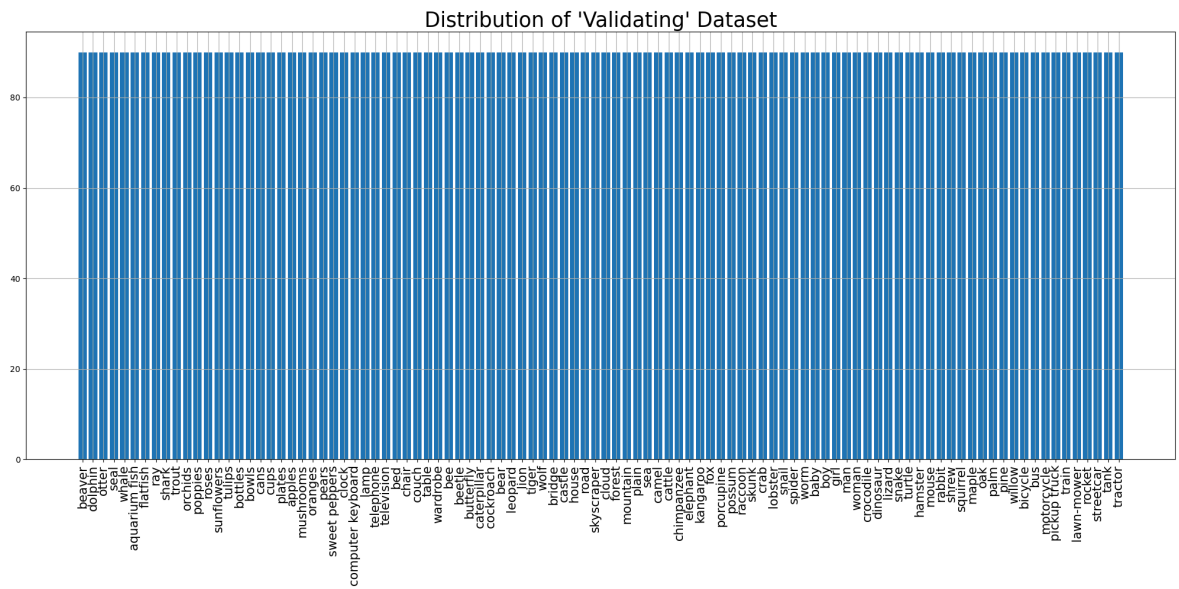


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

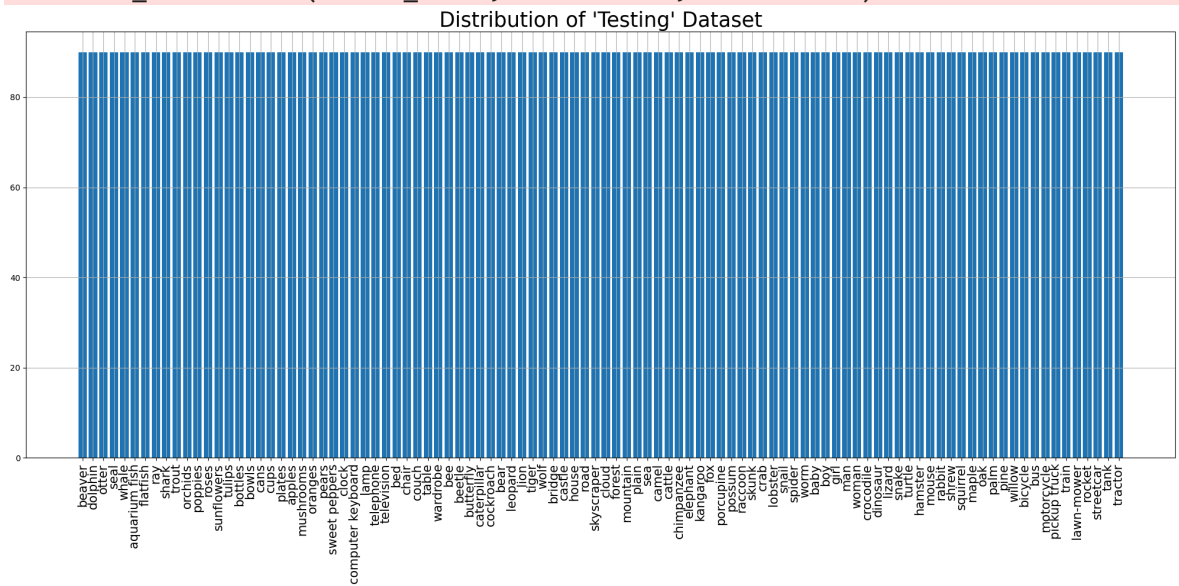
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_15"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_15 (GlobalAveragePooling2D)	(None , 2048)
dense_30 (Dense)	(None , 512)
dropout_15 (Dropout)	(None , 512)
dense_31 (Dense)	(None , 100)

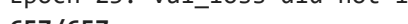
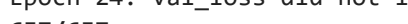
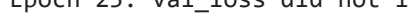
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

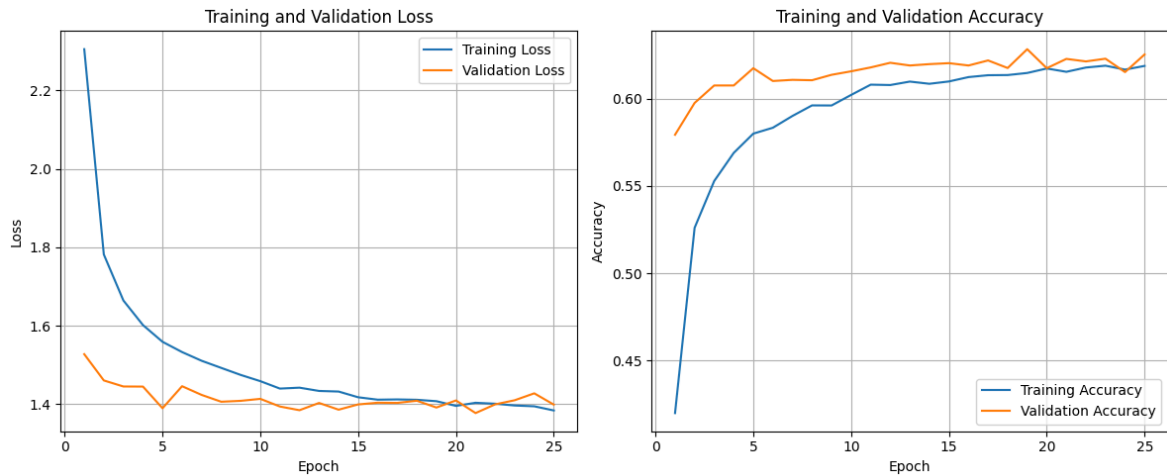
Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 29ms/step - accuracy: 0.3270 - f1_score: 0.3211 - loss: 2.8500 - precision: 0.6538
Epoch 1: val_loss improved from inf to 1.52743, saving model to best_model_v9.keras
657/657 ————— 40s 45ms/step - accuracy: 0.3271 - f1_score: 0.3212 - loss: 2.8492 - precision: 0.6539 - val_accuracy: 0.5793 - val_f1_score: 0.5758 - val_loss: 1.5274 - val_precision: 0.7808
Epoch 2/25
657/657 ————— 0s 24ms/step - accuracy: 0.5140 - f1_score: 0.5101 - loss: 1.8286 - precision: 0.7110
Epoch 2: val_loss improved from 1.52743 to 1.46049, saving model to best_model_v9.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5140 - f1_score: 0.5101 - loss: 1.8286 - precision: 0.7110 - val_accuracy: 0.5976 - val_f1_score: 0.5916 - val_loss: 1.4605 - val_precision: 0.7771
Epoch 3/25
657/657 ————— 0s 24ms/step - accuracy: 0.5469 - f1_score: 0.5432 - loss: 1.6887 - precision: 0.7303
Epoch 3: val_loss improved from 1.46049 to 1.44513, saving model to best_model_v9.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5469 - f1_score: 0.5432 - loss: 1.6886 - precision: 0.7303 - val_accuracy: 0.6076 - val_f1_score: 0.6021 - val_loss: 1.4451 - val_precision: 0.7702
Epoch 4/25
657/657 ————— 0s 24ms/step - accuracy: 0.5638 - f1_score: 0.5601 - loss: 1.6177 - precision: 0.7331
Epoch 4: val_loss improved from 1.44513 to 1.44486, saving model to best_model_v9.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5638 - f1_score: 0.5601 - loss: 1.6177 - precision: 0.7331 - val_accuracy: 0.6076 - val_f1_score: 0.6049 - val_loss: 1.4449 - val_precision: 0.7705
Epoch 5/25
655/657 ————— 0s 24ms/step - accuracy: 0.5774 - f1_score: 0.5744 - loss: 1.5770 - precision: 0.7407
Epoch 5: val_loss improved from 1.44486 to 1.38967, saving model to best_model_v9.keras
657/657 ————— 20s 30ms/step - accuracy: 0.5774 - f1_score: 0.5744 - loss: 1.5769 - precision: 0.7407 - val_accuracy: 0.6174 - val_f1_score: 0.6133 - val_loss: 1.3897 - val_precision: 0.7839
Epoch 6/25
656/657 ————— 0s 24ms/step - accuracy: 0.5785 - f1_score: 0.5751 - loss: 1.5477 - precision: 0.7407
Epoch 6: val_loss did not improve from 1.38967
657/657 ————— 19s 29ms/step - accuracy: 0.5785 - f1_score: 0.5751 - loss: 1.5476 - precision: 0.7407 - val_accuracy: 0.6101 - val_f1_score: 0.6068 - val_loss: 1.4458 - val_precision: 0.7644
Epoch 7/25
657/657 ————— 0s 24ms/step - accuracy: 0.5848 - f1_score: 0.5815 - loss: 1.5260 - precision: 0.7449
Epoch 7: val_loss did not improve from 1.38967
657/657 ————— 19s 29ms/step - accuracy: 0.5848 - f1_score: 0.5815 - loss: 1.5260 - precision: 0.7449 - val_accuracy: 0.6108 - val_f1_score: 0.6081 - val_loss: 1.4236 - val_precision: 0.7668
Epoch 8/25
657/657 ————— 0s 24ms/step - accuracy: 0.5862 - f1_score: 0.5827 - loss: 1.5205 - precision: 0.7455
Epoch 8: val_loss did not improve from 1.38967
657/657 ————— 19s 29ms/step - accuracy: 0.5862 - f1_score: 0.5827 - loss: 1.5204 - precision: 0.7455 - val_accuracy: 0.6106 - val_f1_score: 0.6078

- val_loss: 1.4062 - val_precision: 0.7729
Epoch 9/25
657/657 ————— 0s 24ms/step - accuracy: 0.5925 - f1_score: 0.5894 -
loss: 1.4924 - precision: 0.7491
Epoch 9: val_loss did not improve from 1.38967
657/657 ————— 19s 29ms/step - accuracy: 0.5925 - f1_score: 0.5894
- loss: 1.4923 - precision: 0.7491 - val_accuracy: 0.6137 - val_f1_score: 0.6121
- val_loss: 1.4086 - val_precision: 0.7705
Epoch 10/25
655/657 ————— 0s 24ms/step - accuracy: 0.5990 - f1_score: 0.5961 -
loss: 1.4682 - precision: 0.7580
Epoch 10: val_loss did not improve from 1.38967
657/657 ————— 19s 29ms/step - accuracy: 0.5990 - f1_score: 0.5961
- loss: 1.4682 - precision: 0.7580 - val_accuracy: 0.6157 - val_f1_score: 0.6133
- val_loss: 1.4136 - val_precision: 0.7725
Epoch 11/25
656/657 ————— 0s 24ms/step - accuracy: 0.6049 - f1_score: 0.6017 -
loss: 1.4585 - precision: 0.7579
Epoch 11: val_loss did not improve from 1.38967
657/657 ————— 19s 29ms/step - accuracy: 0.6049 - f1_score: 0.6017
- loss: 1.4585 - precision: 0.7579 - val_accuracy: 0.6179 - val_f1_score: 0.6173
- val_loss: 1.3941 - val_precision: 0.7708
Epoch 12/25
657/657 ————— 0s 24ms/step - accuracy: 0.6021 - f1_score: 0.5991 -
loss: 1.4662 - precision: 0.7578
Epoch 12: val_loss improved from 1.38967 to 1.38465, saving model to best_model_v
9.keras
657/657 ————— 20s 30ms/step - accuracy: 0.6021 - f1_score: 0.5991
- loss: 1.4661 - precision: 0.7578 - val_accuracy: 0.6206 - val_f1_score: 0.6177
- val_loss: 1.3847 - val_precision: 0.7824
Epoch 13/25
657/657 ————— 0s 24ms/step - accuracy: 0.6025 - f1_score: 0.6000 -
loss: 1.4558 - precision: 0.7541
Epoch 13: val_loss did not improve from 1.38465
657/657 ————— 19s 29ms/step - accuracy: 0.6025 - f1_score: 0.6000
- loss: 1.4558 - precision: 0.7541 - val_accuracy: 0.6190 - val_f1_score: 0.6154
- val_loss: 1.4031 - val_precision: 0.7687
Epoch 14/25
657/657 ————— 0s 24ms/step - accuracy: 0.6035 - f1_score: 0.6005 -
loss: 1.4510 - precision: 0.7565
Epoch 14: val_loss did not improve from 1.38465
657/657 ————— 19s 29ms/step - accuracy: 0.6035 - f1_score: 0.6006
- loss: 1.4509 - precision: 0.7565 - val_accuracy: 0.6198 - val_f1_score: 0.6183
- val_loss: 1.3860 - val_precision: 0.7816
Epoch 15/25
655/657 ————— 0s 24ms/step - accuracy: 0.6063 - f1_score: 0.6031 -
loss: 1.4262 - precision: 0.7569
Epoch 15: val_loss did not improve from 1.38465
657/657 ————— 19s 29ms/step - accuracy: 0.6063 - f1_score: 0.6032
- loss: 1.4262 - precision: 0.7569 - val_accuracy: 0.6203 - val_f1_score: 0.6165
- val_loss: 1.3994 - val_precision: 0.7702
Epoch 16/25
656/657 ————— 0s 24ms/step - accuracy: 0.6082 - f1_score: 0.6054 -
loss: 1.4253 - precision: 0.7595
Epoch 16: val_loss did not improve from 1.38465
657/657 ————— 19s 29ms/step - accuracy: 0.6082 - f1_score: 0.6054
- loss: 1.4252 - precision: 0.7595 - val_accuracy: 0.6190 - val_f1_score: 0.6169
- val_loss: 1.4038 - val_precision: 0.7757
Epoch 17/25
655/657 ————— 0s 24ms/step - accuracy: 0.6058 - f1_score: 0.6022 -

loss: 1.4355 - precision: 0.7595
Epoch 17: val_loss did not improve from 1.38465
657/657  **19s** 29ms/step - accuracy: 0.6059 - f1_score: 0.6022
- loss: 1.4354 - precision: 0.7595 - val_accuracy: 0.6219 - val_f1_score: 0.6200
- val_loss: 1.4033 - val_precision: 0.7601
Epoch 18/25
655/657  **0s** 24ms/step - accuracy: 0.6077 - f1_score: 0.6043 -
loss: 1.4372 - precision: 0.7584
Epoch 18: val_loss did not improve from 1.38465
657/657  **19s** 29ms/step - accuracy: 0.6078 - f1_score: 0.6044
- loss: 1.4371 - precision: 0.7585 - val_accuracy: 0.6176 - val_f1_score: 0.6148
- val_loss: 1.4088 - val_precision: 0.7662
Epoch 19/25
657/657  **0s** 24ms/step - accuracy: 0.6104 - f1_score: 0.6073 -
loss: 1.4304 - precision: 0.7581
Epoch 19: val_loss did not improve from 1.38465
657/657  **19s** 29ms/step - accuracy: 0.6105 - f1_score: 0.6073
- loss: 1.4303 - precision: 0.7581 - val_accuracy: 0.6283 - val_f1_score: 0.6251
- val_loss: 1.3915 - val_precision: 0.7747
Epoch 20/25
657/657  **0s** 24ms/step - accuracy: 0.6166 - f1_score: 0.6137 -
loss: 1.4084 - precision: 0.7654
Epoch 20: val_loss did not improve from 1.38465
657/657  **19s** 29ms/step - accuracy: 0.6166 - f1_score: 0.6137
- loss: 1.4084 - precision: 0.7654 - val_accuracy: 0.6174 - val_f1_score: 0.6164
- val_loss: 1.4095 - val_precision: 0.7575
Epoch 21/25
655/657  **0s** 24ms/step - accuracy: 0.6150 - f1_score: 0.6125 -
loss: 1.4162 - precision: 0.7604
Epoch 21: val_loss improved from 1.38465 to 1.37735, saving model to best_model_v
9.keras
657/657  **20s** 30ms/step - accuracy: 0.6150 - f1_score: 0.6125
- loss: 1.4162 - precision: 0.7604 - val_accuracy: 0.6228 - val_f1_score: 0.6189
- val_loss: 1.3773 - val_precision: 0.7791
Epoch 22/25
657/657  **0s** 24ms/step - accuracy: 0.6160 - f1_score: 0.6133 -
loss: 1.4103 - precision: 0.7632
Epoch 22: val_loss did not improve from 1.37735
657/657  **19s** 29ms/step - accuracy: 0.6160 - f1_score: 0.6133
- loss: 1.4103 - precision: 0.7632 - val_accuracy: 0.6213 - val_f1_score: 0.6188
- val_loss: 1.3995 - val_precision: 0.7762
Epoch 23/25
656/657  **0s** 24ms/step - accuracy: 0.6134 - f1_score: 0.6103 -
loss: 1.4135 - precision: 0.7623
Epoch 23: val_loss did not improve from 1.37735
657/657  **19s** 29ms/step - accuracy: 0.6134 - f1_score: 0.6103
- loss: 1.4134 - precision: 0.7623 - val_accuracy: 0.6229 - val_f1_score: 0.6198
- val_loss: 1.4100 - val_precision: 0.7665
Epoch 24/25
655/657  **0s** 24ms/step - accuracy: 0.6131 - f1_score: 0.6106 -
loss: 1.4125 - precision: 0.7624
Epoch 24: val_loss did not improve from 1.37735
657/657  **19s** 29ms/step - accuracy: 0.6132 - f1_score: 0.6106
- loss: 1.4125 - precision: 0.7624 - val_accuracy: 0.6152 - val_f1_score: 0.6126
- val_loss: 1.4277 - val_precision: 0.7631
Epoch 25/25
655/657  **0s** 24ms/step - accuracy: 0.6138 - f1_score: 0.6106 -
loss: 1.4039 - precision: 0.7628
Epoch 25: val_loss did not improve from 1.37735
657/657  **19s** 29ms/step - accuracy: 0.6138 - f1_score: 0.6106

- loss: 1.4038 - precision: 0.7628 - val_accuracy: 0.6253 - val_f1_score: 0.6245
 - val_loss: 1.3989 - val_precision: 0.7669



141/141 ————— **11s** 38ms/step - accuracy: 0.6128 - f1_score: 0.6027

- loss: 1.4039 - precision: 0.7709

Test Accuracy: 62.59%

Test Loss: 1.3842073678970337

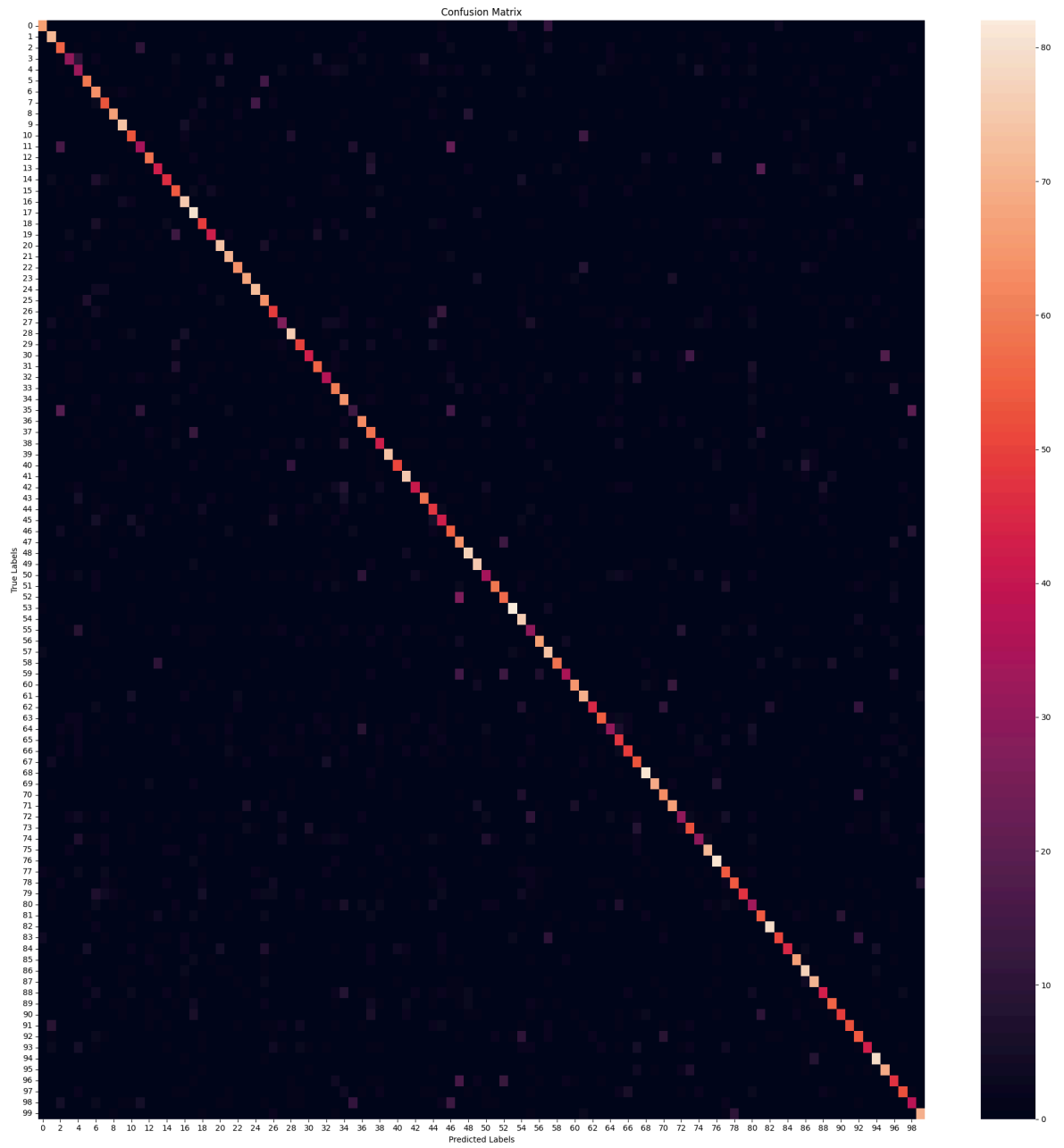
Test Precision: 78.10%

Test F1 Scores (Per Class): [79.51807 73.09644 54.63414 42.253517 32.820507 6
6.666664 54.468082

59.886997	77.456635	78.723404	58.56353	42.76729	67.05202	54.54545
60.52631	52.941174	76.53061	71.74887	53.551907	55.26315	79.569885
72.82051	75.73964	77.27273	78.49462	60.6635	53.260864	36.363632
75.99999	57.80347	58.904106	64.	45.398766	57.692303	56.768555
22.047241	64.61538	53.95348	50.602406	87.57396	63.75	75.75757
56.16438	69.047615	49.48453	43.29896	50.925915	54.468082	80.20833
77.31957	41.717785	71.95121	60.63829	86.31578	65.23605	33.526005
73.33333	67.8899	74.35896	49.64539	76.47058	68.965515	62.068962
66.66666	43.165462	49.230762	59.756096	56.989246	83.07691	81.17647
68.8172	72.43243	38.961033	59.428566	38.66666	78.68852	76.056335
55.958546	64.28571	61.437904	32.835815	56.544495	88.26815	65.80644
62.068962	74.03315	70.96774	74.73684	55.128193	63.276833	61.25
65.4088	51.923065	57.718117	82.7225	70.05075	49.47368	64.59627
44.705875	77.77777					

Average Test F1 Scores:62.19

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.87	0.73	0.80	90
dolphin	0.67	0.80	0.73	90
otter	0.49	0.62	0.55	90
seal	0.58	0.33	0.42	90
whale	0.30	0.36	0.33	90
aquarium fish	0.68	0.66	0.67	90
flatfish	0.44	0.71	0.55	90
ray	0.60	0.59	0.60	90
shark	0.81	0.74	0.77	90
trout	0.76	0.82	0.79	90
orchids	0.58	0.59	0.59	90
poppies	0.50	0.38	0.43	90
roses	0.70	0.64	0.67	90
sunflowers	0.66	0.47	0.55	90
tulips	0.74	0.51	0.61	90
bottles	0.47	0.60	0.53	90
bowls	0.71	0.83	0.77	90
cans	0.60	0.89	0.72	90
cups	0.53	0.54	0.54	90
plates	0.68	0.47	0.55	90
apples	0.77	0.82	0.80	90
mushrooms	0.68	0.79	0.73	90
oranges	0.81	0.71	0.76	90
pears	0.79	0.76	0.77	90
sweet peppers	0.76	0.81	0.78	90
clock	0.53	0.71	0.61	90
computer keyboard	0.52	0.54	0.53	90
lamp	0.49	0.29	0.36	90
telephone	0.69	0.84	0.76	90
television	0.60	0.56	0.58	90
bed	0.77	0.48	0.59	90
chair	0.66	0.62	0.64	90
couch	0.51	0.41	0.45	90
table	0.50	0.67	0.57	90
wardrobe	0.47	0.72	0.57	90
bee	0.38	0.16	0.22	90
beetle	0.60	0.70	0.65	90
butterfly	0.47	0.64	0.54	90
caterpillar	0.55	0.47	0.51	90
cockroach	0.94	0.82	0.88	90
bear	0.73	0.57	0.64	90
leopard	0.69	0.83	0.76	90
lion	0.73	0.46	0.56	90
tiger	0.74	0.64	0.69	90
wolf	0.46	0.53	0.49	90
bridge	0.40	0.47	0.43	90
castle	0.43	0.61	0.51	90
house	0.44	0.71	0.54	90
road	0.75	0.86	0.80	90
skyscraper	0.72	0.83	0.77	90
cloud	0.47	0.38	0.42	90
forest	0.80	0.66	0.72	90
mountain	0.58	0.63	0.61	90
plain	0.82	0.91	0.86	90
sea	0.53	0.84	0.65	90
camel	0.35	0.32	0.34	90
cattle	0.73	0.73	0.73	90
chimpanzee	0.58	0.82	0.68	90

elephant	0.88	0.64	0.74	90
kangaroo	0.69	0.39	0.50	90
fox	0.81	0.72	0.76	90
porcupine	0.62	0.78	0.69	90
possum	0.82	0.50	0.62	90
raccoon	0.73	0.61	0.67	90
skunk	0.61	0.33	0.43	90
crab	0.46	0.53	0.49	90
lobster	0.66	0.54	0.60	90
snail	0.55	0.59	0.57	90
spider	0.77	0.90	0.83	90
worm	0.86	0.77	0.81	90
baby	0.66	0.70	0.68	90
boy	0.71	0.74	0.72	90
girl	0.47	0.33	0.39	90
man	0.61	0.58	0.59	90
woman	0.48	0.32	0.39	90
crocodile	0.77	0.80	0.79	90
dinosaur	0.66	0.90	0.76	90
lizard	0.52	0.60	0.56	90
snake	0.69	0.60	0.64	90
turtle	0.75	0.52	0.61	90
hamster	0.30	0.37	0.33	90
mouse	0.53	0.60	0.57	90
rabbit	0.89	0.88	0.88	90
shrew	0.78	0.57	0.66	90
squirrel	0.82	0.50	0.62	90
maple	0.74	0.74	0.74	90
oak	0.61	0.86	0.71	90
palm	0.71	0.79	0.75	90
pine	0.65	0.48	0.55	90
willow	0.64	0.62	0.63	90
bicycle	0.70	0.54	0.61	90
bus	0.75	0.58	0.65	90
motorcycle	0.45	0.60	0.52	90
pickup truck	0.73	0.48	0.58	90
train	0.78	0.88	0.83	90
lawn-mower	0.64	0.77	0.70	90
rocket	0.47	0.52	0.49	90
streetcar	0.73	0.58	0.65	90
tank	0.47	0.42	0.45	90
tractor	0.78	0.78	0.78	90
accuracy			0.63	9000
macro avg	0.64	0.63	0.62	9000
weighted avg	0.64	0.63	0.62	9000

Model: "sequential_15"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_15 (GlobalAveragePooling2D)	(None , 2048)
dense_30 (Dense)	(None , 512)
dropout_15 (Dropout)	(None , 512)
dense_31 (Dense)	(None , 100)



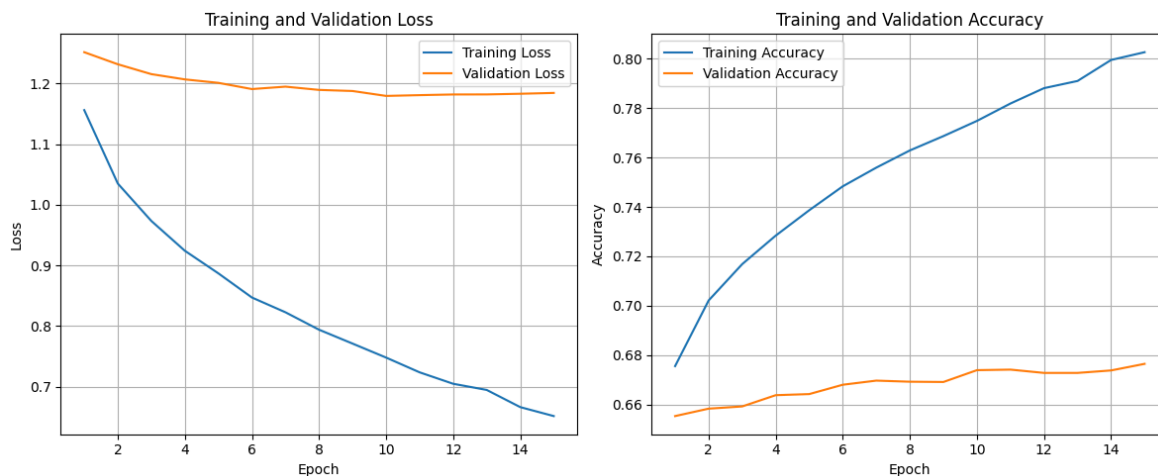
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 26ms/step - accuracy: 0.6648 - f1_score: 0.6630
- loss: 1.1979 - precision: 0.8107
Epoch 1: val_loss improved from inf to 1.25180, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.6649 - f1_score: 0.6630 - loss: 1.1978 - precision: 0.8107 - val_accuracy: 0.6554 - val_f1_score: 0.6545 - val_loss: 1.2518 - val_precision: 0.8018
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7034 - f1_score: 0.7018 - loss: 1.0265 - precision: 0.8394
Epoch 2: val_loss improved from 1.25180 to 1.23189, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7034 - f1_score: 0.7018 - loss: 1.0265 - precision: 0.8394 - val_accuracy: 0.6584 - val_f1_score: 0.6574 - val_loss: 1.2319 - val_precision: 0.8044
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7207 - f1_score: 0.7191 - loss: 0.9537 - precision: 0.8550
Epoch 3: val_loss improved from 1.23189 to 1.21559, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7207 - f1_score: 0.7191 - loss: 0.9538 - precision: 0.8550 - val_accuracy: 0.6593 - val_f1_score: 0.6584 - val_loss: 1.2156 - val_precision: 0.8051
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7359 - f1_score: 0.7344 - loss: 0.8987 - precision: 0.8625
Epoch 4: val_loss improved from 1.21559 to 1.20692, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7359 - f1_score: 0.7344 - loss: 0.8988 - precision: 0.8625 - val_accuracy: 0.6639 - val_f1_score: 0.6632 - val_loss: 1.2069 - val_precision: 0.8060
Epoch 5/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7468 - f1_score: 0.7455 - loss: 0.8503 - precision: 0.8702
Epoch 5: val_loss improved from 1.20692 to 1.20119, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7467 - f1_score: 0.7455 - loss: 0.8503 - precision: 0.8701 - val_accuracy: 0.6643 - val_f1_score: 0.6631 - val_loss: 1.2012 - val_precision: 0.8026
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7552 - f1_score: 0.7544 - loss: 0.8125 - precision: 0.8750
Epoch 6: val_loss improved from 1.20119 to 1.19096, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7552 - f1_score: 0.7544 - loss: 0.8126 - precision: 0.8750 - val_accuracy: 0.6681 - val_f1_score: 0.6670 - val_loss: 1.1910 - val_precision: 0.8069
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7674 - f1_score: 0.7663 - loss: 0.7790 - precision: 0.8825
Epoch 7: val_loss did not improve from 1.19096
1314/1314 ————— 35s 27ms/step - accuracy: 0.7673 - f1_score: 0.7663 - loss: 0.7791 - precision: 0.8825 - val_accuracy: 0.6698 - val_f1_score: 0.6688 - val_loss: 1.1950 - val_precision: 0.8031
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7743 - f1_score: 0.7728 - loss: 0.7537 - precision: 0.8846
Epoch 8: val_loss improved from 1.19096 to 1.18957, saving model to best_model_fine_tune_v9.keras

1314/1314 ————— **36s** 27ms/step - accuracy: 0.7743 - f1_score: 0.7728 - loss: 0.7537 - precision: 0.8846 - val_accuracy: 0.6693 - val_f1_score: 0.6682 - val_loss: 1.1896 - val_precision: 0.8036
Epoch 9/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.7800 - f1_score: 0.7789 - loss: 0.7289 - precision: 0.8897
Epoch 9: val_loss improved from 1.18957 to 1.18775, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.7800 - f1_score: 0.7788 - loss: 0.7290 - precision: 0.8897 - val_accuracy: 0.6692 - val_f1_score: 0.6676 - val_loss: 1.1878 - val_precision: 0.8048
Epoch 10/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.7900 - f1_score: 0.7888 - loss: 0.6961 - precision: 0.8968
Epoch 10: val_loss improved from 1.18775 to 1.17966, saving model to best_model_fine_tune_v9.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.7900 - f1_score: 0.7888 - loss: 0.6962 - precision: 0.8968 - val_accuracy: 0.6740 - val_f1_score: 0.6732 - val_loss: 1.1797 - val_precision: 0.8056
Epoch 11/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.7959 - f1_score: 0.7949 - loss: 0.6763 - precision: 0.8975
Epoch 11: val_loss did not improve from 1.17966
1314/1314 ————— **35s** 27ms/step - accuracy: 0.7959 - f1_score: 0.7949 - loss: 0.6764 - precision: 0.8975 - val_accuracy: 0.6742 - val_f1_score: 0.6730 - val_loss: 1.1809 - val_precision: 0.8060
Epoch 12/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8016 - f1_score: 0.8008 - loss: 0.6553 - precision: 0.8990
Epoch 12: val_loss did not improve from 1.17966
1314/1314 ————— **35s** 26ms/step - accuracy: 0.8016 - f1_score: 0.8007 - loss: 0.6554 - precision: 0.8990 - val_accuracy: 0.6729 - val_f1_score: 0.6721 - val_loss: 1.1820 - val_precision: 0.8024
Epoch 13/15
1312/1314 ————— **0s** 24ms/step - accuracy: 0.8039 - f1_score: 0.8027 - loss: 0.6443 - precision: 0.9008
Epoch 13: val_loss did not improve from 1.17966
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8039 - f1_score: 0.8027 - loss: 0.6444 - precision: 0.9008 - val_accuracy: 0.6729 - val_f1_score: 0.6719 - val_loss: 1.1821 - val_precision: 0.8025
Epoch 14/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8144 - f1_score: 0.8134 - loss: 0.6155 - precision: 0.9069
Epoch 14: val_loss did not improve from 1.17966
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8144 - f1_score: 0.8133 - loss: 0.6156 - precision: 0.9068 - val_accuracy: 0.6739 - val_f1_score: 0.6726 - val_loss: 1.1832 - val_precision: 0.7982
Epoch 15/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.8170 - f1_score: 0.8161 - loss: 0.6013 - precision: 0.9069
Epoch 15: val_loss did not improve from 1.17966
1314/1314 ————— **34s** 26ms/step - accuracy: 0.8170 - f1_score: 0.8161 - loss: 0.6013 - precision: 0.9069 - val_accuracy: 0.6766 - val_f1_score: 0.6751 - val_loss: 1.1846 - val_precision: 0.7980



141/141 ————— **11s** 38ms/step - accuracy: 0.6669 - f1_score: 0.6581

- loss: 1.2012 - precision: 0.7986

Test Accuracy: 67.66%

Test Loss: 1.17820405960083

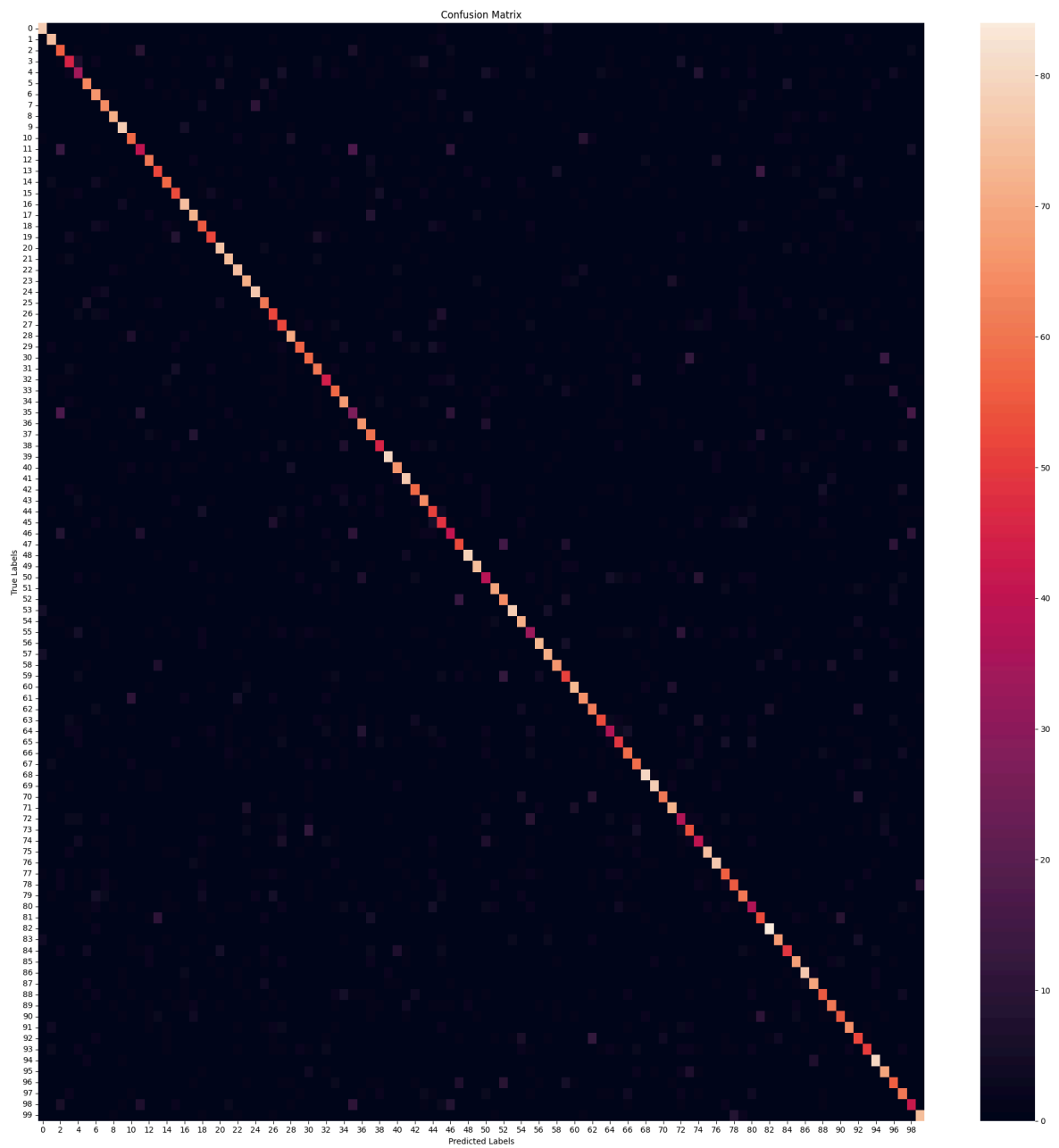
Test Precision: 80.73%

Test F1 Scores (Per Class): [84.7826 80.64516 55.17241 52.941174 38.857136 7
1.99999 67.34693

71.1111	81.60919	84.32432	63.387978	48.78048	68.926544	60.714275
69.047615	60.571426	81.08108	77.41935	60.773476	63.030296	81.72042
80.43478	79.787224	77.173904	83.69564	67.77777	56.521736	53.60824
79.54544	68.263466	64.088394	68.53932	53.012043	61.052628	70.526306
32.72727	70.967735	63.492058	54.21686	90.39548	69.84126	82.7957
63.387978	72.72727	53.124992	50.261772	46.06741	63.09523	86.33879
83.615814	43.33333	76.66666	66.666664	88.63635	73.57513	39.28571
83.42856	74.73684	79.04191	56.66666	82.02247	74.71264	67.3913
66.66666	47.43589	57.309933	65.92178	65.19337	87.43169	85.71427
71.345024	78.260864	41.57303	59.66851	44.44444	82.872925	83.24323
61.87845	66.26506	65.94594	41.340775	57.92349	90.81081	74.03315
59.036137	72.34043	83.69564	77.09496	59.7826	66.66666	61.111103
72.222206	56.216217	60.24096	86.02149	73.79678	56.852787	63.87434
47.999996	80.87432	]				

Average Test F1 Scores:67.51

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.83	0.87	0.85	90
dolphin	0.78	0.83	0.81	90
otter	0.50	0.62	0.55	90
seal	0.56	0.50	0.53	90
whale	0.40	0.38	0.39	90
aquarium fish	0.74	0.70	0.72	90
flatfish	0.62	0.73	0.67	90
ray	0.71	0.71	0.71	90
shark	0.85	0.79	0.82	90
trout	0.82	0.87	0.84	90
orchids	0.62	0.64	0.63	90
poppies	0.54	0.44	0.49	90
roses	0.71	0.68	0.69	90
sunflowers	0.66	0.58	0.62	90
tulips	0.74	0.64	0.69	90
bottles	0.62	0.59	0.61	90
bowls	0.79	0.83	0.81	90
cans	0.75	0.80	0.77	90
cups	0.60	0.61	0.61	90
plates	0.69	0.58	0.63	90
apples	0.79	0.84	0.82	90
mushrooms	0.79	0.82	0.80	90
oranges	0.77	0.83	0.80	90
pears	0.76	0.79	0.77	90
sweet peppers	0.82	0.86	0.84	90
clock	0.68	0.68	0.68	90
computer keyboard	0.55	0.58	0.57	90
lamp	0.50	0.58	0.54	90
telephone	0.81	0.78	0.80	90
television	0.74	0.63	0.68	90
bed	0.64	0.64	0.64	90
chair	0.69	0.68	0.69	90
couch	0.58	0.49	0.53	90
table	0.58	0.64	0.61	90
wardrobe	0.67	0.74	0.71	90
bee	0.36	0.30	0.33	90
beetle	0.69	0.73	0.71	90
butterfly	0.60	0.67	0.63	90
caterpillar	0.59	0.50	0.54	90
cockroach	0.92	0.89	0.90	90
bear	0.67	0.73	0.70	90
leopard	0.80	0.86	0.83	90
lion	0.62	0.64	0.63	90
tiger	0.74	0.71	0.73	90
wolf	0.50	0.57	0.53	90
bridge	0.48	0.53	0.50	90
castle	0.47	0.46	0.46	90
house	0.68	0.59	0.63	90
road	0.85	0.88	0.86	90
skyscraper	0.85	0.82	0.84	90
cloud	0.43	0.43	0.43	90
forest	0.77	0.77	0.77	90
mountain	0.63	0.71	0.67	90
plain	0.91	0.87	0.89	90
sea	0.69	0.79	0.74	90
camel	0.42	0.37	0.39	90
cattle	0.86	0.81	0.83	90
chimpanzee	0.71	0.79	0.75	90

elephant	0.86	0.73	0.79	90
kangaroo	0.57	0.57	0.57	90
fox	0.83	0.81	0.82	90
porcupine	0.77	0.72	0.75	90
possum	0.66	0.69	0.67	90
raccoon	0.77	0.59	0.67	90
skunk	0.56	0.41	0.47	90
crab	0.60	0.54	0.57	90
lobster	0.66	0.66	0.66	90
snail	0.65	0.66	0.65	90
spider	0.86	0.89	0.87	90
worm	0.85	0.87	0.86	90
baby	0.75	0.68	0.71	90
boy	0.77	0.80	0.78	90
girl	0.42	0.41	0.42	90
man	0.59	0.60	0.60	90
woman	0.44	0.44	0.44	90
crocodile	0.82	0.83	0.83	90
dinosaur	0.81	0.86	0.83	90
lizard	0.62	0.62	0.62	90
snake	0.72	0.61	0.66	90
turtle	0.64	0.68	0.66	90
hamster	0.42	0.41	0.41	90
mouse	0.57	0.59	0.58	90
rabbit	0.88	0.93	0.91	90
shrew	0.74	0.74	0.74	90
squirrel	0.64	0.54	0.59	90
maple	0.70	0.76	0.73	90
oak	0.82	0.86	0.84	90
palm	0.78	0.77	0.77	90
pine	0.59	0.61	0.60	90
willow	0.66	0.68	0.67	90
bicycle	0.61	0.61	0.61	90
bus	0.72	0.72	0.72	90
motorcycle	0.55	0.58	0.56	90
pickup truck	0.66	0.56	0.60	90
train	0.83	0.89	0.86	90
lawn-mower	0.71	0.77	0.74	90
rocket	0.52	0.62	0.57	90
streetcar	0.60	0.68	0.64	90
tank	0.49	0.47	0.48	90
tractor	0.80	0.82	0.81	90
accuracy			0.68	9000
macro avg	0.68	0.68	0.68	9000
weighted avg	0.68	0.68	0.68	9000

RMSProp - 0.9 momentum,

```
In [6]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")
```

```

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                    .map(augment_dataset, num_parallel_calls=tf.data.experimen
#                    .batch(batch_size))

```

```

# .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2

```

```

        print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
        print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
        print(sample[1].shape) # Should be (batch_size, 100)
    print(f"Model input shape: {model.input_shape}")
    print(f"Model output shape: {model.output_shape}")
    sample = next(iter(test_dataset.as_numpy_iterator()))
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
    print(sample[0].shape, sample[1].shape) # Should match model input and output
    print("\n")
    #for x, y in test_dataset.take(1):
    #    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
    #for x_batch, y_batch in test_dataset.take(1):
    #    test_loss, test_acc = model.evaluate(x_batch, y_batch)
    #    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

    # Compile the model
    #tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
    model.compile(optimizer=optimizers.RMSprop(learning_rate=1e-3, momentum=0.9,
        loss='categorical_crossentropy',
        metrics=['accuracy', 'precision', 'f1_score']))

    model.summary()

    #### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
    early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
        patience=5, # Num. epochs with no improvement
        restore_best_weights=True)

    #reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
    #    factor=0.1, # Reduce lr by a factor
    #    patience=3, # Num epochs w/ no improvement
    #    min_lr=1e-6, # Min lr
    #    verbose=1)

    #tensorboard = TensorBoard(log_dir='./logs', # Logs directory
    #    histogram_freq=1, # Logs histograms for weights/acc
    #    write_graph=True, # Logs graph of model
    #    write_images=True) # Log images like weight histograms

    checkpoint = ModelCheckpoint('best_model_v11.keras', # Path where the best model is saved
        monitor='val_loss', # What to monitor (validation loss)
        save_best_only=True, # Only save model when val loss is best
        mode='min', # Looking to minimize the validation loss
        verbose=1) # Print out message when saving model

    checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v11.keras', # Path where the best model is saved
        monitor='val_loss', # What to monitor (validation loss)
        save_best_only=True, # Only save model when val loss is best
        mode='min', # Looking to minimize the validation loss
        verbose=1) # Print out message when saving model

    #csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

    # Train the model
    history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
        batch_size=batch_size, callbacks=[checkpoint], verbose=1)

    #### <<<<<<<<Plot Training & Validation Error>>>>>>>>

    # Extract loss and accuracy
    epochs = range(1, len(history.history['loss'])+1)
    train_loss = history.history['loss']
    val_loss = history.history['val_loss']
    train_acc = history.history['accuracy']

```

```

val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v11.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>

# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.RMSprop(learning_rate=1e-5, momentum=0.9,
    loss='categorical_crossentropy',
    metrics=['accuracy', 'precision', 'f1_score']))

model.summary()

```

```

#### <<<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset,
                              batch_size=batch_size, callbacks=[checkpoint_f

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v11.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')

```



```

plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>

169001437/169001437  2s 0us/step

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

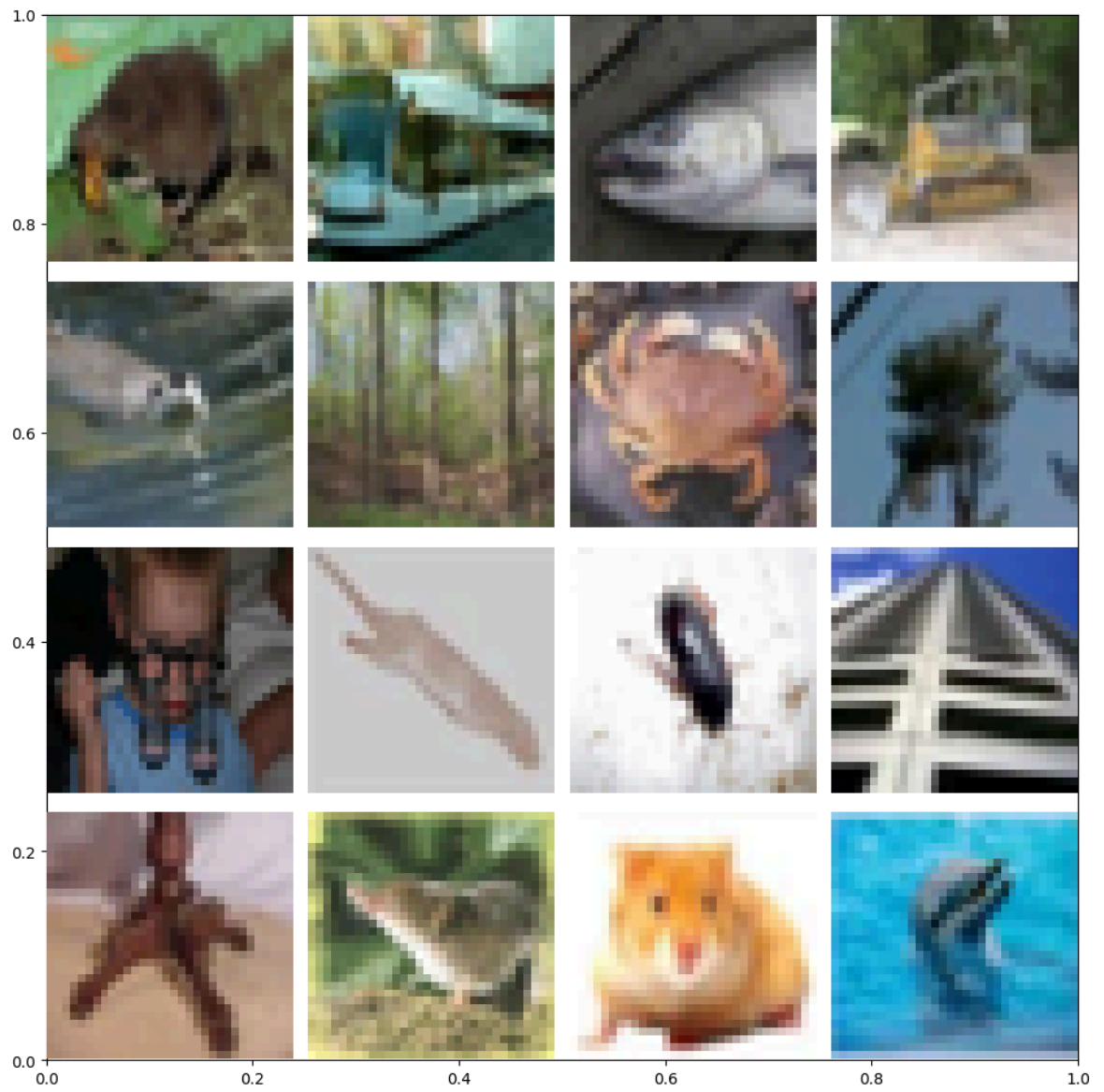
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

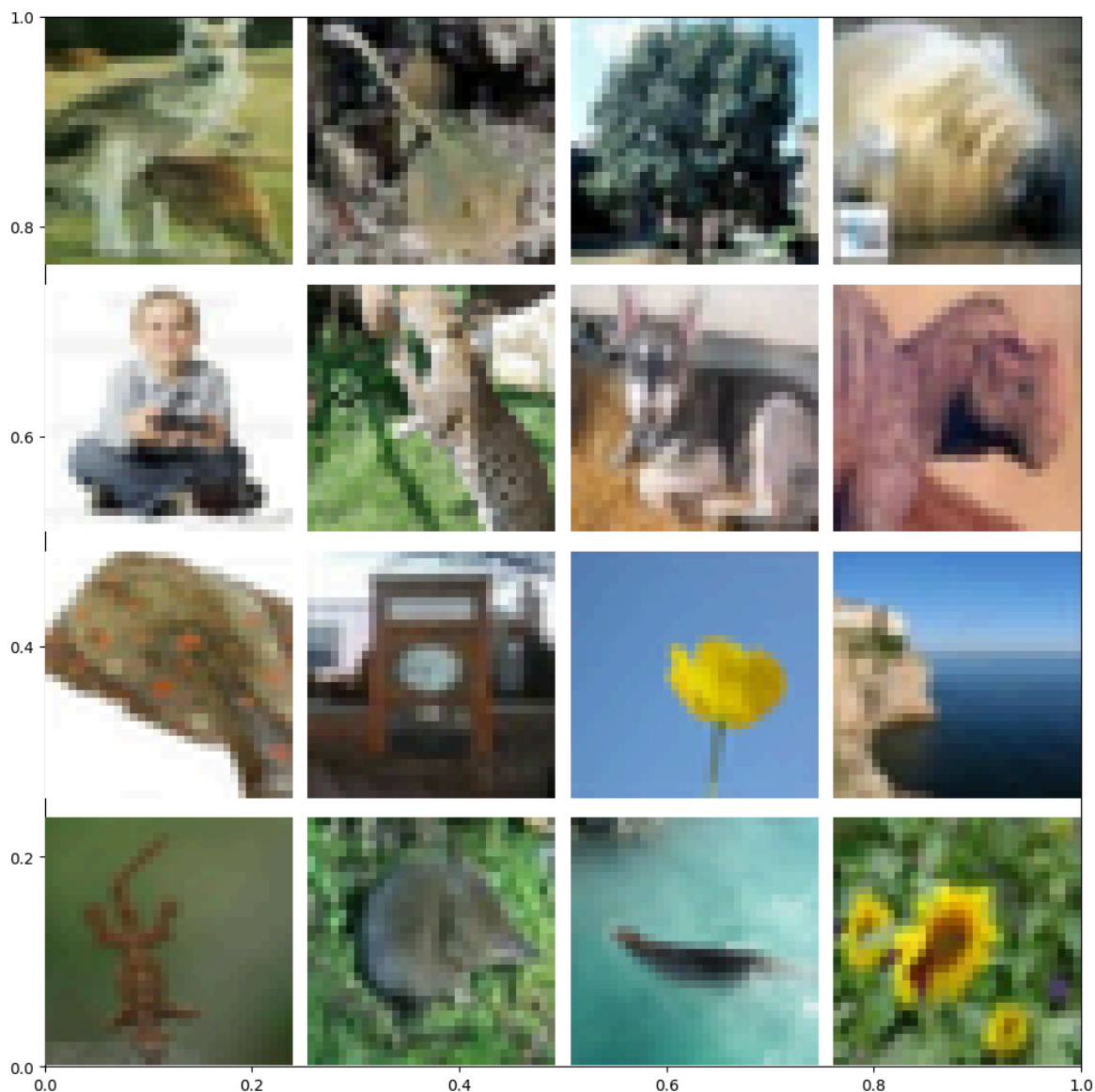
y_test.shape: (9000, 1)

labels[37268].shape: (1,)

imgs[37268].shape: (32, 32, 3)

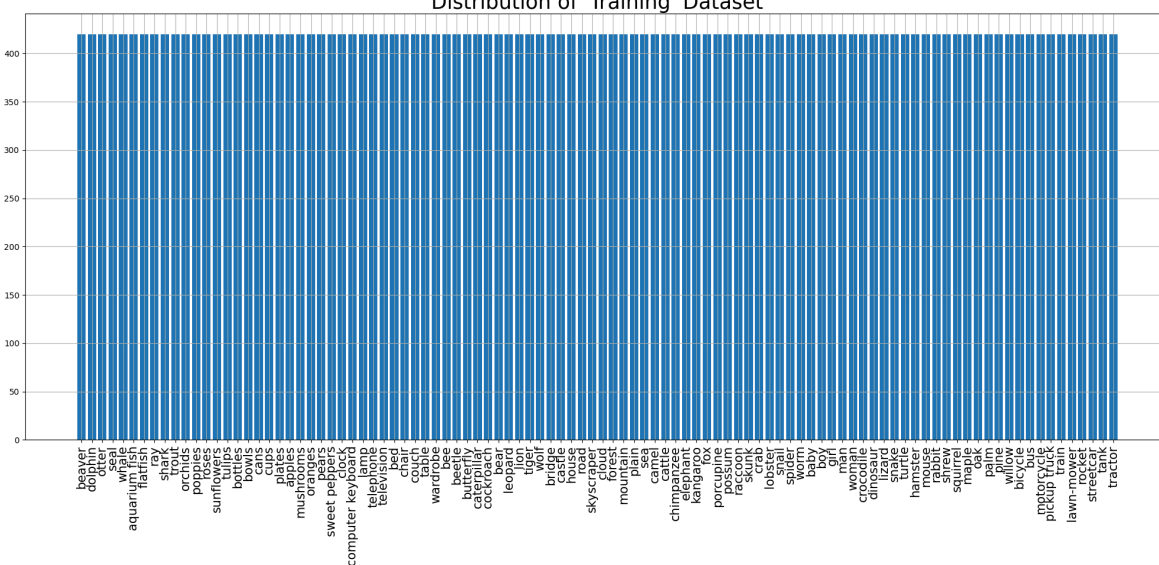


labels[31216].shape: (1,)
imgs[31216].shape: (32, 32, 3)

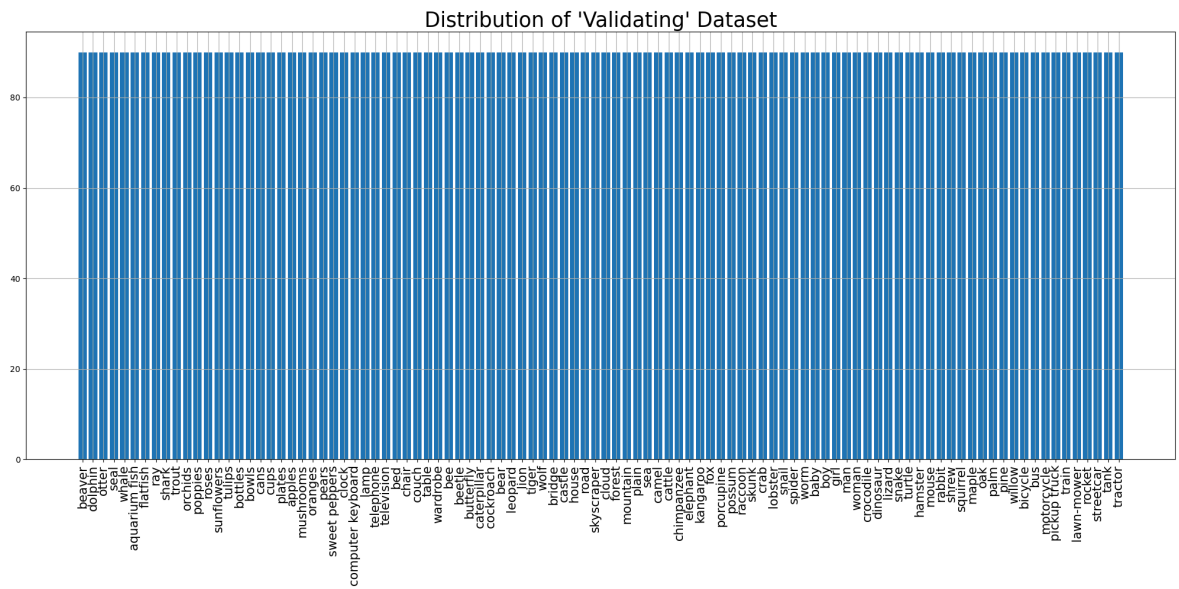


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

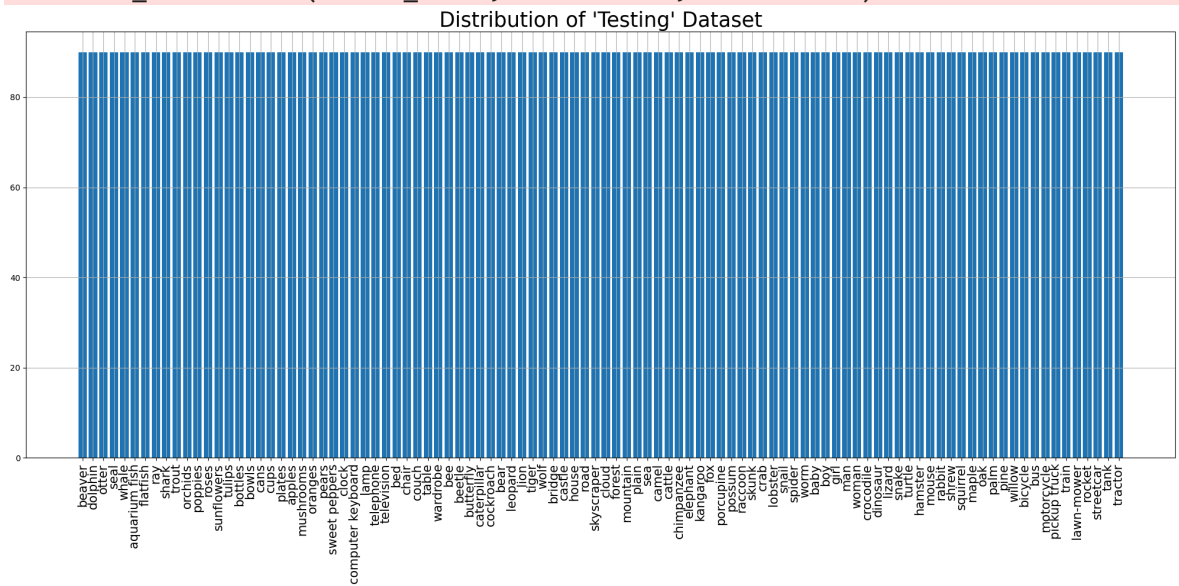
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5
94668760/94668760 ————— 0s 0us/step
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)

◀  ▶

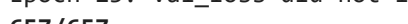
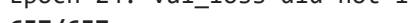
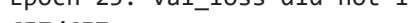
Total params: [24,665,188](#) (94.09 MB)

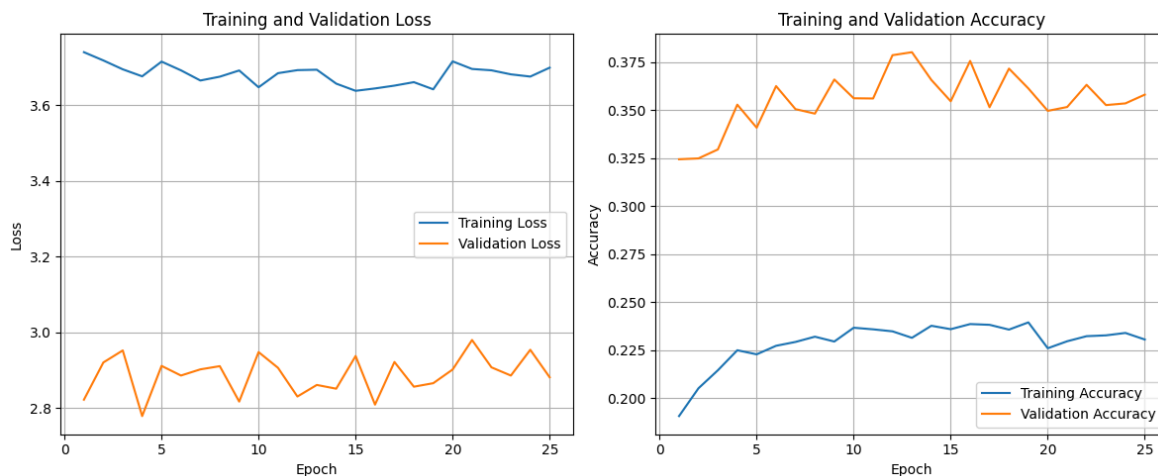
Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 36ms/step - accuracy: 0.1706 - f1_score: 0.1690 - loss: 3.9124 - precision: 0.4587
Epoch 1: val_loss improved from inf to 2.82193, saving model to best_model_v11.keras
657/657 ————— 58s 60ms/step - accuracy: 0.1706 - f1_score: 0.1690 - loss: 3.9121 - precision: 0.4587 - val_accuracy: 0.3243 - val_f1_score: 0.3283 - val_loss: 2.8219 - val_precision: 0.7264
Epoch 2/25
656/657 ————— 0s 24ms/step - accuracy: 0.2047 - f1_score: 0.2252 - loss: 3.7150 - precision: 0.5543
Epoch 2: val_loss did not improve from 2.82193
657/657 ————— 19s 29ms/step - accuracy: 0.2047 - f1_score: 0.2252 - loss: 3.7150 - precision: 0.5543 - val_accuracy: 0.3248 - val_f1_score: 0.3363 - val_loss: 2.9202 - val_precision: 0.7441
Epoch 3/25
656/657 ————— 0s 24ms/step - accuracy: 0.2098 - f1_score: 0.2358 - loss: 3.6964 - precision: 0.5896
Epoch 3: val_loss did not improve from 2.82193
657/657 ————— 19s 29ms/step - accuracy: 0.2098 - f1_score: 0.2358 - loss: 3.6964 - precision: 0.5896 - val_accuracy: 0.3294 - val_f1_score: 0.3404 - val_loss: 2.9520 - val_precision: 0.7430
Epoch 4/25
655/657 ————— 0s 24ms/step - accuracy: 0.2196 - f1_score: 0.2525 - loss: 3.7082 - precision: 0.6013
Epoch 4: val_loss improved from 2.82193 to 2.77862, saving model to best_model_v11.keras
657/657 ————— 20s 30ms/step - accuracy: 0.2197 - f1_score: 0.2525 - loss: 3.7081 - precision: 0.6013 - val_accuracy: 0.3528 - val_f1_score: 0.3590 - val_loss: 2.7786 - val_precision: 0.7515
Epoch 5/25
655/657 ————— 0s 24ms/step - accuracy: 0.2173 - f1_score: 0.2528 - loss: 3.7285 - precision: 0.6186
Epoch 5: val_loss did not improve from 2.77862
657/657 ————— 19s 29ms/step - accuracy: 0.2173 - f1_score: 0.2528 - loss: 3.7284 - precision: 0.6187 - val_accuracy: 0.3408 - val_f1_score: 0.3558 - val_loss: 2.9112 - val_precision: 0.7784
Epoch 6/25
656/657 ————— 0s 24ms/step - accuracy: 0.2204 - f1_score: 0.2572 - loss: 3.6966 - precision: 0.6252
Epoch 6: val_loss did not improve from 2.77862
657/657 ————— 19s 28ms/step - accuracy: 0.2205 - f1_score: 0.2572 - loss: 3.6966 - precision: 0.6253 - val_accuracy: 0.3624 - val_f1_score: 0.3756 - val_loss: 2.8857 - val_precision: 0.7598
Epoch 7/25
657/657 ————— 0s 24ms/step - accuracy: 0.2303 - f1_score: 0.2675 - loss: 3.6682 - precision: 0.6503
Epoch 7: val_loss did not improve from 2.77862
657/657 ————— 19s 29ms/step - accuracy: 0.2303 - f1_score: 0.2675 - loss: 3.6682 - precision: 0.6503 - val_accuracy: 0.3504 - val_f1_score: 0.3722 - val_loss: 2.9020 - val_precision: 0.7688
Epoch 8/25
656/657 ————— 0s 24ms/step - accuracy: 0.2320 - f1_score: 0.2733 - loss: 3.6866 - precision: 0.6489
Epoch 8: val_loss did not improve from 2.77862
657/657 ————— 19s 28ms/step - accuracy: 0.2320 - f1_score: 0.2733 - loss: 3.6865 - precision: 0.6490 - val_accuracy: 0.3481 - val_f1_score: 0.3723 - val_loss: 2.9105 - val_precision: 0.7951
Epoch 9/25
655/657 ————— 0s 24ms/step - accuracy: 0.2212 - f1_score: 0.2645 -

loss: 3.7253 - precision: 0.6661
Epoch 9: val_loss did not improve from 2.77862
657/657 ————— **19s** 29ms/step - accuracy: 0.2212 - f1_score: 0.2646
- loss: 3.7252 - precision: 0.6661 - val_accuracy: 0.3659 - val_f1_score: 0.3834
- val_loss: 2.8169 - val_precision: 0.7744
Epoch 10/25
655/657 ————— **0s** 24ms/step - accuracy: 0.2341 - f1_score: 0.2760 -
loss: 3.6362 - precision: 0.6707
Epoch 10: val_loss did not improve from 2.77862
657/657 ————— **19s** 29ms/step - accuracy: 0.2341 - f1_score: 0.2761
- loss: 3.6363 - precision: 0.6707 - val_accuracy: 0.3561 - val_f1_score: 0.3670
- val_loss: 2.9476 - val_precision: 0.7560
Epoch 11/25
655/657 ————— **0s** 24ms/step - accuracy: 0.2327 - f1_score: 0.2766 -
loss: 3.6963 - precision: 0.6671
Epoch 11: val_loss did not improve from 2.77862
657/657 ————— **19s** 29ms/step - accuracy: 0.2327 - f1_score: 0.2767
- loss: 3.6962 - precision: 0.6672 - val_accuracy: 0.3560 - val_f1_score: 0.3779
- val_loss: 2.9061 - val_precision: 0.7673
Epoch 12/25
657/657 ————— **0s** 24ms/step - accuracy: 0.2346 - f1_score: 0.2782 -
loss: 3.7150 - precision: 0.6729
Epoch 12: val_loss did not improve from 2.77862
657/657 ————— **19s** 29ms/step - accuracy: 0.2346 - f1_score: 0.2782
- loss: 3.7150 - precision: 0.6729 - val_accuracy: 0.3786 - val_f1_score: 0.4006
- val_loss: 2.8303 - val_precision: 0.7942
Epoch 13/25
657/657 ————— **0s** 24ms/step - accuracy: 0.2339 - f1_score: 0.2801 -
loss: 3.6749 - precision: 0.6796
Epoch 13: val_loss did not improve from 2.77862
657/657 ————— **19s** 29ms/step - accuracy: 0.2339 - f1_score: 0.2801
- loss: 3.6750 - precision: 0.6796 - val_accuracy: 0.3801 - val_f1_score: 0.3934
- val_loss: 2.8609 - val_precision: 0.7816
Epoch 14/25
655/657 ————— **0s** 25ms/step - accuracy: 0.2349 - f1_score: 0.2787 -
loss: 3.6811 - precision: 0.6805
Epoch 14: val_loss did not improve from 2.77862
657/657 ————— **21s** 30ms/step - accuracy: 0.2349 - f1_score: 0.2788
- loss: 3.6810 - precision: 0.6805 - val_accuracy: 0.3657 - val_f1_score: 0.3897
- val_loss: 2.8510 - val_precision: 0.7816
Epoch 15/25
657/657 ————— **0s** 24ms/step - accuracy: 0.2313 - f1_score: 0.2752 -
loss: 3.6683 - precision: 0.6714
Epoch 15: val_loss did not improve from 2.77862
657/657 ————— **19s** 29ms/step - accuracy: 0.2313 - f1_score: 0.2752
- loss: 3.6682 - precision: 0.6714 - val_accuracy: 0.3546 - val_f1_score: 0.3809
- val_loss: 2.9373 - val_precision: 0.8032
Epoch 16/25
655/657 ————— **0s** 24ms/step - accuracy: 0.2343 - f1_score: 0.2786 -
loss: 3.6637 - precision: 0.6713
Epoch 16: val_loss did not improve from 2.77862
657/657 ————— **19s** 29ms/step - accuracy: 0.2343 - f1_score: 0.2786
- loss: 3.6636 - precision: 0.6713 - val_accuracy: 0.3756 - val_f1_score: 0.3881
- val_loss: 2.8089 - val_precision: 0.7773
Epoch 17/25
656/657 ————— **0s** 24ms/step - accuracy: 0.2412 - f1_score: 0.2859 -
loss: 3.6309 - precision: 0.6733
Epoch 17: val_loss did not improve from 2.77862
657/657 ————— **19s** 29ms/step - accuracy: 0.2412 - f1_score: 0.2859
- loss: 3.6310 - precision: 0.6733 - val_accuracy: 0.3514 - val_f1_score: 0.3677

- val_loss: 2.9215 - val_precision: 0.7852
Epoch 18/25
655/657  0s 24ms/step - accuracy: 0.2289 - f1_score: 0.2707 -
loss: 3.6641 - precision: 0.6720
Epoch 18: val_loss did not improve from 2.77862
657/657  19s 29ms/step - accuracy: 0.2290 - f1_score: 0.2708
- loss: 3.6640 - precision: 0.6720 - val_accuracy: 0.3716 - val_f1_score: 0.3948
- val_loss: 2.8562 - val_precision: 0.7617
Epoch 19/25
656/657  0s 24ms/step - accuracy: 0.2402 - f1_score: 0.2845 -
loss: 3.6287 - precision: 0.6685
Epoch 19: val_loss did not improve from 2.77862
657/657  19s 29ms/step - accuracy: 0.2402 - f1_score: 0.2845
- loss: 3.6287 - precision: 0.6685 - val_accuracy: 0.3612 - val_f1_score: 0.3899
- val_loss: 2.8655 - val_precision: 0.7905
Epoch 20/25
655/657  0s 24ms/step - accuracy: 0.2262 - f1_score: 0.2723 -
loss: 3.6927 - precision: 0.6821
Epoch 20: val_loss did not improve from 2.77862
657/657  19s 29ms/step - accuracy: 0.2262 - f1_score: 0.2723
- loss: 3.6928 - precision: 0.6821 - val_accuracy: 0.3496 - val_f1_score: 0.3704
- val_loss: 2.9017 - val_precision: 0.7733
Epoch 21/25
657/657  0s 25ms/step - accuracy: 0.2278 - f1_score: 0.2713 -
loss: 3.7123 - precision: 0.6784
Epoch 21: val_loss did not improve from 2.77862
657/657  19s 29ms/step - accuracy: 0.2278 - f1_score: 0.2713
- loss: 3.7122 - precision: 0.6784 - val_accuracy: 0.3516 - val_f1_score: 0.3745
- val_loss: 2.9796 - val_precision: 0.7805
Epoch 22/25
655/657  0s 24ms/step - accuracy: 0.2301 - f1_score: 0.2750 -
loss: 3.7005 - precision: 0.6845
Epoch 22: val_loss did not improve from 2.77862
657/657  19s 29ms/step - accuracy: 0.2301 - f1_score: 0.2750
- loss: 3.7005 - precision: 0.6845 - val_accuracy: 0.3631 - val_f1_score: 0.3738
- val_loss: 2.9073 - val_precision: 0.7698
Epoch 23/25
656/657  0s 24ms/step - accuracy: 0.2292 - f1_score: 0.2714 -
loss: 3.7162 - precision: 0.6766
Epoch 23: val_loss did not improve from 2.77862
657/657  19s 29ms/step - accuracy: 0.2292 - f1_score: 0.2714
- loss: 3.7161 - precision: 0.6766 - val_accuracy: 0.3526 - val_f1_score: 0.3788
- val_loss: 2.8857 - val_precision: 0.7850
Epoch 24/25
657/657  0s 24ms/step - accuracy: 0.2307 - f1_score: 0.2739 -
loss: 3.6834 - precision: 0.6837
Epoch 24: val_loss did not improve from 2.77862
657/657  19s 29ms/step - accuracy: 0.2307 - f1_score: 0.2739
- loss: 3.6834 - precision: 0.6837 - val_accuracy: 0.3534 - val_f1_score: 0.3695
- val_loss: 2.9537 - val_precision: 0.7912
Epoch 25/25
657/657  0s 24ms/step - accuracy: 0.2305 - f1_score: 0.2746 -
loss: 3.7041 - precision: 0.6684
Epoch 25: val_loss did not improve from 2.77862
657/657  19s 29ms/step - accuracy: 0.2305 - f1_score: 0.2746
- loss: 3.7041 - precision: 0.6685 - val_accuracy: 0.3580 - val_f1_score: 0.3787
- val_loss: 2.8810 - val_precision: 0.7871



141/141 ————— **12s** 42ms/step - accuracy: 0.3427 - f1_score: 0.3485

- loss: 2.8270 - precision: 0.7533

Test Accuracy: 35.32%

Test Loss: 2.7936198711395264

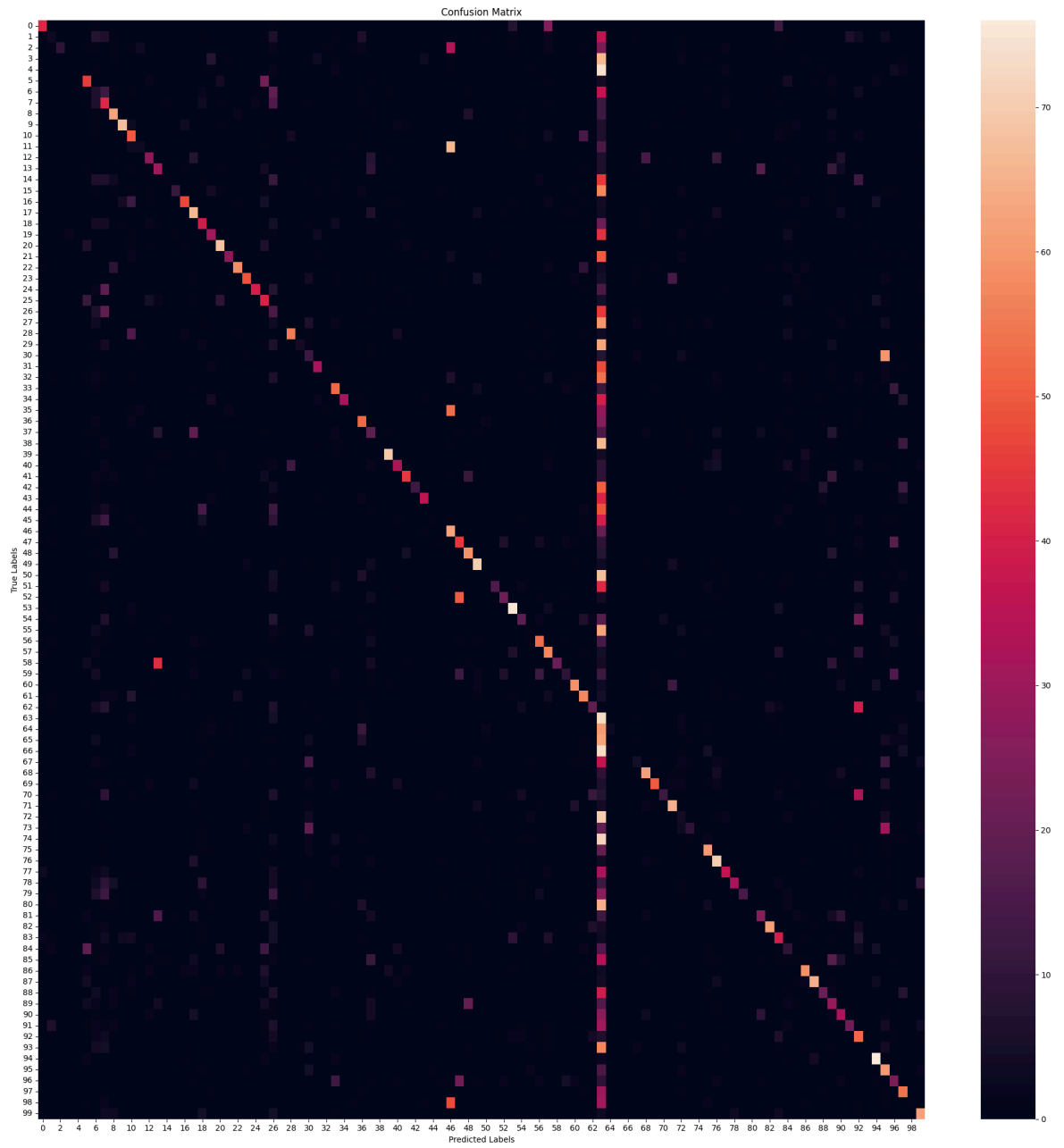
Test Precision: 75.30%

Test F1 Scores (Per Class): [58.741253 5.2173877 19.999998 0. 0.
46.46464

6.542051	27.009642	62.999992	80.722885	47.169804	5.9999986
37.762234	31.472075	2.105262	21.35922	63.51351	60.550453
41.711224	41.33333	72.34043	44.628098	73.41772	65.771805
60.15037	36.12334	9.090905	0.	68.29269	8.421051
12.698409	47.407402	0.	52.525253	52.032516	0.
52.791874	19.587626	0.	81.6568	45.20548	60.810806
23.529411	53.73134	0.	0.	33.96226	40.540535
60.82474	75.268814	0.	29.629627	34.645664	75.37688
29.00763	1.9417455	61.6279	53.023254	37.837833	17.85714
70.23808	58.163254	28.787872	5.630543	6.315788	0.
0.	9.259257	68.508286	67.11409	24.5283	68.06282
5.755391	18.518517	0.	67.77777	66.037735	52.112667
49.999992	26.923075	0.	34.666664	77.01864	42.553185
11.8421	1.9230747	67.42856	78.78787	33.33333	26.6055
37.647053	37.28813	32.704395	0.	77.948715	39.354836
25.806448	51.184826	2.197802	64.921455	]	

Average Test F1 Scores:35.89

282/282 ————— **19s** 39ms/step



	precision	recall	f1-score	support
beaver	0.79	0.47	0.59	90
dolphin	0.12	0.03	0.05	90
otter	1.00	0.11	0.20	90
seal	0.00	0.00	0.00	90
whale	0.00	0.00	0.00	90
aquarium fish	0.43	0.51	0.46	90
flatfish	0.06	0.08	0.07	90
ray	0.19	0.47	0.27	90
shark	0.57	0.70	0.63	90
trout	0.88	0.74	0.81	90
orchids	0.41	0.56	0.47	90
poppies	0.30	0.03	0.06	90
roses	0.51	0.30	0.38	90
sunflowers	0.29	0.34	0.31	90
tulips	0.20	0.01	0.02	90
bottles	0.85	0.12	0.21	90
bowls	0.81	0.52	0.64	90
cans	0.52	0.73	0.61	90
cups	0.40	0.43	0.42	90
plates	0.52	0.34	0.41	90
apples	0.69	0.76	0.72	90
mushrooms	0.87	0.30	0.45	90
oranges	0.85	0.64	0.73	90
pears	0.83	0.54	0.66	90
sweet peppers	0.93	0.44	0.60	90
clock	0.30	0.46	0.36	90
computer keyboard	0.06	0.17	0.09	90
lamp	0.00	0.00	0.00	90
telephone	0.76	0.62	0.68	90
television	0.80	0.04	0.08	90
bed	0.12	0.13	0.13	90
chair	0.71	0.36	0.47	90
couch	0.00	0.00	0.00	90
table	0.48	0.58	0.53	90
wardrobe	0.97	0.36	0.52	90
bee	0.00	0.00	0.00	90
beetle	0.49	0.58	0.53	90
butterfly	0.18	0.21	0.20	90
caterpillar	0.00	0.00	0.00	90
cockroach	0.87	0.77	0.82	90
bear	0.59	0.37	0.45	90
leopard	0.78	0.50	0.61	90
lion	1.00	0.13	0.24	90
tiger	0.82	0.40	0.54	90
wolf	0.00	0.00	0.00	90
bridge	0.00	0.00	0.00	90
castle	0.22	0.70	0.34	90
house	0.34	0.50	0.41	90
road	0.57	0.66	0.61	90
skyscraper	0.73	0.78	0.75	90
cloud	0.00	0.00	0.00	90
forest	0.89	0.18	0.30	90
mountain	0.59	0.24	0.35	90
plain	0.69	0.83	0.75	90
sea	0.45	0.20	0.28	90
camel	0.08	0.01	0.02	90
cattle	0.65	0.59	0.62	90
chimpanzee	0.46	0.63	0.53	90

elephant	1.00	0.23	0.38	90
kangaroo	0.45	0.11	0.18	90
fox	0.76	0.66	0.70	90
porcupine	0.54	0.63	0.58	90
possum	0.45	0.21	0.29	90
raccoon	0.03	0.81	0.06	90
skunk	0.60	0.03	0.06	90
crab	0.00	0.00	0.00	90
lobster	0.00	0.00	0.00	90
snail	0.28	0.06	0.09	90
spider	0.68	0.69	0.69	90
worm	0.85	0.56	0.67	90
baby	0.81	0.14	0.25	90
boy	0.64	0.72	0.68	90
girl	0.08	0.04	0.06	90
man	0.56	0.11	0.19	90
woman	0.00	0.00	0.00	90
crocodile	0.68	0.68	0.68	90
dinosaur	0.57	0.78	0.66	90
lizard	0.71	0.41	0.52	90
snake	0.84	0.36	0.50	90
turtle	1.00	0.16	0.27	90
hamster	0.00	0.00	0.00	90
mouse	0.43	0.29	0.35	90
rabbit	0.87	0.69	0.77	90
shrew	0.41	0.44	0.43	90
squirrel	0.15	0.10	0.12	90
maple	0.07	0.01	0.02	90
oak	0.69	0.66	0.67	90
palm	0.87	0.72	0.79	90
pine	0.58	0.23	0.33	90
willow	0.23	0.32	0.27	90
bicycle	0.41	0.36	0.38	90
bus	0.79	0.24	0.37	90
motorcycle	0.23	0.58	0.33	90
pickup truck	0.00	0.00	0.00	90
train	0.72	0.84	0.78	90
lawn-mower	0.28	0.68	0.39	90
rocket	0.25	0.27	0.26	90
streetcar	0.45	0.60	0.51	90
tank	1.00	0.01	0.02	90
tractor	0.61	0.69	0.65	90
accuracy			0.35	9000
macro avg	0.48	0.35	0.36	9000
weighted avg	0.48	0.35	0.36	9000

```

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)



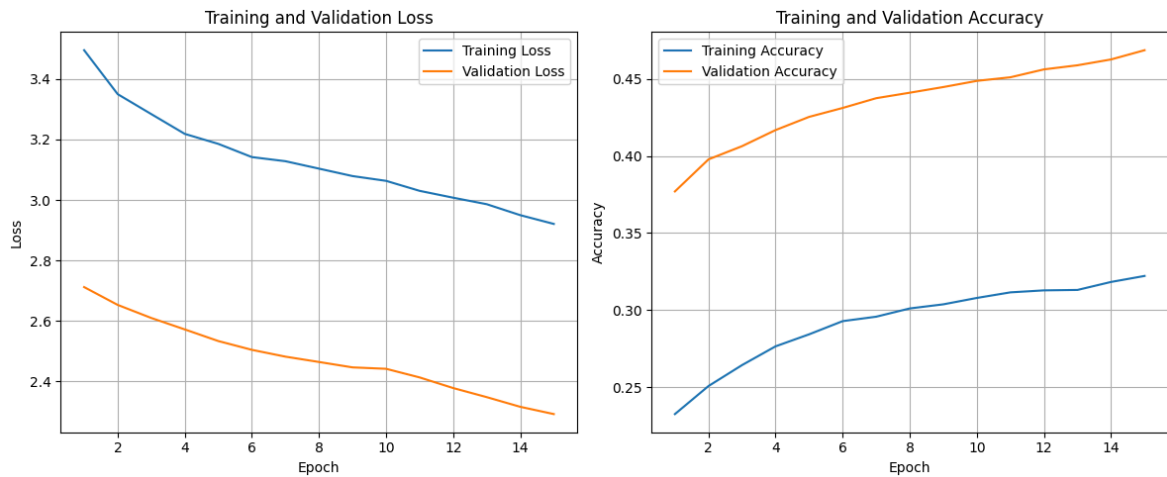
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 27ms/step - accuracy: 0.2264 - f1_score: 0.2603
- loss: 3.5783 - precision: 0.6510
Epoch 1: val_loss improved from inf to 2.71187, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 59s 35ms/step - accuracy: 0.2264 - f1_score: 0.2603 - loss: 3.5781 - precision: 0.6511 - val_accuracy: 0.3770 - val_f1_score: 0.4093 - val_loss: 2.7119 - val_precision: 0.8364
Epoch 2/15
1312/1314 ————— 0s 25ms/step - accuracy: 0.2477 - f1_score: 0.2914 - loss: 3.3665 - precision: 0.7188
Epoch 2: val_loss improved from 2.71187 to 2.65299, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 37s 28ms/step - accuracy: 0.2477 - f1_score: 0.2914 - loss: 3.3665 - precision: 0.7188 - val_accuracy: 0.3978 - val_f1_score: 0.4280 - val_loss: 2.6530 - val_precision: 0.8279
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.2602 - f1_score: 0.3041 - loss: 3.2992 - precision: 0.7321
Epoch 3: val_loss improved from 2.65299 to 2.60983, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.2602 - f1_score: 0.3041 - loss: 3.2992 - precision: 0.7321 - val_accuracy: 0.4063 - val_f1_score: 0.4361 - val_loss: 2.6098 - val_precision: 0.8244
Epoch 4/15
1314/1314 ————— 0s 25ms/step - accuracy: 0.2764 - f1_score: 0.3194 - loss: 3.2217 - precision: 0.7401
Epoch 4: val_loss improved from 2.60983 to 2.57192, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 37s 28ms/step - accuracy: 0.2764 - f1_score: 0.3194 - loss: 3.2217 - precision: 0.7401 - val_accuracy: 0.4168 - val_f1_score: 0.4451 - val_loss: 2.5719 - val_precision: 0.8244
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.2834 - f1_score: 0.3270 - loss: 3.1833 - precision: 0.7402
Epoch 5: val_loss improved from 2.57192 to 2.53347, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.2834 - f1_score: 0.3270 - loss: 3.1833 - precision: 0.7402 - val_accuracy: 0.4253 - val_f1_score: 0.4533 - val_loss: 2.5335 - val_precision: 0.8192
Epoch 6/15
1313/1314 ————— 0s 25ms/step - accuracy: 0.2926 - f1_score: 0.3355 - loss: 3.1404 - precision: 0.7443
Epoch 6: val_loss improved from 2.53347 to 2.50440, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.2926 - f1_score: 0.3355 - loss: 3.1404 - precision: 0.7443 - val_accuracy: 0.4311 - val_f1_score: 0.4572 - val_loss: 2.5044 - val_precision: 0.8186
Epoch 7/15
1314/1314 ————— 0s 25ms/step - accuracy: 0.2951 - f1_score: 0.3373 - loss: 3.1316 - precision: 0.7404
Epoch 7: val_loss improved from 2.50440 to 2.48188, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 37s 28ms/step - accuracy: 0.2951 - f1_score: 0.3373 - loss: 3.1316 - precision: 0.7404 - val_accuracy: 0.4374 - val_f1_score: 0.4634 - val_loss: 2.4819 - val_precision: 0.8197
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.2995 - f1_score: 0.3420 - loss: 3.1125 - precision: 0.7424
Epoch 8: val_loss improved from 2.48188 to 2.46427, saving model to best_model_fine_tune_v11.keras

ne_tune_v11.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.2995 - f1_score: 0.342
0 - loss: 3.1125 - precision: 0.7424 - val_accuracy: 0.4410 - val_f1_score: 0.467
3 - val_loss: 2.4643 - val_precision: 0.8207
Epoch 9/15
1313/1314 ————— 0s 23ms/step - accuracy: 0.3041 - f1_score: 0.3463
- loss: 3.0727 - precision: 0.7455
Epoch 9: val_loss improved from 2.46427 to 2.44641, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.3041 - f1_score: 0.346
3 - loss: 3.0727 - precision: 0.7455 - val_accuracy: 0.4447 - val_f1_score: 0.469
5 - val_loss: 2.4464 - val_precision: 0.8222
Epoch 10/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3084 - f1_score: 0.3488
- loss: 3.0534 - precision: 0.7466
Epoch 10: val_loss improved from 2.44641 to 2.44174, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.3084 - f1_score: 0.348
8 - loss: 3.0535 - precision: 0.7466 - val_accuracy: 0.4487 - val_f1_score: 0.473
9 - val_loss: 2.4417 - val_precision: 0.8272
Epoch 11/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3115 - f1_score: 0.3530
- loss: 3.0271 - precision: 0.7531
Epoch 11: val_loss improved from 2.44174 to 2.41320, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.3115 - f1_score: 0.353
0 - loss: 3.0271 - precision: 0.7531 - val_accuracy: 0.4510 - val_f1_score: 0.475
4 - val_loss: 2.4132 - val_precision: 0.8269
Epoch 12/15
1313/1314 ————— 0s 25ms/step - accuracy: 0.3130 - f1_score: 0.3526
- loss: 2.9998 - precision: 0.7496
Epoch 12: val_loss improved from 2.41320 to 2.37808, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.3130 - f1_score: 0.352
6 - loss: 2.9998 - precision: 0.7496 - val_accuracy: 0.4561 - val_f1_score: 0.481
5 - val_loss: 2.3781 - val_precision: 0.8266
Epoch 13/15
1312/1314 ————— 0s 25ms/step - accuracy: 0.3138 - f1_score: 0.3555
- loss: 2.9741 - precision: 0.7473
Epoch 13: val_loss improved from 2.37808 to 2.34802, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 37s 28ms/step - accuracy: 0.3138 - f1_score: 0.355
5 - loss: 2.9742 - precision: 0.7473 - val_accuracy: 0.4588 - val_f1_score: 0.484
6 - val_loss: 2.3480 - val_precision: 0.8269
Epoch 14/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.3197 - f1_score: 0.3610
- loss: 2.9375 - precision: 0.7532
Epoch 14: val_loss improved from 2.34802 to 2.31602, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.3197 - f1_score: 0.361
0 - loss: 2.9375 - precision: 0.7532 - val_accuracy: 0.4626 - val_f1_score: 0.486
1 - val_loss: 2.3160 - val_precision: 0.8269
Epoch 15/15
1313/1314 ————— 0s 25ms/step - accuracy: 0.3206 - f1_score: 0.3624
- loss: 2.9160 - precision: 0.7554
Epoch 15: val_loss improved from 2.31602 to 2.29204, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— 37s 28ms/step - accuracy: 0.3206 - f1_score: 0.362
4 - loss: 2.9160 - precision: 0.7554 - val_accuracy: 0.4686 - val_f1_score: 0.491
0 - val_loss: 2.2920 - val_precision: 0.8301



141/141 ————— 12s 42ms/step - accuracy: 0.4620 - f1_score: 0.4790

- loss: 2.3481 - precision: 0.8099

Test Accuracy: 46.73%

Test Loss: 2.3066518306732178

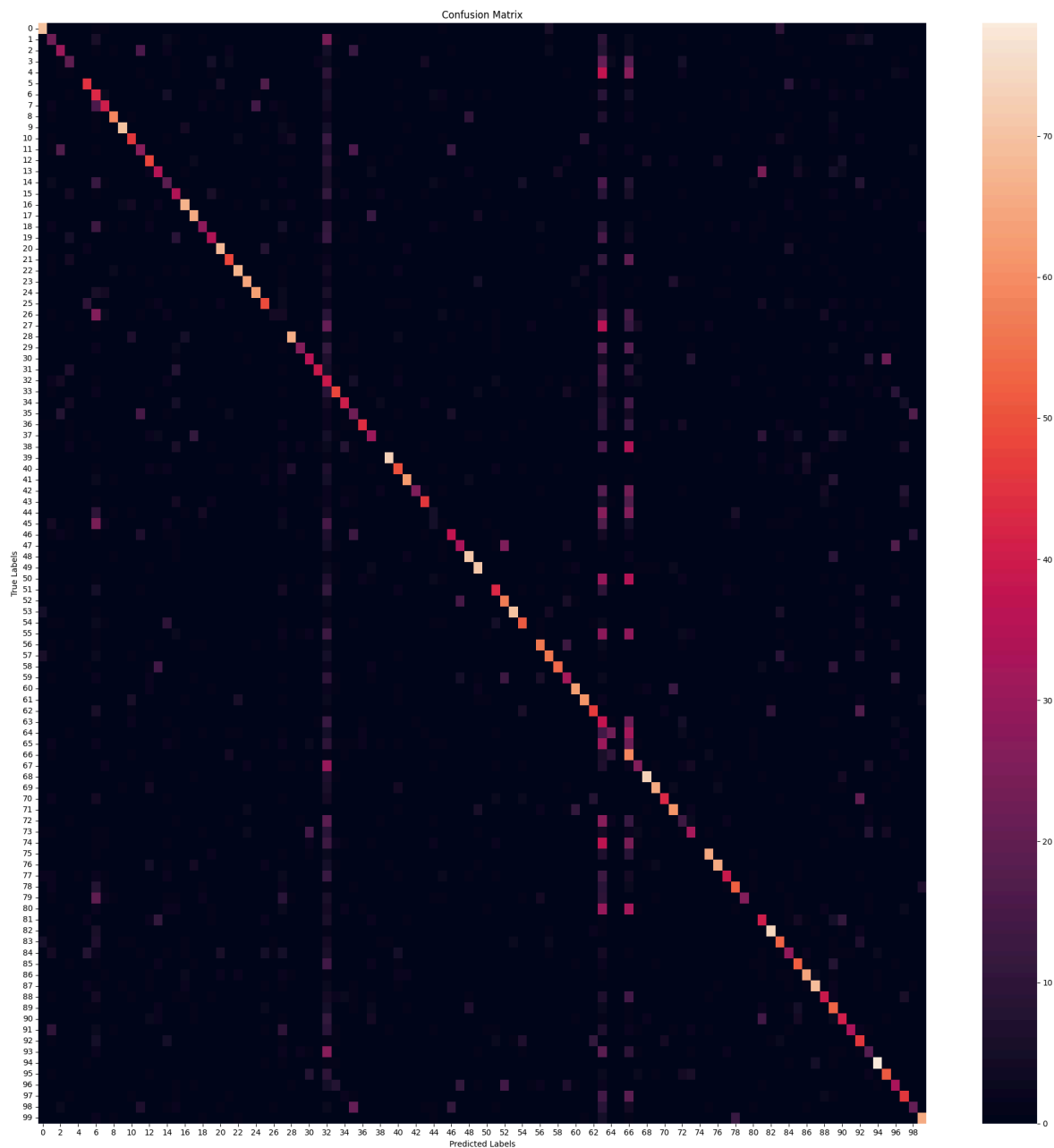
Test Precision: 82.51%

Test F1 Scores (Per Class): [77.77777 25.882345 39.49044 26.845634 0.
55.214714

23.626368	51.612896	74.21383	84.33735	56.097557	33.532932
54.545456	42.424236	24.050629	37.77777	74.86033	71.8232
41.791042	46.258495	76.40449	62.33766	77.01149	78.5276
73.68421	51.851852	8.080808	0.	74.99999	41.6
45.283012	55.71428	10.48387	57.142853	51.282047	22.222218
61.538452	38.509315	0.	86.390526	57.142853	76.543205
43.103443	61.744957	8.130077	1.7699083	47.499996	42.58064
75.93583	75.13227	0.	53.08641	56.43564	84.84848
60.714275	0.	65.88235	67.46988	72.	39.024387
71.99999	72.18934	58.385086	8.77598	32.835815	0.
16.142267	36.363632	82.48587	78.78787	59.154922	68.15642
18.309856	41.558437	0.	78.313255	78.313255	54.166664
60.818714	43.697475	0.	40.201004	81.31868	56.084652
35.15151	58.426964	72.31638	80.23255	49.999992	43.621395
47.12643	50.38167	39.48497	24.32432	86.18784	58.285706
34.92063	51.396645	29.577461	77.8443	]	

Average Test F1 Scores:48.82

282/282 ————— 12s 29ms/step



```
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:  
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w  
ith no predicted samples. Use `zero_division` parameter to control this behavior.  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))  
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:  
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w  
ith no predicted samples. Use `zero_division` parameter to control this behavior.  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))  
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:  
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w  
ith no predicted samples. Use `zero_division` parameter to control this behavior.  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

	precision	recall	f1-score	support
beaver	0.78	0.78	0.78	90
dolphin	0.28	0.24	0.26	90
otter	0.46	0.34	0.39	90
seal	0.33	0.22	0.27	90
whale	0.00	0.00	0.00	90
aquarium fish	0.62	0.50	0.55	90
flatfish	0.16	0.48	0.24	90
ray	0.62	0.44	0.52	90
shark	0.86	0.66	0.74	90
trout	0.91	0.78	0.84	90
orchids	0.62	0.51	0.56	90
poppies	0.36	0.30	0.33	90
roses	0.56	0.53	0.55	90
sunflowers	0.47	0.39	0.43	90
tulips	0.28	0.21	0.24	90
bottles	0.38	0.38	0.38	90
bowls	0.75	0.74	0.75	90
cans	0.71	0.72	0.72	90
cups	0.64	0.31	0.42	90
plates	0.60	0.38	0.46	90
apples	0.77	0.76	0.76	90
mushrooms	0.75	0.53	0.62	90
oranges	0.80	0.74	0.77	90
pears	0.88	0.71	0.79	90
sweet peppers	0.78	0.70	0.74	90
clock	0.49	0.54	0.52	90
computer keyboard	0.44	0.04	0.08	90
lamp	0.00	0.00	0.00	90
telephone	0.77	0.73	0.75	90
television	0.74	0.29	0.42	90
bed	0.52	0.40	0.45	90
chair	0.80	0.43	0.56	90
couch	0.06	0.43	0.10	90
table	0.62	0.53	0.57	90
wardrobe	0.61	0.44	0.51	90
bee	0.20	0.24	0.22	90
beetle	0.83	0.49	0.62	90
butterfly	0.44	0.34	0.39	90
caterpillar	0.00	0.00	0.00	90
cockroach	0.92	0.81	0.86	90
bear	0.59	0.56	0.57	90
leopard	0.86	0.69	0.77	90
lion	0.96	0.28	0.43	90
tiger	0.78	0.51	0.62	90
wolf	0.15	0.06	0.08	90
bridge	0.04	0.01	0.02	90
castle	0.54	0.42	0.47	90
house	0.51	0.37	0.43	90
road	0.73	0.79	0.76	90
skyscraper	0.72	0.79	0.75	90
cloud	0.00	0.00	0.00	90
forest	0.60	0.48	0.53	90
mountain	0.51	0.63	0.56	90
plain	0.95	0.78	0.85	90
sea	0.65	0.57	0.61	90
camel	0.00	0.00	0.00	90
cattle	0.70	0.62	0.66	90
chimpanzee	0.74	0.62	0.67	90

elephant	0.90	0.60	0.72	90
kangaroo	0.43	0.36	0.39	90
fox	0.74	0.70	0.72	90
porcupine	0.77	0.68	0.72	90
possum	0.66	0.52	0.58	90
raccoon	0.05	0.42	0.09	90
skunk	0.50	0.24	0.33	90
crab	0.00	0.00	0.00	90
lobster	0.09	0.66	0.16	90
snail	0.49	0.29	0.36	90
spider	0.84	0.81	0.82	90
worm	0.87	0.72	0.79	90
baby	0.81	0.48	0.60	90
boy	0.69	0.68	0.68	90
girl	0.25	0.14	0.18	90
man	0.50	0.36	0.42	90
woman	0.00	0.00	0.00	90
crocodile	0.86	0.72	0.78	90
dinosaur	0.86	0.72	0.78	90
lizard	0.72	0.43	0.54	90
snake	0.64	0.58	0.61	90
turtle	0.90	0.29	0.44	90
hamster	0.00	0.00	0.00	90
mouse	0.36	0.44	0.40	90
rabbit	0.80	0.82	0.81	90
shrew	0.54	0.59	0.56	90
squirrel	0.39	0.32	0.35	90
maple	0.59	0.58	0.58	90
oak	0.74	0.71	0.72	90
palm	0.84	0.77	0.80	90
pine	0.59	0.43	0.50	90
willow	0.35	0.59	0.44	90
bicycle	0.49	0.46	0.47	90
bus	0.80	0.37	0.50	90
motorcycle	0.32	0.51	0.39	90
pickup truck	0.31	0.20	0.24	90
train	0.86	0.87	0.86	90
lawn-mower	0.60	0.57	0.58	90
rocket	0.33	0.37	0.35	90
streetcar	0.52	0.51	0.51	90
tank	0.40	0.23	0.30	90
tractor	0.84	0.72	0.78	90
accuracy			0.47	9000
macro avg	0.55	0.47	0.49	9000
weighted avg	0.55	0.47	0.49	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

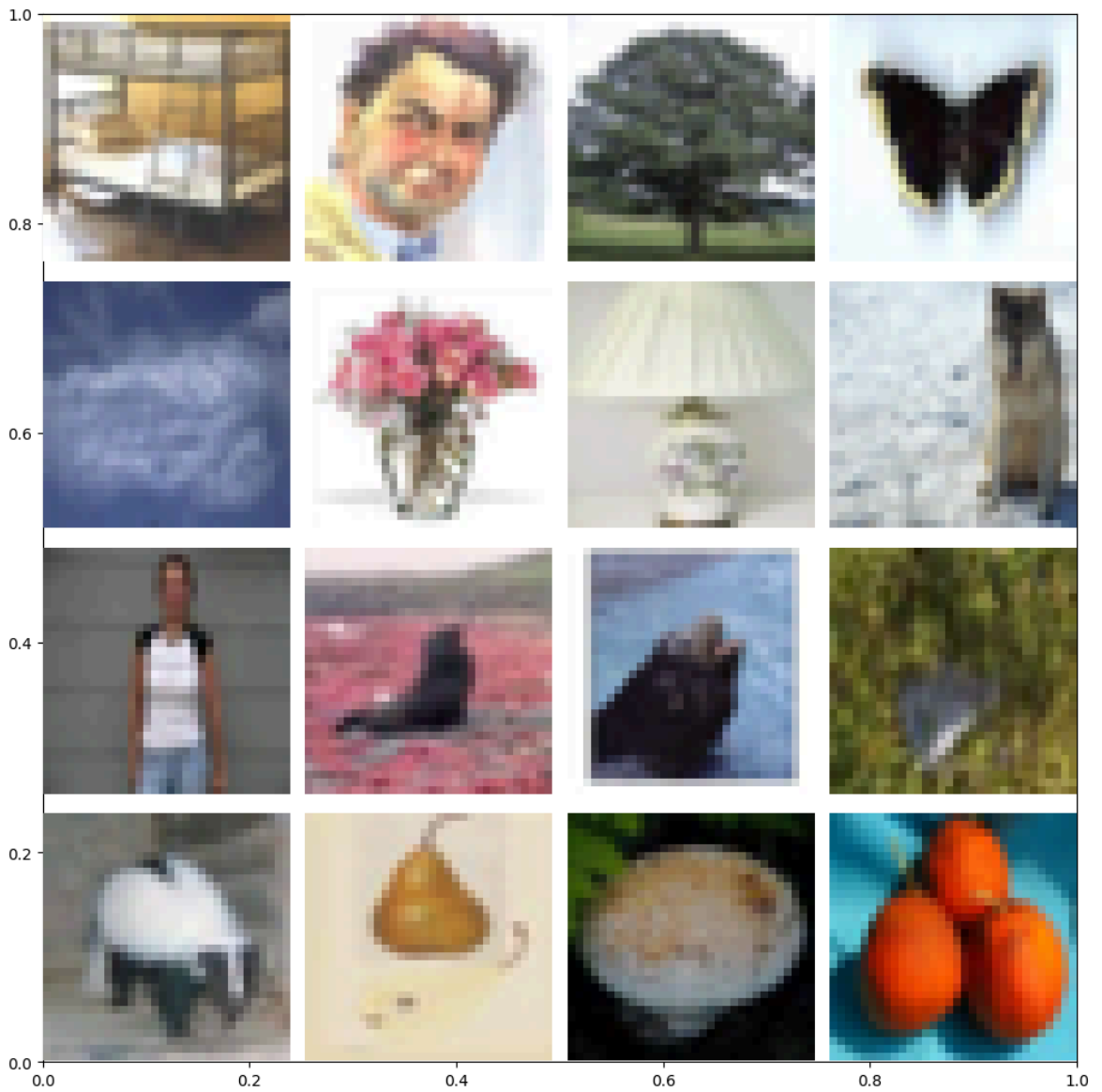
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

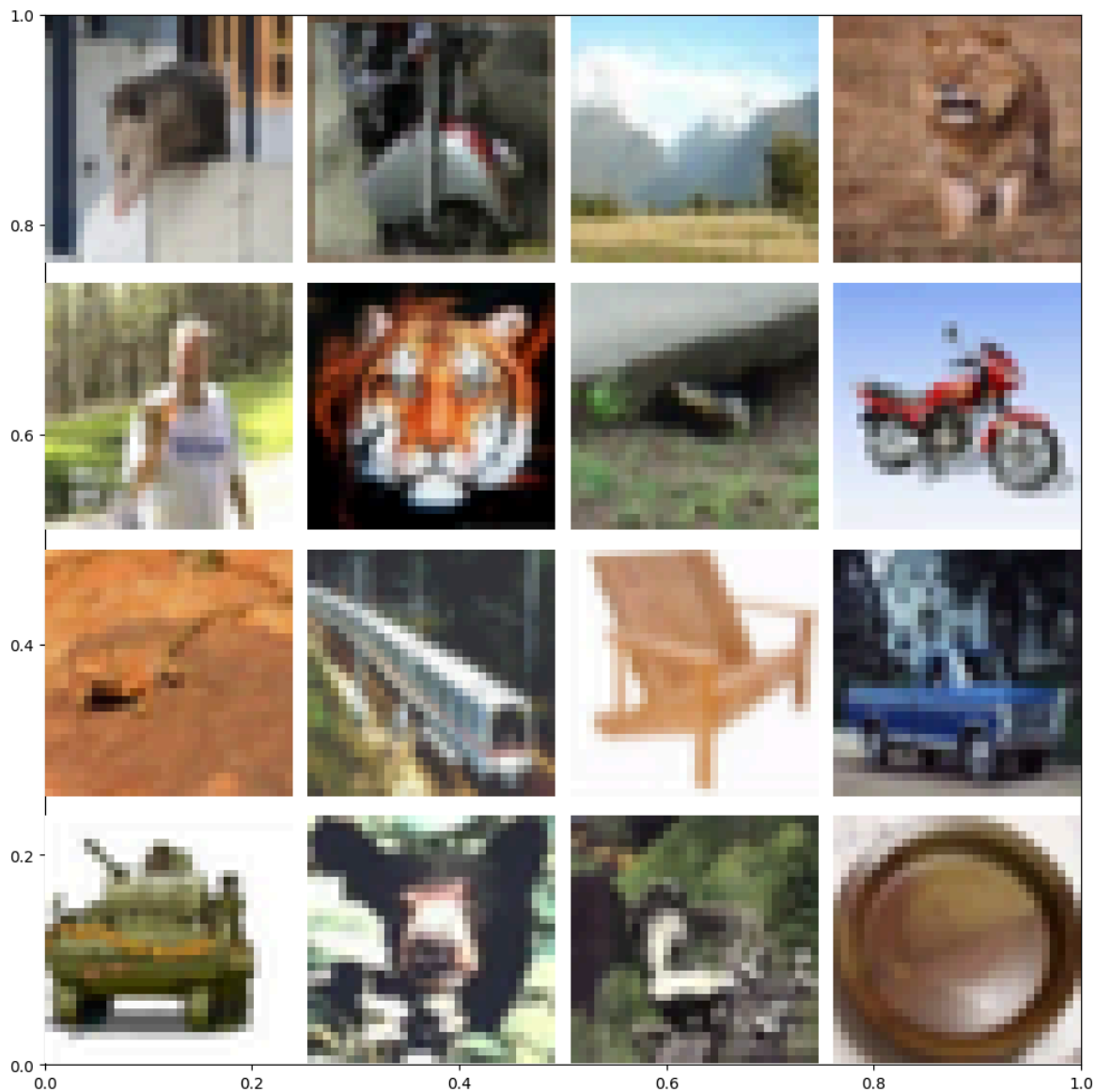
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[3810].shape: (1,)  
imgs[3810].shape: (32, 32, 3)
```

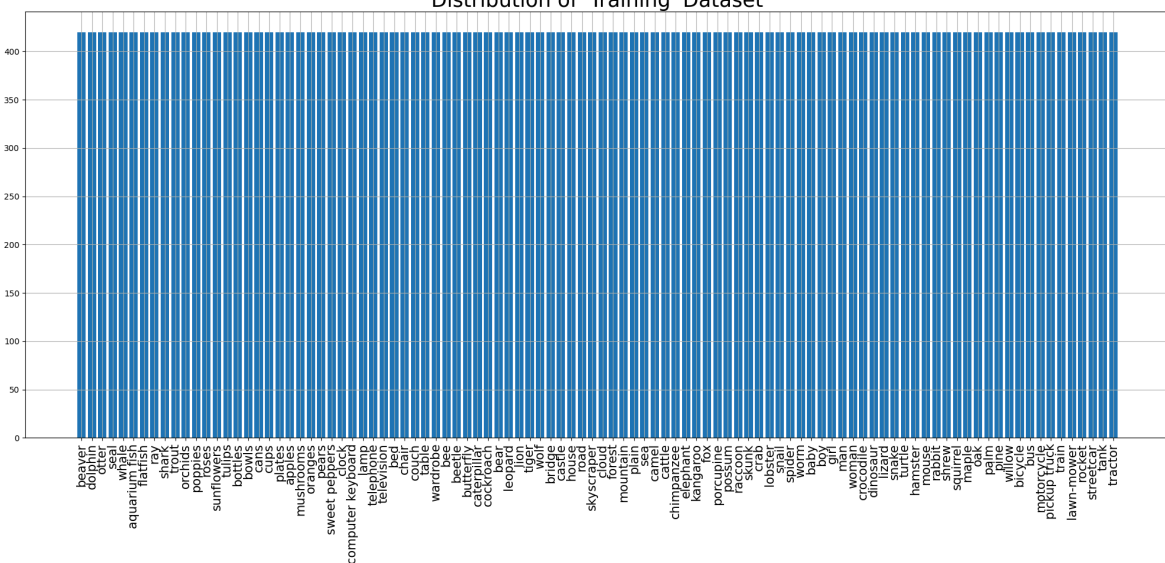


```
labels[11820].shape: (1,)  
imgs[11820].shape: (32, 32, 3)
```

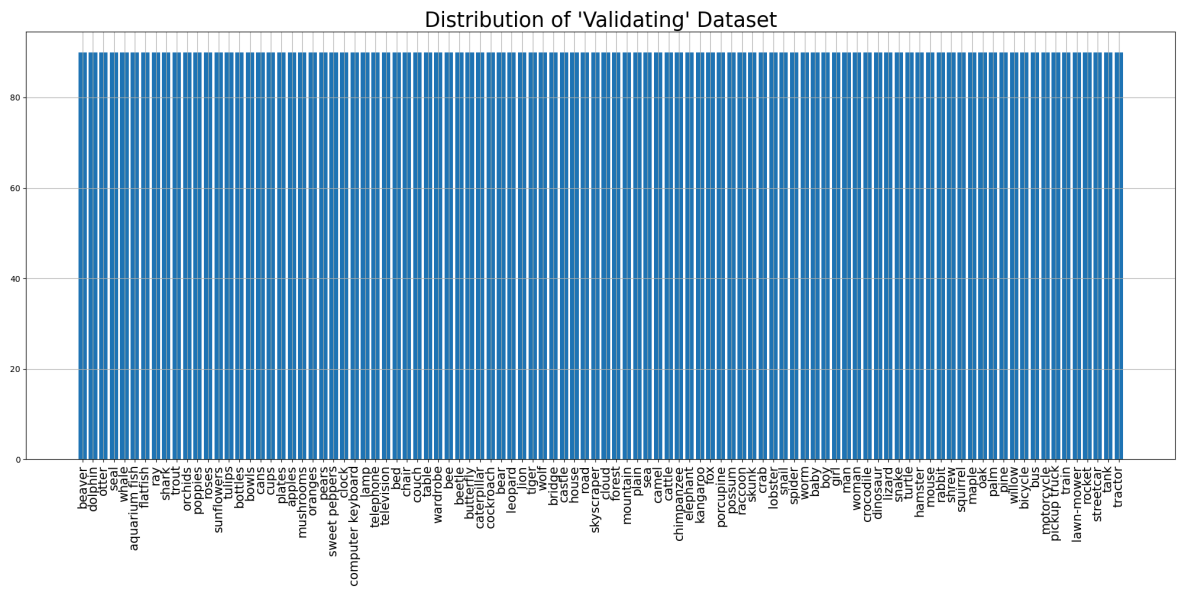


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

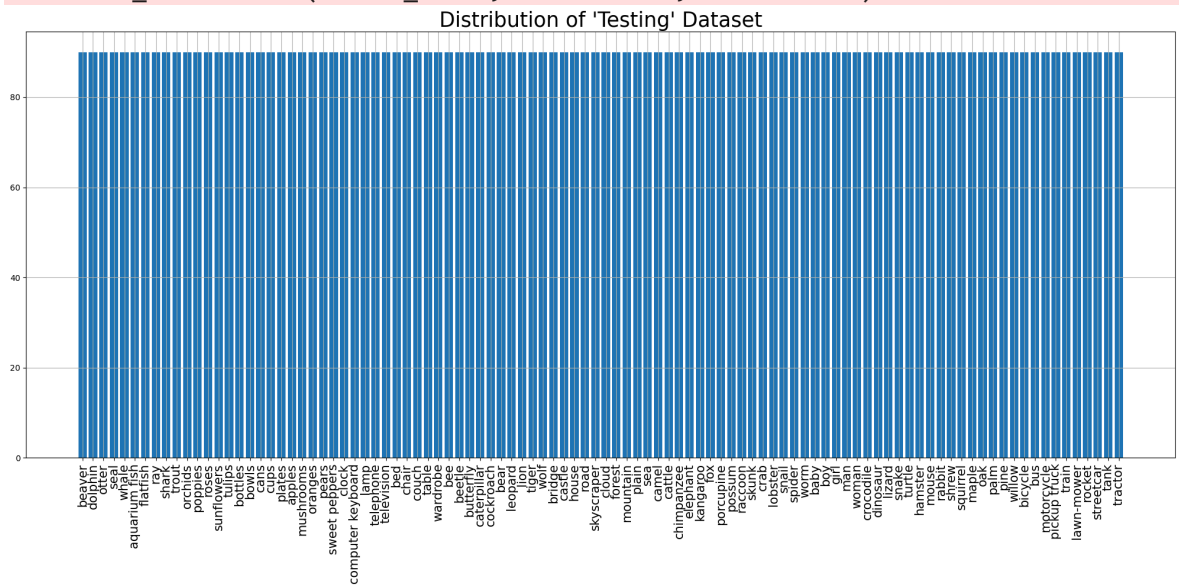
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_1"

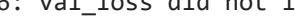
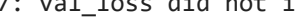
Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)

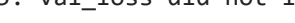
Total params: [24,665,188](#) (94.09 MB)

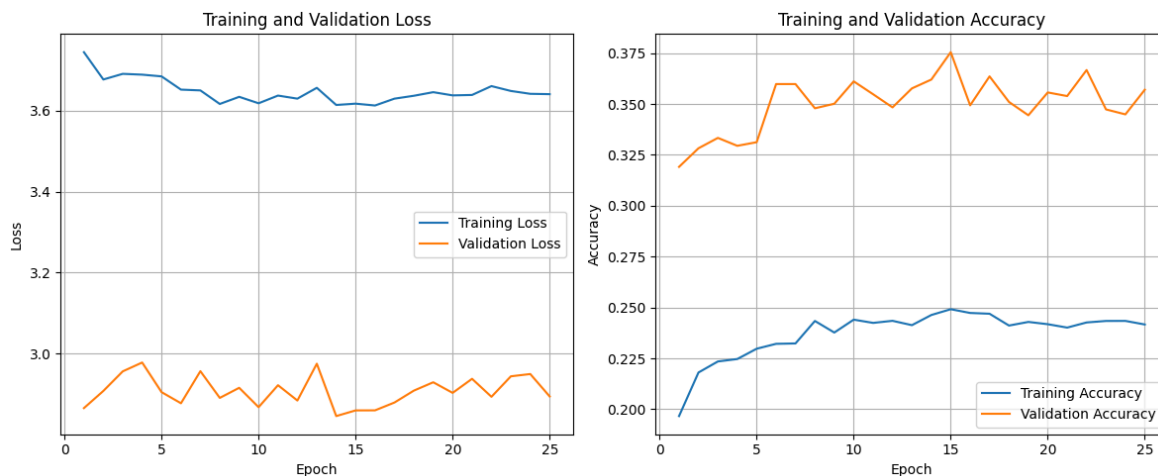
Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 30ms/step - accuracy: 0.1793 - f1_score: 0.1749 - loss: 3.9246 - precision: 0.4698
Epoch 1: val_loss improved from inf to 2.86488, saving model to best_model_v11.keras
657/657 ————— 42s 47ms/step - accuracy: 0.1793 - f1_score: 0.1750 - loss: 3.9243 - precision: 0.4698 - val_accuracy: 0.3191 - val_f1_score: 0.3181 - val_loss: 2.8649 - val_precision: 0.7247
Epoch 2/25
657/657 ————— 0s 24ms/step - accuracy: 0.2166 - f1_score: 0.2361 - loss: 3.6685 - precision: 0.5625
Epoch 2: val_loss did not improve from 2.86488
657/657 ————— 19s 29ms/step - accuracy: 0.2166 - f1_score: 0.2361 - loss: 3.6685 - precision: 0.5625 - val_accuracy: 0.3282 - val_f1_score: 0.3314 - val_loss: 2.9073 - val_precision: 0.7219
Epoch 3/25
655/657 ————— 0s 25ms/step - accuracy: 0.2217 - f1_score: 0.2468 - loss: 3.6957 - precision: 0.5954
Epoch 3: val_loss did not improve from 2.86488
657/657 ————— 19s 29ms/step - accuracy: 0.2218 - f1_score: 0.2468 - loss: 3.6956 - precision: 0.5955 - val_accuracy: 0.3333 - val_f1_score: 0.3297 - val_loss: 2.9557 - val_precision: 0.7148
Epoch 4/25
655/657 ————— 0s 24ms/step - accuracy: 0.2237 - f1_score: 0.2504 - loss: 3.6908 - precision: 0.6015
Epoch 4: val_loss did not improve from 2.86488
657/657 ————— 19s 29ms/step - accuracy: 0.2237 - f1_score: 0.2504 - loss: 3.6908 - precision: 0.6015 - val_accuracy: 0.3294 - val_f1_score: 0.3389 - val_loss: 2.9775 - val_precision: 0.7542
Epoch 5/25
655/657 ————— 0s 24ms/step - accuracy: 0.2279 - f1_score: 0.2585 - loss: 3.6998 - precision: 0.6186
Epoch 5: val_loss did not improve from 2.86488
657/657 ————— 19s 29ms/step - accuracy: 0.2279 - f1_score: 0.2585 - loss: 3.6997 - precision: 0.6186 - val_accuracy: 0.3312 - val_f1_score: 0.3431 - val_loss: 2.9043 - val_precision: 0.7772
Epoch 6/25
655/657 ————— 0s 24ms/step - accuracy: 0.2280 - f1_score: 0.2617 - loss: 3.6622 - precision: 0.6445
Epoch 6: val_loss did not improve from 2.86488
657/657 ————— 19s 29ms/step - accuracy: 0.2280 - f1_score: 0.2617 - loss: 3.6622 - precision: 0.6444 - val_accuracy: 0.3598 - val_f1_score: 0.3632 - val_loss: 2.8768 - val_precision: 0.7333
Epoch 7/25
655/657 ————— 0s 25ms/step - accuracy: 0.2332 - f1_score: 0.2690 - loss: 3.6775 - precision: 0.6430
Epoch 7: val_loss did not improve from 2.86488
657/657 ————— 19s 29ms/step - accuracy: 0.2332 - f1_score: 0.2690 - loss: 3.6774 - precision: 0.6430 - val_accuracy: 0.3598 - val_f1_score: 0.3731 - val_loss: 2.9561 - val_precision: 0.7537
Epoch 8/25
656/657 ————— 0s 24ms/step - accuracy: 0.2406 - f1_score: 0.2781 - loss: 3.6211 - precision: 0.6579
Epoch 8: val_loss did not improve from 2.86488
657/657 ————— 19s 29ms/step - accuracy: 0.2406 - f1_score: 0.2781 - loss: 3.6210 - precision: 0.6579 - val_accuracy: 0.3479 - val_f1_score: 0.3619 - val_loss: 2.8901 - val_precision: 0.7637
Epoch 9/25
656/657 ————— 0s 24ms/step - accuracy: 0.2373 - f1_score: 0.2738 - loss: 3.6537 - precision: 0.6623

Epoch 9: val_loss did not improve from 2.86488
657/657  **19s** 29ms/step - accuracy: 0.2373 - f1_score: 0.2738
- loss: 3.6536 - precision: 0.6623 - val_accuracy: 0.3501 - val_f1_score: 0.3593
- val_loss: 2.9150 - val_precision: 0.7482
Epoch 10/25
656/657  **0s** 25ms/step - accuracy: 0.2394 - f1_score: 0.2753 -
loss: 3.6391 - precision: 0.6620
Epoch 10: val_loss did not improve from 2.86488
657/657  **19s** 29ms/step - accuracy: 0.2394 - f1_score: 0.2754
- loss: 3.6391 - precision: 0.6620 - val_accuracy: 0.3611 - val_f1_score: 0.3750
- val_loss: 2.8671 - val_precision: 0.7827
Epoch 11/25
655/657  **0s** 24ms/step - accuracy: 0.2441 - f1_score: 0.2820 -
loss: 3.6085 - precision: 0.6764
Epoch 11: val_loss did not improve from 2.86488
657/657  **19s** 29ms/step - accuracy: 0.2440 - f1_score: 0.2820
- loss: 3.6086 - precision: 0.6764 - val_accuracy: 0.3548 - val_f1_score: 0.3732
- val_loss: 2.9214 - val_precision: 0.7722
Epoch 12/25
656/657  **0s** 24ms/step - accuracy: 0.2435 - f1_score: 0.2797 -
loss: 3.6477 - precision: 0.6695
Epoch 12: val_loss did not improve from 2.86488
657/657  **21s** 32ms/step - accuracy: 0.2435 - f1_score: 0.2797
- loss: 3.6477 - precision: 0.6696 - val_accuracy: 0.3483 - val_f1_score: 0.3634
- val_loss: 2.8836 - val_precision: 0.7699
Epoch 13/25
657/657  **0s** 24ms/step - accuracy: 0.2397 - f1_score: 0.2810 -
loss: 3.6702 - precision: 0.6867
Epoch 13: val_loss did not improve from 2.86488
657/657  **19s** 29ms/step - accuracy: 0.2397 - f1_score: 0.2810
- loss: 3.6701 - precision: 0.6867 - val_accuracy: 0.3577 - val_f1_score: 0.3623
- val_loss: 2.9746 - val_precision: 0.7558
Epoch 14/25
655/657  **0s** 24ms/step - accuracy: 0.2424 - f1_score: 0.2784 -
loss: 3.6579 - precision: 0.6721
Epoch 14: val_loss improved from 2.86488 to 2.84514, saving model to best_model_v
11.keras
657/657  **20s** 30ms/step - accuracy: 0.2424 - f1_score: 0.2784
- loss: 3.6577 - precision: 0.6722 - val_accuracy: 0.3620 - val_f1_score: 0.3747
- val_loss: 2.8451 - val_precision: 0.7656
Epoch 15/25
656/657  **0s** 24ms/step - accuracy: 0.2458 - f1_score: 0.2851 -
loss: 3.6478 - precision: 0.6832
Epoch 15: val_loss did not improve from 2.84514
657/657  **19s** 29ms/step - accuracy: 0.2458 - f1_score: 0.2851
- loss: 3.6477 - precision: 0.6832 - val_accuracy: 0.3754 - val_f1_score: 0.3910
- val_loss: 2.8591 - val_precision: 0.7772
Epoch 16/25
656/657  **0s** 24ms/step - accuracy: 0.2476 - f1_score: 0.2845 -
loss: 3.6374 - precision: 0.6761
Epoch 16: val_loss did not improve from 2.84514
657/657  **19s** 29ms/step - accuracy: 0.2476 - f1_score: 0.2845
- loss: 3.6373 - precision: 0.6762 - val_accuracy: 0.3493 - val_f1_score: 0.3704
- val_loss: 2.8593 - val_precision: 0.7818
Epoch 17/25
656/657  **0s** 25ms/step - accuracy: 0.2427 - f1_score: 0.2821 -
loss: 3.6319 - precision: 0.6823
Epoch 17: val_loss did not improve from 2.84514
657/657  **20s** 30ms/step - accuracy: 0.2427 - f1_score: 0.2821
- loss: 3.6319 - precision: 0.6823 - val_accuracy: 0.3636 - val_f1_score: 0.3734

- val_loss: 2.8786 - val_precision: 0.7553
Epoch 18/25
656/657  **0s** 24ms/step - accuracy: 0.2392 - f1_score: 0.2805 -
loss: 3.6474 - precision: 0.6851
Epoch 18: val_loss did not improve from 2.84514
657/657  **19s** 29ms/step - accuracy: 0.2392 - f1_score: 0.2806
- loss: 3.6473 - precision: 0.6851 - val_accuracy: 0.3510 - val_f1_score: 0.3610
- val_loss: 2.9080 - val_precision: 0.7699
Epoch 19/25
655/657  **0s** 24ms/step - accuracy: 0.2391 - f1_score: 0.2774 -
loss: 3.6656 - precision: 0.6795
Epoch 19: val_loss did not improve from 2.84514
657/657  **19s** 29ms/step - accuracy: 0.2392 - f1_score: 0.2774
- loss: 3.6655 - precision: 0.6795 - val_accuracy: 0.3444 - val_f1_score: 0.3656
- val_loss: 2.9287 - val_precision: 0.7792
Epoch 20/25
657/657  **0s** 24ms/step - accuracy: 0.2386 - f1_score: 0.2758 -
loss: 3.6396 - precision: 0.6765
Epoch 20: val_loss did not improve from 2.84514
657/657  **19s** 29ms/step - accuracy: 0.2386 - f1_score: 0.2758
- loss: 3.6396 - precision: 0.6766 - val_accuracy: 0.3557 - val_f1_score: 0.3627
- val_loss: 2.9025 - val_precision: 0.7695
Epoch 21/25
655/657  **0s** 24ms/step - accuracy: 0.2379 - f1_score: 0.2788 -
loss: 3.6458 - precision: 0.6846
Epoch 21: val_loss did not improve from 2.84514
657/657  **19s** 29ms/step - accuracy: 0.2379 - f1_score: 0.2788
- loss: 3.6458 - precision: 0.6846 - val_accuracy: 0.3539 - val_f1_score: 0.3740
- val_loss: 2.9372 - val_precision: 0.7727
Epoch 22/25
657/657  **0s** 23ms/step - accuracy: 0.2421 - f1_score: 0.2840 -
loss: 3.6545 - precision: 0.6893
Epoch 22: val_loss did not improve from 2.84514
657/657  **18s** 28ms/step - accuracy: 0.2421 - f1_score: 0.2840
- loss: 3.6545 - precision: 0.6893 - val_accuracy: 0.3667 - val_f1_score: 0.3883
- val_loss: 2.8928 - val_precision: 0.7813
Epoch 23/25
656/657  **0s** 23ms/step - accuracy: 0.2407 - f1_score: 0.2829 -
loss: 3.6433 - precision: 0.6837
Epoch 23: val_loss did not improve from 2.84514
657/657  **18s** 28ms/step - accuracy: 0.2407 - f1_score: 0.2829
- loss: 3.6433 - precision: 0.6837 - val_accuracy: 0.3473 - val_f1_score: 0.3606
- val_loss: 2.9434 - val_precision: 0.7846
Epoch 24/25
657/657  **0s** 23ms/step - accuracy: 0.2431 - f1_score: 0.2831 -
loss: 3.6365 - precision: 0.6912
Epoch 24: val_loss did not improve from 2.84514
657/657  **18s** 28ms/step - accuracy: 0.2431 - f1_score: 0.2831
- loss: 3.6365 - precision: 0.6912 - val_accuracy: 0.3449 - val_f1_score: 0.3539
- val_loss: 2.9491 - val_precision: 0.7662
Epoch 25/25
657/657  **0s** 23ms/step - accuracy: 0.2413 - f1_score: 0.2786 -
loss: 3.6443 - precision: 0.6845
Epoch 25: val_loss did not improve from 2.84514
657/657  **18s** 28ms/step - accuracy: 0.2413 - f1_score: 0.2786
- loss: 3.6443 - precision: 0.6845 - val_accuracy: 0.3570 - val_f1_score: 0.3719
- val_loss: 2.8937 - val_precision: 0.7906



141/141 ————— 12s 40ms/step - accuracy: 0.3547 - f1_score: 0.3640

- loss: 2.8799 - precision: 0.7632

Test Accuracy: 36.24%

Test Loss: 2.8430747985839844

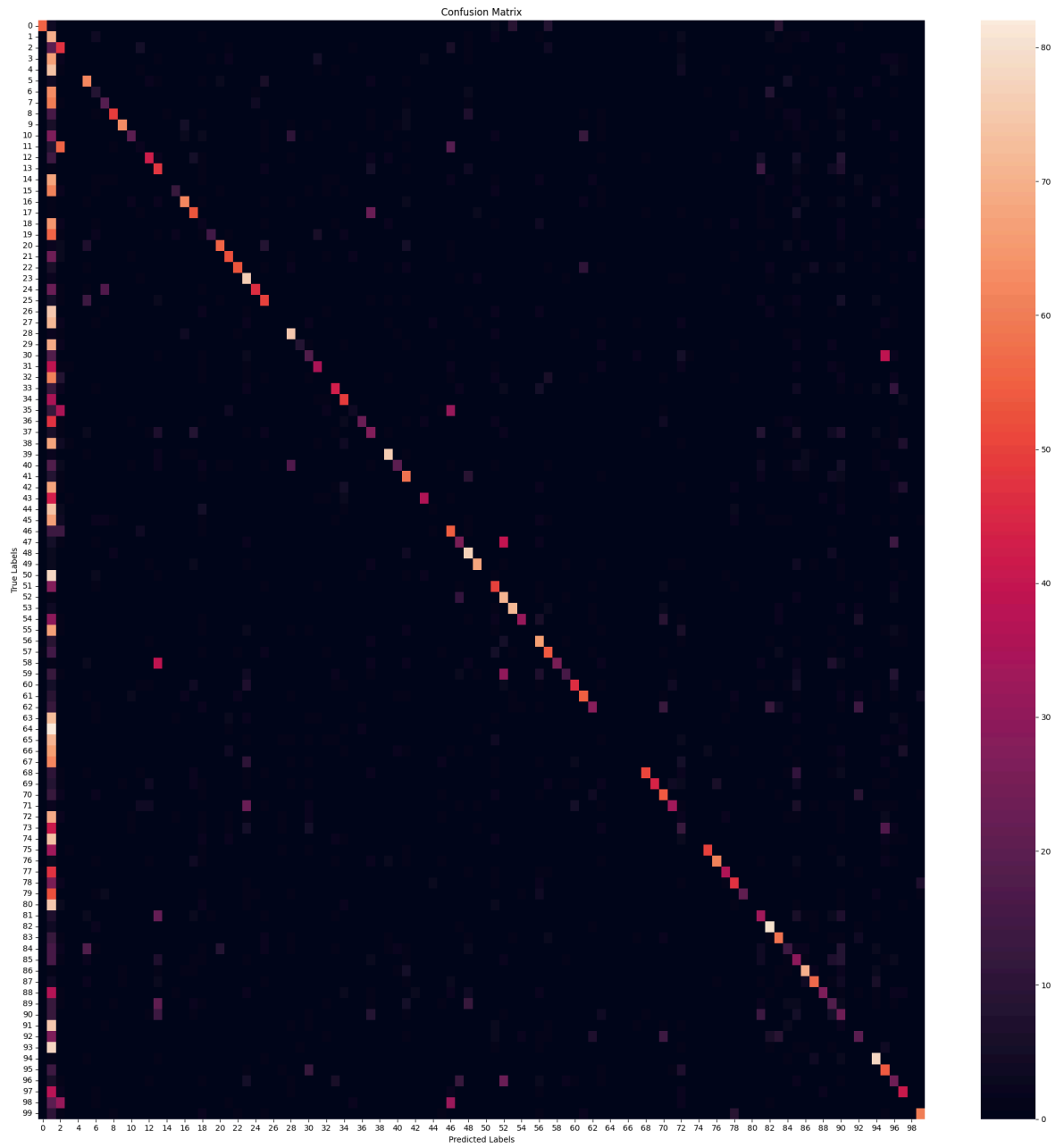
Test Precision: 76.82%

Test F1 Scores (Per Class): [75.675674 4.4929395 26.934093 0. 0.
58.490555

12.499996	28.358204	67.12328	80.769226	33.33333	3.636361
57.33333	35.82089	0.	22.429905	73.809525	58.88888
2.8168967	28.03738	69.99999	63.414627	71.62161	61.044167
64.78873	55.865913	0.	0.	71.77033	16.161615
25.174818	49.29577	0.	55.84415	59.523804	7.999998
39.31624	30.508467	0.	83.9779	33.898304	57.999992
0.	53.73134	2.0618544	0.	43.82469	34.0136
69.36936	80.95238	0.	51.54639	52.398525	82.285706
49.599995	0.	62.559242	61.363636	43.47826	28.070173
62.585033	63.5838	41.176464	0.	0.	0.
0.	0.	70.34482	65.67164	55.10204	48.88888
2.3809476	0.	0.	68.965515	76.24999	54.135334
56.6265	37.168137	0.	34.196888	73.73272	62.702698
20.28985	23.966938	74.8663	69.51219	43.33333	21.428568
19.672129	0.	24.844713	0.	72.89719	47.161568
23.188402	49.10179	0.	72.289154	]	

Average Test F1 Scores:37.40

282/282 ————— 11s 27ms/step



	precision	recall	f1-score	support
beaver	0.97	0.62	0.76	90
dolphin	0.02	0.78	0.04	90
otter	0.18	0.52	0.27	90
seal	0.00	0.00	0.00	90
whale	0.00	0.00	0.00	90
aquarium fish	0.51	0.69	0.58	90
flatfish	0.21	0.09	0.12	90
ray	0.43	0.21	0.28	90
shark	0.88	0.54	0.67	90
trout	0.95	0.70	0.81	90
orchids	0.67	0.22	0.33	90
poppies	0.10	0.02	0.04	90
roses	0.71	0.47	0.56	90
sunflowers	0.27	0.53	0.36	90
tulips	0.00	0.00	0.00	90
bottles	0.71	0.13	0.22	90
bowls	0.79	0.69	0.74	90
cans	0.59	0.59	0.59	90
cups	0.04	0.02	0.03	90
plates	0.88	0.17	0.28	90
apples	0.80	0.62	0.70	90
mushrooms	0.70	0.58	0.63	90
oranges	0.91	0.59	0.72	90
pears	0.47	0.84	0.61	90
sweet peppers	0.88	0.51	0.65	90
clock	0.56	0.56	0.56	90
computer keyboard	0.00	0.00	0.00	90
lamp	0.00	0.00	0.00	90
telephone	0.63	0.83	0.72	90
television	0.89	0.09	0.16	90
bed	0.34	0.20	0.25	90
chair	0.67	0.39	0.49	90
couch	0.00	0.00	0.00	90
table	0.67	0.48	0.56	90
wardrobe	0.63	0.54	0.58	90
bee	0.40	0.04	0.08	90
beetle	0.85	0.26	0.39	90
butterfly	0.31	0.30	0.31	90
caterpillar	0.00	0.00	0.00	90
cockroach	0.84	0.84	0.84	90
bear	0.71	0.22	0.34	90
leopard	0.53	0.64	0.58	90
lion	0.00	0.00	0.00	90
tiger	0.82	0.40	0.54	90
wolf	0.14	0.01	0.02	90
bridge	0.00	0.00	0.00	90
castle	0.34	0.61	0.44	90
house	0.44	0.28	0.34	90
road	0.58	0.86	0.69	90
skyscraper	0.87	0.76	0.81	90
cloud	0.00	0.00	0.00	90
forest	0.48	0.56	0.52	90
mountain	0.39	0.79	0.52	90
plain	0.85	0.80	0.82	90
sea	0.89	0.34	0.50	90
camel	0.00	0.00	0.00	90
cattle	0.55	0.73	0.63	90
chimpanzee	0.63	0.60	0.61	90

elephant	1.00	0.28	0.43	90
kangaroo	0.67	0.18	0.28	90
fox	0.81	0.51	0.63	90
porcupine	0.66	0.61	0.64	90
possum	0.61	0.31	0.41	90
raccoon	0.00	0.00	0.00	90
skunk	0.00	0.00	0.00	90
crab	0.00	0.00	0.00	90
lobster	0.00	0.00	0.00	90
snail	0.00	0.00	0.00	90
spider	0.93	0.57	0.70	90
worm	1.00	0.49	0.66	90
baby	0.51	0.60	0.55	90
boy	0.73	0.37	0.49	90
girl	0.03	0.02	0.02	90
man	0.00	0.00	0.00	90
woman	0.00	0.00	0.00	90
crocodile	0.91	0.56	0.69	90
dinosaur	0.87	0.68	0.76	90
lizard	0.84	0.40	0.54	90
snake	0.62	0.52	0.57	90
turtle	0.91	0.23	0.37	90
hamster	0.00	0.00	0.00	90
mouse	0.32	0.37	0.34	90
rabbit	0.63	0.89	0.74	90
shrew	0.61	0.64	0.63	90
squirrel	0.29	0.16	0.20	90
maple	0.19	0.32	0.24	90
oak	0.72	0.78	0.75	90
palm	0.77	0.63	0.70	90
pine	0.87	0.29	0.43	90
willow	0.23	0.20	0.21	90
bicycle	0.15	0.27	0.20	90
bus	0.00	0.00	0.00	90
motorcycle	0.28	0.22	0.25	90
pickup truck	0.00	0.00	0.00	90
train	0.63	0.87	0.73	90
lawn-mower	0.39	0.60	0.47	90
rocket	0.21	0.27	0.23	90
streetcar	0.53	0.46	0.49	90
tank	0.00	0.00	0.00	90
tractor	0.79	0.67	0.72	90
accuracy			0.36	9000
macro avg	0.46	0.36	0.37	9000
weighted avg	0.46	0.36	0.37	9000

```

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 2048)
dense_2 (Dense)	(None, 512)
dropout_1 (Dropout)	(None, 512)
dense_3 (Dense)	(None, 100)



Total params: 24,665,188 (94.09 MB)
Trainable params: 1,100,388 (4.20 MB)
Non-trainable params: 23,564,800 (89.89 MB)

Epoch 1/15
1312/1314 ————— **0s** 26ms/step - accuracy: 0.2531 - f1_score: 0.2925
- loss: 3.4868 - precision: 0.7184
Epoch 1: val_loss improved from inf to 2.74024, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— **56s** 34ms/step - accuracy: 0.2532 - f1_score: 0.2925 - loss: 3.4867 - precision: 0.7184 - val_accuracy: 0.3823 - val_f1_score: 0.4153 - val_loss: 2.7402 - val_precision: 0.8422

Epoch 2/15
1313/1314 ————— **0s** 24ms/step - accuracy: 0.2723 - f1_score: 0.3195
- loss: 3.3164 - precision: 0.7802
Epoch 2: val_loss improved from 2.74024 to 2.67014, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.2723 - f1_score: 0.3195 - loss: 3.3164 - precision: 0.7802 - val_accuracy: 0.3997 - val_f1_score: 0.4321 - val_loss: 2.6701 - val_precision: 0.8457

Epoch 3/15
1312/1314 ————— **0s** 24ms/step - accuracy: 0.2848 - f1_score: 0.3319
- loss: 3.2211 - precision: 0.7886
Epoch 3: val_loss improved from 2.67014 to 2.58653, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.2848 - f1_score: 0.3319 - loss: 3.2211 - precision: 0.7886 - val_accuracy: 0.4134 - val_f1_score: 0.4411 - val_loss: 2.5865 - val_precision: 0.8422

Epoch 4/15
1313/1314 ————— **0s** 23ms/step - accuracy: 0.2938 - f1_score: 0.3403
- loss: 3.1559 - precision: 0.7860
Epoch 4: val_loss improved from 2.58653 to 2.52081, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— **35s** 26ms/step - accuracy: 0.2938 - f1_score: 0.3403 - loss: 3.1558 - precision: 0.7860 - val_accuracy: 0.4289 - val_f1_score: 0.4511 - val_loss: 2.5208 - val_precision: 0.8446

Epoch 5/15
1312/1314 ————— **0s** 23ms/step - accuracy: 0.3040 - f1_score: 0.3511
- loss: 3.0789 - precision: 0.7926
Epoch 5: val_loss improved from 2.52081 to 2.44732, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— **35s** 26ms/step - accuracy: 0.3040 - f1_score: 0.3511 - loss: 3.0788 - precision: 0.7926 - val_accuracy: 0.4416 - val_f1_score: 0.4636 - val_loss: 2.4473 - val_precision: 0.8445

Epoch 6/15
1314/1314 ————— **0s** 23ms/step - accuracy: 0.3091 - f1_score: 0.3547
- loss: 3.0067 - precision: 0.7869
Epoch 6: val_loss improved from 2.44732 to 2.38644, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— **35s** 26ms/step - accuracy: 0.3091 - f1_score: 0.3547 - loss: 3.0067 - precision: 0.7869 - val_accuracy: 0.4486 - val_f1_score: 0.4706 - val_loss: 2.3864 - val_precision: 0.8456

Epoch 7/15
1314/1314 ————— **0s** 23ms/step - accuracy: 0.3154 - f1_score: 0.3592
- loss: 2.9443 - precision: 0.7987
Epoch 7: val_loss improved from 2.38644 to 2.34221, saving model to best_model_fine_tune_v11.keras
1314/1314 ————— **35s** 26ms/step - accuracy: 0.3154 - f1_score: 0.3592 - loss: 2.9443 - precision: 0.7987 - val_accuracy: 0.4570 - val_f1_score: 0.4768 - val_loss: 2.3422 - val_precision: 0.8408

Epoch 8/15
1312/1314 ————— **0s** 23ms/step - accuracy: 0.3231 - f1_score: 0.3635
- loss: 2.8942 - precision: 0.7950
Epoch 8: val_loss improved from 2.34221 to 2.30573, saving model to best_model_fine_tune_v11.keras

ne_tune_v11.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.3231 - f1_score: 0.3635 - loss: 2.8942 - precision: 0.7950 - val_accuracy: 0.4639 - val_f1_score: 0.4814 - val_loss: 2.3057 - val_precision: 0.8398

Epoch 9/15

1314/1314 ————— 0s 23ms/step - accuracy: 0.3278 - f1_score: 0.3673 - loss: 2.8619 - precision: 0.7989

Epoch 9: val_loss improved from 2.30573 to 2.26957, saving model to best_model_fine_tune_v11.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.3278 - f1_score: 0.3673 - loss: 2.8619 - precision: 0.7989 - val_accuracy: 0.4694 - val_f1_score: 0.4857 - val_loss: 2.2696 - val_precision: 0.8411

Epoch 10/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.3342 - f1_score: 0.3695 - loss: 2.8332 - precision: 0.8017

Epoch 10: val_loss improved from 2.26957 to 2.23521, saving model to best_model_fine_tune_v11.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.3342 - f1_score: 0.3695 - loss: 2.8332 - precision: 0.8017 - val_accuracy: 0.4742 - val_f1_score: 0.4888 - val_loss: 2.2352 - val_precision: 0.8412

Epoch 11/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.3344 - f1_score: 0.3675 - loss: 2.8025 - precision: 0.8056

Epoch 11: val_loss improved from 2.23521 to 2.21195, saving model to best_model_fine_tune_v11.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.3344 - f1_score: 0.3675 - loss: 2.8025 - precision: 0.8056 - val_accuracy: 0.4793 - val_f1_score: 0.4922 - val_loss: 2.2120 - val_precision: 0.8416

Epoch 12/15

1313/1314 ————— 0s 23ms/step - accuracy: 0.3390 - f1_score: 0.3699 - loss: 2.7683 - precision: 0.8051

Epoch 12: val_loss improved from 2.21195 to 2.17408, saving model to best_model_fine_tune_v11.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.3390 - f1_score: 0.3699 - loss: 2.7683 - precision: 0.8051 - val_accuracy: 0.4857 - val_f1_score: 0.4948 - val_loss: 2.1741 - val_precision: 0.8412

Epoch 13/15

1314/1314 ————— 0s 23ms/step - accuracy: 0.3446 - f1_score: 0.3718 - loss: 2.7078 - precision: 0.8057

Epoch 13: val_loss improved from 2.17408 to 2.12804, saving model to best_model_fine_tune_v11.keras

1314/1314 ————— 34s 26ms/step - accuracy: 0.3446 - f1_score: 0.3718 - loss: 2.7078 - precision: 0.8057 - val_accuracy: 0.4929 - val_f1_score: 0.5002 - val_loss: 2.1280 - val_precision: 0.8459

Epoch 14/15

1312/1314 ————— 0s 23ms/step - accuracy: 0.3511 - f1_score: 0.3753 - loss: 2.6663 - precision: 0.8103

Epoch 14: val_loss improved from 2.12804 to 2.09375, saving model to best_model_fine_tune_v11.keras

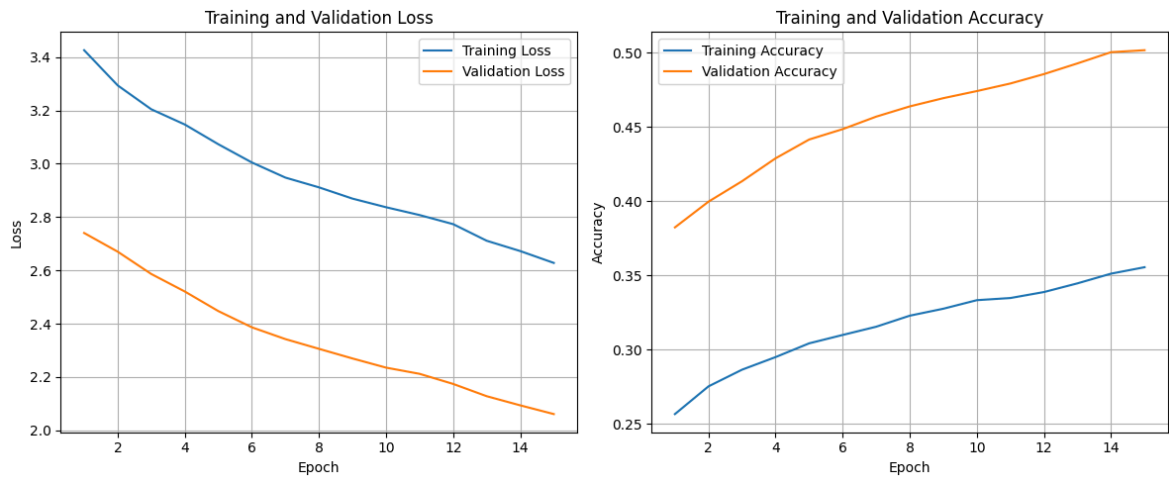
1314/1314 ————— 35s 26ms/step - accuracy: 0.3511 - f1_score: 0.3753 - loss: 2.6663 - precision: 0.8103 - val_accuracy: 0.5004 - val_f1_score: 0.5071 - val_loss: 2.0937 - val_precision: 0.8433

Epoch 15/15

1314/1314 ————— 0s 23ms/step - accuracy: 0.3576 - f1_score: 0.3809 - loss: 2.6185 - precision: 0.8103

Epoch 15: val_loss improved from 2.09375 to 2.06105, saving model to best_model_fine_tune_v11.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.3576 - f1_score: 0.3809 - loss: 2.6185 - precision: 0.8103 - val_accuracy: 0.5018 - val_f1_score: 0.5081 - val_loss: 2.0611 - val_precision: 0.8406

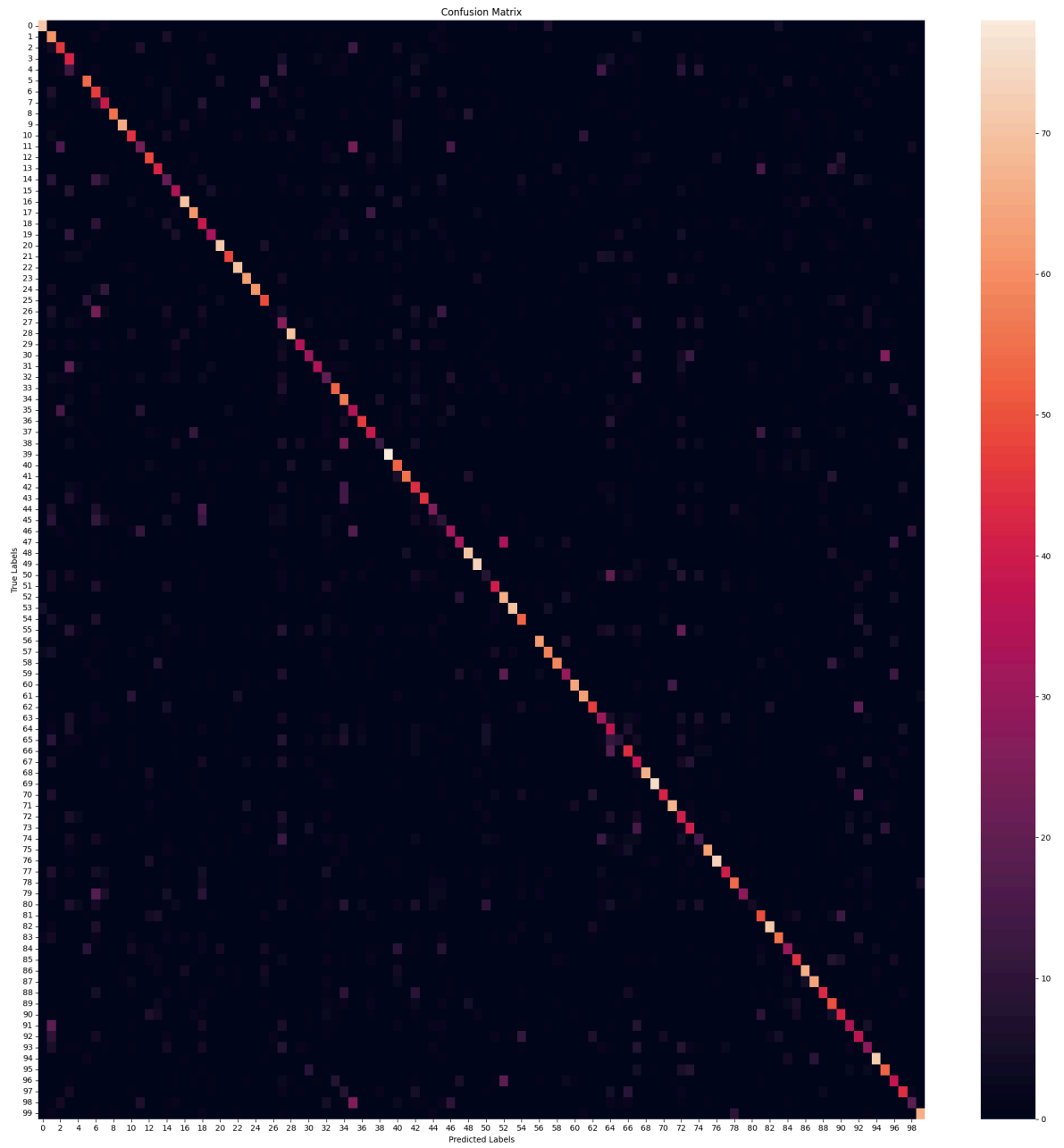


141/141 ————— **11s** 38ms/step - accuracy: 0.4958 - f1_score: 0.4973
- loss: 2.0903 - precision: 0.8368
Test Accuracy: 50.36%
Test Loss: 2.0693130493164062
Test Precision: 84.40%
Test F1 Scores (Per Class): [82.558136 37.308865 49.729725 30.769226 3.17459
92 62.7907

30.81967	43.093918	72.368416	81.24999	48.648643	30.263153
51.851852	52.499992	21.989523	40.476185	77.34807	70.11494
32.36514	44.897953	79.775276	65.306114	80.	74.556206
73.93939	53.551907	3.3057814	18.815329	79.54544	46.052628
40.259735	49.999992	18.867918	56.684486	39.999996	32.22748
60.645157	46.98795	18.320606	90.69767	42.570274	70.96774
35.772354	61.224483	31.645563	11.049719	37.647053	43.53741
74.8663	79.78141	10.666661	51.282047	57.64192	82.84023
56.382973	0.	72.51461	67.05202	73.74999	40.540535
78.048775	72.83237	58.024685	32.804226	29.13385	14.754096
42.926823	29.230764	79.51807	84.26965	57.931023	72.82608
30.258297	47.39884	16.279066	78.26086	84.883705	52.631577
64.670654	43.410847	6.8376036	49.746185	81.14285	66.26506
35.502953	49.999992	72.13114	73.56321	54.43037	47.39336
42.857136	43.589737	34.579433	27.184464	81.81818	56.684486
38.578674	52.694607	27.142849	76.74419	]	

Average Test F1 Scores:50.85

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.87	0.79	0.83	90
dolphin	0.26	0.68	0.37	90
otter	0.48	0.51	0.50	90
seal	0.23	0.47	0.31	90
whale	0.06	0.02	0.03	90
aquarium fish	0.66	0.60	0.63	90
flatfish	0.22	0.52	0.31	90
ray	0.43	0.43	0.43	90
shark	0.89	0.61	0.72	90
trout	0.93	0.72	0.81	90
orchids	0.47	0.50	0.49	90
poppies	0.37	0.26	0.30	90
roses	0.49	0.54	0.52	90
sunflowers	0.60	0.47	0.53	90
tulips	0.21	0.23	0.22	90
bottles	0.44	0.38	0.40	90
bowls	0.77	0.78	0.77	90
cans	0.73	0.68	0.70	90
cups	0.26	0.43	0.33	90
plates	0.58	0.37	0.45	90
apples	0.81	0.79	0.80	90
mushrooms	0.84	0.53	0.65	90
oranges	0.82	0.78	0.80	90
pears	0.80	0.70	0.75	90
sweet peppers	0.81	0.68	0.74	90
clock	0.53	0.54	0.54	90
computer keyboard	0.06	0.02	0.03	90
lamp	0.14	0.30	0.19	90
telephone	0.81	0.78	0.80	90
television	0.56	0.39	0.46	90
bed	0.48	0.34	0.40	90
chair	0.74	0.38	0.50	90
couch	0.16	0.22	0.19	90
table	0.55	0.59	0.57	90
wardrobe	0.29	0.63	0.40	90
bee	0.28	0.38	0.32	90
beetle	0.72	0.52	0.61	90
butterfly	0.51	0.43	0.47	90
caterpillar	0.29	0.13	0.18	90
cockroach	0.95	0.87	0.91	90
bear	0.33	0.59	0.43	90
leopard	0.85	0.61	0.71	90
lion	0.28	0.49	0.36	90
tiger	0.79	0.50	0.61	90
wolf	0.37	0.28	0.32	90
bridge	0.11	0.11	0.11	90
castle	0.40	0.36	0.38	90
house	0.56	0.36	0.44	90
road	0.72	0.78	0.75	90
skyscraper	0.78	0.81	0.80	90
cloud	0.13	0.09	0.11	90
forest	0.61	0.44	0.51	90
mountain	0.47	0.73	0.58	90
plain	0.89	0.78	0.83	90
sea	0.54	0.59	0.56	90
camel	0.00	0.00	0.00	90
cattle	0.77	0.69	0.73	90
chimpanzee	0.70	0.64	0.67	90

elephant	0.84	0.66	0.74	90
kangaroo	0.52	0.33	0.41	90
fox	0.86	0.71	0.78	90
porcupine	0.76	0.70	0.73	90
possum	0.65	0.52	0.58	90
raccoon	0.31	0.34	0.33	90
skunk	0.23	0.41	0.29	90
crab	0.28	0.10	0.15	90
lobster	0.39	0.49	0.43	90
snail	0.22	0.42	0.29	90
spider	0.87	0.73	0.80	90
worm	0.85	0.83	0.84	90
baby	0.76	0.47	0.58	90
boy	0.71	0.74	0.73	90
girl	0.23	0.46	0.30	90
man	0.49	0.46	0.47	90
woman	0.17	0.16	0.16	90
crocodile	0.89	0.70	0.78	90
dinosaur	0.89	0.81	0.85	90
lizard	0.65	0.44	0.53	90
snake	0.70	0.60	0.65	90
turtle	0.70	0.31	0.43	90
hamster	0.15	0.04	0.07	90
mouse	0.46	0.54	0.50	90
rabbit	0.84	0.79	0.81	90
shrew	0.72	0.61	0.66	90
squirrel	0.38	0.33	0.36	90
maple	0.50	0.50	0.50	90
oak	0.71	0.73	0.72	90
palm	0.76	0.71	0.74	90
pine	0.63	0.48	0.54	90
willow	0.41	0.56	0.47	90
bicycle	0.40	0.47	0.43	90
bus	0.52	0.38	0.44	90
motorcycle	0.30	0.41	0.35	90
pickup truck	0.24	0.31	0.27	90
train	0.84	0.80	0.82	90
lawn-mower	0.55	0.59	0.57	90
rocket	0.36	0.42	0.39	90
streetcar	0.57	0.49	0.53	90
tank	0.38	0.21	0.27	90
tractor	0.80	0.73	0.77	90
accuracy			0.50	9000
macro avg	0.54	0.50	0.51	9000
weighted avg	0.54	0.50	0.51	9000

```

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels w
ith no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```